

AVR Assembly Programming

garyPenhook (Dazed_N_Confused)

Contents

1	Why Assembly? Toolchain Setup	1
1.1	What Is Assembly Language?	1
1.1.1	When do you actually need assembly?	1
1.2	Target: ATtiny3217	2
1.3	The AVR-GAS Toolchain	2
1.3.1	Installing the Toolchain	2
1.4	The Build Pipeline	3
1.4.1	Why two link steps?	3
1.5	First Program: Blink	4
1.5.1	Understanding the ATtiny3217 PORT Peripheral	4
1.5.2	The Code	4
1.6	Building and Flashing	6
1.6.1	Step 1: Assemble	6
1.6.2	Step 2: Link	6
1.6.3	Step 3: Convert to HEX	7
1.6.4	Step 4: Inspect (optional but recommended)	7
1.6.5	Step 5: Flash	7
1.7	A Minimal Makefile	8
1.8	What Just Happened — Mental Model	8
1.9	Summary	9
1.10	Exercises	9
2	Number Systems: Binary, Hexadecimal, Decimal, and BCD	10
2.1	Decimal: The System You Already Know	10
2.2	Binary: The CPU's Native Language	10
2.2.1	Counting in Binary	11
2.2.2	Bit Terminology	11
2.2.3	Converting Binary to Decimal	12
2.2.4	Converting Decimal to Binary (Repeated Division)	12
2.2.5	Powers of Two You Must Know	12
2.2.6	Binary Notation in GAS Source	13
2.3	Hexadecimal: Compact Binary Notation	13
2.3.1	Hexadecimal Digits	13
2.3.2	Converting Hex $\leftrightarrow$ Binary (The Fundamental Operation)	13
2.3.3	Converting Hex $\leftrightarrow$ Decimal	14
2.3.4	Converting Decimal to Hex (Repeated Division by 16)	14
2.3.5	Hex Notation in AVR Assembly and C	14
2.4	The Complete 8-bit Map	15
2.5	Little-Endian Byte Order	16
2.6	Number Representation in GAS Assembly Source	17
2.6.1	Assembler Arithmetic	17
2.6.2	Data Directives and Byte Order	17
2.7	BCD: Binary Coded Decimal	18
2.7.1	Packed BCD	18
2.7.2	Unpacked BCD	18
2.7.3	BCD Addition on AVR	19
2.7.4	BCD Subtraction on AVR	21

2.7.5	Converting Binary to Packed BCD	22
2.7.6	Converting Packed BCD to Binary	23
2.7.7	BCD and the Half-Carry Flag	25
2.8	Choosing the Right Representation	25
2.9	Summary	26
2.10	Exercises	27
3	AVR Architecture Overview	28
3.1	Harvard Architecture	28
3.2	ATtiny3217 Memory Map	29
3.3	The Register File	31
3.3.1	Register Pairs (16-bit Pointers)	31
3.3.2	LDI Restriction	32
3.4	The Status Register (SREG)	32
3.4.1	Flag by Flag	32
3.4.2	Reading and Saving SREG	33
3.5	The Stack and Stack Pointer	33
3.5.1	PUSH and POP	34
3.5.2	How CALL Uses the Stack	34
3.6	The Program Counter (PC)	35
3.6.1	Instruction Fetch Pipeline	35
3.7	I/O Register Space	36
3.8	Putting It Together — A Register-State Walk-Through	37
3.9	Key Architectural Constraints Summary	37
3.10	Summary	38
3.11	Exercises	38
4	AVR-AS Syntax and Directives	40
4.1	File Extensions: .S vs .s	40
4.2	Comments	41
4.3	Instruction Syntax	41
4.4	Number Literals	42
4.5	Expressions	42
4.5.1	Built-in Expression Functions	43
4.6	Labels	43
4.6.1	Global vs Local Labels	43
4.6.2	GAS Local Labels (Numeric)	44
4.6.3	File-Scoped Local Labels (.L prefix)	44
4.7	Sections	45
4.7.1	Core Sections	45
4.7.2	Section Flags and Types (Full Form)	46
4.8	Data Directives	46
4.8.1	.byte — 8-bit values	46
4.8.2	.word — 16-bit values (little-endian)	47
4.8.3	.long — 32-bit values	47
4.8.4	.ascii and .asciz — Strings	47
4.8.5	.skip and .space — Reserve Bytes	47
4.8.6	.balign — Alignment	47
4.9	Constant Directives	48
4.9.1	.equ — Define a Constant	48
4.9.2	.set — Reassignable Constant	48
4.9.3	#define — Preprocessor Macro	48
4.10	The .org Directive	48
4.11	Flash Data on AVRxt (ATtiny3217)	49
4.12	The .include Directive	50
4.13	Macro Directives	51
4.14	Full Section Layout Example	51
4.15	Examining the Output with avr-objdump	52

4.16	Quick Reference	53
4.17	Summary	53
4.18	Exercises	54
5	Debugging AVR Assembly with GDB	55
5.1	Why This Chapter Belongs Here	55
5.2	The Files GDB Needs	55
5.3	Keep Symbols That Help Debugging	56
5.4	Starting GDB	57
5.5	Debugging Targets Used Here	57
5.5.1	1. Static ELF Inspection	58
5.5.2	2. ATtiny3217 Curiosity Nano Hardware	58
5.5.3	3. PyAvrOCD Debugging Guide	58
5.6	Core GDB Commands for Assembly	64
5.7	Instruction-Level Stepping	65
5.8	Reading Disassembly	66
5.9	Inspecting Registers and Flags	66
5.10	Inspecting SRAM	67
5.11	Inspecting Flash Data	68
5.12	Breakpoints on Microcontrollers	68
5.13	Watchpoints and Data Breaks	68
5.14	Debugging the Stack	69
5.15	Debugging Interrupts	69
5.16	Debugging Peripheral Code	70
5.17	A Minimal Assembly Debug Example	71
5.18	A Useful .gdbinit for AVR Assembly	72
5.19	Failure Patterns and What to Inspect	73
5.20	Professional Debug Habits	73
5.21	Source Notes	74
6	Instruction Set Basics	75
6.1	Instruction Encoding	75
6.1.1	Single-Word Instructions (16-bit)	75
6.1.2	Two-Word Instructions (32-bit)	76
6.1.3	Cycle Counts at a Glance	76
6.2	Instruction Categories	77
6.3	Data Transfer — Moving Values Around	78
6.3.1	MOV — Copy Register	78
6.3.2	MOVW — Copy Register Pair	78
6.3.3	LDI — Load Immediate	78
6.3.4	CLR and SER — Clear and Set All Bits	78
6.4	Arithmetic Instructions	79
6.4.1	ADD — Add Without Carry	79
6.4.2	ADC — Add with Carry	79
6.4.3	SUB — Subtract Without Carry	80
6.4.4	SBC — Subtract with Carry (Borrow)	80
6.4.5	SUBI — Subtract Immediate	80
6.4.6	SBCI — Subtract Immediate with Carry	80
6.4.7	ADIW — Add Immediate to Word	81
6.4.8	SBIW — Subtract Immediate from Word	81
6.4.9	INC and DEC — Increment / Decrement	81
6.4.10	NEG — Two's Complement Negate	82
6.5	Logic Instructions	82
6.5.1	AND / ANDI — Bitwise AND	82
6.5.2	OR / ORI — Bitwise OR	82
6.5.3	EOR — Bitwise XOR (Exclusive OR)	82
6.5.4	COM — One's Complement (Bitwise NOT)	83
6.5.5	TST — Test Register (Non-Destructive)	83

6.6	Comparison Instructions	83
6.6.1	CP — Compare	83
6.6.2	CPC — Compare with Carry	84
6.6.3	CPI — Compare with Immediate	84
6.6.4	CPSE — Compare, Skip if Equal	84
6.7	Conditional Branches — Full Table	84
6.8	Unsigned vs Signed Comparisons	85
6.9	Flag Effects Summary	86
6.10	Worked Example: Compare, Clamp, Negate	86
6.11	Operand Restrictions Cheat-Sheet	87
6.12	Summary	88
6.13	Exercises	88
7	Load and Store	90
7.1	The Memory-Register Model	90
7.1.1	ATtiny3217 Data Space Is Not Just SRAM	90
7.2	Addressing Modes	91
7.3	Direct Addressing: LDS and STS	91
7.3.1	I/O Registers Above 0x3F	92
7.4	Pointer Registers: X, Y, Z	92
7.5	Indirect Addressing: LD and ST	93
7.5.1	Basic Indirect — No Change to Pointer	93
7.5.2	Post-Increment — Walk Forward Through Memory	93
7.5.3	Pre-Decrement — Walk Backward Through Memory	94
7.5.4	All Three Pointers, All Three Modes	94
7.6	Displacement Addressing: LDD and STD	94
7.7	Cycle and Size Comparison	95
7.8	Reading Flash: LPM	96
7.9	Reading Flash: Mapped Access (AVRxt Only)	97
7.9.1	LPM vs Mapped LD — Which to Use?	97
7.10	Pointer Arithmetic	98
7.10.1	Advancing a Pointer by N Bytes	98
7.10.2	Computing an Array Element Address	98
7.11	Atomicity on ATtiny3217	98
7.12	Full Addressing Mode Reference	99
7.13	Worked Example: Reverse a String In-Place	100
7.14	Summary	101
7.15	Exercises	101
8	Unsigned Integer Arithmetic: 8-bit and 16-bit	103
8.1	What “Unsigned” Means to the CPU	103
8.2	The Status Register (SREG) and Unsigned Flags	104
8.2.1	Flag behavior for INC and DEC	104
8.3	8-bit Unsigned Arithmetic	104
8.3.1	Addition: ADD	104
8.3.2	Immediate Addition: No ADDI — Use SUBI with Negated Constant	105
8.3.3	Adding a Constant to r0-r15	105
8.3.4	Increment: INC	105
8.3.5	Subtract: SUB	106
8.3.6	Immediate Subtraction: SUBI	106
8.3.7	Decrement: DEC	106
8.3.8	Arithmetic vs Logic: ADD vs OR / EOR	106
8.4	Unsigned Overflow, Wraparound, and Saturation	107
8.4.1	Wraparound (Modular) Arithmetic	107
8.4.2	Detecting Overflow	107
8.4.3	Saturating Addition — Clamp to 255	107
8.4.4	Saturating Subtraction — Clamp to 0	107
8.5	8-bit Unsigned Comparisons and Branches	108

8.5.1	CP — Compare Two Registers	108
8.5.2	CPI — Compare with Immediate	108
8.5.3	CPSE — Compare and Skip if Equal	108
8.5.4	Unsigned Branch Instructions	108
8.5.5	The Unsigned Branching Mental Model	109
8.5.6	Unsigned Range Check	109
8.6	Moving to 16-bit: Register Pairs	109
8.6.1	Register Pair Convention	109
8.6.2	Loading a 16-bit Constant	110
8.6.3	Copying a 16-bit Value: MOVW	110
8.6.4	Loading a 16-bit Value from SRAM	110
8.7	16-bit Unsigned Addition	111
8.7.1	The Carry Chain: ADD + ADC	111
8.7.2	ADIW — Add Immediate to Word	111
8.7.3	Adding a Larger Immediate Constant (SUBI + SBCI)	112
8.7.4	General Case: ADD + ADC with a Loaded Constant	112
8.8	16-bit Unsigned Subtraction	112
8.8.1	The Borrow Chain: SUB + SBC	112
8.8.2	SBIW — Subtract Immediate from Word	113
8.8.3	Subtracting a Larger Constant: SUBI + SBCI	113
8.9	16-bit Unsigned Comparisons	113
8.9.1	CP + CPC — Compare Bytes Then Compare with Carry	113
8.9.2	Comparing with a 16-bit Immediate	114
8.10	Incrementing and Decrementing 16-bit Values	114
8.10.1	Why INC and DEC Fail for 16-bit	114
8.10.2	The Correct Patterns	114
8.11	Practical Patterns	115
8.11.1	Pattern 1: 8-bit Counter with Rollover Detection	115
8.11.2	Pattern 2: 16-bit Odometer	115
8.11.3	Pattern 3: 16-bit Elapsed Time Check	116
8.11.4	Pattern 4: Saturating 16-bit Addition	116
8.11.5	Pattern 5: Unsigned Range Check on 16-bit Value	116
8.12	Instruction Reference Table	117
8.13	Common Pitfalls	117
8.13.1	Pitfall 1: Using INC/DEC in a Multi-byte Carry Chain	117
8.13.2	Pitfall 2: High Byte Before Low Byte	118
8.13.3	Pitfall 3: SUBI Negation Confusion	118
8.13.4	Pitfall 4: Reading Z After SBC on the Wrong Byte	118
8.13.5	Pitfall 5: ADIW Supported Pairs Only	118
8.13.6	Pitfall 6: Forgetting SBCI When Using SUBI for 16-bit Constant Add	118
8.14	Summary	119
8.15	Exercises	119
9	Signed Arithmetic, Two's Complement, and Sign Extension	121
9.1	Two's Complement from First Principles	121
9.1.1	The Wrapping Number Line	121
9.1.2	The Full 8-bit Signed Map	122
9.1.3	How to Convert a Decimal Negative to Two's Complement	122
9.1.4	Why the Same Addition Works for Both Signed and Unsigned	122
9.2	The Signed Flags: N, V, and S	123
9.2.1	N — Negative Flag	123
9.2.2	V — Signed Overflow Flag	124
9.2.3	S — Sign Flag	124
9.3	Signed 8-bit Arithmetic Instructions	125
9.3.1	ADD and ADC — Signed Addition	125
9.3.2	SUB and SUBI — Signed Subtraction	126
9.3.3	NEG — Two's Complement Negate	126
9.3.4	Absolute Value	127

9.3.5	COM — One's Complement vs NEG	127
9.4	Detecting and Handling Signed Overflow	127
9.4.1	Post-operation Overflow Check	127
9.4.2	The Four Overflow Scenarios for 8-bit Signed Addition	127
9.4.3	Signed Saturating Addition	128
9.4.4	Signed Saturating Subtraction	128
9.5	Signed Comparison and Branches	129
9.5.1	The Signed Branch Set	129
9.5.2	BRLT and BRGE in Practice	129
9.5.3	Signed Range Check	129
9.5.4	BRMI vs BRLT: When They Differ	130
9.6	Sign Extension	131
9.6.1	The Problem: Widening a Signed Value	131
9.6.2	Why Zero Extension Fails for Signed Values	131
9.6.3	The Sign Extension Pattern	131
9.6.4	Alternative Sign Extension Using SBRS	132
9.6.5	Sign Extension as a Subroutine	132
9.6.6	Sign Extension Before 16-bit Arithmetic	132
9.7	Reading Flags Produced by Signed Operations	133
9.7.1	The N Flag After a Load or Move	133
9.7.2	The Z Flag and Signed Zero	133
9.7.3	Flags After Logical Operations	133
9.8	Common Pitfalls	134
9.8.1	Pitfall 1: Using BRLO Instead of BRLT for Signed Comparisons	134
9.8.2	Pitfall 2: ASR vs LSR for Signed Right Shift	134
9.8.3	Pitfall 3: Forgetting NEG 0x80	134
9.8.4	Pitfall 4: Using COM as a Negate	134
9.8.5	Pitfall 5: Inserting Flag-Altering Instructions Between Arithmetic and Branch	135
9.8.6	Pitfall 6: Sign Extension After Narrowing Operations	135
9.9	Instruction Reference for Signed Arithmetic	135
9.10	Worked Examples	136
9.10.1	Example 1: Signed Temperature Comparison	136
9.10.2	Example 2: Signed Delta Between Two Readings	136
9.10.3	Example 3: Signed Absolute Value	136
9.10.4	Example 4: Three-Way Signed Dispatch	137
9.10.5	Example 5: Sign Extension into a 16-bit Accumulator	137
9.11	Summary	137
9.12	Exercises	138
10	Multi-byte Addition, Subtraction, Comparison, and Negation	140
10.1	The Carry Chain Principle	140
10.2	24-bit Unsigned Arithmetic	141
10.2.1	Register Layout for 24-bit Values	141
10.2.2	24-bit Addition: ADD + ADC + ADC	141
10.2.3	24-bit Subtraction: SUB + SBC + SBC	141
10.2.4	Unsigned Overflow and Underflow Detection	142
10.3	32-bit Unsigned Arithmetic	142
10.3.1	Register Layout for 32-bit Values	142
10.3.2	32-bit Addition: ADD + ADC + ADC + ADC	142
10.3.3	32-bit Subtraction: SUB + SBC + SBC + SBC	143
10.3.4	32-bit Signed Overflow	143
10.4	Extending to N Bytes	143
10.4.1	8-byte (64-bit) Addition	143
10.4.2	Adding a Constant to an N-byte Value	143
10.5	The Zero Flag in Multi-byte Chains	144
10.5.1	The SBC and CPC Zero-Flag Rule	144
10.5.2	Reading Z After a Chain	145

10.5.3	Testing Whether a Multi-byte Value Is Zero (Without Subtraction)	145
10.5.4	Why ADC Does Not Have the Same Z Semantics	145
10.6	Multi-byte Comparison	145
10.6.1	Comparing Two 24-bit Values: CP + CPC + CPC	145
10.6.2	Comparing Two 32-bit Values: CP + CPC + CPC + CPC	146
10.6.3	Signed 32-bit Comparison	146
10.6.4	Comparing a Multi-byte Value with a Constant	146
10.7	Multi-byte Negation	147
10.7.1	The Two Equivalent Methods	147
10.7.2	16-bit Negation (Recap)	148
10.7.3	24-bit Negation (Method 1)	149
10.7.4	32-bit Negation (Method 1)	149
10.7.5	Signed Overflow on Multi-byte Negation	150
10.8	Loading and Storing Multi-byte Values from SRAM	150
10.8.1	Little-Endian Storage	150
10.8.2	Loading with LDS (Direct Addressing)	150
10.8.3	Loading with a Pointer Register (More Efficient)	151
10.8.4	Storing with STS	151
10.8.5	Storing with a Pointer Register	151
10.8.6	Atomicity Warning	151
10.9	Practical Patterns	152
10.9.1	Pattern 1: 32-bit Millisecond Uptime Counter	152
10.9.2	Pattern 2: 32-bit Saturating Accumulator	153
10.9.3	Pattern 3: 32-bit Elapsed Time	153
10.9.4	Pattern 4: 32-bit Range Check	153
10.9.5	Pattern 5: Signed 32-bit Addition with Overflow Detection	154
10.10	Instruction Reference	155
10.11	Common Pitfalls	155
10.11.1	Pitfall 1: Using INC to Increment the High Byte of a Multi-byte Value	155
10.11.2	Pitfall 2: Processing High Byte Before Low Byte	156
10.11.3	Pitfall 3: Reading Z Before the Final Instruction in an SBC Chain	156
10.11.4	Pitfall 4: Using BRLO for Signed Multi-byte Comparison	156
10.11.5	Pitfall 5: COM Is Not NEG	156
10.11.6	Pitfall 6: Multi-byte Compare with Immediate — Forgetting to Load Higher Bytes	157
10.11.7	Pitfall 7: Interrupt-Unsafe Multi-byte Reads	157
10.12	Summary	158
10.13	Exercises	158
11	Bit Math: Masks, Shifts, Rotates, Packing, and Field Extraction	160
11.1	The Bitwise Logic Instructions	160
11.1.1	AND — Bitwise AND	160
11.1.2	OR — Bitwise OR	161
11.1.3	EOR — Bitwise Exclusive OR (XOR)	161
11.1.4	COM — Bitwise Complement (One's Complement)	162
11.1.5	Flag Behaviour Summary for Logic Instructions	162
11.2	Masking: Isolating, Clearing, Setting, and Toggling Bits	162
11.2.1	Clearing Specific Bits (AND)	162
11.2.2	Setting Specific Bits (OR)	163
11.2.3	Toggling Specific Bits (EOR)	163
11.2.4	Isolating a Single Bit (AND, Result in-place)	163
11.2.5	Combining Set and Clear: Read-Modify-Write	163
11.2.6	Common Mask Constants	164
11.3	Shift Instructions	164
11.3.1	Shifts as Bit Movement	164
11.3.2	SWAP — Swap Nibbles	165
11.4	Rotate Instructions	165
11.4.1	Rotates vs Shifts	165

11.4.2	Multi-byte Shift Left (LSL + ROL chain)	165
11.4.3	Multi-byte Shift Right (LSR + ROR chain)	166
11.4.4	Multi-byte Arithmetic Shift Right (ASR + ROR chain)	166
11.4.5	Shifting by N Bits	166
11.5	Bit Test and Skip Instructions	167
11.5.1	SBRC / SBRS — Skip if Bit in Register Clear/Set	167
11.5.2	SBIC / SBIS — Skip if I/O Bit Clear/Set	167
11.5.3	SBI / CBI — Set/Clear Bit in I/O Register	167
11.6	The T Flag: Single-Bit Transfer	168
11.7	Field Extraction: Getting Bits n:m from a Register	168
11.7.1	Single-Bit Extraction to Bit 0	168
11.7.2	Multi-bit Field Extraction (General Case)	169
11.7.3	Extracting the Lower Nibble (bits 3:0)	169
11.7.4	Extracting Specific Byte from a Multi-byte Value	169
11.8	Field Insertion: Putting a Value into a Specific Bit Position	170
11.8.1	Single-Bit Insertion	170
11.8.2	Multi-bit Field Insertion (General Case)	170
11.8.3	Inserting a Lower Nibble into the Upper Nibble (SWAP idiom)	170
11.9	Packing: Combining Two Values into One Byte	171
11.9.1	Pack Two Nibbles into One Byte	171
11.9.2	Pack Eight Signal Lines into One Byte	171
11.9.3	Pack a Boolean Array into a Byte	171
11.10	Unpacking: Splitting a Byte into its Fields	172
11.10.1	Unpack Two Nibbles from One Byte	172
11.10.2	Unpack a Byte into Individual Bits	172
11.11	Practical Patterns	173
11.11.1	Pattern 1: Read a GPIO Port and Extract a Pin State	173
11.11.2	Pattern 2: Write a Specific Field of a Peripheral Register	173
11.11.3	Pattern 3: Reverse the Bits of a Byte	173
11.11.4	Pattern 4: Count Set Bits (Population Count / Hamming Weight)	174
11.11.5	Pattern 5: Extract a 3-bit SPI Device Address from a Protocol Frame	174
11.11.6	Pattern 6: Build a SPI Command Byte from Fields	175
11.11.7	Pattern 7: Rotate a 32-bit Value Through a Peripheral (Shift Register Output)	175
11.12	Instruction Reference	176
11.13	Common Pitfalls	176
11.13.1	Pitfall 1: Using OR to Add Numbers	176
11.13.2	Pitfall 2: EOR Does Not Use an Immediate Operand	177
11.13.3	Pitfall 3: COM Is Not NEG	177
11.13.4	Pitfall 4: Logic Instructions Do Not Preserve C	177
11.13.5	Pitfall 5: SBRC/SBRS Skip Only One Instruction	177
11.13.6	Pitfall 6: Field Mask Must Match the Shift Count	177
11.13.7	Pitfall 7: SBI/CBI Only Reach the Low 32 I/O Addresses	178
11.14	Summary	178
11.15	Exercises	179
12	Fixed-Point Arithmetic	180
12.1	What Fixed-Point Is	180
12.2	Representation in Registers	181
12.3	Addition and Subtraction	181
12.3.1	Q8.8 Addition	181
12.3.2	Q8.8 Subtraction	181
12.4	Multiplication	182
12.4.1	The MUL Family	182
12.4.2	Unsigned Q8.8 Multiplication	182
12.4.3	Signed Q8.8 Multiplication	183
12.4.4	Q1.7 Signed Multiply with FMULS	185
12.4.5	Mixed-Sign Fractional Multiply with FMULSU	185

12.5	Division	186
12.5.1	Divide by a Constant: Multiply by Reciprocal	186
12.5.2	Divide by a Power of Two: Arithmetic Shift Right	186
12.5.3	Variable Division: Software Routine	186
12.6	Type Conversion	186
12.6.1	Integer to Fixed-Point	186
12.6.2	Fixed-Point to Integer (Truncation)	187
12.6.3	Fixed-Point to Integer with Rounding	187
12.6.4	Changing Q Format	187
12.7	Rounding	187
12.8	Saturation	187
12.8.1	Saturating Q8.8 Addition (Unsigned)	187
12.8.2	Saturating Q8.8 Addition (Signed)	188
12.8.3	Overflow Detection After Multiplication	188
12.9	Practical Examples	188
12.9.1	Example 1: ADC Gain Scaling	188
12.9.2	Example 2: First-Order Low-Pass Filter	189
12.9.3	Example 3: PID Derivative Term	190
12.10	Choosing a Format	190
12.11	Summary	191
13	Arithmetic and Logic: 16-bit and Beyond	192
13.1	Multi-Byte Addition and Subtraction	192
13.1.1	The Carry Chain Pattern	193
13.1.2	24-bit and 32-bit	193
13.1.3	Adding a Small Constant to a 16-bit Register Pair	193
13.1.4	Negating a 16-bit Value	194
13.2	Shift and Rotate Instructions	195
13.2.1	16-bit Shift Left (Multiply by 2)	195
13.2.2	16-bit Shift Right, Logical (Unsigned Divide by 2)	195
13.2.3	16-bit Shift Right, Arithmetic (Signed Divide by 2)	196
13.2.4	Shifting by N Bits	196
13.2.5	Extracting a Bit Field	196
13.3	SWAP — Swap Nibbles	196
13.4	MUL — Unsigned 8×8 Multiply	197
13.4.1	Self-Multiply (Square)	197
13.4.2	8-bit Fraction Scaling	197
13.5	MULS — Signed 8×8 Multiply	197
13.6	MULSU — Signed × Unsigned Multiply	198
13.7	Fractional Multiply: FMUL, FMULS, FMULSU	198
13.8	Multiply Summary	199
13.9	Division — Software Algorithm	199
13.9.1	Unsigned 8-bit Division	199
13.9.2	Unsigned 16-bit Division	200
13.9.3	Power-of-2 Division — Just Shift	201
13.10	Worked Example: 8×8 Unsigned Multiply to 16-bit Result	201
13.11	Signed Arithmetic Pitfalls	202
13.11.1	Two Mistakes with Signed Division	202
13.11.2	Overflow Detection	202
13.12	Useful Idioms	203
13.12.1	Test if a value is zero without changing the register	203
13.12.2	Conditional absolute value	203
13.12.3	Saturating add (clamp at 255 instead of wrapping)	203
13.12.4	Saturating subtract (clamp at 0 instead of wrapping)	203
13.12.5	Sign extension: 8-bit signed → 16-bit signed	203
13.13	Summary	203
13.14	Exercises	204

14	Branches and Control Flow	205
14.1	Unconditional Jumps	205
14.1.1	RJMP — Relative Jump	205
14.1.2	JMP — Absolute Jump	205
14.1.3	IJMP — Indirect Jump via Z	206
14.2	Conditional Branches — Mechanics	206
14.2.1	Exact Branch Range	206
14.2.2	Signed vs Unsigned Conditions	207
14.3	Complete Branch Instruction Table	207
14.4	Skip Instructions	208
14.4.1	CPSE — Compare, Skip if Equal	208
14.4.2	SBRC — Skip if Bit in Register is Clear	208
14.4.3	SBRS — Skip if Bit in Register is Set	208
14.4.4	SBIC — Skip if Bit in I/O Register is Clear	208
14.4.5	SBIS — Skip if Bit in I/O Register is Set	209
14.4.6	SBIC/SBIS Address Limit on ATtiny3217	209
14.4.7	Skip Over 2-Word Instructions Carefully	209
14.4.8	Skip vs Branch — When to Use Which	210
14.5	Structured Control Flow Patterns	210
14.5.1	Two-way decision	210
14.5.2	Multi-byte comparisons	210
14.5.3	Guard branch	211
14.5.4	Top-tested loop	211
14.5.5	Bottom-tested loop (preferred in assembly)	211
14.5.6	Counted loop	211
14.5.7	Nested loops	212
14.6	Worked Example: Sum an Array	212
14.7	Jump Tables	213
14.7.1	Address Tables with LPM	214
14.7.2	Large Jump Tables with JMP Entries	215
14.8	Polling Loops	215
14.9	Branch Optimisation Notes	216
14.9.1	Bottom-tested saves one instruction per loop	216
14.9.2	Count down, not up	216
14.9.3	Use SBRC/SBRS for single-bit tests	216
14.9.4	Branch delay slots do not exist on AVR	216
14.10	Summary	216
14.11	Exercises	217
15	Subroutines and the Stack	218
15.1	CALL and RCALL	218
15.1.1	Return Address Layout	218
15.1.2	ICALL	219
15.2	RET and RETI	219
15.3	PUSH and POP	220
15.4	Stack Pointer Facts on ATtiny3217	220
15.4.1	Updating SP Safely	220
15.5	Register Ownership	221
15.5.1	The r1 = 0 Rule	221
15.6	Prologue and Epilogue Patterns	222
15.6.1	Minimal Subroutine	222
15.6.2	Saving Preserved Registers	222
15.6.3	Stack Frame with Y	222
15.6.4	When a Frame Is Worth It	223
15.7	Stack Depth Analysis	223
15.8	Worked Example: Recursive Factorial	224
15.8.1	Stack Trace for factorial(4)	224
15.8.2	8 x 16 Multiply Helper	225

15.9	Tail Jumps	225
15.10	Routine Template	226
15.11	Summary	226
15.12	Exercises	227
16	GPIO	228
16.1	GPIO Fundamentals	228
16.2	Classic DDRx / PORTx / PINx (ATmega-Style)	229
16.3	ATtiny3217 Port Architecture	229
16.3.1	VPORT — Virtual Port (Fast Path)	229
16.3.2	PORT — Full Port Registers (Configuration and Atomic Operations)	230
16.4	IN — Read I/O Register	231
16.5	OUT — Write I/O Register	231
16.6	SBI and CBI — Set / Clear Bit in I/O Register	232
16.7	SBIC and SBIS — Skip if Bit Clear / Set	233
16.8	Configuring a GPIO Output	234
16.9	Configuring a GPIO Input with Pull-Up	234
16.10	Debounce	235
16.10.1	Approach 1: Blocking Time Delay	235
16.10.2	Approach 2: Periodic Sampling (Non-Blocking)	236
16.11	Worked Example: Button-Controlled LED	238
16.11.1	Why PORTA_OUTTGL instead of SBI/CBI?	239
16.11.2	Build and Inspect	240
16.12	Summary	240
16.13	Exercises	240
17	Interrupts	242
17.1	Why Interrupts?	242
17.2	The Interrupt Mechanism	242
17.2.1	What Happens When an Interrupt Fires	242
17.2.2	The I Flag (Global Interrupt Enable)	243
17.2.3	Interrupt Priority	243
17.3	SEI and CLI	244
17.3.1	Critical Sections	244
17.4	RETI — Return from Interrupt	244
17.5	The Interrupt Vector Table	245
17.5.1	Layout on ATtiny3217	245
17.5.2	Defining the Vector Table in .S	246
17.6	ISR Prologue and Epilogue	246
17.6.1	Minimal ISR (uses r16 only)	246
17.6.2	Why This Order?	247
17.6.3	Saving Multiple Registers	247
17.6.4	ISR Stack Budget	247
17.7	Nesting, Reentrancy, and Shared State	248
17.7.1	Do interrupts nest by default? No.	248
17.7.2	Deliberate nesting: the level-1 vector	248
17.7.3	What nesting costs: stack	248
17.7.4	Reentrancy	249
17.7.5	Protecting shared state	249
17.7.6	Rules of thumb	250
17.8	Configuring Pin Interrupts on ATtiny3217	250
17.8.1	PINnCTRL Layout	250
17.8.2	PORT INTFLAGS — Clearing Interrupt Flags	250
17.9	ISR Latency and Timing	251
17.10	Worked Example: Pin Interrupt Toggles LED	251
17.10.1	Execution Flow Walkthrough	253
17.10.2	Why Not Debounce in the ISR?	253
17.10.3	Build and Inspect	253

17.11	Summary	253
17.12	Exercises	254
18	Power and Clock Options	256
18.1	The Default Clock	256
18.2	Clock Sources	256
18.3	Main Prescaler	257
18.4	Configuration Change Protection	257
18.5	Waiting for Clock Switches	258
18.6	Sleep Modes	258
18.6.1	Idle	259
18.6.2	Standby	259
18.6.3	Power-down	259
18.6.4	RTC/PIT clock-source changes	259
18.7	Brown-Out and VLM	259
18.8	Worked Example: Change the Prescaler	259
18.9	Worked Example: Power-Down Wake from PIT	260
18.10	Practical Rules	260
18.11	Exercises	261
19	Timer/Counter	262
19.1	Timer Fundamentals	262
19.1.1	What a Timer Actually Is	262
19.1.2	Prescaler	262
19.2	TCA0 on ATtiny3217	263
19.2.1	TCA0 Register Map	263
19.2.2	CTRLA — Control A	264
19.2.3	CTRLB — Control B	264
19.2.4	INTCTRL and INTFLAGS	264
19.3	Waveform Generation Modes	265
19.3.1	Normal Mode (WGMODE = 0x00)	265
19.3.2	FRQ Mode — Frequency Generation (WGMODE = 0x01)	265
19.3.3	Single-Slope PWM Mode (WGMODE = 0x03)	265
19.4	Computing Timer Values	266
19.5	TCA0 Interrupt Vectors	266
19.6	The ISR	267
19.7	Main: Timer Initialization Sequence	268
19.8	Worked Example: 1 Hz Timer Tick	269
19.8.1	Register Usage	269
19.8.2	Timing Analysis	269
19.8.3	Build Commands	269
19.8.4	Verifying with avr-objdump	269
19.9	Extending: Normal Mode with OVF Interrupt	270
19.10	Extending: Single-Slope PWM	271
19.10.1	WO pins, more channels, and dual-slope	272
19.11	Summary	273
19.12	Exercises	274
20	USART (Serial Communication)	275
20.1	UART Framing	275
20.1.1	The Bit Stream	275
20.1.2	8N1 Format	275
20.2	USART0 Registers on ATtiny3217	276
20.2.1	STATUS — Status Register (0x0804)	276
20.2.2	CTRLB — Control B (0x0806)	276
20.2.3	CTRLA — Control A / Interrupt Enables (0x0805)	277
20.2.4	CTRLC — Frame Format (0x0807)	277
20.3	Baud Rate Calculation	278
20.3.1	The Formula	278

20.4	TX/RX Pins on ATtiny3217	278
20.5	Polling Transmit	279
20.5.1	Transmitting a String from Flash	279
20.6	Polling Receive	280
20.6.1	Error Checking	281
20.7	ISR-Driven Receive	281
20.7.1	USART0_RXC Vector	281
20.7.2	RXC ISR	281
20.8	Initialization Sequence	282
20.9	Worked Example: Echo Terminal	283
20.9.1	Data Flow	283
20.9.2	Build and Test	284
20.9.3	Disassembly Check	284
20.10	Circular Buffer for High-Throughput RX	284
20.10.1	The full-versus-empty problem	285
20.10.2	Power-of-two sizing	285
20.10.3	Why no cli/sei is needed	285
20.10.4	Producer: the RXC ISR	286
20.10.5	Consumer: rx_get	287
20.10.6	The drain loop	287
20.10.7	Companion source files	288
20.11	Summary	288
20.12	Exercises	289
21	Bit-Banging a UART	290
21.1	The Serial Frame	290
21.2	From Baud Rate to Cycle Count	291
21.3	A Delay of an Exact Length	291
21.4	Driving the Pin in Constant Time	291
21.5	Assembling the Frame	292
21.6	The Complete Transmitter	292
21.7	Retargeting and Limits	293
21.8	Summary	294
21.9	Exercises	294
22	SPI & TWI (I2C) Overview	295
22.1	SPI Fundamentals	295
22.1.1	SPI Clock Polarity and Phase (Mode)	295
22.1.2	Timing Diagram (Mode 0)	296
22.2	SPI0 Registers on the ATtiny3217	296
22.2.1	SPI0_CTRLA — Control A	296
22.2.2	SPI0_CTRLB — Control B	297
22.2.3	SPI0_INTFLAGS — Interrupt / Status Flags (non-buffered mode)	297
22.2.4	SPI0_DATA — Transmit / Receive	297
22.3	SPI Initialisation (Master Mode)	298
22.4	Transferring a Byte	298
22.5	The 25LC010A SPI EEPROM	299
22.5.1	Key Characteristics	299
22.5.2	Command Set	299
22.5.3	Status Register (returned by RDSR)	299
22.5.4	CS Framing Rules	300
22.6	Example: Write Byte to SPI EEPROM	300
22.6.1	eprom_wren — Send Write Enable	300
22.6.2	eprom_write_byte — Write One Byte	300
22.6.3	eprom_wait_wip — Poll WIP Until Write Done	300
22.6.4	main — Putting It Together	301
22.6.5	Build and Inspect	301
22.7	TWI (I ² C) Fundamentals	301

22.7.1	Bus Topology	302
22.7.2	Transaction Structure	302
22.7.3	I ² C Speeds	302
22.8	TWI0 Master Mode Registers on ATtiny3217	302
22.8.1	TWI0_MCTRLA — Master Control A	303
22.8.2	TWI0_MSTATUS — Master Status	303
22.8.3	TWI0_MBAUD — Baud Rate Register	303
22.8.4	TWI0_MCTRLB — Master Control B (Bus Commands)	304
22.9	TWI Initialisation	304
22.10	Master Write Transaction (Polling)	305
22.10.1	SBRC vs SBRS for RXACK	306
22.11	Example: Write to PCF8574 I ² C I/O Expander	306
22.11.1	Build	306
22.12	Rendering Strings from Program Memory	307
22.13	A Character LCD over I ² C (HD44780 + PCF8574)	307
22.13.1	Backpack Pin Mapping	308
22.13.2	Clocking a Nibble	308
22.13.3	The Awkward Power-Up Sequence	309
22.14	A Graphical OLED over I ² C (SSD1306)	310
22.14.1	Command vs Data Framing	310
22.14.2	The Font Table in Flash	311
22.14.3	LCD vs OLED: Who Owns the Font?	312
22.15	SPI vs TWI Comparison	312
22.16	Summary	312
22.17	Exercises	313
23	CRC and Data Integrity in Assembly	315
23.1	CRC Basics	315
23.2	Software CRC-8 Register Plan	316
23.3	Flash Test Data	316
23.4	CRC-8 Implementation	316
23.5	Standalone Demo	317
23.6	Table-Driven CRC	318
23.7	ATtiny3217 CRCSCAN Peripheral	318
23.8	Starting a CRCSCAN in Assembly	319
23.9	NMI on CRC Failure	320
23.10	Startup CRC Through Fuses	320
23.11	Summary	321
23.12	Exercises	321
24	Huffman Decoding from Flash	322
24.1	Huffman in One Picture	322
24.2	The Bit-Stream Contract	323
24.3	Two Ways to Store the Table	324
24.4	Reading Flash	324
24.5	A Shared Bit Reader	324
24.6	Decoder 1 — Walking the Tree	325
24.7	Decoder 2 — Canonical Decode	326
24.8	Standalone Demo	328
24.9	Regenerating the Tables	330
24.10	When Not to Use Huffman	330
24.11	Summary	330
24.12	Exercises	331
25	Optimisation Techniques	332
25.1	Build and Inspect the Chapter Examples	332
25.2	Cycle Reference for This Chapter	333
25.3	Baseline: One-Byte Copy Loop	333
25.4	Keep Hot Values in Registers	334

25.5	Loop Unrolling	335
25.6	Tail Handling	336
25.7	Flash Lookup Tables	337
25.7.1	Why ADC ZH, r1 Works	338
25.7.2	Flash Address Limits	338
25.7.3	Verifying Table Placement	338
25.8	Interpolating Between Table Entries	338
25.9	Horner's Method for Polynomials	339
25.10	Size Results from the Current Sources	341
25.11	Reading avr-objdump Correctly	342
25.12	Common Errors in Hand Optimisation	342
25.12.1	Optimising Cold Code	342
25.12.2	Ignoring Setup Cost	342
25.12.3	Forgetting the Final Branch	342
25.12.4	Using a Zero Register Without Clearing It	343
25.12.5	Putting Hot State in SRAM	343
25.12.6	Unrolling Until the Source Becomes Fragile	343
25.13	Optimisation Checklist	343
25.14	Source Notes	343
25.15	Exercises	344
26	UPDI — The Unified Program and Debug Interface	345
26.1	ISP vs UPDI: Why Microchip Changed the Interface	345
26.2	The Physical Layer	346
26.2.1	The Single UPDI Pin	346
26.3	Frame Formats	346
26.3.1	BREAK Character	346
26.3.2	SYNCH Character (0x55)	347
26.3.3	Guard Time	347
26.3.4	Baud Rate Limits	347
26.4	UPDI Enable Sequences	347
26.4.1	Standard Enable (RSTPINCFG = 0x1)	348
26.4.2	High-Voltage (HV) Override	348
26.5	UPDI Disabling	349
26.6	The UPDI Instruction Set	349
26.6.1	LDS — Load Data Space, Direct Address	349
26.6.2	STS — Store Data Space, Direct Address	349
26.6.3	LD — Load Data Space, Indirect (via Pointer Register)	350
26.6.4	ST — Store Data Space, Indirect (via Pointer Register)	350
26.6.5	LDCS — Load UPDI Control/Status Register	350
26.6.6	STCS — Store UPDI Control/Status Register	350
26.6.7	REPEAT — Set Repeat Counter	350
26.6.8	KEY — Send Activation Key or Read SIB	351
26.7	Key-Protected Operations	351
26.7.1	Chip Erase	351
26.7.2	NVM Programming	352
26.7.3	User Row Programming on a Locked Device	352
26.8	The UPDI Register Map	353
26.8.1	STATUSA (0x00) — Status A	353
26.8.2	STATUSB (0x01) — Status B	353
26.8.3	CTRLA (0x02) — Control A	353
26.8.4	CTRLB (0x03) — Control B	354
26.8.5	ASI_KEY_STATUS (0x07)	354
26.8.6	ASI_RESET_REQ (0x08)	354
26.8.7	ASI_CTRLA (0x09)	354
26.8.8	ASI_SYS_CTRLA (0x0A)	354
26.8.9	ASI_SYS_STATUS (0x0B)	355
26.8.10	ASI_CRC_STATUS (0x0C)	355

26.9	Hardware Adapters	355
26.9.1	The nEDBG: Curiosity Nano On-Board Programmer	355
26.9.2	jtag2updi (Legacy Firmware Programmer)	356
26.9.3	SerialUPDI: Direct USB Serial Adapter	356
26.10	pymcuprog: Microchip's Python UPDI Tool	358
26.11	On-Chip Debugging (OCD) via UPDI	359
26.12	Fuses That Affect UPDI Access	359
26.13	Troubleshooting	360
26.13.1	Connection Fails Immediately	360
26.13.2	Error: "UPDI contention" or PESIG = 0x7	360
26.13.3	Error: PESIG = 0x1 (Parity) or 0x2 (Frame)	360
26.13.4	Error: PESIG = 0x6 (Bus error)	360
26.13.5	PESIG = 0x3 (Access Layer Timeout)	360
26.13.6	Device Locked (LOCKSTATUS = 1)	360
26.13.7	Recovery from RSTPINCFG = 0x0 (No UPDI Access)	360
26.14	Complete Wire-Level Walk-Through: Programming One Flash Page	361
26.15	Summary	362
27	Self-Programming Flash: NVMCTRL on tinyAVR 1-Series	363
27.1	tinyAVR 1-Series vs Classic AVR: No SPM, NVMCTRL Instead	363
27.1.1	Architecture Family: AVRXMEGA3	363
27.2	Microchip App Notes Used	364
27.2.1	What Replaces SPM?	364
27.2.2	Programming Interface: UPDI	365
27.3	NVMCTRL Overview	365
27.3.1	Flash Parameters	365
27.3.2	NVMCTRL Register Map	365
27.3.3	NVMCTRL_STATUS — Status Register (0x1002)	366
27.3.4	NVMCTRL Commands (written to NVMCTRL_CTRLA)	366
27.4	The Page Buffer and Write Sequence	366
27.4.1	How the Page Buffer Works	366
27.4.2	Complete Flash Page Write Sequence	367
27.4.3	The Z Pointer Convention	367
27.5	CCP — Configuration Change Protection	367
27.5.1	Why CCP Exists	367
27.5.2	The CCP Register (0x0034)	368
27.5.3	The Four-Instruction Rule	368
27.6	Flash Write Routine in Assembly	368
27.6.1	Register Allocation	368
27.6.2	Full Listing: src/nvmctrl_write.S	369
27.6.3	Instruction Notes	370
27.7	Practical Firmware Update: UART-Driven Self-Update	370
27.7.1	Intel HEX Recap	370
27.7.2	Architecture of selfupdate.S	370
27.7.3	Register Allocation for selfupdate.S	371
27.7.4	USART0 Initialisation at 3.333 MHz	371
27.7.5	Full Listing: src/selfupdate.S	371
27.7.6	Jumping to the Updated Application	372
27.7.7	Uploading Firmware	372
27.8	BOOTEND Fuse: Protecting a Bootloader Region	373
27.8.1	The Problem	373
27.8.2	BOOTEND: Defining a Protected Region	373
27.8.3	Flash Layout With BOOTEND = 0x04	373
27.8.4	Application Entry Point	374
27.8.5	Run-Time Locks: BOOTLOCK and APCWP	374
27.9	Building and Flashing	375
27.9.1	Building nvmctrl_write.S (standalone test)	375
27.9.2	Building selfupdate.S	375

27.9.3	Converting to HEX	375
27.9.4	Flashing with pymcuprog	375
27.9.5	Verifying the Disassembly	376
27.9.6	Building an Application for the Protected Layout	376
27.10	Common Pitfalls	376
27.10.1	Pitfall 1 — CCP Window Violated	376
27.10.2	Pitfall 2 — Page Buffer Not Aligned	377
27.10.3	Pitfall 3 — Writing to Flash During Interrupt	377
27.10.4	Pitfall 4 — Forgetting to Wait for FBUSY Before Each Command	377
27.10.5	Pitfall 5 — Writing the Boot Section From Application Code	377
27.10.6	Pitfall 6 — UPDI Port Floating (PA0)	378
27.10.7	Pitfall 7 — Baud Rate at Non-Default Clock	378
27.11	Summary	378
27.12	Exercises	379
28	Bit Manipulation & Fixed-Point Math	380
28.1	Shift Operations	380
28.1.1	Register Pair Diagram — 16-bit LSL	381
28.2	Shift-and-Add Multiplication	381
28.2.1	Algorithm — Unsigned $8 \times 8 \rightarrow 16$	381
28.2.2	Implementation	381
28.2.3	Signed Multiply — mul8s ($8 \times 8 \rightarrow 16$ signed)	382
28.3	Integer Square Root	383
28.4	Fixed-Point Q8.8 Arithmetic	385
28.4.1	Converting to Q8.8	385
28.4.2	Q8.8 Addition	386
28.4.3	Q8.8 Multiplication ($Q8.8 \times Q8.8 \rightarrow Q8.8$)	386
28.5	BCD Arithmetic	388
28.5.1	Decimal Adjust on AVR	388
28.5.2	H Flag and BCD Correction	388
28.6	Binary to BCD Conversion	389
28.6.1	16-bit Binary to 5-Digit BCD	390
28.6.2	Build and Test	391
28.7	Complete Example: Sensor Reading to 7-Segment Display	391
28.7.1	Scaling: ADC $\rightarrow$ Temperature	391
28.7.2	Register Map for display.S	392
28.7.3	7-Segment LUT	392
28.7.4	Scale ADC to Temperature	392
28.7.5	Extract BCD Digits and Drive SPI	393
28.7.6	Build	394
28.8	Instruction Reference for This Chapter	395
28.9	Common Pitfalls	396
28.9.1	Pitfall 1 — Using R1 Without Zeroing After MUL	396
28.9.2	Pitfall 2 — ASR vs LSR on Signed Values	396
28.9.3	Pitfall 3 — Q8.8 Overflow	396
28.9.4	Pitfall 4 — Decide BCD Corrections From the Original Sum	396
28.9.5	Pitfall 5 — Shift Count for Multi-Byte Values	396
28.10	Summary	397
28.11	Exercises	398
29	Real-Time Patterns	399
29.1	Real-Time Fundamentals	399
29.1.1	Two Scheduling Models	399
29.2	Cooperative Task Scheduler Design	400
29.2.1	Task Representation	400
29.2.2	Tick-Driven Scheduler	400
29.2.3	Task Counter Pattern	401
29.3	TCB0 Configuration — 1 ms Tick	401

29.3.1	TCB0 Register Map	401
29.3.2	Initialization Code	402
29.4	ATtiny3217 Interrupt Vector Table	403
29.5	ISR Prologue and Epilogue	404
29.5.1	Minimal Tick ISR	404
29.6	ISR Latency Analysis	405
29.6.1	Sources of Latency	405
29.6.2	Push/Pop Overhead	406
29.6.3	Eliminating Sources of Latency	406
29.6.4	Measuring Latency	406
29.7	Complete Two-Task Scheduler	406
29.7.1	Hardware Setup — Curiosity Nano	406
29.7.2	Register Allocation	407
29.7.3	Source Code	407
29.7.4	Build	411
29.8	Timing Analysis — Worst-Case Response	411
29.9	Non-Blocking Task Extension	412
29.10	Scaling to More Tasks	412
29.10.1	When to Move to a Preemptive Kernel	413
29.11	Summary	413
30	DDS and Phase Accumulator Math	414
30.1	The Phase Accumulator	414
30.1.1	Mapping Accumulator Value to Phase	415
30.1.2	Visualising the Accumulator	415
30.2	The DDS Frequency Equation	415
30.2.1	Nyquist Limit	415
30.3	Choosing the Accumulator Width	416
30.3.1	8-bit Accumulator	416
30.3.2	16-bit Accumulator	416
30.3.3	32-bit Accumulator	416
30.4	Sample Rate on ATtiny3217	416
30.4.1	TCB0 Sample Clock Calculation	416
30.4.2	TCB0 Register Map (from ATtiny3217 data sheet)	417
30.5	Tuning Word Computation	417
30.5.1	Common Musical Notes	417
30.5.2	Overflow Risk in Tuning Word Calculation	418
30.6	The Waveform Table	418
30.6.1	Sine Table Format	418
30.6.2	Generating the Table	418
30.6.3	Table Placement and Alignment	419
30.7	AVR Assembly Implementation	419
30.7.1	SRAM Variable Layout	419
30.7.2	Register Plan	419
30.7.3	Cycle Budget	420
30.7.4	TCB0 Initialisation	420
30.7.5	TCA0 8-bit PWM Initialisation	421
30.7.6	DDS Interrupt Service Routine	421
30.7.7	Reset Handler and Startup	423
30.8	Complete Source Listing	423
30.8.1	Vector Table	423
30.8.2	Build	424
30.9	Frequency Accuracy and Phase Truncation	424
30.9.1	Two Sources of Frequency Error	424
30.9.2	Phase Truncation and Spurious Tones	425
30.9.3	Worst-Case Spur Frequency	425
30.10	Phase and Frequency Modulation	426
30.10.1	Phase Modulation	426

30.10.2	Frequency Modulation	426
30.10.3	Linear Frequency Sweep (Chirp)	426
30.11	Two-Tone Mixing	427
30.12	32-bit Accumulator Variant	428
30.12.1	SRAM Layout	428
30.12.2	Tuning Word	428
30.12.3	ISR Accumulator Update	428
30.13	Common Pitfalls	429
30.13.1	Pitfall 1 — Aliasing Above Nyquist	429
30.13.2	Pitfall 2 — Overflow in Tuning Word Calculation	429
30.13.3	Pitfall 3 — Unaligned Sine Table	429
30.13.4	Pitfall 4 — Forgetting to Clear the TCB0 Interrupt Flag	429
30.13.5	Pitfall 5 — Phase Accumulator Not Zeroed Before Starting	430
30.13.6	Pitfall 6 — Interrupt Latency Jitter Adds Phase Noise	430
30.14	Relationship to CORDIC	430
30.15	Summary	430
30.16	Exercises	431
31	CORDIC on AVR	433
31.1	Why Use CORDIC on AVR?	433
31.1.1	When Not to Use It	434
31.2	The Core Idea	434
31.2.1	Rotation Mode	434
31.2.2	Vectoring Mode	434
31.3	Fixed-Point Choices for AVR	434
31.3.1	Vector Format: Q1.15	435
31.3.2	Angle Format: BAM16	435
31.3.3	CORDIC Gain	435
31.4	The Angle Table	435
31.4.1	Convergence Range	436
31.5	Rotation Mode for Sine and Cosine	436
31.5.1	Example Values	437
31.6	AVR Assembly Structure	437
31.6.1	Table Layout	437
31.6.2	Core Loop Skeleton	437
31.7	How This Maps to AVR Assembly	438
31.7.1	Example: One Unrolled Stage ($i = 3$)	438
31.7.2	Practical Register Plan	439
31.8	Vectoring Mode: Magnitude and Angle	439
31.8.1	Where AVR Uses This	440
31.9	Example 1: DDS / NCO Sine Generator	440
31.10	Example 2: Tilt Angle from Two Axes	440
31.11	Accuracy, Flash, and Cycle Tradeoffs	441
31.11.1	Iteration Count	441
31.11.2	LUT vs CORDIC	441
31.12	Common Pitfalls	441
31.12.1	Pitfall 1 — Forgetting Quadrant Reduction	441
31.12.2	Pitfall 2 — Using Logical Right Shift on Signed Values	441
31.12.3	Pitfall 3 — Ignoring the Gain	442
31.12.4	Pitfall 4 — Overflow in Narrow Formats	442
31.12.5	Pitfall 5 — Hand-Writing the Loop Before Proving Correctness	442
31.13	Summary	442
31.14	Exercises	443
32	ADC, Analog Comparator, and DAC	444
32.1	The ADC Mental Model	444
32.2	ADC0 Register Path	445
32.3	Choosing an Analog Pin	445

32.4	Choosing the Reference	445
32.4.1	SAMPCAP	446
32.5	Choosing the ADC Clock	446
32.6	Sampling Time	447
32.7	The CALIB Register	447
32.7.1	Software Calibration	448
32.8	Polling Conversion Routine	448
32.9	Complete Example: Read AIN4 and Drive an LED	449
32.10	ISR-Driven Conversion	451
32.10.1	When to Poll and When to Use an ISR	451
32.10.2	Setting Up the Interrupt	451
32.10.3	The ISR	451
32.10.4	Reading the Result in the Main Loop	452
32.10.5	Keeping the ISR Short	452
32.11	Accumulation and Simple Filtering	452
32.11.1	Automatic Sampling Delay Variation (ASDV)	452
32.12	Free-Running Mode	453
32.13	Hardware Sample Accumulation	453
32.14	Window Comparator	454
32.15	Event-Triggered Conversion	455
32.16	Temperature Measurement	455
32.16.1	Signature Row Calibration Values	455
32.16.2	Setup for Temperature Sampling	456
32.16.3	Converting Raw ADC to Kelvin	456
32.16.4	Supply Voltage Estimation	457
32.17	The Analog Comparator (AC0)	457
32.17.1	Inputs and the OUT pin	458
32.17.2	Register path	458
32.17.3	Example: threshold detector on PA7	458
32.17.4	Hysteresis	459
32.18	The Digital-to-Analog Converter (DAC0)	459
32.18.1	Register path and init order	460
32.19	Common Pitfalls	460
32.20	Summary	461
32.21	Exercises	461
33	Integer Filters for ADC and Control	463
33.1	Noise Sources and Filter Choices	463
33.2	Exponential Moving Average	464
33.2.1	Algorithm	464
33.2.2	Fixed-Point Implementation	464
33.2.3	Accumulator Width	464
33.2.4	Cutoff Frequency	465
33.2.5	Assembly: 16-bit Accumulator ($k = 4$)	465
33.2.6	Initialisation	466
33.2.7	Assembly: 24-bit Accumulator ($k = 8$)	467
33.3	Box Filter (N-Sample Moving Average)	468
33.3.1	Algorithm	468
33.3.2	Implementation: $N = 8$ ($M = 3$ shifts)	468
33.3.3	Box Filter Initialisation	469
33.3.4	Frequency Response	470
33.4	Median Filter (3-Sample)	470
33.4.1	Algorithm: Sorting Network	470
33.4.2	SRAM Layout	470
33.4.3	Assembly Implementation	471
33.5	Cascaded EMA (Second-Order Low-Pass)	472
33.5.1	SRAM Layout	472
33.5.2	Implementation	472

33.6	Integer PID Structure	474
33.6.1	Fixed-Point Representation	474
33.6.2	SRAM Layout	474
33.6.3	Proportional Term	475
33.6.4	Integral Term with Anti-Windup	475
33.6.5	Derivative Term	476
33.6.6	Anti-Windup	477
33.6.7	Assembling the Full PID Output	479
33.6.8	Output Clamping	480
33.7	Filter Selection Guide	480
33.7.1	SRAM Cost Summary	481
33.8	Common Pitfalls	481
33.8.1	Pitfall 1 — Using LSR/ROR on a Signed Value	481
33.8.2	Pitfall 2 — Accumulator Overflow	481
33.8.3	Pitfall 3 — Not Initialising the Accumulator	481
33.8.4	Pitfall 4 — Updating the Filter from Two Places	481
33.8.5	Pitfall 5 — Box Filter Zero Not Re-Zeroed After Reset	481
33.8.6	Pitfall 6 — Integral Windup Without Clamping	482
33.8.7	Pitfall 7 — Derivative Amplifying Noise	482
33.9	Summary	482
33.10	Exercises	483
34	Event System and Configurable Custom Logic	484
34.1	Chapter structure (read this first)	484
34.2	Why Route Events in Hardware?	484
34.3	EVSYS Architecture	485
34.4	Asynchronous vs Synchronous Channels	485
34.4.1	Generators (what can drive a channel)	486
34.4.2	Users (what can listen to a channel)	486
34.4.3	A trap: <code>_gc</code> constants do not assemble	486
34.5	Example: Mirror a Pin to an Event-Output Pin	486
34.6	Example: Trigger an ADC Conversion From a Timer Event	487
34.7	CCL Architecture	487
34.8	Example: A LUT as a Simple Gate Driving a Pin	488
34.9	Example: Feeding a LUT From an Event (EVSYS + CCL Together)	489
34.10	What You Learned	489
34.11	Register Quick Reference	490
34.12	Exercises	490
34.13	Write Endurance and Wear	493
34.14	Integrity: Surviving a Botched Write	493
34.15	Erasing	494
34.16	Common Pitfalls	494
34.17	Summary	494
34.18	Exercises	495
35	Fuses and Device Configuration	496
35.1	What a Fuse Is	496
35.2	The Fuse Map	496
35.2.1	The fuses that bite	497
35.3	Reading a Fuse at Run Time	497
35.4	Programming Fuses (UPDI Workflow)	498
35.5	Recovery and UPDI-Safe Habits	498
35.6	Common Pitfalls	498
35.7	Summary	499
35.8	Exercises	499
36	Watchdog and Reset Recovery	500
36.1	The Watchdog Mental Model	500
36.2	Arming the Watchdog	501

36.2.1	Where to pet — and where not to	501
36.2.2	Window mode and the lock	501
36.3	Knowing Why You Reset: RSTCTRL	502
36.4	Software Reset	502
36.5	A Fault-Counter Policy	502
36.6	Common Pitfalls	503
36.7	Summary	503
36.8	Exercises	504
37	Defensive Firmware	505
37.1	Why Defensive Firmware	505
37.2	Verifying Flash: the CRCSCAN Peripheral	505
37.2.1	Applying a run-time scan	506
37.3	Testing RAM	506
37.3.1	The Catch-22 of Testing Your Own Memory	507
37.3.2	March C- in Assembly	507
37.3.3	Wiring It Into the Boot	508
37.4	Version and Identity Metadata	509
37.5	Safe Peripheral Initialization	509
37.6	Orchestrating a Defensive Boot	510
37.7	Common Pitfalls	510
37.8	Summary	511
37.9	Exercises	511
38	C and Assembly Integration	513
38.1	The avr-gcc ABI	513
38.1.1	Register classes	513
38.1.2	Argument passing	513
38.1.3	Return values	514
38.2	Calling Assembly from C	514
38.3	Passing Pointers and Arrays	514
38.4	Calling C from Assembly	515
38.5	Inline Assembly	516
38.5.1	Constraints	516
38.5.2	Clobbers, volatile, and "memory"	516
38.6	Sharing Data and Linker Symbols	517
38.7	Common Pitfalls	517
38.8	Summary	517
38.9	Exercises	518
39	Polynomial and Rational Approximation	519
39.1	Why Not Taylor	519
39.2	Minimax and Equioscillation	519
39.3	Range Reduction	520
39.4	Evaluating in Fixed Point	520
39.4.1	Worked example: Q15 sine	520
39.5	Rational Approximation	521
39.6	Choosing an Approach	522
39.7	Common Pitfalls	522
39.8	Summary	522
39.9	Exercises	523
40	Fixed-Point Matrix Transforms	524
40.1	Matrices in Fixed Point	524
40.2	The Core Operation: Matrix $\times$ Vector	524
40.3	Worked Example: 2×2 Transform	525
40.4	Staying in Range	526
40.5	Composing Transforms	526
40.6	Beyond 2×2 : Homogeneous and Projection	526

40.7	Common Pitfalls	527
40.8	Summary	527
40.9	Exercises	528
41	Build and Linker Workflow	529
41.1	The Pipeline	529
41.2	Separate Compilation	529
41.3	Dropping Unused Code: <code>--gc-sections</code>	530
41.4	Reading the Map File	530
41.5	A Project Makefile	531
41.6	Reproducible Builds	532
41.7	Common Pitfalls	532
41.8	Summary	532
41.9	Exercises	533
A	ATtiny3217 Register Reference	534
A.1	CPU Registers	534
A.1.1	SREG Bit Layout	534
A.1.2	Configuration Change Protection (CCP)	535
A.2	CLKCTRL — Clock Controller	535
A.2.1	MCLKCTRLA — Main Clock Source Select	535
A.2.2	MCLKCTRLB — Main Clock Prescaler	535
A.2.3	MCLKSTATUS — Status (read-only)	536
A.3	SLPCTRL — Sleep Controller	536
A.3.1	CTRLA — Sleep Control	536
A.4	BOD — Brown-Out Detector / Voltage Level Monitor	536
A.4.1	BOD_CTRLA	537
A.4.2	VLM Registers	537
A.5	GPIO Registers	537
A.5.1	Port Base Addresses	537
A.5.2	Per-Port Register Map (offset from base)	537
A.5.3	PORTA Absolute Addresses	538
A.5.4	PORTB Absolute Addresses	538
A.5.5	PORTC Absolute Addresses	539
A.5.6	PINnCTRL Bit Layout	539
A.5.7	Virtual PORT Registers (VPORT) — Fast I/O	539
A.6	Curiosity Nano Pin Assignments	540
A.6.1	LED and Button Quick Reference	540
A.7	TCA0 — Timer/Counter Type A (16-bit)	540
A.7.1	CTRLA — Prescaler and Enable	541
A.7.2	CTRLB — Waveform Mode	541
A.8	TCB0 / TCB1 — Timer/Counter Type B (16-bit)	541
A.8.1	CTRLA — Clock Select and Enable	542
A.8.2	CTRLB — Mode (CMODE)	542
A.9	USART0	542
A.9.1	STATUS Bits	543
A.9.2	CTRLA — Interrupt Enables	543
A.9.3	CTRLB — Enable Bits	543
A.9.4	CTRLC — Frame Format	543
A.9.5	Baud Rate Formula and Common Values	544
A.10	SPI0	544
A.10.1	CTRLA Bits	544
A.10.2	SPI Clock Rates	544
A.11	TWI0 — Two-Wire Interface (I2C)	545
A.11.1	MSTATUS Bits	545
A.11.2	MCTRLB — Master Command	545
A.11.3	Baud Rate Formula	545
A.12	ADC0	545

A.12.1	CTRLA Bits	546
A.12.2	CTRLC — Prescaler and Reference	546
A.12.3	MUXPOS — Input Mux	546
A.12.4	Internal Reference Voltage	547
A.13	NVMCTRL — Non-Volatile Memory Controller	547
A.13.1	CTRLA — NVM Commands	547
A.13.2	STATUS Bits	548
A.13.3	Memory Layout	548
A.14	Interrupt Vector Table	548
A.15	Fuses	549
A.15.1	OSCCFG (0x1282)	549
A.15.2	SYSCFG0 (0x1285)	549
A.15.3	Writing Fuses with avrdude	550
B	AVR Instruction Set Summary	551
B.1	How to Read the AVR Instruction Set Manual	551
B.1.1	What One Instruction Page Contains	551
B.1.2	Start with Syntax, but Trust the Operand Limits	552
B.1.3	Read “Words” Before “Cycles”	552
B.1.4	Relative Addresses Are Not Absolute Addresses	552
B.1.5	Read the SREG Section as “What Can I Test Next?”	553
B.1.6	Do Not Panic About the Boolean Formulas	553
B.1.7	Cycles Are Family-Specific	554
B.1.8	Skip Instructions Need Extra Attention	554
B.1.9	Learn the Address Spaces Separately	554
B.1.10	Aliases and Pseudo-Instructions Can Hide the Real Opcode	555
B.1.11	A Good Manual-Reading Workflow	555
B.1.12	What This Means for the Rest of the Book	555
B.2	Arithmetic and Logic	555
B.3	Shift and Rotate	556
B.4	Bit Operations	556
B.5	Compare and Test	557
B.6	Branch Instructions	557
B.6.1	Skip Instructions	558
B.7	Load and Store	558
B.7.1	Load from Program Memory (Flash)	559
B.8	I/O Instructions	559
B.9	Miscellaneous	559
B.10	Instruction Encoding Summary	559
B.11	GCC ABI Register Conventions	559
C	GAS Directive Reference	561
C.1	Section Directives	561
C.2	Symbol Directives	561
C.3	Data Directives	562
C.4	Alignment Directives	562
C.5	Conditional Assembly	562
C.6	Macro Directives	563
C.7	Include Directives	563
C.8	Listing and Debug Directives	564
C.9	Expression Operators	564
C.10	Common Patterns	564
C.10.1	Define a constant	564
C.10.2	Reserve SRAM variable	564
C.10.3	Initialised variable (copied to SRAM at startup by avr-libc __init)	565
C.10.4	Constant in flash (accessed via LPM)	565
C.10.5	Local labels	565
D	Linker Script Walkthrough	566

D.1	Anatomy of a Linker Script	566
D.1.1	Minimal AVR Linker Script	566
D.2	MEMORY Region Flags	567
D.3	Address Spaces on AVR	568
D.4	VMA vs LMA	568
D.5	KEEP()	569
D.6	Bootloader Placement	569
D.7	Parameter Page at Fixed Flash Address	569
D.8	.noinit Section	570
D.9	Useful Linker Symbols	570
D.10	Inspecting Linker Output	571
D.11	Common Linker Errors	571
E	avrdude Flashing Cheat-Sheet	572
E.1	Basic Syntax	572
E.1.1	-U Flag Format	572
E.2	Most-Used Commands	573
E.2.1	Flash a HEX file	573
E.2.2	Read flash to file	573
E.2.3	Verify only (no write)	573
E.2.4	Erase then write	573
E.2.5	Write EEPROM	573
E.2.6	Read fuse bytes	573
E.2.7	Write fuse bytes	574
E.2.8	Multiple operations in one command	574
E.3	Common Programmers	574
E.3.1	UPDI Devices (ATtiny3217 and tinyAVR 1-series)	574
E.4	Device Part Numbers	575
E.5	Fuse Byte Reference (ATmega328P)	575
E.5.1	Low Fuse (LFUSE)	575
E.5.2	High Fuse (HFUSE)	576
E.5.3	Extended Fuse (EFUSE)	576
E.5.4	Online Fuse Calculator	576
E.6	Generating Intel HEX from ELF	576
E.7	Complete Build + Flash Script	576
E.8	Troubleshooting	577
E.8.1	USBasp Slow Clock	577
E.8.2	Arduino Auto-Reset Circuit	578
F	Inspecting Your Binaries	579
F.1	The worked example	579
F.2	avr-size - how much flash and RAM	580
F.3	avr-objdump - see the machine code	581
F.4	avr-objcopy - make something to flash	583
F.5	avr-nm - list symbols and their addresses	584
F.6	avr-readelf - inspect the ELF structure	585
F.7	avr-addr2line - translate an address to source	586
F.8	Other tools at a glance	587
F.9	Putting it together	587

Chapter 1

Why Assembly? Toolchain Setup

Tools used in this book

Every disassembly listing, symbol size, Intel HEX dump, and cycle count in this book was produced with the GNU AVR toolchain below, targeting the ATtiny3217. Output on other versions should match closely, but exact byte addresses, label spellings, and section layout can shift between releases.

avr-gcc	(GCC) 16.1.0	assembler/linker driver; -mmcu=attiny3217
GNU Binutils	2.45	avr-as, avr-ld, avr-objcopy, avr-objdump, avr-nm, avr-readelf, avr-size, avr-ar
avr-libc		device headers (<avr/io.h>), linker scripts
avrdude		flashing over UPDI (see Appendix E)
avr-gdb		source-level debugging on hardware (Ch 3a)

Source for every example lives under `src/`, organised by chapter. Each routine is assembled exactly as shown; where a listing reports addresses or sizes, those numbers are copied from the real linked ELF, not transcribed by hand. The assembler is the GNU Assembler (`avr-as`); this book uses AT&T/GAS syntax throughout, never a vendor or Intel-syntax assembler.

1.1 What Is Assembly Language?

Every program your computer runs is ultimately a sequence of binary numbers: machine code that the CPU decodes and executes. Assembly language is the human-readable form of that machine code. You are no longer asking a compiler to choose the instructions for you; you are choosing them yourself.

Higher-level languages (C, Python, Rust) are *compiled* or *interpreted* — they add layers of abstraction that trade control for convenience. Assembly gives you almost no abstraction: you decide every instruction, every register, and often every cycle count.

1.1.1 When do you actually need assembly?

Situation	Why assembly wins
Tight timing constraints	You control the exact cycle count
Hardware bring-up / no OS	No runtime support — you write everything
Interrupt service routines	Worst-case latency is deterministic
Extremely small flash budget	No compiler overhead or padding
Learning how CPUs work	Forces deep understanding of architecture
Hand-optimising hot paths	Compiler output is a starting point, not the ceiling

In the embedded world, assembly is not just historical trivia. It still shows up in bootloaders, startup code, interrupt paths, cycle-counted loops, and the smallest pieces of firmware where you cannot afford a vague understanding of what the CPU will do next.

1.2 Target: ATtiny3217

This book uses the **ATtiny3217** — an 8-bit AVR from Microchip’s tinyAVR 1-series. It is a modern, capable microcontroller with:

Feature	Detail
Architecture	AVRxt (enhanced AVR core)
Flash	32 KB
SRAM	2 KB (at 0x3800–0x3FFF)
Clock	Up to 20 MHz internal oscillator
Programming	UPDI (1-wire, replaces ISP)
I/O	New-style PORT peripheral (not classic DDRx/PORTx)
Packages	24-pin VQFN, SOIC, SSOP

The **ATtiny3217 Curiosity Nano** development board is a USB-powered board with an onboard debugger/programmer. It has one yellow LED on **PA3** and exposes all device pins. This book uses this board for all hardware examples.

That choice is deliberate. The chip is modern enough to show the newer AVR peripheral model, but small enough that the architecture is still easy to hold in your head.

Note: All concepts carry over to other tinyAVR 1-series devices (ATtiny3216, ATtiny1617, etc.) and with small adjustments to any AVR.

1.3 The AVR-GAS Toolchain

We use the **GNU Assembler** (`avr-as`), part of the `avr-gcc` toolchain. This is the same assembler `avr-gcc` uses under the hood when it compiles C for AVR. Using it directly keeps the toolchain familiar while removing the compiler as a middleman.

The toolchain has four main tools:

```
avr-as      Assembler: .S source → .o object file
avr-ld      Linker:    .o object(s) → .elf executable
avr-objcopy Converter: .elf → .hex (Intel HEX for flashing)
avr-objdump Inspector: disassemble, inspect sections, count cycles
```

You do not need to memorise all four on day one. The practical workflow is: assemble source into an object file, link it into an executable image, convert that image into something the programmer can flash, then inspect the result when you want to verify what really happened.

1.3.1 Installing the Toolchain

Debian / Ubuntu / Kali:

```
sudo apt install gcc-avr binutils-avr avr-libc avrdude
```

`avr-libc` is needed for the device header files (`<avr/io.h>`) that define register names and addresses.

Arch Linux:

```
sudo pacman -S avr-gcc avr-binutils avr-libc avrdude
```

macOS (Homebrew):

```
brew tap osx-cross/avr
brew install avr-gcc avrdude
```

Verify installation:

```
avr-as --version
avr-ld --version
avr-objcopy --version
```

Expected output (version numbers may differ):

```
GNU assembler (GNU Binutils) 2.45
GNU ld (GNU Binutils) 2.45
GNU objcopy (GNU Binutils) 2.45
```

1.4 The Build Pipeline

Source goes through four stages before reaching the MCU:

```

blink.S
|
|   avr-as -mmcu=attiny3217 -o blink.o blink.S
|
▼
blink.o           ← relocatable object (ELF, machine code + symbol table)
|
|   avr-gcc -mmcu=attiny3217 -nostartfiles -o blink.elf blink.o
|
▼
blink.elf         ← linked executable (ELF, absolute addresses assigned)
|
|   avr-objcopy -O ihex blink.elf blink.hex
|
▼
blink.hex         ← Intel HEX – what avrdude writes to flash
|
|   pymcuprog write -d attiny3217 -t uart -f blink.hex
|
▼
ATTiny3217 flash ← running program

```

If you are new to embedded builds, this pipeline is worth pausing over. The source file is not flashed directly. It is first turned into machine code, then assigned final addresses, then repackaged into a format the programmer can write into the device’s flash memory.

1.4.1 Why two link steps?

We use `avr-gcc` as the linker driver rather than calling `avr-ld` directly because it already knows the right device-specific details: the memory layout, the default linker script, and the expected section placement. We could drive the linker manually, but that adds complexity before it teaches us anything useful.

The `-nostartfiles` flag matters because we are not building a normal C program. We do not want the C runtime to insert its own startup sequence; we want to write the reset path ourselves so we can see exactly how execution begins.

1.5 First Program: Blink

The classic “hello world” of embedded is not printing text. It is making a pin change state in a visible, repeatable way. Here, that means toggling the board LED forever.

Hardware: ATtiny3217 Curiosity Nano, LED0 on PA3.

File: ch01_intro/src/blink.S

1.5.1 Understanding the ATtiny3217 PORT Peripheral

The ATtiny3217 uses a **new-style** PORT peripheral, unlike older AVR devices (ATmega328P etc.) which used DDRx, PORTx, PINx registers in low I/O space.

On ATtiny3217, each port has a block of memory-mapped registers. PORTA lives at base address **0x0400**:

Offset	Register	Function
0x00	PORTA_DIR	Direction: 1 = output, 0 = input
0x01	PORTA_DIRSET	Write 1 to set pin as output (non-destructive)
0x02	PORTA_DIRCLR	Write 1 to set pin as input (non-destructive)
0x03	PORTA_DIRTGL	Write 1 to toggle direction
0x04	PORTA_OUT	Output value
0x05	PORTA_OUTSET	Write 1 to drive pin HIGH (non-destructive)
0x06	PORTA_OUTCLR	Write 1 to drive pin LOW (non-destructive)
0x07	PORTA_OUTTGL	Write 1 to toggle pin (atomic!)
0x08	PORTA_IN	Read current pin state

The SET/CLR/TGL variants are key: they let you change one pin without reading the register first, eliminating the classic read-modify-write race condition.

That is one of the nicest features of the newer AVR GPIO design. Instead of reading the whole port, modifying one bit in software, and writing the whole byte back, the peripheral can perform the bit change for you.

Because these registers are above address 0x3F, we cannot use IN/OUT instructions (which only reach 0x00–0x3F). We use STS (store to SRAM address) instead. The header `<avr/io.h>` defines all register names for us.

So even in this tiny program, the hardware layout is already shaping the instruction choices. We are not using STS by style preference; we are using it because that is the instruction family that can reach these addresses.

1.5.2 The Code

```
#include <avr/io.h>

/* — Vector table ————— */
.section .vectors, "ax", @progbits
.global __vectors

__vectors:
    jmp     main           ; reset vector: first instruction after power-on
```

When the ATtiny3217 resets, execution begins at **byte address 0x0000**. The CPU does not “look for main” by name. It simply fetches whatever instruction is stored at the reset vector and executes it. We place a JMP main there so the first thing the chip does is branch into our real program.

That is an important mental shift if you are coming from C: function names are for the assembler and linker, not for the CPU. The CPU only sees addresses and instructions.

```
.section .text

main:
/* — Stack pointer init — */
ldi    r16, lo8(RAMEND)    ; RAMEND = 0x3FFF → lo8 = 0xFF
out     SPL, r16
ldi    r16, hi8(RAMEND)    ; hi8 = 0x3F
out     SPH, r16
```

SP (the stack pointer) must be valid before any PUSH, CALL, or RCALL. If it is wrong, the CPU will store return addresses and saved registers in the wrong place, and the program will collapse quickly.

On the ATtiny3217, reset hardware already loads SP = RAMEND, so these writes are technically redundant in this exact program. We still do them because they make the startup state explicit, and because this is the pattern you need on many older AVR parts where the hardware does not fully set the stack up for you.

RAMEND expands to 0x3FFF (last byte of ATtiny3217 SRAM). lo8() and hi8() are assembler expressions that extract the low and high bytes of a 16-bit value.

SPL and SPH are in low I/O space (addresses 0x3D and 0x3E), so we reach them with OUT.

```
ldi    r16, (1 << 3)      ; bitmask for PA3 = 0x08
sts     PORTA_DIRSET, r16  ; set PA3 as output
```

LDI loads a constant into a register. STS stores that register value to a memory-mapped peripheral address. PORTA_DIRSET expands to 0x0401, and writing 0x08 there means “set bit 3 of the direction register.”

In plain language: we are telling the chip that PA3 is now an output pin. We are not yet turning the LED on or off; we are only telling the hardware that this pin should be driven by the CPU rather than treated as an input.

```
loop:
sts     PORTA_OUTTGL, r16  ; atomically toggle PA3
```

r16 still holds 0x08 from the DIRSET store above, and nothing in the delay loop below touches r16, so we do not need to reload it on each iteration.

PORTA_OUTTGL flips PA3’s output state. If it was high, it becomes low; if it was low, it becomes high. The useful part is that the flip happens inside the hardware peripheral. We do not have to read the old state first, and there is no risk of losing a concurrent change while doing a software read-modify-write.

```
ldi    r19, 25
delay_outer:
ldi    r25, 0
ldi    r24, 0
delay_inner:
sbw    r24, 1                ; subtract 1 from r25:r24 (2 cycles)
brne   delay_inner          ; branch if not zero (2 cycles)
dec    r19
brne   delay_outer

rjmp   loop
```

SBIW subtracts an immediate from a 16-bit register pair. Here, r25:r24 acts as the inner delay counter. It counts down from 65535 to 0. Each time it reaches zero, BRNE stops looping, DEC r19 reduces the outer counter by one, and the whole process repeats until the outer loop also finishes.

This is not a precise timer. It is a simple busy-wait delay that burns CPU cycles. That

makes it ideal for a first example because you can understand it with nothing more than arithmetic and branches.

After reset, the ATtiny3217 does not normally run at the full oscillator rate. It starts from OSC20M with a **divide-by-6 prescaler**. That gives **3.3 MHz** if the FUSE.OSCCFG frequency select is 20 MHz, or **2.66 MHz** if it is 16 MHz. Assuming the 20 MHz setting, the loop timing is roughly: $25 \times 65536 \times 4 \text{ cycles} \div 3,330,000 \approx \mathbf{2 \text{ seconds}}$ per toggle (about 1 Hz blink). If you later change the clock prescaler, oscillator source, or fuse setting, the blink rate changes proportionally.

That last point is worth remembering early: software delay loops are only as accurate as your clock configuration. Change the clock, and the visible timing changes too.

Instruction introduced: LDI LDI Rd, K — Load immediate value K into register Rd. Rd must be r16–r31. 1 cycle. Affects no flags.

Instruction introduced: OUT OUT A, Rr — Write register Rr to I/O address A (0x00–0x3F). 1 cycle. Affects no flags.

Instruction introduced: STS STS k, Rr — Store register Rr to SRAM address k (0x0000–0xFFFF). 2 cycles. Affects no flags. 32-bit instruction (two 16-bit words).

Instruction introduced: SBIW SBIW Rd, K — Subtract immediate K (0–63) from register pair Rd+1:Rd. Valid pairs: r25:r24, r27:r26 (X), r29:r28 (Y), r31:r30 (Z). 2 cycles. Affects C, Z, N, V, S flags.

Instruction introduced: BRNE BRNE k — Branch if Z flag is clear (last result was not zero). 2 cycles if taken, 1 cycle if not. Range: ± 63 words from current PC.

Instruction introduced: DEC DEC Rd — Decrement register Rd by 1. 1 cycle. Affects Z, N, V, S flags. Does NOT affect C flag — careful when using in multi-byte arithmetic.

Instruction introduced: RJMP RJMP k — Relative jump. PC = PC + k + 1. Range: ± 2047 words. 2 cycles. Affects no flags.

Instruction introduced: JMP JMP k — Absolute jump to any address in 64K word program space. 3 cycles. 32-bit instruction. Affects no flags.

1.6 Building and Flashing

1.6.1 Step 1: Assemble

```
cd book/ch01_intro/src
avr-as -mmcu=attiny3217 -o blink.o blink.S
```

-mmcu=attiny3217 tells the tools exactly which device we are targeting. That choice affects instruction availability, predefined symbols, and which register definitions appear through `#include <avr/io.h>`.

No output on success. On error, avr-as prints the filename, line number, and problem.

1.6.2 Step 2: Link

```
avr-gcc -mmcu=attiny3217 -nostartfiles -o blink.elf blink.o
```

The linker is the stage where your program stops being a pile of pieces and becomes a real image with fixed addresses. It decides where `.vectors` and `.text` live in flash and resolves any symbol references between sections.

-nostartfiles suppresses the C runtime startup (crt0.o). Our .vectors section and main label take its place.

1.6.3 Step 3: Convert to HEX

```
avr-objcopy -O ihex blink.elf blink.hex
```

The .elf is the rich build artifact for tools and humans. The .hex is the lean transport format for programming the chip. Same program, different packaging.

1.6.4 Step 4: Inspect (optional but recommended)

```
avr-objdump -d blink.elf
```

Sample output:

```
blink.elf:      file format elf32-avr
```

Disassembly of section .vectors:

```
00000000 <__vectors>:
   0:  0c 94 02 00      jmp     0x4 <__ctors_end>
```

Disassembly of section .text:

```
00000004 <__ctors_end>:
   4:  0f ef           ldi     r16, 0xFF           ; RAMEND low byte
   6:  0d bf           out     0x3d, r16           ; SPL
   8:  0f e3           ldi     r16, 0x3F           ; RAMEND high byte
  a:  0e bf           out     0x3e, r16           ; SPH
  c:  08 e0           ldi     r16, 0x08           ; PIN3 mask
  e:  00 93 01 04     sts     0x0401, r16         ; PORTA_DIRSET
  ...
```

This is one of the healthiest habits you can build early: do not blindly trust source code. Inspect what the toolchain actually produced. avr-objdump shows the addresses, the machine-code bytes, and the final instruction sequence that will land in flash.

On this device/linker setup, avr-objdump may label the first .text address as __ctors_end; that is still the same address as our main label in this minimal program.

1.6.5 Step 5: Flash

ATTiny3217 Curiosity Nano (onboard debugger via USB):

```
pymcuprog write -d attiny3217 -t uart -f blink.hex
```

UPDI adapter (e.g. FT232R or CH340 with SerialUPDI):

```
avrdude -c serialupdi -p attiny3217 -P /dev/ttyUSB0 -b 115200 \
-U flash:w:blink.hex:i
```

Successful flash output:

```
avrdude: AVR device initialized and ready to accept instructions
avrdude: writing flash (40 bytes):
avrdude: 40 bytes of flash written
avrdude: verifying flash memory against blink.hex:
avrdude: 40 bytes of flash verified
avrdude done. Thank you.
```

The LED on PA3 should now blink at approximately 1 Hz.

1.7 A Minimal Makefile

Typing the build steps by hand is useful once because it teaches the pipeline. Typing them forever is just friction. Save this as `ch01_intro/src/Makefile`:

```
MCU      = attiny3217
TARGET   = blink
SRCS     = blink.S

AS       = avr-as
CC       = avr-gcc
OBJCOPY  = avr-objcopy
OBJDUMP  = avr-objdump
PROG     = serialupdi

ASFLAGS  = -mmcu=$(MCU)
LDFLAGS  = -mmcu=$(MCU) -nostartfiles

.PHONY: all flash disasm clean

all: $(TARGET).hex

$(TARGET).o: $(TARGET).S
    $(AS) $(ASFLAGS) -o $@ $<

$(TARGET).elf: $(TARGET).o
    $(CC) $(LDFLAGS) -o $@ $<

$(TARGET).hex: $(TARGET).elf
    $(OBJCOPY) -O ihex $< $@

flash: $(TARGET).hex
    pymcuprog write -d $(MCU) -t uart -f $<

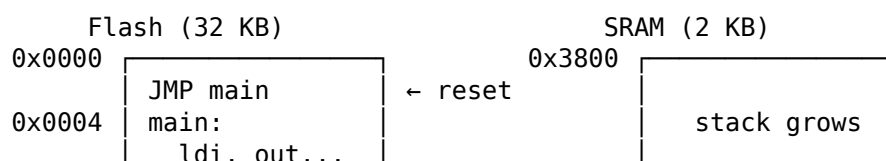
disasm: $(TARGET).elf
    $(OBJDUMP) -d $<

clean:
    rm -f $(TARGET).o $(TARGET).elf $(TARGET).hex
```

Now build with `make`, flash with `make flash`, inspect with `make disasm`.

1.8 What Just Happened — Mental Model

Before moving on, fix this picture in your mind. This is the first real mental model of the whole book:





CPU reads instructions from Flash.

CPU reads/writes variables in SRAM.

Registers (r0–r31) are inside the CPU – fastest possible storage.

The program counter starts at 0x0000 after reset, fetches the JMP at the reset vector, lands in main, runs straight through the setup code, and then spins forever around loop. Nothing mystical is happening. The CPU is just fetching, executing, and updating state one instruction at a time.

1.9 Summary

Concept	Key point
Assembly	One instruction = one CPU operation. No abstraction.
AVR-GAS	GNU Assembler, invoked as <code>avr-as</code> . GAS syntax.
Build pipeline	<code>.S</code> → assemble → <code>.o</code> → link → <code>.elf</code> → convert → <code>.hex</code> → flash
ATtiny3217 PORT	New-style: memory-mapped registers, use STS/LDS. OUTTGL is atomic.
Stack pointer	Must be valid before any subroutine call; on ATtiny3217 reset already loads <code>SP = RAMEND</code> .
Default clock	$\text{OSC20M} \div 6$ after reset: 3.3 MHz or 2.66 MHz, depending on <code>FUSE.0SCCFG</code> .

1.10 Exercises

1. Change the blink rate to 2 Hz (halve the delay loop count). Recalculate the required `r19` value.
2. Use `PORTA_OUTSET` and `PORTA_OUTCLR` instead of `PORTA_OUTTGL`. Make the LED on for a longer period than it is off (asymmetric blink).
3. Run `avr-objdump -d blink.elf` and find:
 - The byte address of the `rjmp loop` instruction.
 - The encoding of `sts 0x0407, r16` — how many bytes does it take?
 - The total size of the `.vectors` section.
4. Add a second LED on PA5. Both LEDs should blink in opposite phases (one on while the other is off).

Next: Chapter 2 — Number Systems: Binary, Hexadecimal, Decimal, and BCD

Chapter 2

Number Systems: Binary, Hexadecimal, Decimal, and BCD

Assembly programming is arithmetic all the way down. Every register holds a number, every address is a number, every instruction is a number stored in flash. Before writing a single instruction you need to be comfortable reading and writing values in all three bases the toolchain uses: decimal (the human default), binary (the hardware's native form), and hexadecimal (the programmer's shorthand for binary). Binary Coded Decimal (BCD) rounds out the chapter as a representation that bridges numeric computation and human-readable output.

This chapter builds each number system from first principles, shows the conversions between them, and then explains where each one appears in AVR assembly source and in real hardware like RTCs and display drivers.

2.1 Decimal: The System You Already Know

Decimal is base 10: ten distinct symbols (0 through 9) and positional notation where each position to the left is worth ten times more than the position to its right.

1	3	7	4			
				ones	= $4 \times 10^0 = 4 \times 1$	= 4
				tens	= $7 \times 10^1 = 7 \times 10$	= 70
				hundreds	= $3 \times 10^2 = 3 \times 100$	= 300
				thousands	= $1 \times 10^3 = 1 \times 1000$	= 1000

Total: 1374

The base-10 system is natural for humans because we have ten fingers. It is not natural for computers. Electronic logic has two stable states (on and off, high and low, charged and uncharged), which maps cleanly to base 2, not base 10. Everything a computer stores or computes is ultimately binary; decimal is a presentation layer applied at the user interface.

2.2 Binary: The CPU's Native Language

Binary is base 2. Only two symbols exist: 0 and 1. Each position is worth twice the position to its right.

1 0 1 1 0 1 1 0

	$2^0 = 1$	$\times 0 = 0$
	$2^1 = 2$	$\times 1 = 2$
	$2^2 = 4$	$\times 1 = 4$
	$2^3 = 8$	$\times 0 = 0$
	$2^4 = 16$	$\times 1 = 16$
	$2^5 = 32$	$\times 1 = 32$
	$2^6 = 64$	$\times 0 = 0$
	$2^7 = 128$	$\times 1 = 128$

Total: $128 + 32 + 16 + 4 + 2 = 182$

The bit pattern 10110110 equals 182 in decimal.

2.2.1 Counting in Binary

Counting up in binary follows the same carry rule as decimal — when a digit reaches its maximum (1 in binary, 9 in decimal) it rolls back to 0 and carries 1 into the next position:

Decimal	Binary
0	0000 0000
1	0000 0001
2	0000 0010
3	0000 0011
4	0000 0100
5	0000 0101
6	0000 0110
7	0000 0111
8	0000 1000
9	0000 1001
10	0000 1010
11	0000 1011
12	0000 1100
...	
127	0111 1111
128	1000 0000
...	
254	1111 1110
255	1111 1111

255 is the largest value in 8 bits; all bits are 1. The next count would be 256, which requires 9 bits: 1 0000 0000. In an 8-bit register, adding 1 to 255 wraps back to 0; an ADD, ADC, ADIW, or SUBI that does this also sets the carry flag. (The dedicated INC instruction wraps 255→0 too, but on AVR it does *not* affect the carry flag — only the Z, N, V, and S flags.)

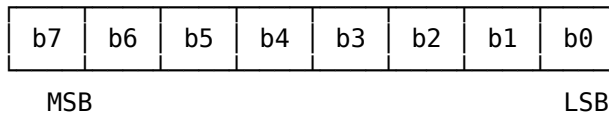
2.2.2 Bit Terminology

Individual binary digits have specific names:

Term	Size	Values	Notes
Bit	1 bit	0 or 1	The atomic unit of binary storage
Nibble	4 bits	0–15	Half a byte; maps to one hex digit
Byte	8 bits	0–255	The basic unit of AVR register storage
Word	2 bytes	0–65535	16-bit; used for addresses, counters
Long	4 bytes	0–4294967295	32-bit; used for timestamps, CRCs

Within a byte, bits are numbered from 0 (the rightmost, least-significant) to 7 (the leftmost, most-significant):

Bit positions in a byte:



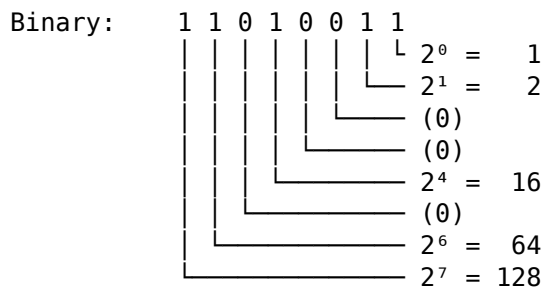
MSB = Most Significant Bit (bit 7 in an 8-bit byte)

LSB = Least Significant Bit (bit 0 in an 8-bit byte)

Bit 7 is the **sign bit** under signed (two's complement) interpretation: if bit 7 is 1, the value is negative. This is explored in depth in Chapter 9 (Signed Arithmetic).

2.2.3 Converting Binary to Decimal

Sum the place values wherever a 1 appears:



Sum: $128 + 64 + 16 + 2 + 1 = 211$

2.2.4 Converting Decimal to Binary (Repeated Division)

Divide repeatedly by 2, collecting remainders from bottom to top:

Convert 211 to binary:

$211 \div 2 = 105$	remainder 1	$\leftarrow$ LSB (bit 0)
$105 \div 2 = 52$	remainder 1	
$52 \div 2 = 26$	remainder 0	
$26 \div 2 = 13$	remainder 0	
$13 \div 2 = 6$	remainder 1	
$6 \div 2 = 3$	remainder 0	
$3 \div 2 = 1$	remainder 1	
$1 \div 2 = 0$	remainder 1	$\leftarrow$ MSB (bit 7)

Read remainders from bottom to top: 1 1 0 1 0 0 1 1 = 0b11010011 = 211 ✓

2.2.5 Powers of Two You Must Know

These values appear constantly in AVR programming:

$2^0 = 1$	$2^8 = 256$
$2^1 = 2$	$2^9 = 512$
$2^2 = 4$	$2^{10} = 1,024$ (1 K)
$2^3 = 8$	$2^{11} = 2,048$ (2 K)
$2^4 = 16$	$2^{12} = 4,096$ (4 K)
$2^5 = 32$	$2^{13} = 8,192$ (8 K)
$2^6 = 64$	$2^{14} = 16,384$ (16 K)
$2^7 = 128$	$2^{15} = 32,768$ (32 K)

The ATtiny3217 has 32 KB = 32,768 bytes of flash (= 2^{15}), 2 KB = 2,048 bytes of SRAM (= 2^{11}), and 256 bytes of EEPROM (= 2^8). These sizes map directly to bit widths: a 15-bit

index spans all of flash, and an 11-bit index spans all of SRAM. (The *absolute* data-space addresses are larger — SRAM is mapped at 0x3800–0x3FFF, so a real SRAM address needs 14 bits — but the size of each region is set by these powers of two.)

2.2.6 Binary Notation in GAS Source

The GNU Assembler accepts a 0b prefix for binary literals:

```
ldi r16, 0b10110110 ; r16 = 182 (same as ldi r16, 182 or ldi r16, 0xB6)
ldi r17, 0b00001111 ; r17 = 15 – lower nibble set, upper nibble clear
ldi r18, 0b11110000 ; r18 = 240 – upper nibble set, lower nibble clear
```

Binary literals are most useful when the bit pattern itself is the point: configuring peripheral registers where individual bits have named meanings, setting up masks, or building values for bitwise operations.

2.3 Hexadecimal: Compact Binary Notation

Hexadecimal (base 16) is not a different way of thinking about numbers — it is a compact shorthand for binary. Each hex digit represents exactly four binary bits (one nibble). Two hex digits represent one byte. This one-to-one correspondence between hex digits and nibbles is why programmers use hex constantly and decimal rarely when dealing with binary hardware.

2.3.1 Hexadecimal Digits

Base 16 needs 16 distinct symbols. The digits 0–9 provide ten, and the letters A–F (or a–f) provide six more:

Hex digit	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

This table is worth memorising. After a few weeks of assembly programming you will read hex digits as their 4-bit patterns automatically.

2.3.2 Converting Hex ↔ Binary (The Fundamental Operation)

Split the byte into two nibbles and convert each nibble independently:

0xB6 → B = 1011, 6 = 0110 → 0b10110110

$0xA0 \rightarrow A = 1010, \quad 0 = 0000 \rightarrow 0b10100000$

$0x3F \rightarrow 3 = 0011, \quad F = 1111 \rightarrow 0b00111111$

The reverse:

$0b11001010 \rightarrow \text{split: } 1100 \mid 1010 \rightarrow C \mid A \rightarrow 0xCA$

$0b00010111 \rightarrow \text{split: } 0001 \mid 0111 \rightarrow 1 \mid 7 \rightarrow 0x17$

$0b11111111 \rightarrow \text{split: } 1111 \mid 1111 \rightarrow F \mid F \rightarrow 0xFF$

This conversion is instantaneous once you know the 16-entry table. No arithmetic required. This is why embedded programmers write register values in hex instead of decimal: $0b10110010$ and $0xB2$ convey the same information, but $0xB2$ is faster to write and the nibble structure is immediately visible.

2.3.3 Converting Hex $\leftrightarrow$ Decimal

For a two-digit hex value $0xHiLo$:

decimal value = $(Hi \times 16) + Lo$

Examples:

$0xB6: (11 \times 16) + 6 = 176 + 6 = 182$

$0x3F: (3 \times 16) + 15 = 48 + 15 = 63$

$0xFF: (15 \times 16) + 15 = 240 + 15 = 255$

$0x80: (8 \times 16) + 0 = 128 + 0 = 128$

$0x10: (1 \times 16) + 0 = 16 + 0 = 16$

$0x01: (0 \times 16) + 1 = 0 + 1 = 1$

For a four-digit hex value $0xB3B2B1B0$:

decimal = $(B3 \times 16^3) + (B2 \times 16^2) + (B1 \times 16) + B0$
 $= (B3 \times 4096) + (B2 \times 256) + (B1 \times 16) + B0$

In practice you rarely convert large hex values to decimal by hand — you use a calculator or let the assembler do it. What matters is the hex $\leftrightarrow$ binary correspondence.

2.3.4 Converting Decimal to Hex (Repeated Division by 16)

Convert 211 to hex:

$211 \div 16 = 13 \text{ remainder } 3 \leftarrow \text{low digit } (0x_3)$

$13 \div 16 = 0 \text{ remainder } 13 \leftarrow \text{high digit } (0xD_)$

Result: $0xD3$

Verify: $(13 \times 16) + 3 = 208 + 3 = 211 \quad \checkmark$

Or convert binary to hex as the two-step shortcut: binary $\rightarrow$ hex is simpler than binary $\rightarrow$ decimal because the nibble grouping aligns perfectly.

2.3.5 Hex Notation in AVR Assembly and C

GAS and avr-gcc both accept $0x$ prefix:

```
ldi r16, 0xFF      ; r16 = 255 (all bits set)
ldi r17, 0x80      ; r17 = 128 (bit 7 set, all others clear)
ldi r18, 0x0F      ; r18 = 15 (lower nibble set)
ldi r19, 0xF0      ; r19 = 240 (upper nibble set)
```


In AVR register and peripheral documentation, values are always given in hex. The SRAM address space starts at 0x3800 on the ATtiny3217; the stack pointer register SPL is at I/O address 0x3D; the PORTB output register is at 0x0424 (PORTB base 0x0420 plus the OUT offset 0x04). These addresses only make visual sense in hex.

2.4 The Complete 8-bit Map

The following table shows decimal, hexadecimal, and binary side by side for every value from 0 to 255. Scan it a few times; the patterns become clear.

Dec	Hex	Binary	Dec	Hex	Binary	Dec	Hex	Binary
0	00	0000 0000	86	56	0101 0110	171	AB	1010 1011
1	01	0000 0001	87	57	0101 0111	172	AC	1010 1100
2	02	0000 0010	88	58	0101 1000	173	AD	1010 1101
3	03	0000 0011	89	59	0101 1001	174	AE	1010 1110
4	04	0000 0100	90	5A	0101 1010	175	AF	1010 1111
5	05	0000 0101	91	5B	0101 1011	176	B0	1011 0000
6	06	0000 0110	92	5C	0101 1100	177	B1	1011 0001
7	07	0000 0111	93	5D	0101 1101	178	B2	1011 0010
8	08	0000 1000	94	5E	0101 1110	179	B3	1011 0011
9	09	0000 1001	95	5F	0101 1111	180	B4	1011 0100
10	0A	0000 1010	96	60	0110 0000	181	B5	1011 0101
11	0B	0000 1011	97	61	0110 0001	182	B6	1011 0110
12	0C	0000 1100	98	62	0110 0010	183	B7	1011 0111
13	0D	0000 1101	99	63	0110 0011	184	B8	1011 1000
14	0E	0000 1110	100	64	0110 0100	185	B9	1011 1001
15	0F	0000 1111	101	65	0110 0101	186	BA	1011 1010
16	10	0001 0000	102	66	0110 0110	187	BB	1011 1011
17	11	0001 0001	103	67	0110 0111	188	BC	1011 1100
18	12	0001 0010	104	68	0110 1000	189	BD	1011 1101
19	13	0001 0011	105	69	0110 1001	190	BE	1011 1110
20	14	0001 0100	106	6A	0110 1010	191	BF	1011 1111
21	15	0001 0101	107	6B	0110 1011	192	C0	1100 0000
22	16	0001 0110	108	6C	0110 1100	193	C1	1100 0001
23	17	0001 0111	109	6D	0110 1101	194	C2	1100 0010
24	18	0001 1000	110	6E	0110 1110	195	C3	1100 0011
25	19	0001 1001	111	6F	0110 1111	196	C4	1100 0100
26	1A	0001 1010	112	70	0111 0000	197	C5	1100 0101
27	1B	0001 1011	113	71	0111 0001	198	C6	1100 0110
28	1C	0001 1100	114	72	0111 0010	199	C7	1100 0111
29	1D	0001 1101	115	73	0111 0011	200	C8	1100 1000
30	1E	0001 1110	116	74	0111 0100	201	C9	1100 1001
31	1F	0001 1111	117	75	0111 0101	202	CA	1100 1010
32	20	0010 0000	118	76	0111 0110	203	CB	1100 1011
33	21	0010 0001	119	77	0111 0111	204	CC	1100 1100
34	22	0010 0010	120	78	0111 1000	205	CD	1100 1101
35	23	0010 0011	121	79	0111 1001	206	CE	1100 1110
36	24	0010 0100	122	7A	0111 1010	207	CF	1100 1111
37	25	0010 0101	123	7B	0111 1011	208	D0	1101 0000
38	26	0010 0110	124	7C	0111 1100	209	D1	1101 0001
39	27	0010 0111	125	7D	0111 1101	210	D2	1101 0010
40	28	0010 1000	126	7E	0111 1110	211	D3	1101 0011
41	29	0010 1001	127	7F	0111 1111	212	D4	1101 0100
42	2A	0010 1010	128	80	1000 0000	213	D5	1101 0101
43	2B	0010 1011	129	81	1000 0001	214	D6	1101 0110

44	2C	0010 1100	130	82	1000 0010	215	D7	1101 0111
45	2D	0010 1101	131	83	1000 0011	216	D8	1101 1000
46	2E	0010 1110	132	84	1000 0100	217	D9	1101 1001
47	2F	0010 1111	133	85	1000 0101	218	DA	1101 1010
48	30	0011 0000	134	86	1000 0110	219	DB	1101 1011
49	31	0011 0001	135	87	1000 0111	220	DC	1101 1100
50	32	0011 0010	136	88	1000 1000	221	DD	1101 1101
51	33	0011 0011	137	89	1000 1001	222	DE	1101 1110
52	34	0011 0100	138	8A	1000 1010	223	DF	1101 1111
53	35	0011 0101	139	8B	1000 1011	224	E0	1110 0000
54	36	0011 0110	140	8C	1000 1100	225	E1	1110 0001
55	37	0011 0111	141	8D	1000 1101	226	E2	1110 0010
56	38	0011 1000	142	8E	1000 1110	227	E3	1110 0011
57	39	0011 1001	143	8F	1000 1111	228	E4	1110 0100
58	3A	0011 1010	144	90	1001 0000	229	E5	1110 0101
59	3B	0011 1011	145	91	1001 0001	230	E6	1110 0110
60	3C	0011 1100	146	92	1001 0010	231	E7	1110 0111
61	3D	0011 1101	147	93	1001 0011	232	E8	1110 1000
62	3E	0011 1110	148	94	1001 0100	233	E9	1110 1001
63	3F	0011 1111	149	95	1001 0101	234	EA	1110 1010
64	40	0100 0000	150	96	1001 0110	235	EB	1110 1011
65	41	0100 0001	151	97	1001 0111	236	EC	1110 1100
66	42	0100 0010	152	98	1001 1000	237	ED	1110 1101
67	43	0100 0011	153	99	1001 1001	238	EE	1110 1110
68	44	0100 0100	154	9A	1001 1010	239	EF	1110 1111
69	45	0100 0101	155	9B	1001 1011	240	F0	1111 0000
70	46	0100 0110	156	9C	1001 1100	241	F1	1111 0001
71	47	0100 0111	157	9D	1001 1101	242	F2	1111 0010
72	48	0100 1000	158	9E	1001 1110	243	F3	1111 0011
73	49	0100 1001	159	9F	1001 1111	244	F4	1111 0100
74	4A	0100 1010	160	A0	1010 0000	245	F5	1111 0101
75	4B	0100 1011	161	A1	1010 0001	246	F6	1111 0110
76	4C	0100 1100	162	A2	1010 0010	247	F7	1111 0111
77	4D	0100 1101	163	A3	1010 0011	248	F8	1111 1000
78	4E	0100 1110	164	A4	1010 0100	249	F9	1111 1001
79	4F	0100 1111	165	A5	1010 0101	250	FA	1111 1010
80	50	0101 0000	166	A6	1010 0110	251	FB	1111 1011
81	51	0101 0001	167	A7	1010 0111	252	FC	1111 1100
82	52	0101 0010	168	A8	1010 1000	253	FD	1111 1101
83	53	0101 0011	169	A9	1010 1001	254	FE	1111 1110
84	54	0101 0100	170	AA	1010 1010	255	FF	1111 1111
85	55	0101 0101						

A few patterns worth noting:

- **0x80 = 128 = 1000 0000**: the boundary between signed-negative and signed-non-negative. Bit 7 is the only bit set.
- **0xFF = 255 = 1111 1111**: all bits set. SER Rd loads this value.
- **0x0F = 15 = 0000 1111**: lower nibble mask.
- **0xF0 = 240 = 1111 0000**: upper nibble mask.
- **0xAA = 170 = 1010 1010** and **0x55 = 85 = 0101 0101**: alternating bit patterns used for hardware stress tests.

2.5 Little-Endian Byte Order

When a value wider than 8 bits lives in AVR SRAM, the bytes are stored in **little-endian** order: the byte at the lowest address holds the least-significant byte (low byte), and each

successive address holds the next more significant byte.

For the 16-bit value 0x1A2B stored at address 0x0200:

Address	Content
0x0200	0x2B (low byte, least significant)
0x0201	0x1A (high byte, most significant)

For the 32-bit value 0xDEADBEEF stored at address 0x0200:

Address	Content
0x0200	0xEF (byte 0 – least significant)
0x0201	0xBE (byte 1)
0x0202	0xAD (byte 2)
0x0203	0xDE (byte 3 – most significant)

Little-endian means “the little end (least significant byte) goes first (at the lowest address).” The GCC AVR compiler, the assembler’s `.word` and `.long` directives, and the LDS/STS load-store instructions all follow this convention. Register pairs in assembly code are written high:low (e.g., `r25:r24`), but in memory the low register lives at the lower address.

2.6 Number Representation in GAS Assembly Source

GAS accepts the same number in any of the three bases. These three lines are identical:

```
ldi r16, 182           ; decimal
ldi r16, 0b10110110    ; binary
ldi r16, 0xB6          ; hex
```

GAS also accepts character literals using single quotes:

```
ldi r16, 'A'           ; r16 = 65 (ASCII code for 'A')
ldi r17, '\n'          ; r17 = 10 (ASCII newline)
ldi r18, '0'           ; r18 = 48 (ASCII digit zero)
```

Character literals are especially useful when building USART output code and working with ASCII display buffers.

2.6.1 Assembler Arithmetic

GAS evaluates constant expressions at assembly time, including mixed-base expressions:

```
ldi r16, (0xFF & ~0x0F) ; r16 = 0xF0 = 240 (upper nibble mask)
ldi r17, (1 << 5)       ; r17 = 0x20 = 32 (bit 5 set)
ldi r18, (0x0F | 0x30)   ; r18 = 0x3F = 63
```

Using expressions like `(1 << 5)` instead of a raw constant makes the intent clear: “bit 5 should be set.”

2.6.2 Data Directives and Byte Order

```
.section .data

byte_val:    .byte  0xAB           ; 1 byte: 0xAB
word_val:    .word  0x1234         ; 2 bytes at consecutive addresses:
                                   ; [addr+0] = 0x34, [addr+1] = 0x12 (little-endian)
long_val:    .long  0xDEADBEEF     ; 4 bytes:
                                   ; [addr+0]=0xEF, [+1]=0xBE, [+2]=0xAD, [+3]=0xDE
```

The `.word` and `.long` directives store values in little-endian order, matching the LDS/STS byte order the CPU expects.

2.7 BCD: Binary Coded Decimal

BCD is a representation system that bridges the gap between binary hardware and decimal human output. Instead of storing a number in pure binary (using all possible bit patterns), BCD reserves exactly 4 bits for each decimal digit. Each nibble encodes one decimal digit from 0 to 9. The values 0xA through 0xF (10–15 in a nibble) are unused — they are illegal BCD digits.

BCD exists because some applications need decimal arithmetic directly: clocks and calendars (where hours are 00–23, minutes 00–59), postal codes, currency amounts, and any case where numbers must be displayed without a binary-to-decimal conversion step.

2.7.1 Packed BCD

Packed BCD stores two decimal digits in one byte: the high nibble holds the tens digit, the low nibble holds the units digit.

Decimal	Packed BCD	Binary
0	0x00	0000 0000
1	0x01	0000 0001
9	0x09	0000 1001
10	0x10	0001 0000
15	0x15	0001 0101
42	0x42	0100 0010
59	0x59	0101 1001
99	0x99	1001 1001

The range of valid packed BCD values in one byte is 0x00–0x99, representing decimal 0–99. This uses 100 of the 256 possible bit patterns — 156 patterns are wasted. The advantage is that converting to a decimal display requires only extracting nibbles, not dividing by 10.

Packed BCD is the format used by virtually all hardware real-time clocks, including the DS1307 and PCF8523 that appear in many AVR designs. Hours, minutes, and seconds are each stored as packed BCD bytes.

Reading a DS1307:

Register value: 0x47

High nibble: 4 → tens digit = 4

Low nibble: 7 → units digit = 7

Decoded: 47 minutes

2.7.2 Unpacked BCD

Unpacked BCD stores one decimal digit per byte in the low nibble (high nibble is zero):

Decimal digit	Unpacked BCD
0	0x00
1	0x01
5	0x05
9	0x09

Unpacked BCD is less memory-efficient but simpler to manipulate. It is also closely related to **ASCII digit encoding**: the ASCII codes for '0' through '9' are 0x30 through 0x39. ASCII digit = unpacked BCD digit + 0x30. This makes ASCII digit output trivial:

```
; Convert a single decimal digit (0-9) in r16 to its ASCII character
ldi r17, '0'           ; r17 = 0x30
add r16, r17           ; r16 = 0x30 + digit (ASCII '0'-'9')
```

And the reverse:

```
; Convert ASCII '0'-'9' in r16 to the digit value 0-9
subi r16, '0'          ; r16 -= 0x30 (subtract ASCII '0')
; r16 is now 0-9
```

2.7.3 BCD Addition on AVR

Adding two packed BCD values with a standard ADD instruction produces a binary sum, which may not be a valid BCD value:

```
  0x29 (= 29 BCD)
+ 0x38 (= 38 BCD)
-----
  0x61 (= 97 decimal in binary)
```

But $29 + 38 = 67$, not 97. The binary add gave $0x61 = 0110\ 0001$, but the correct BCD result is $0x67 = 0110\ 0111$.

The binary add is wrong whenever a nibble sum exceeds 9, because a decimal digit cannot exceed 9. When a nibble overflows past 9, it enters the illegal BCD range 0xA–0xF, and the tens carry does not propagate to the next nibble.

The fix is to add 6 to any nibble whose binary sum exceeded 9 or generated a carry. This is called **BCD correction** or **decimal adjustment**.

Many processor families (x86, Z80, 6502) have a hardware DAA (Decimal Adjust Accumulation) instruction that performs this correction automatically. **The AVR does not have a DAA instruction.** BCD correction must be done in software.

The correction rules, applied after an ADD or ADC:

For the LOW nibble (units digit):

```
If (low nibble of result > 9) OR (H flag = 1):
    add 0x06 to the result
```

For the HIGH nibble (tens digit):

```
If (high nibble of result > 9) OR (C flag = 1):
    add 0x60 to the result
```

The H flag (half-carry, carry out of bit 3 into bit 4) records whether the low nibble overflowed during the binary add — exactly the condition that requires a low-nibble correction.

2.7.3.1 BCD Addition Subroutine

```
/*
 * bcd_add — add two packed BCD bytes
 *   Input:  r16 = BCD addend A (0x00–0x99)
 *           r17 = BCD addend B (0x00–0x99)
 *   Output: r16 = BCD sum (0x00–0x99)
 *           C=1 if sum >= 100 (decimal carry out of tens digit)
 *   Clobbers: r18, r19
 *
 * The binary add's carry and half-carry drive the corrections, but the
```

```

* low-nibble fix would clobber them – so we snapshot the carry into r19
* straight away, read H (still valid) for the low-nibble test, and use real
* ADDs for the corrections so their carry-out stays meaningful.
*/
bcd_add:
    add r16, r17          ; binary add: result may not be valid BCD
    clr r19               ; r19 = pending tens carry-out (0 or 1)
    brcc bcd_add_lowchk   ; did the binary add overflow the byte?
    inc r19               ; yes → carry out of the tens digit
bcd_add_lowchk:
    ; — Low nibble correction: +6 if H=1 (half-carry) or low nibble > 9 —
    brhs bcd_add_lowfix    ; H=1: low nibble carried out of bit 3
    mov r18, r16
    andi r18, 0x0F         ; isolate low nibble
    cpi r18, 0x0A
    brlo bcd_add_lowok     ; low nibble <= 9: no correction needed
bcd_add_lowfix:
    ldi r18, 0x06
    add r16, r18           ; r16 += 6 via real ADD → C is a true byte carry
    brcc bcd_add_lowok
    ldi r19, 1             ; the +6 overflowed the byte → tens carry-out
bcd_add_lowok:
    ; — High nibble correction: +0x60 if a carry is pending or high > 9 —
    tst r19
    brne bcd_add_highfix   ; carry pending → tens digit must wrap
    mov r18, r16
    andi r18, 0xF0         ; isolate high nibble
    cpi r18, 0xA0         ; high nibble > 9? (0xA0 = 10 in high-nibble position)
    brlo bcd_add_done
bcd_add_highfix:
    ldi r18, 0x60
    add r16, r18           ; r16 += 0x60
    ldi r19, 1             ; sum >= 100 → decimal carry out
bcd_add_done:
    lsr r19               ; move tens carry-out (bit 0) into the C flag
    ret

```

Worked trace: 0x29 + 0x38

ADD r16(0x29), r17(0x38):

0x29 + 0x38 = 0x61

Low nibble: 9 + 8 = 17 = 0x11 → bit 3 carry → H=1

High nibble: 2 + 3 + 1(H) = 6, no carry → C=0

Low nibble correction: H=1 → add 6

0x61 + 0x06 = 0x67

Low nibble: 1 + 6 = 7 → 0x67 (valid BCD: '7')

No carry generated from low correction

High nibble correction: C=0; high nibble of 0x67 = 0x60 = 6 → 6 <= 9: no correction

Result: 0x67 = BCD 67 = decimal 67 ✓ (29 + 38 = 67)

Worked trace: 0x75 + 0x48 (= 75 + 48 = 123, BCD overflow)

ADD r16(0x75), r17(0x48):

0x75 + 0x48 = 0xBD

Low nibble: 5 + 8 = 13 = 0xD → no carry out of bit 3, so H=0 (13 < 16)

High nibble: 7 + 4 = 11 = 0xB → C=0 (no overflow out of the byte)

Low correction: $H=0$, but low nibble $0xD > 9 \rightarrow$ add 6

$0xBD + 0x06 = 0xC3$

(no carry out of the byte from the +6; the tens carry stays 0)

High correction: no carry pending, but high nibble $0xC > 9 \rightarrow$ add $0x60$

$0xC3 + 0x60 = 0x123 \rightarrow$ byte portion = $0x23$, $C=1$

Result: $0x23$ with $C=1$

Decoded: BCD 23 with decimal carry $\rightarrow 123 \quad \checkmark \quad (75 + 48 = 123)$

2.7.4 BCD Subtraction on AVR

BCD subtraction requires the same type of correction in the opposite direction. After a binary SUB, correct any nibble whose result required a borrow by subtracting 6:

```
/*
 * bcd_sub - subtract packed BCD (r16 - r17)
 *   Input:  r16 = minuend (0x00-0x99)
 *           r17 = subtrahend (0x00-0x99)
 *   Output: r16 = BCD difference (0x00-0x99)
 *           C=1 if borrow (r16 < r17); the result is then the ten's-complement
 *           BCD form, i.e.  $100 - (r17 - r16)$ 
 *   Clobbers: r18, r19
 *
 * As with bcd_add, the binary subtract's borrow (C) is needed for the
 * tens-digit correction but would be clobbered by the low-nibble fix, so we
 * snapshot it into r19 first and use a real SUB for the correction.
 */
bcd_sub:
    sub r16, r17          ; binary subtract; H = half-borrow, C = byte borrow
    clr r19               ; r19 = pending borrow-out (0 or 1)
    brcc bcd_sub_lowchk   ; did the subtract borrow out of the byte?
    inc r19               ; yes  $\rightarrow$  result is negative
bcd_sub_lowchk:
    ; Low nibble correction: if H=1 (half-borrow), subtract 6
    brhc bcd_sub_lowok    ; H=0: low nibble did not borrow
    ldi r18, 0x06
    sub r16, r18          ; r16 -= 6 via real SUB  $\rightarrow$  C is a true byte borrow
    brcc bcd_sub_lowok
    ldi r19, 1            ; the -6 borrowed past the byte
bcd_sub_lowok:
    ; High nibble correction: if a borrow is pending, subtract 0x60
    tst r19
    breq bcd_sub_done
    ldi r18, 0x60
    sub r16, r18          ; r16 -= 0x60 (forms the ten's-complement BCD)
bcd_sub_done:
    lsr r19               ; move borrow-out (bit 0) into the C flag
    ret
```

Worked trace: $0x73 - 0x28 (= 73 - 28 = 45)$

SUB r16($0x73$), r17($0x28$):

$0x73 - 0x28 = 0x4B$

Low nibble: $3 - 8 = -5 \rightarrow$ borrow from high $\rightarrow H=1$, low result = $3 - 8 + 16 = 11 = 0xB$

High nibble: $7 - 2 - 1(\text{borrow}) = 4 \rightarrow C=0$

Low correction: $H=1 \rightarrow$ subtract 6

$0x4B - 0x06 = 0x45$

High correction: no borrow pending → no correction

Result: $0x45 = \text{BCD } 45$ ✓ ($73 - 28 = 45$)

2.7.5 Converting Binary to Packed BCD

A binary value in the range 0-255 encodes as at most three BCD digits. The standard algorithm is **double-dabble** (also called shift-add-3): it processes the binary input one bit at a time and adjusts the BCD accumulator after each shift.

The rule: before shifting each binary bit into the BCD accumulator, add 3 to any BCD nibble that is ≥ 5 . (This pre-compensates for the value-6 adjustment that would be needed after the shift overflows a nibble past 9.)

Double-dabble for 8-bit input → up to 3 BCD digits (packed into 12 bits):

Registers:

r18 = hundreds digit (bits 11:8 of BCD accumulator – only low nibble used)

r17 = tens and units digits (high nibble = tens, low nibble = units)

r16 = binary input (shifted out MSB first)

```

/*
 * bin8_to_bcd – convert 8-bit binary to 3 packed BCD digits
 * Input:  r16 = binary value (0-255)
 * Output: r18 = hundreds digit (0-2, in low nibble)
 *         r17 = packed BCD: high nibble = tens, low nibble = units
 * Clobbers: r19, r20
 */
bin8_to_bcd:
    clr  r17                ; clear BCD accumulator (tens:units)
    clr  r18                ; clear hundreds digit

    ldi  r20, 8             ; process 8 bits

bin_bcd_loop:
    ; — add-3 if any nibble ≥ 5 (pre-shift correction) —————
    ; Check units nibble
    mov  r19, r17
    andi r19, 0x0F
    cpi  r19, 5
    brlo bin_bcd_skip_units
    subi r17, -3            ; units nibble += 3
bin_bcd_skip_units:
    ; Check tens nibble
    mov  r19, r17
    andi r19, 0xF0
    cpi  r19, 0x50          ; tens ≥ 5 → high nibble ≥ 5
    brlo bin_bcd_skip_tens
    subi r17, -0x30         ; tens nibble += 3
bin_bcd_skip_tens:
    ; Check hundreds nibble (in r18, low nibble)
    mov  r19, r18
    andi r19, 0x0F
    cpi  r19, 5
    brlo bin_bcd_skip_hunds

```



```

    subi r18, -3          ; hundreds nibble += 3
bin_bcd_skip_hunds:

    ; — shift MSB of r16 into LSB of BCD accumulator —————
    lsl  r16              ; MSB of r16 → C
    rol  r17              ; C → LSB of r17 (units nibble ← MSB of binary)
    rol  r18              ; carry from r17 → r18

    dec  r20
    brne bin_bcd_loop

    ret

```

Trace: binary 137 (0x89) → should give hundreds=1, tens=3, units=7 → 0x37 in r17, 0x01 in r18

Each iteration does two things, in order:

1. **Add-3 correction** — for any BCD nibble (units, tens, hundreds) that is currently ≥ 5 , add 3 to that nibble.
2. **Shift left by one** — the MSB of r16 shifts out into C; r17 and then r18 shift left, with C feeding into the LSB of r17.

Starting state: r16 = 0x89 = 1000 1001, r17 = 0x00, r18 = 0x00.

Iter	r16 before shift	Nibbles $\geq 5 \rightarrow$ add-3	r17 after add-3	C from MSB	r17 after shift	r18 after shift
1	1000 1001	none (all 0)	0x00	1	0x01	0x00
2	0001 0010	none (units=1, tens=0)	0x01	0	0x02	0x00
3	0010 0100	none (units=2)	0x02	0	0x04	0x00
4	0100 1000	none (units=4)	0x04	0	0x08	0x00
5	1001 0000	units=8 $\rightarrow$ +3	0x0B	1	0x17	0x00
6	0010 0000	units=7 $\rightarrow$ +3	0x1A	0	0x34	0x00
7	0100 0000	none (units=4, tens=3)	0x34	0	0x68	0x00
8	1000 0000	units=8 $\rightarrow$ +3, tens=6 $\rightarrow$ +0x30	0x9B	1	0x37	0x01

Two iterations are worth checking by hand:

- **Iteration 5.** Pre-shift r17 = 0x08. Units nibble = 8 $\geq$ 5, so add 3: 0x08 + 0x03 = 0x0B. Now shift: the MSB of r16 = 1001 0000 is 1, so C = 1; ROL r17 gives (0x0B << 1) | 1 = 0001 0111 = 0x17.
- **Iteration 8.** Pre-shift r17 = 0x68. Units nibble = 8 $\geq$ 5, add 3 $\rightarrow$ 0x6B. Tens nibble = 6 $\geq$ 5, add 0x30 $\rightarrow$ 0x9B. Shift: MSB of r16 = 1000 0000 is 1, so C = 1; ROL r17 on 0x9B = 1001 1011 gives 0011 0111 = 0x37 with carry-out 1, which ROL r18 then catches as 0x01.

Final: r18 = 0x01 (hundreds = 1), r17 = 0x37 (tens = 3, units = 7) $\rightarrow$ 137 ✓

2.7.6 Converting Packed BCD to Binary

The reverse: extract the digits and compute (hundreds $\times$ 100) + (tens $\times$ 10) + units. For 8-bit values (0–99), only tens and units apply:

```

/*
 * bcd2_to_bin – convert 2-digit packed BCD to binary (0–99)
 * Input:  r16 = packed BCD (0x00–0x99)
 * Output: r16 = binary value (0–99)
 * Clobbers: r17, r18
 */
bcd2_to_bin:
    mov  r17, r16
    andi r17, 0xF0          ; isolate high nibble (tens)
    swap r17                ; move to low nibble: r17 = tens digit (0–9)
    mov  r18, r17           ; r18 = tens
    lsl  r17                ; tens × 2
    lsl  r17                ; tens × 4
    add  r17, r18           ; tens × 5
    lsl  r17                ; tens × 10
    andi r16, 0x0F         ; isolate units nibble
    add  r16, r17           ; binary = (tens × 10) + units
    ret

```

The multiply-by-10 uses the shift-and-add trick: $N \times 10 = (N \ll 3) + (N \ll 1)$. In the code above: $(N \times 4 + N) \times 2 = N \times 10$.

Trace: 0x47 (BCD 47) → should give 47

```

r17 = 0x47 & 0xF0 = 0x40 → swap → 0x04 (tens = 4)
r18 = 4
r17: 4 × 2 = 8, 8 × 2 = 16... wait, let's trace the shifts:
  r17 = 0x04
  lsl r17: 0x08 (= 4 × 2 = 8)
  lsl r17: 0x10 (= 8 × 2 = 16 = 4 × 4)
  add r17, r18: 0x10 + 0x04 = 0x14 (= 4 × 5 = 20)
  lsl r17: 0x28 (= 20 × 2 = 40 = 4 × 10)

```

```

r16 = 0x47 & 0x0F = 0x07 (units = 7)
add r16, r17: 0x07 + 0x28 = 0x2F = 47 ✓

```

For three-digit values (0–255), incorporate hundreds:

```

/* bcd3_to_bin – 3-digit packed BCD to binary
 * Input:  r18 = hundreds digit (0–2, low nibble)
 *         r17 = packed tens:units (0x00–0x99)
 * Output: r16 = binary (0–255)
 * Clobbers: r19, r20
 */
bcd3_to_bin:
    ; Convert r17 (tens:units) to binary subtotal in r16
    mov  r20, r17
    andi r20, 0xF0
    swap r20                ; tens digit
    mov  r19, r20
    lsl  r20
    lsl  r20
    add  r20, r19           ; tens × 5
    lsl  r20                ; tens × 10

    andi r17, 0x0F         ; units digit
    add  r17, r20           ; (tens × 10) + units

    ; Add hundreds × 100
    andi r18, 0x0F         ; hundreds digit (0, 1, or 2)

```

```

mov  r16, r18
ldi  r19, 100
; multiply r16 × 100 (result fits in 8 bits since hundreds ≤ 2 → max 200)
; Use repeated add: r16 × 100 = r16 × 64 + r16 × 32 + r16 × 4
mov  r20, r16
lsl  r20           ; × 2
lsl  r20           ; × 4
mov  r16, r20
lsl  r20           ; × 8
lsl  r20           ; × 16
lsl  r20           ; × 32
add  r16, r20      ; × 36
lsl  r20           ; × 64
add  r16, r20      ; × 100

add  r16, r17      ; add tens+units subtotal
ret

```

2.7.7 BCD and the Half-Carry Flag

The AVR's H (half-carry) flag is specifically designed to support BCD arithmetic. It records whether a carry occurred from bit 3 to bit 4 during an ADD, ADC, or SUB operation — in other words, whether the low nibble overflowed. Since the low nibble holds the units digit in packed BCD, H=1 is precisely the signal that the units digit overflowed and needs correction.

ADD r16, r17 where r16 = 0x08 (BCD 8), r17 = 0x08 (BCD 8):

Binary: 0x08 + 0x08 = 0x10

the addition column by column:

```

  0000 1000
+ 0000 1000
-----
  0001 0000 = 0x10

```

Bit 3: 1 + 1 = 10 → a carry leaves bit 3 and enters bit 4,
so the half-carry flag is set → H=1

H=1 signals: the units nibble overflowed (8 + 8 = 16). Note the low nibble of the result is 0 — so the "low nibble > 9" test alone would miss this; the H flag is what catches it.

→ Add 6 to correct: 0x10 + 0x06 = 0x16 = BCD 16 ✓ (8 + 8 = 16)

This is why H exists in the AVR status register even though most arithmetic ignores it. Chapters that use only binary arithmetic can safely ignore H. BCD arithmetic cannot.

2.8 Choosing the Right Representation

Representation	When to use
Decimal literal	Source code: human-facing constants, loop counts, sensor limits. Use when the numeric quantity is what matters: <code>ldi r16, 100</code>
Binary literal	Source code: bit patterns for peripheral config, masks. Use when individual bits matter: <code>ldi r16, 0b00101101</code>

Hex literal	Source code: addresses, register values, large constants. Use when the nibble structure matters: <code>ldi r16, 0xFF</code>
Pure binary	CPU registers, SRAM, flash. All values are ultimately binary.
Packed BCD	RTC registers (hours, minutes, seconds), display output, numeric entry keyboards, any decimal-digit computation.
Unpacked BCD	ASCII string construction, digit-by-digit output over USART. Digit = ASCII - 0x30; ASCII = digit + 0x30.

2.9 Summary

Decimal (base 10):

Digits 0–9. Place values: 1, 10, 100, 1000, ...
Used in source code for human-legible quantities.

Binary (base 2):

Digits 0 and 1. Place values: 1, 2, 4, 8, 16, 32, 64, 128, ...
Native CPU representation. Prefix: 0b.
Bit 0 = LSB. Bit 7 = MSB / sign bit for 8-bit signed values.
8-bit range: 0–255 unsigned; -128 to +127 signed (two's complement).

Hexadecimal (base 16):

Digits 0–9 and A–F. One hex digit = one nibble = 4 bits.
Two hex digits = one byte. Prefix: 0x.
0b1010 1111 ↔ 0xAF (split at nibble boundary, look up each nibble).

Byte order (AVR SRAM):

Little-endian: lowest address holds the least-significant byte.
Multi-byte variables: low byte at var, high byte at var+1.

BCD – Binary Coded Decimal:

Packed BCD: two decimal digits per byte. High nibble = tens, low = units.
Valid range per byte: 0x00–0x99 (decimal 0–99).
Used in RTCs (DS1307, PCF8523), 7-segment drivers, display output.

Unpacked BCD: one digit per byte (low nibble only).

ASCII digit = unpacked BCD digit + 0x30.

AVR has no DAA instruction. BCD correction is manual:

After ADD/ADC: if low nibble > 9 or H=1: result += 6 (units correction)
 if high nibble > 9 or C=1: result += 0x60 (tens correction)
After SUB: if H=1: result -= 6
 if C=1: result -= 0x60

H flag: carry out of bit 3; signals units-digit overflow for BCD code.

Binary ↔ BCD:

Binary → packed BCD: double-dabble (shift-add-3) algorithm.
Packed BCD → binary: extract digits, compute (tens × 10) + units.
For display: use unpacked BCD → add 0x30 → ASCII character per digit.

2.10 Exercises

- Convert the following decimal values to 8-bit binary and hexadecimal without using a calculator:
 - 43, 128, 200, 255, 0
 Check your work using the full 8-bit map in this chapter.
- Convert the following hex values to binary and decimal:
 - 0x1F, 0xC8, 0x7F, 0xA5, 0x80
 For each, identify whether the value would be positive or negative if interpreted as a signed 8-bit two's complement number.
- A hardware real-time clock returns the following packed BCD register values:
 - Hours register: 0x09
 - Minutes register: 0x47
 - Seconds register: 0x31
 What time does this represent? Write three lines of AVR assembly (using ANDI and SWAP) that extract the tens digit and units digit of the minutes value into separate registers.
- The PORTB output data register is at SRAM address 0x0424 (PORTB base 0x0420 plus the OUT register offset 0x04). Write this address in binary. How many address bits are needed to reach 0x0424? (Hint: what is the smallest power of two greater than 0x0424?)
- Add the packed BCD values 0x57 and 0x46 using the `bcd_add` subroutine. Trace every step: binary add result, H and C flags, each correction applied, and the final result. Verify the result makes sense ($57 + 46 = ?$).
- Subtract the packed BCD values 0x83 – 0x57 using the `bcd_sub` routine. Trace the steps. Verify ($83 - 57 = 26$).
- A sensor returns a binary value of 197. Convert it to packed BCD using the double-dabble algorithm by hand (trace each of the 8 iterations of the loop, showing the shift and the nibble check/correction at each step).
- In a USART output routine, you want to print a two-digit decimal number in r16 (range 0–99) as two ASCII characters. Write the assembly sequence that:
 - Extracts the tens digit (hint: divide by 10, or use packed BCD from the `bin8_to_bcd` routine and extract the high nibble)
 - Converts each digit to its ASCII code
 - Leaves the tens ASCII character in r17 and units ASCII in r16
- A 16-bit address 0x1A3F is stored in SRAM at addresses 0x0200 and 0x0201. Which byte (which value) is at 0x0200? Which is at 0x0201? If you later load this address with LD r24, Y+ followed by LD r25, Y (Y = 0x0200), what values end up in r25:r24?
- Explain why the AVR H (half-carry) flag is not needed for pure unsigned or signed binary arithmetic, but is required for BCD arithmetic. At what point in the ADD instruction does the half-carry get generated, and which nibble does it serve?

Next: Chapter 3 — AVR Architecture Overview

Chapter 3

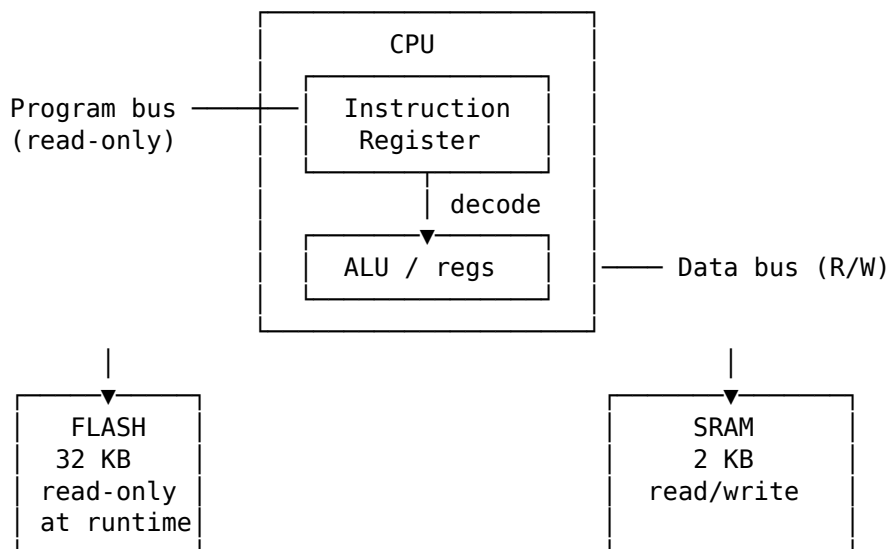
AVR Architecture Overview

Before writing a single instruction, you need a mental model of what the AVR CPU is actually doing: where code lives, where data lives, what the registers are for, and what hidden state changes after each instruction. Without that model, assembly feels like memorising magic spells. With it, most instructions become straightforward and predictable.

The target device in this book, the ATtiny3217, is a useful concrete example: it is an 8-bit AVR running at up to 20 MHz, with 32 KB of flash, 2 KB of SRAM, 256 bytes of EEPROM, a hardware multiplier, and a single-pin UPDI programming and debugging interface. Those numbers matter because assembly programming is always resource-aware. A “small” stack frame or lookup table is only small relative to the actual memory on the chip.

3.1 Harvard Architecture

The AVR is a **Harvard** architecture. This means it has **two separate memory spaces** — one for instructions (flash) and one for data (SRAM) — connected to the CPU by separate buses.



Why does this matter to the programmer?

Because on AVR, “memory” is not one big uniform place. Where a value lives determines how you access it.

When your program is running, the CPU fetches instructions from **flash** automatically. You do not manually load the next instruction yourself; the program counter and instruction fetch hardware handle that in the background. That is why your code can live safely in non-volatile flash while your working data changes constantly in SRAM.

Your **variables**, your **stack**, and most of the peripheral register space you work with as data live in **SRAM/data space**. That is the memory area you read and write with ordinary load/store instructions such as LD, LDS, ST, and STS. So if you push a register, call a sub-routine, store a loop counter, or write to a peripheral register, you are operating in the data space, not in program flash.

Flash is different. It holds your program, and on many AVR parts it also holds constant tables or strings that you do not want to waste SRAM on. But flash is not ordinary read/write RAM. You cannot treat it like a normal destination for ST and expect the program image to change. On the ATtiny3217, writes to flash and EEPROM go through the Nonvolatile Memory Controller (NVMCTRL), using protected command sequences covered later in the book. Older classic AVR devices expose flash self-programming through SPM; do not assume that model applies unchanged to tinyAVR 1-series devices.

This also affects **read-only data** such as lookup tables. On classic AVR, reading data back from flash requires LPM, because flash is on the program side of the machine. On AVRxt parts such as the ATtiny3217, flash is also mapped into the data address space, so some reads can be done through that mapping with ordinary data-memory instructions. Chapter 7 shows both styles and when each applies.

The practical rule is simple:

- If the value is temporary, changing, or part of the stack, think **SRAM/data space**.
- If the value is your code or a fixed table that should survive power loss, think **flash/program space**.

This separation is why you cannot simply think “an address is an address.” On AVR, you must use the instruction that matches the address space you are trying to access.

3.2 ATtiny3217 Memory Map

The ATtiny3217 uses the **AVRxt** core (the GNU toolchain names this architecture `avrxtmega3`). It has an important difference from classic AVR (ATmega328P): **flash is also mapped into the data address space** at offset 0x8000. This means you can read flash with ordinary LD instructions when you use the mapped address — no need for LPM for that access path.

That feature is convenient, but do not let it blur the architectural picture. Flash is still program memory. The chip is simply giving you a second way to read it. Thinking in terms of “program space” versus “data space” is still the right model.

Data Address Space (what LD/ST/LDS/STS see):

0x0000	I/O registers (IN/OUT range)	0x0000–0x003F (64 bytes)
0x003F		
0x0040	Extended I/O / Peripherals (LDS/STS only, not IN/OUT)	0x0040–0x0FFF
0x0FFF		
0x1000	NVM I/O registers and data	0x1000–0x13FF
0x13FF		
0x1400	EEPROM – 256 bytes	0x1400–0x14FF
0x14FF		
0x1500	(reserved)	
0x37FF		

0x3800	SRAM – 2 KB	0x3800–0x3FFF
0x3FFF		
0x4000	(reserved)	
0x7FFF		
0x8000	Flash – 32 KB (read-only) mapped into data space	0x8000–0xFFFF
0xFFFF		

Program Address Space (what the PC addresses, separate bus):

0x0000	Flash – 16K words (32 KB) Byte addr = word addr × 2	
0x3FFF		

(word addresses; byte addresses go to 0x7FFF)

Key numbers to memorise:

Symbol	Value	Meaning
E2START / EEPROM_START	0x1400	First EEPROM byte in data space
E2END / EEPROM_END	0x14FF	Last EEPROM byte
FLASHEND	0x7FFF	Last byte address in flash
RAMSTART	0x3800	First SRAM byte
RAMEND	0x3FFF	Last SRAM byte (init SP here)
MAPPED_PROGMEM_START	0x8000	Flash base in data address space

Key point: When you see a linker script or header file refer to `MAPPED_PROGMEM_START`, it means 0x8000 — where flash appears in the data bus. On classic AVR, flash is *not* in the data bus at all.

If you are new to memory maps, read the diagram like this:

- The **low addresses** are the control area: CPU and peripheral registers.
- The **NVM/EEPROM region** is non-volatile data and device metadata, not general scratch RAM.
- The **middle SRAM block** is your working memory.
- The **high mapped-flash block** is read-only program memory made visible from the data side on AVRxt devices.

That one picture explains why some operations use IN/OUT, some use LDS/STS, and some use flash-specific mechanisms.

There are three common ways an address can appear in real code:

What you mean	Address base	Typical instruction
Program flash as code	0x0000 word address	PC fetch, RJMP, CALL
Program flash as bytes via program bus	0x0000 byte address	LPM with Z
Program flash mapped as data	0x8000 byte address	LD, LDS

This is the source of many off-by-two and off-by-0x8000 mistakes. If a label is used as a branch target, the CPU treats it as a program address. If the same label is used as table data, you must be clear about whether the code is using LPM addressing or mapped-data addressing.

3.3 The Register File

The AVR CPU has **32 general-purpose 8-bit registers**, named `r0` through `r31`. They are directly connected to the ALU. In one CPU clock, the core can fetch register operands, execute an ALU operation, and write the result back to the register file. Compare that to SRAM: a load (LDS) takes 3 cycles; a store (STS) takes 2.

This is one of the main reasons assembly on AVR can be so efficient. If you can keep a value in a register, you avoid repeated trips to SRAM. A large part of “good AVR assembly” is really just good register management.

The ATtiny3217 has a rich AVR instruction set that includes a hardware multiplier. Multiplication is still slower than ordinary register ALU work, but it is a fixed two-cycle hardware operation rather than a long software shift-and-add routine. Chapter 13 uses that distinction when comparing arithmetic strategies.

Register file layout:

r0		General purpose. Some instructions can only use r0
r1		(e.g. MUL stores result in r1:r0).
r2		
r3		r0 is often used as a temporary by the compiler.
...		By convention, keep r1 = 0x00 at all times when
r15		calling C code (GCC ABI requirement).
r16		← LDI can only load immediate into r16–r31.
r17		Most two-operand arithmetic works on any register,
...		but immediate operands restrict you to r16–r31.
r23		
r24		SBIW/ADIW work on r25:r24, r27:r26, r29:r28, r31:r30.
r25		
r26	XL	} X pointer register (r27:r26) – indirect addressing
r27	XH	
r28	YL	} Y pointer register (r29:r28) – indirect + stack frame
r29	YH	
r30	ZL	} Z pointer register (r31:r30) – indirect + flash read
r31	ZH	

3.3.1 Register Pairs (16-bit Pointers)

The top six registers form three **pointer pairs** used for indirect memory access:

Pair	Registers	Name	Special use
X	r27:r26	X-pointer	Indirect load/store
Y	r29:r28	Y-pointer	Indirect load/store, stack frame pointer
Z	r31:r30	Z-pointer	Indirect load/store, LPM, indirect jumps/calls

When you see `LD r0, X` in assembly, it means “load the byte at the address stored in the X register pair into r0.” We cover this in depth in Chapter 7.

In practice, think of X, Y, and Z as your built-in address registers. The ordinary registers hold values; the pointer registers hold locations.

3.3.2 LDI Restriction

LDI — load immediate — **only works on r16–r31**. This is one of the most common beginner mistakes.

```
ldi r5, 42      ; ERROR: illegal operand – r5 not allowed with LDI
ldi r16, 42     ; OK
```

If you need a constant in r0–r15, load it into r16–r31 first, then MOV it:

```
ldi r16, 42
mov r5, r16     ; now r5 = 42
```

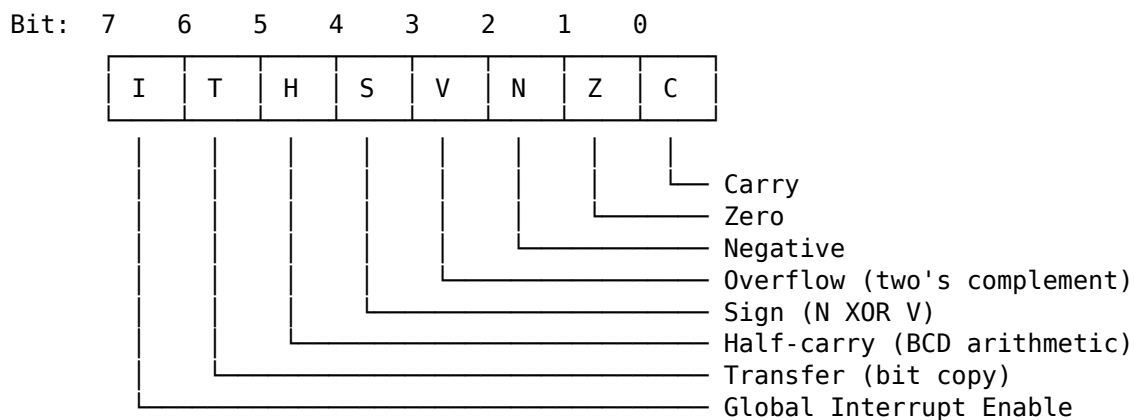
That may feel arbitrary at first, but it is simply part of the AVR instruction encoding. Some instructions have fewer bits available to describe operands, so they only work on part of the register file. You will see this kind of limit often in small instruction-set architectures.

3.4 The Status Register (SREG)

SREG is an 8-bit register at I/O address **0x3F** (accessible with IN/OUT). Every arithmetic and logic instruction updates some of its bits. Branch instructions read these bits to decide whether to jump.

You can think of SREG as the CPU’s short-term memory of “what just happened.” Did the last result become zero? Did it overflow? Was there a carry? Branches do not inspect your registers directly; they inspect these flag bits.

SREG bit layout:



3.4.1 Flag by Flag

C — Carry Set when an addition produces a result > 255 (unsigned overflow) or a subtraction borrows. Also used by shift instructions (the bit shifted out). Used for multi-byte arithmetic (ADC, SBC).

```
ldi r16, 0xFF
ldi r17, 0x01
add r16, r17    ; result = 0x00, C=1 (carry out), Z=1
```

Z — Zero Set when the result of an operation is exactly zero. The most-used flag. BRNE (branch if not equal) branches when Z=0; BREQ branches when Z=1.

```
ldi r16, 1
dec r16        ; r16 = 0, Z=1
```

```
brne somewhere ; NOT taken – Z is set
breq somewhere ; taken – Z is set
```

N — Negative Set when bit 7 of the result is 1 (result is negative in two’s complement).

V — Overflow Set when two’s-complement arithmetic overflows. For example, adding two positive numbers and getting a negative result.

```
ldi r16, 0x7F ; +127 (largest positive signed 8-bit)
ldi r17, 0x01
add r16, r17 ; result = 0x80 = -128, V=1 (signed overflow)
```

S — Sign $S = N \text{ XOR } V$. This is the “true” sign of the result after accounting for overflow. Use BRGE/BRLT (which test S) for signed comparisons, not N alone.

H — Half-carry Carry out of bit 3 into bit 4. It matters for nibble-oriented arithmetic and some compare/add/subtract cases. AVR does **not** have a DAA instruction, so you usually inspect H only in specialised code.

T — Transfer A single-bit scratch pad. BST copies a bit from a register into T; BLD copies T into a register bit. Useful for bit manipulation without clobbering other flags.

I — Global Interrupt Enable When I=1, the CPU responds to interrupts. SEI sets it; CLI clears it. Note: on AVRxt (ATtiny3217), the I bit is **not** cleared by hardware on ISR entry and is **not** set by RETI — that classic-AVR behavior was replaced. Nested-interrupt arbitration is handled by the CPUINT peripheral through its LVL0EX/LVL1EX status bits, which mark whether a normal- or high-priority interrupt is currently executing. Chapter 17 covers this in detail.

3.4.2 Reading and Saving SREG

SREG is a normal I/O register. You can save and restore it:

```
in r0, SREG ; save SREG
cli ; disable interrupts (clear I bit)
; ... critical section ...
out SREG, r0 ; restore SREG (re-enables interrupts if I was set)
```

This pattern appears constantly in ISR prologues and critical sections.

If you remember only one thing about SREG, remember this: flags are part of program state. If your code depends on them, preserve them deliberately. The CPU does not automatically save or restore SREG when entering or leaving an ISR; that is your software’s responsibility.

Instructions introduced: IN / OUT (revisited) IN Rd, A — Load I/O register at address A into Rd. 1 cycle. OUT A, Rr — Write Rr to I/O register at address A. 1 cycle. Range: A = 0x00–0x3F only. For higher addresses, use LDS/STS.

Instruction introduced: SEI SEI — Set Global Interrupt Enable (I bit in SREG). 1 cycle.

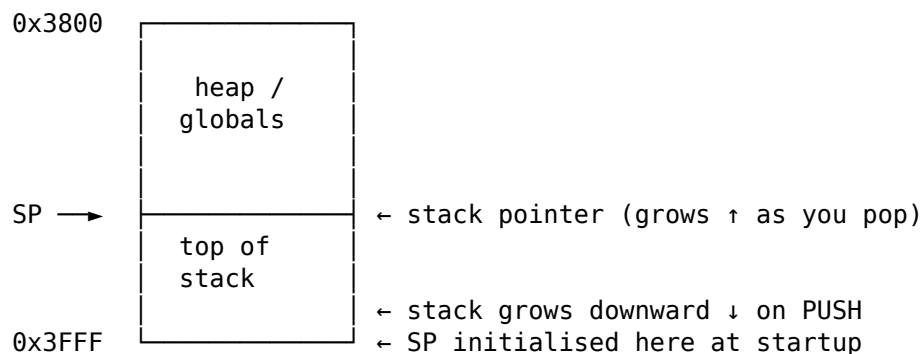
Instruction introduced: CLI CLI — Clear Global Interrupt Enable. 1 cycle. Next instruction always executes before any interrupt is served (pipeline guarantee).

3.5 The Stack and Stack Pointer

The stack is a **last-in, first-out** (LIFO) region of SRAM. The AVR stack **grows downward** — pushing data decrements the stack pointer first, then writes.

That “grows downward” detail matters. On AVR, the top of SRAM is the natural starting point for the stack, and each push moves toward lower addresses. Beginners often imagine the stack growing upward because that feels more intuitive. AVR does the opposite.

SRAM at 0x3800–0x3FFF:



SP is a 16-bit register implemented as two I/O registers: - SPL at I/O address 0x3D (low byte of SP) - SPH at I/O address 0x3E (high byte of SP)

On the ATtiny3217, reset hardware loads SP with RAMEND automatically. You will still see startup code write it explicitly, because that is harmless and keeps examples portable to older AVR parts where software initialisation is required.

So there are really two questions:

- What value should SP start with? RAMEND.
- Who sets it? On ATtiny3217, the hardware does; on some older AVR parts, startup code must do it.

Both bytes are in the IN/OUT address range, so explicit initialisation uses OUT:

```
ldi r16, lo8(RAMEND)    ; 0xFF
out SPL, r16
ldi r16, hi8(RAMEND)    ; 0x3F
out SPH, r16
```

There is one subtle hardware assist here: when software writes SPL, the CPU automatically disables interrupts for up to four instructions, or until the next I/O-memory write, whichever comes first. That prevents an interrupt from observing a half-updated 16-bit stack pointer between the low-byte and high-byte writes. It is still good practice to initialise the stack before enabling interrupts or making subroutine calls.

3.5.1 PUSH and POP

```
push r16    ; SP -= 1, then write r16 to SRAM[SP] (1 cycle on AVRxt)
pop  r16    ; read r16 from SRAM[SP], then SP += 1 (2 cycles)
```

(Older AVR parts list PUSH as 2 cycles; on AVRxt — including the ATtiny3217 — PUSH is 1 cycle. POP remains 2 cycles.)

Push decrements first, then writes. Pop reads first, then increments.

3.5.2 How CALL Uses the Stack

CALL / RCALL automatically pushes the **return address** onto the stack so RET can find it:

```
rcall my_function    ; push PC+1, jump to my_function
; RET in my_function pops the return address and jumps back here
```

On ATtiny3217 (16K word address space), return addresses are 2 bytes (16-bit). The return address stored on the stack is a **word pointer**, not a byte pointer. The least-significant byte is pushed first, at the higher stack address, then the most-significant byte is pushed at the next lower address. On devices with >128KB flash, return addresses are 3 bytes. Always account for this when calculating stack depth.

This is why even a program with “no local variables” still uses stack space. Every nested call consumes stack bytes for the return address, and any saved registers add to that cost.

3.6 The Program Counter (PC)

The Program Counter is a register inside the CPU that holds the **word address** of the next instruction to execute. It is not directly accessible via IN/OUT — you control it only through jump, branch, and call instructions.

In other words, the PC is where control flow lives. Arithmetic changes data; branches and calls change the PC.

```
After reset:      PC = 0x0000    (first word in flash)
After JMP main:   PC = address of first instruction in main
After RCALL foo:  PC = address of foo; old PC+1 on stack
After RET:        PC = popped from stack
```

The PC is 14 bits wide on ATtiny3217 (addressing 0x0000–0x3FFF in word units, covering the full 32KB flash).

Most AVR instructions are one 16-bit word. Some instructions, such as JMP, CALL, LDS, and STS, are two-word instructions. This matters for three reasons:

- The PC counts instruction words, not bytes.
- Skip instructions must skip either one word or two words depending on the next instruction.
- Disassembly addresses may be shown as byte addresses by tools, while the CPU return address is stored as a word pointer.

3.6.1 Instruction Fetch Pipeline

AVR uses a **single-stage pipeline**: while the CPU executes instruction N, it fetches instruction N+1 from flash. This is why most instructions execute in 1 cycle — the fetch is free. Instructions that change the PC (branches, jumps, calls, returns) take 2–4 cycles because the pipeline must be flushed and a new fetch begun.

At the device level, this is the basis for the usual AVR rule of thumb: straight-line register code can approach 1 MIPS per MHz of CLK_CPU. That is not a promise that every instruction is single-cycle; it is a reminder that branches, memory accesses, calls, returns, and peripheral waits are where the extra cycles usually appear.

You do not need to become a pipeline expert to use AVR well. The practical lesson is simpler: straight-line code is cheap, but changing direction costs extra cycles.

Cycle:	1	2	3	4	
Fetch:	N	N+1	N+2	N+3	
Execute:		N	N+1	N+2	← 1 cycle/instruction (normal)

With a taken branch at instruction N:

Fetch:	N	N+1	target	target+1	
Execute:		N	---	target	← N+1 fetch wasted; branch costs 2 cycles

This is why BRNE costs 1 cycle when not taken (no pipeline flush) and 2 cycles when taken (flush + re-fetch from target).

3.7 I/O Register Space

The ATtiny3217 peripherals are controlled through memory-mapped I/O registers. They fall into two zones:

Address range	Access method	Examples
0x0000–0x003F	IN / OUT (fast)	VPORTB_OUT, SREG, SPL, SPH, CCP
0x0040–0x0FFF	LDS / STS (2 words)	PORTB_DIR, TCA0_CNT, USART0_RXDATAL
0x1000–0x13FF	LDS / STS	NVMCTRL, SIGROW, FUSES, USERROW

The low 64 addresses (0x00–0x3F) are the “classic” I/O space reachable with 1-word IN/OUT instructions in 1 cycle. The low half of that space (0x00–0x1F) is also reachable by SBI, CBI, SBIC, and SBIS. Everything above 0x3F needs the 2-word LDS/STS instructions, which take 3 and 2 cycles respectively.

This is one of the most important performance distinctions on AVR. Two registers may both control hardware, but if one is in low I/O space and the other is in extended space, the instruction choices and timing differ.

The ATtiny3217’s important low I/O registers:

I/O Addr	Name	Description
0x05	VPORTB_OUT	Virtual PORTB output register (fast GPIO access)
0x04	VPORTB_DIR	Virtual PORTB direction register
0x3F	SREG	Status Register
0x3E	SPH	Stack Pointer High
0x3D	SPL	Stack Pointer Low
0x34	CCP	Configuration Change Protection (unlock protected registers)
0x1F	GPOR3	General Purpose I/O Register 3 (scratch)
0x1E	GPOR2	General Purpose I/O Register 2 (scratch)
0x1D	GPOR1	General Purpose I/O Register 1 (scratch)
0x1C	GPOR0	General Purpose I/O Register 0 (scratch)

VPORTx_* mirrors let you do single-cycle GPIO with IN/OUT and bit instructions, while the full PORTx_* peripheral block provides the richer DIRSET/OUTCLR/OUTTGL registers in extended I/O space.

GPOR0–3 are four registers with no peripheral function — pure scratch RAM accessible via IN/OUT in a single cycle. Useful for storing flags or counters that need fast ISR access.

Think of them as “fast little mailboxes” between parts of your program. They are not magic, but they can be handy when one-byte state needs to be very cheap to access.

The extended I/O region is where most modern tinyAVR peripherals live. A few important base addresses are:

Base	Peripheral	Why you care
0x0400	PORTA	Full GPIO configuration and atomic set/clear/toggle
0x0600	ADC0	ADC and Peripheral Touch Controller path
0x0800	USART0	Serial port used in later examples
0x0810	TWI0	I2C-compatible bus controller
0x0820	SPI0	SPI controller
0x0A00	TCA0	16-bit timer/counter
0x1000	NVMCTRL	Flash/EEPROM controller
0x1100	SIGROW	Factory calibration and signature data
0x1280	FUSES	Device configuration fuses

This is why examples often prefer header symbols such as `USART0_STATUS` or `PORTA_DIRSET` instead of raw numbers. The symbol tells you which peripheral block you are touching and lets the device header carry the exact address.

3.8 Putting It Together — A Register-State Walk-Through

Let's trace the CPU state through the blink program from Chapter 1, cycle by cycle through the setup section:

The goal here is not to memorise every state transition. It is to get used to thinking in terms of registers, memory side effects, and whether flags changed.

```
; State at reset:
; PC = 0x0000
; SP = 0x3FFF on ATtiny3217 reset hardware
; SREG = 0x00
; Treat general registers as undefined until your code sets them

ldi r16, lo8(RAMEND)    ; r16 = 0xFF. PC → next instr. 1 cycle.
out SPL, r16           ; explicit re-init of SP low byte. 1 cycle.
ldi r16, hi8(RAMEND)    ; r16 = 0x3F. PC → next. 1 cycle.
out SPH, r16           ; explicit re-init of SP high byte. 1 cycle.

ldi r16, (1 << 3)      ; r16 = 0x08. 1 cycle.
sts PORTA_DIRSET, r16   ; Write 0x08 to 0x0401.
                       ; PA3 now output. 2 cycles.

; State now:
; r16 = 0x08, SP = 0x3FFF, PORTA_DIR bit 3 = 1, all flags unchanged
```

Flags: LDI, OUT, STS do **not** affect SREG at all. Only arithmetic, logic, and bit operations change flags.

3.9 Key Architectural Constraints Summary

These are the constraints you will bump into constantly. They look dry, but they explain a large share of beginner errors.

Constraint	Rule
LDI range	r16-r31 only
IN/OUT range	I/O addresses 0x00-0x3F only
SBIW/ADIW pairs	r25:r24, r27:r26, r29:r28, r31:r30 only
Pointer pairs	X = r27:r26, Y = r29:r28, Z = r31:r30
MUL result	Always in r1:r0
Stack direction	Grows downward; PUSH decrements first
Return address	2 bytes on stack (ATtiny3217, ≤128KB flash)
Interrupt response	6 cycles minimum on ATtiny3217 flash sizes > 8 KB

Constraint	Rule
Flash read via LD	Use address + 0x8000 (mapped flash, AVRxt)
Flash read via LPM	Use Z = byte address, reads via program bus
EEPROM address	0x1400–0x14FF in data space, controlled by NVMCTRL
Flash/EEPROM writes	Use NVMCTRL command sequences, not ordinary SRAM stores

“MUL result — Always in r1:r0” vs. the zero register. This is the hardware rule: MUL/MULS/MULSU unconditionally write the 16-bit product to r1:r0. Note that this is *not* in conflict with the convention — used later in this book and by avr-gcc — that r1 holds 0 at all times: MUL is precisely the instruction that temporarily breaks that promise, so code following the convention must copy the product out and run `clr r1` afterwards. The convention and the fix-up are introduced with multi-byte arithmetic (Chapter 10) and revisited in the bit-manipulation (Chapter 26) and ADC (Chapter 30) chapters.

3.10 Summary

Harvard = two buses. Flash for code. SRAM for data.

ATtiny3217 memory map:

```
0x0000–0x003F  I/O (IN/OUT)
0x0040–0x0FFF  Peripherals (LDS/STS)
0x1000–0x13FF  NVMCTRL / SIGROW / FUSES / USERROW region
0x1400–0x14FF  EEPROM (256 bytes)
0x3800–0x3FFF  SRAM (2 KB)
0x8000–0xFFFF  Flash mapped in data space (AVRxt feature)
```

32 registers (r0–r31):

```
r0–r15: general, no LDI
r16–r31: general + LDI + most immediates
r27:r26 = X, r29:r28 = Y, r31:r30 = Z (pointer pairs)
```

SREG (0x3F): C Z N V S H T I – set by ALU, read by branches.

SP = SPH:SPL, initialise to RAMEND. Stack grows down.

On ATtiny3217, reset hardware already loads SP = RAMEND.

PC = word address, 14 bits. Pipeline prefetch = 1 cycle/instr normal.

3.11 Exercises

1. What is the data-bus address of the first byte of flash on ATtiny3217? What is the data-bus address of the last byte of flash?
2. After `ldi r16, 0x80` and `ldi r17, 0x80` and `add r16, r17`, which SREG flags are set? Work it out by hand, then verify with:

```
avr-as -mmcu=attiny3217 -o /tmp/t.o - <<'EOF'
.text
```



```
ldi r16, 0x80
ldi r17, 0x80
add r16, r17
```

EOF

```
avr-objdump -d /tmp/t.o
```

(The disassembly won't show flag values — you must reason through them.)

3. Write a short assembly snippet that:
 - Saves SREG to r0
 - Disables interrupts
 - Does some work (e.g. `ldi r16, 42`)
 - Restores SREG (and thus the interrupt state)
4. Calculate the maximum stack depth for a program that calls a function, which calls another function. How many bytes does the stack use just for return addresses?
5. Why can't you write `ldi r15, 0xFF`? What two-instruction sequence achieves the same result?

Next: Chapter 4 — AVR-AS Syntax and Directives

Chapter 4

AVR-AS Syntax and Directives

Assembly source is more than a list of instructions. Before the CPU can run anything, the assembler has to answer basic questions: where does the code go, what data should be emitted, which names are constants, and which labels refer to real addresses? Those decisions are controlled by **directives** — assembler commands that begin with a dot.

This chapter is about that “language around the instructions.” If Chapter 1 showed how to make the chip do something, Chapter 4 explains how to write assembly source in a way the toolchain can understand and organise correctly.

4.1 File Extensions: .S vs .s

GAS accepts two extensions:

Extension	Preprocessor	Use
.S	Yes — C preprocessor runs first	Always use this
.s	No — raw GAS input	Rarely; only for generated files

The uppercase .S lets you use `#include`, `#define`, `#ifdef`, and all other C preprocessor directives. Since we always want `#include <avr/io.h>` for register names, **always use .S**.

For a beginner, the practical rule is simple: if you are hand-writing AVR assembly for this book, name the file with an uppercase .S and do not think about it further.

The build flow preprocesses .S files before assembly. If you want to inspect that preprocessed output yourself, use the compiler driver:

```
# Inspect the preprocessed source that avr-as will assemble:
avr-gcc -x assembler-with-cpp -mmcu=attiny3217 -E blink.S
```

You normally do not need to run this manually if your build uses `avr-gcc` as the driver, because it sees .S and runs the preprocessor before invoking the assembler. `avr-as` itself does **not** run `cpp`; if you call `avr-as` directly, feed it preprocessed assembly or avoid `#include`/`#define`.

For hand-written examples in this book, prefer this pattern:

```
avr-gcc -x assembler-with-cpp -mmcu=attiny3217 -c blink.S -o blink.o
```

That keeps device-header symbols such as `PORTA_DIRSET`, `RAMEND`, `SREG`, `SPL`, `SPH`, and `MAPPED_PROGMEM_START` available in the source.

Sidebar: look at what you just made. Before diving into directives, it is worth seeing that your source really does turn into machine code. Link the example to an ELF and ask two tools what came out:

```
avr-gcc -mmcu=attiny3217 -nostartfiles -e main blink.S -o blink.elf
avr-objdump -d blink.elf      # disassemble: bytes <-> instructions
avr-size blink.elf           # how much flash/RAM it uses
```

avr-objdump -d shows each instruction next to the bytes it assembled to:

```
00000000 <__ctors_end>:
    0: 03 9a          sbi 0x00, 3

00000002 <loop>:
    2: 0b 9a          sbi 0x01, 3
    4: 03 d0          rcall .+6          ; 0xc <delay>
    6: 0b 98          cbi 0x01, 3
    a: fb cf          rjmp .-10         ; 0x2 <loop>
```

and avr-size reports the footprint (Berkeley format: code, initialised data, zero-init RAM):

text	data	bss	dec	hex	filename
22	0	0	22	16	blink.elf

That `sbi 0x00, 3` is your `sbi VPORTA_DIR, 3` — two bytes of flash. Keeping `objdump -d` and `size` close at hand as you learn the directives below turns every change into something you can immediately see in the output. The “Examining the Output with `avr-objdump`” section at the end of this chapter and Appendix F go deeper.

4.2 Comments

GAS accepts three comment styles. In `.S` files all three work:

```
; This is a comment — semicolon style, common in AVR assembly

// This is a comment — C++ line style

/* This is a
   block comment — C style */
```

Semicolon is traditional in AVR assembly and what you’ll see most. Use block comments for file headers and section separators.

Style matters more in assembly than in many other languages because the code is so compact. A short comment can carry a lot of explanatory weight.

4.3 Instruction Syntax

AVR GAS uses a **destination-first, source-second** convention:

mnemonic destination, source

Examples:

```
mov r16, r17      ; r16 ← r17
add r16, r17      ; r16 ← r16 + r17
```

```
ldi r16, 42      ; r16 ← 42
sts 0x0421, r16  ; mem[0x0421] ← r16
ld  r16, X       ; r16 ← mem[X]
```

This matches GCC output and the Atmel/Microchip instruction set manual.

Instruction syntax is not just cosmetic; it reflects fields in the opcode. On AVR, many instructions cannot encode every register or address. The assembler will reject operands that do not fit the instruction format:

```
ldi r15, 0x55    ; ERROR: LDI can only encode r16-r31
ldi r16, 0x55    ; OK

out 0x3F, r16    ; OK: OUT reaches I/O address 0x00-0x3F
out 0x40, r16    ; ERROR: use STS for extended I/O
sts 0x0040, r16  ; OK: direct store to data space

sbi 0x1C, 0      ; OK: SBI/CBI/SBIC/SBIS reach 0x00-0x1F
sbi 0x20, 0      ; ERROR: above bit-addressable low I/O range
```

Microchip's ATtiny3217 peripheral map puts the four general-purpose I/O registers at 0x1C-0x1F, which makes them directly usable with SBI, CBI, SBIS, and SBIC. Most larger peripherals, such as PORTA at 0x0400 and USART0 at 0x0800, are outside the IN/OUT range and require LDS/STS or pointer-based LD/ST.

4.4 Number Literals

GAS accepts numbers in four bases:

```
ldi r16, 42      ; decimal
ldi r16, 0x2A    ; hexadecimal
ldi r16, 0b00101010 ; binary
ldi r16, 052     ; octal (leading zero – easy to mistake for decimal!)
```

All four produce the same value (42). Avoid octal — the leading-zero trap causes real bugs. Hex and binary are clearest for hardware work.

Character literals also work:

```
ldi r16, 'A'     ; r16 = 65
ldi r16, '\n'    ; r16 = 10
```

4.5 Expressions

GAS evaluates constant expressions at assembly time. You can use:

```
ldi r16, (1 << 4)      ; bit shift – evaluates to 16
ldi r16, (1 << 4) | (1 << 2) ; bitwise OR – evaluates to 20
ldi r16, RAMEND & 0xFF ; mask – evaluates to 0xFF
ldi r16, RAMEND / 256   ; division – evaluates to 0x3F
ldi r16, 10 * 5 - 8     ; arithmetic – evaluates to 42
```

These are evaluated at **assembly time** — no runtime cost. They exist only to make source code readable.

That is an important distinction. These expressions do not generate arithmetic instructions. The assembler does the math once while building the file, and the CPU never sees

the expression itself.

4.5.1 Built-in Expression Functions

GAS provides helper functions for splitting 16-bit values into bytes:

```
ldi r16, lo8(RAMEND)      ; low byte of 0x3FFF = 0xFF
ldi r16, hi8(RAMEND)      ; high byte of 0x3FFF = 0x3F
ldi r16, lo8(0x1234)      ; = 0x34
ldi r16, hi8(0x1234)      ; = 0x12
```

Two more functions matter for AVRxt flash access (see Section 4.11):

```
; pm(label) – convert byte address to word address (for IJMP/ICALL targets)
ldi ZL, lo8(pm(my_func))  ; ZL = word address low byte of my_func
ldi ZH, hi8(pm(my_func))  ; ZH = word address high byte of my_func
```

For table addresses, be explicit about which address space you mean:

```
ldi ZL, lo8(table)        ; flash byte address for LPM
ldi ZH, hi8(table)

ldi ZL, lo8(table + MAPPED_PROGMEM_START) ; data-space address for LD
ldi ZH, hi8(table + MAPPED_PROGMEM_START)
```

The ATtiny3217 datasheet describes both views: LPM sees flash starting at 0x0000, while ordinary data-space loads see mapped flash starting at 0x8000. Those two forms are not interchangeable.

4.6 Labels

A label marks a position in the output. It ends with a colon:

```
main:
    ldi r16, 0xFF

loop:
    dec r16
    brne loop      ; reference label by name – no colon when referencing
```

Labels have two uses: 1. **Jump/branch targets** — as above 2. **Data addresses** — when placed before `.byte`, `.word`, etc.

You can think of a label as a bookmark the assembler turns into an address. Sometimes that address is where execution should jump. Sometimes it is where data begins.

```
table:
    .byte 1, 2, 3, 4      ; "table" now equals the address of byte 1
```

4.6.1 Global vs Local Labels

By default, labels are **local to the object file** — the linker cannot see them unless you explicitly export them:

```
.global main      ; export "main" so the linker can use it as entry point
.global __vectors ; export vector table symbol

main:             ; now visible to linker
...
```

```
internal_helper:    ; NOT global – invisible outside this file
...
```

For functions, you can also tell the linker/debugger what kind of symbol you are defining:

```
.global main
.type    main, @function
main:
    ; ...
    ret
.size    main, .-main
```

`.type` and `.size` do not change the emitted instructions. They improve symbol meta-data, which makes `avr-objdump`, `avr-nm`, and debuggers report function boundaries more cleanly.

4.6.2 GAS Local Labels (Numeric)

Naming every loop, branch target, and temporary label creates a pollution of unique names. GAS solves this with **numeric local labels**:

```
1:          ; define label "1"
    dec r16
    brne 1b          ; "1b" = most recent backward occurrence of label "1"

    tst r16
    breq 2f          ; "2f" = next forward occurrence of label "2"
    ldi r16, 0xFF
2:          ; define label "2"
    ...
```

Rules: - Any number 0–9 (or longer: 10:, 255:) can be a local label - Nb = nearest backward occurrence of N: - Nf = nearest forward occurrence of N: - The same number can be reused anywhere in the file — each definition is distinct. GAS resolves 1b to the *nearest* preceding 1:.

This is invaluable in loops:

```
ldi r18, 10          ; outer loop count
1:          ; outer loop top
    ldi r17, 0
    ldi r16, 0
2:          ; inner loop top
    sbiw r16, 1
    brne 2b          ; back to inner "2:"
    dec r18
    brne 1b          ; back to outer "1:"
```

No naming conflict — 1b and 2b refer to the right labels unambiguously.

This style looks strange at first, but it becomes natural quickly. Named labels are best for important program locations such as `main` or `uart_tx`. Numeric labels are best for short internal loops where a descriptive name would add more clutter than clarity.

4.6.3 File-Scoped Local Labels (`.L` prefix)

Labels starting with `.L` are stripped from the symbol table by the linker and do not appear in debug info. Use them for internal subroutine labels you don't want to export:

```
my_subroutine:
.Lloop:
```

```
dec r16
brne .Lloop
ret
```

.Lloop will not appear in `avr-nm` output or debugger symbol lists.

4.7 Sections

A **section** is a named region of output. The linker arranges sections into the final memory map. Every assembly statement belongs to the *current* section.

If labels are bookmarks, sections are containers. They tell the linker what kind of thing you are defining: executable code, read-only flash data, SRAM data, zero-initialised storage, and so on.

4.7.1 Core Sections

```
.section .text           ; code – goes to flash
.section .data           ; initialised SRAM data – startup code must copy it in
.section .bss            ; zero-initialised SRAM data – startup code must clear it
.section .rodata         ; read-only constants – stay in flash
```

For the ATtiny3217, the linker maps those abstract sections onto a concrete device memory map:

Section	Runtime location	ATtiny3217 address range
.text, .rodata, .vectors	Flash/program memory	Code space 0x0000-0x7FFF bytes
.data	SRAM at runtime, initial image in flash	SRAM 0x3800-0x3FFF
.bss	SRAM, no file contents	SRAM 0x3800-0x3FFF
EEPROM data	EEPROM, NVMCTRL-controlled	Data space 0x1400-0x14FF
Mapped flash reads	Flash through data bus	Data space 0x8000-0xFFFF

The assembler does not know your intent beyond the section and bytes you emit. The linker script is what makes `.text` land in flash and `.bss` reserve SRAM. If you bypass the normal linker script or use custom sections, inspect the linker map and disassembly.

Shorthand forms (equivalent):

```
.text           ; same as .section .text
.data          ; same as .section .data
.bss           ; same as .section .bss
```

You can switch between sections multiple times in one file:

```
.text
foo:
    ldi r16, 5
    ret

.data
my_var: .byte 0
```

```
.text
bar:
    ldi r16, 10
    ret
```

The assembler concatenates all `.text` chunks together, all `.data` chunks together, etc. The order of sections in the output is determined by the linker script, not the source order.

That is why switching sections in the middle of a file is normal. You are not describing the final physical layout directly; you are describing which bytes belong to which category.

4.7.2 Section Flags and Types (Full Form)

The full `.section` directive takes optional flags:

```
.section name, "flags", @type
```

Flag	Meaning
a	Allocatable (takes space in memory)
x	Executable
w	Writable

Type	Meaning
@progbits	Contains data/code
@nobits	Takes space but has no data in file (BSS)

Most of the time you use the shorthand and GAS fills in reasonable defaults. You need the full form for special sections like `.vectors`:

```
.section .vectors, "ax", @progbits
```

This tells the linker: place this section at the start of flash (the linker script maps `.vectors` to address `0x0000`), it contains code (x), and it is allocatable (a).

The vector table is executable code on this AVR target, not a table of raw ISR addresses. Each vector slot contains an instruction, commonly `jmp` handler on ATtiny3217-sized flash parts. That is why `.vectors` uses "ax" rather than plain read-only data flags.

You do not need to memorise all the flags immediately. What matters is knowing that the full form exists when you need something more specific than the common defaults.

4.8 Data Directives

These directives emit bytes into the current section.

That phrase “emit bytes” is worth taking literally. Directives such as `.byte` and `.word` do not create variables in the high-level-language sense. They cause exact byte patterns to be written into the object file.

4.8.1 `.byte` — 8-bit values

```
.byte 0x42          ; one byte: 0x42
.byte 1, 2, 3, 4    ; four bytes
```



```
.byte 'H', 'i', '!'      ; character literals
.byte (1 << 3)           ; expression – evaluates to 8
```

4.8.2 .word — 16-bit values (little-endian)

```
.word 0x1234             ; emits 0x34, 0x12 (little-endian)
.word 0, 1, 2            ; six bytes: three 16-bit words
```

Warning: On AVR, `.word` is 16 bits. In non-AVR GAS targets `.word` can be 32 bits. Never assume word size when reading unfamiliar assembly.

In flash tables used by LPM, remember that bytes still come out in little endian order. If you write `.word 0x1234`, the first byte at the lower address is 0x34 and the second is 0x12. If `Z` points at the label and you execute `lpm r16, Z+`, `r16` receives 0x34 first.

4.8.3 .long — 32-bit values

```
.long 0x12345678         ; emits 78 56 34 12 (little-endian)
```

4.8.4 .ascii and .asciz — Strings

```
.ascii "Hello"           ; 5 bytes: H e l l o – NO null terminator
.asciz "Hello"           ; 6 bytes: H e l l o 0x00 – WITH null terminator
.string "Hello"          ; alias for .asciz
```

Almost always use `.asciz` or `.string` so C string functions work correctly.

If you forget the terminating zero, the bytes are still valid assembly data, but any code that expects a C-style string will keep reading past the intended end.

4.8.5 .skip and .space — Reserve Bytes

```
buf:    .skip 16          ; reserve 16 bytes (content undefined in .text/.data)
buf:    .skip 16, 0       ; reserve 16 bytes, filled with 0x00
```

In `.bss`, content is always zeroed by startup code regardless of fill value.

Do not use `.skip` in `.text` as a cheap delay or timing pad. It emits bytes into flash; those bytes will be interpreted as instructions if execution falls through them. Use real instructions such as `nop`, or `branch around data`.

4.8.6 .balign — Alignment

```
.balign 2                ; align to 2-byte boundary (pad with 0x00 if needed)
.balign 4                ; align to 4-byte boundary
```

Required before `.word` or `.long` data in some contexts. Also used to align code for performance (though on AVR this matters less than on ARM).

AVR instructions are 16-bit words, and some are 32-bit two-word instructions. Keeping code and 16-bit flash data on even byte boundaries makes disassembly and label arithmetic easier to reason about. It also avoids accidental odd-byte entry points when mixing `.byte` tables and executable code in the same section.

4.9 Constant Directives

4.9.1 .equ — Define a Constant

```
.equ LED_PIN, 4
.equ LED_MASK, (1 << LED_PIN) ; expressions work
.equ DELAY, 5000
```

`.equ` defines a **symbol** with a value. It cannot be redefined later. Use it for things that truly never change: pin numbers, masks, magic values.

In practice, `.equ` is how you turn anonymous numeric constants into named ideas. `0x08` is just a number; `LED_MASK` tells the reader what the number means.

Using the constant:

```
ldi r16, LED_MASK ; assembler substitutes the value at assembly time
```

Prefer device-header names over hand-written `.equ` addresses whenever the name exists:

```
#include <avr/io.h>

ldi r16, (1 << 3)
sts PORTA_DIRSET, r16 ; better than .equ PORTA_DIRSET, 0x0401
```

The Microchip device header follows the datasheet address map. Using the header keeps source code tied to the selected `-mmcu=attiny3217` target instead of a copied constant that may be wrong on a different AVR.

4.9.2 .set — Reassignable Constant

```
.set count, 0
.set count, count + 1 ; OK — .set can be redefined
.set count, count + 1 ; count = 2 now
```

`.set` is useful for building values incrementally or for conditional definitions. Equivalent to `#define + #undef` patterns but within GAS.

4.9.3 #define — Preprocessor Macro

Because `.S` files go through `cpp`, you can also use:

```
#define LED_PIN 4
#define LED_MASK (1 << LED_PIN)
#define SET_BIT(reg, bit) ((reg) |= (1 << (bit))) ; functional macro
```

`#define` is processed *before* GAS sees the file; `.equ` is processed *by* GAS. For simple constants, `.equ` is idiomatic in assembly. For complex macros or conditional compilation, `#define`/`#ifdef` are necessary.

If you are unsure which to choose, prefer `.equ` for plain assembly constants. Reach for `#define` only when you really need preprocessor behavior.

4.10 The .org Directive

`.org` sets the **location counter** — the current output offset within the section. It can move the location counter **forward**, creating padding, but it cannot move it backward.

This is a more advanced tool than it first appears. It is useful when a layout must match hardware-defined slots, such as interrupt vectors, but it is also an easy way to create confusing gaps if used casually.

```
.org 0x0000
    jmp main           ; lands exactly at byte address 0

.org 0x0004
    jmp isr_int0       ; next 4-byte JMP slot

.org 0x0008
    jmp isr_pcint0     ; next slot
```

If you use JMP, each vector entry consumes 4 bytes. If you use RJMP, each entry consumes 2 bytes, so the .org spacing changes accordingly. The exact vector table is in the datasheet and in <avr/iotn3217.h> as _VECTOR(n) macros. Chapter 17 builds the full vector table.

The ATtiny3217 has 32 KB of flash, so the normal interrupt vector entries use JMP-sized slots in the examples. Microchip also documents a Compact Vector Table mode through CPUINT.CTRLA.CVT; when enabled, the interrupt table groups normal interrupts into three entries: NMI at vector address 1, priority level 1 at vector address 2, and all priority level 0 interrupts at vector address 3. This book uses the ordinary vector table unless a later chapter explicitly says otherwise.

So the real lesson is not “always use .org.” The lesson is: use it when the hardware requires exact placement, and understand the instruction size when you do.

Warning: .org without context is relative to the *start of the current section*. In .vectors, .org 0 means “start of the vector section”, which the linker places at flash address 0. In .text, .org 0x100 would create a 0x100-byte gap from the section’s start — usually not what you want.

4.11 Flash Data on AVRxt (ATtiny3217)

This is where AVRxt differs from classic AVR in an important way.

This section is easy to skim past, but it matters because it changes how you think about constants in flash. On older AVR parts, flash reads feel like a special case. On AVRxt, they can look much more like ordinary data reads.

Classic AVR (ATmega328P): Flash is NOT in the data address space. To read a byte from flash at runtime, you must use the LPM instruction with the Z register pointing to the flash byte address.

AVRxt (ATtiny3217): Flash IS mapped in the data address space at offset MAPPED_PROGMEM_START (= 0x8000). You can read flash with ordinary LD instructions by adding this offset to the flash address.

The concrete ATtiny3217 memory map behind that statement is:

0x0000-0x003F	I/O registers
0x0040-0x0FFF	Extended I/O / peripheral registers
0x1000-0x13FF	NVM I/O registers and data
0x1400-0x14FF	EEPROM, 256 bytes
0x3800-0x3FFF	SRAM, 2 KB
0x8000-0xFFFF	32 KB flash mapped into data space

Only the last range is the mapped view of program flash. EEPROM is a separate non-volatile memory region controlled by NVMCTRL; do not put ordinary .rodata there by accident.

```
.section .rodata

table:
    .byte 10, 20, 30, 40, 50

.section .text

read_table_entry:                ; r24 = index, returns value in r25
    ldi ZL, lo8(table + MAPPED_PROGMEM_START)
    ldi ZH, hi8(table + MAPPED_PROGMEM_START)
    add ZL, r24                  ; Z now points to table[r24] in data space
    adc ZH, r1                   ; propagate carry (r1 = 0 by convention)
    ld r25, Z                    ; read from flash via data bus
    ret
```

The assembler expression `table + MAPPED_PROGMEM_START` adds 0x8000 to the flash byte address of `table`. The result fits in 16 bits because the table starts in the first 32KB of flash.

Data-space LD access to mapped flash is convenient, but it is not SRAM. The AVR instruction set manual notes that data-memory cycle counts for LD assume internal RAM and that NVM accesses can add implementation-dependent time. For constant tables this usually does not change the code structure, but it matters when doing tight cycle accounting.

Conceptually, you are taking a flash address and translating it into the corresponding data-space view of the same bytes.

Old LPM method still works on AVRxt and is sometimes needed for compatibility:

```
ldi ZL, lo8(table)              ; Z = flash byte address (no +0x8000)
ldi ZH, hi8(table)
lpm r25, Z                      ; read from flash via program bus
```

Both methods read the same data. On AVRxt, LD with mapped address is simpler and more natural. LPM is covered in Chapter 7.

That means you should understand both styles, even if the mapped-flash method is the one you use most often on the ATtiny3217.

4.12 The .include Directive

```
.include "macros.inc"           ; include a GAS file
```

Inserts the contents of another file at the point of the `.include`. The included file is processed by GAS (not cpp), so it cannot use `#define`.

For files that need preprocessor features, use `#include` instead:

```
#include <avr/io.h>              ; cpp include – register definitions
#include "my_macros.h"           ; cpp include – your own header
#include "data_tables.S"         ; GAS include – another assembly file
```

Use `#include <avr/io.h>` exactly once near the top of a `.S` file. That header selects the device-specific header from `-mmcu=attiny3217`; including `<avr/iotn3217.h>` directly works, but it makes the source less portable across build-system target changes.

4.13 Macro Directives

GAS has its own macro system separate from the C preprocessor.

```
.macro delay_ms ms_count
    ldi r19, \ms_count
.Lmacro_outer_\@:
    ldi r18, 0
    ldi r17, 0
.Lmacro_inner_\@:
    sbiw r17, 1
    brne .Lmacro_inner_\@
    dec r19
    brne .Lmacro_outer_\@
.endm
```

Using the macro:

```
delay_ms 100          ; expands inline - \ms_count → 100
delay_ms 500          ; second expansion - \@ ensures unique label names
```

Key syntax: - \param — substitute parameter value - \@ — unique invocation counter (prevents label collisions across expansions) - .endm — end macro definition

GAS macros are powerful but can obscure what code is actually emitted. Prefer subroutines (rcall/ret) over macros for anything that runs more than once, to save flash space.

This is a good place to be disciplined. Macros are tempting because they feel convenient, but they duplicate code at every use site. In assembly, that cost is often visible.

4.14 Full Section Layout Example

This is the structure of a well-organised AVR assembly file:

```
/*
 * myprogram.S — description
 * Build: avr-gcc -x assembler-with-cpp -mmcu=attiny3217 -c myprogram.S
 */

#include <avr/io.h>

/* — Constants ————— */
.equ    MY_PIN,    5
.equ    MY_MASK,   (1 << MY_PIN)

/* — Read-only data (flash) ————— */
.section .rodata

my_table:
    .byte 1, 2, 4, 8, 16, 32, 64, 128

my_string:
    .asciz "ATtiny3217"

/* — Uninitialised SRAM ————— */
.section .bss

rx_buffer: .skip 64      ; 64-byte ring buffer
rx_head:   .skip 1
```

```

rx_tail:      .skip 1

/* — Initialised SRAM (requires startup copy code or CRT startup) — */
.section .data

state:        .byte 0x01      ; starts as 1 in SRAM

/* — Interrupt vector table — */
.section .vectors, "ax", @progbits
.global __vectors

__vectors:
    jmp      main              ; 0x0000 reset

/* — Code — */
.section .text
.global main

main:
    ldi r16, lo8(RAMEND)
    out SPL, r16
    ldi r16, hi8(RAMEND)
    out SPH, r16
    ; ... rest of program

```

That example deliberately uses `.data` and `.bss` to show the sections, but a bare -nostartfiles assembly program must provide the startup behavior itself. If you define `.data`, somebody must copy its initial bytes from flash to SRAM. If you define `.bss`, somebody must clear it. The `avr-libc` C runtime normally does that before `main`; a minimal hand-written reset handler does not unless you write the code.

4.15 Examining the Output with `avr-objdump`

After assembling, inspect what the directives produced:

```

avr-gcc -x assembler-with-cpp -mmcu=attiny3217 -c syntax_demo.S -o syntax_demo.o
avr-gcc -mmcu=attiny3217 -nostartfiles -o syntax_demo.elf syntax_demo.o
avr-objdump -s -d syntax_demo.elf

```

The `-s` flag dumps the **raw contents** of every section as hex. Example output for `.rodata`:

Contents of section `.rodata`:

```

0074 01000101 01000101 01010100 48656c6c  ....Hell
0084 6f204156 5200                o AVR.

```

Cross-reference that output with the source: `morse_dit = [01, 00]`, `morse_dah = [01, 01, 01, 00]`, `greeting = "Hello AVR\0"`. Each byte appears in order. This is how you confirm that your directives produced exactly the layout you intended.

The `-d` flag shows the **disassembly** of `.text`:

```

00000008 <main>:
    8: 0f ef      ldi     r16, 0xFF
    a: 0d bf      out     0x3d, r16
    ...
   1a: e0 e7      ldi     r30, 0x70    ; ZL = lo8(morse_dit + 0x8000)
   1c: f0 e8      ldi     r31, 0x80    ; ZH = hi8(...)

```

Notice `ZH = 0x80` — the `0x8000` mapped flash offset is baked into the immediate at assembly time.

That is a perfect example of why inspecting output is useful: you can see a source-level idea become a real machine-level value.

Two other inspection commands are useful when debugging directives:

```
avr-objdump -h syntax_demo.elf    # section sizes, VMAs, file offsets
avr-nm -n syntax_demo.elf        # symbols sorted by address
```

Use `-h` to confirm that `.bss` takes SRAM space without occupying file bytes. Use `avr-nm` to confirm which labels survived linking; `.L` local labels should not clutter the public symbol list.

4.16 Quick Reference

Directive	Purpose	Example
<code>.equ</code>	Define constant	<code>.equ PIN, 4</code>
<code>.set</code>	Reassignable constant	<code>.set i, 0</code>
<code>.byte</code>	Emit bytes	<code>.byte 1, 2, 3</code>
<code>.word</code>	Emit 16-bit words	<code>.word 0x1234</code>
<code>.long</code>	Emit 32-bit words	<code>.long 0xDEADBEEF</code>
<code>.ascii</code>	String, no null	<code>.ascii "hi"</code>
<code>.asciz</code>	String with null	<code>.asciz "hi"</code>
<code>.skip</code>	Reserve bytes	<code>.skip 16</code>
<code>.balign</code>	Align location counter	<code>.balign 2</code>
<code>.org</code>	Set location counter	<code>.org 0x0000</code>
<code>.global</code>	Export symbol	<code>.global main</code>
<code>.section</code>	Switch/create section	<code>.section .text</code>
<code>.include</code>	Include GAS file	<code>.include "f.S"</code>
<code>.macro/.endm</code>	Define GAS macro	<code>.macro name p1</code>
<code>.type</code>	Mark symbol kind	<code>.type main, @function</code>
<code>.size</code>	Mark symbol size	<code>.size main, .-main</code>

Expression	Meaning	Result for <code>RAMEND=0x3FFF</code>
<code>lo8(x)</code>	Low byte	<code>0xFF</code>
<code>hi8(x)</code>	High byte	<code>0x3F</code>
<code>pm(label)</code>	Word address	<code>addr / 2</code>
<code>(1 << n)</code>	Bit mask	<code>(1 << 4) = 0x10</code>

4.17 Summary

`.S` extension → `cpp` runs first → `#include` works.

Sections:

- `.text` = flash code
- `.rodata` = flash constants (read via mapped `addr + 0x8000` on AVRxt)
- `.data` = SRAM, initialised if your startup code/CRT copies it
- `.bss` = SRAM, zero-init if your startup code/CRT clears it
- `.vectors` = must be first in flash, holds the interrupt vector table

Labels:

```
name:      = define label
.Lname:    = local, stripped from symbol table
1:         = numeric – reusable, resolved by 1f / 1b
```

Constants: `.equ` (fixed), `.set` (reassignable), `#define` (preprocessor).

AVRxt flash read: address + `MAPPED_PROGMEM_START` (0x8000), then LD.

ATtiny3217 data map: EEPROM is 0x1400-0x14FF; SRAM is 0x3800-0x3FFF.

Low I/O: IN/OUT reach 0x00-0x3F; SBI/CBI/SBIS/SBIC reach 0x00-0x1F.

4.18 Exercises

1. Write a `.rodata` section containing a table of the first 8 powers of 2 (1, 2, 4, 8, ...). Write code that reads `table[r24]` using the mapped flash address and returns the value in `r25`.
2. Add a `.bss` variable `byte_count` and a `.data` variable `max_count` initialised to 10. Write code that increments `byte_count` using `LDS/ STS` until it equals `max_count`, then halts.
3. Rewrite the Chapter 1 blink delay using a GAS `.macro` named `delay` that takes one parameter (outer loop count). Show that two calls `delay 25` and `delay 50` both assemble without label conflicts.
4. Assemble `syntax_demo.S` and run:

```
avr-objdump -s syntax_demo.elf
```

Verify the bytes in `.rodata` match the `morse_dit`, `morse_dah`, and `greeting` definitions in the source.
5. What happens if you put `.word 0x1234` inside a `.bss` section? Try it and inspect the linker output. Why does the linker warn or error?

Next: Chapter 5 — Debugging AVR Assembly with GDB

Chapter 5

Debugging AVR Assembly with GDB

Assembly debugging is different from source-level debugging in a high-level language. There is no hidden runtime to explain what happened. The evidence is the program counter, the registers, the status flags, SRAM, I/O registers, the stack pointer, and the instruction stream in flash.

This chapter shows how to debug the assembly style used in this book: GNU AVR assembly syntax in .S files, preprocessed through `avr-gcc` and assembled by `avr-as`. The target board is the ATtiny3217 Curiosity Nano, using its on-board debugger over UPDI. The core habit is simple: build an ELF with symbols, keep labels at meaningful locations, stop at instruction boundaries, inspect the CPU state, and prove what changed after each instruction.

5.1 Why This Chapter Belongs Here

By this point you have seen the source format, sections, labels, symbols, and the build flow that produces an ELF file. That is the right moment to learn debugging. If you wait until after timers, USART, SPI, interrupts, and bootloaders, you will have too many moving parts. If you learn GDB now, every later chapter becomes easier to verify.

The goal is not to memorize every GDB command. The goal is to build a repeatable inspection loop:

1. Build an ELF with useful debug information.
2. Load the ELF into GDB.
3. Stop at a named label.
4. Inspect registers, flags, memory, and disassembly.
5. Step one instruction.
6. Inspect again.
7. Explain exactly what changed.

That loop catches most assembly mistakes faster than guessing.

5.2 The Files GDB Needs

GDB works from an ELF file, not from the final Intel HEX file alone.

The HEX file contains bytes to program into flash. It is good for programming the device, but it does not carry the full symbolic view you want while debugging. The ELF file can contain:

- section addresses
- symbol names such as `main`, `sum_loop`, and `break_here`
- line information connecting source lines to machine instructions
- relocation and address information useful to `objdump`, `avr-nm`, and GDB

For this book's assembly examples, keep the ELF file and debug that:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -g3 -Wa,--gdwarf-2 \
  -c -o gdb_demo.o src/gdb_demo.S

avr-gcc -mmcu=attiny3217 -nostartfiles -g3 \
  -o gdb_demo.elf gdb_demo.o
```

The important pieces are:

Option	Purpose
<code>-mmcu=attiny3217</code>	Selects the device memory layout and instruction variant.
<code>-x assembler-with-cpp</code>	Treats the file as preprocessed assembly.
<code>-g3</code>	Keeps rich debug information in the object and ELF files.
<code>-Wa,--gdwarf-2</code>	Asks the assembler to emit DWARF line information.
<code>-nostartfiles</code>	Keeps the example's reset/vector code under your control.

Do not strip the ELF you plan to debug. Stripping removes the symbolic information that makes GDB useful.

Use `avr-objdump` beside GDB. It gives a static view of exactly what was linked:

```
avr-objdump -h -t -d -S gdb_demo.elf
```

That command shows section placement, symbols, raw instructions, and source interleaving when line information is present.

5.3 Keep Symbols That Help Debugging

GDB is much easier to use when the assembly file exposes useful names.

Prefer this:

```
.global main
.type    main, @function

main:
    ; setup

sum_loop:
    ; loop body

break_here:
    nop
```

Avoid debugging a file where every important place is an anonymous local label:

```
1:
    ; body
    brne 1b
```

Numeric local labels are fine for tight, obvious control flow, but they are poor debugger landmarks. A professional assembly file has labels at state changes, loop entries, peripheral writes, interrupt entries, error traps, and deliberate debug stops.

For functions and major routines, add `.type name, @function`. For data you intend to inspect, give it a symbol and place it in a named section:

```
.section .bss
.global sum_result
sum_result:
    .zero 1
```

Then GDB can inspect it by name:

```
(gdb) x/1xb &sum_result
```

5.4 Starting GDB

Use the AVR-targeted GDB when your toolchain provides it:

```
avr-gdb gdb_demo.elf
```

This chapter assumes `avr-gdb`. It matches the rest of the AVR GNU toolchain and loads the AVR register set and disassembler directly.

Inside GDB, confirm the file:

```
(gdb) info files
(gdb) info functions
(gdb) info variables
```

GDB startup scripts can change behavior. The local GDB tutorial included with this book notes the normal startup-script order: system init file, user init file, then the current directory's `.gdbinit`. If a debug session behaves strangely, start once with startup scripts disabled:

```
avr-gdb --nx gdb_demo.elf
```

Useful startup settings for assembly work:

```
set pagination off
set confirm off
set radix 16
set disassemble-next-line on
display/i $pc
```

The most important line is `display/i $pc`. GDB's own manual recommends it when stepping by machine instruction because it keeps the next instruction visible after every stop.

5.5 Debugging Targets Used Here

This book assumes one assembly syntax and one hardware debug board:

- source files use GNU AVR assembly accepted by `avr-as`

- the build driver is `avr-gcc`
- the debug image is the linked ELF file
- the hardware target is the ATtiny3217 Curiosity Nano and its on-board debugger

5.5.1 1. Static ELF Inspection

This does not run the program. It only inspects the linked ELF:

```
avr-gdb gdb_demo.elf
```

Useful commands:

```
(gdb) disassemble /r main
(gdb) info address sum_loop
(gdb) info files
(gdb) x/12i main
```

This mode is still valuable. Many assembly bugs are visible before execution: wrong section, unexpected instruction encoding, missing symbol, incorrect vector placement, a branch target in the wrong place, or data linked into flash when it was meant for SRAM.

5.5.2 2. ATtiny3217 Curiosity Nano Hardware

The ATtiny3217 uses UPDI, the Unified Program and Debug Interface. Microchip describes UPDI as the single-pin programming and debugging interface for this device family. The ATtiny3217 product documentation lists the device as an 8-bit AVR running up to 20 MHz, with up to 32 KB flash, 2 KB SRAM, 256 bytes of EEPROM, and a single-pin UPDI programming/debug interface.

The ATtiny3217 Curiosity Nano hardware guide documents the board's on-board debugger as the programming and debugging path for the ATtiny3217 using UPDI. That same on-board debugger also provides a virtual serial port over UART and debug GPIO. Those are the observation paths assumed in the rest of this book.

Debugger-to-target connections on the ATtiny3217 Curiosity Nano:

ATtiny3217 pin	Debugger function	Book use
PA0	DBG0, UPDI	program/debug lifeline
PB2	CDC RX, ATtiny3217 USART0 TX	serial output to host
PB3	CDC TX, ATtiny3217 USART0 RX	serial input from host
PA3	debug GPIO1	LED0 and timing/debug marks
PB7	debug GPIO0	optional timing/debug marks

Microchip documents that these debugger connections are tri-stated when the debugger is not actively using the interface. The board also exposes the signals on the edge connector, but the default assumption here is the unmodified Curiosity Nano board with the on-board debugger still connected.

Do not cut the debugger straps for the examples in this book. Microchip notes that disconnecting those straps disables programming, debugging, virtual serial port, and data streaming for the ATtiny3217 mounted on the board.

For this book, the practical rule is simple: treat PA0/UPDI as a debug lifeline. Do not use it as an ordinary GPIO pin in examples unless you also explain how the board can be recovered.

5.5.3 3. PyAvrOCD Debugging Guide

PyAvrOCD is a GDB server for 8-bit AVR targets. GDB does not talk directly to the ATtiny3217 Curiosity Nano. GDB talks to a server over a local TCP port, and the server talks

to the board's debug probe over USB. On this board the probe is the on-board nEDBG, and the MCU debug interface is UPDI.

The debug path is:

```

avr-gdb or a GDB GUI
    |
    | GDB remote serial protocol on localhost:2000
    v
pyavrocd
    |
    | USB to the Curiosity Nano nEDBG probe
    v
ATTtiny3217 over UPDI on PA0

```

This means there are three separate pieces to debug when a session fails:

- the ELF file and symbols that GDB sees
- the PyAvrOCD server process and its command-line options
- the physical/debug connection from nEDBG to the ATTtiny3217 UPDI pin

5.5.3.1 Install and Sanity Check

PyAvrOCD supports several install paths. The recommended method is `pipx`, which isolates the package in its own Python environment:

```

pipx install pyavrocd
pipx ensurepath

```

If `pipx` is not available, `pip install pyavrocd` also works. Pre-compiled binary archives are published on the GitHub releases page and can be extracted directly to `~/.local/bin/`. Any of those paths are fine as long as `pyavrocd` and `avr-gdb` are available from the shell you use for the book examples.

Check the tools first:

```

pyavrocd --version
pyavrocd --help
avr-gdb --version

```

On Linux, PyAvrOCD may need `udev` rules before a normal user can access Microchip debug probes. Download the rules file from <https://pyavrocd.io/99-edbg-debuggers.rules> and copy it to `/etc/udev/rules.d/`, then replug the board. If the board is plugged in and PyAvrOCD still reports that no compatible tool was discovered, fix host USB permissions before changing code, fuses, or wiring.

5.5.3.2 Curiosity Nano Setup

The ATTtiny3217 Curiosity Nano already has the necessary debug probe on the board. For the unmodified board:

- connect the board by USB
- leave the debugger straps intact
- leave PA0/UPDI available for programming and debugging
- do not add capacitive loads, strong resistive loads, or active circuitry to the UPDI line

External UPDI wiring is not needed for this board. PyAvrOCD can also work with external probes on other boards, but the Curiosity Nano case is simpler: USB to the board is the debug connection.

5.5.3.3 Build an ELF for Debugging

Debug the ELF file, not only the HEX file. The ELF carries the sections, symbols, line information, and addresses GDB needs.

Build the companion example from `book/ch03a_gdb_debugging`:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -g3 -Wa,--gdwarf-2 \
  -c -o gdb_demo.o src/gdb_demo.S

avr-gcc -mmcu=attiny3217 -nostartfiles -g3 \
  -o gdb_demo.elf gdb_demo.o
```

For C or mixed C/assembly projects, PyAvrOCD's documentation recommends debug-friendly compiler settings: use debug information, prefer `-Og` while debugging, and avoid `-flto` and `-mrelax`. Passing `-e gdb_demo.elf` is not just advisory: PyAvrOCD inspects the ELF and actively rejects files compiled with `-mrelax` because that option garbles line-number information. For pure assembly, the equivalent discipline is: keep named labels, keep line information, and do not let the linker rewrite jump layout while you are matching source, symbols, and addresses.

5.5.3.4 Start the PyAvrOCD Server

Run PyAvrOCD from the directory that contains `gdb_demo.elf`:

```
pyavrocd -d attiny3217 -i updi -t nedbg -p 2000 -e gdb_demo.elf
```

PyAvrOCD should connect to the probe, identify the target, start the server, and listen for a GDB connection on port 2000.

The options used here are deliberately explicit:

Option	Long form	Meaning
<code>-d attiny3217</code>	<code>--device attiny3217</code>	Selects the target MCU. This is the only mandatory option. Use <code>-d ?</code> to list all supported devices.
<code>-i updi</code>	<code>--interface updi</code>	Selects UPDI. This is the ATtiny3217 debug interface.
<code>-t nedbg</code>	<code>--tool nedbg</code>	Selects the Curiosity Nano on-board debugger. Required when multiple probes are attached.
<code>-p 2000</code>	<code>--port 2000</code>	Selects the TCP port where GDB connects. Port 2000 is the default.
<code>-e gdb_demo.elf</code>	<code>--elf-file gdb_demo.elf</code>	Lets PyAvrOCD inspect the ELF and actively reject files compiled with <code>-mrelax</code> .
<code>-v debug</code>	<code>--verbose debug</code>	Sets log verbosity. Levels: critical, error, warning, info (default), debug, all.
(none)	<code>--reboot-debugger</code>	Reboots the probe before the session. Helps when the probe appears stuck.

Option	Long form	Meaning
-a	--attach	Reconnects without resetting the target. Requires a prior session ended with <code>monitor atexit</code> stay.
-C 750	--comm-speed 750	UPDI communication speed in kbps. Default 750. Avoid values at or below 400 at 16 MHz.

Only `-d` is mandatory. This book keeps the other options visible because visible choices are easier to audit when several probes, ports, or target boards are attached.

Useful variations:

```
pyavrocd -d attiny3217 -i updi -t nedbg -p 2001 -e gdb_demo.elf
pyavrocd -d attiny3217 -i updi -t nedbg -e gdb_demo.elf --verbose debug
pyavrocd -d attiny3217 -i updi -t nedbg -e gdb_demo.elf --reboot-debugger
```

Use a different port if another server still owns port 2000. Use verbose logging when the server starts but the behavior is unclear. Rebooting the debugger can help when the host-side probe state is stale.

5.5.3.5 Connect GDB

Open a second terminal in the same directory:

```
avr-gdb --nx gdb_demo.elf
```

Then connect to PyAvrOCD:

```
set pagination off
set radix 16
set disassemble-next-line on
display/i $pc

target remote :2000
```

At this point GDB is connected to the server, but the target flash may still contain an old image. Load the ELF through PyAvrOCD:

```
load
```

The `load` command matters. It sends the ELF contents through PyAvrOCD so the target flash matches the symbols, addresses, and disassembly GDB is using. A common embedded-debugging mistake is to rebuild the ELF but forget to program the target. After that mistake, breakpoints and symbols describe one program while the chip runs another one.

Now break at a named label and run:

```
break main
break sum_loop
break break_here
continue
```

Then debug at instruction level:

```
si
info registers
```

```
x/8i $pc
x/1xb &sum_result
```

The same commands work from a GUI front end, because the GUI is still driving GDB underneath.

5.5.3.6 Optional: Start a GDB Front End

PyAvrOCD can start another program after the server is up:

```
pyavrocd -d attiny3217 -i updi -t nedbg -p 2000 -e gdb_demo.elf --start gde
```

--start gde launches Gede if it is installed and in PATH. Gede is a GDB front end; PyAvrOCD is still the GDB server. In Gede, configure the program as gdb_demo.elf, use avr-gdb as the debugger, connect to :2000, and run the same startup commands:

```
target remote :2000
load
break main
continue
```

Do not add monitor debugwire enable for the ATtiny3217 Curiosity Nano. That command is for debugWIRE targets. The ATtiny3217 uses UPDI.

5.5.3.7 Persistent Attach

The normal teaching workflow is a fresh session:

1. rebuild the ELF
2. restart PyAvrOCD
3. connect GDB
4. run load
5. set breakpoints by symbol

For bugs that appear only after the program has been running for a while, you may want to leave the target in debug mode and attach later. Before leaving the first GDB session:

```
monitor atexit stay
```

Later, restart PyAvrOCD with attach mode:

```
pyavrocd -d attiny3217 -i updi -t nedbg -p 2000 --attach
```

Then reconnect:

```
target remote :2000
```

Use attach mode deliberately. It is useful when you need it, but it is a poor default for examples because it preserves target state that a beginner may not remember creating.

5.5.3.8 Troubleshooting PyAvrOCD Sessions

If the server does not start:

- Confirm the board is attached by USB and powered.
- Confirm no other program is using the debug probe.
- On Linux, check udev rules and user permissions.
- If several probes are attached, specify --tool nedbg and, if needed, --usbsn.
- Use --verbose debug and read the first critical error.
- Try --reboot-debugger if the probe appears stuck.

If GDB cannot connect:

- Confirm PyAvrOCD is still running.

- Confirm the port number in target remote :PORT matches --port.
- Use a different port if the old one is still busy.
- Start GDB with --nx once to rule out startup-script effects.

If breakpoints do not hit:

- Run load after every rebuild.
- Use info address label and disassemble /r label to confirm the symbol.
- Break at named labels, not guessed numeric addresses.
- Avoid -mrelax while debugging.
- If a reset happens while running, set breakpoints again.
- If needed, try monitor breakpoints hardware.

If stepping or timing looks wrong:

- Remember that stopped debugging changes timing.
- Do not single-step timing-critical peripheral sequences as proof of real-time behavior.
- On UPDI targets, timers continue running while the CPU is stopped by default. Use monitor timer freeze if you need them stopped.
- Use GPIO marks, USART output, a logic analyzer, or an oscilloscope for timing evidence.

If the image looks wrong:

- Rebuild the ELF.
- Restart PyAvrOCD.
- Reconnect GDB.
- Run load.
- Use compare-sections if your GDB/target combination supports it.
- Confirm with avr-objdump -d -S gdb_demo.elf.

If GDB stops with an unexpected signal:

Signal	Meaning
SIGHUP	No OCD connection. The debug probe disconnected or was never enabled.
SIGILL	A BREAK instruction is at the current location. A software breakpoint was not restored after a previous session ended abruptly. Reflash to fix.
SIGBUS	Stack pointer is too low and threatens I/O register space.
SIGSEGV	No program is loaded. Use load, or set monitor onlywhenloaded disable if you intentionally want to run without a loaded image.

If the MCU behaves strangely after a debug session that ended abruptly (for example due to power loss), software breakpoints may not have been restored. Reflash the MCU completely to recover a known-good state.

5.5.3.9 Persistent PyAvrOCD Options

If you always use the same set of options, create a file named pyavrocd.options in the project directory and list arguments there one per line using the @ prefix notation:

```
@pyavrocd.options
```

Where pyavrocd.options contains, for example:

```
--device attiny3217
--interface updi
```

```
--tool nedbg
--port 2000
```

Then invoke PyAvrOCD with just the ELF:

```
pyavrocd @pyavrocd.options -e gdb_demo.elf
```

This keeps the command short without hiding the choices from version control.

5.5.3.10 A Repeatable Debug Script

You can put the GDB side of the session in a small command file, for example `gdb_demo.gdb`:

```
set pagination off
set radix 16
set disassemble-next-line on
display/i $pc

target remote :2000
load
break main
break sum_loop
break break_here
continue
```

Then start the server:

```
pyavrocd -d attiny3217 -i updi -t nedbg -p 2000 -e gdb_demo.elf
```

And start GDB:

```
avr-gdb --nx -x gdb_demo.gdb gdb_demo.elf
```

This makes the debug setup repeatable. If the script stops working, inspect the server output and the first failing GDB command before changing the assembly.

5.6 Core GDB Commands for Assembly

This table is the daily working set.

Command	Use
help command	Show built-in help for one command.
apropos word	Search command help by topic.
file firmware.elf	Load symbol and section information.
info files	Show loaded sections and entry information.
info functions	List known function symbols.
info variables	List known data symbols.
break label	Stop when execution reaches a label.
hbreak label	Request a hardware breakpoint when supported.
delete	Delete breakpoints.
continue	Resume execution until the next stop.

Command	Use
<code>stepi</code> or <code>si</code>	Execute one machine instruction, entering calls.
<code>nexti</code> or <code>ni</code>	Execute one machine instruction, stepping over calls when possible.
<code>finish</code>	Run until the current routine returns.
<code>disassemble /r label</code>	Show instructions and raw opcode bytes.
<code>x/8i \$pc</code>	Examine eight instructions at the program counter.
<code>info registers</code>	Show register state.
<code>p/x \$pc</code>	Print the program counter in hexadecimal.
<code>x/16xb address</code>	Examine 16 bytes of memory in hexadecimal.
<code>set \$reg = value</code>	Change a register, using the target's register names.
<code>set {unsigned char}addr = value</code>	Patch one byte in writable memory.

Register names are target-specific. The GDB manual explicitly points out that `info registers` shows the canonical names for the selected machine. On AVR, check `info registers` before assuming exactly how your GDB build names `r0`, `r1`, `SREG`, `SP`, or `PC`.

5.7 Instruction-Level Stepping

Use instruction stepping when debugging assembly:

```
(gdb) break main
(gdb) continue
(gdb) display/i $pc
(gdb) info registers
(gdb) stepi
(gdb) info registers
```

The difference between `stepi` and `nexti` matters:

Command	Behavior
<code>stepi</code> / <code>si</code>	Execute one instruction. If the instruction calls a subroutine, stop inside it.
<code>nexti</code> / <code>ni</code>	Execute one instruction, but try to step over subroutine calls.

For branch debugging, use `stepi` until you trust the flags. A conditional branch is only as correct as the flags set by the previous instruction.

Example:

```
dec    r18
brne   sum_loop
```

After `dec r18`, inspect `SREG` or the named flags your GDB exposes. The branch does not test `r18` directly; it tests the `Z` flag produced by `DEC`.

For call/return debugging, inspect the stack pointer before and after CALL, RCALL, RET, and RETI. On AVR, the return address is stored on the stack. The return-address size depends on the device's program counter width, so avoid hard-coding a stack-byte assumption across all AVR families. On the ATtiny3217, the flash size is small enough that ordinary examples use a two-byte return address.

5.8 Reading Disassembly

GDB has two common disassembly views:

```
(gdb) disassemble main
(gdb) disassemble /r main
```

The `/r` form shows raw instruction bytes as well as mnemonics. That is useful when you are checking instruction encoding, word order, or whether the linker relaxed a branch or call.

To look around the current program counter:

```
(gdb) x/10i $pc
```

To look at a label:

```
(gdb) x/12i sum_loop
```

Remember that AVR flash is word-oriented at the instruction-set level but ELF tools and GDB often present addresses in bytes. The instruction set manual gives instruction sizes in words; the debugger may display byte addresses. When an address appears to move by 2 for a single instruction, that usually means one 16-bit instruction word.

5.9 Inspecting Registers and Flags

The fastest way to debug AVR assembly is to predict register changes before stepping:

```
(gdb) info registers
(gdb) stepi
(gdb) info registers
```

For a small sequence:

```
ldi    r19, 0x11
ldi    r20, 0x22
add    r19, r20
```

You should predict:

- r19 becomes 0x33
- r20 stays 0x22
- SREG changes according to the ADD result
- PC advances by one instruction after each single-word instruction

For flag-heavy code, write down which instruction last set the flags. Common mistakes include:

- expecting LDI to set flags
- forgetting that INC, DEC, ADD, SUB, CP, CPI, LSL, and shifts affect flags
- placing a harmless-looking instruction between a compare and branch, then accidentally overwriting the flags

- treating carry as a generic error bit instead of the exact carry/borrow state produced by the last arithmetic instruction

When in doubt, single-step and read SREG.

5.10 Inspecting SRAM

Use named symbols for SRAM locations:

```
.section .bss
.global sum_result
sum_result:
.zero 1
```

Then inspect the byte:

```
(gdb) x/1xb &sum_result
```

Other useful memory formats:

```
(gdb) x/16xb &sum_result      # 16 bytes, hex, byte width
(gdb) x/8xh &sum_result       # 8 halfwords
(gdb) x/4xw &sum_result       # 4 words in GDB's display format
```

For I/O registers, use the named address from the device header or a symbol you define yourself. Be aware that reading some peripheral registers has side effects. Status registers often clear flags by writing a one, and some data register reads advance FIFOs or clear interrupt states. A debugger memory window is not always passive.

If you are using PyAvrOCD, there is also a more convenient way to inspect and modify I/O registers directly from GDB:

```
(gdb) monitor ioregister PORTA.DIR      # read one register
(gdb) monitor ioregister PORTA.DIR 0xff # write one register
(gdb) monitor ioregister PORTA.DIRSET.DIRSET0
                                     # read one bitfield
(gdb) monitor ioregister PORTA.DIRSET.DIRSET0 1
                                     # write one bitfield
(gdb) monitor ioregister DIR*          # wildcard search across peripherals
```

The general form for a register is `monitor ioregister <peripheral>.<register> [<integer>]`. The general form for a bitfield is `monitor ioregister <peripheral>.<register>.<field> [<integer>]`. The `<ioreg-expression>` is a shell-style wildcard expression over register and bitfield names, so one command can match several objects. If you omit the `<peripheral>` part, PyAvrOCD searches across peripherals for matching register or bitfield names. If the expression matches a unique register, PyAvrOCD also prints its decoded bitfields. If you provide the optional integer argument, the matched unique register or bitfield is written with that value.

This is especially important when debugging peripherals in later chapters:

- Read the datasheet before repeatedly inspecting a status or data register.
 - Know whether a flag is cleared by writing one.
 - Know whether a register is synchronized to a peripheral clock domain.
 - Know whether a register is protected by a configuration-change mechanism.
-

5.11 Inspecting Flash Data

AVR has separate program and data address spaces. Instructions live in flash. Ordinary SRAM variables live in data memory. Constants may live in flash and be read by LPM.

GDB can disassemble flash reliably because instructions are the normal execution stream:

```
(gdb) x/8i main
```

Raw flash data can be more target-dependent because different tools expose AVR program memory through different address mappings. If GDB's memory view is not obvious, confirm flash contents with `avr-objdump`:

```
avr-objdump -s -j .text gdb_demo.elf
```

For assembly examples, make flash data easy to find:

```
.section .progmem.data, "a", @progbits
.global table_bytes
table_bytes:
    .byte 0x11, 0x22, 0x33, 0x44
```

Then use both tools:

```
avr-nm -n gdb_demo.elf
avr-objdump -s -j .text gdb_demo.elf
```

The default AVR linker script places program-memory input sections into the final flash image, commonly the `.text` output section. That is why the linked ELF is inspected with `-j .text` even though the source used `.progmem.data`. GDB shows what the current target exposes. `objdump` shows what the ELF contains.

5.12 Breakpoints on Microcontrollers

Breakpoints are easy on a desktop process and less abstract on a microcontroller.

A software breakpoint usually means patching code memory with a trap instruction. On a flash-based MCU, that may require special handling because flash is not ordinary writable SRAM. A hardware breakpoint uses on-chip debug resources. Those resources are limited.

Practical rules:

- Prefer breakpoints at labels, not raw numeric addresses.
- Use `hbreak` when the target requires hardware breakpoints.
- Do not assume unlimited breakpoints on a small MCU.
- Keep a deliberate `break_here`: label near important end states.
- For unexpected interrupts, route unused vectors to a trap label such as `unexpected_interrupt: rjmp unexpected_interrupt`.
- Remove stale breakpoints when changing code layout.

On the Curiosity Nano, treat breakpoints as a scarce debug resource and keep your assembly easy to inspect statically with `avr-objdump` even when the hardware session is not attached.

5.13 Watchpoints and Data Breaks

Watchpoints stop when memory changes:

```
(gdb) watch sum_result
```

On embedded targets, watchpoint support depends on the CPU and debug hardware. If a watchpoint is implemented in hardware, there may only be a small number available. If it is implemented in software, it may slow execution heavily.

When watchpoints are limited, instrument the assembly instead:

```
store_result:
    sts    sum_result, r19
break_after_store:
    nop
```

Then break at `break_after_store`.

The label costs nothing in the final instruction stream unless you add an instruction for it. Labels are free debugging handles.

5.14 Debugging the Stack

Stack bugs are common in assembly because the assembler will not protect you from an unbalanced `PUSH`, `POP`, `CALL`, `RET`, interrupt entry, or manual stack write.

Before stepping through subroutines, initialize the stack pointer:

```
ldi    r16, lo8(RAMEND)
out     SPL, r16
ldi    r16, hi8(RAMEND)
out     SPH, r16
```

Then in GDB:

```
(gdb) info registers
(gdb) p/x $sp
(gdb) x/16xb $sp
```

Register naming can vary, so use `info registers` to confirm whether your GDB build calls the stack pointer `$sp`, shows `SPL/SPH`, or exposes both forms.

Checklist for stack debugging:

- Is `SP` initialized before the first `CALL`, `PUSH`, or interrupt?
- Does every `PUSH` have a matching `POP` on every exit path?
- Is `SREG` saved before enabling nested behavior or modifying flags in an ISR?
- Does the routine return with `RET` while an ISR returns with `RETI`?
- Did the code accidentally store ordinary data into the current stack area?

If a return jumps to nonsense, inspect the bytes at the stack pointer before the return instruction. The return address is the evidence.

5.15 Debugging Interrupts

Interrupt debugging requires more discipline than straight-line code.

Give every vector a visible target:

```
.section .vectors, "ax", @progbits
_vectors:
    jmp    main
```

```

jmp    unexpected_interrupt
jmp    tca0_ovf_isr

```

Give every ISR a visible entry and exit:

```

.global tca0_ovf_isr
.type   tca0_ovf_isr, @function

tca0_ovf_isr:
    push    r16
    in      r16, SREG
    push    r16

    ; ISR body

tca0_ovf_isr_exit:
    pop     r16
    out     SREG, r16
    pop     r16
    reti

```

Then you can use:

```

(gdb) break tca0_ovf_isr
(gdb) break tca0_ovf_isr_exit
(gdb) continue

```

At ISR entry, inspect:

- PC
- SP
- saved return address bytes
- SREG
- peripheral interrupt flag register
- interrupt enable register

At ISR exit, inspect:

- whether the peripheral flag was cleared correctly
- whether saved registers were restored
- whether SREG was restored
- whether the stack pointer returned to its entry value
- whether the final instruction is RETI, not RET

When an interrupt never fires, debug in this order:

1. Is the peripheral clocked?
2. Is the peripheral configured?
3. Is the peripheral interrupt flag set?
4. Is the peripheral interrupt enable bit set?
5. Is the global interrupt enable bit set in SREG?
6. Is the vector table at the address the CPU is using?
7. Does the vector point to the intended ISR label?

5.16 Debugging Peripheral Code

For GPIO, timers, USART, SPI, TWI, and NVM code, the debugger can mislead you if you inspect only the instruction stream. Peripheral state often changes outside the CPU pipeline.

Use a three-part method:

1. Step the instruction that writes the control register.
2. Inspect the written register if it is safe to read.
3. Confirm the external effect when possible.

For example, after writing `PORTA_DIRSET`, read back the relevant direction state. After writing `PORTA_OUTTGL`, inspect the output latch and check the pin or LED. After enabling a timer, inspect the timer's control register, then let time pass and inspect the count or interrupt flag.

For communication peripherals, combine GDB with a second observation path:

- virtual COM port for USART
- logic analyzer for SPI/TWI
- oscilloscope for timing-sensitive pins
- Microchip Data Visualizer when using supported board debug features

GDB tells you what the CPU did. External tools tell you what the circuit saw.

5.17 A Minimal Assembly Debug Example

The companion file `src/gdb_demo.S` is intentionally small. It:

- installs a reset vector
- initializes the stack pointer
- loads four flash bytes with LPM
- accumulates them into an 8-bit sum
- stores the result in SRAM
- stops at a named debug label

The flash table is deliberately placed after the halt loop in `.text`, not immediately after the reset vector. This example still installs only the reset vector because interrupts remain disabled, but the table is no longer sitting where a reader would expect interrupt-vector code.

Build it from this chapter directory:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -g3 -Wa,--gdwarf-2 \
  -c -o gdb_demo.o src/gdb_demo.S

avr-gcc -mmcu=attiny3217 -nostartfiles -g3 \
  -o gdb_demo.elf gdb_demo.o
```

Inspect it statically:

```
avr-nm -n gdb_demo.elf
avr-objdump -d -S gdb_demo.elf
```

For static inspection, start GDB:

```
avr-gdb --nx gdb_demo.elf
```

Then run these commands:

```
set pagination off
set radix 16
set disassemble-next-line on
display/i $pc

break main
break sum_loop
```

```
break break_here

info files
disassemble /r main
```

For hardware confirmation, start PyAvrOCD in one terminal:

```
pyavrocd -d attiny3217 -i updi -t nedbg -p 2000 -e gdb_demo.elf
```

Then connect from GDB in a second terminal and load the ELF into target flash:

```
target remote :2000
load
break main
break sum_loop
break break_here
continue
```

The load command programs the target through the on-board debugger. If you also need a standalone HEX file for a separate programming tool, generate it explicitly:

```
avr-objcopy -O ihex gdb_demo.elf gdb_demo.hex
```

At `sum_loop`, step one instruction at a time:

```
info registers
si
info registers
x/8i $pc
```

At `break_here`, inspect the result:

```
x/1xb &sum_result
```

The expected result is `0xaa` because:

```
0x11 + 0x22 + 0x33 + 0x44 = 0xaa
```

If the result differs, debug from evidence:

- Was Z initialized to the flash table address?
- Did LPM `r20`, Z+ load the expected byte?
- Did ADD `r19`, `r20` update the accumulator?
- Did DEC `r18` set Z only on the final iteration?
- Did BRNE `sum_loop` branch exactly three times?
- Did STS `sum_result`, `r19` write SRAM?

Do not guess. Step and inspect.

5.18 A Useful .gdbinit for AVR Assembly

For a project directory, this is a good starting point:

```
set pagination off
set confirm off
set radix 16
set disassemble-next-line on
display/i $pc

define regs
    info registers
```

```

end

define around
    x/8i $pc
end

define hook-stop
    x/i $pc
end

```

The hook-stop command runs every time execution stops. Keep it short. If it prints too much, stepping becomes noisy.

Start with `--nx` when you want to rule out startup-script effects:

```
avr-gdb --nx gdb_demo.elf
```

Start normally when you want the project helpers:

```
avr-gdb gdb_demo.elf
```

5.19 Failure Patterns and What to Inspect

Symptom	Likely area	First inspection
Program starts at the wrong code	vector table or entry address	<code>info files, x/4i 0</code>
Breakpoint never hits	wrong symbol, optimized/changed layout, wrong target image	<code>info break, info address label, compare-sections</code> if supported
Branch goes the wrong way	flags	<code>info registers</code> before and after the flag-setting instruction
Return jumps to nonsense	stack corruption	<code>p/x \$sp, x/16xb \$sp, inspect push/pop balance</code>
SRAM variable unchanged	store not reached or wrong address	<code>break before STS, inspect pointer/address</code>
LPM reads unexpected value	flash table address or address-space mapping	<code>avr-nm, avr-objdump -s, inspect Z</code>
ISR never runs	interrupt enable chain	peripheral flag, peripheral enable bit, global I bit, vector
Curiosity Nano debugger cannot connect	UPDI wiring/fuse/power	target VCC, GND, PA0/UPDI path, fuse state
Debug works once then stops	UPDI pin reused or fuse changed	check PA0 configuration and fuse writes

The point of the table is not to replace thinking. It gives you the first piece of evidence to collect.

5.20 Professional Debug Habits

Write assembly so it can be debugged:

- Use stable labels at important states.

- Keep one instruction per source line.
- Use named SRAM symbols for values you will inspect.
- Keep reset and interrupt vectors readable.
- Save and restore registers in a visible order.
- Keep deliberate trap labels for impossible paths.
- Build with debug information during development.
- Keep the ELF, map file, and objdump listing for every important test image.
- Record the debugger commands that prove a bug is fixed.

When a bug is hard, make the code more observable. Add a label. Store an intermediate value. Toggle a debug pin. Split a dense block into smaller named blocks. Assembly is not self-explaining unless you make the important states visible.

5.21 Source Notes

Facts in this chapter are based on:

- Microchip ATtiny3217 product documentation for device memory sizes and UPDI.
- Microchip ATtiny3217 Curiosity Nano Hardware User Guide sections describing the on-board debugger, UPDI programming/debugging, virtual serial port, and debug GPIO, including PA0/UPDI, PB2/PB3 CDC UART, PA3 debug GPIO1, and PB7 debug GPIO0 connections.
- Microchip ATtiny3217 Curiosity Nano Hardware User Guide sections describing debugger strap disconnection and the warning that cutting the straps disables programming, debugging, virtual serial port, and data streaming for the on-board ATtiny3217.
- PyAvrOCD documentation for its role as an AVR GDB server, support for Curiosity Nano on-board nEDBG probes, command-line options such as `--device`, `--interface`, `--tool`, and `--port`, and the `avr-gdb` remote debugging flow through `target remote`.
- The local GDB 17.2 manual for `stepi`, `nexti`, `x/i`, `display/i $pc`, `info registers`, and `hook-stop`.
- The local Red Hat GDB tutorial PDF for practical startup-script behavior, `--nx`, `help`, `apropos`, and basic debug-session setup.

Chapter 6

Instruction Set Basics

The AVR instruction set has roughly 135 instructions, but a much smaller core set does most of the real work in ordinary firmware. This chapter focuses on that core: how instructions are encoded, which register restrictions matter, how arithmetic and logic actually affect the flags, and how to read the CPU's behaviour from the instruction sequence.

The goal is not to memorise a catalog. The goal is to make the instruction set feel predictable.

This chapter uses the **AVRxt** timing column from Microchip's AVR Instruction Set Manual, because the ATtiny3217 is an AVRxt-family device. Older AVR, AVRe, and AVRxm parts are machine-code compatible for most instructions, but some cycle counts differ. When cycle timing matters, check the column for the actual CPU family instead of copying a number from a different AVR generation.

6.1 Instruction Encoding

Every AVR instruction is encoded as one or two 16-bit words (2 or 4 bytes) stored in flash. The CPU fetches one word per clock cycle. Most instructions are a single word and execute in 1-2 cycles.

That encoding detail matters because it affects both code size and speed. On AVR, the shape of an instruction is often the reason for its operand limits. If an instruction only has a few bits available to describe a register or an immediate, the hardware simply cannot support every possible combination.

The instruction set manual reports instruction sizes in **words**, not bytes. One word is 16 bits. So "Words: 1" means 2 bytes in flash, and "Words: 2" means 4 bytes in flash. That same word-oriented view shows up in branch offsets and return addresses.

6.1.1 Single-Word Instructions (16-bit)

The vast majority. The 16 bits encode the opcode, destination register, and source register or small immediate.

LDI Rd, K – Load Immediate

Encoding: 1110 KKKK dddd KKKK

$\begin{array}{c} \text{1110} \quad \text{KKKK} \quad \text{dddd} \quad \text{KKKK} \\ \text{└───┐} \quad \text{└───┐} \\ \text{opcode} \quad \text{d = Rd - 16 (only r16-r31 → d = 0-15)} \\ \text{K = 8-bit immediate split across bits} \end{array}$

ldi r20, 0xAB:

d = 20 - 16 = 4 = 0b0100

```

K = 0xAB = 0b10101011
→ 1110 1010 0100 1011 = 0xEA4B
ADD Rd, Rr  – Add without Carry
Encoding:  0000 11rd dddd rrrr
           d = Rd (0–31), r = Rr (0–31)

```

```

add r16, r17:
d = 16 = 0b10000, r = 17 = 0b10001
→ 0000 1101 0000 0001 = 0x0D01

```

6.1.2 Two-Word Instructions (32-bit)

These need a full 16-bit address or program address that doesn't fit in the first word:

Instruction	Why 32-bit
LDS Rd, k	Data address k is 16 bits — needs second word
STS k, Rr	Same
JMP k	Program address k is 22 bits — needs second word
CALL k	Same

```

STS k, Rr – Store to SRAM
Word 1: 1001 001r rrrr 0000  (r = Rr, 5 bits)
Word 2: kkkk kkkk kkkk kkkk  (16-bit address k)

```

```

sts PORTB_DIRSET, r16:
PORTB_DIRSET = 0x0421
Word 1: 0x9200 | (16 << 4) = 0x9300
Word 2: 0x0421
→ 4 bytes total in flash

```

This is why STS costs 2 cycles and takes twice the flash of OUT. Always use OUT when the I/O address is $\leq 0x3F$.

This is a recurring AVR theme: the shorter instruction is usually faster, but only if the target address fits the shorter encoding.

The same size difference appears in disassembly. OUT 0x3d, r16 is a single 16-bit word. STS 0x0401, r16 is two words: one word for the opcode/register field and one word for the 16-bit data-space address. The programmer sees one source line either way, but flash sees different byte counts.

6.1.3 Cycle Counts at a Glance

Category	Typical cycles	Notes
Register ops (ADD, AND, MOV...)	1	Pipeline: fetch overlaps
Load indirect (LD Rd, X/Y/Z and $\pm/+q$ variants)	2	Internal SRAM
Load direct (LDS Rd, k)	3	32-bit instruction
Store indirect (ST X/Y/Z, Rr and $\pm/+q$ variants)	1	Internal SRAM
Store direct (STS k, Rr)	2	32-bit instruction
IN / OUT	1	Low I/O space only
Taken branch	2	Pipeline flush
Not-taken branch	1	No flush
RCALL	2	16-bit PC devices such as ATtiny3217
RET	4	Stack pop + control-flow change
CALL	3	32-bit instruction, 16-bit PC
JMP	3	32-bit instruction, no stack use

Category	Typical cycles	Notes
MUL	2	Result in r1:r0
NOP	1	Explicit no-op

The table assumes ordinary internal SRAM/data accesses. Microchip notes that data-memory cycle counts for LD, LDS, ST, and STS assume internal RAM; accesses to non-volatile memory through the data space can take extra time depending on the NVMCTRL implementation. That matters later when reading mapped flash on AVRxt with ordinary LD.

Note on MUL and r1. The “Result in r1:r0” above is the hardware behaviour: MUL (and MULS/MULSU) always write the 16-bit product to r1:r0, unconditionally. This collides with a software convention used throughout the rest of this book — and by `avr-gcc` — that r1 is the *zero register* and holds 0 at all times. MUL is the one common instruction that breaks that promise, so code following the convention must copy the product out and run `clr r1` afterwards. The convention and the required fix-up are introduced with multi-byte arithmetic (Chapter 10) and revisited in the bit-manipulation (Chapter 26) and ADC (Chapter 30) chapters; for now, just note that r1 is not safe to assume zero immediately after a multiply.

6.2 Instruction Categories

AVR Instruction Categories	
Data Transfer	MOV MOVW LDI LD ST LDS STS LPM PUSH POP IN OUT
Arithmetic	ADD ADC SUB SBC SUBI SBCI ADIW SBIW INC DEC NEG MUL MULS MULSU
Logic	AND ANDI OR ORI EOR COM TST CLR SER
Shift / Rotate	LSL LSR ROL ROR ASR SWAP
Bit Operations	SBI CBI SBIC SBIS SBRC SBRS BST BLD SEC CLC SEI CLI BSET BCLR
Branch / Jump	RJMP JMP IJMP RCALL CALL ICALL RET RETI BREX BRNE BRCS BRCC BRSH BRLO BRMI BRPL BRGE BRLT BRVS BRVC BRIE BRID CPSE
MCU Control	NOP SLEEP WDR BREAK

Later chapters cover Load/Store (Chapter 7), Arithmetic (Chapter 13), Branches (Chapter 14), and Subroutines (Chapter 15) in depth. This chapter covers the core register-to-register operations that form the building blocks of everything else.

So think of this chapter as the grammar of AVR instructions. Later chapters combine these pieces into bigger programming patterns.

6.3 Data Transfer — Moving Values Around

6.3.1 MOV — Copy Register

MOV Rd, Rr ; Rd ← Rr

Copies the 8-bit value from Rr to Rd. Source unchanged. No flags affected. Works on any register r0–r31.

MOV is boring in the best possible way. It does exactly what it says, and because it does not touch flags, it is safe to use in the middle of code that depends on the current condition state.

```
ldi r16, 42
mov r0, r16 ; r0 = 42, r16 still = 42
mov r5, r0 ; r5 = 42 (MOV works on r0–r15 unlike LDI)
```

1 cycle. Does not affect SREG.

6.3.2 MOVW — Copy Register Pair

MOVW Rd, Rr ; Rd+1:Rd ← Rr+1:Rr

Copies two registers in one instruction. Rd and Rr must be even-numbered: r0, r2, r4, ... r30.

```
; Copy X pointer (r27:r26) into r1:r0
movw r0, r26 ; r1:r0 = r27:r26

; Copy Z into Y for a secondary pointer
movw r28, r30 ; r29:r28 = r31:r30 (Y = Z)
```

1 cycle. Does not affect SREG. Saves one instruction vs two MOVs.

Whenever you are copying a pointer pair or a 16-bit value, MOVW is worth remembering because it is both shorter and clearer than two separate moves.

Because MOVW uses even register numbers, movw r24, r22 copies r23:r22 into r25:r24. Think “low register named, high register implied.” This is the same low-byte-first convention used by pointer pairs X (r27:r26), Y (r29:r28), and Z (r31:r30).

6.3.3 LDI — Load Immediate

LDI Rd, K ; Rd ← K (K = 0..255)

Load a constant into a register. **r16–r31 only.**

```
ldi r16, 0xFF ; OK
ldi r20, (1 << 4) ; OK – expression evaluated at assembly time
ldi r0, 42 ; ERROR: r0 not allowed
```

1 cycle. Does not affect SREG.

LDI has an 8-bit immediate (K = 0..255) and a 4-bit encoded destination field. That 4-bit field is why only the upper half of the register file is legal: the encoded value is Rd - 16.

6.3.4 CLR and SER — Clear and Set All Bits

These are **pseudo-instructions** (GAS aliases, not separate opcodes):

```
clr r16 ; r16 = 0x00 – assembled as EOR r16, r16
ser r16 ; r16 = 0xFF – assembled as LDI r16, 0xFF (r16–r31 only)
```


CLR works on any register (r0–r31) because it uses EOR. SER only works on r16–r31 because it uses LDI.

This is a good example of how AVR pseudo-instructions work. They look like real instructions in source, but the assembler is translating them into more basic operations for you.

CLR sets Z=1, clears N, V, S. SER affects no flags.

Pseudo-instructions are useful, but do not forget their real flag behavior. CLR is not flag-neutral because it is really EOR Rd,Rd. SER is flag-neutral because it is really LDI Rd,0xFF.

6.4 Arithmetic Instructions

6.4.1 ADD — Add Without Carry

ADD Rd, Rr ; $Rd \leftarrow Rd + Rr$

Adds two registers, stores result in Rd. Affects: H, S, V, N, Z, C.

If you are coming from a higher-level language, read that carefully: the result replaces the original destination register. AVR arithmetic is almost always destructive to the destination operand.

```
ldi r16, 100
ldi r17, 50
add r16, r17 ; r16 = 150, C=0, Z=0, N=0
```

Unsigned overflow (result > 255): C=1, result wraps.

```
ldi r16, 200
ldi r17, 100
add r16, r17 ; 200+100=300 → r16=44, C=1
```

For signed arithmetic, the carry flag is not the overflow test. Signed overflow is reported in V, and signed branches use S (N XOR V). For example, $0x7F + 0x01 = 0x80$ sets V because +127 became -128 in 8-bit two's complement, even though that operation is also just an ordinary 8-bit wraparound.

6.4.2 ADC — Add with Carry

ADC Rd, Rr ; $Rd \leftarrow Rd + Rr + C$

Identical to ADD but adds the carry flag too. Essential for **multi-byte arithmetic** — carry from the low byte automatically propagates into the high byte:

```
; 16-bit add: r17:r16 += r19:r18
add r16, r18 ; low bytes first – may set C
adc r17, r19 ; high bytes – C from previous ADD is included

; 24-bit add: r2:r1:r0 += r5:r4:r3
add r0, r3
adc r1, r4
adc r2, r5 ; each ADC picks up carry from the byte below
```

This low-byte-first pattern is one of the most important habits in AVR assembly. Once it becomes automatic, larger integer arithmetic stops feeling special.

Microchip's manual defines ADC as $Rd \leftarrow Rd + Rr + C$; the carry input is the current C bit before the instruction executes, and the output C bit is then ready for the next byte. That is why no separate "carry variable" exists in multi-byte arithmetic.

6.4.3 SUB — Subtract Without Carry

SUB Rd, Rr ; $Rd \leftarrow Rd - Rr$

Subtracts Rr from Rd. C=1 means borrow (unsigned underflow).

That “C means borrow” convention is easy to trip over because it feels backwards if you learned carry only from addition. On AVR subtraction, carry set means the subtraction had to borrow.

```
ldi r16, 10
ldi r17, 20
sub r16, r17 ; 10-20 = -10 → r16=246 (0xF6), C=1 (borrow)
```

The same instruction can be read three ways:

Interpretation	Example result for 10 - 20
Raw 8-bit bits	0xF6
Unsigned value	246, with C=1 showing borrow
Signed value	-10, with N=1 and S depending on V

The CPU only produces bits and flags. Your later branch instruction decides which interpretation matters.

6.4.4 SBC — Subtract with Carry (Borrow)

SBC Rd, Rr ; $Rd \leftarrow Rd - Rr - C$

Like ADC for addition, SBC propagates borrow in multi-byte subtraction:

```
; 16-bit subtract: r17:r16 -= r19:r18
sub r16, r18 ; low bytes
sbc r17, r19 ; high bytes — includes borrow from low byte
```

Again, the pattern is the same as multi-byte addition: low byte first, then work upward using the carry/borrow chain.

6.4.5 SUBI — Subtract Immediate

SUBI Rd, K ; $Rd \leftarrow Rd - K$ (r16–r31 only)

Subtract a constant without needing a second register. **No ADDI exists** — use SUBI with the negated value, or ADIW/SBIW for 16-bit register pairs.

That missing ADDI surprises many beginners. It is not an omission in the book; it is a real feature of the AVR ISA.

```
ldi r16, 50
subi r16, 7 ; r16 = 43
subi r16, -3 ; r16 = 46 (subtract -3 = add 3)
```

SUBI r16, -3 works because the assembler encodes -3 as the 8-bit constant 0xFD. Subtracting 0xFD from an 8-bit register is the same low-byte result as adding 3. This idiom is common, but use it deliberately; it can be less readable than ADIW when you really mean “add to a 16-bit pointer.”

6.4.6 SBCI — Subtract Immediate with Carry

SBCI Rd, K ; $Rd \leftarrow Rd - K - C$ (r16–r31 only)

Pair with SUBI for 16-bit immediate subtraction:

```
; r17:r16 -= 300 (0x012C)
subi r16, lo8(300) ; r16 -= 0x2C
sbci r17, hi8(300) ; r17 -= 0x01 plus any borrow from low byte
```

6.4.7 ADIW — Add Immediate to Word

ADIW Rd, K ; Rd+1:Rd ← Rd+1:Rd + K (K = 0..63)

Adds a small constant to a 16-bit register pair in one instruction. Only works on r25:r24, r27:r26 (X), r29:r28 (Y), r31:r30 (Z).

```
adiw r24, 1 ; r25:r24 += 1
adiw XL, 4 ; X pointer += 4 (advance 4 bytes)
adiw ZL, 16 ; Z pointer += 16
```

2 cycles. Affects C, Z, N, V, S.

ADIW is especially nice for pointer arithmetic, because X, Y, and Z all live in the supported register-pair set.

The immediate is only 6 bits, so K is limited to 0..63. Larger pointer steps need repeated ADIW, register arithmetic, or explicit low/high-byte addition.

6.4.8 SBIW — Subtract Immediate from Word

SBIW Rd, K ; Rd+1:Rd ← Rd+1:Rd - K (K = 0..63)

Same register restrictions as ADIW. Used in delay loops (Chapter 1) and pointer adjustment.

```
sbiw r24, 1 ; r25:r24 -= 1 — sets Z when result = 0
sbiw ZL, 8 ; Z -= 8
```

6.4.9 INC and DEC — Increment / Decrement

INC Rd ; Rd ← Rd + 1
DEC Rd ; Rd ← Rd - 1

Works on r0–r31. **Critical gotcha: INC and DEC do not affect the carry flag (C).** This catches people trying to use DEC as part of a multi-byte borrow chain.

```
; This loop is WRONG for multi-byte work:
ldi r16, 0
ldi r17, 0
dec r16 ; r16 = 255, C unchanged — cannot chain with SBC!

; Correct 16-bit decrement on a supported pair: use SBIW
sbiw r24, 1 ; r25:r24 -= 1, sets Z correctly
```

Flags affected by INC/DEC: S, V, N, Z — but NOT C.

That is why INC and DEC are great for single-byte counters and a bad fit for arithmetic that needs carry propagation.

Microchip calls out another subtlety: for unsigned values after INC or DEC, only BREQ and BRNE are generally reliable tests. Use them for “did the counter wrap/reach zero?” tests. Do not use BRL0/BRSH after INC or DEC and expect a meaningful unsigned less-than/greater-than result, because C was not updated.

6.4.10 NEG — Two's Complement Negate

NEG Rd ; Rd $\leftarrow$ 0x00 - Rd

Negates the register (flip all bits, then add 1). NEG 0 $\rightarrow$ 0 with C=0. All other values: C=1.

```
ldi r16, 10
neg r16 ; r16 = 246 (0xF6), which is -10 in two's complement
neg r16 ; r16 = 10 again
```

Useful for: converting subtraction direction, computing absolute value.

Whenever you see NEG, think “arithmetic sign change.” Whenever you see COM, think “bit-wise inversion.” They are related, but not interchangeable.

There is one unavoidable two's-complement corner case: NEG 0x80 leaves the bit pattern 0x80. In signed 8-bit arithmetic, -128 has no positive counterpart representable in 8 bits. The V flag reports that overflow case.

6.5 Logic Instructions

6.5.1 AND / ANDI — Bitwise AND

AND Rd, Rr ; Rd $\leftarrow$ Rd AND Rr (r0–r31)
ANDI Rd, K ; Rd $\leftarrow$ Rd AND K (r16–r31 only)

Use AND to **clear** (mask off) specific bits:

```
ldi r16, 0xAB ; 0b10101011
andi r16, 0x0F ; r16 = 0x0B (0b00001011) – upper nibble cleared
andi r16, 0b11111110 ; clear bit 0
```

Affects: S, V(=0), N, Z. Clears V.

Bit masking is one of the most common operations in embedded work. In practice: AND clears bits, OR sets bits, and EOR toggles bits.

ANDI is restricted to r16–r31, but AND works on all 32 registers. If your value is in r0–r15, either use a mask register with AND, or move the value to an upper register before using ANDI.

6.5.2 OR / ORI — Bitwise OR

OR Rd, Rr ; Rd $\leftarrow$ Rd OR Rr
ORI Rd, K ; Rd $\leftarrow$ Rd OR K (r16–r31 only)

Use OR to **set** specific bits:

```
ldi r16, 0x2A
ori r16, 0x80 ; set bit 7  $\rightarrow$  r16 = 0xAA
ori r16, (1 << 3) ; set bit 3
```

ORI has the same upper-register restriction as LDI and ANDI. The opcode has room for an 8-bit immediate and only the reduced destination encoding.

6.5.3 EOR — Bitwise XOR (Exclusive OR)

EOR Rd, Rr ; Rd $\leftarrow$ Rd XOR Rr

Use EOR to **toggle** bits or **invert** a value:

```
ldi r16, 0xFF
ldi r17, 0x0F
eor r16, r17      ; r16 = 0xF0 – lower nibble toggled
eor r16, r16      ; r16 = 0x00 – self-XOR always gives 0 (this is CLR)
```

6.5.4 COM — One's Complement (Bitwise NOT)

COM Rd ; Rd $\leftarrow$ 0xFF - Rd (flip every bit)

```
ldi r16, 0b10101010
com r16      ; r16 = 0b01010101
```

Always sets C=1. Note: COM $\neq$ NEG. COM flips bits; NEG negates (COM + 1).

Because COM always sets C, avoid placing it between the low and high bytes of an ADD/ADC or SUB/SBC carry chain unless changing C is exactly what you intend.

6.5.5 TST — Test Register (Non-Destructive)

TST Rd ; Rd AND Rd $\rightarrow$ set N, Z flags; Rd unchanged

Tests whether a register is zero or negative without modifying it:

```
lds r16, some_var
tst r16
breq var_is_zero      ; Z=1 if r16 was 0
brmi var_is_neg       ; N=1 if bit 7 was set (negative signed)
```

Assembled as AND Rd, Rd.

TST is one of those instructions that mostly exists to make your intent more obvious to a human reader.

Since TST is really AND Rd,Rd, it clears V and sets S from N XOR V. After `tst r16`, BRMI tests bit 7 directly because V is zero.

6.6 Comparison Instructions

These set flags **without storing any result**. Always followed by a conditional branch.

This is the AVR version of “ask a question, then branch on the answer.” The question is encoded in the compare; the answer appears in the flags.

6.6.1 CP — Compare

CP Rd, Rr ; Rd - Rr $\rightarrow$ flags only, result discarded

Exactly like SUB but throws away the result. Use before BREQ, BRNE, BRLO, BRGE, etc.

That makes CP conceptually simple: it performs a subtraction only for the purpose of updating flags.

```
ldi r16, 42
ldi r17, 100
cp r16, r17      ; flags set as if 42 - 100
brlo r16_smaller ; BRLO branches if C=1 (unsigned: Rd < Rr)
```

6.6.2 CPC — Compare with Carry

CPC Rd, Rr ; Rd - Rr - C → flags only

For multi-byte comparison (after comparing low bytes with CP):

```
; Compare r17:r16 with r19:r18 (16-bit unsigned)
cp r16, r18 ; compare low bytes
cpc r17, r19 ; compare high bytes including borrow
brlo less_than ; taken if r17:r16 < r19:r18 unsigned
```

6.6.3 CPI — Compare with Immediate

CPI Rd, K ; Rd - K → flags only (r16–r31 only)

```
ldi r16, 10
cpi r16, 10
breq exactly_ten ; taken
```

6.6.4 CPSE — Compare, Skip if Equal

CPSE Rd, Rr ; if Rd == Rr: skip next instruction

Skips exactly one instruction when equal. Used for compact conditional code:

```
cpse r16, r17
rjmp not_equal ; skipped if r16 == r17
; falls through here if equal
```

CPSE does **not** update SREG. It simply compares and adjusts the program counter if the registers are equal. It also has variable timing: 1 cycle if no skip happens, 2 cycles if it skips a one-word instruction, and 3 cycles if it skips a two-word instruction.

That makes CPSE compact, but not always ideal for cycle-stable code:

```
cpse r16, r17
sts PORTA_OUTTGL, r18 ; two-word instruction: skip path is longer
```

If the next instruction might be two words (LDS, STS, JMP, CALL), count the skip path carefully.

6.7 Conditional Branches — Full Table

Every branch instruction tests one or more SREG flags. The mnemonic tells you what condition causes the branch.

Once you stop reading branch mnemonics as magic words and start reading them as “test this flag pattern,” control flow becomes much easier to reason about.

Mnemonic	Full name	Condition	Flag(s)
BREQ	Branch if Equal	Z = 1	Z
BRNE	Branch if Not Equal	Z = 0	Z
BRCS	Branch if Carry Set	C = 1	C
BRCC	Branch if Carry Cleared	C = 0	C
BRSH	Branch if Same or Higher	C = 0	C ← unsigned ≥
BRLO	Branch if Lower	C = 1	C ← unsigned <
BRMI	Branch if Minus	N = 1	N

BRPL	Branch if Plus	N = 0	N
BRGE	Branch if $\geq$ (signed)	S = 0	$N \oplus V \leftarrow \text{signed} \geq$
BRLT	Branch if $<$ (signed)	S = 1	$N \oplus V \leftarrow \text{signed} <$
BRVS	Branch if Overflow Set	V = 1	V
BRVC	Branch if Overflow Clear	V = 0	V
BRIE	Branch if Interrupts En.	I = 1	I
BRID	Branch if Interrupts Dis.	I = 0	I
BRHS	Branch if Half-carry Set	H = 1	H
BRHC	Branch if Half-carry Clr	H = 0	H
BRTS	Branch if T Set	T = 1	T
BRTC	Branch if T Clear	T = 0	T

All branch instructions: **-64 to +63 words** offset relative to the next instruction. In address terms, the destination must be within PC - 63 through PC + 64 after the branch instruction's normal PC + 1 step. If the target is farther, use JMP or RJMP with an inverted condition:

```
; Branch to far_label if r16 == r17 - but far_label > 63 words away:
cp    r16, r17
brne skip           ; if NOT equal, skip the jump
jmp   far_label
skip:
```

This branch-range limit is another place where instruction encoding leaks into everyday programming. The branch is short because the displacement field is short.

Conditional branches do not modify SREG. They only read the flags left by an earlier instruction. Any instruction inserted between the compare and the branch must be checked for flag effects:

```
cp    r16, r17
mov   r18, r16      ; OK: MOV does not affect flags
breq  equal

cp    r16, r17
andi  r18, 0x0F      ; BAD if you meant to branch on CP: ANDI changes Z/N/V/S
breq  equal
```

6.8 Unsigned vs Signed Comparisons

This trips up almost every beginner. Unsigned and signed comparisons use different branch instructions after the same CP.

```
ldi r16, 0xF0      ; = 240 unsigned, or -16 signed
ldi r17, 0x10      ; = 16 unsigned, or 16 signed

cp    r16, r17      ; 0xF0 - 0x10 = 0xE0. C=0 (no borrow), N=1, V=0, S=1

brlo  below_u       ; C=1? NO → not taken. Correct: 240 > 16 unsigned.
brlt  below_s       ; S=1? YES → taken. Correct: -16 < 16 signed.
```

Rule: **BRLO/BRSH for unsigned. BRLT/BRGE for signed.**

If you take only one rule from this section, take that one. The CPU does not know whether you intended a byte to mean 240 or -16. Your branch choice tells it how to interpret the

flags.

Microchip’s branch summary says the common tests ($Rd < Rr$, $Rd \geq Rr$, and so on) are meaningful immediately after compare/subtract-style instructions such as CP, CPI, SUB, and SUBI. If the preceding instruction was a logical operation, BRL0 still means “C=1”, but it no longer means “less than” unless that flag pattern was intentionally prepared.

6.9 Flag Effects Summary

The table below covers every instruction in this chapter. “—” means the flag is not affected.

Instruction	C	Z	N	V	S	H	
ADD	●	●	●	●	●	●	
ADC	●	●	●	●	●	●	
SUB	●	●	●	●	●	●	
SBC	●	●	●	●	●	●	
SUBI	●	●	●	●	●	●	
SBCI	●	●	●	●	●	●	
ADIW	●	●	●	●	●	—	
SBIW	●	●	●	●	●	—	
INC	—	●	●	●	●	—	← C NOT affected
DEC	—	●	●	●	●	—	← C NOT affected
NEG	●	●	●	●	●	●	
AND / ANDI	—	●	●	0	●	—	← V cleared, C unchanged
OR / ORI	—	●	●	0	●	—	← V cleared, C unchanged
EOR	—	●	●	0	●	—	← V cleared, C unchanged
COM	1	●	●	0	●	—	← C always 1, V cleared
TST	—	●	●	0	●	—	← same flags as AND Rd,Rd
CP / CPI	●	●	●	●	●	●	
CPC	●	●	●	●	●	●	
MOV/MOVW/LDI	—	—	—	—	—	—	
CLR	—	1	0	0	0	—	← same flags as EOR Rd,Rd

The full SREG also contains I, T, H, S, V, N, Z, and C. This table focuses on the arithmetic/logic flags used in this chapter. SEI, CLI, BSET, BCLR, BST, and BLD manipulate the other status bits and are handled more deeply in the interrupt and bit-manipulation chapters.

6.10 Worked Example: Compare, Clamp, Negate

Goal: read an 8-bit value, clamp it to 0–100 range, then negate it if a flag register bit is set.

This example matters because it combines several ideas you will use constantly: compare, branch, clamp by assignment, test a control bit, and optionally apply an arithmetic transform.

```
/*
 * clamp_and_negate:
 *   Input:  r24 = raw value (unsigned), r25 = flags (bit 0 = negate)
 *   Output: r24 = clamped value, optionally negated
 *   Clobbers: r16
 */
clamp_and_negate:
    /* Step 1: clamp to 100 max */
```



```

    cpi    r24, 100          ; compare r24 with 100
    brlo   clamped          ; if r24 < 100 (unsigned), skip clamp
    ldi     r24, 100         ; else clamp to 100
clamped:

/* Step 2: clamp to 0 min – already unsigned so r24 >= 0 always.
 * If the raw value were signed we'd add BRGE check here. */

/* Step 3: negate if bit 0 of r25 is set */
sbrs     r25, 0             ; Skip if Bit in Register Set (bit 0 of r25)
ret      ; bit 0 clear – return as-is
neg      r24                ; negate: r24 = 0 - r24
ret

```

New instructions introduced:

SBRS Rr, b Skip next instruction if bit b of Rr is set. 1 cycle (not skipped), 2 cycles (skipped over 1-word instr), 3 cycles (skipped over 2-word instr). Does not affect SREG.

The skip family all follows the same timing idea on AVRxt:

Instruction	Test location	Skips when
SBRC Rr,b	Register bit	bit is clear
SBRs Rr,b	Register bit	bit is set
SBIC A,b	Low I/O bit 0x00-0x1F	bit is clear
SBIS A,b	Low I/O bit 0x00-0x1F	bit is set
CPSE Rd,Rr	Register compare	registers equal

All skip exactly one following instruction, not a block. If you need to skip multiple operations, skip over a branch:

```

sbrs r25, 0
rjmp no_negate
neg r24
no_negate:

```

Trace through the example:

Call: r24=150, r25=0b00000001 (negate flag set)

```

cpi r24, 100    → 150-100=50, C=0, Z=0
brlo clamped    → C=0: NOT taken (150 >= 100, no branch)
ldi r24, 100    → r24 = 100
clamped:
sbrs r25, 0     → bit 0 of r25 is 1: SKIP next instr
                  (ret is skipped)
neg r24         → r24 = 0 - 100 = -100 (0x9C = -100 signed)
ret

```

Walking through examples like this is how you build intuition. Do not just read the final code; read the flags and the decision points as they evolve.

6.11 Operand Restrictions Cheat-Sheet

This is worth turning into instinct:

Instruction	Rd range	Rr range	Immediate range
MOV	r0–r31	r0–r31	–
MOVW	r0,r2..r30	r0,r2..r30	– (even only)
LDI	r16–r31	–	0–255
CLR	r0–r31	–	–
SER	r16–r31	–	–
ADD/ADC	r0–r31	r0–r31	–
SUB/SBC	r0–r31	r0–r31	–
SUBI/SBCI	r16–r31	–	0–255
ADIW/SBIW	r24,X,Y,Z	–	0–63
INC/DEC	r0–r31	–	–
NEG/COM	r0–r31	–	–
AND/OR/EOR	r0–r31	r0–r31	–
ANDI/ORI	r16–r31	–	0–255
CP	r0–r31	r0–r31	–
CPI	r16–r31	–	0–255
CPC	r0–r31	r0–r31	–
CPSE	r0–r31	r0–r31	–
SBRs/SBRC	r0–r31	–	bit 0–7

6.12 Summary

Instructions: 1 or 2 words (16 or 32 bits).

Two-word: LDS/STS carry a 16-bit data address; JMP/CALL carry a larger program address field.

AVRxt timing in this chapter: LDS=3 cycles, LD=2 cycles, STS=2 cycles, ST=1 cycle for internal SRAM; mapped NVM can add cycles.

Register rules:

LDI, SUBI, SBCI, ANDI, ORI, SER, CPI → r16–r31 only.

ADIW, SBIW → r24, X(r26), Y(r28), Z(r30) only.

Everything else → r0–r31.

Carry chains: ADD+ADC, SUB+SBC, SUBI+SBCI – always low byte first.

INC/DEC do NOT touch C – do not use them to propagate multi-byte carry/borrow.

NEG ≠ COM: NEG = two's complement, COM = bitwise invert.

Comparisons:

Unsigned: BRL0 (<), BRSH (≥), BRCS/BRCC

Signed: BRLT (<), BRGE (≥)

Sign bit: BRMI (N=1), BRPL (N=0)

Branch range: -64..+63 word offset. Use JMP/RJMP patterns for farther targets.

Skip instructions skip one instruction only and cost 1/2/3 cycles depending on whether no skip, one-word skip, or two-word skip occurs.

6.13 Exercises

1. Add two 24-bit numbers stored in r2:r1:r0 and r5:r4:r3. Store the result in r2:r1:r0. What happens to the carry if the result exceeds 24 bits?
2. Write a sequence to compute $r16 = r16 * 4$ without using MUL. (Hint: shifting left by 1 is the same as multiplying by 2.) Use only ADD instructions.

3. Write a loop that counts from 0 to 255 in r16, then wraps and repeats forever. Use INC and BREQ. Why is BREQ the right branch here?
 4. Given r16 = 0xAB and r17 = 0x3C, predict the result and all six flag states (C, Z, N, V, S, H) after each operation — before assembling. For instructions that do not affect C or H, write “unchanged” for that flag:
 - ADD r16, r17
 - SUB r16, r17 (fresh r16=0xAB each time)
 - AND r16, r17
 - EOR r16, r17
 5. Write clamp_and_negate to also handle the case where r24 starts as 0 and the negate flag is set. What does NEG 0 produce and is that correct for your use case?
 6. Why does SUBI r16, -1 add 1 to r16? What is the bit pattern of -1 as an 8-bit two’s complement number, and what does subtracting it do?
-

Next: Chapter 7 — Load and Store

Chapter 7

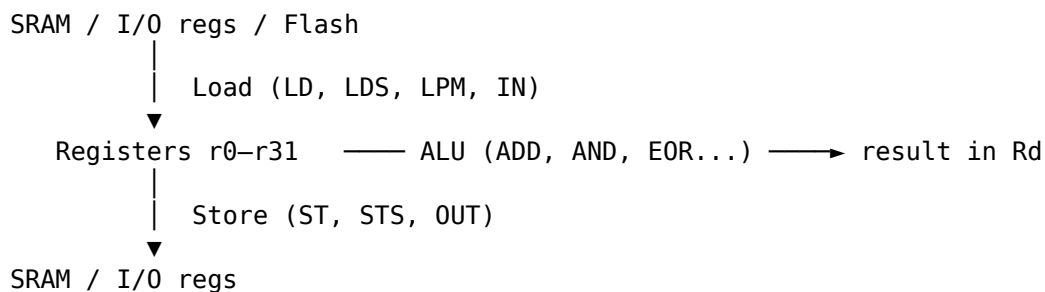
Load and Store

Every useful program moves data. In AVR assembly, **every data movement is explicit**: you choose the exact instruction that moves each byte, the exact addressing mode, and often the exact pointer update behaviour. This chapter covers the main ways to get data into registers, get it back out to memory, and read constants from flash on the ATtiny3217.

7.1 The Memory-Register Model

The AVR CPU can only operate on data that is in **registers** (r0-r31). To work on a byte in SRAM, you must load it into a register first, operate, then store it back. There is no “operate directly on memory” instruction.

That rule is so central that it is worth repeating in plain language: if the value is not in a register yet, the ALU cannot touch it.



Every instruction in this chapter is either a load (memory → register) or a store (register → memory).

Once you see that pattern, a lot of AVR code becomes easier to read. A routine is often just alternating phases: bring bytes into registers, do work there, write results back out.

7.1.1 ATtiny3217 Data Space Is Not Just SRAM

The load/store instructions use the AVR **data address space**, not just the 2 KB internal SRAM block. On ATtiny3217, Microchip’s memory map puts several different resources into that 16-bit address space:

Data address range	Resource
0x0000-0x003F	64 I/O registers

Data address range	Resource
0x0040-0x0FFF	Extended I/O/peripheral range; Microchip documents 960 implemented Ext. I/O registers plus reserved space
0x1000-0x13FF	NVM I/O registers and data
0x1400-0x14FF	EEPROM, 256 bytes
0x3800-0x3FFF	Internal SRAM, 2 KB
0x8000-0xFFFF	Flash mapped into data space, 32 KB

That is why the same LD instruction can read SRAM, an I/O register, EEPROM space, or mapped flash. The address decides which hardware block answers.

For normal variables, the assembler and linker place `.bss` and `.data` in SRAM at 0x3800-0x3FFF. Do not treat EEPROM or flash as scratch SRAM: they are nonvolatile memory regions controlled by NVMCTRL and have different write rules and timing.

7.2 Addressing Modes

The AVR supports five ways to specify a memory address:

Mode	Syntax	Address source	Instruction
Direct	LDS Rd, k	16-bit constant k in instruction	LDS / STS
Indirect	LD Rd, X	Value in X, Y, or Z pointer pair	LD / ST
Post-increment	LD Rd, X+	X (used), then X += 1	LD / ST
Pre-decrement	LD Rd, -X	X -= 1, then X (used)	LD / ST
Indirect + displacement	LDD Rd, Y+q	Y + constant q (0-63)	LDD / STD

These are not five completely different ideas. They are variations on one theme: where does the address come from, and should the pointer change as a side effect?

7.3 Direct Addressing: LDS and STS

```
LDS Rd, k          ; Rd ← SRAM[k]      (k = 16-bit address)
STS k, Rr          ; SRAM[k] ← Rr
```

The address is encoded directly in the instruction as a second 16-bit word. Both are **32-bit (two-word) instructions**.

That makes LDS/STS very convenient to write, but relatively expensive in both flash space and cycles.

```
.section .bss
my_var: .skip 1          ; reserve one byte in SRAM

.section .text
ldi r16, 0xAB
sts my_var, r16          ; write 0xAB to my_var's SRAM address
lds r17, my_var          ; r17 = 0xAB
```

Cycle costs:

Instruction	Cycles
LDS Rd, k	3
STS k, Rr	2

These AVRxt cycle counts assume internal SRAM or ordinary data-memory access. Microchip's instruction manual notes that data-memory accesses to NVM need at least one extra cycle, with the exact timing depending on the device's NVMCTRL implementation. Treat EEPROM and mapped flash as slower than SRAM unless the device data sheet gives a tighter bound for the specific access.

When to use LDS/STS: accessing a single named variable. For loops over arrays, indirect addressing (Section 7.4) is faster and smaller.

So LDS/STS are great for “go touch this exact place once.” They are a poor fit for repeated traversal through memory.

7.3.1 I/O Registers Above 0x3F

IN/OUT cannot reach peripheral registers with addresses above 0x3F. Use data-space instructions (LD/LDS/LDD and ST/STS/STD) for those addresses. For a fixed named peripheral register, LDS/STS are the usual direct form:

```
ldi r16, (1 << 4)
sts PORTB_DIRSET, r16    ; PORTB_DIRSET = 0x0421, above 0x3F → STS
```

For addresses 0x00–0x3F, always prefer IN/OUT (1 cycle, 1 word) over LDS/STS (3/2 cycles, 2 words).

That is one of the easiest AVR optimisations to make because it is mechanical: if the register is in low I/O space, use the I/O instructions.

There is one more I/O-address boundary to remember:

- IN and OUT can address I/O locations 0x00–0x3F.
- SBI, CBI, SBIC, and SBIS can address only 0x00–0x1F.

ATtiny3217 exposes virtual port registers (VPORTx) in the low I/O range so simple GPIO operations can use IN/OUT. The full PORTx peripheral registers live in extended I/O space, so examples such as PORTB_DIRSET use STS.

7.4 Pointer Registers: X, Y, Z

The three pointer register pairs are 16-bit registers built from adjacent general-purpose registers:

X = r27 : r26	XH : XL
Y = r29 : r28	YH : YL
Z = r31 : r30	ZH : ZL

To load an address into a pointer pair, use two LDI instructions:

```
; Point X at buf (a label in .bss)
ldi XL, lo8(buf)      ; XL = r26 = low byte of buf's address
ldi XH, hi8(buf)      ; XH = r27 = high byte
```

Now X holds the 16-bit address of buf and can be used with LD/ST.

This is the beginning of pointer-based programming in AVR. Instead of naming an address directly in every instruction, you put the address in X, Y, or Z once and then operate through that pointer.

Practical convention:

- Use X for simple source or destination pointers.
- Use Y when you want a stable base pointer for LDD/STD, especially for structs or stack-frame-style code.
- Use Z for flash table reads, because LPM requires Z and mapped-flash examples commonly use Z too.

The ATtiny3217 data space fits in 64 KB, so X, Y, and Z are enough. Larger AVR devices can add RAMPX, RAMPY, RAMPZ, or RAMPD to extend addressing beyond 64 KB, but that extra machinery is not part of normal ATtiny3217 code.

7.5 Indirect Addressing: LD and ST

7.5.1 Basic Indirect — No Change to Pointer

```
LD  Rd, X      ; Rd ← SRAM[X]      (X unchanged)
LD  Rd, Y      ; Rd ← SRAM[Y]
LD  Rd, Z      ; Rd ← SRAM[Z]
ST  X, Rr      ; SRAM[X] ← Rr      (X unchanged)
ST  Y, Rr
ST  Z, Rr
```

On AVRxt (ATtiny3217): LD Rd, X/Y/Z is **2 cycles**, ST X/Y/Z, Rr is **1 cycle**. Use when you want to read/write the same address multiple times without moving the pointer.

This is the pointer equivalent of “stay where you are and keep working here.”

```
; Read, modify, write a single SRAM byte
ldi XL, lo8(my_var)
ldi XH, hi8(my_var)
ld  r16, X      ; r16 = my_var
ori r16, 0x80   ; set bit 7
st  X, r16      ; write back – X still points to my_var
```

7.5.2 Post-Increment — Walk Forward Through Memory

```
LD  Rd, X+      ; Rd ← SRAM[X], then X += 1
ST  X+, Rr      ; SRAM[X] ← Rr, then X += 1
```

The increment happens **after** the access. The full 16-bit pointer is incremented, so 0x38FF correctly becomes 0x3900. X is ready to point to the next byte. Perfect for forward array traversal:

This is one of the most natural addressing modes on AVR. It lets the memory access and the pointer update happen in the same instruction.

```
; Zero-fill buf (8 bytes)
ldi XL, lo8(buf)
```

```
ldi XH, hi8(buf)
ldi r16, 0
ldi r18, 8
1:
    st X+, r16      ; write 0, advance X
    dec r18
    brne 1b
```

7.5.3 Pre-Decrement — Walk Backward Through Memory

```
LD Rd, -X          ; X -= 1, then Rd ← SRAM[X]
ST -X, Rr          ; X -= 1, then SRAM[X] ← Rr
```

The decrement happens **before** the access. The full 16-bit pointer is decremented before the memory cycle. To walk an array backwards, point X one past the last element:

Pre-decrement often feels odd the first time you see it, but it matches how stacks and reverse traversals naturally work.

```
; Sum array backwards into r17
ldi XL, lo8(buf + 8) ; one past end
ldi XH, hi8(buf + 8)
clr r17
ldi r18, 8
1:
    ld r16, -X      ; X--, then load
    add r17, r16
    dec r18
    brne 1b
```

7.5.4 All Three Pointers, All Three Modes

X, Y, and Z all support LD/ST, LD+/ST+, and LD-/ST-. The only difference is which register pair holds the address:

So the real question is rarely “which instruction family do I need?” More often it is “which pointer register should own this job?”

```
ld r16, X+      ; load via X, post-increment
ld r16, Y+      ; load via Y, post-increment
ld r16, Z+      ; load via Z, post-increment — also used for LPM
```

Avoid using a pointer register as both the pointer and the source or destination register in auto-increment/decrement forms. The AVR instruction manual marks Z auto-update combinations such as LD r30, Z+, LD r31, Z+, LD r30, -Z, LD r31, -Z, ST Z+, r30, ST Z+, r31, ST -Z, r30, and ST -Z, r31 as undefined because r30 and r31 are part of Z itself. The same rule applies to LPM r30, Z+ and LPM r31, Z+.

7.6 Displacement Addressing: LDD and STD

Y and Z support a **constant displacement** added to the pointer without modifying it. X does **not** support displacement.

```
LDD Rd, Y+q      ; Rd ← SRAM[Y + q]    (Y unchanged, q = 0–63)
LDD Rd, Z+q      ; Rd ← SRAM[Z + q]
STD Y+q, Rr      ; SRAM[Y + q] ← Rr
STD Z+q, Rr
```


On AVRxt: LDD is 2 cycles and STD is 1 cycle. Displacement q must be a constant 0-63 — it cannot be a register.

Displacement mode is useful because it lets one pointer stay parked at a base address while you reach nearby fields by constant offset.

The 0-63 displacement limit is encoded directly in the 16-bit instruction. If a field is at offset 64 or later, adjust the base pointer first with ADIW or use normal pointer arithmetic, then access through Y or Z.

Primary use: struct member access. Point Y at the base of a struct, then access each member by offset:

```
; Struct layout in SRAM:
; offset 0: status (1 byte)
; offset 1: count (1 byte)
; offset 2: value (2 bytes, little-endian)
; offset 4: flags (1 byte)

.equ STATUS_OFF, 0
.equ COUNT_OFF, 1
.equ VALUE_OFF, 2
.equ FLAGS_OFF, 4

; Point Y at the struct
ldi YL, lo8(my_struct)
ldi YH, hi8(my_struct)

; Read all fields without moving Y
ldd r16, Y+STATUS_OFF ; r16 = status
ldd r17, Y+COUNT_OFF ; r17 = count
ldd r18, Y+VALUE_OFF ; r18 = value low byte
ldd r19, Y+VALUE_OFF+1 ; r19 = value high byte
ldd r20, Y+FLAGS_OFF ; r20 = flags
```

Secondary use: stack frame pointer. The compiler uses Y as the frame pointer, accessing local variables via LDD r, Y+offset. Chapter 15 covers this in the context of subroutines.

In other words, LDD/STD are the AVR equivalent of “base pointer + fixed offset” addressing.

7.7 Cycle and Size Comparison

Mode	Instruction	Cycles (LD/ST)	Words	Pointer moves?
Direct	LDS / STS	3 / 2	2	No
Indirect	LD / ST	2 / 1	1	No
Post-increment	LD+ / ST+	2 / 1	1	+1 after
Pre-decrement	LD- / ST-	2 / 1	1	-1 before
Displacement	LDD / STD	2 / 1	1	No (Y/Z only)
I/O register	IN / OUT	1	1	N/A (addr 0-3F)

Cycle counts in this table are the AVRxt base costs for internal SRAM and low I/O style accesses. AVRxt made every indirect store one cycle faster than the load that mirrors it. Accesses that hit NVM regions, including EEPROM and mapped flash, can take additional cycles.

Rule of thumb: - **Named variable, one access:** LDS/STS - **Named variable, repeated access:** load address into pointer, then LD/ST - **Array traversal:** LD+/ST+ or LD-/ST- - **Struct**

fields: LDD/STD with Y as base - **I/O registers $\leq 0x3F$:** IN/OUT

Those rules are worth internalising because they cover most real choices in ordinary firmware.

7.8 Reading Flash: LPM

On all AVR devices, LPM reads one byte from the **program bus** (flash) into a register. Z holds the byte address of the flash location to read.

This is the classic AVR way to read constants stored in flash. If you learned AVR on older parts, LPM is probably the method you saw first.

```
LPM Rd, Z          ; Rd ← Flash[Z]          (Z unchanged)
LPM Rd, Z+         ; Rd ← Flash[Z], Z++ (post-increment)
LPM                ; r0 ← Flash[Z]          (deprecated form)
```

3 cycles. Z must be the byte address (not word address).

That byte-address detail matters because some other AVR concepts, like jump targets, use word-oriented program addressing instead.

On ATtiny3217, flash is 32 KB, so ordinary LPM is enough for the whole program memory. ELPM and RAMPZ matter on larger AVR devices with more than 64 KB of program memory, but they do not buy anything on this MCU.

For hand-written flash tables intended for LPM, put the bytes in program-memory-addressed storage such as `.progmem.data`, or in `.text` after executable code. Do not blindly use a `.rodata` symbol as an LPM address with modern `avr-ld` scripts: `.rodata` may have a mapped data-space VMA (`0x8000` plus the flash load address). Check `avr-nm` or `avr-objdump -h` when in doubt.

```
.section .progmem.data, "a", @progbits
lookup:
    .byte 0, 1, 1, 2, 3, 5, 8, 13    ; Fibonacci

.section .text

    ; Load lookup[3] into r16
    ldi ZL, lo8(lookup + 3)
    ldi ZH, hi8(lookup + 3)
    lpm r16, Z                      ; r16 = 2 (lookup[3])
```

Copying a flash table to SRAM with LPM:

```
ldi ZL, lo8(lookup)
ldi ZH, hi8(lookup)
ldi XL, lo8(dest_buf)
ldi XH, hi8(dest_buf)
ldi r18, 8

copy:
    lpm r16, Z+                    ; load from flash, Z++
    st X+, r16                     ; store to SRAM, X++
    dec r18
    brne copy
```

7.9 Reading Flash: Mapped Access (AVRxt Only)

On AVRxt (ATtiny3217), flash is mapped into the **data address space** at offset `MAPPED_PROGMEM_START = 0x8000`. You can read it with ordinary LD instructions — no LPM needed.

This is one of the nicest quality-of-life improvements on the ATtiny3217. It lets flash reads look much more like normal memory reads.

```
LD Rd, Z          ; Rd ← DataMem[Z]
```

If $Z \geq 0x8000$, the data bus reads from flash. If $Z < 0x8000$, it reads SRAM or I/O registers. The CPU doesn't care which — the address selects the bus.

That means one instruction family, LD, can read from very different physical resources depending on the address you place in the pointer.

The ATtiny3217 data sheet states that the whole 32 KB flash is mapped from 0x8000 and is accessible with normal LD/ST instructions as well as LPM. In ordinary application code, use this mapped view mainly for reads. Flash writes are NVMCTRL-controlled operations, not arbitrary byte stores like SRAM writes.

```
.section .progmem.data, "a", @progbits
lookup:
    .byte 0, 1, 1, 2, 3, 5, 8, 13

.section .text

    ; Load lookup[3] into r16 via mapped flash
    ldi ZL, lo8(lookup + MAPPED_PROGMEM_START + 3)
    ldi ZH, hi8(lookup + MAPPED_PROGMEM_START + 3)
    ld  r16, Z          ; r16 = 2
```

Copying with mapped LD — same loop as SRAM copy:

```
ldi ZL, lo8(lookup + MAPPED_PROGMEM_START)
ldi ZH, hi8(lookup + MAPPED_PROGMEM_START)
ldi XL, lo8(dest_buf)
ldi XH, hi8(dest_buf)
ldi r18, 8

copy_mapped:
    ld  r16, Z+          ; load from mapped flash, Z++
    st  X+, r16          ; store to SRAM, X++
    dec r18
    brne copy_mapped
```

7.9.1 LPM vs Mapped LD — Which to Use?

	LPM	Mapped LD
Available on	All AVR	AVRxt only (tinyAVR 1-series, megaAVR 0-series)
Cycles	3	2 base, plus NVM wait time if any
Words	1	1
Z value	Flash byte address	Flash byte address + 0x8000
Pointer modes	Z, Z+ only	X, Y, Z — all modes including displacement
ATtiny3217 coverage	Whole 32 KB flash	Whole 32 KB flash at 0x8000-0xFFFF

On ATtiny3217 with 32 KB flash: **prefer mapped LD** when AVRxt-specific code is acceptable. It uses all pointer modes and reads flash and SRAM with the same instruction family, which is usually the simpler mental model. It is also usually faster than LPM, but remember the NVM wait-state caveat above.

LPM is still worth knowing because it is portable across AVR families, but for this chip, mapped LD is usually the cleaner choice.

7.10 Pointer Arithmetic

7.10.1 Advancing a Pointer by N Bytes

Post-increment advances by one byte per memory access. To skip N bytes, use ADIW:

```
adiw ZL, 4      ; Z += 4 (skip 4 bytes)
adiw XL, 16     ; X += 16
```

ADIW only handles values 0–63. For larger steps, use ADD/ADC on the pair directly:

So pointer arithmetic on AVR comes in two flavors: compact/specialized (ADIW/SBIW) and general/manual (ADD/ADC).

```
; Z += 200
ldi r16, 200
add ZL, r16
ldi r16, 0
adc ZH, r16      ; propagate carry into high byte
```

Or use r1 (which the ABI keeps at 0) for the carry:

```
ldi r16, 200
add ZL, r16
adc ZH, r1       ; r1 = 0 by convention – just propagates carry
```

7.10.2 Computing an Array Element Address

```
; Address of array[i]: base + i * element_size (element_size = 1 here)
; r24 = index, Z = array base
add ZL, r24      ; Z += index (1-byte elements)
adc ZH, r1       ; carry
ld r16, Z        ; load array[i]
```

For 2-byte elements:

```
; i * 2 = i << 1
lsl r24          ; r24 *= 2 (left shift)
rol r25          ; shift carry into r25 (if index > 127)
add ZL, r24
adc ZH, r25
```

This is the same reasoning as in multi-byte arithmetic: build the low byte result first, then propagate carry into the high byte.

7.11 Atomicity on ATtiny3217

Some AVR families implement XCH, LAC, LAS, and LAT, but the ATtiny3217's AVRxt core does **not**. If you need interrupt-safe bit updates on GPIO, use the dedicated peripheral

registers such as `PORTx_OUTSET`, `PORTx_OUTCLR`, and `PORTx_OUTTGL` instead of a read-modify-write sequence.

This matters because “load-modify-store” is not automatically safe when an ISR or another hardware path can touch the same state concurrently.

```
ldi r16, (1 << 3)
sts PORTA_OUTSET, r16      ; set PA3 without reading PORTA_OUT first
sts PORTA_OUTCLR, r16     ; clear PA3 atomically
sts PORTA_OUTTGL, r16     ; toggle PA3 atomically
```

For ordinary SRAM bytes shared with an ISR, there is no single instruction on this MCU that performs an atomic arbitrary read-modify-write. Use a critical section (IN/CLI/OUT SREG) when the update must not be interrupted.

So the practical split is:

- for GPIO, prefer the hardware set/clear/toggle registers;
- for shared SRAM state, protect the update with interrupt control when needed.

7.12 Full Addressing Mode Reference

Instruction	Addr calc	Pointer after	Cycles (AVRxt)	Words
LDS Rd, k	k	—	3	2
STS k, Rr	k	—	2	2
LD Rd, X	X	unchanged	2	1
LD Rd, X+	X	X + 1	2	1
LD Rd, -X	X - 1	X - 1	2	1
ST X, Rr	X	unchanged	1	1
ST X+, Rr	X	X + 1	1	1
ST -X, Rr	X - 1	X - 1	1	1
LD Rd, Y	Y	unchanged	2	1
LD Rd, Y+	Y	Y + 1	2	1
LD Rd, -Y	Y - 1	Y - 1	2	1
LDD Rd, Y+q	Y + q	unchanged	2	1
ST Y, Rr	Y	unchanged	1	1
ST Y+, Rr	Y	Y + 1	1	1
ST -Y, Rr	Y - 1	Y - 1	1	1
STD Y+q, Rr	Y + q	unchanged	1	1
LD Rd, Z	Z	unchanged	2	1
LD Rd, Z+	Z	Z + 1	2	1
LD Rd, -Z	Z - 1	Z - 1	2	1
LDD Rd, Z+q	Z + q	unchanged	2	1
ST Z, Rr	Z	unchanged	1	1
ST Z+, Rr	Z	Z + 1	1	1
ST -Z, Rr	Z - 1	Z - 1	1	1
STD Z+q, Rr	Z + q	unchanged	1	1
LPM Rd, Z	Flash[Z]	unchanged	3	1
LPM Rd, Z+	Flash[Z]	Z + 1	3	1
IN Rd, A	I/O[A] (0–3F)	—	1	1
OUT A, Rr	I/O[A] (0–3F)	—	1	1

PUSH Rr	SRAM[SP]	SP - 1	1	1
POP Rd	SRAM[SP+1]	SP + 1	2	1

Stores ST/STD and PUSH run in **1 cycle** on AVRxt — that is one cycle faster than the matching loads, and one cycle faster than the same instructions on AVRe (ATmega) cores. Loads (LD/LDD/POP) and LDS keep the AVRe timings.

7.13 Worked Example: Reverse a String In-Place

Reverse an ASCII string stored in SRAM. Classic two-pointer approach.

This example is useful because it combines several addressing modes in one small routine: forward scan with Z+, fixed access through X and Z, then converging pointers for the swap loop.

```

/*
 * str_reverse – reverse a null-terminated string in SRAM in-place.
 * Input: X = pointer to first byte of string
 * Clobbers: X, Z, r16, r17, r18
 */
str_reverse:
    ; Find the end: walk Z to the null terminator
    movw ZL, XL                ; Z = X (copy pointer)
1:
    ld    r16, Z+              ; load byte, Z++
    tst   r16                  ; null?
    brne 1b                    ; no – keep walking

    ; Z is now one past the null. Step back twice: past null, to last char.
    sbiw  ZL, 2                ; Z now points to last character

    ; Now swap X[front] and Z[back], moving inward
2:
    cp    XL, ZL                ; has front crossed back? (low bytes)
    cpc   XH, ZH                ; (high bytes)
    brsh done                  ; BRSH: unsigned >=, front >= back → done

    ld    r16, X                ; r16 = front char
    ld    r17, Z                ; r17 = back char
    st    X+, r17               ; store back char at front, advance front
    st    Z, r16                ; store front char at back
    sbiw  ZL, 1                ; retreat back pointer
    rjmp  2b

done:
    ret

```

Trace with “AVR\0” at address 0x3810:

Initial: X = 0x3810 ('A'), Z = 0x3812 ('R')

Iter 1: swap 'A' and 'R' → X=0x3811, Z=0x3811

Iter 2: XL=0x11 >= ZL=0x11 → BRSH taken → done

Result in SRAM: “RVA\0” OK

Notice how little arithmetic the routine needs. Most of the real work is being done by the addressing modes themselves.

7.14 Summary

LDS/STS – direct, 16-bit address in instruction, 3/2 cycles, 2 words.
Use for named variables, single accesses.

LD/ST – indirect via X, Y, or Z. AVRxt: LD = 2 cycles, ST = 1 cycle.
1 word each.
no mode – pointer unchanged
X+/Y+/Z+ – post-increment (forward traversal)
-X/-Y/-Z – pre-decrement (backward traversal)

LDD/STD – Y or Z plus constant offset 0–63. AVRxt: LDD = 2 cycles,
STD = 1 cycle. Struct member access. X does NOT support
displacement.

PUSH/POP – AVRxt: PUSH = 1 cycle, POP = 2 cycles.

LPM – flash read via program bus. Z = byte address. 3 cycles.

Mapped LD – AVRxt: LD with Z = flash addr + 0x8000. 2-cycle base
cost plus possible NVM wait time. Preferred on ATtiny3217.

ADIW/SBIW – adjust pointer by constant 0–63 in 2 cycles.

ADD+ADC – adjust pointer by any 16-bit value.

ATtiny3217 data space: I/O 0x0000-0x003F, extended I/O 0x0040-0x0FFF,
EEPROM 0x1400-0x14FF, SRAM 0x3800-0x3FFF, mapped flash 0x8000-0xFFFF.

ATtiny3217 has no XCH/LAC/LAS/LAT. Use PORTx set/clear/toggle registers for
GPIO, and critical sections for shared SRAM updates.

7.15 Exercises

1. Write a subroutine `memset(X, val, n)` that fills `n` bytes starting at address `X` with value `val` (in `r16`). Use `ST X+` in the loop.
2. Write `memcpy(X, Z, n)` that copies `n` bytes from `Z` to `X`. Both source and destination are SRAM. Then modify it so the source is flash (use mapped LD for ATtiny3217).
3. Given the struct:
offset 0: type (1 byte)
offset 1: length (1 byte)
offset 2: data (4 bytes)
Write code using LDD/STD with Y to read type, increment length, and zero out the data field — all without moving Y.
4. What is the maximum array size (in elements, 1 byte each) you can access with LDD `Y+q`? What happens if you need element 64? Write code that handles this using ADIW and basic LD.
5. Assemble `loadstore.S`, run `avr-nm -n loadstore.elf`, then run `avr-objdump -s -j .text loadstore.elf`. Find `powers_of_two` and verify the bytes match 1, 2, 4, 8, 16, 32, 64, 128. Then find the address of `buf_lpm` in the `.bss` section – what address does it get?
6. Why does the `str_reverse` subroutine use `CP+CPC` instead of just `CP` to compare X and Z? What would go wrong with only `CP XL, ZL`?

Next: Chapter 8 — Unsigned Integer Arithmetic: 8-bit and 16-bit

Chapter 8

Unsigned Integer Arithmetic: 8-bit and 16-bit

Previous chapters introduced the AVR register file, the load/store model, and the instruction set at a survey level. This chapter narrows to a single topic and goes deep: **unsigned integer arithmetic** on 8-bit registers and 16-bit register pairs.

Unsigned arithmetic is where most embedded programs start. Sensor readings, buffer indices, byte counters, millisecond timestamps, and ADC results are all unsigned. Getting this layer of arithmetic right — including overflow detection, borrow tracking, and 16-bit carry chains — is the foundation that every later chapter builds upon.

The chapter covers the complete unsigned integer toolkit: all relevant instructions, the flag semantics specific to unsigned operations, two-byte carry chains, comparisons, and a set of practical worked examples with traced execution.

8.1 What “Unsigned” Means to the CPU

The AVR CPU operates on bytes. Every register holds 8 bits, and every 8-bit arithmetic instruction produces an 8-bit result. The CPU does not inherently know whether a register represents a temperature reading, a packet sequence number, or a raw index. The bits are just bits.

Unsigned interpretation means you choose to read those 8 bits as a value in the range 0 to 255:

bit pattern	unsigned value
0b00000000	0
0b00000001	1
0b01111111	127
0b10000000	128
0b10101010	170
0b11111111	255

There is no sign bit. No two’s complement. All 256 bit patterns map directly to distinct non-negative integers. The key consequence: arithmetic that exceeds 255 wraps back to 0, and arithmetic that goes below 0 wraps to 255. The carry flag records that a wrap happened.

For 16-bit values built from two registers, the range is 0 to 65535. The same wrapping principle applies, now spanning two bytes connected by the carry chain.

The CPU itself is neutral. The same ADD instruction produces the same bit result regardless of interpretation. What changes is which flags you check and which branch instructions you use afterward. Unsigned code uses C and Z; signed code uses S, V, and N. The moment you choose BRL0 instead of BRLT, you are telling the CPU to apply unsigned interpretation.

8.2 The Status Register (SREG) and Unsigned Flags

Every arithmetic instruction updates the Status Register (SREG). For unsigned arithmetic, the relevant flags are:

Bit	Name	Unsigned meaning
0	C	Carry (addition) or Borrow (subtraction). ADD/ADC: C=1 means result exceeded 255 (or 65535 for 16-bit). SUB/SBC: C=1 means result went below 0 (borrow occurred). Compare: C=1 means $R_d < R_r$ (unsigned less-than).
1	Z	Zero. Z=1 when the result is exactly 0x00 (or 0x0000 for 16-bit).
5	H	Half-carry. Carry out of bit 3 into bit 4. Mostly used for BCD. For pure unsigned arithmetic, you rarely consult H directly.

The signed flags (V, N, S) are still computed by the hardware after unsigned operations, but you ignore them when working with unsigned data.

A critical point about flag timing: **flags reflect the last instruction that updated them.** Any instruction between your arithmetic and your branch can change the flags. A MOV, CLR, AND, or even some pseudo-instructions all touch flags. Develop the habit of keeping compare/arithmetic results adjacent to the branch that reads them.

8.2.1 Flag behavior for INC and DEC

INC and DEC are the exception to the rule. They update S, V, N, Z but **do not change the carry flag (C)**. This is by design — they are designed for simple counters, not for feeding carry into a multi-byte chain. More on this in the section on 16-bit arithmetic.

8.3 8-bit Unsigned Arithmetic

8.3.1 Addition: ADD

ADD Rd, Rr ; $R_d \leftarrow R_d + R_r$ (any r0–r31)

1 cycle. Flags: H, S, V, N, Z, C.

The fundamental unsigned addition. Both operands are any register; the result replaces Rd.

```
ldi r16, 100 ; r16 = 100
ldi r17, 50 ; r17 = 50
add r16, r17 ; r16 = 150, C=0, Z=0
```

No carry means the result is exact. With larger values:

```
ldi r16, 200 ; r16 = 200
ldi r17, 100 ; r17 = 100
add r16, r17 ; 200+100=300, wrapped: r16 = 44, C=1
```

The mathematical result 300 does not fit in 8 bits. The CPU stores $300 \bmod 256 = 44$ in r16 and sets $C=1$ to record that the result overflowed the 8-bit range.

C=1 after ADD means unsigned overflow. The stored 8-bit result is modulo 256 of the true answer.

8.3.2 Immediate Addition: No ADDI — Use SUBI with Negated Constant

AVR has no ADDI instruction. To add a compile-time constant to a register, subtract its two's-complement negation using SUBI:

SUBI Rd, K ; $Rd \leftarrow Rd - K$ (r16–r31 only, $K = 0..255$)

```
; r16 += 10 - there is no ADDI, so:
subi r16, -10 ; subtract -10 is the same as add 10
               ; assembler encodes -10 as 0xF6 and computes 8-bit sub
```

The assembler accepts negative literals and encodes them modulo 256. The instruction performs $Rd - 0xF6$, which is numerically identical to $Rd + 10$ for the unsigned result bits. C is set if the result would have wrapped, which for adding 10 is the same carry as adding 10 directly.

This idiom is common and correct. If it feels odd, convince yourself: subtracting -10 wraps exactly the same as adding 10 in two's-complement modular arithmetic.

For adding values already in registers, use ADD. SUBI is for compile-time constants only.

8.3.3 Adding a Constant to r0-r15

SUBI only works on r16–r31. For lower registers, load the constant into a scratch register and use ADD:

```
; r5 += 42 (r5 is in lower half, cannot use SUBI)
ldi r16, 42 ; scratch register
add r5, r16 ; r5 = r5 + 42
```

8.3.4 Increment: INC

INC Rd ; $Rd \leftarrow Rd + 1$ (any r0–r31)

1 cycle. Flags: S, V, N, Z — **not C**.

INC is a convenient shorthand for adding 1, but the missing carry flag is a critical restriction. Use it only when you do not need the carry afterward:

```
ldi r16, 0
loop:
    ; ... do work ...
    inc r16 ; advance counter
    cpi r16, 10
    brne loop ; repeat while r16 != 10
```

The loop counter never overflows into multi-byte territory, so the absent carry is irrelevant. This is the intended use case.

```
; WRONG: trying to detect overflow via C
ldi r16, 255
inc r16 ; r16 = 0 (wraps), but C is NOT set
brcs overflow ; never branches - C wasn't updated by INC
```

When you need carry-aware addition of 1, use ADD Rd, some_one_register with a register holding 1, or use ADIW for 16-bit pairs.

8.3.5 Subtract: SUB

SUB Rd, Rr ; $Rd \leftarrow Rd - Rr$ (any r0–r31)

1 cycle. Flags: H, S, V, N, Z, C.

```
ldi r16, 100
ldi r17, 40
sub r16, r17 ; r16 = 60, C=0
```

C=0 means no borrow: the result is non-negative and exact.

```
ldi r16, 10
ldi r17, 30
sub r16, r17 ; 10-30 = -20, wrapped: r16 = 236 (0xEC), C=1
```

C=1 after SUB means unsigned underflow (borrow). The stored result is the two's complement of the "deficit," equivalently $256 + (10 - 30) = 236$.

For most unsigned code, a borrow means the subtraction went wrong or you should take a different branch. The pattern is:

```
cp r16, r17 ; compare without changing r16
brlo too_small ; branch if r16 < r17 unsigned (C=1)
sub r16, r17 ; only subtract when r16 >= r17
```

8.3.6 Immediate Subtraction: SUBI

SUBI Rd, K ; $Rd \leftarrow Rd - K$ (r16–r31 only, K = 0..255)

1 cycle.

```
ldi r16, 100
subi r16, 25 ; r16 = 75
subi r16, 100 ; 75 - 100 = -25, wrapped: r16 = 231, C=1
```

8.3.7 Decrement: DEC

DEC Rd ; $Rd \leftarrow Rd - 1$ (any r0–r31)

1 cycle. Flags: S, V, N, Z — **not C**.

Same restriction as INC. Use it for simple counters where the carry is not needed:

```
ldi r20, 8 ; loop 8 times
countdown:
; ... do work ...
dec r20
brne countdown ; BRNE reads Z, not C — safe with DEC
```

BRNE tests Z, which DEC does update correctly. This is the reliable pattern for DEC-based loops.

8.3.8 Arithmetic vs Logic: ADD vs OR / EOR

A common beginner mistake is using OR to combine two numbers and expecting addition. OR performs bitwise union, not addition:

```
ldi r16, 0b00001111 ; = 15
ldi r17, 0b00001111 ; = 15
or r16, r17 ; r16 = 0b00001111 = 15 ← no carry possible
add r16, r17 ; r16 = 0b00011110 = 30 ← correct addition
```

OR is for bit manipulation. ADD is for numeric addition.

8.4 Unsigned Overflow, Wraparound, and Saturation

8.4.1 Wraparound (Modular) Arithmetic

When C=1 after ADD, the result has wrapped. Wraparound is useful when you deliberately want modular arithmetic — for example, a repeating 8-bit phase counter that cycles 0→255→0→255:

```
loop:
    lds r16, phase
    add r16, step      ; advance phase – wraps naturally at 255→0
    sts phase, r16
    rjmp loop
```

No carry check needed; the wrap is the desired behavior.

8.4.2 Detecting Overflow

When you need to detect that a wrap happened, test C immediately after ADD or use a pre-check:

```
; Post-check: detect overflow after ADD
add r16, r17
brcs overflow_handler ; C=1 → result wrapped

; Pre-check: verify headroom before adding
ldi r17, 10
cpi r16, 246          ; is r16 > 245? (256 - 10 - 1 = 245)
brsh overflow_handler ; unsigned >=246 + 10 would exceed 255
add r16, r17          ; safe to add
```

8.4.3 Saturating Addition — Clamp to 255

For UI values, brightness levels, and sensor readings, wraparound is wrong: 255 + 1 should stay at 255, not jump to 0. This is called *saturating* arithmetic:

```
; Saturating 8-bit add: result = min(r16 + r17, 255)
add r16, r17
brcc no_clamp      ; C=0: no overflow, result is correct
ser r16            ; C=1: clamp to 0xFF = 255
no_clamp:
```

SER Rd is an alias for LDI Rd, 0xFF and loads 255 into Rd. After the clamp, r16 holds the saturated result.

8.4.4 Saturating Subtraction — Clamp to 0

```
; Saturating 8-bit subtract: result = max(r16 - r17, 0)
sub r16, r17
brcc no_underflow  ; C=0: no borrow, result is correct
clr r16            ; C=1: clamp to 0
no_underflow:
```

These two patterns appear constantly in embedded firmware wherever user-facing quantities have natural upper or lower bounds.

8.5 8-bit Unsigned Comparisons and Branches

Compare instructions perform a subtraction and set flags without storing the result. They are always paired with a conditional branch.

8.5.1 CP — Compare Two Registers

CP Rd, Rr ; Rd - Rr → flags only, Rd and Rr unchanged

```
ldi r16, 42
ldi r17, 100
cp r16, r17 ; flags set as if 42 - 100 → C=1 (borrow), Z=0
brlo r16_smaller ; C=1 → r16 < r17 unsigned → branch taken
```

8.5.2 CPI — Compare with Immediate

CPI Rd, K ; Rd - K → flags only (r16–r31 only)

```
cpi r16, 200
brlo r16_below_200 ; r16 < 200 unsigned → branch
brsh r16_at_or_above; r16 >= 200 unsigned → branch
```

8.5.3 CPSE — Compare and Skip if Equal

CPSE Rd, Rr ; if Rd == Rr: skip next instruction

A 1-2 cycle conditional skip, useful for simple equality without an explicit branch target:

```
cpse r16, r17 ; if r16 == r17: skip the rjmp
rjmp not_equal
; equal path continues here
```

8.5.4 Unsigned Branch Instructions

These are the branches to use after unsigned comparisons:

Instruction	Condition	Flag tested	Meaning
BREQ	Rd == Rr	Z=1	Equal
BRNE	Rd != Rr	Z=0	Not equal
BRLO	Rd < Rr (unsigned)	C=1	Below (same as BRCS)
BRSH	Rd >= Rr (unsigned)	C=0	Same or higher (same as BRCC)
BRCS	carry set	C=1	Alias for BRLO
BRCC	carry clear	C=0	Alias for BRSH

There are no BRGT or BRLE instructions for unsigned comparisons. These cases require two tests:

```
; Branch if r16 > r17 unsigned (strictly greater)
cp r16, r17
breq equal ; r16 == r17: not greater
brsh greater_than ; r16 >= r17 and not equal → r16 > r17

; Branch if r16 <= r17 unsigned
cp r16, r17
brlo less_than ; C=1 → r16 < r17
```

```
breq equal          ; Z=1 → r16 == r17
; if neither: r16 > r17
```

8.5.5 The Unsigned Branching Mental Model

Unsigned branches decode cleanly from the subtraction model:

CP Rd, Rr computes: Rd - Rr

If Rd >= Rr (unsigned): subtraction succeeds with no borrow → C=0

If Rd < Rr (unsigned): subtraction borrows → C=1

Therefore:

BRLO ≡ branch if C=1 after CP ≡ Rd < Rr unsigned
 BRSH ≡ branch if C=0 after CP ≡ Rd >= Rr unsigned

This maps directly to BRCS/BRCC. The names BRLO and BRSH just make the unsigned comparison intent explicit in source code.

8.5.6 Unsigned Range Check

A common embedded task: check whether a byte is inside a range [lo, hi]:

```
; Is r16 in the range [30, 80]?
cpi r16, 30
brlo out_of_range    ; r16 < 30 unsigned → too low
cpi r16, 81          ; hi + 1
brsh out_of_range    ; r16 >= 81 unsigned → too high
; here: 30 <= r16 <= 80
```

The upper bound check uses hi+1 because BRSH tests >=. If hi is 255, use a separate CPI r16, 255 + BRNE rather than risking the 256 that does not fit in the immediate.

8.6 Moving to 16-bit: Register Pairs

Eight bits holds values up to 255. Many real quantities require more: millisecond uptime counters, distances in millimetres, raw 10-bit ADC readings, buffer byte counts for USART frames. These need 16-bit unsigned arithmetic.

AVR registers are 8 bits wide. 16-bit values are held in **register pairs**: two adjacent registers used together.

8.6.1 Register Pair Convention

The notation Rh:Rl means the high byte is in Rh and the low byte is in Rl. The numeric value is:

$$\text{value} = (\text{Rh} \times 256) + \text{Rl}$$

Any adjacent pair of registers can hold a 16-bit value, but three specific pairs are also hardware pointer registers with dedicated addressing instructions:

Pair name	High register	Low register	Notes
X	r27 (XH)	r26 (XL)	Indirect load/store pointer

Pair name	High register	Low register	Notes
Y	r29 (YH)	r28 (YL)	Indirect load/store; frame pointer in GCC ABI
Z	r31 (ZH)	r30 (ZL)	Indirect load/store; flash read pointer

For arithmetic (not pointers), any even-register pair works. By GCC convention, function results are returned in r25:r24. General 16-bit temporaries commonly use r25:r24, r23:r22, r21:r20, and so on downward in pairs.

8.6.2 Loading a 16-bit Constant

```
; r25:r24 = 1000 (0x03E8)
ldi r24, lo8(1000)      ; r24 = 0xE8 (low byte)
ldi r25, hi8(1000)      ; r25 = 0x03 (high byte)
```

lo8() and hi8() are assembler expressions that extract the low or high byte of a 16-bit constant. The assembler evaluates these at assembly time.

```
; Named constant example
.equ MAX_SAMPLES, 512

ldi r24, lo8(MAX_SAMPLES) ; r24 = 0x00
ldi r25, hi8(MAX_SAMPLES) ; r25 = 0x02
; r25:r24 = 512
```

8.6.3 Copying a 16-bit Value: MOVW

MOVW Rd, Rr ; Rd+1:Rd ← Rr+1:Rr (Rd, Rr must be even)

1 cycle. Copies two registers in one instruction. Assembler allows either the low register or the named alias:

```
movw r24, r22      ; r25:r24 ← r23:r22
movw r28, r26      ; Y ← X
movw r30, r24      ; Z ← r25:r24
```

This is faster and clearer than two separate MOV instructions, and it reads naturally as a 16-bit register copy.

8.6.4 Loading a 16-bit Value from SRAM

```
; Load 16-bit variable from SRAM (little-endian: low byte at lower address)
lds r24, var_lo      ; load low byte
lds r25, var_hi      ; load high byte

; Using X pointer (after loading address into X)
ld r24, X+           ; load low byte, X advances
ld r25, X            ; load high byte
```


AVR stores multi-byte values in **little-endian** order: low byte at the lower address. The GCC toolchain follows this convention for all variables. When reading 16-bit values from SRAM, always load the low byte from the lower address.

8.7 16-bit Unsigned Addition

8.7.1 The Carry Chain: ADD + ADC

There is no 16-bit ADD instruction. You build 16-bit addition from two 8-bit operations by using the carry flag as a bridge between bytes.

```
ADD Rd, Rr      ; low bytes – may produce carry
ADC Rh, Rsh     ; high bytes – includes carry from previous ADD
```

The mnemonic ADC stands for **Add with Carry**: $Rh \leftarrow Rh + Rsh + C$.

The order is mandatory: always process the low byte first. The carry from the low byte must be in C when the ADC executes. Processing high before low produces wrong results.

```
; 16-bit add: r25:r24 = r25:r24 + r23:r22
add r24, r22    ; low bytes: r24 = r24 + r22, may set C
adc r25, r23    ; high bytes: r25 = r25 + r23 + C
```

Example trace: 500 (0x01F4) + 700 (0x02BC) = 1200 (0x04B0):

```
r24 = 0xF4 (low 500),   r25 = 0x01 (high 500)
r22 = 0xBC (low 700),   r23 = 0x02 (high 700)
```

```
ADD r24, r22: 0xF4 + 0xBC = 0x1B0 → r24 = 0xB0, C=1
ADC r25, r23: 0x01 + 0x02 + 1(C) = 0x04 → r25 = 0x04, C=0
```

Result: r25:r24 = 0x04B0 = 1200 ✓

After the chain: - C=0: exact result - C=1: unsigned overflow — result exceeded 65535 and wrapped to (true_result mod 65536)

```
; Detecting 16-bit unsigned overflow
add r24, r22
adc r25, r23
brcs overflow_16 ; C=1 after ADC → overflow
```

8.7.2 ADIW — Add Immediate to Word

ADIW Rd, K ; $Rd+1:Rd \leftarrow Rd+1:Rd + K$ (K = 0..63)

2 cycles. Works only on the four special pairs:

Low-register operand	Pair
r24	r25:r24
r26 / XL	r27:r26 = X
r28 / YL	r29:r28 = Y
r30 / ZL	r31:r30 = Z

```
adiw r24, 1 ; r25:r24 += 1 - 2 cycles
adiw XL, 4 ; X pointer += 4
adiw ZL, 16 ; Z += 16
```

ADIW is the preferred way to increment a 16-bit pair when the step fits in 0–63. Two cycles instead of two separate instructions, and the carry semantics are correct.

Flags: C, Z, N, V, S. Does not update H.

When K is 0, ADIW Rd, 0 still updates Z and N — useful to test whether a 16-bit pair is zero without modifying it (though TST on each byte is more commonly used for that purpose).

8.7.3 Adding a Larger Immediate Constant (SUBI + SBCI)

For constants larger than 63 or not on the ADIW-supported pairs, use SUBI+SBCI with negated values — the same pattern as 8-bit immediate add, extended to two bytes:

```
; r25:r24 += 1000 (1000 = 0x03E8, larger than 63)
subi r24, lo8(-1000)    ; r24 -= lo8(-1000) = subtract -(0x18) = add 0xE8
sbc i r25, hi8(-1000)   ; r25 -= hi8(-1000) + borrow
```

The GNU assembler computes `lo8(-1000)` and `hi8(-1000)` at assembly time. `-1000` in 16-bit two's complement is `0xFC18`. So `lo8(-1000) = 0x18` and `hi8(-1000) = 0xFC`. The CPU computes `r24 - 0x18` (subtract —low of 1000 = add low of 1000) and `r25 - 0xFC - C` (which adds the high byte of 1000 corrected by carry).

Both registers must be in r16–r31 for SUBI/SBCI.

8.7.4 General Case: ADD + ADC with a Loaded Constant

When neither ADIW nor SUBI/SBCI applies (wrong register range, constant too large), load the constant into a temporary pair and use ADD+ADC:

```
; r25:r24 += 5000 (r25:r24 is the only free pair in r16-r31)
ldi r22, lo8(5000)
ldi r23, hi8(5000)
add r24, r22
adc r25, r23
```

This is the general case. Two LDI loads, two arithmetic instructions. Use it when ADIW and SUBI+SBCI don't apply.

8.8 16-bit Unsigned Subtraction

8.8.1 The Borrow Chain: SUB + SBC

```
; 16-bit subtract: r25:r24 = r25:r24 - r23:r22
sub r24, r22    ; low bytes: r24 = r24 - r22, may set C (borrow)
sbc r25, r23    ; high bytes: r25 = r25 - r23 - C
```

SBC stands for **Subtract with Carry (borrow)**: $R_h \leftarrow R_h - R_{sh} - C$.

The pattern mirrors the addition chain. Low byte first. The borrow from the low byte is in C when SBC executes on the high byte.

Example trace: 300 (0x012C) – 500 (0x01F4) → underflow:

```
r24 = 0x2C (low 300), r25 = 0x01 (high 300)
r22 = 0xF4 (low 500), r23 = 0x01 (high 500)
```

```
SUB r24, r22: 0x2C - 0xF4 = -0xC8, borrow → r24 = 0x38, C=1
```

```
SBC r25, r23: 0x01 - 0x01 - 1(C) = -1, borrow → r25 = 0xFF, C=1
```

Result: `r25:r24 = 0xFF38 = 65336` (unsigned wrap of 300–500)

Final C=1 → unsigned underflow occurred

```
; Detecting 16-bit underflow
sub  r24, r22
sbc  r25, r23
brcs underflow_16    ; C=1 after SBC → result wrapped
```

Note on SBC and the Z flag: the AVR instruction set defines a special rule: SBC only clears Z when the byte result is nonzero; it does not set Z when the byte result is zero. This means Z=1 after the complete chain only if **all bytes** in the chain computed to zero. Do not read Z after only the first SBC in a multi-byte chain and expect it to mean “the 16-bit result is zero.” Read Z only after the final (most-significant-byte) SBC.

8.8.2 SBIW — Subtract Immediate from Word

SBIW Rd, K ; Rd+1:Rd ← Rd+1:Rd - K (K = 0..63)

Same register restrictions as ADIW (r24, X, Y, Z). 2 cycles.

```
sbiw r24, 1    ; r25:r24 -= 1
sbiw ZL, 8     ; Z -= 8
```

SBIW is commonly used in countdown loops:

```
ldi  r24, lo8(1000)
ldi  r25, hi8(1000)
loop:
    ; ... do work for one step ...
    sbiw r24, 1
    brne loop    ; Z=1 when r25:r24 == 0
```

BRNE reads Z, which SBIW updates correctly for the full 16-bit value. This loop runs exactly 1000 times.

8.8.3 Subtracting a Larger Constant: SUBI + SBCI

```
; r25:r24 -= 300 (r25:r24 in r16-r31)
subi r24, lo8(300)
sbci r25, hi8(300)
```

Unlike addition, this is straightforward: no negation needed. lo8(300) = 0x2C, hi8(300) = 0x01. The CPU subtracts these from the pair with the borrow propagated through SBCI.

8.9 16-bit Unsigned Comparisons

8.9.1 CP + CPC — Compare Bytes Then Compare with Carry

To compare two 16-bit unsigned values, chain CP with CPC:

```
; Compare r25:r24 with r23:r22 (16-bit unsigned)
cp  r24, r22    ; compare low bytes: (r24 - r22), sets C, Z
cpc r25, r23    ; compare high bytes including borrow: (r25 - r23 - C)
brlo r24r25_less ; BRL0: branch if r25:r24 < r23:r22 unsigned
```

After the chain, C and Z reflect the comparison of the full 16-bit value: - Z=1: the pairs are equal - C=1: r25:r24 < r23:r22 unsigned (unsigned less-than) - C=0 and Z=0: r25:r24 > r23:r22

The same Z-flag rule applies to CPC as to SBC: CPC only clears Z, never sets it for one byte in isolation. Z=1 after the full chain only if both bytes were equal.

```
; 16-bit: is count >= threshold?
; r25:r24 = count, r23:r22 = threshold
cp r24, r22
cpc r25, r23
brlo below_threshold ; count < threshold → branch
; here: count >= threshold
```

8.9.2 Comparing with a 16-bit Immediate

There is no 16-bit CPI. Use CPI for the low byte and SBCI for the high byte comparison (after zeroing a scratch register for the carry):

```
; r25:r24 == 1000?
cpi r24, lo8(1000) ; compare low byte
ldi r16, hi8(1000)
cpc r25, r16 ; compare high byte with borrow
breq equal_to_1000
```

8.10 Incrementing and Decrementing 16-bit Values

8.10.1 Why INC and DEC Fail for 16-bit

INC Rl increments the low register but does not set C, so the high register has no way to learn that a carry happened:

```
; WRONG 16-bit increment:
ldi r24, 0xFF
ldi r25, 0x00
inc r24 ; r24 wraps to 0x00, but C is NOT updated
; r25 stays 0x00 – r25:r24 is now 0x0000 instead of 0x0100!
```

Always use ADIW, ADD+ADC, or SUBI+SBCI for 16-bit increments.

8.10.2 The Correct Patterns

For ± 1 to ± 63 on a supported pair:

```
adiw r24, 1 ; r25:r24 += 1 – correct
sbiw r24, 1 ; r25:r24 -= 1 – correct
```

For ± 1 on any pair in r16–r31:

```
; r21:r20 += 1
subi r20, lo8(-1) ; r20 -= -1 = r20 += 1
sbc1 r21, hi8(-1) ; r21 -= 0xFF - C = r21 += 0 + C (propagates carry)
```

Wait — hi8(-1) is 0xFF. So this would compute $r21 - 0xFF - C$, which is not what we want. The correct form for +1 on a non-ADIW pair in r16–r31:

```
; r21:r20 += 1 (both in r16-r31)
ldi r16, 1
ldi r17, 0
add r20, r16
adc r21, r17
```

Or more efficiently, keep a zero register available and use:

```

; r21:r20 += 1
inc r20          ; low byte
brne no_carry    ; if r20 didn't wrap to 0, no carry needed
inc r21          ; carry: high byte needs increment
no_carry:

```

This trick works because when INC wraps 0xFF to 0x00, Z is set (the result is zero). Testing Z lets you determine whether carry needs to propagate. This is the one sanctioned use of INC in a multi-byte context: you test Z, not C.

For a decrement equivalent:

```

; r21:r20 -= 1
tst r20          ; test r20 without modifying it
breq borrow      ; r20 == 0 → decrement would wrap, need to borrow
dec r20          ; r20 > 0 → simple decrement, no borrow
rjmp dec_done
borrow:
dec r20          ; r20 wraps 0x00 to 0xFF (correct for 2's complement)
dec r21          ; propagate borrow to high byte
dec_done:

```

For most 16-bit decrement needs, SBIW is cleaner:

```

sbiw r24, 1      ; r25:r24 -= 1 – the right tool when on a valid pair

```

8.11 Practical Patterns

8.11.1 Pattern 1: 8-bit Counter with Rollover Detection

Count events from 0 to 255, then detect each complete rollover:

```

; r16 = event counter (0-255, unsigned)
on_event:
    inc r16
    brne still_counting ; Z=0: didn't wrap to 0, no rollover
    ; Z=1: counter wrapped 255→0, handle rollover
    rcall on_rollover
still_counting:
    ret

```

INC is appropriate here: we test Z (which INC does update), not C.

8.11.2 Pattern 2: 16-bit Odometer

Accumulate a 16-bit distance counter (in millimetres, for example):

```

; r25:r24 = total_mm (16-bit accumulator)
; r22 = step_mm (8-bit step per tick, always small)
;
; We need 8-bit + 16-bit: zero-extend r22 first

add_distance:
    clr r23          ; zero-extend r22 to 16-bit: r23:r22 = 0:step_mm
    add r24, r22
    adc r25, r23
    brcc distance_ok ; C=0: no overflow, done
    ser r24          ; overflow: saturate to 0xFFFF

```

```

    ser r25
distance_ok:
    ret

```

This accumulates distance with saturating overflow at 65535 mm $\approx$ 65.5 m.

8.11.3 Pattern 3: 16-bit Elapsed Time Check

Compare an 8-bit timestamp snapshot against a 16-bit running timer:

```

; r25:r24 = current_time (16-bit, incremented in interrupt)
; r22:r21 = deadline (16-bit, set when timer was started)
;
; Is current_time >= deadline?

check_deadline:
    cp r21, r22      ; wait – wrong order! must compare pair to pair
    ; Correct:
    cp r24, r22      ; low bytes of current vs deadline
    cpc r25, r23      ; high bytes (deadline high in r23)
    brsh deadline_reached ; current_time >= deadline
    ret
deadline_reached:
    ; ... handle timeout ...
    ret

```

8.11.4 Pattern 4: Saturating 16-bit Addition

Add two 16-bit unsigned values, clamping at 65535:

```

; r25:r24 = sat16_add(r25:r24, r23:r22)
; Result in r25:r24; saturated to 0xFFFF on overflow
sat16_add:
    add r24, r22
    adc r25, r23
    brcc sat16_done ; C=0: no overflow
    ser r24          ; C=1: clamp to 0xFFFF
    ser r25
sat16_done:
    ret

```

8.11.5 Pattern 5: Unsigned Range Check on 16-bit Value

Is a 16-bit value in the range [lo16, hi16]?

```

; Is r25:r24 in [100, 5000]?
; lo16=100 (0x0064), hi16=5000 (0x1388)
;
; Check r25:r24 >= 100
    ldi r22, lo8(100)
    ldi r23, hi8(100)
    cp r24, r22
    cpc r25, r23
    brlo out_of_range ; r25:r24 < 100

; Check r25:r24 <= 5000 (equivalently: r25:r24 < 5001)
    ldi r22, lo8(5001)
    ldi r23, hi8(5001)
    cp r24, r22

```

```

cpc r25, r23
brsh out_of_range      ; r25:r24 >= 5001

; In range [100, 5000]
; ...
rjmp done
out_of_range:
; ...
done:

```

8.12 Instruction Reference Table

The following table collects every unsigned arithmetic and comparison instruction with its operand constraints, cycle count, and which flags it updates. The flag column uses ✓ for updated, × for not touched, and – for forced-clear.

Instruction	Operands	Cycles	H	C	Z	N	S	V
ADD Rd, Rr	r0-r31, r0-r31	1	✓	✓	✓	✓	✓	✓
ADC Rd, Rr	r0-r31, r0-r31	1	✓	✓	✓	✓	✓	✓
SUB Rd, Rr	r0-r31, r0-r31	1	✓	✓	✓	✓	✓	✓
SBC Rd, Rr	r0-r31, r0-r31	1	✓	✓	*	✓	✓	✓
SUBI Rd, K	r16-r31	1	✓	✓	✓	✓	✓	✓
SBCI Rd, K	r16-r31	1	✓	✓	*	✓	✓	✓
ADIW Rd, K	r24/XL/YL/ZL	2	×	✓	✓	✓	✓	✓
SBIW Rd, K	r24/XL/YL/ZL	2	×	✓	✓	✓	✓	✓
INC Rd	r0-r31	1	×	×	✓	✓	✓	✓
DEC Rd	r0-r31	1	×	×	✓	✓	✓	✓
NEG Rd	r0-r31	1	✓	✓	✓	✓	✓	✓
CP Rd, Rr	r0-r31, r0-r31	1	✓	✓	✓	✓	✓	✓
CPC Rd, Rr	r0-r31, r0-r31	1	✓	✓	*	✓	✓	✓
CPI Rd, K	r16-r31	1	✓	✓	✓	✓	✓	✓
CPSE Rd, Rr	r0-r31, r0-r31	1/2	×	×	×	×	×	×
MOV Rd, Rr	r0-r31, r0-r31	1	×	×	×	×	×	×
MOVW Rd, Rr	even, even	1	×	×	×	×	×	×
LDI Rd, K	r16-r31	1	×	×	×	×	×	×

* SBC/SBCI/CPC: only clears Z, never sets it – Z stays 1 if the previous byte was also zero. Read Z only after the last instruction in a chain.

8.13 Common Pitfalls

8.13.1 Pitfall 1: Using INC/DEC in a Multi-byte Carry Chain

As stressed above: INC and DEC do not update C. Mixing them into an ADD+ADC or SUB+SBC chain silently corrupts the carry.

```

; WRONG:
add r24, r22
inc r25      ; r25 += 1 regardless of carry – broken!

; CORRECT:
add r24, r22
adc r25, r23 ; r23 must be zero if you only want to propagate carry

```

If you genuinely want to add just 1 to the high byte only when carry is set:

```
ldi r23, 0
add r24, r22
adc r25, r23      ; r25 += 0 + C – propagates carry only
```

8.13.2 Pitfall 2: High Byte Before Low Byte

```
; WRONG – processes high byte first, carry is 0 at that point:
add r25, r23
adc r24, r22      ; r24 now adds a carry from the wrong byte

; CORRECT – always low byte first:
add r24, r22
adc r25, r23
```

The carry produced by ADC on the low byte has nowhere to go. The mathematical result is wrong for any input where the low byte overflows.

8.13.3 Pitfall 3: SUBI Negation Confusion

The immediate value for SUBI when you want to add is negated:

```
; To add 10 to r16:
subi r16, -10      ; correct – subtract -10 = add 10
subi r16, 10       ; WRONG – this subtracts 10
```

Using a comment documenting the intent helps:

```
subi r16, -10      ; r16 += 10 (no ADDI exists)
```

8.13.4 Pitfall 4: Reading Z After SBC on the Wrong Byte

Z after a multi-byte SBC/CPC chain is only meaningful after the last (most-significant) instruction in the chain:

```
cp r24, r22      ; low byte
cpc r25, r23      ; high byte
; Z is now valid for the 16-bit comparison – read it here
breq equal
; DO NOT read Z from the cp alone
```

8.13.5 Pitfall 5: ADIW Supported Pairs Only

```
adiw r20, 1      ; WRONG – r20 is not in the supported set
adiw r24, 1      ; CORRECT – r24 is one of r24, XL, YL, ZL
```

If the pair is not r24, XL, YL, or ZL, use ADD+ADC or SUBI+SBCI instead.

8.13.6 Pitfall 6: Forgetting SBCI When Using SUBI for 16-bit Constant Add

```
; r25:r24 += 300:
subi r24, lo8(-300)
; MISSING sbci r25, hi8(-300) ← the high byte is never updated!
```


Every SUBI on the low byte of a 16-bit operation must be paired with SBCI on the high byte.

8.14 Summary

8-bit unsigned:

```
ADD Rd, Rr      - add, C=1 on overflow
SUBI Rd, -K     - add immediate K (no ADDI); r16-r31 only
INC Rd         - add 1, but C NOT updated
SUB Rd, Rr     - subtract, C=1 on borrow/underflow
SUBI Rd, K      - subtract immediate; r16-r31 only
DEC Rd         - subtract 1, but C NOT updated
Overflow: C=1 after ADD means result > 255 (wrapped)
Underflow: C=1 after SUB means result < 0 (wrapped)
Saturate: BRCC/SER for clamp-at-255; BRCC/CLR for clamp-at-0
```

8-bit unsigned compare and branch:

```
CP Rd, Rr / CPI Rd, K - compare without storing result
BRLO  $\equiv$  BRCS        - branch if Rd < Rr (C=1)
BRSH  $\equiv$  BRCC        - branch if Rd  $\geq$  Rr (C=0)
BREQ / BRNE        - branch if equal / not equal (Z)
```

16-bit register pairs:

```
Any two adjacent registers; write high:low (e.g. r25:r24)
LDI r24, lo8(K) / LDI r25, hi8(K) - load 16-bit constant
MOVW Rd, Rr - copy pair in 1 cycle (even register numbers)
```

16-bit unsigned add:

```
ADD r24, r22 / ADC r25, r23 - low byte FIRST, then high byte + carry
ADIW Rd, K                  - +=0..63, 2 cycles, r24/XL/YL/ZL only
SUBI r24, lo8(-K) / SBCI r25, hi8(-K) - add immediate to r16-r31 pair
```

16-bit unsigned subtract:

```
SUB r24, r22 / SBC r25, r23 - low byte first
SBIW Rd, K                  - -=0..63, 2 cycles, r24/XL/YL/ZL only
SUBI r24, lo8(K) / SBCI r25, hi8(K) - subtract immediate
```

16-bit unsigned compare:

```
CP r24, r22 / CPC r25, r23 - then BRLO / BRSH / BREQ
Z is valid only after the final instruction in the chain.
```

Key rules:

1. Always process low byte first in multi-byte chains.
 2. INC/DEC do not update C – never use them in a carry chain.
 3. SBC/CPC only clear Z, never set it in the middle of a chain.
 4. ADIW/SBIW: four pairs only (r24, X, Y, Z); K = 0..63.
 5. SUBI Rd, -K to add K (no ADDI exists).
-

8.15 Exercises

1. Write a subroutine `add8_saturate(r16, r17) → r16` that returns `min(r16 + r17, 255)`. Verify with inputs (200, 100) and (100, 50).
2. Write a subroutine `sub8_saturate(r16, r17) → r16` that returns `max(r16 - r17, 0)`. Verify with inputs (10, 30) and (100, 40).

3. Implement an 8-bit modular counter that counts 0→9 then wraps to 0. On each call to `count_step`, increment the counter and set `Z=1` in `SREG` if the counter just wrapped. (Hint: compare with 10 after incrementing.)
4. Write a 16-bit accumulator that sums values arriving one at a time in `r16` (zero-extended to 16-bit). Accumulate into `r25:r24`. Saturate at 65535. How many additions of 255 fit before saturation?
5. Given two 16-bit timestamps `start` (`r23:r22`) and `now` (`r25:r24`), compute `elapsed = now - start`. What happens if `now` has wrapped past 65535 since `start` was recorded? Under what condition does the subtraction still give the correct elapsed time modulo 65536?
6. Write a 16-bit unsigned range check: return (via `Z`) whether `r25:r24` falls in the range [200, 3000] inclusive. Use `CP+CPC` chains. Test with values 199, 200, 1500, 3000, and 3001.
7. Implement `mul8_scale(r16, r17) → r16`: multiply `r16` by the fraction `r17` where `r17` represents 0/255 to 255/255. Return the scaled result in `r16`. (Hint: use `MUL` and take the high byte of the 16-bit product.) This is the 8-bit fraction scaling trick introduced in Chapter 13.
8. Write a subroutine that counts the number of consecutive zero-bytes in an SRAM buffer starting at address in `X` (`r27:r26`), up to a maximum of 255. Return the count in `r16`. Use a 16-bit loop counter for the maximum even if the count return is 8-bit.

Next: Chapter 9 — Signed Arithmetic, Two's Complement, and Sign Extension

Chapter 9

Signed Arithmetic, Two's Complement, and Sign Extension

The previous chapter treated every byte as a number from 0 to 255. That works for sensor raw counts, buffer indices, and packet lengths. It does not work for temperature deltas, motor direction, calibration offsets, or any quantity that can go below zero.

This chapter covers the other half of AVR integer arithmetic: **signed values**. The instructions are mostly the same — ADD, SUB, CP — but the representation changes, new flags become important (V, N, S), the branch instructions change (BRLT and BRGE instead of BRLO and BRSH), and a new operation, sign extension, becomes necessary whenever you widen a signed value.

Everything here applies to 8-bit operations. The sign-extension section shows how to connect an 8-bit signed value to the 16-bit arithmetic in the next chapter.

9.1 Two's Complement from First Principles

Before examining any AVR instruction, it is worth understanding exactly what two's complement is and why it was chosen as the standard representation for signed integers.

9.1.1 The Wrapping Number Line

An 8-bit register can hold 256 distinct bit patterns: 0x00 through 0xFF. In unsigned interpretation these map 1-to-1 to the integers 0 through 255. In signed two's complement interpretation the mapping shifts: the upper half of bit patterns (0x80 through 0xFF) represents negative integers.

The simplest mental model is a clock face, but with 256 positions instead of 12:

0x00 = 0	
0xFF = -1	0x01 = 1
0xFE = -2	0x02 = 2
0xFD = -3	0x03 = 3
.	.
0x81 = -127	0x7F = 127
0x80 = -128	

Moving clockwise adds 1. Moving counter-clockwise subtracts 1. Incrementing 0x7F (127) clockwise reaches 0x80 (−128). Decrementing 0x00 counter-clockwise reaches 0xFF (−1). The number line is a ring; it wraps at both ends.

This wrapping view explains why two's complement addition works without any special-casing: the CPU adds bit patterns modulo 256, and those same patterns happen to represent the correct signed result, as long as the true answer fits in the representable range [−128, +127].

9.1.2 The Full 8-bit Signed Map

Bit pattern	Signed value	Unsigned value
0x00	0	0
0x01	+1	1
0x02	+2	2
...		
0x7E	+126	126
0x7F	+127	127
— sign boundary —		
0x80	−128	128
0x81	−127	129
0x82	−126	130
...		
0xFE	−2	254
0xFF	−1	255

Three facts stand out:

1. **Bit 7 is the sign bit.** Patterns with bit 7 = 0 are non-negative (0 to 127). Patterns with bit 7 = 1 are negative (−128 to −1). The CPU's N flag is simply a copy of bit 7 of the result.
2. **Zero is positive.** The single value 0x00 is the boundary non-negative value. There is no −0.
3. **The range is asymmetric.** There are 128 non-negative values (0 to 127) and 128 negative values (−128 to −1). The maximum positive value is +127, but the minimum negative value is −128, not −127. This asymmetry produces the only overflow case that has no correct answer.

9.1.3 How to Convert a Decimal Negative to Two's Complement

Two steps: take the bit pattern of the absolute value, invert all bits, add 1.

Example: −10 in 8-bit two's complement.

```

10 in binary:  0b00001010  (= 0x0A)
Invert bits:   0b11110101  (= 0xF5)
Add 1:         0b11110110  (= 0xF6)

```

Result: −10 is represented as 0xF6.

Verification: 0x0A + 0xF6 = 0x100. Discard the ninth bit: 0x00. So 10 + (−10) = 0. ✓

This is exactly what the NEG instruction computes. The invert-and-add-1 identity is also why COM Rd followed by INC Rd is equivalent to NEG Rd.

9.1.4 Why the Same Addition Works for Both Signed and Unsigned

The single most important property of two's complement: addition is the same operation for signed and unsigned values.

The 8-bit ADD instruction computes $(Rd + Rr) \bmod 256$ — a simple modular sum. The bit pattern of that sum is the correct two's complement representation of the signed sum, as long as the true mathematical answer falls in $[-128, +127]$.

Example: $(-10) + (-20) = -30$ in signed 8-bit

As bit patterns:

$$0xF6 (= -10) + 0xEA (= -20) = 0x1E0$$

Discard carry (the ninth bit):

$$0xE0 = -32$$

Wait — that is wrong. Let me recalculate.

$$0xF6 + 0xEA:$$

$$\begin{array}{r} F6 \\ + EA \\ ---- \\ 1E0 \end{array}$$

Discard carry $\rightarrow 0xE0 = 0b11100000$

$0xE0$ as signed: bit 7 = 1, so negative.

Two's complement of $0xE0$: invert = $0x1F$, +1 = $0x20 = 32$. So $0xE0 = -32$.

$-10 + (-20) = -30$, but we got -32 . Something is wrong in the example.

Let me redo: -20 in 8-bit two's complement:

$$20 = 0x14$$

$$\text{Invert: } 0xEB$$

$$\text{Add 1: } 0xEC$$

So $-20 = 0xEC$.

$$0xF6 + 0xEC:$$

$$\begin{array}{r} F6 \\ + EC \\ ---- \\ 1E2 \end{array}$$

Discard carry $\rightarrow 0xE2$

$0xE2$ as signed: invert = $0x1D$, +1 = $0x1E = 30 \rightarrow 0xE2 = -30$. ✓

$-10 + (-20) = -30$. Correct!

Example (continued): $(-10) + (-20) = -30$

$$\begin{aligned} &0xF6 (= -10) + 0xEC (= -20) \\ &= 0x1E2 \rightarrow \text{discard carry} \rightarrow 0xE2 (= -30) \quad \checkmark \end{aligned}$$

The carry that is discarded is not a sign of an error here — it is expected and handled by the carry flag ($C=1$ from the ADD). For signed arithmetic you ignore C and watch V instead.

9.2 The Signed Flags: N, V, and S

For signed arithmetic, three flags matter:

9.2.1 N — Negative Flag

N = bit 7 of the result

N=1 means the result's most significant bit is 1, which under signed interpretation means the result is negative. N=0 means the result is zero or positive.

N is simply bit 7 of the 8-bit result, copied into SREG. It is fast and cheap: the CPU reads it directly from the ALU output.

N alone is not always reliable for signed comparisons. When overflow occurs, bit 7 of the result is wrong (the sign flipped incorrectly). That is why the S flag exists.

9.2.2 V — Signed Overflow Flag

V = 1 when the mathematical result exceeds the range $[-128, +127]$

Overflow happens when two same-sign operands produce a result with the opposite sign — which is impossible in correct arithmetic and always indicates wrap.

The formal hardware definition for ADD:

$V_ADD = (Rd7 \text{ AND } Rr7 \text{ AND } !R7) \text{ OR } (!Rd7 \text{ AND } !Rr7 \text{ AND } R7)$

Where Rd7 is bit 7 of the destination operand before the add, Rr7 is bit 7 of the source operand, and R7 is bit 7 of the result. In English:

- **(positive) + (positive) → (negative):** V=1. Two non-negative values cannot produce a negative result in true arithmetic; the sum overflowed +127.
- **(negative) + (negative) → (positive):** V=1. Two negative values cannot produce a zero-or-positive result; the sum underflowed -128.
- **(positive) + (negative) → anything:** V=0. Adding opposite signs always shrinks the magnitude; the result is guaranteed to fit.
- **(negative) + (positive) → anything:** V=0. Same reason.

For SUB and CP, the formula adapts to subtraction but the rule is the same: opposite-sign results from same-sign inputs signal overflow.

Concrete overflow examples:

$+127 + 1 = ?$

$0x7F + 0x01 = 0x80 = 128 \text{ unsigned} = -128 \text{ signed}$

V=1 (positive + positive → negative: overflow)

$-128 - 1 = ?$

$0x80 - 0x01 = 0x7F = 127 \text{ unsigned} = +127 \text{ signed}$

V=1 (negative - positive added extra magnitude: overflow) [sub as add of negated]

$+100 + +100 = ?$

$0x64 + 0x64 = 0xC8 = 200 \text{ unsigned} = -56 \text{ signed}$

V=1 (positive + positive → negative: overflow)

$-100 + -100 = ?$

$0x9C + 0x9C = 0x138 \rightarrow \text{discard carry} \rightarrow 0x38 = +56 \text{ signed}$

V=1 (negative + negative → positive: overflow)

$+50 + -30 = ?$

$0x32 + 0xE2 = 0x114 \rightarrow 0x14 = +20 \text{ signed}$

V=0 (opposite-sign operands never overflow)

9.2.3 S — Sign Flag

$S = N \text{ XOR } V$

The S flag is the “corrected sign bit” for signed comparisons. It accounts for the case where overflow has flipped the result's sign bit to the wrong value.

The clearest way to understand S is through a concrete failure of N:

CP r16, r17 where r16 = 0x80 (-128), r17 = 0x01 (+1)

CPU computes: $0x80 - 0x01 = 0x7F = +127$

N = 0 (result 0x7F has bit 7 = 0 → looks positive)

V = 1 (overflow: $-128 - 1$ should be -129 , wrapped to $+127$)

S = N XOR V = 0 XOR 1 = 1

Correct answer: $-128 < +1$, so r16 < r17. Should produce a "less than" signal.

BRMI: reads N=0 → would NOT branch → WRONG

BRLT: reads S=1 → DOES branch → CORRECT

And the reverse case, where a signed subtraction overflows in the other direction:

CP r16, r17 where r16 = 0x7F (+127), r17 = 0xFF (-1)

CPU computes: $0x7F - 0xFF = 0x80$ (borrow from wrap)

Hmm, more carefully: $0x7F - 0xFF = 0x7F + 0x01 - 0x100 = 0x80$ with C=1

N = 1 (result 0x80 has bit 7 = 1 → looks negative)

V = 1 (overflow: $+127 - (-1) = +128$ exceeds $+127$)

S = N XOR V = 1 XOR 1 = 0

Correct answer: $+127 \geq -1$, so r16 $\geq$ r17. Should NOT branch for less-than.

BRMI: reads N=1 → DOES branch → WRONG

BRLT: reads S=0 → does NOT branch → CORRECT

The rule: use BRLT/BRGE for signed comparisons. Never use BRMI/BRPL for signed less-than/greater-than decisions. BRMI tests raw bit 7; BRLT tests S which is corrected for overflow.

BRMI and BRPL have their own legitimate uses: testing whether a result is simply negative or non-negative when overflow is known impossible, or testing the sign bit of a fresh load that did not go through arithmetic.

9.3 Signed 8-bit Arithmetic Instructions

9.3.1 ADD and ADC — Signed Addition

The ADD and ADC instructions are physically identical to unsigned addition. The same opcodes, the same result bits. Only the flags you check afterward determine whether you are doing signed or unsigned arithmetic.

```
ldi r16, 0x32 ; = +50 signed
ldi r17, 0xEC ; = -20 signed (0xEC = two's complement of 20)
add r16, r17 ; 0x32 + 0xEC = 0x1E → 0x1E = +30 signed ✓
; C=1 (unsigned carry occurred), V=0 (signed OK)
```

Ignore C. For signed arithmetic, C=1 does not mean error — it just records a carry out of the top bit that has no signed meaning. Watch V instead.

```
; Overflow example
ldi r16, 0x64 ; = +100 signed
ldi r17, 0x64 ; = +100 signed
add r16, r17 ; 0x64 + 0x64 = 0xC8 = -56 signed ← wrong!
; C=0, V=1, N=1, S = N XOR V = 0
brvs signed_overflow ; V=1 → overflow detected
```

9.3.2 SUB and SUBI — Signed Subtraction

Again, the same instructions as unsigned subtraction. The hardware cannot tell the difference.

```
ldi r16, 0x0A    ; = +10
ldi r17, 0x1E    ; = +30
sub r16, r17      ; 10 - 30 = -20 → 0xEC
                  ; C=1 (unsigned borrow), V=0 (signed OK), N=1, S=1
```

C=1 from a signed subtraction does not necessarily mean overflow. -20 is a perfectly valid signed result.

```
; Signed subtraction overflow: -128 - 1
ldi r16, 0x80    ; = -128
ldi r17, 0x01    ; = +1
sub r16, r17      ; 0x80 - 0x01 = 0x7F = +127 signed ← wrong!
                  ; C=0, V=1, N=0, S = N XOR V = 1
brvs signed_overflow
```

9.3.3 NEG — Two's Complement Negate

NEG Rd ; Rd ← 0x00 - Rd

NEG computes the additive inverse: the value that, when added back, produces zero. It is a single-instruction “change sign.”

NEG Rd:

Rd = 0x00 - Rd (computed as: COM Rd, then INC Rd)

Flags: H, S, V, N, Z, C

C = 1 if Rd was nonzero; C = 0 if Rd was 0x00

Z = 1 if result is 0x00

V = 1 if Rd = 0x80 (the one unrepresentable case)

N = bit 7 of result

S = N XOR V

The normal cases:

```
ldi r16, 10      ; = +10 = 0x0A
neg r16          ; r16 = 0 - 10 = -10 = 0xF6, C=1, V=0

ldi r16, 0xF6    ; = -10
neg r16          ; r16 = 0 - (-10) = +10 = 0x0A, C=1, V=0

ldi r16, 0       ; = 0
neg r16          ; r16 = 0 - 0 = 0, C=0, Z=1
```

The 0x80 corner case:

```
ldi r16, 0x80    ; = -128 (the most negative 8-bit signed value)
neg r16          ; 0x00 - 0x80 = 0x80 (result wraps!)
                  ; V=1 (overflow), C=1, N=1
                  ; r16 is still 0x80 — unchanged
```

The true mathematical result of $-(-128)$ is +128. But +128 does not exist in 8-bit signed arithmetic (the maximum is +127). The hardware stores 0x80 and sets V=1 to record the failure. If your code calls NEG on a value that might be -128, check V afterward or ensure by design that -128 cannot appear.

9.3.4 Absolute Value

Using NEG to compute $|x|$:

```
; |r16| (signed 8-bit input, unsigned 8-bit result – valid for -127..+127)
sbrc r16, 7      ; if bit 7 clear (non-negative): skip NEG
neg r16         ; bit 7 set: negate to make positive
; r16 now holds |input| (undefined for input = -128; see note above)
```

For safety when -128 is possible:

```
; |r16| with -128 guard
cpi r16, 0x80    ; is it -128?
breq abs_clamp   ; if so, clamp (or error)
sbrc r16, 7
neg r16
rjmp abs_done
abs_clamp:
ldi r16, 127     ; saturate |-128| to 127 (the closest representable value)
abs_done:
```

9.3.5 COM — One's Complement vs NEG

COM Rd inverts all bits: $Rd \leftarrow 0xFF - Rd$. It is bitwise NOT, not arithmetic negation. COM and NEG produce different results for all nonzero values:

```
r16 = 0x05 (= +5):
  COM r16 → 0xFA (= -6 signed) ← NOT +5
  NEG r16 → 0xFB (= -5 signed) ← correct negation
```

COM is for bit manipulation and for building multi-byte NEG sequences. NEG is for signed arithmetic negation. Do not confuse them.

9.4 Detecting and Handling Signed Overflow

9.4.1 Post-operation Overflow Check

After any ADD, ADC, SUB, or SBC that operates on signed values, test V immediately:

```
add r16, r17
brvs overflow_handler ; V=1 → signed overflow
; safe path – result is in [-128, +127]
```

```
sub r16, r17
brvs overflow_handler
```

BRVS branches if $V=1$. BRVC branches if $V=0$.

9.4.2 The Four Overflow Scenarios for 8-bit Signed Addition

For ADD Rd, Rr, $V=1$ only in these two situations — and $V=0$ always when the inputs have opposite signs:

Rd sign	Rr sign	$V=1$ when...
+	+	result > +127 (positive sum too large)
-	-	result < -128 (negative sum too small)
+	-	impossible – V always 0
-	+	impossible – V always 0

Table of the boundary cases:

Operation	Decimal	Hex result	V
+127 + +1	+128	0x80	1 (overflows to -128)
+127 + +127	+254	0xFE	1 (overflows to -2)
-128 + (-1)	-129	0x7F	1 (overflows to +127)
-128 + (-128)	-256	0x00	1 (overflows to 0)
+100 + +50	+150	0x96	1 (overflows to -106)
+100 + (-50)	+50	0x32	0 (opposite signs, safe)
-100 + +50	-50	0xCE	0 (opposite signs, safe)
+64 + +64	+128	0x80	1 (overflows to -128)

9.4.3 Signed Saturating Addition

Clamp the result to $[-128, +127]$ instead of wrapping:

```
; Saturating signed add: r16 = clamp(r16 + r17, -128, +127)
add r16, r17
brvc sat_add_ok          ; V=0: no overflow, result is correct

; Overflow occurred. Determine direction from the sign of one operand.
; If both positive (N=0 in r17 before add), result should be +127.
; If both negative (N=1 in r17 before add), result should be -128.
; Use the sign bit of the source operand to decide.
sbrs r17, 7              ; if r17 was negative (bit 7 set), skip next
ldi r16, 0x7F             ; both positive → clamp to +127
brvc sat_add_ok           ; skip the negative clamp
ldi r16, 0x80             ; both negative → clamp to -128
sat_add_ok:
```

A cleaner approach saves r17's sign before the add:

```
; Saturating signed add: r16 += r17, clamped to [-128, +127]
; Preserves r17.
sat8_sadd:
    mov r18, r17          ; save copy of addend
    add r16, r18
    brvc sat_ok           ; no overflow
    ; overflow: saturate based on the sign of the saved addend
    tst r18
    brmi sat_neg          ; addend was negative → result must clamp to -128
    ldi r16, 0x7F         ; addend was positive → clamp to +127
    ret
sat_neg:
    ldi r16, 0x80         ; clamp to -128
sat_ok:
    ret
```

9.4.4 Signed Saturating Subtraction

```
; Saturating signed subtract: r16 = clamp(r16 - r17, -128, +127)
sat8_ssub:
    mov r18, r17
    sub r16, r18
    brvc ssub_ok
    ; overflow during subtraction: direction is opposite of the subtrahend's sign
    tst r18
    brmi ssub_pos        ; subtrahend was negative → r16 - (neg) went above +127
```

```

    ldi    r16, 0x80      ; subtrahend was positive → r16 - (pos) went below -128
    ret
ssub_pos:
    ldi    r16, 0x7F
ssub_ok:
    ret

```

9.5 Signed Comparison and Branches

9.5.1 The Signed Branch Set

After CP Rd, Rr or CPI Rd, K, these branches test signed relationships:

Instruction	Condition (signed)	Flag tested	Notes
BRLT	Rd < Rr (signed <)	S = 1	S = N XOR V
BRGE	Rd >= Rr (signed >=)	S = 0	S = N XOR V
BRMI	result < 0	N = 1	tests raw bit 7 of result
BRPL	result >= 0	N = 0	tests raw bit 7 of result
BRVS	signed overflow occurred	V = 1	
BRVC	no signed overflow	V = 0	
BREQ	Rd == Rr	Z = 1	same for signed and unsigned
BRNE	Rd != Rr	Z = 0	same for signed and unsigned

9.5.2 BRLT and BRGE in Practice

```

; Signed comparison: branch if r16 < r17 (signed)
ldi r16, 0xF0      ; = -16 signed
ldi r17, 0x10      ; = +16 signed
cp  r16, r17       ; -16 - (+16) = -32; C=1, N=1, V=0, S=1
brlt r16_less      ; S=1 → taken (correct: -16 < +16)

```

```

; Threshold check: if temperature < -10°C, take action
; temperature in r16, signed
ldi r17, 0xF6      ; = -10 (two's complement)
cp  r16, r17
brlt too_cold      ; signed less-than

```

9.5.3 Signed Range Check

Is the signed value in r16 within [-40, +85]?

```

; Range check: is r16 in [-40, +85] (signed)?
ldi r17, 0xD8      ; = -40 (0x100 - 40 = 0xD8)
cp  r16, r17
brlt out_of_range_low ; r16 < -40 (signed)

ldi r17, 85        ; = +85
cp  r16, r17
brgt_pattern:
; No BRGT instruction – need two steps for "strictly greater"
breql in_range     ; equal to 85 is still in range
brlt in_range      ; r16 < 85 → in range
rjmp out_of_range_high ; r16 > 85

```

```

in_range:
    ; ...
    rjmp check_done
out_of_range_low:
out_of_range_high:
check_done:

```

A cleaner form using BRGE for the upper bound:

```

; Range check: is r16 in [-40, +85] (signed)?
ldi r17, 0xD8          ; = -40
cp r16, r17
brlt range_fail_signed ; r16 < -40

ldi r17, 86            ; upper + 1 = 86
cp r16, r17
brge range_fail_signed ; r16 >= 86

; in range [-40, +85]
rjmp range_pass_signed
range_fail_signed:
range_pass_signed:

```

The upper bound uses 86 (not 85) because BRGE tests $\geq$. The check $r16 \geq 86$ is equivalent to $r16 > 85$.

9.5.4 BRMI vs BRLT: When They Differ

BRMI tests N (raw bit 7 of result). BRLT tests S (N XOR V). They disagree when overflow occurs:

Case 1: $r16 = -128$ (0x80), $r17 = +1$ (0x01)
 CP $r16, r17$ computes $0x80 - 0x01 = 0x7F$
 N=0 (result bit 7 is 0, looks positive)
 V=1 (overflow: $-128 - 1$ should be -129)
 S = $0 \text{ XOR } 1 = 1$

BRMI checks N=0 → does NOT branch → says " $r16 \geq r17$ " → WRONG ($-128 < +1$)
 BRLT checks S=1 → DOES branch → says " $r16 < r17$ " → CORRECT

Case 2: $r16 = +127$ (0x7F), $r17 = -1$ (0xFF)
 CP $r16, r17$ computes $0x7F - 0xFF$
 $0x7F - 0xFF$: borrow from 0x80 gives 0x80, C=1
 N=1 (result bit 7 is 1, looks negative)
 V=1 (overflow: $127 - (-1) = +128$, exceeds +127)
 S = $1 \text{ XOR } 1 = 0$

BRMI checks N=1 → DOES branch → says " $r16 < r17$ " → WRONG ($+127 > -1$)
 BRLT checks S=0 → does NOT branch → says " $r16 \geq r17$ " → CORRECT

Practical rule: - Use BRMI/BRPL only to test whether a freshly computed or loaded value is negative or non-negative — when no overflow is possible or expected. - Use BRLT/BRGE after any CP/CPI/SUB where the operands might have extreme signed values that cause overflow.

When in doubt, always use BRLT/BRGE. They are always correct for signed comparisons. BRMI/BRPL are a shortcut valid only in specific circumstances.

9.6 Sign Extension

9.6.1 The Problem: Widening a Signed Value

When you compute with 16-bit values (addresses, timer counts, multi-byte results), you often need to combine an 8-bit signed value with a 16-bit quantity. Before doing so, you must **sign-extend** the 8-bit value to 16 bits.

Sign extension preserves the mathematical signed value when increasing the bit width. The rule: fill the new high bits with copies of the sign bit.

8-bit signed	16-bit sign-extended	
0x05 (= +5)	0x0005 (= +5)	high byte = 0x00 (sign bit was 0)
0x7F (+127)	0x007F (+127)	high byte = 0x00
0xFF (= -1)	0xFFFF (= -1)	high byte = 0xFF (sign bit was 1)
0x80 (-128)	0xFF80 (-128)	high byte = 0xFF
0xD8 (= -40)	0xFFD8 (= -40)	high byte = 0xFF

9.6.2 Why Zero Extension Fails for Signed Values

A common mistake is zero-extending a signed value — clearing the high byte to 0x00 regardless of the value's sign:

```
; WRONG — zero extension applied to signed value:
ldi r16, 0xFF      ; = -1 signed
clr r17             ; r17:r16 = 0x00FF = +255 — mathematically wrong!
```

0x00FF as a 16-bit signed value is +255. But -1 as a 16-bit signed value is 0xFFFF. Zero extension changed the mathematical meaning.

```
; WRONG again:
ldi r16, 0xD8      ; = -40 signed
clr r17             ; r17:r16 = 0x00D8 = +216 — completely wrong!
```

Zero extension is correct only for unsigned values (see Chapter 8). For signed values, you must sign-extend.

9.6.3 The Sign Extension Pattern

```
; Sign-extend r16 (8-bit signed) into r17:r16 (16-bit signed)
; After: r17 = 0x00 if r16 was non-negative, 0xFF if r16 was negative
clr r17             ; assume non-negative: high byte = 0
sbrc r16, 7         ; if bit 7 is CLEAR (non-negative): skip the SER
ser r17             ; bit 7 set: value is negative, high byte = 0xFF
; r17:r16 is now the correct 16-bit signed representation
```

SBRC Rr, b skips the next instruction if bit b of register Rr is **clear** (0). So sbrc r16, 7 skips ser r17 when bit 7 of r16 is 0 (non-negative). When bit 7 is 1, SBRC does not skip and SER loads 0xFF into r17.

Three cycles in the non-negative path (CLR + SBRC skip + nothing), three cycles in the negative path (CLR + SBRC no-skip + SER). Balanced and branchless.

Trace:

```
Input r16 = 0x05 (+5):
CLR r17    → r17 = 0x00
SBRC r16, 7: bit 7 of 0x05 = 0 → skip SER
Result: r17:r16 = 0x0005 = +5 ✓
```

```
Input r16 = 0xFF (-1):
  CLR r17    → r17 = 0x00
  SBRC r16, 7: bit 7 of 0xFF = 1 → do NOT skip
  SER r17    → r17 = 0xFF
  Result: r17:r16 = 0xFFFF = -1 ✓
```

```
Input r16 = 0x80 (-128):
  CLR r17    → r17 = 0x00
  SBRC r16, 7: bit 7 of 0x80 = 1 → do NOT skip
  SER r17    → r17 = 0xFF
  Result: r17:r16 = 0xFF80 = -128 ✓
```

9.6.4 Alternative Sign Extension Using SBRS

If you prefer to branch on the set case:

```
clr    r17
sbrs   r16, 7      ; if bit 7 SET: execute next instruction (then fall through)
rjmp   sext_done   ; bit 7 clear: jump to done (no sign extension needed)
ser    r17         ; bit 7 set: sign-extend
sext_done:
```

This takes an extra word and one extra cycle in the non-negative path. The CLR + SBRC + SER form is more concise and preferred.

9.6.5 Sign Extension as a Subroutine

```
/*
 * sext8to16 – sign-extend r16 to r17:r16
 * Input:  r16 = 8-bit signed value
 * Output: r17:r16 = 16-bit signed value
 * Clobbers: r17
 * Cycles: 3 (non-negative), 3 (negative), +4 for rcall/ret
 */
sext8to16:
  clr    r17
  sbrc   r16, 7
  ser    r17
  ret
```

9.6.6 Sign Extension Before 16-bit Arithmetic

After sign-extending, you can use the 16-bit unsigned arithmetic from Chapter 5a, treating the 16-bit pair as signed. The same ADD+ADC and SUB+SBC chains work correctly for signed 16-bit arithmetic, with V set appropriately at the end by the high-byte instruction.

```
; signed 8-bit r16 + signed 8-bit r18 → signed 16-bit result in r25:r24

; Sign-extend r16 → r17:r16
  clr    r17
  sbrc   r16, 7
  ser    r17

; Sign-extend r18 → r19:r18
  clr    r19
  sbrc   r18, 7
  ser    r19
```

```

; 16-bit signed add (same instructions as unsigned)
    add    r16, r18
    adc    r17, r19      ; V=1 if signed 16-bit overflow occurred

; Copy to return registers
    movw   r24, r16      ; r25:r24 = result

```

The result in r25:r24 is a signed 16-bit value. V=1 if the signed 16-bit result overflowed; otherwise the mathematical value is exact.

9.7 Reading Flags Produced by Signed Operations

9.7.1 The N Flag After a Load or Move

After LDS, LD, MOV, or other data-transfer instructions that **do not affect SREG**, N is whatever it was before. These instructions are not arithmetic — they do not update flags. You cannot test N after a load.

Use TST to set flags from a loaded value:

```

lds    r16, temperature    ; load signed temperature – flags NOT updated
tst    r16                  ; Rd AND Rd → sets N, Z, clears V; Rd unchanged
brmi   is_negative         ; N=1 means r16 < 0 (since TST clears V, S=N)

```

After TST, V=0, so S = N XOR 0 = N. This means BRMI and BRLT behave identically after TST. Using BRMI after TST is therefore correct.

9.7.2 The Z Flag and Signed Zero

Z=1 means the result is exactly zero, regardless of signed or unsigned interpretation. Zero is non-negative in signed arithmetic. After TST, Z=1 and N=0 when the value is zero:

```

tst    r16
breq   is_zero             ; Z=1: r16 == 0
brmi   is_negative        ; N=1, Z=0: r16 < 0
; here: r16 > 0 (positive and non-zero)

```

This three-way dispatch (negative / zero / positive) is a common pattern in signed firmware.

9.7.3 Flags After Logical Operations

AND, ANDI, OR, ORI, EOR, COM, TST all force V=0. This means:

- After any of these, S = N XOR 0 = N.
- BRLT and BRMI produce identical results after a logical op.
- BRGE and BRPL produce identical results after a logical op.

This is fine for sign tests on fresh values, but do not run a logical instruction between a signed arithmetic result and the V-based branch that depends on it.

9.8 Common Pitfalls

9.8.1 Pitfall 1: Using BRLO Instead of BRLT for Signed Comparisons

```
ldi r16, 0xF0      ; = -16 signed
ldi r17, 0x10      ; = +16 signed
cp r16, r17        ; C=0 (no unsigned borrow - 0xF0 > 0x10 unsigned)
brlo wrong_branch  ; C=0 → NOT taken → says "r16 >= r17" → WRONG!
brlt correct_branch ; S=1 → taken → says "r16 < r17" → CORRECT
```

The CPU computed $0xF0 - 0x10 = 0xE0$. No borrow occurred, so $C=0$. To the unsigned view, $240 > 16$. But in signed arithmetic, $-16 < +16$, which is captured by $S=1$.

Always use BRLT/BRGE for signed comparisons.

9.8.2 Pitfall 2: ASR vs LSR for Signed Right Shift

To divide a signed value by 2, use ASR (arithmetic shift right), not LSR (logical shift right). ASR copies bit 7 (the sign bit) into the vacated position. LSR inserts a 0, which incorrectly makes negative values positive:

```
ldi r16, 0xFE      ; = -2 signed
lsr r16            ; r16 = 0x7F = +127 ← WRONG (unsigned right shift)
                  ; LSR inserts 0 at bit 7 regardless of sign

ldi r16, 0xFE      ; = -2 signed
asr r16            ; r16 = 0xFF = -1 ← CORRECT (-2 / 2 = -1)
                  ; ASR copies bit 7 into bit 7 (sign-extended shift)

ldi r16, 0xFC      ; = -4 signed
asr r16            ; r16 = 0xFE = -2 (-4 / 2 = -2) ✓
asr r16            ; r16 = 0xFF = -1 (-2 / 2 = -1) ✓
```

ASR rounds toward negative infinity (floor division). This differs from C's signed right shift behaviour, which is implementation-defined, but GCC on AVR generates ASR for its signed divisions by powers of two.

9.8.3 Pitfall 3: Forgetting NEG 0x80

Code that unconditionally calls NEG on a signed value is subtly broken:

```
; BROKEN – returns wrong result for input = -128
abs8:
    sbrc r16, 7
    neg r16      ; NEG 0x80 = 0x80, V=1 – still -128!
    ret
```

If your domain guarantees the input is in $[-127, +127]$, this is safe. If -128 can appear, guard it explicitly (see the absolute value section above).

9.8.4 Pitfall 4: Using COM as a Negate

COM is one's complement (flip all bits). NEG is two's complement (flip all bits and add 1). COM is not a sign change:

```
ldi r16, 0x01      ; = +1
com r16            ; r16 = 0xFE = -2 ← NOT -1!
neg r16            ; r16 = 0xFF = -1 ← correct negate of +1
```


9.8.5 Pitfall 5: Inserting Flag-Altering Instructions Between Arithmetic and Branch

```
add r16, r17
mov r18, r16      ; safe: MOV does not alter flags
brvs overflow     ; V still valid ✓

add r16, r17
andi r18, 0x01    ; ANDI clears V! V=0 after this regardless of add result
brvs overflow     ; V is now 0, overflow is never detected ✗
```

Check every instruction between the arithmetic and its branch for flag effects. The safest approach: keep the branch immediately after the arithmetic, or save intermediate results to a scratch register without using logical ops.

9.8.6 Pitfall 6: Sign Extension After Narrowing Operations

If you mask an 8-bit value and then sign-extend, the mask may clear the sign bit, corrupting the extension:

```
; WRONG – mask destroys the sign bit before sign extension
andi r16, 0x7F      ; clears bit 7 – value is now always non-negative!
clr r17
sbrc r16, 7         ; bit 7 is always 0 after ANDI 0x7F → always skips SER
ser r17             ; never reached
```

Sign-extend first, then mask, or design the mask to preserve bit 7.

9.9 Instruction Reference for Signed Arithmetic

Instruction	Action	Flags set/cleared	Notes
ADD Rd, Rr	$Rd = Rd + Rr$	H,S,V,N,Z,C	V=1 on signed overflow
SUB Rd, Rr	$Rd = Rd - Rr$	H,S,V,N,Z,C	V=1 on signed overflow
SUBI Rd, K	$Rd = Rd - K$	H,S,V,N,Z,C	r16-r31 only
ADC Rd, Rr	$Rd = Rd + Rr + C$	H,S,V,N,Z,C	for multi-byte chains
SBC Rd, Rr	$Rd = Rd - Rr - C$	H,S,V,N,Z,C	for multi-byte chains
NEG Rd	$Rd = 0 - Rd$	H,S,V,N,Z,C	V=1 only if $Rd=0x80$
COM Rd	$Rd = 0xFF - Rd$	C=1, S,V=0,N,Z	bitwise NOT, not negate
ASR Rd	$Rd = Rd \gg 1$ (signed)	S,V,N,Z,C	sign bit preserved
CP Rd, Rr	flags from $Rd - Rr$	H,S,V,N,Z,C	no result stored
CPI Rd, K	flags from $Rd - K$	H,S,V,N,Z,C	r16-r31 only
TST Rd	flags from $Rd \text{ AND } Rd$	S,V=0,N,Z	Rd unchanged; C unchanged

Branch	Condition	Flag	Use after
BRLT	$Rd < Rr$ signed	S=1	CP/CPI for signed less-than
BRGE	$Rd \geq Rr$ signed	S=0	CP/CPI for signed greater-or-equal
BRMI	result < 0	N=1	TST/arithmetic when overflow impossible
BRPL	result ≥ 0	N=0	TST/arithmetic when overflow impossible
BRVS	overflow	V=1	after signed ADD/SUB to detect overflow
BRVC	no overflow	V=0	after signed ADD/SUB to confirm safe result
BREQ	equal	Z=1	signed or unsigned
BRNE	not equal	Z=0	signed or unsigned

9.10 Worked Examples

9.10.1 Example 1: Signed Temperature Comparison

A sensor returns a signed 8-bit Celsius reading (−40 to +85). Take action if it exceeds a signed threshold stored in SRAM.

```
/*
 * check_temp – compare temperature against threshold
 * Inputs:  r16 = current temperature (signed 8-bit)
 *          r17 = threshold (signed 8-bit)
 * Output:  Z=1 if temp == threshold
 *          C=0 and Z=0 if temp > threshold (use brge after call)
 *          C=1 via S flag interpretation: use brlt
 * Use:  rcall check_temp; brlt too_cold; brge warm_enough
 */
check_temp:
    cp    r16, r17    ; signed compare: sets S for BRLT/BRGE
    ret
```

Calling code:

```
lds    r16, temperature ; signed reading
ldi    r17, 0xF6        ; threshold: −10°C
rcall  check_temp
brlt   heater_on        ; temperature < −10 → heat
rjmp   heater_off
```

9.10.2 Example 2: Signed Delta Between Two Readings

Compute the difference between two signed readings and take action based on the magnitude and direction:

```
/*
 * signed_delta – r16 = new – old (signed, may overflow for extreme values)
 * Input:  r16 = new reading, r17 = old reading (both signed 8-bit)
 * Output: r16 = signed delta (new – old)
 *          V=1 if overflow (new and old differ by more than 127/128)
 */
signed_delta:
    sub    r16, r17    ; r16 = new – old
    ret

; Example use: was the change positive (warming) or negative (cooling)?
lds    r16, temp_new
lds    r17, temp_old
rcall  signed_delta
brvs   delta_overflow ; readings too far apart to trust the delta
brmi   cooling         ; delta < 0 → temperature dropped
breq   stable         ; delta = 0 → no change
; here: delta > 0 → temperature rose
warming:
```

9.10.3 Example 3: Signed Absolute Value

```
/*
 * abs8_safe – |r16|, handles −128 by saturating to +127
 * Input:  r16 = signed 8-bit value (−128..+127)
 * Output: r16 = |r16| (0..+127, or +127 if input was −128)
```

```

/*
abs8_safe:
    sbrc    r16, 7          ; bit 7 clear → non-negative → no action
    rjmp    do_neg
    ret
do_neg:
    cpi     r16, 0x80       ; is it the special -128 case?
    brne    simple_neg
    ldi     r16, 127        ; -128 has no exact positive form: saturate to +127
    ret
simple_neg:
    neg     r16             ; all other negatives: negate normally
    ret

```

9.10.4 Example 4: Three-Way Signed Dispatch

Test a signed value and dispatch to one of three handlers:

```

/*
* signed_dispatch – branch on sign of r16
*   Input:  r16 = signed 8-bit value
*   Effect: branches to negative_handler, zero_handler, or positive_handler
*/
signed_dispatch:
    tst     r16             ; sets N and Z; clears V; C unchanged
    breq    zero_handler    ; Z=1: r16 == 0
    brmi    neg_handler     ; N=1 (V=0 from TST so S=N): r16 < 0
    ; fall through: r16 > 0
    rjmp    pos_handler

```

9.10.5 Example 5: Sign Extension into a 16-bit Accumulator

Accumulate signed 8-bit ADC readings into a 16-bit signed sum:

```

/*
* accum_signed – add signed 8-bit r16 to 16-bit signed sum in r25:r24
*   Input:  r16 = signed 8-bit sample, r25:r24 = running sum (16-bit signed)
*   Output: r25:r24 = updated sum
*   Does not saturate – caller must check for overflow if needed.
*/
accum_signed:
    clr     r17
    sbrc    r16, 7          ; sign-extend r16 into r17:r16
    ser     r17
    add     r24, r16
    adc     r25, r17         ; V=1 if 16-bit signed overflow occurred
    ret

```

9.11 Summary

Two's complement:

8-bit signed range: -128 (0x80) to +127 (0x7F).

Bit 7 = sign bit; 0 = non-negative, 1 = negative.

Convert negative N: invert all bits of |N|, then add 1.

NEG 0x80 = 0x80 (only value that cannot be negated – V=1 to record this).

Flags for signed arithmetic:

- N = bit 7 of result (raw sign bit – unreliable when overflow occurred).
- V = signed overflow (result left [-128, +127]). Set only when same-sign operands produce opposite-sign result.
- S = N XOR V (corrected sign – reliable for all signed comparisons).

Signed instructions:

- ADD, SUB, ADC, SBC, SUBI, SBCI – same opcodes as unsigned, V flag differs.
- NEG – two's complement negate; handle 0x80 corner case explicitly.
- ASR – arithmetic right shift; preserves sign bit (signed ÷2).
- COM – bitwise NOT; NOT a sign change. Always sets C=1.

Signed branches (after CP/CPI/SUB):

- BRLT: Rd < Rr signed (reads S=N⊕V) – always correct
- BRGE: Rd ≥ Rr signed (reads S=N⊕V) – always correct
- BRMI: result bit 7 = 1 (reads N) – correct only when V=0 (no overflow)
- BRPL: result bit 7 = 0 (reads N) – correct only when V=0 (no overflow)
- BRVS/BRVC: overflow detection after arithmetic
- BREQ/BRNE: equal/not-equal (same for signed and unsigned)

Rule: use BRLT/BRGE for signed less-than/greater-than comparisons.
use BRMI/BRPL only after TST or when overflow is guaranteed absent.

Sign extension (8-bit signed → 16-bit signed):

```
clr Rh
sbrc Rl, 7      ; skip SER if bit 7 is clear (non-negative)
ser Rh          ; bit 7 set: fill high byte with 0xFF
```

Common pitfalls:

- BRL0/BRSH are unsigned – never use them for signed comparisons.
- LSR is unsigned right shift – use ASR for signed ÷2.
- COM is bitwise NOT – use NEG for arithmetic sign change.
- Zero extension is wrong for signed widening – always sign-extend.
- Logical ops (AND, OR, EOR) force V=0 – do not insert them before BRVS.
- NEG 0x80 = 0x80 with V=1 – guard this case when input range includes -128.

9.12 Exercises

- Predict the result, V, N, and S flags after each operation. Do this before assembling, then verify:
 - ldi r16, 0x60; ldi r17, 0x60; add r16, r17 (= +96 + +96)
 - ldi r16, 0x90; ldi r17, 0x90; add r16, r17 (= -112 + -112)
 - ldi r16, 0x7F; ldi r17, 0x80; add r16, r17 (= +127 + -128)
 - ldi r16, 0x80; subi r16, 1 (= -128 - 1)
- Write a signed comparison subroutine `cmp_s8(r16, r17)` that returns:
 - r18 = 0xFF (-1) if r16 < r17 signed
 - r18 = 0x00 (0) if r16 == r17
 - r18 = 0x01 (+1) if r16 > r17 signed
- Implement a signed 8-bit division by 4 using ASR. Verify:
 - 100 / 4 = 25
 - -100 / 4 = -25
 - -1 / 4 = 0 (floor toward -∞)
 - -4 / 4 = -1 Why does ASR give floor division rather than truncation toward zero?
- Sign-extend the four values 0x00, 0x7F, 0x80, and 0xFF from 8-bit to 16-bit using the CLR + SBRC + SER pattern. Write out the expected r17:r16 for each case before running

the code.

5. The following code tries to check whether a signed value is in $[-50, +50]$ but contains a bug. Find and fix it:

```
cpi r16, 0xCE    ; -50 in two's complement
brlo too_small  ; WRONG: this is an unsigned branch
cpi r16, 51
brsh too_large  ; WRONG
```

6. Implement `accum_signed` (from Worked Example 5) with 16-bit signed saturation: clamp the sum to $[-32768, +32767]$ instead of wrapping. What flags do you check and what values do you load for each clamp direction?
7. Explain why BRLT uses $S = N \text{ XOR } V$ rather than N alone. Construct a concrete example (specific register values and a CP instruction) where BRLT gives the correct answer but BRMI gives the wrong one.
8. Write a subroutine `signed_min(r16, r17) → r16` that returns the smaller of two signed 8-bit values. Use CP and one conditional move pattern (you will need to copy r17 conditionally — there is no CMOV on AVR, so use a BRGE + MOV combination).

Next: Chapter 10 — Multi-byte Addition, Subtraction, Comparison, and Negation

Chapter 10

Multi-byte Addition, Subtraction, Comparison, and Negation

Chapters 8 and 9 established the rules for 8-bit and 16-bit arithmetic. This chapter takes the same carry-chain principle and pushes it to 24 bits, 32 bits, and arbitrary width. The operations — addition, subtraction, comparison, and negation — stay the same. What changes is understanding exactly how the carry flag threads across three or more bytes, how the zero flag behaves in long chains, and how to negate multi-byte two's complement values correctly.

The chapter covers unsigned and signed interpretations throughout, addresses the SRAM storage conventions that govern multi-byte variable layout, and closes with practical sub-routines for 32-bit counters, accumulators, and comparisons.

10.1 The Carry Chain Principle

Chapter 8 introduced the two-byte carry chain for 16-bit arithmetic:

```
add  r24, r22      ; low bytes: r24 = r24 + r22, C updated
adc  r25, r23      ; high bytes: r25 = r25 + r23 + C
```

The principle behind this is universal: a binary addition or subtraction of any width is a sequence of 8-bit operations, each feeding its carry or borrow into the next byte via the C flag.

The invariant is simple:

```
First byte:  ADD or SUB  – consumes no carry, produces a carry
Middle bytes: ADC or SBC – consumes carry from the byte below, produces a carry
Last byte:   ADC or SBC  – consumes carry from the byte below; C/V after this
                                instruction reflect the entire operation
```

The CPU does not need to know how many bytes you intend to chain. It computes the 8-bit result and sets C. The next ADC or SBC reads that C. You decide when to stop. That is the entire mechanism.

Three rules that never change:

1. Always start at the least-significant byte.
2. The first byte uses ADD or SUB. Every subsequent byte uses ADC or SBC.
3. Read C and V only after the final instruction in the chain.

10.2 24-bit Unsigned Arithmetic

10.2.1 Register Layout for 24-bit Values

A 24-bit value occupies three registers. By convention the notation is:

Rh:Rm:Rl – high : middle : low

The numeric value is:

$$\text{value} = (\text{Rh} \times 65536) + (\text{Rm} \times 256) + \text{Rl}$$

A 24-bit unsigned value spans 0 to 16,777,215 (0xFFFFFFFF). Common uses: a 24-bit timer tick counter, a cumulative distance in micrometres, or a signed audio sample expressed in 24-bit linear PCM.

No single-register constraint applies: any three registers can hold a 24-bit value. A common arrangement in GCC-generated code is r22:r21:r20 or r24:r23:r22.

10.2.2 24-bit Addition: ADD + ADC + ADC

```
; 24-bit unsigned add: r26:r25:r24 = r26:r25:r24 + r22:r21:r20
add r24, r20      ; byte 0 (low):    r24 = r24 + r20,      C updated
adc r25, r21      ; byte 1 (mid):    r25 = r25 + r21 + C,  C updated
adc r26, r22      ; byte 2 (high):   r26 = r26 + r22 + C,  C final
```

Three instructions. The first is ADD; the rest are ADC. The order is fixed: low byte first, high byte last.

Worked trace: 0x00FFA0 + 0x000160 = 0x010100

r26:r25:r24 = 0x00 : 0xFF : 0xA0

r22:r21:r20 = 0x00 : 0x01 : 0x60

```
ADD r24, r20:  0xA0 + 0x60 = 0x100 → r24 = 0x00, C = 1
ADC r25, r21:  0xFF + 0x01 + 1(C) = 0x101 → r25 = 0x01, C = 1
ADC r26, r22:  0x00 + 0x00 + 1(C) = 0x01 → r26 = 0x01, C = 0
```

Result: r26:r25:r24 = 0x01:0x01:0x00 = 0x010100 = 65792 ✓

After the chain: - C = 0: result fits in 24 bits - C = 1: unsigned overflow — result exceeded 0xFFFFFFFF and wrapped

10.2.3 24-bit Subtraction: SUB + SBC + SBC

```
; 24-bit unsigned subtract: r26:r25:r24 = r26:r25:r24 - r22:r21:r20
sub r24, r20     ; byte 0 (low):    r24 = r24 - r20,      C=borrow
sbc r25, r21     ; byte 1 (mid):    r25 = r25 - r21 - C
sbc r26, r22     ; byte 2 (high):   r26 = r26 - r22 - C,  C final
```

Worked trace: 0x010000 – 0x000001 = 0x00FFFF (no overflow)

r26:r25:r24 = 0x01 : 0x00 : 0x00

r22:r21:r20 = 0x00 : 0x00 : 0x01

```
SUB r24, r20:  0x00 - 0x01 → borrow → r24 = 0xFF, C = 1
SBC r25, r21:  0x00 - 0x00 - 1(C) → borrow → r25 = 0xFF, C = 1
SBC r26, r22:  0x01 - 0x00 - 1(C) → 0x00 → r26 = 0x00, C = 0
```

Result: r26:r25:r24 = 0x00:0xFF:0xFF = 0x00FFFF = 65535 ✓

After the chain: - C = 0: result is non-negative (no borrow) - C = 1: unsigned underflow
— result went below 0 and wrapped

10.2.4 Unsigned Overflow and Underflow Detection

```
; After 24-bit add:
add r24, r20
adc r25, r21
adc r26, r22
brcs overflow_24    ; C=1 after final ADC → result > 0xFFFFFF

; After 24-bit subtract:
sub r24, r20
sbc r25, r21
sbc r26, r22
brcs underflow_24   ; C=1 after final SBC → result < 0
```

10.3 32-bit Unsigned Arithmetic

32-bit values appear constantly in embedded firmware: millisecond uptime counters, cumulative pulse counts from motor encoders, 32-bit CRC accumulators, and Unix timestamps. At a 1 ms tick rate, a 16-bit counter rolls over in 65 seconds; a 32-bit counter rolls over in 49 days.

10.3.1 Register Layout for 32-bit Values

Four registers. Notation: Rhh:Rhl:Rlh:Rll (highest byte to lowest byte):

value = (Rhh × 16777216) + (Rhl × 65536) + (Rlh × 256) + Rll

GCC returns 32-bit results in r25:r24:r23:r22 (most significant in r25). General temporaries often use r19:r18:r17:r16.

10.3.2 32-bit Addition: ADD + ADC + ADC + ADC

```
; 32-bit unsigned add: r25:r24:r23:r22 += r19:r18:r17:r16
add r22, r16      ; byte 0 (lowest):
adc r23, r17      ; byte 1
adc r24, r18      ; byte 2
adc r25, r19      ; byte 3 (highest): C=1 on unsigned overflow
```

The pattern is uniform: one ADD followed by N−1 ADCs, lowest byte first.

Worked trace: 0x00FFFFFF + 0x00000001 = 0x01000000

r25:r24:r23:r22 = 0x00 : 0xFF : 0xFF : 0xFF
r19:r18:r17:r16 = 0x00 : 0x00 : 0x00 : 0x01

ADD r22, r16: 0xFF + 0x01 = 0x100 → r22 = 0x00, C = 1
ADC r23, r17: 0xFF + 0x00 + 1 = 0x100 → r23 = 0x00, C = 1
ADC r24, r18: 0xFF + 0x00 + 1 = 0x100 → r24 = 0x00, C = 1
ADC r25, r19: 0x00 + 0x00 + 1 = 0x01 → r25 = 0x01, C = 0

Result: r25:r24:r23:r22 = 0x01000000 ✓

10.3.3 32-bit Subtraction: SUB + SBC + SBC + SBC

```
; 32-bit unsigned subtract: r25:r24:r23:r22 -= r19:r18:r17:r16
sub  r22, r16      ; byte 0 (lowest)
sbc  r23, r17      ; byte 1
sbc  r24, r18      ; byte 2
sbc  r25, r19      ; byte 3 (highest): C=1 on unsigned underflow
```

10.3.4 32-bit Signed Overflow

After a 32-bit chain, the final ADC or SBC instruction sets V for signed overflow. V=1 means the signed mathematical result fell outside $[-2,147,483,648, +2,147,483,647]$.

```
; Signed overflow detection after 32-bit add:
add  r22, r16
adc  r23, r17
adc  r24, r18
adc  r25, r19
brvs signed_overflow_32 ; V=1 → result left signed 32-bit range
```

10.4 Extending to N Bytes

The pattern generalises linearly. For an N-byte add:

```
add  R_byte0, R_other_byte0 ; first byte: ADD
adc  R_byte1, R_other_byte1 ; all subsequent bytes: ADC
adc  R_byte2, R_other_byte2
; ... repeat for each byte ...
adc  R_byteN-1, R_other_byteN-1 ; last byte: ADC, C final here
```

For an N-byte subtract, replace ADD with SUB and all ADC with SBC.

10.4.1 8-byte (64-bit) Addition

A 64-bit counter fits eight registers. The pattern extends straightforwardly:

```
; 64-bit add: r7:r6:r5:r4:r3:r2:r1_lo:r0_lo += r15:r14:r13:r12:r11:r10:r9:r8
; (using r0–r7 for accumulator, r8–r15 for addend)
; Note: r1 must be zero per GCC ABI – use r0 if standalone assembly only
add  r0, r8
adc  r1, r9
adc  r2, r10
adc  r3, r11
adc  r4, r12
adc  r5, r13
adc  r6, r14
adc  r7, r15
```

AVR has 32 general-purpose 8-bit registers. A 64-bit counter needs 8 of them. A 32-bit counter is far more common in practice, but the mechanism is the same.

10.4.2 Adding a Constant to an N-byte Value

For small constants (0–63) on a supported register pair, ADIW handles two bytes in one instruction. For wider constants or unsupported pairs, load the constant and use ADD+ADC:

```

; 32-bit: r25:r24:r23:r22 += 1 (increment)
; Option 1: INC chain (only for +1, uses Z not C – see pitfalls)
; Option 2: add via zero registers
clr  r16          ; r16 = 0
ldi  r17, 1       ; r17 = 1
add  r22, r17     ; low byte += 1
adc  r23, r16     ; middle bytes += 0 + carry
adc  r24, r16
adc  r25, r16

```

For the common += 1 case on a 32-bit value where the low word is in a supported ADIW pair, a faster approach avoids loading a constant:

```

; 32-bit increment: r25:r24:r23:r22 += 1
; r23:r22 is not an ADIW pair, but r25:r24 is – must use ADD+ADC approach
ldi  r16, 1
ldi  r17, 0
add  r22, r16
adc  r23, r17
adc  r24, r17
adc  r25, r17

```

For adding a 32-bit constant to a 32-bit value in r16–r31 registers, load the constant into a temporary quad and use ADD+ADC:

```

; r25:r24:r23:r22 += 86400 (seconds per day, for timestamp arithmetic)
; 86400 = 0x00015180
ldi  r16, lo8(86400) ; r16 = 0x80
ldi  r17, byte2(86400) ; r17 = 0x51
ldi  r18, byte3(86400) ; r18 = 0x01
ldi  r19, byte4(86400) ; r19 = 0x00
add  r22, r16
adc  r23, r17
adc  r24, r18
adc  r25, r19

```

The assembler expression `byte2(K)` extracts bits 15:8, `byte3(K)` extracts bits 23:16, and `byte4(K)` extracts bits 31:24 of a 32-bit constant K.

10.5 The Zero Flag in Multi-byte Chains

10.5.1 The SBC and CPC Zero-Flag Rule

The SBC (Subtract with Carry) and CPC (Compare with Carry) instructions follow a special rule for the Z flag that is different from every other instruction:

SBC and CPC only clear Z. They never set Z.

This means: if Z was already 0 (previous byte was nonzero), Z stays 0 after SBC. If Z was already 1 (previous byte was zero), SBC leaves Z=1 only if the current byte also produces a zero result.

The hardware rationale: Z can be used as a “still all zeros so far” accumulator across a multi-byte chain. After the final SBC or CPC, Z=1 means every byte in the chain computed to zero — the entire multi-byte value is zero.

Compare this to ADC, which sets Z normally based on its own byte result alone. For equality testing across a subtraction chain, SBC’s special Z behaviour is exactly right. For addition, you need a different approach (see below).

10.5.2 Reading Z After a Chain

```
; 32-bit subtraction with equality check:
sub  r22, r16
sbc  r23, r17
sbc  r24, r18
sbc  r25, r19
; Now: Z=1 iff all four bytes computed to zero iff r25:r24:r23:r22 == r19:r18:r17:r16
breq values_equal
```

Do not read Z after only some of the SBC instructions and expect it to reflect the whole value:

```
sub  r22, r16
sbc  r23, r17      ; ← Z is NOT yet valid for the 32-bit result; never read Z here
sbc  r24, r18
sbc  r25, r19      ; ← only here is Z valid for the full 32-bit result
```

10.5.3 Testing Whether a Multi-byte Value Is Zero (Without Subtraction)

For checking whether a register group is zero without performing a subtraction, OR all bytes together and test Z:

```
; Is r25:r24:r23:r22 == 0?
mov  r0, r22
or   r0, r23
or   r0, r24
or   r0, r25
breq is_zero      ; Z=1: all four bytes were zero
```

OR only sets Z if the result byte is zero, which happens only when all four sources had zero bits in every position — i.e. all were 0x00.

Alternatively, OR the upper bytes into a scratch and use TST:

```
; Inline 32-bit zero test (clobbers r0):
mov  r0, r22
or   r0, r23
or   r0, r24
or   r0, r25      ; r0 = bitwise OR of all four bytes
; Z=1: all four were 0x00 → r25:r24:r23:r22 == 0
breq all_zero
```

10.5.4 Why ADC Does Not Have the Same Z Semantics

ADC sets and clears Z normally — it does not preserve a running “still zero” state. This means you cannot use BREQ after an ADD+ADC chain to test whether two multi-byte values are equal. Use SUB+SBC instead when you need equality as a side effect.

10.6 Multi-byte Comparison

10.6.1 Comparing Two 24-bit Values: CP + CPC + CPC

To compare two 24-bit values without changing either, use CP for the low byte and CPC for each subsequent byte. CPC (“Compare with Carry”) is to comparison what SBC is to subtraction.

```

; Compare r26:r25:r24 with r22:r21:r20 (24-bit unsigned)
cp   r24, r20      ; low bytes: sets C, Z (no carry in)
cpc  r25, r21      ; mid bytes: subtracts borrow from CP
cpc  r26, r22      ; high bytes: subtracts borrow from CPC
; After the chain:
;   Z=1: r26:r25:r24 == r22:r21:r20
;   C=0, Z=0: r26:r25:r24 > r22:r21:r20 (unsigned)
;   C=1: r26:r25:r24 < r22:r21:r20 (unsigned)
brlo a_less_than_b ; unsigned less-than
brsh a_ge_b        ; unsigned >=
breq a_equal_b     ; exactly equal

```

CPC carries the same special Z semantics as SBC: it only clears Z, never sets it. Z=1 after the final CPC means the entire multi-byte value matched.

Full 24-bit comparison example:

```

; Is counter (r26:r25:r24) >= limit (r22:r21:r20)?
cp   r24, r20
cpc  r25, r21
cpc  r26, r22
brsh counter_reached_limit ; C=0 → counter >= limit

```

10.6.2 Comparing Two 32-bit Values: CP + CPC + CPC + CPC

```

; Compare r25:r24:r23:r22 with r19:r18:r17:r16 (32-bit unsigned)
cp   r22, r16      ; byte 0 (lowest)
cpc  r23, r17      ; byte 1
cpc  r24, r18      ; byte 2
cpc  r25, r19      ; byte 3 (highest): flags final here
brlo a32_lt_b32    ; r25:r24:r23:r22 < r19:r18:r17:r16 (unsigned)
breq a32_eq_b32
; else: r25:r24:r23:r22 > r19:r18:r17:r16

```

10.6.3 Signed 32-bit Comparison

For signed 32-bit comparison, use BRLT and BRGE after the CPC chain. The V and S flags produced by the final CPC instruction reflect signed overflow and the corrected sign — exactly as for 8-bit and 16-bit signed comparisons.

```

; Signed 32-bit compare: r25:r24:r23:r22 vs r19:r18:r17:r16
cp   r22, r16
cpc  r23, r17
cpc  r24, r18
cpc  r25, r19
brlt signed_a_lt_b ; S=1 → a < b (signed)
brge signed_a_ge_b ; S=0 → a >= b (signed)
breq a_eq_b

```

The rule is identical to ch05b: BRLT reads S (= N XOR V), which handles overflow correctly. Never use BRLO for signed comparisons.

10.6.4 Comparing a Multi-byte Value with a Constant

There is no multi-byte CPI. For the low byte use CPI; for higher bytes use CPC against a register loaded with the constant byte:

```

; Is r25:r24:r23:r22 == 86400? (86400 = 0x00015180)
cpi r22, lo8(86400)           ; compare low byte
ldi r0, byte2(86400)
cpc r23, r0
ldi r0, byte3(86400)
cpc r24, r0
ldi r0, byte4(86400)         ; r0 = 0x00 for byte 4 of 86400
cpc r25, r0
breq equals_86400

```

Note: r0 is clobbered by this sequence. If r0 must be preserved, use a different scratch register in r16-r31.

For a 32-bit value in r16-r31, use CPI for the low byte and load into a scratch register for each subsequent CPC:

```

; Is r25:r24:r23:r22 >= 0x00800000? (unsigned greater-or-equal to 8388608)
cpi r22, 0x00                ; lo8
ldi r16, 0x00
cpc r23, r16                  ; byte2
ldi r16, 0x80
cpc r24, r16                  ; byte3
ldi r16, 0x00
cpc r25, r16                  ; byte4 (hi8)
brsh value_ge_8M

```

10.7 Multi-byte Negation

Negation in two's complement means computing $0 - \text{value}$. For a single byte, NEG Rd does this directly. For multi-byte values, there is no multi-byte NEG instruction. You must construct the operation yourself.

10.7.1 The Two Equivalent Methods

There are two standard ways to negate an N-byte two's complement value:

Method 1: COM all bytes, then add 1.

This is the textbook two's complement definition: flip all bits (COM), then increment (add 1). For a multi-byte value:

```

; Negate r25:r24:r23:r22 (32-bit, in-place)
; Method 1: COM each byte, then add 1
com r22
com r23
com r24
com r25
; Now r25:r24:r23:r22 = bitwise NOT of original value
; Add 1 to complete the two's complement negate:
ldi r16, 1
ldi r17, 0
add r22, r16
adc r23, r17
adc r24, r17
adc r25, r17

```

Method 2: NEG the low byte, SBC each higher byte from zero.

```

; Negate r25:r24:r23:r22 (32-bit, in-place)
; Method 2: NEG low byte, propagate borrow through SBC
neg r22                ; r22 = 0 - r22; C=1 if r22 was nonzero, C=0 if zero
; For each subsequent byte: subtract from 0, minus the borrow
; COM+INC equivalent: (0 - byte - borrow) = (~byte + 1 - borrow) when borrow=1
; Use a zero register to do: 0 - Rn - C
clr r0                 ; r0 = 0 (scratch; clobbers r0)
sbc r0, r0              ; This is WRONG – we want 0 - r23 - C, not r0 - r0 - C

```

Wait — SBC r0, r0 computes $r0 - r0 - C = -C$, not $0 - r23 - C$. The correct form requires the source register to hold the byte we want to negate:

```

; Negate r25:r24:r23:r22 (32-bit, in-place) – Method 2 (correct)
neg r22                ; r22 = 0 - r22, C=1 if r22≠0
; For each subsequent byte: COM then propagate borrow
; SBC Rd, Rr computes Rd = Rd - Rr - C
; We want: Rn = ~Rn - C + 1 (which equals 0 - Rn - C when the original NEG gave C=1)
; Actually: Method 2 is more cleanly expressed as:
; Rn = ~Rn - borrow (complement and subtract borrow from higher NEG/SBC steps)
; This equals: Rn = NOT(Rn) - C = 0xFF - Rn - C = -(Rn + C)
; The identity: 0 - Rn - C = ~Rn + 1 - C (when C=1 from the previous stage)
; A clean implementation:
com r23                ; ~r23
sbc r23, r1             ; r23 = ~r23 - 0 - C (r1 = 0)
; Actually this gives ~r23 - C, not 0 - r23 - C.

```

The arithmetic identity:

$$\begin{aligned}
 0 - Rn - C &= \sim Rn + 1 - C \\
 &= \sim Rn + (1 - C)
 \end{aligned}$$

When the previous byte produced $C=1$ (borrow), $1 - C = 0$, so the byte becomes $\sim Rn$. When the previous byte produced $C=0$ (no borrow, only happens if all lower bytes were zero), $1 - C = 1$, so the byte becomes $\sim Rn + 1$.

The COM + SBC from zero pattern using $r1 = 0$:

```

; Correct Method 2 for 32-bit negate using COM + SBC with r1=0:
neg r22                ; r22 = 0 - r22, sets C correctly
com r23                ; r23 = ~r23
sbci r23, 0             ; r23 = ~r23 - 0 - C = ~r23 - C
                        ; which equals 0 - orig_r23 - (C from prev) ✓
com r24
sbci r24, 0
com r25
sbci r25, 0

```

Method 1 (COM all + ADD chain) is simpler and less error-prone. Prefer it.

10.7.2 16-bit Negation (Recap)

```

; Negate r25:r24 (16-bit)
; Method 1: COM + add 1
com r24
com r25
adiw r24, 1             ; adiw is faster than add+adc for +1 on a supported pair

```

Or using NEG + SBC:

```

; Negate r25:r24 (16-bit, alternate)
neg r24                ; r24 = 0 - r24, C=1 unless r24 was 0

```

```
neg  r25          ; r25 = 0 - r25
sbc  r25, r1      ; r25 -= 0 + C(from neg r24) → r25 = -(r25) - C ✓
```

Wait — this does NOT give the right result because the second NEG already computed $0 - r25$, and then subtracts the carry. Let's verify:

Original: $r25:r24 = 0xAB:0xCD$

Negate: should give $0xFF:0xFF - 0xAB:0xCD + 1 = 0x5433$

Step 1: NEG r24: $0 - 0xCD = 0x33$, $C=1$ ($0xCD \neq 0$)

Step 2: NEG r25: $0 - 0xAB = 0x55$

Step 3: SBC r25, r1: $0x55 - 0 - 1(C) = 0x54$

Result: $r25:r24 = 0x54:0x33 = 0x5433$ ✓

The NEG + NEG + SBC form works for 16-bit.

For 24-bit and 32-bit, **Method 1 (COM all bytes + add 1)** is clearer:

10.7.3 24-bit Negation (Method 1)

```
; Negate r26:r25:r24 (24-bit, in-place)
com  r24
com  r25
com  r26
; Add 1 to the bitwise complement:
ldi  r16, 1
ldi  r17, 0
add  r24, r16
adc  r25, r17
adc  r26, r17
```

10.7.4 32-bit Negation (Method 1)

```
; Negate r25:r24:r23:r22 (32-bit, in-place)
com  r22
com  r23
com  r24
com  r25
ldi  r16, 1
ldi  r17, 0
add  r22, r16
adc  r23, r17
adc  r24, r17
adc  r25, r17
```

Trace: negate 0x00000001 → should give 0xFFFFFFFF

After COM chain: $r25:r24:r23:r22 = 0xFF:0xFF:0xFF:0xFE$

ADD r22, r16: $0xFE + 0x01 = 0xFF$, $C=0$

ADC r23, r17: $0xFF + 0x00 + 0 = 0xFF$, $C=0$

ADC r24, r17: $0xFF + 0x00 + 0 = 0xFF$, $C=0$

ADC r25, r17: $0xFF + 0x00 + 0 = 0xFF$, $C=0$

Result: $0xFFFFFFFF = -1$ (signed 32-bit) ✓

Trace: negate 0xFFFFFFFF → should give 0x00000001

After COM chain: $r25:r24:r23:r22 = 0x00:0x00:0x00:0x00$

ADD r22, r16: $0x00 + 0x01 = 0x01$, $C=0$

ADC r23-r25: all $0+0+0 = 0$, $C=0$
 Result: 0x00000001 ✓

Trace: negate 0x00000000 → should give 0x00000000

After COM chain: 0xFF:0xFF:0xFF:0xFF
 ADD r22: 0xFF + 1 = 0x100 → r22=0x00, C=1
 ADC r23: 0xFF + 0 + 1 = 0x100 → r23=0x00, C=1
 ADC r24: 0xFF + 0 + 1 = 0x100 → r24=0x00, C=1
 ADC r25: 0xFF + 0 + 1 = 0x100 → r25=0x00, C=1
 Result: 0x00000000, C=1 ✓
 (C=1 is the expected carry-out for negating zero in two's complement)

10.7.5 Signed Overflow on Multi-byte Negation

For any two's complement value, negating is well-defined for every value except the most-negative representable integer. Negating 0x80000000 (−2,147,483,648 for 32-bit signed) yields 2,147,483,648, which exceeds +2,147,483,647 and wraps back to 0x80000000.

After Method 1 negation, test for this case:

```
; 32-bit negate with overflow check
com r22
com r23
com r24
com r25
ldi r16, 1
ldi r17, 0
add r22, r16
adc r23, r17
adc r24, r17
adc r25, r17
; V=1 after the final ADC iff signed overflow occurred
brvs neg32_overflow      ; result is 0x80000000, same as input
```

10.8 Loading and Storing Multi-byte Values from SRAM

10.8.1 Little-Endian Storage

AVR stores multi-byte values in **little-endian** order: the byte with the lowest address is the least-significant byte. This matches how the C compiler lays out uint32_t variables.

For a 32-bit variable at SRAM address var32:

Address:	var32+0	var32+1	var32+2	var32+3
Content:	byte 0	byte 1	byte 2	byte 3
	(low)			(high)

10.8.2 Loading with LDS (Direct Addressing)

```
; Load 32-bit value from SRAM into r25:r24:r23:r22
lds r22, var32+0      ; low byte
lds r23, var32+1
lds r24, var32+2
lds r25, var32+3      ; high byte
```

Each LDS is a 3-cycle, 2-word instruction on the ATtiny3217 (AVRxt). Four LDS instructions = 12 cycles, 8 words of flash.

10.8.3 Loading with a Pointer Register (More Efficient)

Using the X, Y, or Z pointer with post-increment:

```
; Load 32-bit value at address in X into r25:r24:r23:r22
ld  r22, X+      ; low byte, X advances to +1
ld  r23, X+      ; byte 1, X advances to +2
ld  r24, X+      ; byte 2, X advances to +3
ld  r25, X        ; high byte, X points at last byte (not past it)
```

Or using X+ throughout if X should point past the last byte afterward:

```
ld  r22, X+
ld  r23, X+
ld  r24, X+
ld  r25, X+      ; X now points one past the end of the 32-bit value
```

LD Rd, X+ is 2 cycles on AVRxt. Four loads = 8 cycles, half the flash of four LDS instructions (one word each instead of two) and four cycles faster.

10.8.4 Storing with STS

```
; Store r25:r24:r23:r22 to SRAM at var32
sts  var32+0, r22
sts  var32+1, r23
sts  var32+2, r24
sts  var32+3, r25
```

10.8.5 Storing with a Pointer Register

```
; Store r25:r24:r23:r22 at address in Y
st   Y+, r22
st   Y+, r23
st   Y+, r24
st   Y,  r25      ; final byte: no post-increment if Y should stay at last byte
```

10.8.6 Atomicity Warning

Loading or storing a multi-byte variable that is written by an interrupt service routine is not atomic. If an interrupt fires between the first and last LD instruction, the loaded value may be inconsistent (half from the old value, half from the new).

The standard fix: disable interrupts around the load/store:

```
cli      ; disable interrupts
lds  r22, counter+0
lds  r23, counter+1
lds  r24, counter+2
lds  r25, counter+3
sei      ; re-enable interrupts
```

For an 8-byte value, this means holding interrupts off for 8 LDS instructions = 24 cycles on AVRxt. Keep the critical section as short as possible.

10.9 Practical Patterns

10.9.1 Pattern 1: 32-bit Millisecond Uptime Counter

A 32-bit tick counter incremented in a timer interrupt, read safely from main code:

```
.section .data
tick_count: .space 4           ; 32-bit, little-endian, initialized to zero

.section .text

; In the timer ISR (interrupt service routine):
timer_isr:
    push r24                   ; save clobbered registers
    push r25
    push r22
    push r23
    push r16
    in  r16, SREG
    push r16

    lds r22, tick_count+0
    lds r23, tick_count+1
    lds r24, tick_count+2
    lds r25, tick_count+3

    ldi r16, 1
    ldi r17, 0
    add r22, r16
    adc r23, r17
    adc r24, r17
    adc r25, r17

    sts tick_count+0, r22
    sts tick_count+1, r23
    sts tick_count+2, r24
    sts tick_count+3, r25

    pop r16
    out SREG, r16
    pop r23
    pop r22
    pop r25
    pop r24
    reti

; In main code: atomic read of tick_count into r25:r24:r23:r22
read_ticks:
    cli
    lds r22, tick_count+0
    lds r23, tick_count+1
    lds r24, tick_count+2
    lds r25, tick_count+3
    sei
    ret
```

10.9.2 Pattern 2: 32-bit Saturating Accumulator

Accumulate 8-bit unsigned samples into a 32-bit unsigned sum, clamping at 0xFFFFFFFF rather than wrapping:

```
/*
 * add32_sat8 – add unsigned 8-bit r16 to 32-bit accumulator r25:r24:r23:r22
 * Saturates at 0xFFFFFFFF. Clobbers r17.
 */
add32_sat8:
    clr    r17
    add    r22, r16        ; zero-extend r16 to 32-bit and add
    adc    r23, r17
    adc    r24, r17
    adc    r25, r17
    brcc   sat8_done       ; C=0: no overflow
    ser    r22             ; C=1: saturate to 0xFFFFFFFF
    ser    r23
    ser    r24
    ser    r25
sat8_done:
    ret
```

10.9.3 Pattern 3: 32-bit Elapsed Time

Given a 32-bit start time and current time (both in milliseconds), compute the elapsed milliseconds. This works correctly even if the counter has rolled over past 0xFFFFFFFF back toward zero (one rollover only):

```
/*
 * elapsed32 – compute elapsed time
 * Input:  r25:r24:r23:r22 = start_time, r19:r18:r17:r16 = current_time
 * Output: r25:r24:r23:r22 = elapsed = current_time - start_time
 * (Result is correct modulo 2^32 – handles one counter rollover)
 */
elapsed32:
    sub    r22, r16
    sbc    r23, r17
    sbc    r24, r18
    sbc    r25, r19
    ret
```

Unsigned subtraction modulo 2^{32} gives the correct elapsed time regardless of rollover, because $\text{current} - \text{start}$ in 32-bit modular arithmetic gives the right answer as long as at most one rollover occurred between the two times.

10.9.4 Pattern 4: 32-bit Range Check

Is a 32-bit unsigned value in the range [lo32, hi32]?

```
/*
 * in_range32 – check if r25:r24:r23:r22 is in [lo32, hi32]
 * Input:  r25:r24:r23:r22 = value to test
 *         lo32 and hi32 = 32-bit constants defined in .equ or .word
 * Output: Z=1 if in range, Z=0 if out of range
 * Clobbers: r19:r18:r17:r16
 */
in_range32:
    ; Check value >= lo32
    ldi    r16, lo8(lo32)
```

```

ldi r17, byte2(lo32)
ldi r18, byte3(lo32)
ldi r19, byte4(lo32)
cp r22, r16
cpc r23, r17
cpc r24, r18
cpc r25, r19
brlo out_of_range32      ; value < lo32

; Check value <= hi32 (equivalently: value < hi32 + 1)
ldi r16, lo8(hi32 + 1)
ldi r17, byte2(hi32 + 1)
ldi r18, byte3(hi32 + 1)
ldi r19, byte4(hi32 + 1)
cp r22, r16
cpc r23, r17
cpc r24, r18
cpc r25, r19
brsh out_of_range32      ; value >= hi32 + 1, i.e. value > hi32

; In range – signal via Z flag
; (Both CP chains consumed Z; now signal "in range" by setting Z explicitly)
clr r16
tst r16                  ; Z=1 (r16 is 0 after CLR)
ret
out_of_range32:
ldi r16, 1
tst r16                  ; Z=0 (r16 is 1)
ret

```

In practice, firmware usually branches directly to handlers rather than returning a flag; the above shows the comparison structure.

10.9.5 Pattern 5: Signed 32-bit Addition with Overflow Detection

```

/*
 * sadd32 – signed 32-bit add with overflow check
 * Input:  r25:r24:r23:r22 = a, r19:r18:r17:r16 = b
 * Output: r25:r24:r23:r22 = a + b
 *         V=1 if signed overflow (result outside [-2^31, 2^31 - 1])
 */
sadd32:
add r22, r16
adc r23, r17
adc r24, r18
adc r25, r19      ; V=1 here iff signed 32-bit overflow occurred
ret

```

Calling code that handles overflow:

```

rcall sadd32
brvs handle_s32_overflow

```

10.10 Instruction Reference

The following table covers the instructions used in multi-byte arithmetic chains. All cycle counts are for the ATtiny3217.

Instruction	Operands	Cycles	Flags updated	Notes
ADD Rd, Rr	r0–r31, r0–r31	1	H, S, V, N, Z, C	First byte in chain
ADC Rd, Rr	r0–r31, r0–r31	1	H, S, V, N, Z, C	Subsequent bytes; adds C
SUB Rd, Rr	r0–r31, r0–r31	1	H, S, V, N, Z, C	First byte in chain
SBC Rd, Rr	r0–r31, r0–r31	1	H, S, V, N, Z, *C	Subsequent bytes; Z only cleared
CP Rd, Rr	r0–r31, r0–r31	1	H, S, V, N, Z, C	Compare first byte (no write)
CPC Rd, Rr	r0–r31, r0–r31	1	H, S, V, N, Z, *C	Compare subsequent bytes; Z only cleared
NEG Rd	r0–r31	1	H, S, V, N, Z, C	Negate single byte; use in multi-byte via COM+chain
COM Rd	r0–r31	1	C=1, S, V=0, N, Z	Bitwise NOT; building block for multi-byte NEG
ADIW Rd, K	r24/XL/YL/ZL; K=0-63	2	S, V, N, Z, C; no H	16-bit += constant; use for +1 on 16-bit pairs
SBIW Rd, K	r24/XL/YL/ZL; K=0-63	2	S, V, N, Z, C; no H	16-bit -= constant
LDS Rd, k	r0–r31	3	none	Load byte from SRAM (direct)
STS k, Rr	r0–r31	2	none	Store byte to SRAM (direct)
LD Rd, X/Y/Z	r0–r31	2	none	Load byte (pointer; +/- variants same cycle count)
ST X/Y/Z, Rr	r0–r31	1	none	Store byte (pointer; +/- variants same cycle count)

*SBC, SBCI, CPC: only CLEAR Z (never set Z); Z=1 iff all bytes in the chain produced zero.

10.11 Common Pitfalls

10.11.1 Pitfall 1: Using INC to Increment the High Byte of a Multi-byte Value

INC does not set the carry flag (C). If you use INC to propagate a carry from a lower byte to a higher byte, the increment will happen unconditionally rather than only when carry is set:

```
; WRONG – INC is not carry-conditional
add r22, r16
adc r23, r17
adc r24, r18
inc r25 ; INC runs regardless of carry from ADC – broken!

; CORRECT
add r22, r16
adc r23, r17
adc r24, r18
adc r25, r19 ; r19 must be 0 if you only want carry propagation
```

When you want to propagate carry into a high byte that has no corresponding source operand (e.g., zero-extending a smaller value into a larger one), keep a zero register available and use ADC:

```
clr r17 ; zero register for carry propagation
add r22, r16 ; low byte: r16 is the actual operand
adc r23, r17 ; r23 += 0 + C (propagates carry only)
```

```
adc r24, r17      ; same
adc r25, r17
```

10.11.2 Pitfall 2: Processing High Byte Before Low Byte

The carry produced by ADD flows into the first ADC. If you process high before low, the carry from the wrong byte contaminates the result:

```
; WRONG – high byte first
adc r25, r19      ; C here is from some previous instruction, not from r22's ADD
add r22, r16

; CORRECT – low byte always first
add r22, r16
adc r23, r17
adc r24, r18
adc r25, r19
```

10.11.3 Pitfall 3: Reading Z Before the Final Instruction in an SBC Chain

Z is accumulated across SBC and CPC instructions. Reading it mid-chain gives the result for only the bytes processed so far:

```
sub r22, r16
sbc r23, r17      ; Z here: only valid for r23:r22 vs r17:r16 – NOT the 32-bit result
sbc r24, r18
sbc r25, r19      ; Z here: valid for full 32-bit comparison
breq equal_32
```

10.11.4 Pitfall 4: Using BRLO for Signed Multi-byte Comparison

After CP+CPC+...+CPC, use BRLT/BRGE for signed and BRLO/BRSH for unsigned. The V and S flags produced by the final CPC correctly reflect signed overflow.

```
; WRONG for signed comparison:
cp r22, r16
cpc r23, r17
cpc r24, r18
cpc r25, r19
brlo signed_less  ; WRONG: BRLO reads C, which is unsigned less-than

; CORRECT for signed comparison:
cp r22, r16
cpc r23, r17
cpc r24, r18
cpc r25, r19
brlt signed_less  ; CORRECT: BRLT reads S = N XOR V
```

10.11.5 Pitfall 5: COM Is Not NEG

For multi-byte negation, COM is not the whole story. COM Rd computes $\sim R_d$, which is one less than the negation. After COM-ing all bytes you must add 1 to complete the two's complement:

```
; WRONG – COM alone is not a negate
com r22
com r23
```

```

com r24
com r25          ; r25:r24:r23:r22 = ~original, NOT -original

; CORRECT — COM then add 1
com r22
com r23
com r24
com r25
ldi r16, 1
ldi r17, 0
add r22, r16
adc r23, r17      ; these complete the two's complement negate
adc r24, r17
adc r25, r17

```

10.11.6 Pitfall 6: Multi-byte Compare with Immediate — Forgetting to Load Higher Bytes

CPI only works on r16–r31 and compares one byte. Higher bytes of a constant must be loaded into a scratch register before CPC:

```

; WRONG — CPC reads r16 each time, but r16 doesn't change
cpi r22, lo8(K)
cpc r23, r16      ; r16 still has lo8(K), not byte2(K)!
cpc r24, r16
cpc r25, r16

; CORRECT — load each byte of the constant separately
cpi r22, lo8(K)
ldi r16, byte2(K)
cpc r23, r16
ldi r16, byte3(K)
cpc r24, r16
ldi r16, byte4(K)
cpc r25, r16

```

10.11.7 Pitfall 7: Interrupt-Unsafe Multi-byte Reads

Any multi-byte variable modified by an ISR and read by main code must be read atomically. A multi-byte read is not atomic by default — an interrupt can fire between two LDS instructions:

```

; WRONG — susceptible to torn read if ISR modifies counter32
lds r22, counter32+0
; ← ISR fires here, counter32 rolls over
lds r23, counter32+1 ; now mismatched with r22!

; CORRECT — atomic read
cli
lds r22, counter32+0
lds r23, counter32+1
lds r24, counter32+2
lds r25, counter32+3
sei

```

10.12 Summary

The carry chain principle:

Any-width ADD: first byte = ADD, subsequent bytes = ADC (low byte FIRST)

Any-width SUB: first byte = SUB, subsequent bytes = SBC (low byte FIRST)

Any-width CMP: first byte = CP, subsequent bytes = CPC

Final flags (C, V, Z, N, S) are valid only after the LAST instruction in the chain.

24-bit add: ADD r24,r20 / ADC r25,r21 / ADC r26,r22

24-bit sub: SUB r24,r20 / SBC r25,r21 / SBC r26,r22

24-bit cmp: CP r24,r20 / CPC r25,r21 / CPC r26,r22

32-bit add: ADD r22,r16 / ADC r23,r17 / ADC r24,r18 / ADC r25,r19

32-bit sub: SUB r22,r16 / SBC r23,r17 / SBC r24,r18 / SBC r25,r19

32-bit cmp: CP r22,r16 / CPC r23,r17 / CPC r24,r18 / CPC r25,r19

Unsigned overflow/underflow: C=1 after final ADC/SBC

Signed overflow: V=1 after final ADC/SBC; V=1 after final CPC for signed compare

Zero flag in SBC/CPC chains:

SBC and CPC only CLEAR Z. They never set it.

Z=1 after the chain iff EVERY byte produced a zero result.

Read Z only after the LAST instruction in the chain.

Multi-byte zero test without subtraction:

OR all bytes together into a scratch; Z=1 iff all bytes were 0x00.

Signed multi-byte compare: use BRLT/BRGE (reads S = N XOR V)

Unsigned multi-byte compare: use BRL0/BRSH (reads C)

Multi-byte negation (any width):

COM all bytes, then add 1 via ADD+ADC chain (Method 1 – prefer this)

NEG low byte, then COM + SBCI/SBC each higher byte (Method 2 – error-prone)

Signed overflow iff the value was the most-negative representable integer.

SRAM multi-byte layout: little-endian (lowest address = lowest byte)

Load/store: LDS/STS (direct) or LD/ST with X/Y/Z pointer (shorter, faster)

Interrupt safety: CLI + load/store all bytes + SEI for ISR-shared variables

INC/DEC pitfall: these do NOT update C – never use them inside a carry chain.

10.13 Exercises

1. Write a 24-bit unsigned adder subroutine: add24(r26:r25:r24, r22:r21:r20) → r26:r25:r24. Detect overflow via C after the chain and saturate to 0xFFFFFFFF if overflow occurs. Trace the inputs (0xFFFFF0, 0x000020) and (0x0000FF, 0x000001).
2. Implement a 32-bit two's complement negation subroutine using Method 1 (COM + add 1). Test with inputs 0x00000001, 0x7FFFFFFF, 0x80000000, and 0x00000000. For which input does signed overflow occur? What does V indicate?
3. Write a 32-bit comparison subroutine that returns (via Z flag and the SREG S flag) whether two 32-bit signed values are less-than, equal, or greater-than. Call it from a small main that tests all three outcomes.
4. A 32-bit millisecond counter rolls over every ~49.7 days. Given a start timestamp start32 and a current timestamp now32, write a function has_elapsed(now32, start32, delay32) → bool that returns true if now32 - start32 ≥ delay32 using only unsigned 32-bit subtraction and comparison. Explain why this works even across a rollover.

5. Write a 32-bit saturating subtract subroutine: `r25:r24:r23:r22 -= r19:r18:r17:r16`, clamped to `0x00000000` on underflow. Verify with inputs (`0x00000010`, `0x00000020`) → should give 0.
6. Given a 24-bit signed value in `r26:r25:r24` (two's complement), sign-extend it to 32 bits in `r25:r24:r23:r22`. Describe the pattern (extend `r26`'s sign bit across a new fourth byte) and implement it using `SBRC + SER / CLR`.
7. An SRAM buffer at address `buf` holds three consecutive 32-bit unsigned values (little-endian). Write a code sequence that loads all three into register quads `r25:r24:r23:r22`, `r19:r18:r17:r16`, and `r11:r10:r9:r8` using a single Y-pointer with post-increment. How many flash words does this consume compared to twelve separate LDS instructions?
8. Modify the 32-bit uptime counter ISR from the Practical Patterns section to also maintain a 32-bit rollover counter that increments each time the uptime counter wraps from `0xFFFFFFFF` back to `0x00000000`. What flag do you test after the increment, and where in the ISR does the test go?

Next: Chapter 11 — Bit Math: Masks, Shifts, Rotates, Packing, and Field Extraction

Chapter 11

Bit Math: Masks, Shifts, Rotates, Packing, and Field Extraction

Arithmetic adds and subtracts numbers. Bit math operates on individual bits and groups of bits regardless of numeric value. It is how you configure peripheral registers, isolate sensor channels, assemble protocol frames, move data through hardware shift registers, and pack two small values into one byte. Every embedded program uses these techniques constantly.

This chapter covers the complete AVR bitwise toolkit: the logical instructions (AND, OR, EOR, COM), the shift and rotate family, single-bit test and manipulation instructions, and the patterns built from them — masking, field extraction, field insertion, packing, and unpacking.

Shifts and rotates used for arithmetic (multiplying or dividing by powers of two) are revisited in Chapter 13. This chapter treats them as bit-movement tools.

11.1 The Bitwise Logic Instructions

The four bitwise logic instructions compute their results one bit at a time. Bit N of the result depends only on bit N of each operand — never on any other bit position. There are no carries, no overflow, no numeric interpretation.

11.1.1 AND — Bitwise AND

AND Rd, Rr ; Rd ← Rd AND Rr (any r0–r31)
ANDI Rd, K ; Rd ← Rd AND K (r16–r31 only, K = 0..255)

1 cycle. Flags: S, V=0, N, Z. C unchanged.

The AND truth table for each bit pair:

A	B	A AND B
—	—	—
0	0	0
0	1	0
1	0	0
1	1	1

AND produces 1 only when both inputs are 1. Every other combination gives 0.

The primary use of AND is **masking**: the mask register controls which bits of the source are preserved (mask bit = 1) and which are forced to zero (mask bit = 0):

```
ldi r16, 0b10110110 ; source: r16 = 0xB6
ldi r17, 0b00001111 ; mask: keep only lower nibble

and r16, r17
; r16 = 0b00000110 = 0x06
;      ^^^^ - only the lower 4 bits survived
```

AND forces bits to 0. It cannot force bits to 1. To force bits to 1, use OR.

Flags: AND always clears V. The N flag reflects bit 7 of the result. Z=1 if the result is 0x00. Because V is forced clear, S = N XOR V = N, which means BRMI and BRLT behave identically after AND.

11.1.2 OR — Bitwise OR

```
OR Rd, Rr ; Rd ← Rd OR Rr (any r0–r31)
ORI Rd, K ; Rd ← Rd OR K (r16–r31 only)
```

1 cycle. Flags: S, V=0, N, Z. C unchanged.

OR truth table:

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

OR produces 1 when either input is 1. It forces bits to 1 wherever the mask has a 1, and leaves bits unchanged wherever the mask has a 0:

```
ldi r16, 0b00000110 ; r16 = 0x06
ldi r17, 0b11000000 ; mask: set bits 7 and 6

or r16, r17
; r16 = 0b11000110 = 0xC6
; ^^ - bits 7 and 6 forced to 1; all others unchanged
```

OR forces bits to 1. It cannot force bits to 0. The combination of AND (force to 0) and OR (force to 1) is how read-modify-write operations on peripheral registers work.

11.1.3 EOR — Bitwise Exclusive OR (XOR)

```
EOR Rd, Rr ; Rd ← Rd EOR Rr (any r0–r31)
```

1 cycle. Flags: S, V=0, N, Z. C unchanged. No immediate form — use a scratch register if a constant mask is needed.

EOR truth table:

A	B	A EOR B
0	0	0
0	1	1
1	0	1
1	1	0

EOR produces 1 when the inputs differ. It **toggles** bits wherever the mask has a 1, and leaves bits unchanged wherever the mask has a 0:

```
ldi r16, 0b10110110 ; r16 = 0xB6
ldi r17, 0b00001111 ; mask: toggle lower nibble

eor r16, r17
; r16 = 0b10111001 = 0xB9
;      ^^^^ - lower nibble toggled: 0110 → 1001
```

EOR has two special cases worth knowing:

```
eor r16, r16 ; r16 = 0x00 (register XOR itself = clear to zero)
; This is the preferred way to clear r0–r31. Aliases: CLR Rd.
; 1 cycle, sets Z=1, does NOT affect C.

eor r16, r17 ; Also used for: test if r16 == r17 without storing result
; (result is 0 iff they were equal, setting Z=1)
```

CLR Rd is the assembler alias for EOR Rd, Rd. Always use CLR in source for clarity.

11.1.4 COM — Bitwise Complement (One's Complement)

COM Rd ; $Rd \leftarrow 0xFF - Rd = \sim Rd$ (any r0–r31)

1 cycle. Flags: C=1 (always), S, V=0, N, Z.

COM inverts every bit. It is bitwise NOT:

```
ldi r16, 0b10110110 ; r16 = 0xB6
com r16
; r16 = 0b01001001 = 0x49
```

Primary uses: - Building inverted masks at runtime when the positive mask is already computed - First step of two's complement negation (COM then INC — same as NEG) - Multi-byte negation (COM all bytes, then add 1 — see Chapter 10)

COM is **not** arithmetic negation. COM r16 gives $\sim r16$; NEG r16 gives $-r16 = \sim r16 + 1$. For single-byte negate, use NEG. For multi-byte negate, use COM on every byte then add 1.

11.1.5 Flag Behaviour Summary for Logic Instructions

All four logic instructions (AND, OR, EOR, COM) clear V to zero and set N and Z based on the result byte. C is unchanged by AND, OR, and EOR; COM sets C=1.

Because V=0 after a logic instruction, $S = N \text{ XOR } V = N$, so: - BRMI and BRLT are equivalent after a logic instruction - BRPL and BRGE are equivalent after a logic instruction

This matters when you use AND or EOR to test a value and then branch on the sign.

11.2 Masking: Isolating, Clearing, Setting, and Toggling Bits

A **mask** is a byte (or word) used as the second operand of a logic instruction to select which bits of the first operand are affected. The mask is the tool; AND, OR, and EOR are the operations.

11.2.1 Clearing Specific Bits (AND)

To clear bits n and m of a register, AND with a mask that has 0 in positions n and m and 1 everywhere else:

```
; Clear bits 3 and 1 of r16
andi r16, 0b11110101    ; mask: ~(BIT(3) | BIT(1)) = 0xF5
; Bits 3 and 1 are now 0; all other bits unchanged
```

The mask for “clear bit N” is $0xFF \wedge (1 \ll N)$ or equivalently $\sim(1 \ll N)$, computed at assembly time:

```
andi r16, ~(1 << 3)      ; clear bit 3 (assembler evaluates ~(1<<3) = 0xF7)
andi r16, ~(1 << 5)      ; clear bit 5
```

Mnemonic: AND with 0 clears; AND with 1 preserves.

11.2.2 Setting Specific Bits (OR)

To set bits n and m, OR with a mask that has 1 in those positions:

```
; Set bits 5 and 2 of r16
ori  r16, (1 << 5) | (1 << 2)    ; mask = 0b00100100 = 0x24
; Bits 5 and 2 are now 1; all other bits unchanged
```

Mnemonic: OR with 1 sets; OR with 0 preserves.

11.2.3 Toggling Specific Bits (EOR)

To flip bits without knowing their current state, EOR with a mask that has 1 in those positions:

```
; Toggle bits 7 and 0 of r16
ldi  r17, (1 << 7) | (1 << 0)    ; mask = 0b10000001 = 0x81
eor  r16, r17
; Bits 7 and 0 are inverted; all other bits unchanged
```

11.2.4 Isolating a Single Bit (AND, Result in-place)

Test whether a specific bit is set by AND-ing with a single-bit mask:

```
; Is bit 4 of r16 set?
mov  r17, r16
andi r17, (1 << 4)              ; r17 = 0 if bit 4 was 0; r17 = 0x10 if bit 4 was 1
breq bit4_clear                 ; Z=1: bit 4 was 0
; here: bit 4 was 1
```

The result is nonzero (but not necessarily 1) when the bit is set — it equals the mask value. Use BREQ/BRNE to branch on the result; do not compare with 1.

11.2.5 Combining Set and Clear: Read-Modify-Write

Peripheral register configuration is almost always read-modify-write: read the current register value, change only the relevant bits, write back. This preserves other bits that other code may have set.

```
; Configure PORTB: set bit 5 (output high), clear bit 3 (output low)
lds  r16, VPORTB_OUT           ; read current output register
ori  r16, (1 << 5)              ; set bit 5
andi r16, ~(1 << 3)            ; clear bit 3
sts  VPORTB_OUT, r16           ; write back
```

The order — read, modify, write — is the universal peripheral control pattern.

11.2.6 Common Mask Constants

Mask	Binary	Purpose
0x0F	0000 1111	Isolate lower nibble (units digit, low 4 bits)
0xF0	1111 0000	Isolate upper nibble (tens digit, high 4 bits)
0x7F	0111 1111	Clear bit 7 (sign bit in signed, MSB mask)
0x80	1000 0000	Isolate bit 7 (sign bit test)
0x01	0000 0001	Isolate bit 0 (odd/even test, LSB)
(1 << N)	varies	Single-bit mask for bit N (N = 0..7)
~(1 << N)	varies	Clear-bit mask for bit N

11.3 Shift Instructions

Shifts move all bits left or right by one position. The bit that falls off the end goes into the carry flag; the vacated position is filled according to the shift type.

All shift instructions operate on a single register in 1 cycle.

LSL Rd – Logical Shift Left

$C \leftarrow [7][6][5][4][3][2][1][0] \leftarrow 0$

Bit 7 $\rightarrow$ C. Bit 0 filled with 0. Equivalent to ADD Rd, Rd.

Effect: $Rd \times 2$ (unsigned). Overflows into C when result > 255.

Flags: H, C, Z, N, V, S (H and V are set; $V = N \text{ XOR } C$ after shift)

LSR Rd – Logical Shift Right

$0 \rightarrow [7][6][5][4][3][2][1][0] \rightarrow C$

Bit 0 $\rightarrow$ C. Bit 7 filled with 0.

Effect: $Rd \div 2$ (unsigned). Remainder (0 or 1) in C.

Flags: C, Z, N=0, $V=N_{\text{before}} \text{ XOR } C$, S

ASR Rd – Arithmetic Shift Right

$b7 \rightarrow [7][6][5][4][3][2][1][0] \rightarrow C$

Bit 0 $\rightarrow$ C. Bit 7 preserved (copied from its current value).

Effect: $Rd \div 2$ (signed, rounds toward $-\infty$). Remainder in C.

Flags: C, Z, N, $V=N \text{ XOR } C$, S

11.3.1 Shifts as Bit Movement

Shifts are not just arithmetic: they physically move bits to a new position. This is how you position a field for insertion or extraction.

Move bit 3 into bit 0 (extract bit 3 to the LSB):

```
ldi r16, 0b00101000 ; r16 bit 3 is set
lsr r16              ; bit 3  $\rightarrow$  bit 2, bit 0  $\rightarrow$  C
lsr r16              ; bit 2  $\rightarrow$  bit 1
lsr r16              ; bit 1  $\rightarrow$  bit 0
; r16 = 0b00000101 (bit 3 is now in bit 0, though bit 2 is also set from the original bit 5)
; Better: use the ANDI-before-shift pattern for clean extraction (see Field Extraction)
```

A cleaner pattern: mask first, then shift:

```
ldi r16, 0b00101000 ; source
andi r16, (1 << 3)  ; isolate bit 3: r16 = 0b00001000
lsr r16              ;  $\rightarrow$  0b00000100
lsr r16              ;  $\rightarrow$  0b00000010
lsr r16              ;  $\rightarrow$  0b00000001 (bit 3 value is now in bit 0)
```

Alternatively, test the bit directly with SBRC/SBRS (see below) and build the result bit-by-bit.

11.3.2 SWAP — Swap Nibbles

SWAP Rd ; bits 7:4 ↔ bits 3:0

1 cycle. No flags affected.

SWAP exchanges the upper and lower nibbles of a register:

```
ldi r16, 0xAB
swap r16 ; r16 = 0xBA
```

SWAP is a one-cycle substitute for four LSR (or LSL) instructions when working with nibble-aligned fields. It is the tool for moving the upper nibble to the lower position for extraction:

```
; Extract the upper nibble of r16 into the lower nibble of r17
mov r17, r16
swap r17 ; upper nibble → lower nibble (garbage in upper nibble)
andi r17, 0x0F ; clear the garbage upper nibble
; r17 = upper nibble of r16 as a 4-bit value (0–15)
```

And the reverse — move a low nibble to the upper nibble:

```
; Move lower nibble of r16 into upper nibble
andi r16, 0x0F ; clear upper nibble first
swap r16 ; lower nibble → upper nibble (0s in lower nibble)
```

11.4 Rotate Instructions

Rotates shift bits through the carry flag. The carry flag acts as a 9th bit in the rotation ring.

ROL Rd — Rotate Left through Carry
 C ← [7][6][5][4][3][2][1][0] ← (old C)
 Bit 7 → C. Bit 0 gets the old value of C.
 Flags: H, C, Z, N, V, S

ROR Rd — Rotate Right through Carry
 (old C) → [7][6][5][4][3][2][1][0] → C
 Bit 0 → C. Bit 7 gets the old value of C.
 Flags: C, Z, N, V, S

11.4.1 Rotates vs Shifts

A shift discards the outgoing bit (it goes into C but the vacated position is filled with 0 regardless of C). A rotate feeds C back in on the vacated side.

This distinction is critical for multi-byte operations: when chaining shifts across multiple registers, rotates are the correct tool because they carry bits from one register into the next.

11.4.2 Multi-byte Shift Left (LSL + ROL chain)

```
; 16-bit logical shift left: r25:r24 <<= 1
lsl r24 ; low byte: bit 7 → C, 0 fills bit 0
rol r25 ; high byte: old C → bit 0, bit 7 → C (new C = overflow)
```

```
; 32-bit logical shift left: r25:r24:r23:r22 <=> 1
lsl  r22
rol  r23
rol  r24
rol  r25
```

Each additional byte just adds another ROL. The carry from the previous register flows into the LSB of the next.

11.4.3 Multi-byte Shift Right (LSR + ROR chain)

```
; 16-bit logical shift right: r25:r24 >>= 1
lsr  r25      ; high byte first: bit 0 → C, 0 fills bit 7
ror  r24      ; low byte: old C → bit 7, bit 0 → C

; 32-bit logical shift right
lsr  r25
ror  r24
ror  r23
ror  r22
```

Multi-byte right shift processes high byte first (unlike addition, which processes low byte first). The carry flows from the high byte down into the low byte.

11.4.4 Multi-byte Arithmetic Shift Right (ASR + ROR chain)

```
; 16-bit arithmetic shift right (signed ÷2): r25:r24 >>= 1
asr  r25      ; high byte: sign bit preserved, bit 0 → C
ror  r24      ; low byte: C → bit 7, bit 0 → C
```

Use ASR only on the most-significant byte. Every lower byte uses ROR, which propagates the carry without forcing anything.

11.4.5 Shifting by N Bits

AVR has no multi-bit shift instruction. Repeat the pair for each bit:

```
; 16-bit: r25:r24 >>= 3 (logical)
lsr  r25
ror  r24
lsr  r25
ror  r24
lsr  r25
ror  r24
```

For large shifts ($N = 4$), SWAP + ANDI/ORI is often faster on 8-bit registers. For shifts by 8 (full byte), simply move registers:

```
; r25:r24 >>= 8 (shift right by one full byte)
mov  r24, r25 ; what was the high byte becomes the low byte
clr  r25      ; high byte is now 0

; r25:r24 <=> 8 (shift left by one full byte)
mov  r25, r24 ; what was the low byte becomes the high byte
clr  r24      ; low byte is now 0
```


11.5 Bit Test and Skip Instructions

These instructions test a single bit and conditionally skip the next instruction. They avoid the need to mask, compare, and branch.

11.5.1 SBRC / SBRS — Skip if Bit in Register Clear/Set

SBRC Rr, b ; skip next instruction if bit b of Rr is 0 (Clear)
 SBRS Rr, b ; skip next instruction if bit b of Rr is 1 (Set)

1 cycle if not skipping; 2 cycles if skipping (3 if the skipped instruction is 2 words). b = 0..7. Operand Rr = any r0-r31. No flags affected.

These are the cleanest way to act on a single bit:

```
; Execute handler if bit 4 of r16 is set
sbrs r16, 4      ; skip next if bit 4 SET
rjmp skip_handler ; bit 4 clear: skip handler
rcall handle_bit4 ; bit 4 set: handle it
skip_handler:

; Conditional assignment: if bit 2 of r16 set, r17 = 0xFF, else r17 = 0x00
clr r17
sbrs r16, 2      ; skip CLR/SER sequence if bit 2 set
rjmp bit2_clear
ser r17          ; bit 2 set: r17 = 0xFF
bit2_clear:
```

SBRC and SBRS are the preferred way to branch on a single bit. They are more readable and often faster than ANDI + BREQ/BRNE.

11.5.2 SBIC / SBIS — Skip if I/O Bit Clear/Set

SBIC A, b ; skip next if bit b of I/O register A is 0
 SBIS A, b ; skip next if bit b of I/O register A is 1

A = I/O address 0..31 (the lower I/O space, accessible without IN/OUT). b = 0..7. No flags affected.

These operate directly on peripheral registers without loading them into a general-purpose register:

```
; Wait until USART transmit data register is empty (UDRE bit in status)
wait_udre:
    sbis UCSR0A, UDRE0 ; skip next if UDRE bit SET (register ready)
    rjmp wait_udre      ; UDRE clear: not ready, loop
; here: UDRE is set – safe to write
```

SBIC and SBIS only reach the bottom 32 I/O addresses (0x00–0x1F). Peripheral registers above that range must use IN + SBRC/SBRS.

11.5.3 SBI / CBI — Set/Clear Bit in I/O Register

SBI A, b ; set bit b in I/O register A
 CBI A, b ; clear bit b in I/O register A

1 cycle on AVRxt (ATtiny3217); 2 cycles on classic AVR devices. A = I/O address 0..31. b = 0..7. No flags affected.

These are atomic single-bit operations on I/O space — no read-modify-write cycle in software is needed:

```
sbi  VPORTB_DIR, 5      ; set PORTB pin 5 as output
cbi  VPORTB_OUT, 5      ; drive PORTB pin 5 low
sbi  VPORTB_OUT, 5      ; drive PORTB pin 5 high
```

SBI and CBI are faster than the ANDI/ORI read-modify-write pattern for the I/O registers they can reach.

11.6 The T Flag: Single-Bit Transfer

The T flag in SREG is a general-purpose bit storage cell. It is not set by arithmetic; it only changes via BST and BLD.

```
BST Rr, b      ; T ← bit b of Rr (Bit STore into T)
BLD Rd, b      ; bit b of Rd ← T (Bit LoAD from T)
```

1 cycle each. Only T is affected by BST; only bit b of Rd is affected by BLD.

The T flag lets you copy an individual bit from one register to any bit position in another register without clobbering either register:

```
; Copy bit 5 of r16 into bit 2 of r17
bst  r16, 5      ; T ← bit 5 of r16
bld  r17, 2      ; bit 2 of r17 ← T
; r16 and r17 are otherwise unchanged
```

Compare with the AND/OR approach that requires scratch registers and clobbers more of the destination:

```
; Same operation without T flag (more cumbersome):
mov  r18, r16
lsr  r18          ; move bit 5 toward bit 0 (need 5 shifts)
lsr  r18
lsr  r18
lsr  r18
lsr  r18          ; r18 bit 0 = original r16 bit 5
andi r18, 0x01    ; isolate
lsl  r18
lsl  r18          ; move to bit 2
andi r17, ~(1 << 2) ; clear bit 2 in destination
or   r17, r18     ; insert
```

The BST/BLD pair is the right tool whenever you need to copy a single bit between non-corresponding positions.

11.7 Field Extraction: Getting Bits n:m from a Register

A **field** is a contiguous run of bits at a known position within a byte. Extracting a field means isolating those bits and shifting them down to bit 0 so the result is a plain integer (0 to $2^{\text{width}} - 1$).

11.7.1 Single-Bit Extraction to Bit 0

```
; Extract bit 5 of r16 into bit 0 of r17 (result: 0 or 1)
mov  r17, r16
andi r17, (1 << 5) ; isolate bit 5: r17 = 0 or 0x20
```

```

lsr r17          ; 0x20 → 0x10
lsr r17          ; 0x10 → 0x08
lsr r17          ; 0x08 → 0x04
lsr r17          ; 0x04 → 0x02
lsr r17          ; 0x02 → 0x01
; r17 = 0 (bit was 0) or 1 (bit was 1)

```

With BST, this is two instructions:

```

; Extract bit 5 of r16 into bit 0 of r17 using T flag
bst r16, 5
bld r17, 0          ; r17 bit 0 = old r16 bit 5; other bits of r17 unchanged

```

If you want a clean 0-or-1 result with other bits zeroed:

```

clr r17
bst r16, 5
bld r17, 0          ; r17 = 0 or 1

```

11.7.2 Multi-bit Field Extraction (General Case)

To extract bits [hi:lo] from a register (a field of width hi-lo+1 starting at bit lo):

1. Mask to isolate the field
2. Shift right by lo positions to bring it to bit 0

```

; Extract bits 6:4 (3-bit field at offset 4) from r16 into r17
; Example: r16 = 0b10110100, field bits 6:4 = 0b011 = 3

```

```

mov r17, r16
andi r17, 0b01110000 ; mask = 0x70 = bits 6, 5, 4
lsr r17              ; bits 6:4 → bits 5:3
lsr r17              ; bits 5:3 → bits 4:2
lsr r17              ; bits 4:2 → bits 3:1
lsr r17              ; bits 3:1 → bits 2:0
; r17 = field value (0-7)

```

The number of shifts equals the field's starting bit position (lo).

For fields at nibble boundaries, SWAP eliminates four shifts:

```

; Extract upper nibble (bits 7:4) of r16 into lower nibble of r17
mov r17, r16
swap r17          ; upper nibble → lower nibble
andi r17, 0x0F     ; clear upper nibble (which now holds old lower nibble)
; r17 = upper nibble value (0-15)

```

11.7.3 Extracting the Lower Nibble (bits 3:0)

```

mov r17, r16
andi r17, 0x0F     ; r17 = bits 3:0 (lower nibble), 0-15

```

One instruction. No shift needed because the field is already at bit 0.

11.7.4 Extracting Specific Byte from a Multi-byte Value

When a 16-bit value is in r25:r24, extracting the high byte is just a MOV:

```

mov r16, r25          ; r16 = high byte of r25:r24
mov r16, r24          ; r16 = low byte of r25:r24

```

No shift, no mask. The byte boundary aligns with the register boundary.

11.8 Field Insertion: Putting a Value into a Specific Bit Position

Inserting a field is the reverse of extraction: shift the value up to its target position, mask off any overflow, clear the target bits in the destination, then OR the shifted value in.

11.8.1 Single-Bit Insertion

```
; Set or clear bit 5 of r16 based on whether r17 bit 0 is set
andi r17, 0x01          ; isolate the source bit (0 or 1)
lsl r17                 ; bit 0 → bit 1
lsl r17                 ; bit 1 → bit 2
lsl r17                 ; bit 2 → bit 3
lsl r17                 ; bit 3 → bit 4
lsl r17                 ; bit 4 → bit 5 (r17 = 0 or 0x20)
andi r16, ~(1 << 5)    ; clear bit 5 in destination
or r16, r17             ; insert the bit
```

Again, BST/BLD is cleaner for single-bit work:

```
; Copy bit 0 of r17 into bit 5 of r16
bst r17, 0
bld r16, 5
```

11.8.2 Multi-bit Field Insertion (General Case)

Insert a w-bit value in r17 (bits [w-1:0]) into bits [hi:lo] of r16, where lo = start position and w = hi - lo + 1:

1. Mask r17 to exactly w bits (prevent overflow into adjacent fields)
2. Shift r17 left by lo positions
3. AND r16 with the inverted field mask to clear the target bits
4. OR the shifted value into r16

```
; Insert bits 2:0 of r17 into bits 6:4 of r16 (3-bit field at offset 4)
; Field mask: bits 6:4 = 0b01110000 = 0x70

andi r17, 0x07          ; step 1: keep only 3 bits (0-7)
lsl r17                 ; step 2: shift up 4 positions (field offset = 4)
lsl r17
lsl r17
lsl r17                 ; r17 = (value & 0x07) << 4 = value in bits 6:4

andi r16, ~0x70         ; step 3: clear bits 6:4 in destination (mask = 0x8F)
or r16, r17             ; step 4: insert the field
```

11.8.3 Inserting a Lower Nibble into the Upper Nibble (SWAP idiom)

```
; Insert lower nibble of r17 into upper nibble of r16
; (Leaves lower nibble of r16 unchanged)
andi r17, 0x0F          ; ensure upper nibble of r17 is clear
swap r17                ; lower nibble → upper nibble
```

```
andi r16, 0x0F      ; clear upper nibble of destination
or   r16, r17       ; merge
```

11.9 Packing: Combining Two Values into One Byte

Packing puts two separate values into the two halves (nibbles) of a single byte. This is common for: - Sending packed sensor data over a 1-byte protocol field - Storing two 4-bit values in one SRAM byte to save space - Building BCD digits from separate tens and units values

11.9.1 Pack Two Nibbles into One Byte

```
; r16 = packed byte from low nibble of r17 (tens) and low nibble of r18 (units)
; r17 = tens digit (0-9, stored in low nibble)
; r18 = units digit (0-9, stored in low nibble)

andi r17, 0x0F      ; ensure only low nibble (defensive)
andi r18, 0x0F      ; same
swap r17            ; move tens to high nibble position
or   r17, r18        ; merge units into low nibble
mov  r16, r17        ; r16 = packed result
```

Trace: tens=4 (r17=0x04), units=7 (r18=0x07)

```
andi r17, 0x0F:  r17 = 0x04
andi r18, 0x0F:  r18 = 0x07
swap r17:        r17 = 0x40  (4 in upper nibble)
or   r17, r18:   r17 = 0x47  (packed BCD 47) ✓
```

11.9.2 Pack Eight Signal Lines into One Byte

When reading a parallel bus where each signal is in a different register, pack them into a single byte using BST/BLD:

```
; Build a byte from 8 individual signal bits:
; bit 7 = MOSI (bit 0 of r20)
; bit 6 = MISO (bit 0 of r21)
; ... etc.
; Result in r16.

clr  r16
bst  r20, 0 ; MOSI
bld  r16, 7
bst  r21, 0 ; MISO
bld  r16, 6
; ... (repeat for remaining bits) ...
```

11.9.3 Pack a Boolean Array into a Byte

Eight boolean values (one per register, non-zero = true) into one flag byte:

```
; r8..r15 each hold 0 or nonzero (boolean)
; Pack: r16 bit N = 1 iff r(8+N) is nonzero

clr  r16
```

```
ldi r17, 8
ldi r30, lo8(r8_array)    ; point Z at r8 using indirect addressing workaround
; (In practice, use SRAM-backed booleans and load via LD)
; For registers: use CPSE + SBI per bit or BST approach
```

A direct approach when the source values are already in known registers:

```
; Compact: bit 0 = nonzero(r8), bit 1 = nonzero(r9), ...
clr r16
tst r8
breq pack_b0_done
ori r16, (1 << 0)
pack_b0_done:
tst r9
breq pack_b1_done
ori r16, (1 << 1)
pack_b1_done:
; ... (repeat for each bit) ...
```

11.10 Unpacking: Splitting a Byte into its Fields

Unpacking reads a packed byte and distributes its fields into separate registers for processing.

11.10.1 Unpack Two Nibbles from One Byte

```
; r16 = packed byte (e.g., BCD 0x47)
; Extract high nibble → r17, low nibble → r18

mov r17, r16
swap r17          ; upper nibble → lower nibble
andi r17, 0x0F    ; r17 = upper nibble value (4 in this example)

mov r18, r16
andi r18, 0x0F    ; r18 = lower nibble value (7 in this example)
```

11.10.2 Unpack a Byte into Individual Bits

Distribute 8 bits from r16 into individual boolean registers r8–r15 (one bit per register):

```
; Unpack r16: each bit → separate register (0 or 1)
bst r16, 0 ; bit 0
bld r8, 0
clr r8
bst r16, 0
bld r8, 0 ; r8 bit 0 = source bit 0; but other bits of r8 are 0 from CLR
```

A cleaner idiom using LSR + carry:

```
; Unpack r16 bit-by-bit into r8..r15 using shifts
; After each LSR, carry holds the outgoing bit

lsr r16          ; bit 0 → C
clr r8
adc r8, r8        ; r8 = C (0 or 1)
```

```

lsr r16      ; original bit 1 → C
clr r9
adc r9, r9    ; r9 = C

; ... (repeat for r10–r15) ...

```

ADC Rd, Rd computes $Rd + Rd + C$. With CLR r8 first: $0 + 0 + C = C$. So `clr + adc Rd`, Rd loads C into bit 0 of Rd in one step.

11.11 Practical Patterns

11.11.1 Pattern 1: Read a GPIO Port and Extract a Pin State

```

; Read PORTB input and test whether pin 3 is high
lds r16, VPORTB_IN      ; read PORTB input register
sbrs r16, 3              ; skip next if pin 3 is HIGH
rjmp pin3_low
; pin3_high:
rjmp pin3_done
pin3_low:
pin3_done:

```

11.11.2 Pattern 2: Write a Specific Field of a Peripheral Register

```

; Set the clock prescaler field (bits 2:0) of a config register to 0b101
; without disturbing other bits

lds r16, CLK_CONFIG_REG ; read current config
andi r16, 0b1111000     ; clear bits 2:0
ori r16, 0b00000101     ; set bits 2:0 to 0b101
sts CLK_CONFIG_REG, r16 ; write back

```

11.11.3 Pattern 3: Reverse the Bits of a Byte

Reverse all 8 bits (bit 0 ↔ bit 7, bit 1 ↔ bit 6, etc.) — used for LSB-first to MSB-first conversion in bit-banged protocols:

```

/*
 * bit_reverse8 – reverse all 8 bits of r16
 * Input:  r16 = byte to reverse
 * Output: r16 = bit-reversed byte
 * Clobbers: r17, r18
 */
bit_reverse8:
    ldi r18, 8      ; 8 bits to process
    clr r17         ; result accumulator
bit_rev_loop:
    lsl r17         ; make room for next bit
    lsr r16         ; LSB of r16 → C
    rol r17         ; C → bit 0 of r17 (via rotate)
    dec r18
    brne bit_rev_loop
    mov r16, r17
    ret

```

Trace: r16 = 0b10110001

Iteration 1: lsl r17(0)→0, lsr r16→01011000 C=1, rol r17→0b00000001

Iteration 2: lsl r17→0b00000010, lsr r16→00101100 C=0, rol r17→0b00000010

...continuing...

Final: r16 = 0b10001101 (bits reversed) ✓

11.11.4 Pattern 4: Count Set Bits (Population Count / Hamming Weight)

Count the number of 1 bits in a byte:

```
/*
 * popcount8 – count set bits in r16
 * Input:  r16 = byte to examine
 * Output: r17 = number of set bits (0–8)
 * Clobbers: r18
 */
popcount8:
    clr r17          ; count = 0
    ldi r18, 8        ; 8 iterations

popcount_loop:
    lsr r16          ; bit 0 → C
    adc r17, r1       ; r17 += C (r1 = 0 per ABI)
    dec r18
    brne popcount_loop
    ret
```

ADC r17, r1 computes $r17 + 0 + C = r17 + C$. Since $r1 = 0$ (GCC ABI invariant), this adds 1 to the count only when a 1 bit was shifted out.

11.11.5 Pattern 5: Extract a 3-bit SPI Device Address from a Protocol Frame

A SPI command byte has the format: [7:5] = opcode, [4:2] = device address, [1:0] = flags.

```
; r16 = incoming command byte
; Extract device address (bits 4:2) into r17

mov r17, r16
andi r17, 0b0001100    ; mask bits 4:2 (= 0x1C)
lsr r17                ; bit 2 offset: shift right twice
lsr r17
; r17 = device address (0–7)

; Also extract opcode (bits 7:5) into r18
mov r18, r16
andi r18, 0b11100000    ; mask bits 7:5 (= 0xE0)
lsr r18
lsr r18
lsr r18
lsr r18
lsr r18
; r18 = opcode (0–7)

; Extract flags (bits 1:0) into r19
mov r19, r16
```



```
andi r19, 0b00000011    ; mask bits 1:0 (= 0x03)
; r19 = flags (0-3) – already at bit 0, no shift needed
```

11.11.6 Pattern 6: Build a SPI Command Byte from Fields

Reverse of the above — assemble a command byte from opcode, address, and flags:

```
; r20 = opcode (0-7), r21 = device address (0-7), r22 = flags (0-3)
; Build command byte in r16
```

```
andi r20, 0x07          ; ensure 3 bits
lsl  r20
lsl  r20
lsl  r20
lsl  r20
lsl  r20                ; opcode in bits 7:5

andi r21, 0x07          ; ensure 3 bits
lsl  r21
lsl  r21                ; address in bits 4:2

andi r22, 0x03          ; ensure 2 bits (already at bits 1:0)

or   r20, r21
or   r20, r22
mov  r16, r20           ; r16 = assembled command byte
```

Trace: opcode=5 (0b101), address=3 (0b011), flags=2 (0b10)

```
opcode: 0b00000101 → <<5 → 0b10100000
address: 0b00000011 → <<2 → 0b00001100
flags: 0b00000010 (no shift)
```

```
OR: 0b10100000 | 0b00001100 | 0b00000010 = 0b10101110 = 0xAE ✓
Decode: bits 7:5 = 101 = 5 ✓, bits 4:2 = 011 = 3 ✓, bits 1:0 = 10 = 2 ✓
```

11.11.7 Pattern 7: Rotate a 32-bit Value Through a Peripheral (Shift Register Output)

Output a 32-bit value MSB-first via a GPIO pin by shifting bit by bit:

```
/*
 * shift_out32 – output r25:r24:r23:r22 MSB-first on DATA pin
 * Assumes DATA = PORTB pin 1, CLK = PORTB pin 0
 * 32 clock pulses are generated. No return value.
 */
shift_out32:
    ldi  r20, 32          ; 32 bits to send

shift_out_loop:
    ; Output MSB of r25 on DATA pin
    sbrs r25, 7           ; skip next if bit 7 is set
    cbi  VPORTB_OUT, 1    ; bit 7 = 0: DATA low
    sbrs r25, 7           ; skip next if bit 7 is clear
    sbi  VPORTB_OUT, 1    ; bit 7 = 1: DATA high

    ; Pulse clock
    sbi  VPORTB_OUT, 0    ; CLK high
```

```

cbi  VPORTB_OUT, 0          ; CLK low

; Shift left: next MSB moves into bit 7 of r25
lsl  r22
rol  r23
rol  r24
rol  r25

dec  r20
brne shift_out_loop
ret

```

11.12 Instruction Reference

Instruction	Operands	Cyc	Flags	Notes
AND Rd, Rr	r0–r31, r0–r31	1	S,V=0,N,Z; C unch	Bitwise AND
ANDI Rd, K	r16–r31; K=0–255	1	S,V=0,N,Z; C unch	AND with immediate
OR Rd, Rr	r0–r31, r0–r31	1	S,V=0,N,Z; C unch	Bitwise OR
ORI Rd, K	r16–r31; K=0–255	1	S,V=0,N,Z; C unch	OR with immediate
EOR Rd, Rr	r0–r31, r0–r31	1	S,V=0,N,Z; C unch	Bitwise XOR; CLR Rd = EOR Rd,Rd
COM Rd	r0–r31	1	C=1,S,V=0,N,Z	Bitwise NOT (~Rd)
LSL Rd	r0–r31	1	H,C,Z,N,V,S	Shift left; bit 7→C; 0→bit 0
LSR Rd	r0–r31	1	C,Z,N=0,V,S	Shift right; 0→bit 7; bit 0→C
ASR Rd	r0–r31	1	C,Z,N,V,S	Arith shift right; b7 preserved
ROL Rd	r0–r31	1	H,C,Z,N,V,S	Rotate left through C
ROR Rd	r0–r31	1	C,Z,N,V,S	Rotate right through C
SWAP Rd	r0–r31	1	none	Swap nibbles; no flags
BST Rr, b	r0–r31; b=0–7	1	T only	T ← bit b of Rr
BLD Rd, b	r0–r31; b=0–7	1	none	bit b of Rd ← T
SBRC Rr, b	r0–r31; b=0–7	1/2	none	Skip if bit b of Rr = 0
SBRs Rr, b	r0–r31; b=0–7	1/2	none	Skip if bit b of Rr = 1
SBIC A, b	A=0–31; b=0–7	1/2	none	Skip if I/O bit clear; low I/O only
SBIS A, b	A=0–31; b=0–7	1/2	none	Skip if I/O bit set; low I/O only
SBI A, b	A=0–31; b=0–7	2	none	Set I/O bit (atomic)
CBI A, b	A=0–31; b=0–7	2	none	Clear I/O bit (atomic)
CLR Rd	r0–r31	1	Z=1,N=0,V=0,S=0	Alias for EOR Rd,Rd; C unchanged
SER Rd	r16–r31	1	none	Alias for LDI Rd,0xFF
TST Rd	r0–r31	1	Z,N,V=0,S	Alias for AND Rd,Rd; Rd unchanged

11.13 Common Pitfalls

11.13.1 Pitfall 1: Using OR to Add Numbers

OR is bitwise union, not addition. When the same bit is set in both operands, OR produces 1 (it does not carry):

```

ldi  r16, 0b00001111 ; = 15
ldi  r17, 0b00001111 ; = 15
or   r16, r17         ; r16 = 0b00001111 = 15 ← not 30!
add  r16, r17         ; r16 = 0b00001110 = 30 ← correct

```

Use ADD for numeric addition. Use OR for bit setting.

11.13.2 Pitfall 2: EOR Does Not Use an Immediate Operand

There is no EORI instruction. To XOR with a constant, load the constant into a scratch register first:

```
; Toggle bits 3 and 0 of r16
; WRONG – no EORI instruction:
; eori r16, 0x09

; CORRECT:
ldi r17, 0x09
eor r16, r17
```

11.13.3 Pitfall 3: COM Is Not NEG

COM computes $\sim R_d$ (flip all bits). NEG computes $-R_d$ (two's complement negation = $\sim R_d + 1$). They differ by 1 for all nonzero inputs:

```
ldi r16, 0x01      ; +1
com r16            ; r16 = 0xFE = -2 ← NOT -1
neg r16           ; r16 = 0xFF = -1 ← correct negate
```

11.13.4 Pitfall 4: Logic Instructions Do Not Preserve C

AND, OR, and EOR leave C unchanged. If you plan to use C after a masking operation, the logic instruction will not corrupt it — but if you intended C to reflect the logic result, it does not.

```
add r16, r17      ; C=1 (carry occurred)
andi r16, 0xF0    ; C is still 1 from the ADD – ANDI did not change it
brcs still_from_add ; this branch uses the carry from the ADD, not the ANDI
```

Keep arithmetic and bit-manipulation flag reading clearly separated.

11.13.5 Pitfall 5: SBRC/SBRS Skip Only One Instruction

SBRC and SBRS skip exactly one instruction — not a block of code. If the skipped instruction is itself a branch (RJMP, BREQ, etc.), that one instruction is skipped:

```
; Execute a block if bit 3 is set
sbrs r16, 3      ; skip next instruction if bit 3 SET
rjmp block_done  ; skipped when bit 3 set → fall through to block
; block executes here (when bit 3 is set)
block:
    ; ...
    rjmp block_done
block_done:
```

The skip + rjmp pattern is the standard idiom for conditional block execution with SBRC/SBRS.

11.13.6 Pitfall 6: Field Mask Must Match the Shift Count

When inserting a field, the number of shift positions must exactly equal the field's starting bit number, and the field mask must cover exactly those bits:

```
; Insert 3-bit value (bits 2:0 of r17) into bits 5:3 of r16
; Correct: shift by 3, mask = 0b00111000 = 0x38
andi r17, 0x07      ; 3 bits of value
lsl r17
```

```

lsl r17
lsl r17          ; shifted up by 3 (bits 5:3 position)
andi r16, ~0x38  ; clear bits 5:3 in destination
or r16, r17      ; insert

; WRONG: shift by 4 (misaligned):
; lsl r17 × 4 → value in bits 6:4, not 5:3 – off by one

```

11.13.7 Pitfall 7: SBI/CBI Only Reach the Low 32 I/O Addresses

SBI, CBI, SBIC, and SBIS can only address I/O registers at addresses 0x00–0x1F in the I/O space (which corresponds to 0x20–0x3F in the data space). Higher peripheral registers require IN + ANDI/ORI + OUT or LDS/ANDI/ORI/STS.

On the ATtiny3217, most peripherals use the VPORT mechanism: each port's virtual registers (at 0x00–0x1F I/O space) are accessible with SBI/CBI.

11.14 Summary

Bitwise logic:

```

AND Rd, Rr / ANDI Rd, K  – force bits to 0 (mask to clear)
OR  Rd, Rr / ORI  Rd, K  – force bits to 1 (mask to set)
EOR Rd, Rr              – toggle bits (no immediate form)
COM Rd                  – invert all bits (~Rd); C=1 always; NOT the same as NEG
All: V=0 after; S = N; C unchanged (except COM sets C=1).

```

Masking idioms:

```

Clear bits N: ANDI Rd, ~(1<<N)      – mask has 0 at N, 1 elsewhere
Set bit N:    ORI  Rd, (1<<N)        – mask has 1 at N, 0 elsewhere
Toggle bit N: load mask, EOR          – no EORI instruction
Test bit N:   ANDI Rd, (1<<N) + BREQ/BRNE, or SBRC/SBRS (preferred)
Read-modify-write: LDS/IN → ANDI → ORI → STS/OUT

```

Shifts (1 cycle each):

```

LSL – left, 0 fills bit 0, bit 7 → C (unsigned ×2)
LSR – right, 0 fills bit 7, bit 0 → C (unsigned ÷2)
ASR – right, sign bit preserved (signed ÷2, rounds toward −∞)
ROL – rotate left through C (chain for multi-byte left shift: LSL + ROL...)
ROR – rotate right through C (chain for multi-byte right shift: LSR high, ROR low...)
SWAP – swap nibbles, no flags (shortcut for 4-position shift)

```

Multi-byte shifts: left = LSL low, ROL high; right = LSR high, ROR low.

Shift by N: repeat the pair N times, or use register-move for byte shifts.

Single-bit operations:

```

SBRC/SBRS Rr, b – skip next if bit b of Rr is 0/1 (preferred for branches)
BST/BLD         – copy bit to/from T flag (copy between non-aligned positions)
SBI/CBI A, b    – set/clear I/O bit atomically (I/O 0x00–0x1F only)
SBIC/SBIS A, b  – skip if I/O bit clear/set (same range)

```

Field extraction:

```

Lower nibble: ANDI Rd, 0x0F
Upper nibble: MOV + SWAP + ANDI 0x0F
Arbitrary [hi:lo]: ANDI with field mask, then LSR × lo

```

Field insertion (value into bits [hi:lo] of destination):

1. ANDI value with width mask (prevent overflow)
2. LSL value by lo positions (align to target)
3. ANDI dest with ~field mask (clear target bits)
4. OR dest, value (insert)

Packing two nibbles: ANDI + SWAP one, ANDI the other, OR together.

Unpacking nibbles: MOV + SWAP + ANDI 0x0F for high; ANDI 0x0F for low.

Bit reversal: 8-iteration loop: LSR source → C → ROL accumulator.

Population count: 8-iteration loop: LSR source → C → ADC count, r1.

11.15 Exercises

1. A peripheral status register contains these fields: [7:6] = mode (2 bits), [5:3] = channel (3 bits), [2:1] = rate (2 bits), [0] = enable (1 bit). Write the assembly sequence to:
 - a. Extract the channel field into r17 (result in bits 2:0 of r17, 0-7)
 - b. Set the enable bit without affecting any other field
 - c. Write mode=2, channel=5, rate=1, enable=1 as a single assembled byte
2. A bitfield byte in r16 encodes eight boolean flags. Write a subroutine `get_flag(r16, r17) → r18` where r17 is the bit number (0-7) and r18 returns 0 or 1. Hint: use a shift loop or look up whether there's a single-instruction approach using BST.
3. Implement `bit_reverse8` without a loop: unroll all 8 iterations using LSR + ROL pairs and verify the result for input 0b10110001. How many instruction words does the unrolled version use compared to the loop?
4. A 16-bit value in r25:r24 needs to be shifted right by 3 bits (logical). Write the 6-instruction sequence (LSR + ROR × 5). Then write a version that shifts by 8 bits using only register moves — how many instructions?
5. Write a subroutine `pack_nybbles(r16, r17) → r16` that takes the low nibble of r16 as the upper nibble and the low nibble of r17 as the lower nibble of the result. Verify with inputs r16=0x04, r17=0x09 → 0x49.
6. The GPIO input register holds a byte where bits 7, 5, 3, and 1 are connected to four buttons (bit = 1 when pressed). Write a sequence that compacts these four button states into bits 3:0 of r17, discarding the gaps. (Hint: extract each bit with BST/BLD or with shifts and OR.)
7. Implement `popcount8` without a loop by using the parallel bit-count technique: first count pairs, then nibbles, then the full byte using only AND, ADD, and shift operations. (Hint: for two-bit groups, `count = (x & 0x55) + ((x >> 1) & 0x55)`.) How does cycle count compare to the loop version?
8. Write a read-modify-write sequence that atomically (with respect to the rest of the main-loop code — not interrupt-safe) configures bits 5:3 of a peripheral control register `PE-RIPH_CTRL` to the value 0b110 (6), without disturbing any other bits. Write it twice: once for a register reachable by SBI/CBI, and once for a register that requires LDS/STS.
9. A 32-bit shift register protocol requires data MSB-first. Rewrite `shift_out32` from the Practical Patterns section to also capture 32 incoming MISO bits into r7:r6:r5:r4 using ROR on the captured byte after each clock pulse.
10. Using only logic instructions (no ADD, no SUB), compute:
 - a. `r16 AND (r16 - 1)` — what does this value represent about r16? (Hint: think about what the subtraction does to the lowest set bit.)
 - b. `r16 OR (r16 - 1)` — similarly, what does this represent? Implement both using registers (you may use SUBI for the -1 step) and test with r16 = 0b00101000.

Next: Chapter 12 — Fixed-Point Arithmetic

Chapter 12

Fixed-Point Arithmetic

Floating-point hardware does not exist on the ATtiny3217. When you need fractional values — sensor gains, filter coefficients, percentages, scaled ADC readings — you carry the decimal point in your head and use integer arithmetic to do the work. This is fixed-point arithmetic.

This chapter covers fixed-point arithmetic from the representation through every operation you will need in practice:

1. Q-format: how the decimal point is encoded in an integer register.
2. Range and resolution for the formats used most on 8-bit AVR.
3. Addition and subtraction.
4. Multiplication — the hard part, worked out in full.
5. The AVR fractional multiply instructions: FMUL, FMULS, FMULSU.
6. Division and reciprocal multiplication.
7. Conversion between fixed-point and integer.
8. Rounding and truncation.
9. Saturation and overflow detection.
10. Three complete worked examples: ADC gain scaling, a first-order low-pass filter, and a PID derivative term.

12.1 What Fixed-Point Is

The CPU stores integers. Fixed-point assigns a constant scale factor to those integers so they represent non-integer values. If you decide that the integer 256 means 1.0, then 128 means 0.5 and 384 means 1.5. You and the code agree on the scale. The CPU just does integer math.

The formal notation is **Q-format**. A Q_{m.n} number has:

- m integer bits above the binary point
- n fractional bits below the binary point
- Total width: m + n bits (for unsigned), m + n + 1 bits (for signed, one bit for the sign)

The value represented is:

$$\text{value} = \text{raw_integer} \times 2^{-n}$$

Some examples, all unsigned:

Format	Width	Range	Resolution	Scale
Q0.8	8-bit	0 to 255/256	1/256	raw/256
Q1.7	8-bit	0 to 255/128	1/128	raw/128

Q4.4	8-bit	0 to 15.9375	1/16	raw/16
Q8.8	16-bit	0 to 255.996	1/256	raw/256
Q8.24	32-bit	0 to 255.9999999	1/16777216	raw/2 ²⁴

Signed Q-format adds a two's-complement sign bit. A signed Q1.7 number has 1 integer bit, 7 fractional bits, and 1 implicit sign bit — total 8 bits, range -1.0 to $+127/128$.

The most common formats on 8-bit AVR:

Format	Width	Unsigned range	Signed range	Typical use
Q0.8	8	[0, 1)	—	Blend factors, alpha
Q1.7	8	[0, 2)	[-1, +127/128]	Unit fractions, trig
Q4.4	8	[0, 16)	[-8, +7.9375]	Small scaled values
Q8.8	16	[0, 256)	[-128, +127.996]	General purpose
Q8.16	24	[0, 256)	[-128, +127.99999]	High-res sensor data
Q16.16	32	[0, 65536)	[-32768, +32767.99]	Extended precision

12.2 Representation in Registers

A Q8.8 value occupies two registers. By convention in this chapter:

Register pair	Contents
---------------	----------

r_H : r_L	integer byte : fractional byte
-----------	--------------------------------

For example, 3.5 in Q8.8: $-3.5 \times 256 = 896 = 0x0380$ - r_H = 0x03, r_L = 0x80

There is no special instruction or flag for fixed-point. The CPU treats the register pair as a plain 16-bit integer. Only your interpretation gives it meaning.

12.3 Addition and Subtraction

Fixed-point addition and subtraction are identical to integer addition and subtraction. The scales cancel:

$$(a \times 2^{-n}) + (b \times 2^{-n}) = (a + b) \times 2^{-n}$$

Both operands must be in the same Q format. If they are not, align them first by shifting the lower-resolution operand.

12.3.1 Q8.8 Addition

```
; Add two Q8.8 values.
; A in r17:r16, B in r19:r18 → result in r17:r16.
add r16, r18      ; fractional bytes
adc r17, r19      ; integer bytes, carry from fractional
```

Overflow: if the integer byte wraps, the addition overflowed. The V and C flags from ADC signal this, identical to unsigned 16-bit overflow detection from chapter 5a.

12.3.2 Q8.8 Subtraction

```
; Subtract: A = A - B.
sub r16, r18      ; fractional bytes
sbc r17, r19      ; integer bytes, borrow from fractional
```

12.4 Multiplication

Multiplication is where fixed-point diverges from plain integer work. When you multiply two Q8.8 numbers the result has twice as many fractional bits and needs to be shifted back:

$$(a \times 2^{-8}) \times (b \times 2^{-8}) = (a \times b) \times 2^{-16}$$

To express the result as Q8.8 (scale 2^{-8}), divide the raw 32-bit product by 2^8 — that is, discard the lowest 8 bits and keep the next 16:

```
raw 32-bit product:  bits[31:24] bits[23:16] bits[15:8] bits[7:0]
                    overflow  integer    fractional sub-frac
                    ——— Q8.8 result ———
```

The AVR has no 16×16 multiply instruction, so you build it from four 8×8 partial products using MUL.

12.4.1 The MUL Family

Instruction	Operands	Result	Register constraints
MUL Rd, Rr	unsigned × unsigned	R1:R0 = Rd×Rr	Rd, Rr ∈ R0–R31
MULS Rd, Rr	signed × signed	R1:R0 = Rd×Rr	Rd, Rr ∈ R16–R31
MULSU Rd, Rr	signed × unsigned	R1:R0 = Rd×Rr	Rd, Rr ∈ R16–R23
FMUL Rd, Rr	unsigned × unsigned	R1:R0 = (Rd×Rr)<<1	Rd, Rr ∈ R16–R23
FMULS Rd, Rr	signed × signed	R1:R0 = (Rd×Rr)<<1	Rd, Rr ∈ R16–R23
FMULSU Rd, Rr	signed × unsigned	R1:R0 = (Rd×Rr)<<1	Rd, Rr ∈ R16–R23

MUL always places the result in R1:R0. **The GCC/avr-libc ABI assumes R1 is zero at all times except during a multiply.** Clear R1 with `clr r1` after every MUL family instruction before any call, interrupt return, or branch that may reach code expecting R1 = 0.

12.4.2 Unsigned Q8.8 Multiplication

Label registers: A_H:A_L = r17:r16, B_H:B_L = r19:r18. The 32-bit product occupies a four-register accumulator p3:p2:p1:p0.

Partial products:

```
P00 = A_L × B_L → contributes to bits [15:0]
P01 = A_L × B_H → contributes to bits [23:8]
P10 = A_H × B_L → contributes to bits [23:8]
P11 = A_H × B_H → contributes to bits [31:16]
```

Q8.8 result = p2:p1 (integer byte p2, fractional byte p1). p0 is the sub-fractional byte used for rounding. p3 indicates overflow.

```
; mul_q8_8_unsigned – unsigned Q8.8 × Q8.8 → Q8.8
;
; Input:  A in r17:r16 (r17=integer, r16=fractional)
;         B in r19:r18 (r19=integer, r18=fractional)
; Output: result in r22:r21 (r22=integer, r21=fractional)
;         r20 = sub-fractional byte (for rounding)
;         r23 = overflow byte (non-zero = result exceeded Q8.8 range)
; Clobbers: r0, r1 (cleared on exit)
```

```
mul_q8_8_unsigned:
; P00 = A_L × B_L
```



```

mul    r16, r18      ; R1:R0 = A_L × B_L (unsigned)
movw   r20, r0       ; p1:p0 = R1:R0
clr    r22
clr    r23

; P01 = A_L × B_H
mul    r16, r19      ; R1:R0 = A_L × B_H
add    r21, r0       ; p1 += low byte
adc    r22, r1       ; p2 += high byte + carry
clr    r1
adc    r23, r1       ; p3 += carry

; P10 = A_H × B_L
mul    r17, r18      ; R1:R0 = A_H × B_L
add    r21, r0
adc    r22, r1
clr    r1
adc    r23, r1

; P11 = A_H × B_H
mul    r17, r19      ; R1:R0 = A_H × B_H
add    r22, r0
adc    r23, r1
clr    r1           ; restore R1 = 0

ret
; Result: Q8.8 integer byte in r22, fractional byte in r21.
; For truncated result use r22:r21 directly.
; For rounded result see rounding section below.

```

Worked trace — $1.5 \times 1.5 = 2.25$:

1.5 in Q8.8 = 0x0180 → r17=0x01, r16=0x80 B = same: r19=0x01, r18=0x80

P00: 0x80 × 0x80 = 0x4000 → p1:p0 = 0x40:0x00

P01: 0x80 × 0x01 = 0x0080 → p1 += 0x80 → p1=0xC0, p2=0x00

P10: 0x01 × 0x80 = 0x0080 → p1 += 0x80 → p1=0x40 (carry!), p2=0x01

P11: 0x01 × 0x01 = 0x0001 → p2 += 0x01 → p2=0x02

p3:p2:p1:p0 = 0x00:0x02:0x40:0x00

Q8.8 result = p2:p1 = 0x02:0x40 = 2.25 ✓

12.4.3 Signed Q8.8 Multiplication

When the integer bytes are signed (two's complement), the partial products use mixed-sign multiplies:

```

P00 = A_L × B_L      – unsigned × unsigned → MUL
P01 = B_H × A_L      – signed   × unsigned → MULSU (Rd=B_H signed)
P10 = A_H × B_L      – signed   × unsigned → MULSU (Rd=A_H signed)
P11 = A_H × B_H      – signed   × signed   → MULS

```

MULSU Rd, Rr treats Rd as signed and Rr as unsigned. Both Rd and Rr must be in R16–R23. After MULSU, the C flag equals bit 15 of R1:R0 (the sign of the 16-bit partial product). SBC Rx, Rx uses that C flag to produce a sign-extension byte: 0xFF if C=1, 0x00 if C=0. This byte is added into the top accumulator register to correctly sign-extend the partial product to 32 bits. This is the pattern documented in Microchip Application Note **AVR201** (Using the AVR Hardware Multiplier).

```

; mul_q8_8_signed - signed Q8.8 × Q8.8 → Q8.8
;
; Input:  A in r17:r16 (r17=signed integer, r16=unsigned fractional)
;         B in r19:r18 (r19=signed integer, r18=unsigned fractional)
;         All registers must be in R16–R23 (MULS/MULSU constraint).
; Output: Q8.8 result in r22:r21 (r22=signed integer, r21=fractional)
;         r20 = sub-fractional byte (for rounding)
;         r23 = sign-extended overflow byte
; Clobbers: r0, r1 (cleared on exit)

mul_q8_8_signed:
; P00: A_L × B_L (unsigned × unsigned)
mul    r16, r18
movw   r20, r0          ; p1:p0 = R1:R0
clr    r22
clr    r23

; P01: B_H(signed) × A_L(unsigned)
mulsu  r19, r16          ; MULSU Rd=r19(signed B_H), Rr=r16(unsigned A_L)
; C flag = bit15 of result = sign of partial product
sbc    r23, r23          ; r23 = 0xFF (negative product) or 0x00 (positive)
add    r21, r0           ; accumulate low byte
adc    r22, r1           ; accumulate high byte
adc    r23, r1           ; r23 += r1 + carry (r1 is high byte of product)
clr    r1               ; restore R1 = 0

; P10: A_H(signed) × B_L(unsigned)
mulsu  r17, r18          ; MULSU Rd=r17(signed A_H), Rr=r18(unsigned B_L)
sbc    r23, r23          ; sign-extend partial product
add    r21, r0
adc    r22, r1
adc    r23, r1
clr    r1

; P11: A_H(signed) × B_H(signed)
muls   r17, r19          ; MULS: both R16–R31 constraint, r17 and r19 qualify
add    r22, r0
adc    r23, r1
clr    r1

ret
; Q8.8 result: signed integer byte in r22, fractional byte in r21.
---

```

The FMUL Family: Fractional Multiply

The FMUL instructions are purpose-built for Q1.7 arithmetic. They multiply **and** shift the result left by 1 bit in a single instruction cycle:

FMUL Rd, Rr: $R1:R0 = (\text{unsigned Rd} \times \text{unsigned Rr}) \ll 1$ FMULS Rd, Rr: $R1:R0 = (\text{signed Rd} \times \text{signed Rr}) \ll 1$ FMULSU Rd, Rr: $R1:R0 = (\text{signed Rd} \times \text{unsigned Rr}) \ll 1$

Operands: Rd, Rr ∈ R16–R23.

Flags: C = bit 15 of the unshifted product (i.e. bit 16 of the shifted result, overflow indicator for Q1.7). Z = result is zero.

Why the Left Shift?

In Q1.7 format, each value has 1 integer bit and 7 fractional bits. When two Q1.7 numbers are multiplied, the raw 16-bit product has 2 integer bits and 14 fractional bits (Q2.14). The two integer bits are always a sign-extended copy of each other for values in $[-1, +1]$. Shifting left by 1 removes the redundant copy, returning a Q1.15 result in R1:R0 where R1 holds the Q1.7 high byte.

Q1.7 $\times$ Q1.7 product: [I][I][f6..f0][f6..f0] (Q2.14, 16-bit) After left shift: [I][f6..f0][f6..f0][0] (Q1.15, 16-bit) High byte R1: [I][f6..f0] (Q1.7 result)

Q0.8 Multiply with MUL

For unsigned Q0.8 (values 0 to 255/256), use plain `MUL` and keep the high byte R1:

```
``asm
; Q0.8  $\times$  Q0.8  $\rightarrow$  Q0.8.
; a in r16, b in r17. Result in r1 (Q0.8). r0 = sub-fractional byte.
mul    r16, r17      ; R1:R0 = a  $\times$  b (16-bit unsigned product)
; R1 = (a  $\times$  b)  $\gg$  8 = Q0.8 result.
; R0 = sub-fractional (used for rounding).
clr    r1            ; restore R1 = 0
```

Why R1? A Q0.8 value has implicit scale $1/256$. The product of two Q0.8 integers a and b is $(a \times b)$, and the Q0.8 result value is $(a \times b) / 256$, which is exactly R1 (the high byte of the 16-bit product).

Do not use FMUL for Q0.8. FMUL shifts the product left by 1, yielding $R1 = (a \times b) \gg 7$, which is twice the correct value. FMUL is designed specifically for Q1.7 format (scale $1/128$).

12.4.4 Q1.7 Signed Multiply with FMULS

For signed Q1.7 (values -1.0 to $+127/128$):

```
; Signed Q1.7  $\times$  Q1.7  $\rightarrow$  Q1.7.
; Inputs in r16 (signed), r17 (signed). Result in r1 (signed Q1.7).
fmuls  r16, r17
; R1 = Q1.7 result. C flag set if result would overflow Q1.7.
clr    r1
```

Special case: FMULS(-1.0 , -1.0) sets $C=1$ because $+1.0$ cannot be represented in signed Q1.7 (maximum is $+127/128$). Check C after FMULS when overflow is possible and saturate if needed.

12.4.5 Mixed-Sign Fractional Multiply with FMULSU

Used when one operand is a signed coefficient and the other is an unsigned signal sample (common in filters and control loops):

```
; signed Q1.7 coefficient in r16, unsigned Q0.8 sample in r17.
; Result in R1:R0 (signed Q1.15, or take R1 as signed Q1.7).
fmulsu r16, r17
clr    r1
```

12.5 Division

AVR has no divide instruction. Fixed-point division is handled two ways:

12.5.1 Divide by a Constant: Multiply by Reciprocal

Dividing by a constant d is equivalent to multiplying by $1/d$. Pre-compute the reciprocal as a Q0.16 constant and use a 16-bit multiply:

Example: divide Q8.8 value by 3.

$1/3$ in Q0.16 = round($65536 / 3$) = 21845 = 0x5555

```
; x / 3 using Q0.16 reciprocal
; x in r17:r16 (Q8.8)
ldi    r18, lo8(21845)      ; 0x55
ldi    r19, hi8(21845)      ; 0x55
; 16x16 unsigned multiply, keep bits[31:16] as the result
; (shifting right by 16 after multiply by 1/2^16 reciprocal)
; Full 32-bit product needed; bits [31:16] = x/3 in Q8.8.
```

The 32-bit product of a Q8.8 value by a Q0.16 reciprocal gives a Q8.24 full product; bits [31:16] of that product are the Q8.8 result.

12.5.2 Divide by a Power of Two: Arithmetic Shift Right

```
; Divide Q8.8 value in r17:r16 by 4 (shift right 2).
asr    r17                ; arithmetic shift right of high byte
ror    r16                ; rotate right of low byte (with carry from asr)
asr    r17
ror    r16
```

ASR preserves the sign bit. ROR passes the carry bit into the MSB, completing the 16-bit right shift. Two iterations divide by 4.

12.5.3 Variable Division: Software Routine

For variable divisors, use a software long-division routine or restructure the algorithm to use only multiplies (often possible in control and signal processing code).

12.6 Type Conversion

12.6.1 Integer to Fixed-Point

Shift left by n bits (the number of fractional bits):

```
; Convert 8-bit unsigned integer in r16 to Q8.8 in r17:r16.
mov    r17, r16           ; integer part
clr    r16                ; fractional part = 0
```

For Q4.4 from a 4-bit integer: shift left 4.

```
; 8-bit integer in r16 → Q4.4 in r16 (same byte, fractional bits zeroed).
; Integer value must be in [0,15] to fit.
swap   r16                ; swap nibbles: integer nibble → high nibble
andi   r16, 0xF0          ; zero the low (fractional) nibble
```

12.6.2 Fixed-Point to Integer (Truncation)

Discard the fractional byte:

```
; Q8.8 in r17:r16 → integer in r17 (truncated toward zero).
; r16 (fractional byte) is discarded.
```

For Q4.4: shift right 4.

```
swap    r16
andi    r16, 0x0F      ; keep high nibble as integer in low nibble
```

12.6.3 Fixed-Point to Integer with Rounding

Add 0.5 (half LSB of the integer part) before truncating:

```
; Q8.8 in r17:r16 → rounded integer in r17.
ldi     r18, 0x80      ; 0.5 in Q0.8
add     r16, r18        ; add 0.5 to fractional byte
adc     r17, r1         ; carry into integer byte (r1 = 0)
; r17 is now the rounded integer result.
```

12.6.4 Changing Q Format

To convert from Q_m._n to Q_m._k (same total width, different split):

- If $k > n$ (more fractional bits): shift left $(k - n)$ bits.
- If $k < n$ (fewer fractional bits): shift right $(n - k)$ bits (truncate or round).

12.7 Rounding

Truncation is the default (just drop bits). Rounding to nearest adds 0.5 ULP (unit in the last place of the result) before truncating.

For Q8.8 multiplication, r20 holds the sub-fractional byte. Round by adding the MSB of r20 as a carry into the result:

```
; After mul_q8_8_unsigned, result in r22:r21, sub-frac in r20.
lsl     r20              ; shift sub-frac MSB into carry
adc     r21, r1           ; round fractional byte (r1 = 0)
adc     r22, r1           ; carry into integer byte
; r22:r21 = rounded Q8.8 result.
```

12.8 Saturation

When fixed-point addition or multiplication overflows, the result wraps. For control systems and audio processing, **saturation arithmetic** clamps the result to the representable maximum or minimum instead.

12.8.1 Saturating Q8.8 Addition (Unsigned)

```
; Saturating add: r17:r16 + r19:r18 → r17:r16, clamped to [0, 255.996].
add     r16, r18
adc     r17, r19
brcc    .no_overflow_add ; C clear = no overflow
```

```
ldi    r16, 0xFF          ; saturate to 0xFF:0xFF (maximum Q8.8)
ldi    r17, 0xFF
.no_overflow_add:
```

12.8.2 Saturating Q8.8 Addition (Signed)

For signed Q8.8, overflow occurs when two positives produce a negative or two negatives produce a positive. The V flag signals this:

```
; Signed saturating add.
add    r16, r18
adc    r17, r19
brvs   .signed_overflow    ; V set = signed overflow
rjmp   .done_add
.signed_overflow:
; If result MSB is 0 after overflow, we went positive→negative (underflow).
; Set to 0x8000 (minimum). Otherwise set to 0x7FFF (maximum).
ldi    r16, 0x00
ldi    r17, 0x80          ; assume underflow: minimum = -128.0
sbrs   r17, 7             ; if result MSB was 1 (went negative→positive)
ldi    r16, 0xFF          ; maximum = +127.996
sbrs   r17, 7
ldi    r17, 0x7F
.done_add:
```

12.8.3 Overflow Detection After Multiplication

After mul_q8_8_unsigned, r23 is non-zero if the result exceeds 255.996:

```
tst    r23
brne   .multiply_overflow
```

12.9 Practical Examples

12.9.1 Example 1: ADC Gain Scaling

An ADC returns a 10-bit result (0–1023). You want to convert it to millivolts given $V_{CC} = 3300$ mV:

$$\begin{aligned} \text{mV} &= \text{adc_result} \times (3300 / 1024) \\ &= \text{adc_result} \times 3.22265625 \end{aligned}$$

Express the gain 3.22265625 as Q8.8: $3.22265625 \times 256 = 825 = 0x0339$. Gain: r_gain_H = 0x03, r_gain_L = 0x39.

The ADC result is a 10-bit integer. Treat it as Q8.8 by storing it as-is in a 16-bit pair (the value is an integer, so fractional part = 0x00):

```
; ADC result in r25:r24 (10-bit integer, max 0x03FF).
; Gain 0x0339 in r21:r20.
; Result (millivolts, Q8.8) in r23:r22:r21:r20 after multiply.

; Load gain
ldi    r20, lo8(825)      ; 0x39
ldi    r21, hi8(825)      ; 0x03

; The ADC integer value treated as Q8.8 (fractional byte = 0):
```

```

; A_H:A_L = r25:r24, B_H:B_L = r21:r20

; P00 = 0 × gain_L = 0 (A_L = 0 since integer)
; P01 = 0 × gain_H = 0
; P10 = adc_L × gain_L
mul    r24, r20      ; R1:R0 = adc_L × gain_L
movw   r22, r0       ; p1:p0

; P11 is split because adc is 16-bit:
; P10' = adc_H × gain_L
mul    r25, r20
add    r23, r0       ; note: result fits in r23 for typical ADC values

; P01' = adc_L × gain_H
mul    r24, r21
add    r23, r0
clr    r1

; P11' = adc_H × gain_H
mul    r25, r21
add    r23, r0       ; overflow lands here (can be checked)
clr    r1

; mV result (Q8.8 integer part) in r23 for typical values.
; For exact result including fractional, r23:r22 is the Q8.8 mV value.

```

For a cleaner implementation, call `mul_q8_8_unsigned` directly with the ADC result as the Q8.8 value (integer byte = high bits, fractional byte = 0).

12.9.2 Example 2: First-Order Low-Pass Filter

A single-pole IIR low-pass filter:

$$y[n] = y[n-1] + \alpha \times (x[n] - y[n-1])$$

where $\alpha \in (0, 1)$ is the smoothing factor in Q0.8. A smaller α gives more smoothing.

```

; Low-pass filter step.
; x (new sample) in r17:r16 (Q8.8)
; y (previous output) in r19:r18 (Q8.8)
; alpha in r20 (Q0.8, e.g. 0x1A ≈ 0.1 for heavy smoothing)
; Result (new y) written back to r19:r18.
;
; Steps:
; 1. err = x - y (Q8.8 subtraction)
; 2. adj = alpha × err (Q0.8 × Q8.8 multiply, keep Q8.8 result)
; 3. y = y + adj

; Step 1: err = x - y (r17:r16 - r19:r18)
sub    r16, r18      ; fractional
sbc    r17, r19      ; integer (with borrow)
; err in r17:r16

; Step 2: adj = alpha × err
; This is Q0.8 × Q8.8. The result is Q8.8:
; alpha (Q0.8) × err_H (Q8.8 integer) → partial Q8.8
; alpha (Q0.8) × err_L (Q8.8 fractional) → partial sub-fractional
;
; Result = (alpha × err_H) giving integer and fractional parts,

```

```

;           plus (alpha × err_L) >> 8 as the fractional contribution.

mul    r20, r16      ; alpha × err_L (unsigned × unsigned)
mov    r22, r1       ; save high byte (fractional contribution)
clr    r1

mul    r20, r17      ; alpha × err_H (unsigned × signed → needs MULSU)
; For signed err_H, use Mulsu:
; mulsu r17, r20 (but r17 may not be in r16-r23 – adjust if needed)
; Assuming registers are arranged in R16–R23:
add    r22, r0       ; add low byte of (alpha × err_H)
clc
adc    r23, r1       ; add high byte (integer of result)
clr    r1
; adj in r23:r22 (Q8.8)

; Step 3: y = y + adj
add    r18, r22      ; fractional
adc    r19, r23      ; integer
; Updated y in r19:r18.

```

12.9.3 Example 3: PID Derivative Term

A PID controller derivative term scaled by a Q8.8 coefficient Kd:

$$D = K_d \times (\text{error} - \text{prev_error})$$

```

; error in r17:r16 (Q8.8, signed)
; prev_error in r19:r18 (Q8.8, signed)
; Kd in r21:r20 (Q8.8, unsigned coefficient)
; Result D in r22:r21 (Q8.8)

; delta = error - prev_error
sub    r16, r18
sbc    r17, r19
; delta in r17:r16

; D = Kd × delta (signed Q8.8 multiply)
; Use mul_q8_8_signed with r17:r16 and r21:r20.
; Call mul_q8_8_signed – result in r22:r21.
rcall  mul_q8_8_signed

```

12.10 Choosing a Format

Situation	Recommended format
Blend factor, alpha 0–1	Q0.8 (unsigned)
Unit trig values –1 to +1	Q1.7 (signed) with FMULS
ADC scaling, general fractions	Q8.8
Control coefficients (medium range)	Q8.8 signed
High-resolution sensor accumulator	Q8.16 or Q16.16
Intermediate multiply accumulator	Use 32-bit (four registers)

Pick the narrowest format that covers your range with the resolution you need. Wider formats cost more instructions per operation. Q8.8 is the default starting point for most general-purpose fixed-point work on 8-bit AVR.

12.11 Summary

Fixed-point arithmetic on AVR is integer arithmetic with a documented scale. The key rules:

- **Same Q format to add/subtract.** The hardware does plain ADD/SUB.
- **Multiply shifts the binary point:** $Q8.8 \times Q8.8 = Q16.16$ raw; extract bits [23:8] to get the Q8.8 result.
- **Four partial products** for 16×16 : use MUL, MULS, MULSU according to signedness. Follow Microchip AVR201 for the complete signed version.
- **FMUL family** is purpose-built for Q1.7/Q0.8: one instruction multiplies and shifts. Operands must be in R16–R23. Clear R1 after every MUL-family instruction.
- **Always clear R1** before any code that assumes $R1 = 0$ (calls, returns, interrupts).
- **Saturation prevents wrap-around** for control and audio code.
- **Pre-computed reciprocals** replace division by constants.

Chapter 13

Arithmetic and Logic: 16-bit and Beyond

Chapters 8 and 9 covered unsigned and signed 8-bit and 16-bit arithmetic. Chapter 10 extended the carry-chain principle to 24-bit, 32-bit, and wider values. Chapter 11 covered bit manipulation: masks, field extraction and insertion, packing, and the bitwise logic instructions. This chapter completes the toolkit: shift and rotate instructions used for arithmetic, the hardware multiply family, and software division.

Real programs quickly outgrow a single register. Addresses are 16-bit, timers are often 16-bit, counters spill past 255, and scaled sensor values rarely fit in a single byte for long. This chapter shows how AVR handles arithmetic once one byte is no longer enough.

The key idea is simple: AVR does not magically become a 16-bit machine when you want a 16-bit result. You build larger arithmetic explicitly out of 8-bit pieces, and the carry flag is what ties those pieces together.

13.1 Multi-Byte Addition and Subtraction

The AVR has no 16-bit ADD instruction. You build it from two 8-bit instructions using the carry flag as a bridge between bytes.

That sounds more awkward than it is. Once you trust the carry chain, multi-byte arithmetic becomes a repeatable pattern rather than a special trick.

Microchip's instruction summary is useful here because it separates the two classes of arithmetic flags:

Flag	Meaning in this chapter
C	Carry for unsigned addition; borrow for unsigned subtraction
Z	Result is zero
N	Result bit 7 is set for 8-bit ops, or bit 15 for ADIW/SBIW
V	Two's-complement signed overflow
S	$N \text{ xor } V$, used by signed branches such as BRLT/BRGE
H	Half-carry/half-borrow between bits 3 and 4, mainly for BCD-style code

For multi-byte arithmetic, the important flags are C for unsigned carry/borrow and V/S/N for signed interpretation of the final high byte.

13.1.1 The Carry Chain Pattern

Always start with the least-significant byte. Never with the most-significant.

```
; 16-bit add: r25:r24 += r23:r22
add r24, r22      ; low bytes – may produce carry
adc r25, r23      ; high bytes – ADC includes carry from ADD
```

```
; 16-bit subtract: r25:r24 -= r23:r22
sub r24, r22      ; low bytes – may produce borrow (C=1)
sbc r25, r23      ; high bytes – SBC subtracts borrow
```

The carry flag is the pipeline between bytes. Break the chain and the high byte has the wrong value.

That is why instruction order matters here. Arithmetic is not just about the operands; it is also about preserving the flag state from one byte to the next.

13.1.2 24-bit and 32-bit

The pattern extends to any width:

```
; 32-bit add: r3:r2:r1:r0 += r7:r6:r5:r4
add r0, r4
adc r1, r5
adc r2, r6
adc r3, r7
```

```
; 32-bit subtract: r3:r2:r1:r0 -= r7:r6:r5:r4
sub r0, r4
sbc r1, r5
sbc r2, r6
sbc r3, r7
```

After an addition chain: if C=1, unsigned overflow occurred (result too large for N bytes). After a subtraction chain, C=1 means unsigned borrow/underflow. If V=1, signed overflow occurred.

So after a multi-byte operation, the flags still matter. The CPU is telling you whether the final N-byte result wrapped or overflowed in the signed sense.

One subtle detail from the AVR instruction manual: SBC preserves the previous Z value when its own byte result is zero. That is deliberate. It means a multi-byte subtract or compare leaves Z=1 only if **all** bytes in the chain were zero/equal.

13.1.3 Adding a Small Constant to a 16-bit Register Pair

ADIW handles constants 0–63 in 2 cycles, but only on the upper even register pairs:

Low register operand	Full pair
r24	r25:r24
r26 / XL	XH:XL
r28 / YL	YH:YL
r30 / ZL	ZH:ZL

```
adiw r24, 1      ; r25:r24 += 1 – 2 cycles, 1 word
adiw ZL, 16     ; Z += 16
```

SBIW has the same register-pair and immediate limits, but subtracts:

```
sbiw r24, 1      ; r25:r24 -= 1
sbiw ZL, 16     ; Z -= 16
```

ADIW/SBIW update Z, C, N, V, and S. They do not update H. That differs from byte ADD/ADC/SUB/SBC, which do update H.

For constants that fit the target width, use SUBI+SBCI with negated values — because there is no ADDI:

```
; r25:r24 += 100
subi r24, lo8(-100)    ; subtract -100 = add 100
sbcI r25, hi8(-100)    ; propagate carry
```

SUBI and SBCI only work on registers r16-r31. If your value is in a lower register pair, move it or use a register-register add/subtract sequence.

For a 16-bit constant, use the same pattern:

```
; r25:r24 += 1000
subi r24, lo8(-1000)
sbcI r25, hi8(-1000)
```

If the destination pair is not in r16-r31, load the constant into a register pair and use ADD+ADC:

```
; r25:r24 += 1000
ldi r22, lo8(1000)
ldi r23, hi8(1000)
add r24, r22
adc r25, r23
```

This gives you three practical tools for “add a constant”:

- ADIW when the constant is small and the register pair is supported
- SUBI/SBCI when the destination pair is in r16-r31
- ADD/ADC when you want the general case

13.1.4 Negating a 16-bit Value

Two’s complement negate: invert all bits, add 1.

```
; negate r25:r24
com r24      ; ~low
com r25      ; ~high
adiw r24, 1  ; + 1 (or: ldi r16,1 / add r24,r16 / adc r25,r1)
```

Or equivalently using NEG + SBC:

```
neg r25      ; high = 0 - high
neg r24      ; low = 0 - low, C=1 if low was non-zero
sbc r25, r1   ; subtract the borrow from high (r1 must be zero)
```

That order looks backwards, but it is intentional: NEG r24 must run before the final SBC so the carry flag tells whether the low-byte negate borrowed from the high byte.

The reason is that it fits naturally into the carry/borrow model the CPU already uses for subtraction.

A correct register-register form that avoids ADIW is:

```
com r24
com r25
ldi r16, 1
```

```
add  r24, r16
adc  r25, r1      ; r1 must be zero
```

For signed 8-bit values, NEG has one important edge case: 0x80 (-128) cannot be represented as +128 in signed 8-bit two's complement, so the result stays 0x80 and V is set.

13.2 Shift and Rotate Instructions

Shifts move bits left or right. The bit shifted out goes into the carry flag. Rotates put the old carry flag back in on the vacated side.

If addition and subtraction are about numeric value, shifts and rotates are about bit position. They are the bridge between arithmetic and bit-level work.

LSL Rd – Logical Shift Left

$C \leftarrow [7][6][5][4][3][2][1][0] \leftarrow 0$
 $Rd \ast= 2$ (unsigned). Old bit 7 $\rightarrow C$.

LSR Rd – Logical Shift Right

$0 \rightarrow [7][6][5][4][3][2][1][0] \rightarrow C$
 $Rd /= 2$ (unsigned). Old bit 0 $\rightarrow C$. Bit 7 forced to 0.

ASR Rd – Arithmetic Shift Right

$b7 \rightarrow [7][6][5][4][3][2][1][0] \rightarrow C$
 $Rd /= 2$ (signed). Bit 7 preserved (sign extension).

ROL Rd – Rotate Left through Carry

$C \leftarrow [7][6][5][4][3][2][1][0] \leftarrow C(\text{old})$
 Like LSL but old C enters at bit 0.

ROR Rd – Rotate Right through Carry

$C(\text{old}) \rightarrow [7][6][5][4][3][2][1][0] \rightarrow C$
 Like LSR but old C enters at bit 7.

All shift/rotate instructions: **1 cycle**. The instruction summary lists C, Z, N, and V as directly affected; S follows from $N \text{ xor } V$. LSL and ROL also affect H because they are aliases for ADD Rd,Rd and ADC Rd,Rd. LSR, ASR, and ROR do not affect H. T and I are not affected. SWAP is separate: it affects no flags.

That makes them cheap enough to use constantly, especially for scaling by powers of two or for moving carries across byte boundaries.

13.2.1 16-bit Shift Left (Multiply by 2)

```
lsl  r24      ; low byte: bit 7 → C
rol  r25      ; high byte: C → bit 0 (carry from low byte propagates)
```

13.2.2 16-bit Shift Right, Logical (Unsigned Divide by 2)

```
lsr  r25      ; high byte: bit 0 → C (MSB filled with 0)
ror  r24      ; low byte: C → bit 7
```

13.2.3 16-bit Shift Right, Arithmetic (Signed Divide by 2)

```
asr r25      ; high byte: sign bit preserved, bit 0 → C
ror r24      ; low byte: C → bit 7
```

13.2.4 Shifting by N Bits

AVR does not have a multi-bit shift instruction (unlike x86's `SHL r, cl`). Repeat for each bit:

```
; r16 <=< 3 (multiply by 8)
lsl r16
lsl r16
lsl r16
```

For large shifts on 8-bit values, `SWAP` is faster when shifting by 4:

```
; r16 <=< 4
swap r16      ; swap nibbles: bits 7:4 ↔ bits 3:0
andi r16, 0xF0 ; zero lower nibble (was upper before swap)
```

```
; r16 >>= 4 (logical)
swap r16
andi r16, 0x0F ; zero upper nibble
```

This is a good reminder that on AVR, repeated simple instructions often replace one “fancier” instruction from a larger ISA.

13.2.5 Extracting a Bit Field

Example: extract bits 5:3 of r16 into bits 2:0 of r17:

```
mov r17, r16
andi r17, 0b00111000 ; mask bits 5:3
lsr r17               ; shift right 3 times
lsr r17
lsr r17               ; r17 = bits 5:3 in positions 2:0
```

13.3 SWAP — Swap Nibbles

`SWAP Rd` ; bits 7:4 ↔ bits 3:0

1 cycle. Does not affect any flags. Pure bit rearrangement.

```
ldi r16, 0xAB
swap r16 ; r16 = 0xBA
```

Primary uses: - Fast $\times 16$ or $\div 16$ on values that fit in a nibble - BCD digit manipulation - Building two-nibble values from separate sources:

```
; Pack two 4-bit values (r16 low nibble, r17 low nibble) into one byte
andi r16, 0x0F ; keep only low nibble of r16
swap r17       ; move r17's low nibble to high position
andi r17, 0xF0 ; keep only high nibble of r17
or r16, r17    ; r16 = r17_nibble : r16_nibble
```

`SWAP` is a good example of an instruction that looks niche until you start doing display code, BCD work, or byte packing. Then it becomes surprisingly useful.

13.4 MUL — Unsigned 8×8 Multiply

MUL Rd, Rr ; r1:r0 ← Rd × Rr (unsigned)

2 cycles. Result always in **r1:r0** regardless of operands.

ATtiny3217 has the hardware multiplier block, so this is a real two-cycle instruction, not a library loop. The multiplier takes two 8-bit inputs and produces a 16-bit result; the integer multiply instructions differ only in how they interpret the inputs.

The fixed result location is the most important thing to remember about MUL. The product does not stay near the source operands. It always lands in r1:r0.

```
ldi r16, 12
ldi r17, 25
mul r16, r17 ; r1:r0 = 300 = 0x012C
movw r24, r0 ; copy result to a usable register pair
clr r1 ; RESTORE r1 = 0 – this is mandatory (see below)
```

Why must you clear r1?

By the GCC AVR ABI, r1 must always be 0 when calling C functions or when C code calls assembly. MUL sets r1 to the high byte of the product. If you forget CLR r1, later code that assumes r1=0 (like ADC Rr, r1 for carry propagation) will produce wrong results.

Develop the habit: every MUL/MULS/MULSU is immediately followed by CLR r1 unless you are certain no C code is involved and you consume r1 before any carry-chain arithmetic.

Even if you are not mixing with C today, it is still a good discipline. The “r1 should be zero” convention is useful across the whole codebase.

13.4.1 Self-Multiply (Square)

```
; r25:r24 = r16 * r16 (r16 squared)
mul r16, r16
movw r24, r0
clr r1
```

MUL allows both operands to be the same register.

13.4.2 8-bit Fraction Scaling

A common embedded pattern: scale an 8-bit value by an 8-bit fraction using the high byte of the product as a “multiply then shift right 8” operation:

This is a classic fixed-point trick: keep the multiply, throw away the low byte, and treat the high byte as the scaled result.

```
; Approximate: result = (val * fraction) >> 8
; val in r16, fraction in r17 (0x00=0%, 0xFF≈100%, 0x80=50%)
mul r16, r17
mov r16, r1 ; take only the high byte → effectively >> 8
clr r1
```

13.5 MULS — Signed 8×8 Multiply

MULS Rd, Rr ; r1:r0 ← (signed)Rd × (signed)Rr

Both operands treated as signed (-128..127). Result is signed 16-bit. **r16-r31 only** for both operands.

```
ldi r16, 0xF6      ; -10 in two's complement
ldi r17, 5
muls r16, r17      ; r1:r0 = -50 = 0xFFCE
movw r24, r0
clr r1
```

```
ldi r16, 0xF6      ; -10
ldi r17, 0xFB      ; -5
muls r16, r17      ; r1:r0 = 50 = 0x0032
movw r24, r0
clr r1
```

13.6 MULSU — Signed × Unsigned Multiply

MULSU Rd, Rr ; r1:r0 ← (signed)Rd × (unsigned)Rr

Rd (r16-r23) is signed; Rr (r16-r23) is unsigned. Result is signed 16-bit.

Used in fixed-point arithmetic where one operand is a coefficient (signed scaling factor) and the other is a measurement (unsigned raw value):

That signed/unsigned mix shows up often in calibration and correction code.

```
; Scale a raw ADC reading (0-255, unsigned) by a signed calibration
; coefficient (-128..127), result in r25:r24.
; Example: reading=200 (0xC8), coeff=-3 (0xFD)
ldi r16, 0xFD      ; signed coefficient -3
ldi r17, 200       ; unsigned reading
mulsu r16, r17     ; r1:r0 = -600 = 0xFDA8
movw r24, r0
clr r1
```

13.7 Fractional Multiply: FMUL, FMULS, FMULSU

The ATtiny3217 multiplier also supports fractional multiply instructions:

```
FMUL   Rd, Rr      ; unsigned fractional
FMULS  Rd, Rr      ; signed fractional
FMULSU Rd, Rr      ; signed × unsigned fractional
```

These still compute an 8×8 product into r1:r0 in 2 cycles, but they shift the 16-bit product left by one bit before storing it. Microchip describes the typical input format as 1.7 fixed point, where the binary point is between bits 6 and 7. The left shift puts the high byte back into the same fractional scale that DSP-style code usually wants.

Operand limits:

Instruction	Rd range	Rr range
FMUL	r16-r23	r16-r23
FMULS	r16-r23	r16-r23
FMULSU	r16-r23	r16-r23

Like the integer multiply family, fractional multiply writes `r1:r0` and affects only `Z` and `C`. Clear `r1` after consuming the product if the surrounding code follows the GCC AVR ABI.

Example: signed Q1.7-style scale. `0x40` represents `+0.5`, so multiplying `0.5` by `0.5` gives `0.25`, represented by `0x20` in the high byte:

```
ldi    r16, 0x40    ; +0.5 in signed 1.7 fixed point
ldi    r17, 0x40    ; +0.5
fmuls  r16, r17     ; r1:r0 = (0x40 * 0x40) << 1 = 0x2000
mov    r18, r1      ; high byte = 0x20, which represents +0.25
clr    r1
```

Fractional multiply is powerful, but be precise about your fixed-point format. For signed fractional edge cases such as `0x80 * 0x80`, Microchip notes that the shifted result can overflow two's-complement interpretation and software must handle that if it matters.

13.8 Multiply Summary

Instruction	Rd range	Rr range	Signed?	Result
MUL Rd, Rr	r0–r31	r0–r31	both unsigned	r1:r0 unsigned
MULS Rd, Rr	r16–r31	r16–r31	both signed	r1:r0 signed
MULSU Rd, Rr	r16–r23	r16–r23	Rd signed, Rr unsigned	r1:r0 signed
FMUL*	r16–r23	r16–r23	fractional	(product << 1) in r1:r0

All: 2 cycles. Result always in `r1:r0`. Always CLR `r1` after.

All multiply instructions affect only `C` and `Z`.

13.9 Division — Software Algorithm

AVR has no divide instruction. Division is implemented with a shift-subtract loop: the same long division algorithm you learned in school, but in binary.

So division is not something the hardware does for you in one step. You are trading simplicity of concept for cost in cycles.

13.9.1 Unsigned 8-bit Division

dividend / divisor → quotient, remainder

The algorithm processes one bit at a time from MSB to LSB:

For each bit (MSB to LSB):

1. Shift MSB of dividend into remainder LSB (remainder <= 1, `r[0] = d_msb`)
2. If remainder >= divisor:
 - remainder -= divisor
 - set current quotient bit = 1
- Else:
 - quotient bit = 0

If that seems abstract, think of it this way: each iteration asks, “Can the current remainder afford one more divisor?” If yes, subtract and record a 1 in the quotient. If not, record a 0 and continue.

Assembly implementation (clean restoring version):

```

/*
 * div8u – unsigned 8-bit division
 * Input:  r16 = dividend, r17 = divisor
 * Output: r18 = quotient, r19 = remainder
 * Clobbers: r20
 * Cycles: roughly 9-10 cycles per bit, plus call/return if used as a subroutine
 */
div8u_clean:
    clr r18                ; quotient = 0
    clr r19                ; remainder = 0
    ldi r20, 8             ; 8 bits
1:
    lsl r16                ; MSB of dividend → C
    rol r19                ; remainder = (r19 << 1) | C
    lsl r18                ; quotient <=< 1, make room for next bit
    cp r19, r17
    brlo 2f               ; remainder < divisor: quotient bit stays 0
    sub r19, r17           ; remainder -= divisor
    ori r18, 1             ; quotient bit = 1
2:
    dec r20
    brne 1b
    ret

```

Trace: 7 / 3:

```

bit 7: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 6: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 5: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 4: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 3: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 2: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 1: remainder=0→1, 1<3 → quotient bit 0 → r18=0b00000000
bit 0: remainder=1→3, 3>=3 → sub→0, quotient bit 1 → r18=0b00000001

```

```

quotient = 0b000000010 = 2 OK
remainder = 1 OK (7 = 2×3 + 1)

```

Tracing a few divisions by hand is worth the time. After that, the loop stops looking magical and starts looking mechanical.

13.9.2 Unsigned 16-bit Division

Extend the pattern to 16-bit dividend, 8-bit divisor (common for scaled sensor values):

This is the same algorithm again, just stretched over more bits.

```

/*
 * div16u8 – divide 16-bit by 8-bit
 * Input:  r25:r24 = dividend, r16 = divisor
 * Output: r25:r24 = quotient, r18 = remainder
 * Clobbers: r19, r20, r21, r22, r23
 */
div16u8:
    clr r18                ; remainder low = 0
    clr r22                ; remainder high = 0
    clr r23                ; zero register for 16-bit subtract
    ldi r19, 16            ; 16 bits
    clr r20                ; quotient low
    clr r21                ; quotient high

```

```

1:      lsl  r24                ; shift MSB of dividend → C
      rol  r25
      rol  r18                ; remainder <= 1, | C
      rol  r22                ; keep the 9th remainder bit
      lsl  r20                ; quotient <= 1
      rol  r21
      tst  r22                ; if high byte is nonzero, remainder >= divisor
      brne 3f
      cp   r18, r16
      brlo 2f
3:      sub  r18, r16
      sbc  r22, r23
      ori  r20, 1              ; quotient bit = 1
2:      dec  r19
      brne 1b
      movw r24, r20            ; quotient → r25:r24
      ret

```

Both routines assume the divisor is nonzero. AVR has no hardware divide-by-zero trap; if zero is possible, test it before entering the loop and choose an explicit policy such as saturating the quotient, returning zero, or branching to an error handler.

13.9.3 Power-of-2 Division — Just Shift

If the divisor is a compile-time power of 2, skip the loop entirely:

```

; Unsigned: r16 /= 8 (= 2^3)
lsr r16
lsr r16
lsr r16

; Signed: r16 /= 4 (rounds toward -∞, which is floor division)
asr r16
asr r16

; 16-bit unsigned: r25:r24 /= 4
lsr r25
ror r24
lsr r25
ror r24

```

Always use shifts for power-of-2 division. The loop costs ~72 cycles; three LSRs cost 3.

That is not a small optimization. It is the difference between “expensive software division” and “almost free.”

13.10 Worked Example: 8×8 Unsigned Multiply to 16-bit Result

Full annotated example with verification by objdump.

```

/*
 * Input:  r24 = a (0–255), r22 = b (0–255)
 * Output: r25:r24 = a * b (0–65025)

```

```

* ABI-compliant: returns 16-bit result in r25:r24.
*/
mul8_to_16:
    mul    r24, r22      ; r1:r0 = a * b
    movw   r24, r0       ; r25:r24 = result
    clr    r1            ; restore ABI r1=0
    ret

```

Disassembly of this function (avr-objdump -d):

```

<mul8_to_16>:
  0:  86 9f          mul     r24, r22
  2:  c0 01          movw   r24, r0
  4:  11 24          eor    r1, r1      ; CLR alias
  6:  08 95          ret

```

MUL is a single 16-bit word (2 bytes). The two-cycle cost comes from the hardware multiplier computing the 16-bit product in parallel with the next fetch. MOVW is another single word. The entire routine is 4 instructions, 8 bytes, 8 cycles including RET.

This is a good example of why knowing the ISA matters. A tiny hand-written routine can be both obvious and efficient once you know the dedicated instruction exists.

13.11 Signed Arithmetic Pitfalls

13.11.1 Two Mistakes with Signed Division

Mistake 1: Using LSR (logical) instead of ASR (arithmetic) to divide a signed value.

```

ldi r16, 0xFE      ; = -2 signed
lsr r16            ; r16 = 0x7F = 127 ← WRONG (unsigned right shift)
asr r16            ; r16 = 0xFF = -1 ← CORRECT (-2 / 2 = -1)

```

Mistake 2: Using BRLO/BRSH (unsigned) instead of BRLT/BRGE (signed) for signed comparisons. Covered in Chapter 6 but worth repeating whenever you work with signed arithmetic.

The CPU does not know your intent. A byte is just a bit pattern until your instruction choice tells the machine whether to treat it as signed or unsigned.

13.11.2 Overflow Detection

After a signed operation, V=1 means overflow. The result is wrong as a signed value:

```

ldi r16, 100      ; +100
ldi r17, 100      ; +100
add r16, r17      ; result = 200, but 200 > 127 → V=1 (signed overflow)
brvs overflow_handler

```

After unsigned addition, C=1 means overflow (result > 255 / 65535). After unsigned subtraction, C=1 means borrow/underflow.

That signed/unsigned split is one of the most common sources of subtle bugs in embedded arithmetic.

13.12 Useful Idioms

13.12.1 Test if a value is zero without changing the register

```
tst r16 ; Z=1 if r16=0, N=1 if negative – r16 unchanged
```

TST sets flags. The point is that it lets you ask “is this register zero or negative right now?” without changing the register value itself.

13.12.2 Conditional absolute value

```
; |r16|
sbrcc r16, 7 ; skip if bit 7 clear (positive)
neg r16 ; negate if negative
```

13.12.3 Saturating add (clamp at 255 instead of wrapping)

```
add r16, r17 ; add
brcc 1f ; no carry: result fits in 8 bits
ser r16 ; carry: clamp to 0xFF
1:
```

This is a very common embedded pattern because sensor and UI code often prefers clamping over wraparound.

13.12.4 Saturating subtract (clamp at 0 instead of wrapping)

```
sub r16, r17
brcc 1f ; no borrow (C=0): result >= 0
clr r16 ; borrow: clamp to 0
1:
```

13.12.5 Sign extension: 8-bit signed → 16-bit signed

```
; r16 (signed) → r17:r16 (signed 16-bit)
clr r17
sbrcc r16, 7 ; if bit 7 clear: r17 = 0 (positive, done)
ser r17 ; bit 7 set: r17 = 0xFF (negative, sign extend)
; r17:r16 is now the correct signed 16-bit representation
```

13.13 Summary

Multi-byte arithmetic:

Always low byte first. Use ADC/SBC to chain carry between bytes.

ADIW/SBIW: $\pm 0-63$ to 16-bit pair, 2 cycles.

SUBI+SBCI: add constants to r16-r31 pairs (negate the immediate).

Shifts:

LSL/LSR – logical (unsigned), 0 inserted.

ASR – arithmetic (signed), sign bit preserved.

ROL/ROR – rotate through carry – chains shifts across bytes.

SWAP – nibble swap, 1 cycle, use for $\times 16$ / $\div 16$.

Multiply:

MUL – unsigned $\times$ unsigned $\rightarrow$ r1:r0. Always CLR r1 after.

MULS – signed $\times$ signed $\rightarrow$ r1:r0. r16–r31 only.

MULSU – signed $\times$ unsigned $\rightarrow$ r1:r0. r16–r23 only.

FMUL* – fractional multiply, r16–r23 only, product shifted left once.

All: 2 cycles. Result in r1:r0.

Division: no hardware DIV. Use shift-subtract loop (~72 cycles for 8-bit).

Power-of-2 divisors: use LSR/ASR instead – $O(\log_2 n)$ cycles.

Signed pitfalls:

Use ASR not LSR for signed right shift.

Use BRLT/BRGE not BRLO/BRSH for signed branch.

V flag = signed overflow, C flag = unsigned carry/borrow.

13.14 Exercises

1. Write a 16-bit multiply: $r25:r24 = r25:r24 \times r23:r22$ (both inputs unsigned). You cannot use MUL for the full 16×16 product directly (it only handles 8×8). Decompose into four 8×8 multiplies and accumulate. Hint: $(a_hi \times 256 + a_lo) \times (b_hi \times 256 + b_lo)$.
2. Extend div16u8 to handle divide-by-zero explicitly. Choose a policy (for example: quotient = 0xFFFF, remainder = dividend low byte, or branch to an error handler) and document it.
3. Write a function that counts the number of 1-bits in r16 (popcount). Use LSL + ADC r17, r1 pattern: after each LSL, C holds the shifted-out bit; ADC r17, r1 adds C to the accumulator (r1=0).
4. Implement saturating 16-bit addition: $r25:r24 + r23:r22$, clamped to 0xFFFF if overflow. The 16-bit carry flag is in C after the second ADC.
5. Verify the div8u_clean routine:
 - Trace $100 / 7$ by hand (expected: quotient=14, remainder=2).
 - Assemble and inspect: use `avr-objdump -d` to confirm the instruction sequence, then trace the bit loop manually.
6. Without MUL, compute $r16 \times 10$ using only shifts and adds. ($10 = 8 + 2 = (1 \ll 3) + (1 \ll 1)$.)

Next: Chapter 14 — Branches and Control Flow

Chapter 14

Branches and Control Flow

Every non-trivial program has decisions and loops. In assembly, all of them reduce to one fundamental act: change the program counter, or deliberately let it continue to the next instruction. This chapter covers the main ways AVR does that — unconditional jumps, conditional branches, and single-instruction skips — and shows the assembly patterns behind decisions, loops, and multi-way dispatch.

The key mental shift is that high-level control flow is just structured PC manipulation.

14.1 Unconditional Jumps

14.1.1 RJMP — Relative Jump

`RJMP k` ; PC $\leftarrow$ PC + k + 1 (k = signed offset, -2048..+2047 words)

2 cycles. 1 word. Reachable destinations are PC - 2047 through PC + 2048 instruction words (the offset k is encoded as 12-bit signed, -2048..+2047, and the CPU computes PC + k + 1). That covers about ± 4 KB of code on either side of the RJMP.

```
rjmp loop      ; jump to label "loop" - GAS computes the offset
rjmp .         ; infinite loop (jump to current address, offset = -1)
```

RJMP covers most local control-flow transfers — nearly 4K instruction words (just under 8 KB of code bytes). Use it by default.

That “use it by default” rule is practical: RJMP is smaller and faster than JMP, and most control-flow targets are close enough to fit.

14.1.2 JMP — Absolute Jump

`JMP k` ; PC $\leftarrow$ k (k = 22-bit program word address)

3 cycles. 2 words (32-bit instruction). JMP carries a 22-bit absolute word address in its encoding, which is more than enough to cover the whole 16 K words (32 KB) of ATtiny3217 flash, or any other AVR device released so far.

```
jmp main        ; jump anywhere - no range limit
jmp reset_handler
```

Use JMP only when the target is out of RJMP range, or in the vector table where you need a guaranteed 2-word slot. For everything else, RJMP is smaller and faster.

So the usual decision is simple:

- nearby target: RJMP

- far target or fixed-size vector entry: JMP

14.1.3 IJMP — Indirect Jump via Z

IJMP ; PC $\leftarrow$ Z (Z holds word address of target)

2 cycles. 1 word. Z contains the **word address** (byte address $\div$ 2) of the destination. Use pm(label) to convert a label to its word address:

```
ldi ZL, lo8(pm(target))
ldi ZH, hi8(pm(target))
ijmp ; jump to target
```

Primary use: **jump tables**. Covered in Section 14.7.

Indirect jumps are less common in beginner code, but they are the step that takes you from simple branch chains to table-driven dispatch.

14.2 Conditional Branches — Mechanics

All conditional branches work identically:

1. Test one or more bits in SREG.
2. If the condition is true: PC += offset + 1 (branch taken, 2 cycles).
3. If the condition is false: PC += 1 (not taken, 1 cycle).

Range: reachable destinations are PC - 63 through PC + 64 instruction words. The offset is encoded as 7-bit signed (-64..+63), and the CPU computes PC + k + 1. If the target is farther, invert the condition and use JMP/RJMP:

```
; Branch to far_target if r16 == 0 - but far_target > 63 words away:
tst r16
brne skip ; if NOT zero, skip the jump (inverted condition)
jmp far_target ; now unconditionally jump to far target
skip:
```

This pattern — invert condition, skip over jump — is the standard way to handle far conditional transfers.

14.2.1 Exact Branch Range

The branch operand is a 7-bit signed word offset: -64..+63. Because the CPU adds the offset to PC + 1, the reachable destination range is:

PC - 63 through PC + 64 ; destination instruction word addresses

That is why datasheets often describe conditional branches as reaching PC - 63 to PC + 64, while programmers usually summarize the range as “about ± 64 words.” Both descriptions refer to the same encoding.

All AVR branch and jump offsets are in **instruction words**, not bytes. This matters when you compare a disassembly address, a linker error, and the instruction manual:

- RJMP offset: signed word offset, -2048..+2047; reachable destinations PC - 2047 .. PC + 2048
- conditional branch offset: signed word offset, -64..+63; reachable destinations PC - 63 .. PC + 64
- flash byte addresses: twice the word address on classic AVR notation

GAS hides most of this with labels, but the distinction matters for jump tables, manual address arithmetic, and datasheet tables that list program addresses in words.

14.2.2 Signed vs Unsigned Conditions

The CPU does not know whether a byte is signed or unsigned. You choose the branch mnemonic that interprets the flags the way your data should be read.

After CP Rd, Rr or CPI Rd, K, the comparison is internally $Rd - Rr$ or $Rd - K$.

Relation	Unsigned branch	Signed branch
<	BRLO / BRCS	BRLT
>=	BRSH / BRCC	BRGE
==	BREQ	BREQ
!=	BRNE	BRNE

Example: 0x80 is 128 unsigned but -128 signed.

```
ldi r16, 0x80
cpi r16, 1
brlo unsigned_less    ; false: 128 is not < 1
brlt signed_less      ; true: -128 is < 1
```

This is the most common comparison bug in hand-written assembly: using BRLO/BRSH for signed values or BRLT/BRGE for unsigned values.

14.3 Complete Branch Instruction Table

Mnemonic	Alias for	Condition	Flags tested	Typical use after
BREQ	BRBS 1,k	Z = 1	Z	CP, CPI, TST, SUB
BRNE	BRBC 1,k	Z = 0	Z	CP, CPI, DEC, INC
BRCS	BRBS 0,k	C = 1	C	ADD, SUB, LSL, LSR
BRCC	BRBC 0,k	C = 0	C	ADD, SUB
BRLO	BRCS	C = 1	C	CP, CPI (unsigned <)
BRSH	BRCC	C = 0	C	CP, CPI (unsigned ≥)
BRLT	BRBS 4,k	S = 1 (N≠V)	N, V	CP, CPI (signed <)
BRGE	BRBC 4,k	S = 0 (N=V)	N, V	CP, CPI (signed ≥)
BRMI	BRBS 2,k	N = 1	N	ADD, SUB (signed neg)
BRPL	BRBC 2,k	N = 0	N	ADD, SUB (signed pos)
BRVS	BRBS 3,k	V = 1	V	ADD, SUB (ovf detect)
BRVC	BRBC 3,k	V = 0	V	ADD, SUB
BRIE	BRBS 7,k	I = 1	I	SEI/CLI context
BRID	BRBC 7,k	I = 0	I	SEI/CLI context
BRHS	BRBS 5,k	H = 1	H	BCD arithmetic
BRHC	BRBC 5,k	H = 0	H	BCD arithmetic
BRTS	BRBS 6,k	T = 1	T	BST/BLD sequences
BRTC	BRBC 6,k	T = 0	T	BST/BLD sequences

BRBS b, k (Branch if Bit in SREG Set) and BRBC b, k (Branch if Bit in SREG Clear) are the generic forms. Every named branch is an alias for one of these two — GAS assembles BREQ as BRBS 1, k (bit 1 = Z flag).

In practice you almost always write the named forms, because BREQ expresses intent better than “branch if SREG bit 1 is set.”

14.4 Skip Instructions

Skips are single-instruction conditional skips — they bypass exactly the next instruction if a condition is met. No label needed. They are the most compact way to write small conditionals.

If branches are for “go somewhere else,” skips are for “ignore the next thing if this condition holds.”

14.4.1 CPSE — Compare, Skip if Equal

CPSE Rd, Rr ; if Rd == Rr: skip next instruction

1 cycle (no skip), 2 cycles (skip 1-word), 3 cycles (skip 2-word).

That timing detail matters because the skipped instruction still occupies space in the stream even though it does not execute.

```
cpse r16, r17
rjmp not_equal ; skipped if r16 == r17 – falls through to equal path
; equal path here
```

CPSE does **not** set SREG. It compares only for the purpose of deciding whether to skip the next instruction. If later code needs Z, C, N, V, or S from the comparison, use CP/CPI instead.

14.4.2 SBRC — Skip if Bit in Register is Clear

SBRC Rr, b ; if bit b of Rr == 0: skip next instruction

Tests bit b (0-7) of register Rr (r0-r31). No flags affected.

```
; Process bit 0 of flags register r16
sbrc r16, 0 ; skip if bit 0 clear
rcall handle_flag0 ; only called if bit 0 was set
```

14.4.3 SBRS — Skip if Bit in Register is Set

SBRS Rr, b ; if bit b of Rr == 1: skip next instruction

```
sbrs r16, 7 ; skip if bit 7 set (value is negative signed)
rjmp positive_path ; only taken if bit 7 was clear
; negative path falls through here
```

14.4.4 SBIC — Skip if Bit in I/O Register is Clear

SBIC A, b ; if bit b of I/O[A] == 0: skip (A = 0x00-0x1F only)

Tests a bit in an I/O register **in a single cycle without loading into a register first**. The address range is limited to 0x00-0x1F (the lower half of the IN/OUT space).

```
; Wait until a flag bit in GPIOR0 is set by an ISR
wait_for_flag:
    sbis GPIOR0, 0 ; skip if bit 0 of GPIOR0 is set
    rjmp wait_for_flag ; not set: loop
; bit 0 now set – proceed
```

14.4.5 SBIS — Skip if Bit in I/O Register is Set

SBIS A, b ; if bit b of I/O[A] == 1: skip (A = 0x00–0x1F only)

```
; Check if a GPIOR flag is clear
sbic GPIOR0, 2 ; skip if bit 2 clear
rcall clear_handler ; only if bit 2 was set
```

14.4.6 SBIC/SBIS Address Limit on ATtiny3217

On ATtiny3217, SBIC/SBIS reach I/O addresses **0x00–0x1F** only. The regular PORT peripheral registers (PORTA_IN, PORTB_IN, etc.) are at 0x0400+ — far beyond this limit. To test a regular PORT pin register, you must load it first:

```
; Read PA3 state into r16, test bit 3
lds r16, PORTA_IN ; load port input register
sbrc r16, 3 ; skip if PA3 is low
rcall pa3_high_handler
```

The registers that ARE reachable with SBIC/SBIS on ATtiny3217:

I/O Addr	Register	Useful bits
0x02	VPORTA_IN	Fast pin reads for Port A
0x06	VPORTB_IN	Fast pin reads for Port B
0x0A	VPORTC_IN	Fast pin reads for Port C
0x1C	GPIOR0	User-defined flags (bit 0–7)
0x1D	GPIOR1	User-defined flags
0x1E	GPIOR2	User-defined flags
0x1F	GPIOR3	User-defined flags

GPIOR0–3 are key scratch flag registers in low I/O space: readable in 1 cycle with SBIC/SBIS, no load required. VPORTx_IN is the other major use: it gives you fast bit tests on real pins without touching the slower PORTx_IN registers in extended I/O space.

```
; Wait while PA3 is low using the virtual port input register
wait_pa3_high:
    sbis VPORTA_IN, 3 ; VPORTA_IN is I/O address 0x02
    rjmp wait_pa3_high
```

The regular PORTx registers are still the full peripheral interface. The virtual ports map only the common DIR, OUT, IN, and INTFLAGS registers into low I/O space so bit instructions can reach them.

14.4.7 Skip Over 2-Word Instructions Carefully

Skip instructions can skip either a 1-word instruction or a 2-word instruction. On AVRxt, the timing is:

Case	Cycles
Condition false, no skip	1
Skip a 1-word instruction	2
Skip a 2-word instruction	3

This is legal:

```
sbrc r16, 0
jmp far_handler ; 2-word instruction, skipped as a whole if bit 0 clear
```

But it is easy to misread during review because the skipped instruction takes two words in flash. For small conditionals, prefer skipping a 1-word RJMP or RCALL when the target range allows it.

14.4.8 Skip vs Branch — When to Use Which

Situation	Use
Skip 1 instruction on a bit condition	SBRC/SBRS/SBIC/SBIS
Skip 1 instruction on equality	CPSE
Branch to a label	BREQ, BRNE, etc.
More than 1 instruction in the conditional body	Branch

Skips are best for guard clauses and flag-polling loops. Branches are best for bodies with multiple instructions.

That is the simplest rule for choosing between them: one-instruction body, consider a skip; larger body, use a branch.

14.5 Structured Control Flow Patterns

14.5.1 Two-way decision

```
; r16 = a, r17 = b, result → r18
cp   r16, r17           ; a - b
brsh use_a              ; unsigned a >= b
mov  r18, r17
rjmp done
use_a:
mov  r18, r16
done:
```

Pattern: compare/test -> branch to one path -> execute the other path -> rjmp over the first path -> join at a common label.

That “invert the condition” step is one of the most common assembly habits. You often branch around the code you do not want rather than branch directly into every desired path.

14.5.2 Multi-byte comparisons

For 16-bit values, compare the low byte first with CP, then the high byte with CPC so the carry/borrow flows into the high-byte comparison.

```
; Compare unsigned a = r17:r16 with b = r19:r18
cp   r16, r18           ; low byte
cpc  r17, r19           ; high byte with carry from low-byte compare
brlo a_below_b          ; unsigned a < b
brsh a_same_or_above_b  ; unsigned a >= b
```

For equality, the same CP/CPC sequence leaves Z set only if both bytes are equal:

```
cp   r16, r18
cpc  r17, r19
breq equal16
```

For signed 16-bit compares, use the same CP/CPC sequence but finish with BRLT or BRGE. The signed condition comes from the final high-byte comparison, which is the byte containing the sign bit.

14.5.3 Guard branch

```
tst r16
brne continue      ; NOT zero: skip the return
ret                ; zero: return early
continue:
```

For this pattern, a branch is clearer than forcing CPSE into it:

```
tst r16
breq early_return
rjmp continue
early_return:
ret
continue:
```

This is a good example of a broader rule: the shortest control-flow sequence is not always the clearest one.

14.5.4 Top-tested loop

```
top_test:
    tst r16
    breq top_done      ; condition false: exit
    dec r16            ; body
    rjmp top_test
top_done:
```

Pre-test — loop may execute zero times. Costs one extra RJMP per iteration.

That is why a top-tested loop is often not the most efficient assembly shape.

14.5.5 Bottom-tested loop (preferred in assembly)

```
do_body:
    dec r16            ; body
    brne do_body       ; condition: not zero → repeat
```

This is the most efficient loop form. No RJMP back — the branch IS the loop-back. One instruction of overhead per iteration instead of two.

This is one of the most useful structural transformations in assembly: if the loop body must run at least once, put the test at the bottom.

Use this shape when the body runs at least once:

```
; Count-down loop: r18 iterations, guaranteed >= 1
ldi r18, N
loop:
    ; body
    dec r18
    brne loop          ; one branch instruction, no rjmp
```

14.5.6 Counted loop

Count **down** rather than up — comparison against 0 is free (BRNE after DEC):

```

; Count-down (preferred):
ldi r18, N
1:
    ; body
    dec r18
    brne 1b

; Count-up (necessary when index used inside body):
clr r18
1:
    ; body (r18 = current index)
    inc r18
    cpi r18, N          ; costs one extra CPI per iteration
    brne 1b

```

That is why countdown loops appear everywhere in AVR code. Zero is the cheapest comparison target on the machine.

14.5.7 Nested loops

```

ldi r19, 8          ; outer counter
outer:
    ldi r18, 64      ; inner counter – reload every outer iteration
    inner:
        ; body
        dec r18
        brne inner
    dec r19
    brne outer

```

Critical: reload the inner counter at the top of the outer loop body, **before** the inner loop starts. A common mistake is loading it once before the outer loop — it reaches zero after the first outer iteration and never loops again.

This bug is common because the loop still looks nested at a glance. The reload placement is what actually makes it nested.

14.6 Worked Example: Sum an Array

Sum 8 unsigned bytes from flash, store 16-bit result in SRAM.

```

/* sum_array – sums ARRAY_LEN bytes from flash
 * Uses LPM: Z is the flash byte address of test_array.
 * Result in r25:r24
 */

.section .progmem.data
test_array:
    .byte 10, 20, 30, 40, 50, 60, 70, 80    ; sum = 360 = 0x0168
.equ ARRAY_LEN, 8

.section .text

sum_array:
    ldi ZL, lo8(test_array)
    ldi ZH, hi8(test_array)

```

```

    clr r24                ; sum_lo = 0
    clr r25                ; sum_hi = 0
    ldi r18, ARRAY_LEN    ; counter
loop:
    lpm r16, Z+            ; byte = flash[Z], Z++
    add r24, r16          ; sum_lo += byte
    adc r25, r1            ; sum_hi += carry (r1 = 0)
    dec r18
    brne loop             ; bottom-tested loop: no rjmp needed

; r25:r24 = 360 = 0x0168
ret

```

LPM reads through the program-memory bus. On AVRxt devices such as the ATtiny3217, flash is also mapped into the data address space starting at 0x8000, but LPM is the portable and explicit way to say “read from flash” in a small example like this.

Step-by-step trace:

Iter	r18	Z points to	r16	r25:r24
1	7	[20]	10	0x000A
2	6	[30]	20	0x001E
3	5	[40]	30	0x003C
4	4	[50]	40	0x0064
5	3	[60]	50	0x0096
6	2	[70]	60	0x00D2
7	1	[80]	70	0x0118
8	0	—	80	0x0168 OK

Why ADC r25, r1 and not ADC r25, r17? ADD r24, r16 sets C if the low-byte sum overflows, and ADC r25, r1 immediately consumes that carry. Using r1 (=0) adds only the carry bit, correctly propagating any overflow from the low byte without needing a dedicated zero register. The later DEC r18 updates N, Z, and V for the loop test, but it does not affect C.

This is a good example of how flag discipline and dataflow interact. The right instruction is not just about values in registers; it is about which flag bit you are intentionally consuming next.

14.7 Jump Tables

For multi-way branches, a jump table is faster than a chain of comparisons. In the compact form below, each table entry is an executable RJMP instruction. IJMP jumps into the selected table entry, and that RJMP then jumps to the handler.

Conceptually, a jump table turns a small command number into an indexed branch.

```

; r16 = command index (0, 1, or 2)

; Bounds check first:
cpi r16, 3                ; 3 = number of table entries
brsh jtable_default       ; >= 3: out of range

; Build Z = pm(jtable) + r16
; Each RJMP in the table is 1 word, so index directly.
ldi ZL, lo8(pm(jtable))
ldi ZH, hi8(pm(jtable))

```

```

add  ZL, r16          ; offset by selector index
adc  ZH, r1           ; carry
ijmp                ; jump to selected RJMP entry in the table

jtable:
    rjmp cmd0         ; selector 0
    rjmp cmd1         ; selector 1
    rjmp cmd2         ; selector 2

jtable_default:
    rcall cmd_default
    rjmp jtable_done

cmd0: rcall handler0 ; rjmp table_done implicit via fall-through pattern
      rjmp jtable_done
cmd1: rcall handler1
      rjmp jtable_done
cmd2: rcall handler2
      rjmp jtable_done
jtable_done:

```

How it works:

`pm(jtable)` gives the **word address** of `jtable`. Adding the selector index (in words) moves Z to the corresponding RJMP instruction. IJMP loads the program counter with that word address, so execution continues at the selected table entry. The table entry then executes and jumps to the actual handler. Each entry is 1 word (RJMP = 1 word), so the index scales correctly.

That two-step jump looks strange the first time you see it, but it buys you compact entries and simple indexing.

Cycle cost: bounds check (2) + LDI×2 (2) + ADD+ADC (2) + IJMP (2) + RJMP from table (2) = **10 cycles**. A chain of CPI/BREQ for 3 entries costs up to $3 \times (1+2) = 9$ cycles on the slowest path — comparable. Jump tables win decisively with more entries.

So jump tables are not automatically better for tiny dispatches. They become attractive once the table is large enough that linear comparison chains stop being cheap.

14.7.1 Address Tables with LPM

Another jump-table style stores raw handler addresses as data and loads the selected entry with LPM. That is useful when you want the table to be data rather than executable branch instructions, but it costs more instructions because each address is two bytes and must be loaded into Z before IJMP.

```

; r16 = selector index, handlers are within 64K words
lsl  r16              ; byte offset = index * 2
ldi  ZL, lo8(addr_table) ; LPM uses a flash byte address
ldi  ZH, hi8(addr_table)
add  ZL, r16
adc  ZH, r1

lpm  r30, Z+          ; low byte of handler word address
lpm  r31, Z           ; high byte of handler word address
ijmp

addr_table:
    .word pm(cmd0)

```



```
.word pm(cmd1)
.word pm(cmd2)
```

Use the executable-RJMP table when handlers are within RJMP range and code size matters. Use an address table when you need data-like table handling or when the table is generated by other tools.

14.7.2 Large Jump Tables with JMP Entries

If handlers are far from the table (RJMP out of range), use JMP (2 words) instead. Then each table entry is 2 words, so multiply the index by 2:

```
; Index × 2 for 2-word entries
lsl r16 ; r16 *= 2
ldi ZL, lo8(pm(jtable))
ldi ZH, hi8(pm(jtable))
add ZL, r16
adc ZH, r1
ijmp

jtable:
    jmp cmd0 ; 2 words each
    jmp cmd1
    jmp cmd2
```

On devices with more than 64K words of program memory, indirect jumps may also need EIJMP and the EIND register. ATtiny3217 has 32 KB of flash, so plain IJMP is enough for all program-memory destinations on this device.

14.8 Polling Loops

A polling loop spins until a condition becomes true — set by hardware or an ISR.

Polling is simple and sometimes appropriate, but it is also the most CPU-hungry waiting strategy because the core keeps executing the test loop continuously.

```
; Wait for USART0 receive complete (bit 7 of USART0_STATUS = RXCIF)
; Cannot use SBIS: address too high. Must use LDS + SBRS.
wait_rxc:
    lds r16, USART0_STATUS ; load USART status
    sbrc r16, 7 ; skip if RXCIF bit set
    rjmp wait_rxc ; not set: loop
    lds r16, USART0_RXDATA1 ; read the received byte
```

For low I/O registers (GPOR), SBIS eliminates the load:

```
; ISR sets bit 0 of GPOR0 when data ready
wait_ready:
    sbis GPOR0, 0 ; 1 cycle: skip if bit 0 set
    rjmp wait_ready ; 2 cycles: loop back
; 3 cycles per iteration vs 5 cycles with LDS+SBRS
```

That difference is exactly why low-I/O flag registers are so useful on AVR.

For real pins on ATtiny3217, prefer VPORTx_IN when the loop only needs a single pin state:

```
; Wait for PB2 high through the virtual port
wait_pb2:
```

```
sbis VPORTB_IN, 2      ; VPORTB_IN is I/O address 0x06
rjmp wait_pb2
```

Use regular `PORTx.IN` with `LDS` when you need the full peripheral register view, or when the code is shared with a device that lacks the same virtual-port layout.

14.9 Branch Optimisation Notes

14.9.1 Bottom-tested saves one instruction per loop

```
; top-tested: 3 instructions per iteration (body + dec + brne + rjmp)
; bottom-tested: 2 instructions per iteration (body + dec + brne)
```

For a loop that runs 1000 times, that's 1000 saved cycles.

Over one or two iterations this does not matter. Over hot loops, it matters a lot.

14.9.2 Count down, not up

`DEC r18 + BRNE` is **two instructions with no extra comparison**. Counting up needs `INC + CPI N + BRNE` — three instructions.

This is one of those patterns that looks like a small optimization until you write enough firmware to see it everywhere.

14.9.3 Use `SBRC/SBRS` for single-bit tests

Instead of:

```
andi r16, (1 << 3)
brne bit3_set
```

Write:

```
sbrs r16, 3
rjmp bit3_clear
; bit3_set path falls through
```

Saves one instruction and doesn't destroy `r16`.

Preserving the register value is often as important as saving the instruction.

14.9.4 Branch delay slots do not exist on AVR

Unlike ARM or MIPS, the AVR has no branch delay slot. The instruction after a branch is NOT executed if the branch is taken. No surprises.

That makes AVR control flow pleasantly literal: if you branch away, the next sequential instruction does not secretly run first.

14.10 Summary

Unconditional:

- RJMP — destinations `PC-2047..PC+2048`, 2 cycles, 1 word. Default choice.
- JMP — 22-bit absolute word address, 3 cycles, 2 words. For far targets or vector-table entries.

IJMP – Z = word address, 2 cycles. For jump tables.

Conditional branches: destinations PC-63..PC+64, 2 cycles taken / 1 not taken.

Unsigned: BRL0 (<), BRSH ($\geq$), BREQ (=), BRNE ($\neq$)

Signed: BRLT (<), BRGE ($\geq$), BRMI (neg), BRPL (pos)

Far target: invert condition, skip over JMP.

Skip instructions (1–3 cycles, no label needed):

CPSE	Rd, Rr	– skip if Rd == Rr
SBRC	Rr, b	– skip if bit b of Rr clear
SBRS	Rr, b	– skip if bit b of Rr set
SBIC	A, b	– skip if bit b of I/O[A] clear (A $\leq$ 0x1F)
SBIS	A, b	– skip if bit b of I/O[A] set (A $\leq$ 0x1F)

Loop patterns:

bottom-tested: most efficient – branch is the loop-back (no extra RJMP)

counted: count down with DEC+BRNE – no CPI needed

nested: reload inner counter at start of each outer iteration

Jump tables: IJMP with Z = pm(table) + index. 0(1) dispatch for table entries.

Low-I/O targets such as `VPORTx\_IN` and `GPIOR0–3` are reachable with `SBIC`/`SBIS`. `GPIORx` make especially good fast flag bytes between ISR and main loop.

14.11 Exercises

1. Rewrite the array-sum example to sum **signed** 8-bit values into a signed 16-bit result. What changes? (Hint: ADC r25, r1 is still correct for unsigned carry, but how do you handle sign extension of each negative byte before adding?)
2. Write a function find_min(X, n) that finds the minimum byte value in an array of n bytes at SRAM address X. Return the minimum in r24. Use a bottom-tested loop.
3. Extend the jump table example to 8 entries. What is the cycle cost of the dispatch compared to a chain of 8 CPI/BREQ pairs (slowest path)?
4. Write a zero-terminated byte-string length routine: given X pointing to the first byte in SRAM, return the length (not counting the zero terminator) in r24. Use a bottom-tested loop with LD X+ and TST.
5. Implement a byte-search routine: scan n bytes at X for value r22, return pointer to first match in X (or 0 if not found). Use CPSE to detect a match.
6. The polling loop sbis GPIOR0, 0 / rjmp wait burns CPU cycles continuously. Name one hardware peripheral on ATtiny3217 that would let you sleep the CPU instead of polling, waking only when the condition is true. (Covered in Chapter 17 — but think about it now.)

Next: Chapter 15 — Subroutines and the Stack

Chapter 15

Subroutines and the Stack

Subroutines let you package a sequence of instructions once and reuse it from many places. The stack is what makes that possible: it records the return address, holds temporary saved registers, and gives each active call its own scratch space when registers are not enough.

This chapter stays at the assembly level. The goal is to understand exactly what RCALL, CALL, RET, PUSH, POP, and stack frames do on ATtiny3217.

15.1 CALL and RCALL

RCALL k ; push return address, PC <- PC + k + 1 (-2048..+2047 words)
CALL k ; push return address, PC <- k (absolute target)

Both instructions push the return address, then branch to the target. The return address is the program-memory word address of the instruction after the call.

On ATtiny3217, the Program Counter is 14 bits wide for 32 KB of flash, so the stored return address is **2 bytes**. Larger AVR devices with wider program counters can use larger return addresses, but this chapter assumes the ATtiny3217.

Instruction	Words	Cycles	Range
RCALL k	1	2	relative, -2048..+2047 words
CALL k	2	3	absolute program address

Use RCALL for nearby subroutines. Use CALL only when the target is outside the relative range or when a fixed 2-word instruction is required.

```
rcall delay_tick ; compact local call
call reset_handler ; absolute call
```

The return address is pushed as a **word pointer**, not as a flash byte address. For example, if the next instruction is at byte address 0x0006, the stored return word address is 0x0003.

15.1.1 Return Address Layout

The ATtiny3217 data sheet specifies that the least significant byte of the return address is pushed first and ends up at the higher SRAM address. The stack grows downward, so after a call the stack looks like this:

Before RCALL, SP = 0x3FFF

```
0x3FFF  [free]
0x3FFE  [free]
```

After RCALL target, return word address = 0x1234

```
0x3FFF  0x34      ; return low byte, higher address
0x3FFE  0x12      ; return high byte
0x3FFD  [free]    ; SP now points here
```

You normally let RET handle this, but the byte order matters when debugging a corrupted stack in memory.

15.1.2 ICALL

ICALL ; push return address, PC <- Z

ICALL calls the word address held in Z (r31:r30). It is the subroutine version of IJMP.

```
ldi ZL, lo8(pm(handler))
ldi ZH, hi8(pm(handler))
icall
```

Use it for dispatch through a table or for a selected handler address. On ATtiny3217, plain ICALL is enough for all program-memory destinations. Devices with more than 64K words of program memory may need EICALL and the EIND register.

15.2 RET and RETI

```
RET      ; pop return address into PC
RETI     ; pop return address into PC; complete interrupt return
```

RET returns from a normal subroutine. RETI returns from an interrupt handler. On every AVR core, including AVRxt, the AVR Instruction Set Manual specifies that RETI sets the Global Interrupt Enable bit (SREG.I = 1). On AVRxt devices such as ATtiny3217, RETI additionally signals the CPUINT controller that the current interrupt level is finished — it clears the matching LVL0EX, LVL1EX, or NMIEX bit in CPUINT.STATUS so the controller can dispatch the next interrupt. That second effect is why RET must never be used to exit an ISR even when the I flag happens to be set.

Instruction	Cycles on AVRxt with 16-bit PC	Use
RET	4	normal subroutine return
RETI	4	interrupt return

Do not use RETI for ordinary subroutines. Do not use RET at the end of an interrupt handler; the interrupt controller needs the RETI instruction to finish the interrupt return correctly.

Every call path must return with the stack pointer at the same position it had on subroutine entry. If a subroutine pushes three bytes and pops only two, RET will fetch one byte of ordinary data as part of the return address.

15.3 PUSH and POP

```
PUSH Rr      ; SRAM[SP] <- Rr, then SP decreases by 1
POP  Rd      ; SP increases by 1, then Rd <- SRAM[SP]
```

On AVRxt, PUSH is 1 cycle and POP is 2 cycles. Both are 1-word instructions. The stack grows from higher SRAM addresses toward lower SRAM addresses.

```
push r16
push r17
; use r16 and r17
pop  r17      ; last pushed, first popped
pop  r16
```

Example starting with SP = 0x3FFF:

```
push r16:
  SP = 0x3FFE
  SRAM[0x3FFF] = r16
```

```
push r17:
  SP = 0x3FFD
  SRAM[0x3FFE] = r17
  SRAM[0x3FFF] = r16
```

```
pop r17:
  r17 = SRAM[0x3FFE]
  SP = 0x3FFE
```

The practical rule is simple: pop in the exact reverse order of pushes.

15.4 Stack Pointer Facts on ATtiny3217

The stack pointer is the 16-bit CPU.SP register, exposed as SPL and SPH in I/O space. The ATtiny3217 SRAM range is:

```
SRAM start: 0x3800
SRAM end:   0x3FFF
Size:       2 KB
```

After reset, the stack pointer is automatically set to the highest internal SRAM address. In standalone examples this book still initializes it explicitly:

```
ldi r16, lo8(RAMEND)
ldi r17, hi8(RAMEND)
out SPH, r17
out SPL, r16
```

If you move the stack yourself, set it before any subroutine call executes and before interrupts are enabled. A valid custom stack pointer must point inside SRAM and leave enough room above the lowest possible stack address.

15.4.1 Updating SP Safely

When writing a 16-bit stack pointer value manually, write SPH before SPL:

```
out SPH, r29
out SPL, r28
```

Microchip documents a hardware protection detail: writing SPL automatically disables interrupts for up to four instructions, or until the next I/O-memory write, whichever comes first. Writing high first and low second keeps the observable update coherent during that protected window.

15.5 Register Ownership

A reusable subroutine needs a register contract. This book uses the following assembly convention unless a routine documents otherwise:

Register	Meaning in this book's examples
r0	Scratch, often used by MUL. Not preserved.
r1	Zero register. Keep as 0 except during MUL sequences.
r2-r17	Preserved by the subroutine if used.
r18-r27	Scratch for the subroutine.
r28:r29 (Y)	Preserved if used; preferred frame pointer.
r30:r31 (Z)	Scratch pointer; used by LPM, ICALL, IJMP, tables.
SREG	Not preserved by ordinary routines unless documented.
SP	Must be balanced on every return path.

That gives two useful habits:

- short-lived temporary values go in scratch registers
- values that must survive a nested call should be pushed or kept in preserved registers saved by the callee

Example: a subroutine that uses r14 and r15 must save them:

```
worker:
    push r14
    push r15

    ; body may use r14/r15

    pop  r15
    pop  r14
    ret
```

Example: a caller that needs r20 after a nested call must save it itself:

```
push r20
rcall helper    ; helper may use scratch registers
pop  r20
```

15.5.1 The r1 = 0 Rule

AVR multiply instructions store their result in r1:r0. That means r1 may become non-zero after MUL, MULS, or MULSU.

Many AVR idioms use r1 as a zero register:

```
add r24, r16
adc r25, r1    ; add carry only
```

If r1 is not zero, this sequence is wrong. Clear r1 after every multiply sequence unless the routine immediately consumes and overwrites it before any code can observe it.

```
mul  r16, r17      ; r1:r0 = product
movw r24, r0
clr  r1            ; restore zero register
```

15.6 Prologue and Epilogue Patterns

A prologue is the entry sequence that saves state and allocates stack space. An epilogue restores that state and returns.

15.6.1 Minimal Subroutine

```
small_worker:
    ; use only scratch registers
    ret
```

If a routine only uses scratch registers and has no nested state to preserve, it needs no pushes.

15.6.2 Saving Preserved Registers

```
accumulate:
    push r12
    push r13

    ; body uses r12:r13

    pop  r13
    pop  r12
    ret
```

The order is part of correctness. Push low/high or high/low as you prefer, but the pops must be the exact reverse.

15.6.3 Stack Frame with Y

Use a stack frame when a routine needs per-call storage that cannot live in registers. Y (r29:r28) is the natural frame pointer because LDD and STD support displacement addressing from Y.

This prologue allocates four local bytes:

```
with_frame:
    push r28
    push r29
    in   r28, SPL      ; Y = SP, currently below saved r29
    in   r29, SPH
    sbiw r28, 4        ; reserve 4 bytes below saved Y
    out  SPH, r29
    out  SPL, r28
```

After this prologue, Y equals the new SP. The local bytes are addressed as Y+1 through Y+4:

Higher addresses

Y+8 return low byte


```

Y+7    return high byte
Y+6    saved r28
Y+5    saved r29
Y+4    local byte 4
Y+3    local byte 3
Y+2    local byte 2
Y+1    local byte 1
Y+0    current SP value, below the allocated local area

```

Lower addresses

The saved Y bytes are above the local area. After `sbiw`, the active local bytes are normally accessed through `Y+1..Y+N`. Keep the layout written down when using stack frames. Off-by-one errors here corrupt saved registers or return addresses.

Access example:

```

ldi r16, 10
std Y+1, r16
ldi r16, 20
std Y+2, r16
ldd r17, Y+1

```

Epilogue:

```

    adiw r28, 4           ; Y/SP points below saved r29; POP pre-increments
    out  SPH, r29
    out  SPL, r28
    pop  r29
    pop  r28
    ret

```

15.6.4 When a Frame Is Worth It

Situation	Pattern
no saved state	no prologue
one or two preserved registers	push/pop only
several per-call temporary bytes	Y frame
recursion	stack storage for every value needed after the nested call
interrupt handler	save SREG and every register the handler modifies

15.7 Stack Depth Analysis

Stack overflow on a small AVR usually has no exception or fault signal. The stack simply overwrites SRAM data. You need to estimate the worst case.

For one active call:

stack bytes = return address bytes + pushed registers + local allocation

For ATtiny3217:

```

return address = 2 bytes
SRAM           = 0x3800..0x3FFF

```

Examples:

```
simple_add:
    2 return bytes + 0 pushes + 0 locals = 2 bytes
```

```
with_frame:
    2 return bytes + 2 saved Y bytes + 4 locals = 8 bytes
```

```
factorial recursive frame:
    2 return bytes + 1 saved n byte = 3 bytes
```

Peak stack use is the deepest active chain, not the total number of calls over time. If a main routine calls alpha, alpha calls beta, and beta calls gamma, add the frame sizes for all three at the deepest point.

Inspect SRAM allocation with:

```
avr-nm -n subroutines.elf
avr-size -A subroutines.elf
```

Then reserve a safety margin for interrupts. If an interrupt can occur at the deepest subroutine point, add the interrupt handler's pushes and return address to the worst-case stack budget.

15.8 Worked Example: Recursive Factorial

Factorial is a compact example because every recursive level needs to remember one byte: the current n .

```
0! = 1
1! = 1
n! = n * (n - 1)!
```

This routine uses:

```
Input:   r24 = n
Output:  r25:r24 = n!
Limit:   n <= 8 for a 16-bit result
```

8! = 40320 = 0x9D80, which fits in 16 bits. 9! = 362880, which does not.

```
factorial:
    cpi    r24, 2
    brlo   .Lbase           ; 0! and 1! return 1

    push   r24               ; save n for after the recursive call
    dec    r24
    rcall  factorial         ; r25:r24 = (n - 1)!

    pop     r16              ; r16 = original n
    rcall  mul8x16           ; r25:r24 = r16 * r25:r24, low 16 bits
    ret

.Lbase:
    clr    r25
    ldi    r24, 1
    ret
```

15.8.1 Stack Trace for factorial(4)

This trace shows only the saved n bytes. Each RCALL also pushes a 2-byte return address.

```
factorial(4):
  push 4
  factorial(3):
    push 3
    factorial(2):
      push 2
      factorial(1):
        return 1
      pop 2, compute 2 * 1 = 2
    pop 3, compute 3 * 2 = 6
  pop 4, compute 4 * 6 = 24
```

Peak active calls for factorial(4):

```
factorial(4), factorial(3), factorial(2), factorial(1)
4 return addresses = 4 * 2 = 8 bytes
saved n values      = 3 * 1 = 3 bytes
mul8x16 call        = 2 bytes while multiply helper is active
```

At the deepest recursive point before unwinding, the saved-n cost is 3 bytes. During each multiply helper call, one additional return address is active.

15.8.2 8 x 16 Multiply Helper

```
/*
 * mul8x16 - 8-bit x 16-bit -> low 16 bits
 * Input:   r16 = 8-bit factor
 *          r25:r24 = 16-bit factor
 * Output:  r25:r24 = low 16 bits of product
 * Clobbers r0, r1, r22, r23
 */
mul8x16:
  mul    r16, r24          ; low factor byte
  movw   r22, r0           ; r23:r22 = low partial
  mul    r16, r25          ; high factor byte
  add    r23, r0           ; add shifted contribution
  clr    r1
  movw   r24, r22
  ret
```

The helper intentionally truncates the product to 16 bits. That is fine for the documented factorial range, but it would be wrong for a routine promising a full 24-bit result.

15.9 Tail Jumps

A tail jump is a jump to another subroutine as the final action. It avoids creating another return address on the stack.

```
; extra return layer:
wrapper:
  rcall target
  ret

; tail jump:
wrapper:
  rjmp target          ; target's RET returns to wrapper's caller
```

Savings:

RCALL + RET replaced by RJMP = 4 cycles saved on ATtiny3217
 temporary return address avoided = 2 stack bytes on ATtiny3217

That cycle saving is RCALL (2) + RET (4), replaced by RJMP (2).

Use a tail jump only when the current routine has already restored any saved registers and returned SP to its entry value. If there is cleanup left to do, use a normal call and return.

15.10 Routine Template

Use this as a checklist, not as mandatory boilerplate:

```
.global routine_name
routine_name:
    ; Save preserved registers used by this routine.
    ; push r14
    ; push r15

    ; Optional Y frame.
    ; push r28
    ; push r29
    ; in    r28, SPL
    ; in    r29, SPH
    ; sbiw  r28, N
    ; out   SPH, r29
    ; out   SPL, r28

    ; Body.
    ; Put result in the documented output registers.

    ; Optional Y frame cleanup.
    ; adiw  r28, N
    ; out   SPH, r29
    ; out   SPL, r28
    ; pop   r29
    ; pop   r28

    ; Restore preserved registers in reverse order.
    ; pop   r15
    ; pop   r14
ret
```

For every routine, document:

- input registers
- output registers
- clobbered registers
- preserved registers used internally
- maximum stack use
- numerical limits, if any

15.11 Summary

RCALL k - call relative target, 2 cycles, 1 word, +/-2048-word offset.

CALL k - call absolute target, 3 cycles, 2 words.
 ICALL - call target in Z, 2 cycles.
 RET - pop return address, 4 cycles.
 RETI - interrupt return, 4 cycles. Sets SREG.I = 1 and clears the active level bit in CPUINT.STATUS on AVRxt.

ATtiny3217 return address:

2 bytes, stored as a program-memory word pointer.
 Low byte is pushed first and resides at the higher SRAM address.

Stack:

SRAM range: 0x3800..0x3FFF.
 Grows downward.
 PUSH stores at SP, then decreases SP by 1.
 POP increases SP by 1, then loads from SP.
 CALL/RCALL/ICALL decrease SP by 2.
 RET/RETI increase SP by 2.

SP updates:

CPU.SP is SPL/SPH.
 Write SPH before SPL for manual 16-bit updates.
 Writing SPL masks interrupts for up to four instructions or until the next I/O-memory write.

Register discipline:

r1 is the zero register; clear it after multiply instructions.
 Push/pop preserved registers in strict reverse order.
 SP must be balanced on every return path.

Stack usage:

return address bytes + pushed registers + local allocation.
 Add interrupt-handler stack use to the worst-case budget when interrupts can fire at the deepest point.

15.12 Exercises

1. Write `sum_range`: input `r24 = start`, `r22 = end`; output `r25:r24 = sum` of all integers from start through end inclusive. Verify 1 through 10 gives 55.
2. For `with_frame`, draw the stack after push `r28`, after push `r29`, after `sbiw r28, 4`, and after each `std Y+q` store. Assume SP starts at 0x3FFF.
3. Calculate peak stack use for `factorial(8)`, including recursive return addresses, saved `n` bytes, and the active `mul8x16` call during unwinding.
4. Rewrite `factorial` as a counted loop. Compare stack use against the recursive version.
5. Write a byte-swap routine: input `X` points to byte `A` and `Z` points to byte `B`. Swap the two SRAM bytes using one scratch register pair and no stack local storage.
6. Add a stack guard byte at the low end of an allowed stack region. Write a small check routine that branches to `stack_error` if the guard byte has changed.

Next: Chapter 16 — GPIO

Chapter 16

GPIO

GPIO (General-Purpose Input/Output) is the mechanism that connects the CPU to the physical world: sensors, LEDs, switches, relays, and anything else wired to a pin. This chapter covers the four dedicated GPIO instructions — IN, OUT, SBI, CBI — together with SBIC/SBIS for pin-state testing, and builds up to a fully debounced button-controlled LED.

The ATtiny3217 uses a **dual-register model** (VPORT + PORT) that differs from the classic DDRx/PORTx/PINx model found on ATmega parts. Both models are covered here; the worked example targets ATtiny3217.

That difference matters because a lot of AVR examples online assume the older ATmega naming. If you only memorize the old names, modern tinyAVR code will look stranger than it really is.

16.1 GPIO Fundamentals

Every GPIO pin has three properties the firmware controls:

Direction 0 = input (pin floats / follows external signal)
 1 = output (pin is driven high or low by the CPU)

Output value 0 = drive low (when direction = output)
 1 = drive high (when direction = output)

When direction = input:
0 = no pull-up (pin floats if nothing drives it)
1 = weak pull-up to VCC (prevents floating)

Input Readback: reflects actual voltage on pin

On all AVR devices, GPIO is controlled through I/O registers. The exact register names and addresses differ between classic ATmega and the newer tinyAVR 1-series, but the concepts are identical.

So before worrying about the register map, keep the simple mental model in mind: configure direction, choose driven value or pull-up behavior, then read the pin state back when needed.

16.2 Classic DDRx / PORTx / PINx (ATmega-Style)

Most AVR examples online target **ATmega328P** (used on the Arduino Uno). That device uses this three-register model per port:

DDRx	– Data Direction Register.	Bit n: 0 = input, 1 = output.
PORTx	– Output Data / Pull-Up.	Bit n: driven value (output) or pull-up enable (input).
PINx	– Input Data.	Reading bit n gives actual pin state; on many classic AVRs, writing 1 toggles the corresponding PORTx output latch.

On ATmega328P these are in the low I/O space (addresses 0x00–0x1F), so IN, OUT, SBI, CBI, SBIC, and SBIS all work directly:

```
/* ATmega328P example – NOT ATtiny3217 */

/* Configure PB5 (Arduino LED) as output */
sbi  DDRB, 5          ; DDRB bit 5 = 1 (output)

/* Drive LED on */
sbi  PORTB, 5         ; PORTB bit 5 = 1 (high)

/* Drive LED off */
cbi  PORTB, 5         ; PORTB bit 5 = 0 (low)

/* Read button on PD2 (input, pull-up enabled) */
cbi  DDRD, 2          ; DDRD bit 2 = 0 (input)
sbi  PORTD, 2         ; enable pull-up (input + PORTx bit = 1)
in   r16, PIND        ; r16 = all 8 bits of PIND
sbrc r16, 2           ; skip if bit 2 = 0 (button pressed)
rjmp not_pressed
```

ATtiny3217 does not have DDRx/PORTx/PINx registers. It uses the PORT peripheral described in the next section. If you see online examples that reference DDRB or PORTC, they are targeting classic ATmega devices — not the ATtiny3217 used in this book.

That is one of the easiest ways to get misled by otherwise-correct AVR code: it may be valid for the wrong AVR family.

16.3 ATtiny3217 Port Architecture

The ATtiny3217 exposes each physical port through **two register sets**:

16.3.1 VPORT — Virtual Port (Fast Path)

VPORT registers are in the low I/O address space (0x00–0x0B). Because they are below address 0x20, all six GPIO instructions work on them: IN, OUT, SBI, CBI, SBIC, SBIS.

Register	I/O addr	Memory addr	Width	Access
VPORTA_DIR	0x00	0x0000	8-bit	R/W
VPORTA_OUT	0x01	0x0001	8-bit	R/W
VPORTA_IN	0x02	0x0002	8-bit	R/W*
VPORTA_INTFLAGS	0x03	0x0003	8-bit	R/W(W1C)
VPORTB_DIR	0x04	0x0004	8-bit	R/W
VPORTB_OUT	0x05	0x0005	8-bit	R/W
VPORTB_IN	0x06	0x0006	8-bit	R/W*

VPORTB_INTFLAGS	0x07	0x0007	8-bit	R/W(W1C)
VPORTC_DIR	0x08	0x0008	8-bit	R/W
VPORTC_OUT	0x09	0x0009	8-bit	R/W
VPORTC_IN	0x0A	0x000A	8-bit	R/W*
VPORTC_INTFLAGS	0x0B	0x000B	8-bit	R/W(W1C)

W1C = Write 1 to Clear (write 1 to a flag bit to clear it)

R/W* = read pin state; writing 1 toggles the matching OUT bit

Note on tinyAVR 1-series addressing: On classic ATmega, there is a 0x20 offset between I/O addresses and memory addresses (I/O 0x00 = memory 0x0020). On ATtiny3217, this offset does not exist — I/O address N = memory address N. `_SFR_MEM8(0x0000)` and I/O address 0x00 are the same location.

VPORT registers are **aliases** of the full PORT registers. Writing to VPORTA_DIR is identical to writing to PORTA_DIR.

Microchip documents the VPORT mapping as four regular PORT registers:

Regular PORT register Virtual PORT register

PORTx.DIR	VPORTx.DIR
PORTx.OUT	VPORTx.OUT
PORTx.IN	VPORTx.IN
PORTx.INTFLAGS	VPORTx.INTFLAGS

Only those high-traffic registers are mirrored in VPORT. Pin configuration still lives in the full PORTx.PINnCTRL registers, so pull-ups, input inversion, and interrupt-sense setup require LDS/STS.

The big idea is that VPORT gives you the fast instruction-set view of a port, while the full PORT block gives you the richer configuration interface.

16.3.2 PORT — Full Port Registers (Configuration and Atomic Operations)

The full PORT peripheral sits in the extended memory range (above 0x3F) and cannot be accessed with IN/OUT/SBI/CBI. Use LDS/STS.

PORTA base: 0x0400

PORTB base: 0x0420

PORTC base: 0x0440

Offset	Register	Description
+0x00	DIR	Direction (0=in, 1=out) — full byte read/write
+0x01	DIRSET	Atomic direction set (write 1 to set bits)
+0x02	DIRCLR	Atomic direction clear (write 1 to clear bits)
+0x03	DIRTGL	Atomic direction toggle
+0x04	OUT	Output value — full byte read/write
+0x05	OUTSET	Atomic output set (write 1 to set bits)
+0x06	OUTCLR	Atomic output clear (write 1 to clear bits)
+0x07	OUTTGL	Atomic output toggle (write 1 to toggle bits)
+0x08	IN	Input value; write 1 to toggle matching OUT bit
+0x09	INTFLAGS	Interrupt flags — write 1 to clear a flag
+0x0A	Reserved	Reserved on ATtiny3217
+0x0B	Reserved	
+0x0C	Reserved	
+0x0D	Reserved	
+0x0E	Reserved	
+0x0F	Reserved	

+0x10	PIN0CTRL	Pin 0: ISC[2:0] + PULLUPEN + INVEN
+0x11	PIN1CTRL	Pin 1 configuration
...		
+0x17	PIN7CTRL	Pin 7 configuration

The **SET/CLR/TGL** variants perform atomic read-modify-write in hardware: a single STS `PORTA_OUTSET`, `r16` can set individual bits without disabling interrupts. Use these for multi-bit changes where atomicity matters (e.g., if an ISR also touches the same port).

This is one of the nicest parts of the newer AVR GPIO design. Instead of forcing software to read a whole port byte, modify one bit, and write the whole byte back, the peripheral can do the bit update internally.

The **PINnCTRL** registers at offsets 0x10-0x17 hold: - ISC[2:0] (bits 2:0) — interrupt sense (covered in Chapter 17) - PULLUPEN (bit 3) — enables the internal pull-up resistor - INVEN (bit 7) — inverts the pin logic

After reset, Microchip specifies all PORT outputs as tri-stated and the digital input buffers enabled. The PINnCTRL reset value is 0x00, which means no inversion, pull-up disabled, and ISC=0x0 (INTDISABLE: interrupt disabled, input buffer enabled).

Writing 1 to OUTTGL toggles the corresponding OUT bit. Microchip also documents that writing 1 to bit `n` of `PORTx.IN` or `VPORTx.IN` toggles the matching OUT bit, but this book uses OUTTGL because it makes the intent obvious and keeps input reads separate from output writes.

16.4 IN — Read I/O Register

IN Rd, A
 Operands: Rd ∈ {R0..R31}, A ∈ {0..63} (I/O address)
 Operation: Rd ← I/O(A)
 Flags: None
 Cycles: 1
 Encoding: 1011 0AA dddd AAAA

IN reads one byte from an I/O register into a general-purpose register. On ATtiny3217, VPORT registers sit at I/O addresses 0x00-0x0B and are within this range.

Think of IN as “take a snapshot of this register so the CPU can inspect it.”

```
in  r16, VPORTA_IN      ; r16 = current state of PORTA pins (8 bits)
in  r17, VPORTB_OUT     ; r17 = current output values of PORTB
in  r18, SREG           ; r18 = Status Register (a critical use of IN)
```

One common idiom: read SREG before a critical section, write it back after.

This shows up here because GPIO work and interrupt control often meet in the same small critical sections.

```
in  r24, SREG           ; save interrupt enable state
cli                               ; disable interrupts (Chapter 17)
; ... critical section ...
out SREG, r24           ; restore interrupt enable state exactly
```

16.5 OUT — Write I/O Register

OUT A, Rr

Operands: $A \in \{0..63\}$, $Rr \in \{R0..R31\}$
 Operation: $I/O(A) \leftarrow Rr$
 Flags: None
 Cycles: 1
 Encoding: 1011 1AAr rrrr AAAA

OUT writes one byte from a register to an I/O register.

Where IN captures a register value, OUT pushes a whole register value back into the I/O space in one step.

```
out  VPORTA_OUT, r16      ; drive all 8 PORTA output pins at once
out  VPORTB_DIR, r17      ; set direction for all 8 PORTB pins
out  SPL, r16             ; set Stack Pointer Low byte
```

Writing all 8 bits at once with OUT is often less convenient than the single-bit SBI/CBI instructions. Use OUT when you want to change multiple pins simultaneously (e.g., output a byte to an 8-bit bus).

So OUT is not the default for every GPIO task. It is the right tool when the whole byte matters.

16.6 SBI and CBI — Set / Clear Bit in I/O Register

SBI A, b
 Operands: $A \in \{0..31\}$, $b \in \{0..7\}$ ← I/O addresses 0x00–0x1F only
 Operation: $I/O(A).b \leftarrow 1$
 Flags: None
 Cycles: 1 on AVRxt (ATtiny3217), 2 on older AVRe devices

CBI A, b
 Operands: $A \in \{0..31\}$, $b \in \{0..7\}$
 Operation: $I/O(A).b \leftarrow 0$
 Flags: None
 Cycles: 1 on AVRxt (ATtiny3217), 2 on older AVRe devices

SBI and CBI are **atomic** single-bit operations. On ATtiny3217, the hardware internally does read-modify-write in one indivisible step: no interrupt can observe a partial state between the read and the write.

That atomicity is why these instructions are still so valuable even though the newer PORT peripheral has richer register options.

Range restriction: SBI/CBI only reach I/O addresses 0x00–0x1F (the lower 32 I/O registers). The VPORT registers (0x00–0x0B) are all within this range. Full PORT registers (0x0400+) require LDS/STS.

```
sbi  VPORTA_DIR, 3        ; set PA3 as output  (VPORTA_DIR = I/O 0x00)
cbi  VPORTA_OUT, 3        ; drive PA3 low      (VPORTA_OUT = I/O 0x01)
sbi  VPORTA_OUT, 3        ; drive PA3 high
sbi  VPORTB_DIR, 3        ; set PB3 as output
cbi  VPORTB_DIR, 5        ; set PB5 as input
```

When to use SBI/CBI vs. full PORT OUTSET/OUTCLR:

Situation	Instruction	Why
Set or clear one pin in VPORT	SBI/CBI	1-word instruction; 1 cycle on ATtiny3217
Set/clear bits, ISR may also touch port	STS PORTA_OUTSET	Guaranteed atomic
Set multiple bits at once	STS PORTA_OUTSET	Single write, no loop

If you remember only one distinction here, remember this one:

- SBI/CBI are ideal for one bit in low I/O space
- OUTSET/OUTCLR/OUTTGL are ideal for atomic updates in the full port block

16.7 SBIC and SBIS — Skip if Bit Clear / Set

SBIC A, b

Operands: $A \in \{0..31\}$, $b \in \{0..7\}$

Operation: if $I/O(A).b = 0$, skip next instruction

Flags: None

Cycles: 1 (no skip), 2 (skip 1-word instruction), 3 (skip 2-word)

SBIS A, b

Operands: $A \in \{0..31\}$, $b \in \{0..7\}$

Operation: if $I/O(A).b = 1$, skip next instruction

Flags: None

Cycles: 1 (no skip), 2 (skip 1-word instruction), 3 (skip 2-word)

These are the fastest way to branch on a single I/O pin state. Instead of `IN + SBRC/SBRS`, a single instruction reads and conditionally skips in one operation.

That is why SBIC and SBIS feel so natural in polling loops: test plus control flow happens in one instruction.

Wait for a button press (PB2 active-low):

```
.Lwait:
    sbis VPORTB_IN, 2      ; skip if PB2 = 1 (not pressed)
    rjmp .Lpressed        ; PB2 = 0: button is pressed
    rjmp .Lwait
```

Equivalent without SBIS:

```
in    r16, VPORTB_IN      ; 1 cy: read all PORTB pins
sbrs  r16, 2              ; 1 cy: skip if bit 2 = 1
rjmp  .Lpressed           ; 2 cy: branch if bit was 0 (pressed)
rjmp  .Lwait
```

The SBIC/SBIS version saves one register (r16 untouched) and avoids the `IN` instruction.

Small savings like that matter in tight loops and ISR-adjacent code.

Skip behavior for 2-word instructions: SBIC/SBIS skip exactly one instruction slot. If the instruction immediately after is a 2-word instruction (`JMP`, `CALL`, `LDS`, `STS`), the skip costs 3 cycles (the full 2-word instruction is skipped). Skip over a 1-word instruction costs 2 cycles.

So the timing of a skip depends not just on the skip instruction itself, but also on what comes immediately after it.

```
; Good: skip over a 1-word RJMP (common pattern)
sbic VPORTB_IN, 2           ; skip if PB2 = 0 (pressed)
rjmp .Lnot_pressed          ; 1 word – skipped in 2 cycles

; Also fine: skip over a 2-word STS
sbis VPORTA_IN, 3           ; skip if PA3 = 1
sts PORTA_OUTTGL, r16       ; 2 words – skipped in 3 cycles
```

16.8 Configuring a GPIO Output

Two equivalent approaches on ATtiny3217:

Method A — SBI on VPORT (single pin, 1 cycle on ATtiny3217):

```
sbi VPORTA_DIR, 3           ; PA3 = output
sbi VPORTA_OUT, 3          ; initial value: high (Curiosity Nano LED off)
```

Method B — STS with PORT DIRSET/OUTCLR (atomic, any number of pins):

```
ldi r16, (1 << 3)
sts PORTA_DIRSET, r16       ; PA3 = output, other bits unchanged
sts PORTA_OUTSET, r16       ; PA3 driven high: Curiosity Nano LED off
```

Both methods produce identical hardware results for a single pin. Method B is preferred when configuring multiple pins in a single write:

That means you can choose the style based on clarity and scaling, not because one is inherently more “correct” for a single pin.

```
ldi r16, (1 << 3) | (1 << 5) ; configure PA3 and PA5
sts PORTA_DIRSET, r16         ; both pins become outputs
sts PORTA_OUTCLR, r16         ; both driven low
```

16.9 Configuring a GPIO Input with Pull-Up

On ATtiny3217, pin direction defaults to input (0) at reset, so only the pull-up needs configuration. Pull-up lives in the PINnCTRL register, which is above the I/O space — LDS/STS required.

PINnCTRL register layout:

```
Bit: 7      6:4      3      2:0
      INVEN  reserved  PULLUPEN  ISC[2:0]
```

```
PULLUPEN (bit 3) = 0: pull-up off (pin floats if nothing drives it)
                  = 1: pull-up on (pin pulled toward VCC)
ISC[2:0]         = 0x00: interrupt disabled, input buffer enabled
                  = 0x04: interrupt and digital input buffer disabled
                  (other values in Chapter 17)
PORT_PULLUPEN_bm = 0x08
```

The floating input problem: If a digital input pin is not connected to anything and has no pull-up, its voltage floats between GND and VCC. The CPU reads random logic levels that change with temperature, noise, and nearby switching signals. Always enable the

pull-up when using a button wired to GND, add an external pull resistor, or drive the pin from a source you control.

An unconnected digital input is not neutral; it is unreliable.

```
; PB2 as input with pull-up (button wired to GND)
; Direction is already 0 (input) – nothing to do for DIR.
ldi r16, PORT_PULLUPEN_bm      ; = 0x08
sts PORTB_PIN2CTRL, r16        ; PORTB_PIN2CTRL = memory 0x0432

; PB3 as input, NO pull-up (externally driven signal, e.g. UART RX)
ldi r16, 0
sts PORTB_PIN3CTRL, r16        ; ISC = disabled, PULLUPEN = 0
```

Reading a pin:

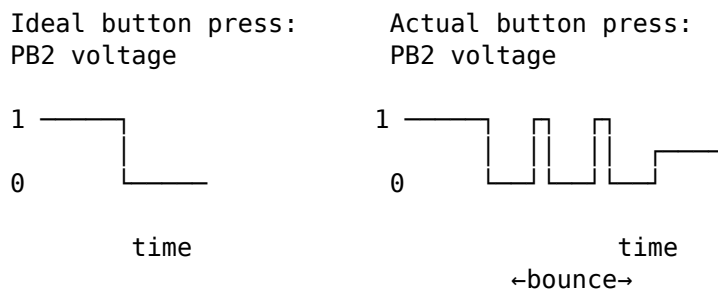
```
in r16, VPORTB_IN              ; read all 8 PORTB pins
sbrc r16, 2                    ; skip if bit 2 = 0 (button pressed)
; ... bit 2 = 1: button not pressed ...

; Or more directly:
sbis VPORTB_IN, 2              ; skip if PB2 = 1 (not pressed)
rjmp .Lpressed
```

16.10 Debounce

Mechanical switch contacts **bounce**: when you press or release a button, the contacts make and break contact several times in rapid succession before settling. The CPU is fast enough to see each bounce as a separate button event.

So debouncing is not a cosmetic improvement. It is what turns a noisy physical event into a single logical event.



There are two families of software fix. The first blocks: detect an edge, then busy-wait through the bounce. The second samples the pin periodically and only believes a change once it has been stable for a while. The blocking version is simplest; the periodic version is what most production firmware uses, because it never stalls the CPU.

16.10.1 Approach 1: Blocking Time Delay

After detecting a press, wait a few milliseconds (bounce settles in < 10 ms for most switches), then re-check. Do the same on release.

```
; Wait for press
.Lpoll:
sbis VPORTB_IN, BTN_BIT        ; skip if not pressed
rjmp .Lpressed
rjmp .Lpoll
```

```

.Lpressed:
    rcall delay_20ms          ; wait for bounce to settle

    ; Optional: confirm still pressed (guards against glitches)
    sbic VPORTB_IN, BTN_BIT   ; skip if still pressed (PB2 = 0)
    rjmp .Lpoll              ; was a glitch – restart

    ; Confirmed: genuine press
    ; ... handle event ...

    ; Wait for release
.Lwait_release:
    sbis VPORTB_IN, BTN_BIT   ; skip if released (PB2 = 1)
    rjmp .Lwait_release
    rcall delay_20ms          ; debounce the release
    rjmp .Lpoll

```

For typical tactile buttons, 20 ms is a safe debounce window. Some mechanical switches need 50 ms.

That delay is a human-timescale compromise: long enough to outlast bounce, but short enough that the user does not notice it.

The blocking method is fine for a toy, but it has real costs:

While `rcall delay_20ms` runs:

Nothing else happens – no other inputs, no display, no protocol.

Two buttons need interleaved waits and the code gets tangled fast.

A long bounce window directly adds latency to everything else.

16.10.2 Approach 2: Periodic Sampling (Non-Blocking)

Instead of pressing pause, sample the pin on a fixed schedule — say every 5 ms from a timer interrupt — and feed each sample into a small filter that only accepts a change after several samples agree. Between samples the CPU is free to do anything else. This is the standard production technique.

The filter used here is a **shift-register integrator**. Keep a byte that holds the last eight raw samples (1 = pressed, for an active-low button). Each tick, shift it left and drop the newest sample into bit 0:

```
history = (history << 1) | sample
```

After eight ticks the byte holds a full window. Two values matter:

```

history == 0xFF    last 8 samples all "pressed"    -> confirmed pressed
history == 0x00    last 8 samples all "released"   -> confirmed released
anything else      still bouncing                  -> ignore

```

A clean **press event** is the moment the debounced state goes from released to pressed. With a 5 ms tick, eight agreeing samples span ~40 ms, which easily outlasts contact bounce while staying imperceptible. A short bounce glitch never fills the window with a single value, so it is rejected automatically.

The reusable core reads the pin, updates the window, and toggles LED0 on each confirmed press edge. It is short enough to run inside the timer ISR:

```

; debounce_update – one sample; updates btn_history / btn_state in SRAM.
; Clobbers r24, r25, T, SREG.
debounce_update:
    in     r24, VPORTB_IN
    bst    r24, BTN_BIT          ; T = pin level (1 = released)

```

```

clr    r24
bld    r24, 0                ; r24 bit0 = pin level
ldi    r25, 1
eor    r24, r25              ; bit0 = !pin -> 1 means pressed

lds    r25, btn_history
lsl    r25                    ; history <= 1
or     r25, r24              ; | sample
sts    btn_history, r25

cpi    r25, 0xFF              ; full window pressed?
breq   .Ldb_pressed
cpi    r25, 0x00              ; full window released?
breq   .Ldb_released
ret                                ; still bouncing: no state change

.Ldb_pressed:
lds    r24, btn_state
cpi    r24, 1
breq   .Ldb_ret              ; already pressed: not a new edge
ldi    r24, 1
sts    btn_state, r24
ldi    r24, (1 << LED_BIT)
sts    PORTA_OUTTGL, r24     ; one clean press -> toggle LED0

.Ldb_ret:
ret

.Ldb_released:
clr    r24
sts    btn_state, r24
ret

```

The state byte is what makes this edge-triggered rather than level-triggered: the LED toggles once when the window first fills with presses, and not again until the button has been released (window all zeros) and pressed afresh. Holding the button down does not retrigger it.

The periodic call comes from a timer. Using TCA0 in FRQ (CTC) mode with a period of ~5 ms (Chapter 11 covers the setup), the interrupt handler is just a save/sample/clear-flag wrapper:

```

tca0_cmp0_isr:
push   r16
in     r16, SREG
push   r16
push   r24
push   r25

rcall  debounce_update

ldi    r16, TCA_SINGLE_CMP0_bm ; clear CMP0 flag (write-1-to-clear)
sts    TCA0_SINGLE_INTFLAGS, r16

pop    r25
pop    r24
pop    r16
out    SREG, r16
pop    r16
reti

```

main configures the LED output, the button input with its pull-up, the timer, enables interrupts, and then idles — every button event is handled by the tick. The full program is [debounce.S](#).

Blocking delay:

Simple to write.
Stalls the whole program.
One button at a time.
Fine for quick tests.

Periodic sampling:

Non-blocking; CPU free between ticks.
Scales to many buttons (one byte each).
Naturally produces clean edge events.
The production default.

A few practical variations:

Window length:

8 samples (one byte) is convenient. Use a 16-bit history for a longer, noisier switch, or fewer required samples for a faster response.

Many buttons at once:

Keep one history byte per button, or use a "vertical counter" that debounces all eight bits of a port in parallel with a couple of adds.

Event vs. state:

This core toggles the LED directly. In larger programs, set an event flag (or push to a queue) on the press edge and act on it in the main loop.

16.11 Worked Example: Button-Controlled LED

LED0 on PA3 (Curiosity Nano, active-low). External button on PB2 (active-low, internal pull-up). Each confirmed press toggles the LED state.

This example combines almost every GPIO concept in one small program: output setup, input pull-up, active-low logic, skip-based polling, debounce, and atomic output update.

Hardware:

ATtiny3217

VCC
|
PA3 — Curiosity Nano LED0 (on board, active-low)

PB2 —[internal pull-up]——— VCC

PB2 —[button]——— GND

Source file: src/gpio.S

```
#include <avr/io.h>

.equ LED_BIT,      3           ; PA3, Curiosity Nano LED0
.equ BTN_BIT,      2           ; PB2
.equ DEBOUNCE_COUNT, 16666     ; ~20 ms at 3.3 MHz (sbiw+brne loop)

.section .vectors, "ax", @progbits
.global __vectors
__vectors:
    rjmp main

.section .text
.global main

main:
```



```

ldi r16, hi8(RAMEND)
out SPH, r16
ldi r16, lo8(RAMEND)
out SPL, r16

sbi VPORTA_DIR, LED_BIT ; PA3 = output
sbi VPORTA_OUT, LED_BIT ; LED off (active-low)

ldi r16, PORT_PULLUPEN_bm ; 0x08
sts PORTB_PIN2CTRL, r16 ; PB2: input + pull-up, no interrupt

.Lloop:
sbis VPORTB_IN, BTN_BIT ; skip if PB2 = 1 (not pressed)
rjmp .Lpressed
rjmp .Lloop

.Lpressed:
rcall delay_20ms
sbic VPORTB_IN, BTN_BIT ; skip if PB2 is still low (still pressed)
rjmp .Lloop ; bounce/glitch: ignore

ldi r16, (1 << LED_BIT)
sts PORTA_OUTTGL, r16 ; atomic toggle PA3

.Lwait_release:
sbis VPORTB_IN, BTN_BIT ; skip if PB2 = 1 (released)
rjmp .Lwait_release
rcall delay_20ms
rjmp .Lloop

delay_20ms:
ldi r26, lo8(DEBOUNCE_COUNT)
ldi r27, hi8(DEBOUNCE_COUNT)
.Ldly_loop:
sbiw r26, 1 ; 2 cycles
brne .Ldly_loop ; 2 cycles taken
ret

```

16.11.1 Why PORTA_OUTTGL instead of SBI/CBI?

SBI VPORTA_OUT, 3 sets the bit; CBI VPORTA_OUT, 3 clears it. Neither alone toggles — you'd need to read the current state first. The naive approach:

```

; Non-atomic toggle – dangerous if ISR touches PORTA_OUT
in r16, VPORTA_OUT
ldi r17, (1 << LED_BIT)
eor r16, r17 ; flip LED bit
out VPORTA_OUT, r16 ; write back

```

Three instructions. An interrupt that fires between IN and OUT and modifies another PA pin will have its change overwritten by the OUT. The PORTA_OUTTGL approach is atomic: the hardware performs the read, XOR, and write in one step.

That is the real lesson: the shorter code is not just shorter, it is safer when something else can touch the same port state.

16.11.2 Build and Inspect

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o gpio.o src/gpio.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o gpio.elf gpio.o
avr-objdump -d gpio.elf      # inspect disassembly
avr-objcopy -O ihex gpio.elf gpio.hex  # generate flashable HEX
```

Check the disassembly around `.Lpressed` to confirm that `sts PORTA_OUTTGL` encodes as a 2-word `sts` instruction (opcode `0x92..`) with the 16-bit address `0x0407`.

16.12 Summary

IN Rd, A	– read I/O register A (0–63) into Rd.	1 cycle.
OUT A, Rr	– write Rr to I/O register A (0–63).	1 cycle.
SBI A, b	– set bit b of I/O register A (0–31).	1 cycle on AVRxt. Atomic.
CBI A, b	– clear bit b of I/O register A (0–31).	1 cycle on AVRxt. Atomic.
SBIC A, b	– skip next if I/O(A).b = 0.	1/2/3 cycles.
SBIS A, b	– skip next if I/O(A).b = 1.	1/2/3 cycles.

SBI/CBI/SBIC/SBIS range: I/O addresses 0x00–0x1F only.

VPORT registers (0x00–0x0B) are within this range: all six instructions apply.

Full PORT registers (0x0400+) need LDS/STS.

ATtiny3217 GPIO quick-reference:

Configure output:	<code>sbi VPORTA_DIR, n</code> or <code>sts PORTA_DIRSET, r16</code>
Drive high:	<code>sbi VPORTA_OUT, n</code> or <code>sts PORTA_OUTSET, r16</code>
Drive low:	<code>cbi VPORTA_OUT, n</code> or <code>sts PORTA_OUTCLR, r16</code>
Toggle output:	<code>sts PORTA_OUTTGL, r16</code> (atomic)
Configure input:	(DIR default = 0 – no action required)
Enable pull-up:	<code>sts PORTB_PIN2CTRL, r16</code> (PORT_PULLUPEN_bm = 0x08)
Read input:	<code>in r16, VPORTB_IN</code> or <code>sbic/sbis VPORTB_IN, n</code>

Debounce: after detecting edge, wait ~20 ms, re-check, wait for release, wait ~20 ms again before returning to poll loop.

16.13 Exercises

1. Reconfigure the example so the LED is on PC0 instead of PA3. Which registers change? Which instructions change? Is there any difference in the generated machine code size?
2. Write a routine `toggle_n_times` that accepts a count in r24 and toggles PA3 that many times with a 100 ms delay between each toggle. Use `PORTA_OUTTGL` and a subroutine-based delay.
3. The debounce loop calls `delay_20ms` twice: once after press, once after release. Under what button-press pattern could the LED be toggled without a real press if you removed the release debounce?
4. Rewrite the button poll using `SBIC` instead of `SBIS`. Draw the new control flow. Are the two approaches equivalent in terms of cycles per loop iteration?
5. How many cycles does the `delay_20ms` subroutine take from `RCALL` to the final `RET`? Show the sum: prologue, loop body × N, final loop exit, return.
6. Add a second button on PA5 (active-low, pull-up). When `BTN1` (PB2) is pressed, turn the LED on; when `BTN2` (PA5) is pressed, turn it off. Use only `SBIC`/`SBIS` — no `IN` instruction.
7. In `debounce.S`, `btn_history` starts at 0xFF and `btn_state` at 0. Trace what happens on the first eight ticks if the button is held down from reset. On which tick does the first

- press edge fire, and why does starting the history at 0xFF (rather than 0x00) avoid a false press at power-up?
8. Extend `debounce_update` to debounce two buttons at once by keeping a second history/state pair. How many SRAM bytes does each additional button cost, and does the timer tick rate need to change?
 9. Replace the direct LED toggle in `debounce_update` with an event: set a byte `btn_event` to 1 on the press edge, and have the idle loop in `main` poll it, toggle the LED, and clear it. What does this decoupling buy you in a larger program?
-

Next: Chapter 17 — Interrupts

Chapter 17

Interrupts

Polling burns CPU cycles checking whether something has happened. An **interrupt** lets hardware signal the CPU directly: the CPU pauses whatever it was doing, runs a short handler (ISR), and resumes exactly where it left off. This chapter covers the interrupt mechanism, the ATtiny3217 vector table, the SEI/CLI/RETI instructions, and the discipline required to write a correct ISR.

This is where the program stops being a single straight-line thread of control and starts reacting to the outside world asynchronously.

17.1 Why Interrupts?

The polling loop from Chapter 16:

```
.Lloop:
    sbis VPORTB_IN, BTN_BIT    ; check button every iteration
    rjmp .Lpressed
    rjmp .Lloop
```

This consumes 100% of the CPU checking one pin. In a real application, the CPU may need to: update a display, process serial data, manage a state machine — all while remaining responsive to button presses. Polling every possible event source and handling each one in priority order becomes unwieldy.

Interrupts invert the model: the CPU runs its main work and is **notified** when an event occurs. The ISR handles the event in a bounded time, then execution resumes with main code unmodified.

That last requirement is the hard part. An interrupt is only useful if the main code can continue as though nothing important was damaged by the pause.

17.2 The Interrupt Mechanism

17.2.1 What Happens When an Interrupt Fires

On ATtiny3217, think of interrupt entry as a short fixed hardware sequence: finish the current instruction, push the PC, then execute the vector entry. With the JMP-based vector table used by ATtiny3217, the first ISR instruction starts after **6 cycles minimum**: one cycle to finish the current instruction, two cycles to store the PC, and three cycles to execute the vector JMP.

That means interrupt latency is not arbitrary. It is mostly predictable, which is one reason small microcontrollers work well for real-time event handling.

1. CPU finishes the current instruction.
2. PC is pushed onto the stack (2 bytes on ATtiny3217).
3. CPUINT records the active interrupt level.
4. The interrupt vector entry executes (`JMP`` in this chapter's layout).
5. CPU begins the first instruction of the ISR body.

When the ISR executes RETI:

1. PC is popped from the stack ($SP += 2$).
2. CPUINT returns to the previous interrupt state.
3. CPU resumes the instruction after where it was interrupted.

The interrupted code is **unaware** an interrupt occurred — as long as the ISR leaves all registers and SREG flags exactly as it found them.

That “as long as” is the whole discipline of ISR writing.

17.2.2 The I Flag (Global Interrupt Enable)

SREG bit layout:

```

Bit:  7    6    5    4    3    2    1    0
      I    T    H    S    V    N    Z    C
      ↑
      Global Interrupt Enable
      1 = interrupts enabled
      0 = interrupts disabled (masked)

```

The I flag is a master switch. Even if a peripheral has its interrupt configured and pending, the CPU will not respond until $I = 1$. The AVR Instruction Set Manual specifies that RETI sets $SREG.I = 1$ on every AVR core, AVRxt included; AVRxt also tracks the active priority level in CPUINT.STATUS (covered below), which is what blocks same-or-lower-priority pre-emption while a handler is running. You can also control I explicitly with SEI, CLI, or a write to SREG from inside the handler.

So enabling a peripheral interrupt is never the whole story. The peripheral must be configured, and the global interrupt gate must also be open.

Microchip gives the initialization order explicitly:

1. Configure CPUINT only if the defaults are not enough.
2. Configure the peripheral interrupt condition and enable that source.
3. Set $SREG.I$ globally with SEI.

For this chapter, the CPUINT defaults are enough: normal vector table, fixed priority by vector address, no round-robin scheduling, and no level-1 vector. Advanced CPUINT bits such as IVSEL, CVT, LVL0RR, and LVL1VEC are left unchanged.

17.2.3 Interrupt Priority

On ATtiny3217, lower vector address = higher priority. RESET (0x0000) has the highest priority. If two interrupts become pending simultaneously, the one with the lower vector address fires first.

CPUINT tracks the current interrupt level. By default, all maskable interrupt vectors are level 0, so another level 0 interrupt is kept pending until the current handler returns. One vector can optionally be assigned to level 1 (high priority), and a level 1 interrupt can interrupt a level 0 handler.

For most firmware, leave the default level-0 behavior alone unless there is a clear timing reason to introduce a high-priority interrupt.

CPUINT also has a Compact Vector Table mode (CPUINT.CTRLA.CVT). When enabled, all level-0 interrupts share one vector. This saves flash but requires a shared dispatcher that checks interrupt flags. This book uses the normal table because one vector per source is easier to inspect in assembly.

17.3 SEI and CLI

SEI

Operation: SREG.I $\leftarrow$ 1
 Flags: I = 1
 Cycles: 1
 Note: The instruction immediately after SEI executes before any pending interrupt fires.

CLI

Operation: SREG.I $\leftarrow$ 0
 Flags: I = 0
 Cycles: 1
 Note: Guaranteed to prevent any interrupt from firing after CLI.

The one-instruction delay after SEI is important:

```
; Guaranteed: the RJMP executes before any pending interrupt fires.
sei
rjmp .Lidle
```

That behavior prevents awkward races where an interrupt could fire immediately between SEI and the next instruction.

17.3.1 Critical Sections

CLI/SEI create a **critical section** — a code region that cannot be interrupted:

```
cli                ; disable interrupts – begin critical section
; ... read or write shared variables that the ISR also touches ...
sei                ; re-enable interrupts – end critical section
```

A cleaner pattern that preserves the interrupt state rather than unconditionally re-enabling:

```
in   r24, SREG      ; save current SREG (including I flag)
cli                ; disable interrupts
; ... critical section ...
out  SREG, r24      ; restore SREG exactly (may or may not re-enable)
```

This is safe if the caller had interrupts disabled — SEI would have incorrectly re-enabled them, but restoring SREG preserves their disabled state.

This is the pattern to prefer in reusable code, because it does not assume that interrupts were enabled before the critical section began.

17.4 RETI — Return from Interrupt

RETI

Operation: PC $\leftarrow$ stack (pop 2 bytes), SP $\leftarrow$ SP + 2;
 SREG.I $\leftarrow$ 1;

CPUINT clears the active LVL0EX / LVL1EX / NMIEX bit
 Flags: I = 1. The rest of SREG is NOT restored automatically.
 Cycles: 4

On ATtiny3217, RETI is required to tell CPUINT that the interrupt handler has completed and to return the interrupt controller to the previous priority level. RET would pop the return address but leave CPUINT.STATUS showing an active handler, so the controller would refuse to dispatch the next same-or-lower-priority interrupt:

Using RET at end of ISR:	Using RETI at end of ISR:
PC returns, but CPUINT still thinks an ISR is active.	PC returns and CPUINT exits the interrupt state cleanly.
Same/lower-priority requests may remain blocked.	Pending interrupts can be serviced after one instruction.

The gap between executing the OUT SREG, r24 (in the epilogue) and RETI is safe because CPUINT has not yet completed the current interrupt. Pending interrupts are not serviced until after RETI, and Microchip documents that pending interrupts are served only after one instruction of the resumed program has executed.

That is why restoring SREG before RETI is correct even when the saved I bit is 1.

17.5 The Interrupt Vector Table

17.5.1 Layout on ATtiny3217

The vector table occupies the first 124 bytes of flash (31 entries × 4 bytes each). ATtiny3217 has more than 8 KB of flash, so avr-libc defines _VECTOR_SIZE as 4 bytes and every entry must fill a 2-word slot. This book uses one JMP instruction per vector entry.

Byte addr	Vector num	Peripheral	Trigger condition
0x0000	—	RESET	Power-on, external reset, WDT
0x0004	1	CRCSCAN_NMI	CRC scan failure (non-maskable)
0x0008	2	BOD_VLM	Voltage level monitor
0x000C	3	PORTA_PORT	Any PORTA pin with ISC configured
0x0010	4	PORTB_PORT	Any PORTB pin with ISC configured
0x0014	5	PORTC_PORT	Any PORTC pin with ISC configured
0x0018	6	RTC_CNT	RTC counter/compare
0x001C	7	RTC_PIT	RTC periodic interrupt
0x0020	8	TCA0_LUNF/TCA0_OVF	Timer/Counter A low underflow/overflow
0x0024	9	TCA0_HUNF	TCA high-byte underflow (split)
0x0028	10	TCA0_LCMP0/CMP0	TCA compare channel 0
0x002C	11	TCA0_LCMP1/CMP1	TCA compare channel 1
0x0030	12	TCA0_CMP2/LCMP2	TCA compare channel 2
0x0034	13	TCB0_INT	Timer/Counter B0
0x0038	14	TCB1_INT	Timer/Counter B1
0x003C	15	TCD0_OVF	Timer/Counter D overflow
0x0040	16	TCD0_TRIG	TCD trigger
0x0044	17	AC0_AC	Analog Comparator 0
0x0048	18	AC1_AC	Analog Comparator 1
0x004C	19	AC2_AC	Analog Comparator 2
0x0050	20	ADC0_RESRDY	ADC result ready
0x0054	21	ADC0_WCOMP	ADC window compare
0x0058	22	ADC1_RESRDY	ADC1 result ready
0x005C	23	ADC1_WCOMP	ADC1 window compare
0x0060	24	TWI0_TWIS	TWI slave
0x0064	25	TWI0_TWIM	TWI master

0x0068	26	SPI0_INT	SPI
0x006C	27	USART0_RXC	USART RX complete
0x0070	28	USART0_DRE	USART data register empty
0x0074	29	USART0_TXC	USART TX complete
0x0078	30	NVMCTRL_EE	EEPROM ready

17.5.2 Defining the Vector Table in .S

```
.section .vectors, "ax", @progbits
.global __vectors
__vectors:
    jmp main                /* 0x0000: RESET – must be first */
    jmp isr_default         /* 0x0004: CRCSCAN_NMI */
    jmp isr_default         /* 0x0008: BOD_VLM */
    jmp isr_default         /* 0x000C: PORTA_PORT */
    jmp portb_isr           /* 0x0010: PORTB_PORT ← our handler */
    jmp isr_default         /* 0x0014: PORTC_PORT */
    /* ... continue through vector 30 ... */
```

Each consecutive JMP is exactly 4 bytes, so the n th entry lands at byte address $n \times 4$ automatically — no .org directives needed.

Unhandled vectors should point to a catch-all handler that just RETIs, not to main (re-running main on a spurious interrupt would re-initialize your whole program). A simple trap for debugging: rjmp . (infinite loop).

Which default you choose depends on your goal:

- reti if you want unused interrupts to be harmless
- rjmp . if you want unexpected interrupts to halt the system for debugging

```
isr_default:
    reti                    /* ignore unexpected interrupts silently */
```

17.6 ISR Prologue and Epilogue

An ISR may fire at any point during the main code: between any two instructions. The main code has no way to know whether an ISR ran during a particular period. Therefore:

Every register written by an ISR must be saved before use and restored before RETI.

SREG must always be saved. Any instruction that modifies flags (arithmetic, logic, shifts, compares, even SBIW) will corrupt the flags that the interrupted code was relying on.

These are the core ISR rules. Violating them may not fail immediately, which is why ISR bugs can be so frustrating to diagnose.

17.6.1 Minimal ISR (uses r16 only)

```
my_isr:
    push r16                ; save r16 (1 cycle, 1 byte stack)
    in r16, SREG            ; save SREG into r16
    push r16               ; save SREG on stack (1 cycle, 1 byte)

    ; ... ISR body, may freely use r16 ...
```



```

pop    r16           ; restore SREG value into r16
out    SREG, r16     ; restore SREG
pop    r16           ; restore r16
reti

```

17.6.2 Why This Order?

Step	Stack contents (grows down)	SP points to
(on ISR entry)	[ret_hi][ret_lo]	top of ret addr
push r16	[ret_hi][ret_lo][r16]	saved r16
in r16, SREG	r16 = SREG value	
push r16	[ret_hi][ret_lo][r16][SREG_val]	← SP
... body ...		
pop r16	r16 = SREG_val, stack shrinks	
out SREG, r16	SREG restored	
pop r16	r16 = original r16 value	
reti	pop return addr, exit CPUINT ISR state, jump back	

On ATtiny3217, `OUT SREG, r16` restores H, S, V, N, Z, C, T, and the saved value of I. Immediately afterwards, `RETI` sets `I = 1` unconditionally and clears the active level in `CPUINT.STATUS`. If the ISR was reached with interrupts enabled (the usual case), the `OUT SREG, r16` and the `RETI` agree about I. If the saved SREG had `I = 0` — for example because the main code was inside a `CLI/SEI` critical section when the interrupt was already pending — `RETI` will re-enable interrupts as it returns, which is the documented behavior on all AVR cores.

That separation is one of the main reasons `RETI` exists as a distinct instruction instead of reusing ordinary `RET`.

17.6.3 Saving Multiple Registers

```

my_isr:
    push    r16
    push    r17
    push    r18
    in      r16, SREG
    push    r16           ; SREG last – now on top of stack

    ; ... ISR body using r16, r17, r18 ...

    pop     r16           ; SREG value
    out     SREG, r16
    pop     r18           ; restore in reverse order
    pop     r17
    pop     r16
    reti

```

17.6.4 ISR Stack Budget

Every `PUSH` costs 1 cycle on AVRxt and 1 byte of stack. The two-register minimal ISR (`r16` + `SREG`) uses: - 2 bytes for the return address (pushed automatically by CPU) - 2 bytes for the two `PUSH` instructions

Total: 4 bytes of stack per invocation. If main code uses 200 bytes of stack and ISRs nest, add the ISR frame size to your stack budget.

So ISR design is not just about speed. It is also about bounded stack usage.

Budget for the sum, not for whichever handler is largest. A stack that is fine until the day a level-1 interrupt lands at the deepest point of a level-0 ISR is a stack that fails intermittently in the field — the hardest kind of bug to catch.

17.7.4 Reentrancy

A routine is **reentrant** if a second invocation can begin before the first one has returned, without either corrupting the other. A routine is reentrant when it touches no shared mutable state — it works only in registers and on its own stack frame — or when every shared access it makes is atomic.

Reentrancy matters the moment the same code can run in two contexts at once. Two ways that happens here:

1. A subroutine called from BOTH main and an ISR.
(The ISR fires while main is partway through the routine.)
2. A subroutine called from BOTH a level-0 and the level-1 ISR.
(The level-1 interrupt fires while the level-0 ISR is in the routine.)

The classic non-reentrant trap is a shared variable updated with a read-modify-write. Suppose both main and an ISR run `tick_count += 1` on a 16-bit counter:

```
/* NOT reentrant: load / add / store is three steps, and the value is 16-bit */
lds  r24, tick_count
lds  r25, tick_count+1
adiw r24, 1
sts  tick_count, r24          /* ← ISR firing anywhere in here corrupts */
sts  tick_count+1, r25        /* the count: lost update or torn value */
```

If the ISR increments the same counter between main's load and store, main writes back a stale value and the ISR's increment is lost. The same hazard applies between a level-0 and a level-1 handler.

17.7.5 Protecting shared state

There is no single answer; the cheapest correct option depends on the access:

Access pattern	Protection needed
One writer, one reader, single byte (e.g. a ready flag, a ring-buffer head/tail index)	None. A byte LDS/STS is atomic and a single writer cannot race itself. This is the SPSC pattern from the circular-buffer section of Chapter 12.
Multi-byte value, OR read-modify-write, OR a struct touched as a unit	Critical section on the LOWER-priority side: CLI / work / SEI, or the save-SREG pattern from the Critical Sections section above. CLI masks ALL maskable interrupts – level 1 included – so it also fences off the preemptor.
State shared with a non-maskable interrupt (CRCSCAN_NMI, BOD)	You cannot mask it. Restrict the NMI to writing ONE atomic flag that the main loop polls; never share an RMW with it.

The asymmetry is the useful part: only the side that can be interrupted has to guard. An ISR reading a counter that only main writes with a critical section needs no guard of its own, because main's CLI already kept the ISR out during the write. Push shared data toward single-writer, byte-sized, atomic designs and most of these guards disappear.

17.7.6 Rules of thumb

- Leave everything at level 0 unless a specific latency requirement forces otherwise. Level 0 means no nesting and the simplest possible reasoning.
- Do not SEI inside an ISR expecting to enable nesting – on AVRxt it does not.
- If one source genuinely cannot wait, make it the level-1 vector and keep its handler tiny. Do not try to make a long handler preemptible from within.
- Prefer a single-writer atomic flag over a shared read-modify-write. When you must share an RMW, guard it with a critical section on the interruptible side.

17.8 Configuring Pin Interrupts on ATtiny3217

Classic ATmega devices have dedicated INT0/INT1 external interrupt pins. ATtiny3217 takes a different approach: **any pin** can generate an interrupt by configuring its PINnCTRL register.

This is more flexible than the old dedicated-pin model, but it also means you think in terms of per-pin configuration rather than special global interrupt pins.

17.8.1 PINnCTRL Layout

Bit:	7	6:4	3	2:0
	INVEN	reserved	PULLUPEN	ISC[2:0]

PULLUPEN = 1: enable internal pull-up (bit 3 = 0x08)

ISC[2:0]: Interrupt Sense Configuration

0x00	=	PORT_ISC_INTDISABLE_gc	-	interrupt disabled, input buffer on
0x01	=	PORT_ISC_BOTHEDGES_gc	-	interrupt on both edges (rise + fall)
0x02	=	PORT_ISC_RISING_gc	-	interrupt on rising edge (low → high)
0x03	=	PORT_ISC_FALLING_gc	-	interrupt on falling edge (high → low)
0x04	=	PORT_ISC_INPUT_DISABLE_gc	-	input buffer off (ADC pins, saves power)
0x05	=	PORT_ISC_LEVEL_gc	-	interrupt while pin is low (level triggered)

For a button wired to GND (active-low, fires on press = falling edge):

```
ldi r16, (PORT_PULLUPEN_bm | PORT_ISC_FALLING_gc) ; = 0x08 | 0x03 = 0x0B
sts PORTB_PIN2CTRL, r16 ; PB2: pull-up + falling edge interrupt
```

Microchip describes the PORT interrupt source as INTn in PORTx.INTFLAGS being raised according to ISC in PORTx.PINnCTRL. One PORTx_PORT vector covers the whole port, so the ISR must inspect INTFLAGS if more than one pin on that port can interrupt.

When changing interrupt sense settings, avoid doing it while an interrupt is being synchronized. The datasheet notes that changing ISC during synchronization may lose a request, and re-enabling an input after INPUT_DISABLE may request an interrupt even with different settings.

17.8.2 PORT INTFLAGS — Clearing Interrupt Flags

When an interrupt fires, the corresponding bit in PORT_INTFLAGS is set. The flag is **sticky**: it stays set until explicitly cleared. If you return from the ISR without clearing the flag, the interrupt fires again immediately.

That sticky-flag behavior is easy to forget the first time you write an ISR. If the flag is not cleared, the hardware still believes the event is pending.

Clearing: write 1 to the flag bit (write-1-to-clear, or W1C semantics).

Method A — STS with full PORT address (any bit):

```
ldi r16, (1 << BTN_BIT)      ; bit 2 = PB2
sts PORTB_INTFLAGS, r16      ; write 1 to bit 2 → clears flag
```

Method B — SBI on VPORTB_INTFLAGS (single bit, faster):

```
sbi VPORTB_INTFLAGS, BTN_BIT ; VPORTB_INTFLAGS = I/O 0x07
                           ; SBI writes 1 to bit 2 → clears flag
```

SBI writing a 1 to a W1C register clears the flag. This is counterintuitive (SBI normally “sets” a bit) but correct: the hardware interprets a write-1 as “clear this flag”.

This is one of those peripheral rules you simply have to memorize: W1C means “write one to clear,” even if the instruction name sounds like the opposite.

The flag for PORTB_PORT covers all PORTB pins. If multiple pins are configured with ISC ≠ 0, read PORTB_INTFLAGS first to determine which pin(s) fired before clearing.

17.9 ISR Latency and Timing

With the JMP vector style used here, the CPU reaches the first ISR instruction in **6 cycles minimum** from acknowledgement on ATtiny3217: one cycle to finish the ongoing instruction, two cycles to store the PC on the stack, and three cycles for the vector-table JMP. If the interrupt arrives during a multi-cycle instruction, that instruction must finish first, so total observed latency is higher.

```
Finish current instruction
Push 2-byte PC to stack
Execute vector-table JMP
Begin ISR body
```

At 3.3 MHz, one cycle is about 303 ns, so 6 cycles is about 1.8 us before the ISR body starts, not counting any extra wait for a multi-cycle interrupted instruction or synchronizer delay in the peripheral itself.

That is fast enough for many GPIO, timer, and serial-response tasks, but still slow enough that cycle counting matters in tight timing work.

The minimal ISR prologue shown here (PUSH r16, IN r16, SREG, PUSH r16) adds **3 cycles** before the first useful ISR work: - PUSH = 1 cycle - IN = 1 cycle - PUSH = 1 cycle

For time-critical ISRs, minimize the prologue: save only what you use.

In ISR code, every unnecessary instruction is paid on every event.

17.10 Worked Example: Pin Interrupt Toggles LED

Curiosity Nano LED0 on PA3, external button on PB2 (falling edge). The ISR toggles the LED; main loops forever doing nothing.

This is intentionally minimal so the interrupt path is easy to see in isolation.

Source file: src/interrupts.S

```
#include <avr/io.h>

.equ LED_BIT, 3
.equ BTN_BIT, 2
.equ BTN_CTRL, 0x0B      /* PORT_PULLUPEN_bm | PORT_ISC_FALLING_gc */
```

```

/* — Vector table ————— */
.section .vectors, "ax", @progbits
.global __vectors
__vectors:
    jmp main          /* 0x0000: RESET */
    jmp isr_default   /* 0x0004: CRCSCAN_NMI */
    jmp isr_default   /* 0x0008: BOD_VLM */
    jmp isr_default   /* 0x000C: PORTA_PORT */
    jmp portb_isr     /* 0x0010: PORTB_PORT */
    jmp isr_default   /* 0x0014: PORTC_PORT */
/* ... full source lists the remaining vectors as isr_default ... */

/* — Catch-all ISR ————— */
.section .text

isr_default:
    reti

/* — PORTB ISR ————— */
/*
* Prologue:    PUSH r16, IN r16-SREG, PUSH r16          (3 cycles, 2 stack)
* Body:        LDI r16, STS PORTA_OUTTGL, SBI VPORTB_INTFLAGS
* Epilogue:    POP r16, OUT SREG-r16, POP r16, RETI      (9 cycles, 0 stack)
*/
portb_isr:
    push r16
    in r16, SREG
    push r16

    ldi r16, (1 << LED_BIT)
    sts PORTA_OUTTGL, r16 /* toggle PA3 / LED0 */
    sbi VPORTB_INTFLAGS, BTN_BIT /* clear flag (WIC via SBI) */

    pop r16
    out SREG, r16
    pop r16
    reti

/* — main ————— */
.global main

main:
    ldi r16, hi8(RAMEND)
    out SPH, r16
    ldi r16, lo8(RAMEND)
    out SPL, r16

    sbi VPORTA_DIR, LED_BIT /* PA3 = output */
    sbi VPORTA_OUT, LED_BIT /* LED off (active-low) */

    ldi r16, BTN_CTRL
    sts PORTB_PIN2CTRL, r16 /* PB2: pull-up + falling edge interrupt */

    sei /* enable global interrupts */

.Lidle:
    rjmp .Lidle /* CPU idles, ISR handles everything */

```

17.10.1 Execution Flow Walkthrough

Power-on:

RESET vector (0x0000) → JMP main → main executes
 Configures PA3 output, PB2 pull-up + ISC, SEI
 Enters .Lidle loop

User presses button:

PB2 falls from 1 → 0 (falling edge)
 PORTB_PIN2CTRL ISC detects edge → sets PORTB_INTFLAGS bit 2
 CPU detects pending interrupt (I=1, so enabled)
 Finishes current RJMP .Lidle instruction
 Pushes PC onto stack and records the active interrupt level in CPUINT
 Fetches PORTB_PORT vector (0x0010) → JMP portb_isr
 Executes portb_isr:
 push r16 / save SREG
 sts PORTA_OUTTGL → PA3 toggles (LED changes state)
 sbi VPORTB_INTFLAGS, 2 → clears flag
 restore SREG / pop r16
 RETI → pop PC, exit CPUINT ISR state, jump back to .Lidle

User presses button again:

Same sequence, LED toggles back

17.10.2 Why Not Debounce in the ISR?

The Chapter 16 polling example used a 20 ms delay loop after each press. That loop busy-waits, which is unacceptable inside an ISR (other time-critical events would be missed, and ISR latency would explode to 20 ms).

Options for interrupt-based debouncing: 1. **Hardware debounce**: RC + Schmitt trigger on the button line. 2. **Timer debounce**: in the ISR, start a timer; the real handler fires when the timer expires (Chapter 19). Until the timer expires, disable the pin interrupt. 3. **Level-based re-arm**: change ISC to ISC_LEVEL or use a state machine in main that re-enables the interrupt after a delay.

For this example, the button is held long enough that multiple edges do not cause visible problems. Finished firmware should use method 2 or 3.

The example demonstrates the interrupt mechanism cleanly; a finished button interface should add a real debounce strategy.

17.10.3 Build and Inspect

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o interrupts.o src/interrupts.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o interrupts.elf interrupts.o
avr-objdump -d interrupts.elf
avr-objcopy -O ihex interrupts.elf interrupts.hex
```

Inspect the disassembly to confirm: - The vector table starts at address 0x0000 with a sequence of jmps. - portb_isr is at the address pointed to by the entry at 0x0010. - The sts PORTA_OUTTGL inside portb_isr is a 4-byte instruction. - sbi VPORTB_INTFLAGS, 2 is a 2-byte instruction.

17.11 Summary

SEI – set I flag (enable global interrupts). 1 cycle.

CLI – clear I flag (disable global interrupts). 1 cycle.
 RETI – return from ISR: pop PC, set SREG.I = 1, clear active
 CPUINT.STATUS level. 4 cycles. Never use RET here.

Interrupt response sequence (ATtiny3217, 6 cycles minimum with JMP vectors):
 finish current instruction → push PC → execute vector JMP → ISR

ISR discipline:

1. PUSH all registers used. (1 cy, 1 stack byte each)
2. IN r16, SREG → PUSH r16. (save SREG – always)
3. ISR body.
4. POP r16 → OUT SREG, r16. (restore SREG)
5. POP all registers in reverse order.
6. RETI. (not RET!)

Vector table: .section .vectors – sequential JMP instructions.
 4 bytes each. Entry n at byte address n×4. RESET at 0x0000.
 Unhandled vectors → isr_default: reti.

ATtiny3217 pin interrupt config:

```
sts PORTx_PINnCTRL, r16
  PORT_PULLUPEN_bm      = 0x08  (bit 3: pull-up)
  PORT_ISC_FALLING_gc    = 0x03  (bits 2:0: falling edge)
  PORT_ISC_RISING_gc     = 0x02
  PORT_ISC_BOTHEDGES_gc = 0x01
```

Clearing INTFLAGS (W1C):

```
sts PORTx_INTFLAGS, r16      (STS to full PORT – any bit)
sbi VPORTx_INTFLAGS, n       (faster – SBI writes 1 → clears flag)
```

Must be done before RETI or interrupt fires again.

17.12 Exercises

1. The current ISR saves r16 and SREG. If the ISR body also needed to use r17 and r18, where exactly (before/after which existing instruction) should the PUSH r17 and PUSH r18 go? Write the complete modified prologue and epilogue.
2. What happens if you replace `reti` with `ret` at the end of `portb_isr`? Describe the symptom you would observe and explain why it occurs in terms of CPUINT interrupt state.
3. The idle loop is `rjmp .Lidle`. If you replaced it with the AVR sleep instruction (opcode 0x9588), the CPU would enter idle sleep mode and halt until the next interrupt. What changes would be required in main to enable sleep mode? (Hint: the SLPCTRL peripheral needs to be configured — look at Chapter 1's linker-script walkthrough or the ATtiny3217 datasheet section on SLPCTRL.)
4. Configure PB3 for a second button that also fires on falling edge. In `portb_isr`, read `PORTB_INTFLAGS` to determine which pin fired, clear only that flag, and call one of two handlers: `led_on` or `led_off`. Write the complete modified ISR.
5. Using `avr-objdump -d interrupts.elf`, count the exact number of bytes used by the vector table. Does it match 31 vectors × 4 bytes = 124 bytes? What byte address does the first non-vector instruction appear at?
6. The ISR uses `sts PORTA_OUTTGL, r16` (a 4-byte instruction, 2 cycles). Could you replace it with a 2-byte instruction using the VPORT registers? If so, write the alternative. If not, explain why.
7. A developer ports an Arduino sketch and, wanting a timer ISR to be interruptible, adds `sei` as the first instruction of a level-0 TCB0 handler. They observe no change in behavior: a pending USART RX interrupt still does not preempt the timer ISR. Explain why.

- in terms of `CPUINT.STATUS` and the I flag. What single register write would actually let the USART RX interrupt preempt the timer handler, and what new hazard does that introduce for any variable both handlers touch?
8. `main` and a level-0 ISR both maintain a shared 16-bit `event_count` with a read-modify-write increment. Show the critical-section-protected version of `main`'s increment using the save-SREG pattern, and explain why the ISR's own increment needs no guard.

Next: Chapter 18 — Power and Clock Options

Chapter 18

Power and Clock Options

Clock and power control sit between GPIO/interrupts and the timing peripherals. Before you trust a timer period, a USART baud rate, or a delay loop, you need to know what clock the CPU and peripheral bus are using. Before you put a device to sleep, you need to know which clock domains stop and which interrupt sources can wake the part again.

This chapter covers the ATtiny3217 clock controller (CLKCTRL), the sleep controller (SLPCTRL), and the brown-out detector / voltage level monitor (BOD / VLM) from the point of view of assembly code.

18.1 The Default Clock

After reset, the ATtiny3217 uses the internal 16/20 MHz oscillator (OSC20M) as the main clock source. On the Curiosity Nano parts used in this book, the oscillator fuse selects 20 MHz and the main prescaler starts enabled with a divide-by-6 setting:

$$20 \text{ MHz} / 6 = 3.333333 \text{ MHz}$$

That default is why earlier examples use 3.333 MHz for rough delay loops and timer math. If your firmware changes the prescaler, every software delay, timer TOP value, PWM period, and asynchronous serial baud calculation must be checked again.

The important distinction:

CLK_MAIN	selected oscillator before the main prescaler
CLK_PER	prescaled clock distributed to CPU-visible peripherals
CLK_CPU	CPU/SRAM/NVM clock derived from the same main clock path

Many datasheet tables and register descriptions say CLK_PER because the peripheral bus sees the prescaled clock. In most examples in this book, that is also the effective CPU instruction clock.

18.2 Clock Sources

CLKCTRL_MCLKCTRLA.CLKSEL chooses the main clock source:

CLKSEL	Source
0x0	OSC20M: 16/20 MHz internal oscillator
0x1	OSCULP32K: 32.768 kHz internal ultra-low-power oscillator
0x2	XOSC32K: 32.768 kHz external crystal oscillator
0x3	EXTCLK: external clock input

The internal oscillator is the normal choice for examples and general firmware. It needs no external parts and starts quickly. The 32 kHz sources are useful for low-power timing, RTC/PIT work, or applications where a watch crystal gives better long-term timing than the internal ULP oscillator.

External clock options require board-level care. The datasheet warns that when switching to an external main clock, hardware waits for clock edges before the switch completes. If the selected external clock is not present, firmware may not be able to switch away again without a reset. Treat external clock selection as a board configuration decision, not as a casual run-time experiment.

18.3 Main Prescaler

CLKCTRL_MCLKCTRLB controls the main prescaler:

Bit:	7:5	4:1	0
	-	PDIV	PEN

PEN = 0 disables the prescaler and passes CLK_MAIN through undivided. PEN = 1 enables the prescaler, with PDIV selecting the division factor:

PDIV	Division
0x0	/2
0x1	/4
0x2	/8
0x3	/16
0x4	/32
0x5	/64
0x8	/6
0x9	/10
0xA	/12
0xB	/24
0xC	/48

The reset value of MCLKCTRLB is 0x11: PDIV = 0x8, PEN = 1, so the prescaler divides by 6.

Use documented field values together with header bit-position and bit-mask names when possible:

```
.equ MCLKCTRLB_DIV24, ((0x0B << CLKCTRL_PDIV_gp) | CLKCTRL_PEN_bm)

ldi r17, MCLKCTRLB_DIV24
rcall clock_write_mclkctrlb
```

That is clearer and safer than hard-coding 0x17.

18.4 Configuration Change Protection

Clock registers that can destabilize the system are protected by CCP (Configuration Change Protection). To write one of these registers, write the I/O protection key (0xD8) to CPU_CCP, then write the protected register immediately.

For example, this subroutine writes a new value to CLKCTRL_MCLKCTRLB. The new prescaler byte is passed in r17:

```
.equ CCP_IOREG_KEY, 0xD8

clock_write_mclkctrlb:
```

```
ldi    r16, CCP_IOREG_KEY
sts    CPU_CCP, r16
sts    CLKCTRL_MCLKCTRLB, r17
ret
```

Do not insert a call, branch, long calculation, or unrelated register write between CPU_CCP and the protected register write. The protected write must land inside the timed CCP window.

18.5 Waiting for Clock Switches

When firmware changes the main clock source, CLKCTRL_MCLKSTATUS.SOSC reports that the system oscillator is still changing. Status bits also report whether the selected oscillator is stable:

Bit	Meaning
EXTS	external clock has started
XOSC32KS	32.768 kHz crystal oscillator is stable
OSC32KS	internal 32.768 kHz oscillator is stable
OSC20MS	16/20 MHz oscillator is stable
SOSC	main clock source switch in progress

A conservative clock-source switch follows this shape:

```
; configure/enable the new oscillator if needed
; write CLKCTRL_MCLKCTRLA through CCP

.lwait_switch:
lds    r16, CLKCTRL_MCLKSTATUS
sbrc   r16, CLKCTRL_SOSC_bp
rjmp   .lwait_switch
```

Prescaler-only changes do not normally need this wait loop, but they still need the CCP unlock.

18.6 Sleep Modes

SLPCTRL_CTRLA selects the sleep mode and enables sleep entry:

SMODE	Mode
0x0	Idle
0x1	Standby
0x2	Power-down

SEN must be set before executing the SLEEP instruction.

```
.equ   SLPCTRL_MODE_PDOWN, (0x02 << SLPCTRL_SMODE_gp)

ldi    r16, (SLPCTRL_MODE_PDOWN | SLPCTRL_SEN_bm)
sts    SLPCTRL_CTRLA, r16
sleep
```

Sleep does not mean “resume later by magic.” An enabled interrupt source must wake the device. The CPU wakes, executes the ISR, returns with RETI, and then continues at the instruction after SLEEP.

18.6.1 Idle

Idle stops CPU execution but leaves peripherals running. All interrupt sources can wake the device. Use Idle when you want lower power without changing much about the rest of the system.

18.6.2 Standby

Standby stops more of the system. Peripherals that need to keep working must support and enable their own RUNSTDBY bit. Wake-up time may include oscillator start-up time unless the needed oscillator is already running.

18.6.3 Power-down

Power-down is the deepest sleep mode. The datasheet lists BOD, WDT, and the RTC PIT as active. Wake sources are limited: pin-change interrupt, PIT, VLM, TWI address match, and CCL. Only the PIT part of the RTC is available in Power-down.

That limitation is important. A TCA0 compare interrupt will not wake a Power-down system because TCA0 is not running there. Use PIT or a pin-change interrupt for deep-sleep wakeups.

18.6.4 RTC/PIT clock-source changes

The RTC and PIT use CLK_RTC, which is configured via RTC_CLKSEL. The datasheet notes that the clock source for CLK_RTC must only be changed when the RTC/PIT peripheral is disabled. In practice that means: disable RTC (RTC_RTCE=0) and PIT (RTC_PITEN=0), wait for any synchronization-busy flags to clear, then write RTC_CLKSEL, then re-enable the function you need.

18.7 Brown-Out and VLM

The Brown-Out Detector monitors VDD against a fuse-selected threshold and can reset the device before voltage drops too low for reliable operation. The Voltage Level Monitor is an early-warning interrupt source set above the BOD threshold.

Key points for firmware:

- The BOD level and the Active/Idle BOD operating mode are loaded from fuses at reset and cannot be changed by normal software.
- The Standby/Power-down BOD mode is also loaded from fuses, but the SLEEP field in BOD_CTRLA can be changed at run time through CCP.
- VLM follows the BOD mode. If BOD is disabled, enabling the VLM interrupt alone is not enough.
- VLM can wake from Power-down and can give code a chance to save state before a brown-out reset threshold is reached.

For many small examples, the right policy is simple: leave BOD fuse policy alone and do not spend power-tuning effort until the application has a measured current budget.

18.8 Worked Example: Change the Prescaler

src/clock_prescale.S shows a minimal firmware image that changes the main prescaler through CCP and blinks LED0. It starts from the factory default 3.333 MHz, switches to 20 MHz, blinks quickly, then switches to 416.7 kHz (20 MHz / 48) and blinks slowly.

Build:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o clock_prescale.o src/clock_prescale.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o clock_prescale.elf clock_prescale.o
avr-objdump -d clock_prescale.elf
```

The important routine is short:

```
clock_write_mclkctrlb:
    ldi    r16, CCP_IOREG_KEY
    sts    CPU_CCP, r16
    sts    CLKCTRL_MCLKCTRLB, r17
    ret
```

If the LED timing changes when this program runs, the CPU really did change speed. That is a crude test, but it is a useful first confirmation before you start recalculating timer and baud-rate values.

18.9 Worked Example: Power-Down Wake from PIT

src/sleep_pit.S configures the RTC PIT from the internal 1 kHz clock, enables a 1024-cycle PIT interrupt, selects Power-down sleep, and sleeps in the foreground loop. Each PIT interrupt wakes the device and toggles LED0.

The foreground loop is intentionally boring:

```
.Lsleep_forever:
    sleep
    rjmp  .Lsleep_forever
```

The useful work happens in the wake ISR. This is the usual low-power shape: configure a wake source, enable global interrupts, enter sleep, do brief work after wake, and go back to sleep.

18.10 Practical Rules

1. Decide the clock frequency before writing timer, USART, ADC, or delay-loop code.
2. Use header bit masks/positions such as CLKCTRL_PDIV_gp and CLKCTRL_PEN_bm instead of scattering raw register bytes through code.
3. Keep the CCP unlock and protected write adjacent.
4. Do not enter sleep until at least one valid wake interrupt is configured and global interrupts are enabled.
5. Use Idle for simple interrupt-driven waiting, Standby for peripheral-assisted waiting, and Power-down only when the wake source is known to work there.
6. Treat BOD fuses as part of the hardware power policy. Runtime code can adjust only the sleep-mode BOD behavior.

Microchip source notes:

- ATtiny3216/3217 Complete Datasheet DS40002205A, sections 10 (CLKCTRL), 11 (SLPCTRL), and 17 (BOD).
- Microchip online peripheral address map lists SLPCTRL at 0x0050, CLKCTRL at 0x0060, and BOD at 0x0080.
- The avr-libc device header iotn3217.h defines the register names, bit masks, and bit positions used by the assembly examples.

18.11 Exercises

1. Change `clock_prescale.S` to use `/24` instead of `/48`. What byte should `MCLKCTRLB` receive?
2. Recalculate the Chapter 19 `TCA0_CMP0` value for a 20 MHz clock with the timer prescaler still set to 1024.
3. Change `sleep_pit.S` from PIT period field value `0x09` (1024 cycles) to `0x0A` (2048 cycles). How often should the LED toggle from the 1 kHz PIT clock?
4. Replace the PIT wake source with a PORT pin-change wake source. Which sleep modes can still wake from that interrupt?
5. Enable VLM interrupt-on-crossing in a test program. Why should this be tested with a current-limited bench supply instead of a normal USB cable?

Chapter 19

Timer/Counter

Timers are the heartbeat of embedded firmware. They let you generate precise waveforms, schedule periodic work, measure elapsed time, and produce PWM signals — all without busy-waiting and without tying up the CPU. This chapter covers the TCA0 (Timer/Counter Type A) peripheral on the ATtiny3217, its main waveform-generation modes, and the interrupts each mode can generate. The worked example produces a 1 Hz timer tick that toggles LED0 from a CMP0 compare-match interrupt.

This is where you start handing time-sensitive repetition over to hardware instead of burning CPU cycles in delay loops.

19.1 Timer Fundamentals

19.1.1 What a Timer Actually Is

A timer is a hardware counter clocked by a divided version of the CPU clock. Each clock tick, the counter register (CNT) increments by 1. When CNT reaches a programmed limit (TOP), something happens — an interrupt fires, an output pin toggles, CNT resets — depending on the mode. The CPU is not involved; the peripheral manages everything in hardware.

That is the mental model to keep in view: a timer is just a counter plus rules about what to do when the counter reaches certain values.

The key equation:

$$f_{\text{interrupt}} = f_{\text{cpu}} / (\text{prescaler} \times (\text{TOP} + 1))$$

Rearranged for TOP given a desired frequency:

$$\text{TOP} = (f_{\text{cpu}} / (\text{prescaler} \times f_{\text{desired}})) - 1$$

Most timer configuration problems reduce to this equation plus careful attention to which register plays the role of TOP in the chosen mode.

19.1.2 Prescaler

The prescaler divides the CPU clock before it reaches the counter. Without it, a 16-bit counter at 20 MHz would overflow in 3.3 ms — useful for microsecond timing but not for a 1 Hz blink. Available prescaler values on TCA0:

Prescaler	Divides by	Max period @ 3.333 MHz (16-bit counter)
-----------	------------	---

DIV1	1	19.7 ms (65535 ticks)
------	---	-----------------------

DIV2	2	39.3 ms
DIV4	4	78.6 ms
DIV8	8	157.3 ms
DIV16	16	314.6 ms
DIV64	64	1.26 s
DIV256	256	5.03 s
DIV1024	1024	20.1 s

For a 1 Hz timer tick, DIV1024 gives 20 seconds of headroom with a 16-bit counter — plenty of room.

So the prescaler is what lets one timer peripheral cover both fast and slow timescales.

19.2 TCA0 on ATtiny3217

ATtiny3217 has one Timer/Counter Type A (TCA0) and two Timer/Counter Type B (TCB0, TCB1). TCA0 is the most versatile:

- 16-bit counter (CNT)
- Three compare channels (CMP0, CMP1, CMP2) each with an output pin option
- Three waveform generation modes
- Single mode (16-bit) or split mode (two independent 8-bit timers)

This chapter uses **single mode** (the default).

That is the easiest mode to reason about because the whole 16-bit counter stays together as one timer.

19.2.1 TCA0 Register Map

Register	Address	Width	Purpose
TCA0_SINGLE_CTRLA	0x0A00	8	Prescaler, enable
TCA0_SINGLE_CTRLB	0x0A01	8	Waveform generation mode, compare enable
TCA0_SINGLE_CTRLC	0x0A02	8	Compare output value override
TCA0_SINGLE_CTRLD	0x0A03	8	Split mode enable (leave 0 for single)
TCA0_SINGLE_CTRLCLR	0x0A04	8	Command/update bits clear side
TCA0_SINGLE_CTRLSET	0x0A05	8	Command/update bits set side
TCA0_SINGLE_CTRLFCLR	0x0A06	8	Buffer-valid flags clear side
TCA0_SINGLE_CTRLFSET	0x0A07	8	Buffer-valid flags set side
Reserved	0x0A08	8	Reserved
TCA0_SINGLE_EVCTRL	0x0A09	8	Event control
TCA0_SINGLE_INTCTRL	0x0A0A	8	Interrupt enable bits
TCA0_SINGLE_INTFLAGS	0x0A0B	8	Interrupt flags (W1C)
TCA0_SINGLE_DBGCTRL	0x0A0E	8	Debug run override
TCA0_SINGLE_TEMP	0x0A0F	8	Temporary bits for 16-bit access
TCA0_SINGLE_CNT	0x0A20	16	Current counter value (L at 0x20, H at 0x21)
TCA0_SINGLE_PER	0x0A26	16	Period (TOP in normal/PWM mode)
TCA0_SINGLE_CMP0	0x0A28	16	Compare 0 (also TOP in FRQ mode)
TCA0_SINGLE_CMP1	0x0A2A	16	Compare 1
TCA0_SINGLE_CMP2	0x0A2C	16	Compare 2
TCA0_SINGLE_PERBUF	0x0A36	16	Period buffer (double-buffered update)
TCA0_SINGLE_CMP0BUF	0x0A38	16	CMP0 buffer
TCA0_SINGLE_CMP1BUF	0x0A3A	16	CMP1 buffer
TCA0_SINGLE_CMP2BUF	0x0A3C	16	CMP2 buffer

All TCA0 registers are outside the 0x00-0x3F I/O range reached by IN/OUT, so they require LDS/STS. The 16-bit registers use two consecutive bytes: the L suffix register at the lower

address must be read or written first (low byte first for 16-bit accesses on AVR).

That means timer code is one of the places where the full memory-mapped peripheral model becomes very visible.

19.2.2 CTRLA — Control A

Bit:	7:4	3:1	0
	_____	_____	_____
	reserved	CLKSEL	ENABLE

CLKSEL[2:0] (occupies bits 3:1):

```

TCA_SINGLE_CLKSEL_DIV1_gc = (0x00 << 1) prescaler /1
TCA_SINGLE_CLKSEL_DIV2_gc = (0x01 << 1) prescaler /2
TCA_SINGLE_CLKSEL_DIV4_gc = (0x02 << 1) prescaler /4
TCA_SINGLE_CLKSEL_DIV8_gc = (0x03 << 1) prescaler /8
TCA_SINGLE_CLKSEL_DIV16_gc = (0x04 << 1) prescaler /16
TCA_SINGLE_CLKSEL_DIV64_gc = (0x05 << 1) prescaler /64
TCA_SINGLE_CLKSEL_DIV256_gc = (0x06 << 1) prescaler /256
TCA_SINGLE_CLKSEL_DIV1024_gc = (0x07 << 1) prescaler /1024

```

ENABLE (bit 0):

```

1 = timer running
0 = timer stopped (CNT holds its value)

```

Write ENABLE last after configuring the mode and TOP — avoids glitches where the timer runs briefly with incorrect settings.

This is a recurring embedded pattern: configure first, enable last.

19.2.3 CTRLB — Control B

Bit:	7	6	5	4	3	2:0
	—	CMP2EN	CMP1EN	CMP0EN	ALUPD	WGMODE[2:0]

CMP2EN/CMP1EN/CMP0EN: 1 = compare output drives the associated pin

ALUPD: auto-lock update of double-buffered registers on OVF/TOP

WGMODE[2:0]:

```

TCA_SINGLE_WGMODE_NORMAL_gc = 0x00 Normal — count to PER, interrupt on OVF
TCA_SINGLE_WGMODE_FRQ_gc    = 0x01 Frequency — count to CMP0, toggle pin,
                                interrupt on CMP0
0x02                        Reserved
TCA_SINGLE_WGMODE_SINGLESLOPE_gc = 0x03 PWM — count to PER, duty via CMPn
TCA_SINGLE_WGMODE_DSTOP_gc      = 0x05 Dual-slope PWM (top)
TCA_SINGLE_WGMODE_DSBOTH_gc     = 0x06 Dual-slope (top + bottom)
TCA_SINGLE_WGMODE_DSBOTTOM_gc  = 0x07 Dual-slope (bottom)

```

19.2.4 INTCTRL and INTFLAGS

Bit:	6	5	4	0
	CMP2	CMP1	CMP0	OVF

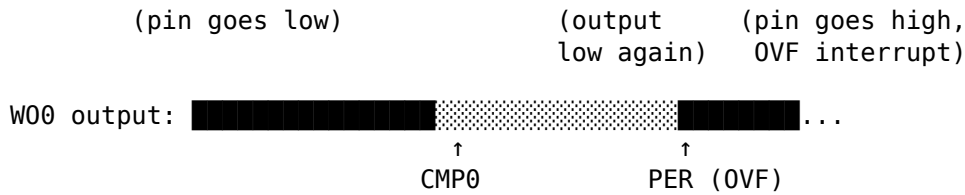
INTCTRL: writing 1 to a bit enables that interrupt.

INTFLAGS: bit set when interrupt fires. Clear by writing 1 (W1C).

```

TCA_SINGLE_OVF_bm = 0x01 Overflow / Underflow
TCA_SINGLE_CMP0_bm = 0x10 Compare 0 match
TCA_SINGLE_CMP1_bm = 0x20 Compare 1 match

```

Duty cycle $\approx \text{CMP0} / (\text{PER} + 1)$

CNT counts to PER. WO0 is high from 0 to CMP0, then low from CMP0 to PER. Duty cycle is approximately $\text{CMP0} / (\text{PER} + 1)$; CMP0 = BOTTOM produces a static low output and CMP0 > TOP produces a static high output. OVF interrupt fires when CNT wraps. This is the standard PWM mode for motor control, LED dimming, etc.

PWM mode is where the timer stops being just a scheduler and starts being a waveform generator.

PWM frequency:

$f_{\text{pwm}} = f_{\text{cpu}} / (\text{prescaler} \times (\text{PER} + 1))$
Duty $\approx \text{CMP0} / (\text{PER} + 1)$

19.4 Computing Timer Values

For a 1 Hz interrupt from a 3.333 MHz CPU using FRQ mode and prescaler 1024:

$\text{TOP} = (f_{\text{cpu}} / (\text{prescaler} \times f_{\text{desired}})) - 1$
 $= (3333333 / (1024 \times 1)) - 1$
 $= 3255.2 - 1$
 $= 3254.2 \rightarrow \text{round to } 3254$

Actual frequency check:

$f = 3333333 / (1024 \times (3254 + 1))$
 $= 3333333 / 3333120$
 $= 1.0000639 \text{ Hz (error } < 0.01\%)$

The 16-bit value 3254 in hex is 0x0CB6:

$3254 = 0x0CB6$
 $\text{lo8}(3254) = 0xB6$
 $\text{hi8}(3254) = 0x0C$

Low byte must be written first. The ATtiny3217 uses a 16-bit write latch: writing CMP0L latches the value, and writing CMP0H completes the transfer to the 16-bit register atomically. Reversing the order produces a glitch where the intermediate (wrong) value is briefly seen by the timer.

This is not stylistic ceremony. The write order is part of the hardware interface contract.

```
ldi r16, lo8(3254)
sts TCA0_SINGLE_CMP0L, r16 /* writes 0xB6, latches */
ldi r16, hi8(3254)
sts TCA0_SINGLE_CMP0H, r16 /* writes 0x0C, transfers 0x0CB6 atomically */
```

19.5 TCA0 Interrupt Vectors

From the ATtiny3217 vector table (Chapter 17):

Byte addr	Vector	Peripheral	Trigger
0x0020	8	TCA0_LUNF/OVF	Low underflow / overflow
0x0024	9	TCA0_HUNF	Split mode high-byte underflow
0x0028	10	TCA0_LCMP0/CMP0	Compare 0 match
0x002C	11	TCA0_LCMP1/CMP1	Compare 1 match
0x0030	12	TCA0_CMP2/LCMP2	Compare 2 match

In FRQ mode, the **TCA0_CMP0** interrupt (vector 10, address 0x0028) fires on every compare match. FRQ mode also reaches TOP at CMP0, so the OVF flag can be set at TOP, but this example does not enable the OVF interrupt. It enables only CMP0 and clears only CMP0.

So the interrupt source depends on the chosen mode, not just on the peripheral name.

To place `tca0_cmp0_isr` at vector 10, the vector table needs `jmp tca0_cmp0_isr` as vector entry 10 (counting RESET as entry 0):

```
__vectors:
    jmp main          /* 0x0000: RESET      */
    jmp isr_default   /* 0x0004: CRCSCAN    */
    jmp isr_default   /* 0x0008: BOD        */
    jmp isr_default   /* 0x000C: PORTA      */
    jmp isr_default   /* 0x0010: PORTB      */
    jmp isr_default   /* 0x0014: PORTC      */
    jmp isr_default   /* 0x0018: RTC_CNT    */
    jmp isr_default   /* 0x001C: RTC_PIT    */
    jmp isr_default   /* 0x0020: TCA0_OVF   */
    jmp isr_default   /* 0x0024: TCA0_HUNF  */
    jmp tca0_cmp0_isr /* 0x0028: TCA0_CMP0 ← */
```

19.6 The ISR

The TCA0 CMP0 ISR follows the same prologue/epilogue discipline from Chapter 17. It adds one step: **clear the INTFLAGS bit before returning**, or the interrupt fires again immediately.

```
tca0_cmp0_isr:
    push    r16
    in      r16, SREG
    push    r16

    ldi     r16, (1 << LED_BIT)
    sts     PORTA_OUTTGL, r16      /* toggle LED0 on PA3 (active low) */

    ldi     r16, TCA_SINGLE_CMP0_bm /* = 0x10 */
    sts     TCA0_SINGLE_INTFLAGS, r16 /* clear CMP0 flag (W1C) */

    pop     r16
    out     SREG, r16
    pop     r16
    reti
```

PORTA_OUTTGL is a toggle register — writing a 1 to bit *n* toggles PORTAn without affecting other bits. It is at address 0x0407 (outside I/O space, requires STS). Writing $(1 \ll \text{LED_BIT}) = 0x08$ toggles only PA3.

TCA0_SINGLE_INTFLAGS uses W1C semantics: writing TCA_SINGLE_CMP0_bm (bit 4 = 1) clears only the CMP0 flag. If you needed to clear all flags simultaneously: `ldi r16, 0xFF; sts`

TCA0_SINGLE_INTFLAGS, r16.

As with every interrupt source on AVR, forgetting to clear the flag turns one interrupt into an endless interrupt loop.

19.7 Main: Timer Initialization Sequence

Order matters. The safe sequence is:

1. Set CTRLB (waveform mode) – timer is not running yet
2. Write CMP0L then CMP0H – latch and load TOP value
3. Set INTCTRL – enable the interrupt we want
4. Set CTRLA – prescaler + enable bit (starts the timer)
5. SEI – enable global interrupts

Starting the timer before configuring CMP0 would cause the timer to run briefly with CNT=0 and CMP0=0, firing immediately. Starting before INTCTRL is set means the first match is missed silently (no interrupt enabled yet).

Initialization order matters because hardware begins reacting as soon as the enable bit is set.

main:

```
ldi    r16, hi8(RAMEND)
out    SPH, r16
ldi    r16, lo8(RAMEND)
out    SPL, r16

sbi     VPORTA_DIR, LED_BIT      /* PA3 = output (LED0 on Curiosity Nano) */
sbi     VPORTA_OUT, LED_BIT      /* LED off initially (active low: HIGH = off) */

/* 1. Waveform mode */
ldi     r16, TCA_SINGLE_WGMODE_FRQ_gc
sts     TCA0_SINGLE_CTRLB, r16

/* 2. TOP value: CMP0 = 3254 */
ldi     r16, lo8(3254)
sts     TCA0_SINGLE_CMP0L, r16
ldi     r16, hi8(3254)
sts     TCA0_SINGLE_CMP0H, r16

/* 3. Enable CMP0 interrupt */
ldi     r16, TCA_SINGLE_CMP0_bm
sts     TCA0_SINGLE_INTCTRL, r16

/* 4. Start timer: prescaler 1024, enable */
ldi     r16, (TCA_SINGLE_CLKSEL_DIV1024_gc | TCA_SINGLE_ENABLE_bm)
sts     TCA0_SINGLE_CTRLA, r16

/* 5. Global interrupt enable */
sei
```

.Lidle:

```
rjmp    .Lidle
```

19.8 Worked Example: 1 Hz Timer Tick

Full source: `src/timer_ctc.S`

The complete program combines the vector table, the CMP0 ISR, and main into one file. No shared state is needed between main and the ISR — main initializes hardware and parks in the idle loop; the ISR does all the work.

That makes this example a good demonstration of timer-driven design: main sets the policy, the timer peripheral generates the schedule, and the ISR performs the periodic action.

19.8.1 Register Usage

`r16`: `scratch` – used in prologue/epilogue and ISR body
(saved/restored each invocation)

No other registers needed. The simplest possible ISR.

LED0 on Curiosity Nano is active low (PA3 high = LED off, PA3 low = LED on). `PORTA_OUTTGL` toggles PA3 on every CMP0 match. With a 1 Hz interrupt, the LED changes state once per second, so a complete on-off cycle takes about two seconds.

19.8.2 Timing Analysis

Event	Cycles
TCA0 CMP0 match (hardware)	0
Interrupt request to CPU	1–2 (sync)
CPU finishes current RJMP	2
Push PC, enter CPUINT ISR state	2
JMP <code>tca0_cmp0_isr</code>	3
Total to ISR first instruction	~8
ISR prologue (PUSH + IN + PUSH)	3 cycles (1+1+1)
Toggle (LDI + STS)	3 cycles (1+2)
Clear flag (LDI + STS)	3 cycles (1+2)
ISR epilogue (POP + OUT + POP + RETI)	9 cycles
Total ISR duration	~18 cycles

At 3.333 MHz: $18 \text{ cycles} \times 300 \text{ ns} \approx 5.4 \text{ } \mu\text{s}$ ISR runtime per tick

The timer period is $3333333 / (1024 \times 3255) \approx 1.0 \text{ s}$. The ISR runtime ($\sim 5.4 \text{ } \mu\text{s}$) is 0.00054% of the period — nearly zero CPU overhead for the tick.

This is exactly why timers are so valuable. The CPU only pays for the tiny ISR runtime, not for the whole one-second wait.

19.8.3 Build Commands

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o timer_ctc.o src/timer_ctc.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o timer_ctc.elf timer_ctc.o
avr-objcopy -O ihex timer_ctc.elf timer_ctc.hex

# Inspect disassembly
avr-objdump -d timer_ctc.elf
```

19.8.4 Verifying with avr-objdump

Expected output (excerpts):

```

00000000 <__vectors>:
  0: XX XX XX XX  jmp  main
  4: ...
 28: XX XX XX XX  jmp  tca0_cmp0_isr    (TCA0_CMP0 vector at 0x0028)

<tca0_cmp0_isr>:
  push r16
  in   r16, 0x3f          ; SREG = 0x3f in I/O space
  push r16
  ldi  r16, 0x08          ; (1 << 3) = PA3
  sts  0x0407, r16        ; PORTA_OUTTGL
  ldi  r16, 0x10          ; TCA_SINGLE_CMP0_bm
  sts  0x0a0b, r16        ; TCA0_SINGLE_INTFLAGS
  pop  r16
  out  0x3f, r16          ; restore SREG
  pop  r16
  reti

```

Check that `tca0_cmp0_isr` is at the address that vector 0x0028 points to.

19.9 Extending: Normal Mode with OVF Interrupt

For cases where you want a fixed period and don't need compare-match output, use Normal mode with the OVF interrupt. Change two registers:

```

/* Normal mode – OVF fires when CNT reaches PER */
ldi  r16, TCA_SINGLE_WGMODE_NORMAL_gc    /* = 0x00 */
sts  TCA0_SINGLE_CTRLB, r16

ldi  r16, lo8(3254)
sts  TCA0_SINGLE_PERL, r16               /* PER (not CMP0) */
ldi  r16, hi8(3254)
sts  TCA0_SINGLE_PERH, r16

ldi  r16, TCA_SINGLE_OVF_bm              /* = 0x01: overflow interrupt */
sts  TCA0_SINGLE_INTCTRL, r16

```

The ISR changes slightly — clear OVF instead of CMP0:

```

tca0_ovf_isr:
  push r16
  in   r16, SREG
  push r16

  ldi  r16, (1 << LED_BIT)
  sts  PORTA_OUTTGL, r16

  ldi  r16, TCA_SINGLE_OVF_bm             /* = 0x01 */
  sts  TCA0_SINGLE_INTFLAGS, r16         /* clear OVF flag */

  pop  r16
  out  SREG, r16
  pop  r16
  reti

```

And the vector changes to entry 8 (0x0020 — TCA0_OVF):


```

jmp tca0_ovf_isr    /* 0x0020: TCA0_OVF */
jmp isr_default     /* 0x0024: TCA0_HUNF */
jmp isr_default     /* 0x0028: TCA0_CMP0 */

```

Both modes produce identical 1 Hz interrupt behavior. FRQ mode is marginally simpler (no PER register to configure separately).

That kind of tradeoff is common in timer work: several modes can solve the same problem, but one usually fits the mental model more directly.

19.10 Extending: Single-Slope PWM

In single-slope PWM the counter ramps from BOTTOM (0) up to TOP (PER), resets, and repeats. WO0 is **set** at BOTTOM and **cleared** when the counter reaches CMP0, so the duty cycle is $\text{CMP0} / (\text{PER} + 1)$. No interrupt is needed — the hardware drives the pin on its own.

The companion file `src/ch11_timer/timer_pwm.S` is a complete, runnable program that puts 25% duty at ~3.33 kHz on PB0 — the WO0 pin (per the datasheet's Multiplexed Signals table, WO0's default pin is PB0, not PA0):

```

#include <avr/io.h>

.equ TCA_WGMODE_SINGLESLOPE, 0x03    /* CTRLB WGMODE[2:0] = single-slope */

.section .text
.global main

main:
    /* WO0 pin (PB0) as output so the timer can drive it */
    sbi  VPORTB_DIR, 0

    /* period: PER = 999 → (999 + 1) = 1000 ticks = 300 us ≈ 3.33 kHz */
    ldi  r16, lo8(999)
    ldi  r17, hi8(999)
    sts  TCA0_SINGLE_PERL, r16
    sts  TCA0_SINGLE_PERH, r17

    /* duty: CMP0 = 250 → 250/1000 = 25% */
    ldi  r16, lo8(250)
    ldi  r17, hi8(250)
    sts  TCA0_SINGLE_CMP0L, r16
    sts  TCA0_SINGLE_CMP0H, r17

    /* waveform mode: single-slope PWM, enable compare-channel 0 output */
    ldi  r16, TCA_WGMODE_SINGLESLOPE | TCA_SINGLE_CMP0EN_bm
    sts  TCA0_SINGLE_CTRLB, r16

    /* start the timer: prescaler DIV1 (CLKSEL = 0), ENABLE */
    ldi  r16, TCA_SINGLE_ENABLE_bm
    sts  TCA0_SINGLE_CTRLA, r16

loop:
    rjmp loop                                /* PWM runs entirely in hardware */

```

Build and check it directly:

```
$ avr-gcc -mmcu=attiny3217 -nostartfiles timer_pwm.S -o timer_pwm.elf
$ avr-size timer_pwm.elf
   text    data     bss       dec      hex filename
    40       0       0       40       28 timer_pwm.elf
```

The pin must be set as output (the `sbi VPOR_TB_DIR, 0` above) **before** enabling the timer, or WO0 stays disconnected from the pin.

A note on the constants: `TCA_SINGLE_WGMODE_SINGLESLOPE_gc` and the `CLKSEL*_gc` names exist in `iotn3217.h` only as C enum members, so they do **not** assemble in a `.S` file — write the literal value instead (here `0x3` for the mode via `.equ`, and `0` for the `DIV1` prescaler). The `_bm` masks (`CMP0EN`, `ENABLE`) are `#defines` and work directly.

This is an important milestone: once PWM is configured, the peripheral can keep producing the waveform even if the CPU goes off and does something unrelated.

To change duty cycle at runtime, write a new value to `CMP0BUF` (the buffer). The hardware transfers the buffer into `CMP0` automatically at the next `UPDATE` condition (`BOTTOM` in single-slope mode), preventing glitches mid-cycle. The `ALUPD` bit in `CTRLB` is only needed when several compare channels must update together — it auto-manages the lock-update bit so all enabled buffers transfer on the same cycle.

19.10.1 WO pins, more channels, and dual-slope

In normal (16-bit) mode `TCA0` exposes three PWM channels, each on its own pin:

Channel	WO pin (default)	Alternate (via <code>PORTMUX</code>)
<code>CMP0</code>	<code>PB0</code>	<code>PB3</code>
<code>CMP1</code>	<code>PB1</code>	<code>PB4</code>
<code>CMP2</code>	<code>PB2</code>	<code>PB5</code>

Enable each with its `CMPnEN` bit in `CTRLB` and set the matching pin as output. All three share one `PER`, so they run at the same frequency with independent duty cycles — convenient for driving the three channels of an RGB LED.

The Curiosity Nano’s on-board LED is on `PA3`, which is **not** a normal-mode WO pin, so `TCA0` cannot dim it directly. Drive an external LED on a WO pin (`PB0`), or route a WO signal to `PA3` through the Event System and Configurable Custom Logic (see that chapter).

For center-aligned PWM — a pulse symmetric about the period midpoint, which cuts switching noise in motor and H-bridge drives — use a dual-slope mode: the counter ramps **up** to `PER` and back **down** to 0, so one full period spans $2 \times \text{PER}$ ticks. WO0 is cleared on the `CMP0` match during the up-ramp and set again on the `CMP0` match during the down-ramp, so the active pulse straddles the `BOTTOM` point.

The three dual-slope modes differ only in *when* the overflow (OVF) interrupt fires — useful when you want an ISR aligned to a particular point in the cycle:

WGMODE	value	counter behavior	OVF fires at
<code>SINGLESLOPE</code>	<code>0x3</code>	$0 \rightarrow \text{PER} \rightarrow 0$ (sawtooth)	TOP (reference)
<code>DSTOP</code>	<code>0x5</code>	$0 \rightarrow \text{PER} \rightarrow 0$ (triangle)	TOP
<code>DSBOTH</code>	<code>0x6</code>	$0 \rightarrow \text{PER} \rightarrow 0$ (triangle)	TOP and BOTTOM
<code>DSBOTTOM</code>	<code>0x7</code>	$0 \rightarrow \text{PER} \rightarrow 0$ (triangle)	BOTTOM

The duty-cycle and frequency math differs from single-slope. Because a period is $2 \times \text{PER}$ ticks (not `PER + 1`):

```
f_pwm = f_cpu / (prescaler * 2 * PER)
duty   = CMP0 / PER
```

The companion file `src/ch11_timer/timer_pwm_ds.S` puts the same 25% duty at ~ 3.33 kHz on `PB0`, but center-aligned. Only two things change from the single-slope program: the `WGMODE` value, and the `PER/CMP0` magnitudes (halved, since each tick is now counted twice per period):

```

.equ TCA_WGMODE_DSBOTTOM, 0x07 /* dual-slope, OVF at BOTTOM */

sbi   VPORTB_DIR, 0 /* PB0 = W00 output */

/* PER = 500 → 2 × 500 = 1000 ticks = 300 us ≈ 3.33 kHz */
ldi   r16, lo8(500)
ldi   r17, hi8(500)
sts   TCA0_SINGLE_PERL, r16
sts   TCA0_SINGLE_PERH, r17

/* CMP0 = 125 → 125/500 = 25% duty */
ldi   r16, lo8(125)
ldi   r17, hi8(125)
sts   TCA0_SINGLE_CMP0L, r16
sts   TCA0_SINGLE_CMP0H, r17

ldi   r16, TCA_WGMODE_DSBOTTOM | TCA_SINGLE_CMP0EN_bm
sts   TCA0_SINGLE_CTRLB, r16

ldi   r16, TCA_SINGLE_ENABLE_bm
sts   TCA0_SINGLE_CTRLA, r16

```

Dual-slope trades half the duty-cycle resolution (TOP is reached at PER, not $2 \times \text{PER}$) for a pulse that is symmetric about its center — the property that keeps two halves of an H-bridge from switching at the same instant. Runtime duty updates still go through CMP0BUF; in dual-slope the buffered value transfers at BOTTOM, so a mid-cycle write never produces a runt pulse.

19.11 Summary

TCA0 modes:

NORMAL (0x00): CNT → PER → reset; OVF interrupt.
 FRQ (0x01): CNT → CMP0 → reset; CMP0 interrupt; W00 toggles (if enabled).
 SS-PWM (0x03): CNT → PER → reset; W00 set at BOTTOM, cleared on CMP0; OVF int.
 DS-PWM (0x05/6/7): CNT $\leftrightarrow$ PER (triangle); center-aligned W00; period = $2 \times \text{PER}$.

Frequency formula:

single-slope: $f = f_{\text{cpu}} / (\text{prescaler} \times (\text{TOP} + 1))$, $\text{duty} = \text{CMP0} / (\text{PER} + 1)$
 dual-slope: $f = f_{\text{cpu}} / (\text{prescaler} \times 2 \times \text{PER})$, $\text{duty} = \text{CMP0} / \text{PER}$
 $\text{TOP} = (f_{\text{cpu}} / (\text{prescaler} \times f)) - 1$ (single-slope)

16-bit register write order: LOW byte first, then HIGH byte (latch).

Initialization order:

1. CTRLB (mode) 2. CMP0/PER (TOP) 3. INTCTRL 4. CTRLA (start) 5. SEI

Interrupt flag clear: write TCA_SINGLE_CMP0_bm (or TCA_SINGLE_OVF_bm) to TCA0_SINGLE_INTFLAGS before RETI — or interrupt fires again immediately.

ISR discipline: identical to Chapter 17.

PUSH all used regs → IN/PUSH SREG → body → POP/OUT SREG → POP regs → RETI.

ATtiny3217 TCA0 vectors:

0x0020 TCA0_OVF — overflow
 0x0028 TCA0_CMP0 — compare 0 match (FRQ mode trigger)

```
0x002C  TCA0_CMP1
0x0030  TCA0_CMP2
```

19.12 Exercises

1. The example uses prescaler 1024 and $CMP0 = 3254$. Calculate the $CMP0$ value needed for a 2 Hz blink with the same prescaler. What is the exact frequency error as a percentage?
2. Modify the example to blink the LED three times fast (150 ms on, 150 ms off) then pause for 700 ms. First reconfigure the timer for a 150 ms tick, then use a counter variable in SRAM to step through the pattern.
3. Calculate PER and $CMP0$ values for a 1 kHz PWM signal on WO0 with 25% duty cycle, using prescaler DIV1 at 3.333 MHz. Write the complete initialization sequence in assembly.
4. The ISR clears `TCA0_SINGLE_INTFLAGS` with an STS instruction. STS is a 4-byte, 2-cycle instruction. `TCA0_SINGLE_INTFLAGS` is at 0x0A0B — not in the VPORT I/O range (0x00–0x1F), so SBI cannot be used here. Confirm this by checking whether 0x0A0B mapped to any VPORT address. What is the smallest instruction count you can use to clear the flag?
5. Add a second compare channel ($CMP1$) to the FRQ-mode example. $CMP1$ should fire at 2 Hz (while $CMP0$ still fires at 1 Hz). What constraint does FRQ mode place on $CMP1$'s relationship to $CMP0$? (Hint: re-read the FRQ mode description — CNT resets at $CMP0$, not PER.)
6. Using `avr-objdump -d timer_ctc.elf`, verify that the `sts TCA0_SINGLE_CMP0L` instruction is encoded as a 4-byte instruction. What are the two 16-bit words of its encoding? (Refer to the AVR instruction set manual for the STS 32-bit format.)

Next: Chapter 20 — USART (Serial Communication)

Chapter 20

USART (Serial Communication)

USART (Universal Synchronous/Asynchronous Receiver/Transmitter) is the simplest way to communicate between a microcontroller and a PC. One wire transmits (TX), one receives (RX), and data flows as a timed sequence of bits. No clock line is needed — both sides agree on a baud rate (bits per second) in advance. This chapter covers the ATtiny3217's USART0 peripheral: register setup, polling transmit, polling receive, ISR-driven receive, and a complete echo terminal example.

This is often the first peripheral that makes a microcontroller feel “interactive” because it gives you a direct text path to a terminal.

20.1 UART Framing

20.1.1 The Bit Stream

USART uses NRZ (Non-Return-to-Zero) encoding: the line is high (idle) when no data is being sent. Each byte is framed with:

Idle: _____
Start: _____ | ← start bit (line goes low, 1 bit period)
 | D0 – D1 – D2 – D3 – D4 – D5 – D6 – D7 |
Stop: _____ (line high)

Timeline at 9600 baud (1 bit = 104.2 μ s):

1 bit	1 bit	... 8 bits ...	1 stop	idle ...
start	D0 LSB		D7 MSB	

One character transmission = 1 start + 8 data + 1 stop = **10 bit periods**. At 9600 baud: $10 \times 104.2 \mu\text{s} = 1.042 \text{ ms}$ per byte $\rightarrow$ maximum ~ 960 bytes/second.

That simple 10-bit framing cost is worth remembering. Even before software overhead, UART throughput is lower than the raw baud number suggests.

20.1.2 8N1 Format

8N1 means: - **8** data bits (LSB first) - **N** — no parity bit - **1** stop bit

This is the most common UART configuration and the ATtiny3217 USART0 default after reset.

So for ordinary 8-bit serial text, the default framing is already the framing you want.

20.2 USART0 Registers on ATtiny3217

Register	Address	Purpose
USART0_RXDATA_L	0x0800	Receive data low byte (read = receive byte)
USART0_RXDATA_H	0x0801	Receive data high byte (error flags + 9th bit)
USART0_TXDATA_L	0x0802	Transmit data (write = queue byte to send)
USART0_TXDATA_H	0x0803	Transmit data high byte (9th bit when used)
USART0_STATUS	0x0804	Status flags (RXCIF, TXCIF, DREIF, etc.)
USART0_CTRLA	0x0805	Interrupt enables, RS-485 mode
USART0_CTRLB	0x0806	TX/RX enable, start-frame detect, RX mode
USART0_CTRLC	0x0807	Frame format: baud mode, char size, parity, stop bits
USART0_BAUD_L	0x0808	Baud rate register low byte
USART0_BAUD_H	0x0809	Baud rate register high byte
USART0_CTRLD	0x080A	Auto-baud width field
USART0_DBGCTRL	0x080B	Debug run override
USART0_EVCTRL	0x080C	IrDA event input enable
USART0_TXPLCTRL	0x080D	IrDA transmitter pulse length
USART0_RXPLCTRL	0x080E	IrDA receiver pulse length

All are outside the 0x00-0x3F I/O range reached by IN/OUT, so use LDS/STS.

That means USART code on ATtiny3217 naturally uses the “full peripheral register” access style rather than the short IN/OUT style.

20.2.1 STATUS — Status Register (0x0804)

Bit:	7	6	5	4	3	2	1	0
	RXCIF	TXCIF	DREIF	RXSIF	ISFIF	—	BDF	WFB

- RXCIF — RX Complete Interrupt Flag: 1 when unread byte in RXDATA_L.
Cleared automatically by reading RXDATA_H.
- TXCIF — TX Complete Interrupt Flag: 1 when last byte shifted out fully.
Cleared by writing 1 (W1C).
- DREIF — Data Register Empty Flag: 1 when TXDATA_L is ready for new byte.
Cleared automatically when TXDATA_L is written.
On reset, DREIF = 1 (TX is ready immediately).
- RXSIF — Receive Start Interrupt Flag, used with start-frame detection.
- ISFIF — Inconsistent Sync Field Interrupt Flag, used with auto-baud/LIN modes.
- BDF — Break Detected Flag, used with break/sync detection.
- WFB — Wait For Break control/status bit.

Received-byte error flags are not in STATUS. They are stored with the received byte in RXDATA_H: BUFOVF at bit 6, FERR at bit 2, PERR at bit 1, and DATA[8] at bit 0 for 9-bit frames. Read RXDATA_H before RXDATA_L when you need those error bits.

For polling TX: wait until DREIF = 1, then write TXDATA_L. For polling RX: wait until RXCIF = 1, then read RXDATA_L. For ISR-driven RX: enable RXCIE in CTRLA; ISR fires when RXCIF sets.

Those three sentences are the whole USART mental model in miniature: one flag for “ready to transmit,” one flag for “data received,” and one interrupt-enable bit if you want hardware to notify you instead of polling.

20.2.2 CTRLB — Control B (0x0806)

Bit:	7	6	5	4	3	2:1	0
	RXEN	TXEN	—	SFDEN	ODME	RXMODE	MPCM

RXEN — 1 = receive enabled (USART monitors RXD pin)

TXEN – 1 = transmit enabled (USART drives TXD pin)
 SFDEN – 1 = start-frame detection enabled
 ODME – 1 = open-drain mode enabled
 RXMODE – 0 = normal (16x oversampling), 1 = CLK2X (8x), 2 = GENAUTO, 3 = LINAUTO
 MPCM – 1 = multiprocessor communication mode

Bit constants (from avr/io.h):

USART_TXEN_bm = 0x40
 USART_RXEN_bm = 0x80
 USART_SFDEN_bm = 0x10
 USART_ODME_bm = 0x08
 USART_RXCIE_bm is in CTRLA (see below)

20.2.3 CTRLA — Control A / Interrupt Enables (0x0805)

Bit:	7	6	5	4	3	2	1:0
	RXCIE	TXCIE	DREIE	RXSIE	LBME	ABEIE	RS485

RXCIE – RX Complete Interrupt Enable: fires ISR when RXCIF = 1
 TXCIE – TX Complete Interrupt Enable
 DREIE – Data Register Empty Interrupt Enable
 RXSIE – Receive Start Interrupt Enable
 LBME – Loop-back Mode Enable
 ABEIE – Auto-baud Error Interrupt Enable

Bit constants:

USART_RXCIE_bm = 0x80
 USART_TXCIE_bm = 0x40
 USART_DREIE_bm = 0x20

20.2.4 CTRLC — Frame Format (0x0807)

Bit:	7:6	5:4	3	2:0
	CMODE	PMODE	SBMODE	CHSIZE

CMODE (bits 7:6): communication mode

0x00 = USART_CMODE_ASYNCHRONOUS_gc – async (normal UART)
 0x01 = USART_CMODE_SYNCHRONOUS_gc – sync
 0x02 = USART_CMODE_IRCOM_gc – IrDA
 0x03 = USART_CMODE_MSPI_gc – SPI master

SBMODE (bit 3): stop bits

0 = 1 stop bit
 1 = 2 stop bits

PMODE (bits 5:4): parity

field 0x0 = disabled
 field 0x2 = even
 field 0x3 = odd

CHSIZE (bits 2:0):

0x03 = 8-bit (USART_CHSIZE_8BIT_gc) – most common

Default after reset = 0x03 = async, 8-bit, no parity, 1 stop = 8N1.
 No write needed if 8N1 is desired.

20.3 Baud Rate Calculation

20.3.1 The Formula

USART0 on ATtiny3217 uses a 16-bit baud register (BAUDL + BAUDH) with a fractional format. For **asynchronous normal mode** (16× oversampling):

$$\begin{aligned}\text{BAUD_REG} &= (\text{f_cpu} \times 64) / (16 \times \text{baud_rate}) \\ &= (\text{f_cpu} \times 4) / \text{baud_rate}\end{aligned}$$

The × 64 and ÷ 16 come from the internal fractional divider format — the top 10 bits of BAUD_REG are the integer part, and the bottom 6 bits are the fractional part.

You do not need to memorize the internal fractional details to use the peripheral well, but they explain why the formula looks stranger than a simple clock-divider equation.

For 9600 baud at 3.333 MHz:

$$\begin{aligned}\text{BAUD_REG} &= (3333333 \times 64) / (16 \times 9600) \\ &= 21333312 / 153600 \\ &= 1388.88875 \\ &\rightarrow \text{round to } 1389 = 0x056D\end{aligned}$$

$$\begin{aligned}\text{Actual baud rate} &= (3333333 \times 64) / (16 \times 1389) \\ &= 21333312 / 22224 \\ &= 9599.23 \text{ bps} \quad (-0.008\% \text{ error} - \text{negligible})\end{aligned}$$

Common baud rates at 3.333 MHz:

Baud	BAUD_REG (decimal)	BAUD_REG (hex)	Error
9600	1389	0x056D	-0.01%
19200	694	0x02B6	0.06%
38400	347	0x015B	0.06%
57600	231	0x00E7	0.21%
115200	116	0x0074	-0.22%

Writing BAUD (low byte first):

```
ldi r16, lo8(1389) /* = 0x6D */
sts USART0_BAUDL, r16
ldi r16, hi8(1389) /* = 0x05 */
sts USART0_BAUDH, r16
```

As with other 16-bit peripheral values on AVR, the write order is part of the hardware interface, not just style.

20.4 TX/RX Pins on ATtiny3217

USART0 uses **PORTB** by default:

Function	Pin	Direction	Notes
TXD	PB2	Output	USART overrides the pin when TXEN=1
RXD	PB3	Input	Leave as input (default); no pull-up needed

Microchip documents that the transmitter overrides normal port operation for the TXD pin when TXEN=1. The example still sets PB2 as an output before enabling the transmitter so the idle level is well-defined during setup.

```
sbi VPORTB_DIR, 2 /* PB2 = output for TXD */
/* PB3 (RXD) = input by default – no configuration needed */
```


Alternative pin mapping (PORTMUX_USARTROUTEA register) allows moving USART0 to other pins, but this chapter uses the default mapping.

20.5 Polling Transmit

The simplest TX approach: spin until DREIF = 1, then write the byte.

Polling transmit is usually acceptable because the program already knows when it wants to send something.

AVR Instruction: LDS Rd, A – load from SRAM/peripheral
SBRS Rd, b – skip next if bit b of Rd is set

```
/*
 * usart_tx_byte – transmit r16 via polling
 * Clobbers: r17
 */
usart_tx_byte:
    push    r17
.Lwait_dreif:
    lds     r17, USART0_STATUS
    sbrs    r17, USART_DREIF_bp    /* bit 5 – skip if DREIF = 1          */
    rjmp    .Lwait_dreif           /* DREIF = 0: TX busy, loop        */
    sts     USART0_TXDATA, r16     /* write byte: clears DREIF, starts TX */
    pop     r17
    ret
```

USART_DREIF_bp is the bit position (5) of DREIF in the STATUS register. SBRS r17, 5 skips the RJMP when bit 5 is 1 (DREIF set = TX ready).

So the loop is doing exactly one question over and over: “is the transmit data register empty yet?”

20.5.0.1 Why Not SBIS?

SBIS and SBIC only work with the bit-accessible I/O registers at data addresses 0x00–0x1F. On the ATtiny3217 (a modern AVRxt core) the I/O space is mapped directly into data space starting at 0x0000 — there is no +0x20 offset like on classic ATmega parts, so the bit-accessible window is simply 0x00–0x1F. USART0_STATUS is at 0x0804 — far outside that range. Use LDS + SBRS for peripheral registers.

20.5.0.2 Cycle Cost of Polling

Each loop iteration: LDS (3 cy) + SBRS (1 cy, not skipping) + RJMP (2 cy) = 6 cycles. At 3.333 MHz, 6 cycles = 1.8 µs per iteration. At 9600 baud, one bit period is 104 µs — the polling loop runs ~58 times per bit period. Acceptable for simple programs; for power-sensitive applications, use DREIE interrupt instead.

That is the tradeoff in one paragraph: polling is simple, but it spends CPU time checking a condition that usually is not ready yet.

20.5.1 Transmitting a String from Flash

```
/*
 * usart_tx_string – send bytes from flash
 * ZL:ZH = pointer to first byte, r18 = length
 * Clobbers: r16, r17, r18, Z
 */
```

```

usart_tx_string:
    tst    r18
    breq   .Ltx_done
.Ltx_loop:
    lpm    r16, Z+           /* load byte from flash, post-increment Z */
    rcall  usart_tx_byte
    dec    r18
    brne   .Ltx_loop
.Ltx_done:
    ret

```

String data in flash:

```

.Lready_str:
    .byte  'R', 'e', 'a', 'd', 'y', '\r', '\n'
.equ    READY_LEN, 7
.balign 2

```

Calling it:

```

ldi    ZL, lo8(.Lready_str)
ldi    ZH, hi8(.Lready_str)
ldi    r18, READY_LEN
rcall  usart_tx_string

```

The `.byte` directive places raw bytes in the `.text` section at the label address. `LPM Z+` loads each byte from flash and advances `Z`. `READY_LEN` is a constant counting the bytes (7 = 5 chars + CR + LF). The `.balign 2` keeps the next instruction word-aligned after the 7-byte string.

This is a nice place where Chapter 7's flash-reading model becomes immediately useful in application code.

20.6 Polling Receive

Wait until `RXCIF = 1`, then read `RXDATAL`. Reading clears the flag.

Polling receive is conceptually symmetric with polling transmit, but it is much more wasteful in practice because the CPU does not know when the next incoming byte will arrive.

```

/*
 * usart_rx_byte – receive one byte into r16 (polling)
 * Clobbers: r17
 */
usart_rx_byte:
    push   r17
.Lwait_rxcif:
    lds    r17, USART0_STATUS
    sbrs   r17, USART_RXCIF_bp /* bit 7 – skip if RXCIF = 1 (byte ready) */
    rjmp   .Lwait_rxcif
    lds    r16, USART0_RXDATAL /* read byte – clears RXCIF */
    pop    r17
    ret

```

`USART_RXCIF_bp = 7` (bit position). `SBRs r17, 7` skips the loop branch when the RX byte is ready.

20.6.1 Error Checking

Before reading RXDATAH, the error flags in RXDATAH should be checked:

```
.Lwait_rxcif:
    lds    r17, USART0_STATUS
    sbrs   r17, USART_RXCIF_bp
    rjmp   .Lwait_rxcif
    lds    r17, USART0_RXDATAH      /* read high byte first for error flags */
    andi   r17, (USART_FERR_bm | USART_BUFOVF_bm | USART_PERR_bm)
    brne   .Lrx_error              /* jump to error handler if any flag set */
    lds    r16, USART0_RXDATAH      /* no error – read data */
```

RXDATAH contains the error flags (FERR, BUFOVF, PERR) and optionally the 9th data bit. For 8N1, only the error bits matter. RXDATAH must be read before RXDATA — reading RXDATA consumes the byte and discards the error flags.

That read order is another hardware contract: get the status context first, then consume the byte.

For the simple echo example, error checking is omitted for clarity.

20.7 ISR-Driven Receive

Polling TX is fine for transmit — the CPU controls when it sends. But polling RX is wasteful: the CPU spins doing nothing until a byte arrives, unable to perform other work. The RXCIE interrupt solves this: the ISR fires only when a byte is ready, and main code runs uninterrupted between bytes.

This is one of the clearest examples of when interrupts are worth the added complexity.

20.7.1 USART0_RXC Vector

From the ATtiny3217 vector table (Chapter 17):

Byte addr	Vector	Peripheral	Trigger
0x006C	27	USART0_RXC	RXCIF = 1 (byte received)
0x0070	28	USART0_DRE	DREIF = 1 (TX data register empty)
0x0074	29	USART0_TXC	TXCIF = 1 (TX shift register empty)

The USART0_RXC vector is at byte address 0x006C. In the vector table, it is entry 27 (counting RESET as entry 0) — the 28th slot in the table:

```
__vectors:
    jmp    main          /* 0x0000: entry 0  RESET */
    ...                /* entries 1–26 */
    jmp    usart0_rxc_isr /* 0x006C: entry 27 USART0_RXC */
    jmp    isr_default   /* 0x0070: entry 28 USART0_DRE */
    jmp    isr_default   /* 0x0074: entry 29 USART0_TXC */
    jmp    isr_default   /* 0x0078: entry 30 NVMCTRL_EE */
```

20.7.2 RXC ISR

Reading RXDATAH inside the ISR **automatically clears RXCIF**. No explicit flag clear is needed (unlike TCA0 INTFLAGS which requires W1C).

That makes USART receive interrupts a little cleaner than many timer or GPIO interrupt sources.

```

usart0_rxc_isr:
    push    r16
    push    r17
    in      r16, SREG
    push    r16

    lds     r16, USART0_RXDATAL    /* read byte – clears RXCIF automatically */
    rcall   usart_tx_byte          /* echo it back (polling TX) */

    pop     r16
    out     SREG, r16
    pop     r17
    pop     r16
    reti

```

Note r17 is pushed/popped because `usart_tx_byte` clobbers it. The ISR calls a subroutine — this is legal but requires that the callee’s clobbers are also saved in the prologue.

The important rule is local and mechanical: if an ISR calls a helper, the ISR must also preserve any registers the helper uses.

20.7.2.1 Calling Subroutines from ISRs

Calling `RCALL` inside an ISR pushes another 2 bytes onto the stack. The total stack depth for this ISR:

On ISR entry (CPU pushes return addr):	2 bytes
PUSH r16:	1 byte
PUSH r17:	1 byte
PUSH r16 (SREG):	1 byte
RCALL <code>usart_tx_byte</code> (CPU pushes addr):	2 bytes
PUSH r17 (in <code>usart_tx_byte</code>):	1 byte
Total stack depth:	8 bytes

On ATtiny3217, SRAM is 2 KB. Typical use of a few hundred bytes of stack is safe, but deep call trees inside ISRs require careful accounting.

So ISR design is not just about correctness and latency; it is also about bounded stack growth.

20.7.2.2 Blocking TX Inside an ISR

The ISR calls `usart_tx_byte`, which polls `DREIF`. While polling, the ISR is still active, so another `USART0_RXC` interrupt will not run until `RETI`. This is safe here because at 9600 baud the TX wait is at most one byte period (~1.042 ms) and no other time-critical ISR is competing. If more bytes arrive while the ISR is blocked, they wait in the receive path; if software does not drain them in time, `BUFOVF` is reported with the received frame. In higher-throughput designs, use a circular buffer: the `RXC` ISR puts the byte in SRAM, the `DREIE` interrupt sends from the buffer, and the `RXC` ISR returns immediately.

That distinction is important: a simple echo ISR is fine for a teaching example, but blocking inside an ISR does not scale well.

20.8 Initialization Sequence

```

main:
    /* Set Stack */
    ldi    r16, hi8(RAMEND)
    out    SPH, r16
    ldi    r16, lo8(RAMEND)
    out    SPL, r16

    /* Set TXD idle high before TXEN overrides the pin */
    sbi    VPORTB_OUT, TXD_BIT      /* PB2 */
    sbi    VPORTB_DIR, TXD_BIT      /* PB2 */

    /* Baud rate: 9600 at 3.333 MHz → BAUD_REG = 1389 = 0x056D */
    ldi    r16, BAUD_LO             /* 0x6D */
    sts    USART0_BAUDL, r16
    ldi    r16, BAUD_HI             /* 0x05 */
    sts    USART0_BAUDH, r16

    /* CTRLC defaults = 8N1 async – no write needed */

    /* Enable TX + RX */
    ldi    r16, (USART_TXEN_bm | USART_RXEN_bm)
    sts    USART0_CTRLB, r16        /* TXEN + RXEN */
    /* RXCIE lives in CTRLA – separate write */
    ldi    r16, USART_RXCIE_bm
    sts    USART0_CTRLA, r16

    sei

    /* Send startup message */
    ldi    ZL, lo8(.Lready_str)
    ldi    ZH, hi8(.Lready_str)
    ldi    r18, READY_LEN
    rcall  usart_tx_string

.Lidle:
    rjmp   .Lidle

```

Note on CTRLB vs CTRLA: TXEN and RXEN are in CTRLB (0x0806). RXCIE is in CTRLA (0x0805). They are separate registers — two STS writes.

This is exactly the sort of small register-detail bug that can waste a lot of debug time if you assume all enable bits live together.

20.9 Worked Example: Echo Terminal

Full source: `src/usart_echo.S`

The program transmits `Ready\r\n` on startup, then echoes every received byte back to the sender. No main-loop processing — all receive logic is in the ISR.

That makes the data path very easy to reason about: receive a byte, trigger the ISR, send the same byte back.

20.9.1 Data Flow

PC terminal → USB-UART adapter → RXD (PB3) → USART0_RXDATA
 RXCIF = 1 → USART0_RXC ISR fires

```

reads RXDATA (clears RXCIF)
calls usart_tx_byte(r16)
polls DREIF → writes TXDATA
TXD (PB2) → USB-UART adapter → PC terminal (sees same char echoed back)

```

20.9.2 Build and Test

```

# Build
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o usart_echo.o src/usart_echo.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o usart_echo.elf usart_echo.o
avr-objcopy -O ihex usart_echo.elf usart_echo.hex

# Flash
avrdude -c serialupdi -p attiny3217 -P /dev/ttyUSB0 -U flash:w:usart_echo.hex

# Connect serial terminal at 9600 8N1
# Linux/macOS:
screen /dev/ttyUSB0 9600
# or:
minicom -b 9600 -D /dev/ttyUSB0

```

20.9.3 Disassembly Check

```
avr-objdump -d usart_echo.elf
```

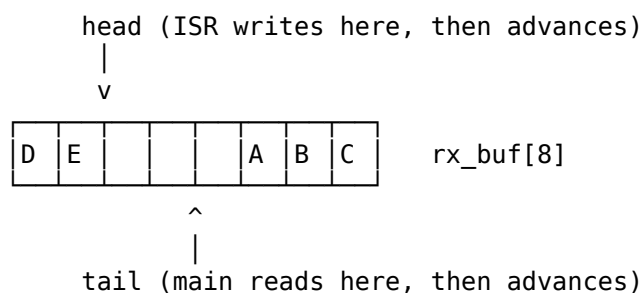
Key things to verify: - Vector at 0x006C (jmp usart0_rxc_isr) — counts 27 entries from RESET. - sts 0x0808, r16 for BAUDL, sts 0x0809, r16 for BAUDH. - sts 0x0806, r16 for CTRLB, sts 0x0805, r16 for CTRLA. - lds r16, 0x0800 for RXDATA inside the ISR. - lds r17, 0x0804 for STATUS in the TX polling loop.

20.10 Circular Buffer for High-Throughput RX

Full source: src/usart_rxbuf.S

The echo ISR copies one byte directly and blocks while it transmits. At 115200 baud with rapid bursts, the CPU may not finish the TX echo before the next byte arrives — BUFOVF sets and bytes are lost. The fix is to make the ISR do the least possible work: drop the received byte into SRAM and return immediately. Main code drains SRAM whenever it has spare cycles. The data structure that sits between them is the circular buffer (also called a ring buffer or FIFO).

A circular buffer is a fixed array plus two indices that chase each other around it. The ISR is the **producer**: it writes at head and advances head. Main code is the **consumer**: it reads at tail and advances tail. When an index walks off the end of the array it wraps back to zero — the array's two ends are logically joined into a ring.



Stored, unread bytes: A B C D E (tail → ... → head, wrapping)
 Free slots: two between head and tail

This decouples “a byte arrived now” from “main code is ready to process it now.” The two sides run at completely independent rates; the buffer absorbs the difference up to its capacity.

20.10.1 The full-versus-empty problem

Two indices give a deceptively simple emptiness test: the buffer is empty when `head == tail`. The trouble is that a *completely full* buffer also brings head back around to equal tail. The same condition means two opposite things, and the producer and consumer cannot tell which.

There are two standard fixes:

Option A – keep a separate count byte

Track an explicit element count. `empty = (count == 0)`, `full = (count == SIZE)`. Costs one more byte, AND both sides must update `count` – so it is no longer true that each variable has a single writer (see “Why no cli/sei” below).

Option B – sacrifice one slot

← used here

Never let the buffer fill completely. Declare it full when advancing head WOULD make it equal tail. A `SIZE`-element array then holds `SIZE-1` bytes.
`empty = (head == tail)`, `full = ((head+1) wrapped == tail)`.
 Costs one slot of capacity, keeps each index single-writer.

This chapter uses Option B. Losing one slot is cheap; keeping each index owned by exactly one side is what makes the buffer safe without disabling interrupts, which is the whole point of the next subsection.

20.10.2 Power-of-two sizing

Wrapping an index means computing $(\text{index} + 1) \bmod \text{SIZE}$. The AVR has no divide instruction, and a general modulo is a costly subroutine. But if `SIZE` is a power of two, the modulo collapses to a single ANDI:

`SIZE = 16 = 0b10000` `mask = SIZE - 1 = 15 = 0x0F`

`index = 15 → +1 = 16 = 0b10000, & 0x0F = 0b00000 = 0` (wraps to 0)
`index = 7 → +1 = 8 = 0b01000, & 0x0F = 0b01000 = 8` (no wrap)

So `andi r17, 0x0F` does the wrap in one cycle. Always size a ring buffer to a power of two unless you have a strong reason not to. This is the same trick the box-filter window uses in the digital-filters chapter.

20.10.3 Why no cli/sei is needed

The ISR (producer) and main code (consumer) touch shared SRAM, which normally demands a critical section. Here it does not, and the reason is worth understanding because it is the single most useful property of this structure.

The argument rests on two facts:

1. Single writer per index.

Only the ISR writes `rx_head`. Only main writes `rx_tail`.
 Each side merely READS the index the other side owns.

2. Byte-sized loads and stores are atomic on AVR.

`rx_head` and `rx_tail` are one byte each. LDS / STS move the whole value in one indivisible bus access – there is no half-updated index for the other

side to catch (unlike a 16-bit value, which takes two instructions and can tear – see the 16-bit register read note in Chapter 11).

Given those, every interleaving is safe. When main reads `rx_head`, it sees either the value before the ISR's update or the value after — never garbage. If it sees the old value, it simply processes one fewer byte and picks it up on the next pass; nothing is lost or corrupted. The producer's view of `rx_tail` is symmetric. This is a *single-producer / single-consumer* (SPSC) lock-free queue, and it is the canonical reason embedded code reaches for a ring buffer.

One ordering detail matters in the producer: store the **data byte first**, then publish the new head. A consumer that observes the advanced head must be guaranteed the byte is already in the slot. Writing head last enforces that.

20.10.4 Producer: the RXC ISR

The buffered ISR is shorter in spirit than the echo version — it never blocks on TX — but it must do one thing the sketch above omitted: check for a full buffer before writing, and **still read RXDATA even when dropping the byte**. Reading RXDATA is what clears RXCIF; skip it and the interrupt re-fires forever.

```
/* SRAM (.bss) layout:
 * rx_buf: .space RXBUF_SIZE ; 16-byte ring (holds up to 15 bytes)
 * rx_head: .space 1 ; write index – ISR owns
 * rx_tail: .space 1 ; read index – main owns
 * rx_ovf: .space 1 ; dropped-byte counter (diagnostics)
 */
.equ RXBUF_SIZE, 16
.equ RXBUF_MASK, RXBUF_SIZE - 1 /* 0x0F */

usart0_rxc_isr:
    push r16
    push r17
    push r18
    push ZL
    push ZH
    in r16, SREG
    push r16

    lds r16, USART0_RXDATA /* ALWAYS read – clears RXCIF */
    lds r17, rx_head /* r17 = head */
    mov r18, r17
    inc r18
    andi r18, RXBUF_MASK /* r18 = (head + 1) wrapped */
    lds ZL, rx_tail /* ZL = tail (scratch, reused below) */
    cp r18, ZL
    breq .Lrx_full /* next == tail → buffer full, drop */

    ldi ZL, lo8(rx_buf)
    ldi ZH, hi8(rx_buf)
    add ZL, r17 /* Z = &rx_buf[head] */
    adc ZH, __zero_reg__
    st Z, r16 /* store byte FIRST */
    sts rx_head, r18 /* then publish new head (commit) */
    rjmp .Lrx_done

.Lrx_full:
```



```

    lds    r16, rx_ovf
    inc    r16
    sts    rx_ovf, r16                /* count the dropped byte */

.Lrx_done:
    pop    r16
    out    SREG, r16
    pop    ZH
    pop    ZL
    pop    r18
    pop    r17
    pop    r16
    reti

```

20.10.5 Consumer: rx_get

rx_get returns the oldest unread byte in r16, or signals “empty.” A return flag is cheaper than a sentinel value (which would collide with a real 0xFF data byte), so this uses the T flag in SREG: T = 1 means the buffer was empty and r16 is meaningless; T = 0 means r16 holds a valid byte.

```

/* rx_get – pop one byte from the ring buffer
 * Returns: T = 1 if empty (r16 undefined)
 *          T = 0 and r16 = byte if one was available
 * Clobbers: r16, r17, Z
 */
rx_get:
    lds    r16, rx_tail
    lds    r17, rx_head
    cp     r16, r17
    brne   .Lrx_have                /* head != tail → a byte is waiting */
    set                                          /* T = 1 → empty */
    ret

.Lrx_have:
    ldi    ZL, lo8(rx_buf)
    ldi    ZH, hi8(rx_buf)
    add    ZL, r16
    adc    ZH, __zero_reg__
    ld     r17, Z                    /* r17 = rx_buf[tail] */
    inc    r16
    andi   r16, RXBUF_MASK
    sts    rx_tail, r16              /* publish new tail (consume) */
    mov    r16, r17                  /* return byte in r16 */
    clt                                          /* T = 0 → valid byte */
    ret

```

20.10.6 The drain loop

Main code now does the work the ISR used to do. The structure is the same one you will use for any buffered peripheral: poll for data, process it, repeat.

```

.Lmain_loop:
    rcall  rx_get
    brts   .Lmain_loop              /* T = 1 → empty, keep polling */
    rcall  usart_tx_byte             /* T = 0 → r16 has a byte; echo it */
    rjmp   .Lmain_loop

```

The behaviour at the terminal is identical to the echo example — every byte comes back — but the RX path no longer blocks. The ISR spends only a handful of cycles per byte, so it keeps up with bursts that would overrun the blocking echo. When main code falls behind, bytes queue in SRAM instead of being lost; only a sustained overrun (more than `RXBUF_SIZE-1` bytes backed up) drops data, and `rx_ovf` records exactly how many. In a real design `process_byte` replaces the echo `rcall` — parsing a command, filling a packet, and so on.

20.10.7 Companion source files

Three sources accompany this section:

- `src/usart_rxbuf.S` The RX program above: producer ISR, `rx_get`, drain loop.
- `src/ringbuf.S` The same ring buffer factored into a reusable, peripheral-agnostic module — `rb_init` / `rb_put` / `rb_get` operating on a control block (head, tail, mask, then the data) passed in `Y`. No USART coupling; link one instance per buffer (RX, TX, a log, ...). This is the version to reach for in real code.
- `src/usart_txbuf.S` The mirror image: a TX ring buffer drained by the `USART0_DRE` interrupt, so transmission is non-blocking too. `tx_put` drops a byte in SRAM and enables `DREIE`; the `DRE` ISR sends one byte per fire and `DISABLES DREIE` when the buffer empties (`DREIF` is level-sensitive, so a left-on `DREIE` would re-fire forever). This is a reference implementation for Exercise 5(a).

The standalone `ringbuf.S` is worth studying for one detail the inline versions hide: because it addresses a caller-supplied control block, the wrap mask lives in the block (`rb_init` records it) rather than as an assemble-time `.equ`. The same routine then serves a 16-byte RX buffer and a 64-byte log with no code change — the power-of-two requirement is the only constraint.

20.11 Summary

USART0 pins (default): TXD = PB2 (output), RXD = PB3 (input)

Baud rate register:

`BAUD_REG = (f_cpu × 64) / (16 × baud)` [async normal mode]

At 3.333 MHz, 9600 bps → `BAUD_REG = 1389 = 0x056D`

Write: `BAUDL` first (low byte), then `BAUDH`.

Registers to configure:

- `USART0_BAUDL` / `BAUDH` — baud rate (write low byte first)
- `USART0_CTRLC` — frame format (default = 8N1, no write needed)
- `USART0_CTRLB` — `TXEN_bm` (0x40) | `RXEN_bm` (0x80)
- `USART0_CTRLA` — `RXCIE_bm` (0x80) for ISR-driven RX

STATUS flags:

`DREIF` (bit 5) — TX data register empty: 1 = ready for new byte

`RXCIF` (bit 7) — RX complete: 1 = byte ready in `RXDATAL`

Both checked with: `LDS Rd, USART0_STATUS` then `SBRs/SBRc`

Polling TX:

wait `DREIF = 1` → `STS USART0_TXDATAL`, `Rr` → `DREIF` auto-clears

Polling RX:

wait `RXCIF = 1` → `LDS Rd, USART0_RXDATA1` → `RXCIF` auto-clears

ISR-driven RX (`USART0_RXC`, vector 27, address `0x006C`):

Enable: `USART_RXCIE_bm` in `USART0_CTRLA`

ISR reads `RXDATA1` → clears `RXCIF` automatically

No manual flag clear needed (unlike `TCA0_INTFLAGS`)

ISR rules: same as always.

Save `r16+SREG` minimum. Save any register clobbered by callees.

Stack depth = 2 (`ret addr`) + saved regs + callee frames.

20.12 Exercises

1. Calculate `BAUD_REG` for 115200 baud at 3.333 MHz. What is the percentage error? If the error exceeds 2%, the link is likely to be unreliable — is 115200 feasible at this clock speed?
2. The polling TX function checks `DREIF` each iteration. At 9600 baud and 3.333 MHz, how many poll iterations occur while waiting for a byte to finish transmitting? (Use the `LDS+SBR5+RJMP` cycle count.)
3. The ISR calls `usart_tx_byte`, which polls `DREIF`. During this polling, could a second `USART0_RXC` interrupt fire? Explain why or why not. What would happen to the second received byte?
4. Modify `usart_echo.S` to echo each byte with its hex representation. Receiving `A` (`0x41`) should transmit the string `41\r\n`. Write the `byte_to_hex` subroutine that converts `r16` (`0x00-0xFF`) into two ASCII hex digits in `r17` (high nibble) and `r16` (low nibble).
5. The circular buffer in “Circular Buffer for High-Throughput RX” decouples RX from main code. Extend it in two ways:
 - (a) Add a symmetric **TX** ring buffer driven by the `USART0_DRE` interrupt: `tx_put` enqueues a byte and enables `DREIE`; the `DRE` ISR sends one byte from the buffer per interrupt and disables `DREIE` when the buffer empties. This makes transmission non-blocking too.
 - (b) Replace the sacrifice-a-slot scheme (Option B) with the count-byte scheme (Option A) so the buffer can hold all 16 bytes. Explain why the count byte now needs a `cli/sei` guard around its update, whereas the single-writer head/tail indices did not.
6. Using `avr-objdump -d usart_echo.elf`, locate the `LDS r17, USART0_STATUS` instruction in `usart_tx_byte`. What is its 32-bit encoding? What address (`0x0804`) appears in which bytes of the instruction?

Next: Bit-Banging a UART

Chapter 21

Bit-Banging a UART

The USART chapter used dedicated hardware to send serial data: you load a byte and the peripheral clocks it out at the configured baud rate. Bit-banging does the same job with no peripheral at all — just an ordinary GPIO pin and a loop that counts CPU cycles. It is slower to design and burns the CPU while it runs, but it works on any pin, on a part whose USART is busy or absent, and it is the purest demonstration of why assembly matters: here, the timing *is* the code.

This chapter builds a software UART transmitter. The receiver is harder and is discussed at the end. The transmitter is the right place to learn the core skill — making a loop take an exact, known number of cycles — because the same technique drives every timing-critical protocol: WS2812 LEDs (where a bit is ~350 ns), 1-Wire, software SPI, and DHT-style sensors.

This chapter covers:

1. The asynchronous serial frame, and why timing is the whole problem
2. Turning a baud rate into a cycle count
3. A delay loop with an exact, known duration
4. Driving a pin in constant time regardless of the data bit
5. Assembling the frame so every bit costs the same
6. The complete `uart_putc`, annotated cycle by cycle
7. Retargeting the baud rate and clock; the limits of bit-banging

21.1 The Serial Frame

Asynchronous serial (the “UART” line format) sends one byte as a frame of bits, each held on the wire for one **bit time**. There is no clock line — the receiver recovers timing from the falling edge of the start bit, then samples each following bit at its expected center. That only works if sender and receiver agree on the bit time to within a couple of percent.

For 8N1 (8 data bits, no parity, 1 stop bit), idle-high, **LSB first**:

idle	start	d0	d1	d2	d3	d4	d5	d6	d7	stop	idle
1	→ 0	→	8 data bits						→	1	→ 1
	↑									↑	
	falling edge									line returns high	
	(receiver syncs here)										

The transmitter’s only job is to put each of those ten bits on the pin and hold it for exactly one bit time. Everything in this chapter is in service of “hold it for exactly one bit time.”

21.2 From Baud Rate to Cycle Count

A baud rate is bits per second. One bit time, measured in CPU cycles, is:

$$\text{BIT_CYCLES} = f_{\text{cpu}} / \text{baud}$$

On the factory-default ATtiny3217 clock ($\text{OSC20M} / 6 = 3.333 \text{ MHz}$) at 9600 baud:

$$\text{BIT_CYCLES} = 3333333 / 9600 = 347.2 \rightarrow 347 \text{ cycles}$$

Rounding to 347 gives an actual rate of $3333333 / 347 = 9606 \text{ baud}$ — an error of **+0.06%**. A UART receiver samples at mid-bit and tolerates roughly $\pm 2\text{-}3\%$ of accumulated error across a frame, so a fraction of a percent is comfortable.

The task is now concrete: **make each bit cell take exactly 347 CPU cycles**. That is an instruction-counting problem, and on AVR every instruction's cycle count is fixed and documented, so it is solvable exactly.

21.3 A Delay of an Exact Length

The workhorse is a counted delay loop:

```
ldi    r19, COUNT
.Ldly:
dec    r19
brne   .Ldly
```

On the ATtiny3217 core, DEC is 1 cycle, BRNE is 2 cycles when the branch is taken and 1 when it falls through. For COUNT iterations the loop body costs:

```
(COUNT-1) iterations taken:  (1 + 2) cycles each
1 final iteration:           (1 + 1) cycles
loop total:                  3*COUNT - 1
plus the LDI:                 3*COUNT cycles
```

So LDI + loop is exactly $3 * \text{COUNT}$ cycles. To hit a target that is not a multiple of three, pad with NOPs (1 cycle each). That is the entire trick: a multiple-of-three delay from the loop, plus up to two NOPs to trim.

21.4 Driving the Pin in Constant Time

A subtle trap: the obvious “if the bit is 1 set the pin, else clear it” branches, and a taken branch costs a different number of cycles than a skipped one — so a 1 bit and a 0 bit would have *different* periods, smearing the timing in a data-dependent way. The fix is an idiom that takes the same time either way:

```
sbrc   r24, 0           ; skip next if frame bit 0 is clear
sbi    VPORTB_OUT, TXD_BIT ; bit = 1: drive high
sbrs   r24, 0           ; skip next if frame bit 0 is set
cbi    VPORTB_OUT, TXD_BIT ; bit = 0: drive low
```

SBI/CBI are 1 cycle on this core; SBRC/SBRS are 1 cycle normally and 2 when they skip a one-word instruction. Trace both values:

```
bit = 1: sbrc (no skip, 1) + sbi (1) + sbrs (skip cbi, 2)      = 4 cycles
bit = 0: sbrc (skip sbi, 2) + sbrs (no skip, 1) + cbi (1)     = 4 cycles
```

Four cycles regardless of the data — the skip simply lands on the other instruction. The pin is driven correctly and the bit cell's length never depends on what is being sent.

(VPORTB_OUT is at I/O address 0x05, in the SBI/CBI-addressable low range, so single-cycle bit writes are available.)

21.5 Assembling the Frame

Rather than special-case the start and stop bits, pack the whole 10-bit frame — plus idle ones — into a 16-bit shift register and emit it one bit at a time with the *same* code. Then start, data, and stop bits all cost identical cycles.

```

bit:   15 14 13 12 11 10 9 | 8 7 6 5 4 3 2 1 | 0
      ----- idle/stop -----      d7 d6 d5 d4 d3 d2 d1 d0      start
      1 1 1 1 1 1 1      ←----- data (LSB first) ----- 0

```

So the low byte is data << 1 (shifting the start 0 into bit 0), and the high byte is 0xFE with the data's top bit dropped into its bit 0:

```

mov    r23, r24                ; keep the original data
lsl    r24                     ; low byte = data<<1, bit0 = 0 (start)
ldi    r25, 0xFE               ; high byte: bits 9..15 = 1 (stop + idle)
bst    r23, 7                  ; T = data bit 7
bld    r25, 0                  ; high byte bit 0 = data bit 7 (frame bit 8)

```

```

nop

dec    r18                /* next bit  (1)          */
brne   .Lbit              /* (2 taken)        */
ret

```

Adding up one bit cell:

```

output idiom (sbrc/sbi/sbrs/cbi)  4
shift (lsr + ror)                  2
ldi + delay loop                  336  (= 3 * 112)
nop + nop                          2
dec + brne (taken)                 3
-----
per bit                            347 cycles -> 9606 baud

```

Every one of the ten bit cells runs this identical path, so each is 347 cycles (the final stop bit is one cycle shorter, since its BRNE falls through — the line stays idle-high afterward, so it does not matter). The byte goes out LSB first, framed by a low start bit and a high stop bit, at 9600 baud.

main sets PB2 idle-high and an output, then sends a short message a byte at a time. PB2 is the pin USART0 uses on the Curiosity Nano, so the bit-banged output reaches the on-board debugger's serial port exactly as the hardware USART's would — a terminal at 9600 8N1 shows Hi!.

21.7 Retargeting and Limits

To change the baud rate or clock, recompute $\text{BIT_CYCLES} = f_{\text{cpu}} / \text{baud}$, then solve $3 \times \text{DELAY_COUNT} + \text{NOP_PAD} + 11 = \text{BIT_CYCLES}$ for the loop count and pad (the constant 11 is the fixed per-bit overhead: output 4, shift 2, nop-slots, loop tail). At very high baud the delay shrinks until the 11-cycle overhead dominates and the rate can no longer be hit precisely; at very low baud COUNT may exceed 255 and the delay needs a 16-bit counter or a nested loop.

Bit-banging trades hardware for cycles, and the trade has sharp edges:

Hardware USART	Bit-banged UART
-----	-----
Clocks bits in the background	Burns the CPU for the whole frame
Fixed pins	Any GPIO pin
Interrupt on each byte	An interrupt mid-frame corrupts timing
Baud from a divider register	Baud from a hand-counted loop

The third row is the one that bites: because the timing lives in the instruction stream, an interrupt firing mid-frame adds its latency to a bit cell and skews the rest of the byte. Bit-banged transmit usually runs with interrupts disabled for the duration of the frame, or budgets a worst-case ISR that is far shorter than the timing margin.

Receive is harder still. The transmitter chooses when each edge happens; the receiver must *find* the start edge (typically a pin-change interrupt or a tight polling loop), wait half a bit time to reach the first bit's center, then sample every bit time after that — all while tolerating the sender's clock being a little off. It is doable with the same cycle-counting tools, but the timing budget is tighter and the edge cases (framing errors, noise on the start edge) are real. For two-way serial on a part with a spare USART, use the hardware.

21.8 Summary

Bit-banging: emit a serial frame from a GPIO pin by counting CPU cycles.

Bit time: `BIT_CYCLES = f_cpu / baud` (3.333 MHz, 9600 -> 347 cycles)

Exact delay: `ldi + (dec; brne) loop = 3*COUNT cycles; trim with NOPs.`

Constant-time pin: `sbrc/sbi/sbrs/cbi` drives the pin in 4 cycles whether the bit is 0 or 1 (the skip lands on the other instruction) – no data-dependent timing.

Frame as a shift register: pack `start(0) + 8 data + stop(1) + idle(1s)` into a 16-bit value; emit bit 0, shift right, `x10` – every bit cell identical.

Caveats: CPU is busy for the whole frame; disable interrupts (or bound ISR latency) so nothing skews the timing; receive is markedly harder.

21.9 Exercises

1. Recompute `DELAY_COUNT` and `NOP_PAD` for 19200 baud at 3.333 MHz. What is the actual baud and the error? How much delay is left once the 11-cycle overhead is removed?
2. The output idiom takes 4 cycles for both bit values. Rewrite the pin drive with an ordinary `BRNE/RJMP` branch and show, cycle by cycle, how a 1 bit and a 0 bit end up with different periods.
3. `uart_putc` runs with interrupts wherever the caller left them. Add the `CLI/SEI` guard around the frame and explain why a single timer ISR firing between two bits would corrupt the byte without it.
4. Change the clock to 20 MHz (no /6 prescaler) and retarget for 115200 baud. Does the 11-cycle overhead still leave room to hit the rate? What is the smallest delay you can express, and what baud does that cap correspond to?
5. Extend main to send a NUL-terminated string instead of a fixed-length one: loop until the loaded byte is zero. Which register holds the byte, and where does the terminator test go?
6. Sketch the receive side: a pin-change interrupt catches the start edge, then the handler delays half a bit time and samples eight bits one bit time apart. Why the *half* bit time first, and what goes wrong if it is omitted?

Next: SPI & TWI (I2C) Overview

Chapter 22

SPI & TWI (I2C) Overview

Serial communication protocols let microcontrollers talk to sensors, memory chips, displays, and other devices using only a handful of wires. This chapter covers the two most common synchronous serial buses used in embedded systems: SPI (Serial Peripheral Interface) and TWI (Two-Wire Interface, Microchip’s name for I²C). Both are present on the ATtiny3217 as hardware peripherals; both are accessed through memory-mapped registers using LDS/STS, exactly as with USART0 in Chapter 20.

This chapter is less about talking to a terminal and more about talking to other chips. It ends with two worked display examples — a character LCD and a graphical OLED — that turn a string in flash into something a human can read.

22.1 SPI Fundamentals

SPI is a full-duplex, synchronous serial bus. Four wires connect one master to one or more slave devices:

Master (ATtiny3217)		Slave (25LC010A EEPROM)
MOSI (PA1)	→	SI (Master Out Slave In)
MISO (PA2)	←	SO (Master In Slave Out)
SCK (PA3)	→	SCK (clock, generated by master)
CS (PA4)	→	C $\bar{S}$ (chip select, active-low)

Full duplex: every clock pulse simultaneously shifts one bit out on MOSI and samples one bit in on MISO. An 8-bit transfer is 8 clock pulses. If the slave has nothing meaningful to return (a pure command), the byte received by the master is discarded (or a dummy byte 0xFF is sent to clock the slave’s response out).

That full-duplex behavior is one of the first things that makes SPI feel different from UART. You are always sending and receiving at the same time, even if one direction is only carrying dummy data.

Chip select: SPI has no built-in addressing. Instead, the master drives CS low to select a particular slave before the transfer begins, then drives CS high after the transfer ends. Each slave needs its own CS line.

So SPI trades protocol simplicity for pin count: the bus itself is simple, but every slave needs a dedicated select signal.

22.1.1 SPI Clock Polarity and Phase (Mode)

The idle state of SCK and the edge on which data is sampled define four modes:

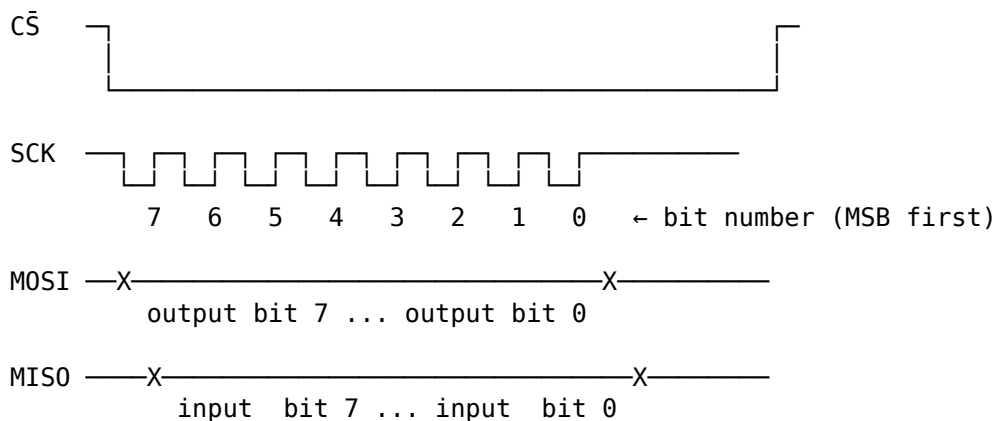
Mode	CPOL	CPHA	SCK idle	Data captured on
0	0	0	low	rising edge ← 25LC010A compatible
1	0	1	low	falling edge
2	1	0	high	falling edge
3	1	1	high	rising edge ← also 25LC010A compatible

The 25LC010A EEPROM used in this chapter works in Mode 0 or Mode 3. We use Mode 0 (CPOL=0, CPHA=0) — SCK idles low, data captured on the rising edge.

In practice, “which SPI mode?” is one of the first things you check in a datasheet when a device does not respond correctly.

On ATtiny3217, the default SPI0 pins are PA1=MOSI, PA2=MISO, PA3=SCK, and PA4=SS. The datasheet also lists alternate SPI0 positions on PORTC: PC2=MOSI, PC1=MISO, PC0=SCK, and PC3=SS, selected through PORTMUX. This chapter keeps the default PA pins and uses PA4 as a software-controlled chip select.

22.1.2 Timing Diagram (Mode 0)



Data is set up before the rising edge and sampled on the rising edge.

22.2 SPI0 Registers on the ATtiny3217

The ATtiny3217 SPI0 peripheral uses five memory-mapped registers:

Register	Address	Purpose
SPI0_CTRLA	0x0820	Master/slave, clock, data order, enable
SPI0_CTRLB	0x0821	Buffer mode, slave-select, SPI mode
SPI0_INTCTRL	0x0822	Interrupt enables
SPI0_INTFLAGS	0x0823	Transfer-complete flag (IF), write-collision (WRCOL)
SPI0_DATA	0x0824	Transmit / receive data register

22.2.1 SPI0_CTRLA — Control A

Bit:	7	6	5	4	3	2	1	0
	—	DORD	MASTER	CLK2X	—	PRESC[1:0]	ENABLE	

Field	Bits	Description
DORD	6	Data order: 0 = MSB first (normal), 1 = LSB first
MASTER	5	1 = master mode, 0 = slave mode
CLK2X	4	Double the clock speed (halves effective prescaler)

Field	Bits	Description
PRESC	2:1	Clock prescaler: 00=/4, 01=/16, 10=/64, 11=/128
ENABLE	0	1 = enable SPI peripheral

At 3.333 MHz with PRESC=DIV4 and CLK2X=0: $SCK = 3.333 \text{ MHz} / 4 \approx \mathbf{833 \text{ kHz}}$. The 25LC010A maximum SCK is 10 MHz, so the default chapter setting is well inside the EEPROM limit. On this CPU clock, SPI0 can run as fast as $f_{\text{CPU}}/2 \approx 1.667 \text{ MHz}$ by setting CLK2X.

22.2.2 SPI0_CTRLB — Control B

Bit: 7 6 5 4 3 2 1 0
 BUFEN BUFWR — — — SSD MODE[1:0]

Field	Bits	Description
BUFEN	7	Enable buffer mode (more complex; not used here)
BUFWR	6	Buffer write mode enable (buffer mode only)
SSD	2	Slave Select Disable — set to 1 when driving CS manually
MODE	1:0	SPI mode: 00=Mode0, 01=Mode1, 10=Mode2, 11=Mode3

SSD must be set when driving CS as a GPIO (as we do in this chapter). If SSD=0 and the hardware SS pin (PA4) is driven low by external logic, the ATtiny3217 will switch itself to slave mode — a common source of confusion.

This is one of those details that feels obscure until it breaks the whole bus.

22.2.3 SPI0_INTFLAGS — Interrupt / Status Flags (non-buffered mode)

Bit: 7 6 5 4 3 2 1 0
 IF WRCOL — — — — — —

Flag	Bit	Set when	Cleared by
IF	7	8-bit transfer complete	Read INTFLAGS, then access DATA
WRCOL6		Write collision (SPI0_DATA written mid-transfer)	Read INTFLAGS, then access DATA

IF at bit position SPI_IF_bp (= 7). Poll with SBRS.

22.2.4 SPI0_DATA — Transmit / Receive

Reading SPI0_DATA after polling IF returns the received byte and completes the flag-clear sequence, because the poll already read SPI0_INTFLAGS. Writing SPI0_DATA (when IF is set or before the first transfer) queues the byte to be shifted out. Both transmit and receive share the same register address — write to transmit, read to receive.

22.3 SPI Initialisation (Master Mode)

Before any transfer, configure the three output pins, leave MISO as input, set SSD, set MODE, then enable as master.

That order is deliberate: get the pins and mode right first, then turn the peripheral on.

```
/* Pin directions: MOSI (PA1), SCK (PA3), CS (PA4) → output */
ldi r16, (1 << 1) | (1 << 3) | (1 << 4)
out VPORTA_DIR, r16

/* CS high (deselect) */
sbi VPORTA_OUT, 4

/* SPI0_CTRLB: SSD=1 (manual CS), MODE=0 (SPI mode 0) */
ldi r16, SPI_SSD_bm
sts SPI0_CTRLB, r16

/* SPI0_CTLRA: master, DIV4 prescaler, MSB first, enable */
ldi r16, (SPI_MASTER_bm | SPI_ENABLE_bm)
sts SPI0_CTLRA, r16
```

SPI_MASTER_bm = (1 << SPI_MASTER_bp) = 0x20. SPI_ENABLE_bm = (1 << SPI_ENABLE_bp) = 0x01. Both are defined in <avr/io.h> for the ATtiny3217.

22.4 Transferring a Byte

The hardware handles the 8-bit shift; the CPU's job is to: 1. Write the byte to SPI0_DATA (starts the transfer). 2. Poll SPI0_INTFLAGS until IF = 1 (transfer done). 3. Read SPI0_DATA (clears IF; returns received byte).

Once you see SPI transfers this way, most device protocols reduce to “send a command byte, maybe send an address byte, maybe send or receive data bytes.”

```
/*
 * spi_transfer – exchange one byte over SPI
 * In: r16 = byte to send
 * Out: r16 = byte received
 * Clobbers: r17
 */
spi_transfer:
    sts SPI0_DATA, r16          /* write tx byte → starts 8-bit transfer */
.Lwait_if:
    lds r17, SPI0_INTFLAGS
    sbrc r17, SPI_IF_bp         /* spin until IF (bit 7) = 1 */
    rjmp .Lwait_if
    lds r16, SPI0_DATA          /* read received byte – clears IF */
    ret
```

Cycle count at 3.333 MHz with SCK = f/4:

Item	Duration
8 SPI bits	8 × 4 CPU cycles = 32 cycles on SCK
Missed poll	6 cycles (LDS 3 + SBRS 1 + R JMP 2)
Final poll	5 cycles (LDS 3 + taken SBRS skip 2)
DATA read	3 cycles (LDS r16, SPI0_DATA)

The exact number of poll iterations depends on where the CPU is inside the loop when IF becomes set. With this code and f/4 SCK, DATA is typically read about 40 CPU cycles after the transfer starts.

22.5 The 25LC010A SPI EEPROM

The 25LC010A is a 1-Kbit (128×8 -bit) SPI EEPROM. It communicates using single-byte commands followed by optional address and data bytes.

22.5.1 Key Characteristics

Parameter	Value
Capacity	128 bytes (1 Kbit)
Address width	7 bits (0x00 – 0x7F)
Page size	16 bytes (for page write)
SPI modes	Mode 0 and Mode 3
Max SCK	10 MHz
Write cycle time	5 ms maximum
Supply voltage	2.5 V – 5.5 V

22.5.2 Command Set

Opcode	Hex	Bytes after opcode	Description
READ	0x03	addr, [data...]	Read one or more bytes
WRITE	0x02	addr, data[...]	Write one byte (or page)
WREN	0x06	–	Set Write Enable Latch (WEL)
WRDI	0x04	–	Clear Write Enable Latch
RDSR	0x05	[dummy]	Read Status Register
WRSR	0x01	data	Write Status Register

Write Enable Latch (WEL): the EEPROM resets WEL after every write. You must send WREN (its own CS-framed transaction) before every WRITE command.

That makes accidental writes less likely, but it also means your software has to treat “enable writing” as part of every write sequence, not as a one-time setup step.

22.5.3 Status Register (returned by RDSR)

Bit:	7	6	5	4	3	2	1	0
	–	–	–	–	BP1	BP0	WEL	WIP

Flag	Bit	Meaning
WIP	0	Write In Progress — 1 while write cycle is running
WEL	1	Write Enable Latch — 1 after WREN, 0 after write completes
BP	3:2	Block-protect bits (control write-protect regions)

Poll WIP = 0 before issuing another WRITE command.

In other words, the WRITE command only starts the EEPROM’s internal work. The chip still needs time afterwards to commit the byte into nonvolatile storage.

22.5.4 CS Framing Rules

Each command must be bracketed by CS low/high transitions. Raising CS before a complete transfer aborts the current command. The typical patterns:

This is the other half of SPI correctness besides mode. Many slave devices care not just about the bytes, but about the exact CS framing around them.

```
Write Enable:          CS↓ → WREN(0x06) → CS↑
Write byte:           CS↓ → WRITE(0x02) → ADDR → DATA → CS↑
Read status (poll WIP): CS↓ → RDSR(0x05) → DUMMY → CS↑
                      read status byte from DUMMY transfer
```

22.6 Example: Write Byte to SPI EEPROM

The full annotated source is at `ch13_spi_twi/src/spi_eeprom.S`. Here we walk through the key subroutines.

22.6.1 eeprom_wren — Send Write Enable

```
eeprom_wren:
    rcall cs_assert          /* CS low */
    ldi    r16, EEPROM_WREN  /* 0x06 */
    rcall spi_transfer       /* shift out opcode (received byte discarded) */
    rcall cs_deassert        /* CS high — WEL is now set */
    ret
```

22.6.2 eeprom_write_byte — Write One Byte

```
/*
 * r24 = 8-bit address, r22 = data byte
 */
eeprom_write_byte:
    rcall eeprom_wren        /* WEL must be set before each write */

    rcall cs_assert
    ldi    r16, EEPROM_WRITE /* 0x02 */
    rcall spi_transfer       /* opcode */
    mov    r16, r24
    rcall spi_transfer       /* address */
    mov    r16, r22
    rcall spi_transfer       /* data */
    rcall cs_deassert        /* CS↑ latches the write; 5 ms cycle starts */
    ret
```

22.6.3 eeprom_wait_wip — Poll WIP Until Write Done

```
eeprom_wait_wip:
.lwip_poll:
    rcall cs_assert
    ldi    r16, EEPROM_RDSR /* 0x05 */
    rcall spi_transfer       /* send RDSR opcode */
    ldi    r16, 0xFF         /* dummy: clocks out the status register */
    rcall spi_transfer       /* r16 = status byte after return */
    rcall cs_deassert
```

```

sbrc  r16, WIP_BIT          /* skip rjmp if WIP = 0 (write done) */
rjmp  .Lwip_poll
ret

```

After `spi_transfer` with the dummy byte, `r16` holds the status register. `SBRC r16, 0` skips the `RJMP` when bit 0 (WIP) is clear — loop exits.

This is a common SPI pattern: send a command, then send a dummy byte only to clock the response back from the slave.

22.6.4 main — Putting It Together

```

main:
    /* SP, pins, SPI0 init (see full source) */

    ldi  r24, 0x00          /* EEPROM address */
    ldi  r22, 0xA5          /* data byte */
    rcall eeprom_write_byte /* WREN + WRITE command sequence */
    rcall eeprom_wait_wip   /* wait up to 5 ms for write cycle */

.Lhalt:
    rjmp .Lhalt

```

22.6.5 Build and Inspect

```

# Assemble and link with the GCC driver so <avr/io.h> is preprocessed
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o spi_eeprom.o src/spi_eeprom.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o spi_eeprom.elf spi_eeprom.o

# Disassemble – verify instruction encoding
avr-objdump -d spi_eeprom.elf

# Generate flashable HEX
avr-objcopy -O ihex spi_eeprom.elf spi_eeprom.hex

```

Expected `objdump` output for `spi_transfer`:

```

0000007e <spi_transfer>:
 7e: 00 93 24 08  sts 0x0824, r16    ; SPI0_DATA ← tx byte
 82: 10 91 23 08  lds r17, 0x0823    ; read SPI0_INTFLAGS
 86: 17 ff                sbrs r17, 7        ; wait for IF (bit 7)
 88: fc cf                rjmp 0x82      ; loop
 8a: 00 91 24 08  lds r16, 0x0824    ; read received byte, clears IF
 8e: 08 95                ret

```

(Exact symbol labels and addresses vary; with `-nostartfiles`, `objdump` may also show `__ctors_end` at the same address as the first text symbol. The important part is the `LDS/STS` encoding: `24 08` encodes address `0x0824` in little-endian.)

That is another good reminder that on AVR, peripheral access cost depends on where the register lives.

22.7 TWI (I²C) Fundamentals

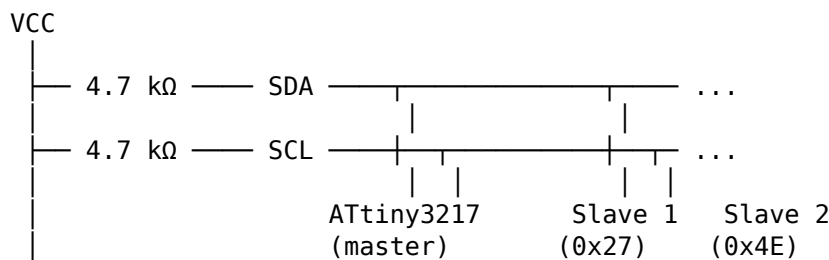
TWI uses two wires: SDA (data) and SCL (clock). Both lines are open-drain with pull-up resistors — any device can pull a line low, but no device drives it high (the pull-up does

that). This allows multi-master arbitration and clock stretching by slaves.

On ATtiny3217, the default TWI0 pins are PB1=SDA and PB0=SCL. Alternate TWI0 signals are available on PA1=SDA and PA2=SCL through PORTMUX. The bus still needs pull-up resistors in either location.

This is a very different bus philosophy from SPI. TWI uses fewer wires and built-in addressing, but the protocol is more stateful and the electrical behavior is less direct.

22.7.1 Bus Topology

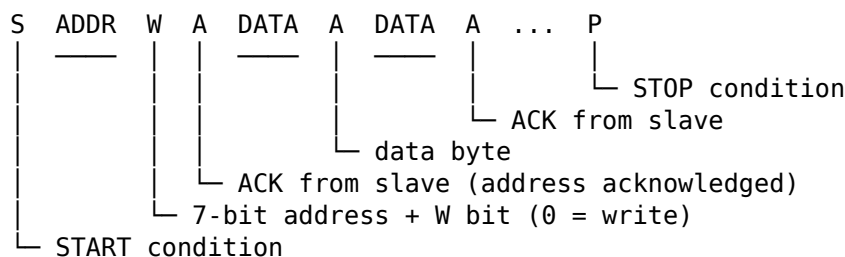


Every device has a 7-bit address (0x00–0x7F). The master sends the address at the start of each transaction; only the matching slave responds.

So TWI solves the “which slave am I talking to?” problem in-band, with an address byte, instead of with separate CS wires.

22.7.2 Transaction Structure

A master write transaction (master sends data to slave) looks like:



- **START (S):** SDA falls while SCL is high.
- **Address + R/W:** 7-bit address followed by a Read(1) or Write(0) bit.
- **ACK (A):** slave pulls SDA low during the 9th SCK pulse to acknowledge.
- **STOP (P):** SDA rises while SCL is high.

22.7.3 I²C Speeds

Mode	Max SCL	Common use
Standard	100 kHz	Most sensors, EEPROMs
Fast	400 kHz	High-speed peripherals
Fast-Plus	1 MHz	ATtiny3217 supports this

22.8 TWI0 Master Mode Registers on ATtiny3217

The ATtiny3217’s TWI0 peripheral has separate register sets for master and slave operation. We use master-mode registers only.

Register	Address	Purpose
TWI0_CTRLA	0x0810	SDA setup/hold time, fast-mode-plus enable
TWI0_DBGCTRL	0x0812	Debug run control
TWI0_MCTRLA	0x0813	Master enable, interrupts, smart mode
TWI0_MCTRLB	0x0814	Flush, ACK action, bus command (STOP etc.)
TWI0_MSTATUS	0x0815	WIF, RIF, CLKHOLD, RXACK, ARBLOST, BUSERR, BUSSTATE
TWI0_MBAUD	0x0816	SCL baud rate register
TWI0_MADDR	0x0817	Send START + address (write to initiate)
TWI0_MDATA	0x0818	Master data (read = receive, write = transmit)

22.8.1 TWI0_MCTRLA — Master Control A

Bit:	7	6	5	4	3	2	1	0
	RIEN	WIEN	—	QCEN	TIMEOUT[1:0]		SMEN	ENABLE

Field	Description
RIEN	Read Interrupt Enable (fires on RIF)
WIEN	Write Interrupt Enable (fires on WIF)
QCEN	Quick Command Enable (ACK sent automatically)
TIMEOUT	Bus timeout (00 = disabled)
SMEN	Smart Mode Enable (ACK sent automatically on read)
ENABLE	Enable TWI master

For polling mode (no interrupts): set only TWI_ENABLE_bm.

22.8.2 TWI0_MSTATUS — Master Status

Bit:	7	6	5	4	3	2	1	0
	RIF	WIF	CLKHOLD	RXACK	ARBLOST	BUSERR	BUSSTATE[1:0]	

Flag	Description
RIF	Read Interrupt Flag — byte received and ready in MDATA
WIF	Write Interrupt Flag — address/data write accepted, ACK received
CLKHOLD	Clock hold — SCL is being stretched; write MDATA/MCTRLB to release
RXACK	Received ACK: 0 = ACK (slave acknowledged), 1 = NACK
ARBLOST	Arbitration lost (multi-master: another master won the bus)
BUSERR	Bus error (illegal START/STOP condition detected)
BUSSTATE	00=unknown, 01=idle, 10=owner, 11=busy

WIF is the flag you poll most: it sets after the master successfully sends an address byte or a data byte. Check RXACK immediately after — if RXACK=1 the slave sent NACK (not present or busy).

That RXACK polarity is easy to misread because success is encoded as zero.

22.8.3 TWI0_MBAUD — Baud Rate Register

$$f_{\text{SCL}} = f_{\text{CPU}} / (10 + 2 \times \text{BAUD} + f_{\text{CPU}} \times T_{\text{rise}})$$

Ignoring T_{rise} (assume short wires, small capacitance):

$$\text{BAUD} = (f_{\text{CPU}} / f_{\text{SCL}} - 10) / 2$$

At 3.333 MHz for 100 kHz:

$$\text{BAUD} = (3333333/100000 - 10) / 2 = 11.67$$

Use **12** if you want to stay just below 100 kHz when ignoring rise time: $3333333 / (10 + 2 \times 12) \approx 98.0 \text{ kHz}$.

```
.equ TWI_BAUD_100K, 12          /* 3.333 MHz → ~98 kHz SCL, ignoring rise time */
```

22.8.4 TWI0_MCTRLB — Master Control B (Bus Commands)

Write to TWI0_MCTRLB to issue a bus command after each transaction phase. The MCMD field (bits 1:0) controls the state machine, while ACKACT (bit 2) selects whether the next acknowledge action sends ACK or NACK.

Bit:	7	6	5	4	3	2	1	0
	—	—	—	—	FLUSH	ACKACT	MCMD[1:0]	

ACKACT = 0 sends ACK; ACKACT = 1 sends NACK. ACKACT and MCMD can be written in the same STS. The ACK action matters for host reads; during a host write, the peripheral waits for the next data byte or command.

MCMD value	Symbolic name	Effect
0b00	TWI_MCMD_NOACT_gc	No action
0b01	TWI_MCMD_REPSTART_gc	ACK/NACK action, then repeated START
0b10	TWI_MCMD_RECVTRANS_gc	ACK/NACK action, then receive/transmit
0b11	TWI_MCMD_STOP_gc	ACK/NACK action, then STOP

To send STOP after a write: sts TWI0_MCTRLB, TWI_MCMD_STOP_gc

22.9 TWI Initialisation

```
/* TWI0 pins (default mux): SDA = PB1, SCL = PB0
 * Both open-drain – no DDR configuration needed.
 * The TWI peripheral takes control of the pins when ENABLE = 1.
 * External 4.7 kΩ pull-ups to VCC required. */

/* Set baud rate: 100 kHz at 3.333 MHz (must be written while disabled) */
ldi r16, TWI_BAUD_100K          /* 12 */
sts TWI0_MBAUD, r16

/* Enable master */
ldi r16, TWI_ENABLE_bm          /* 0x01 */
sts TWI0_MCTRLA, r16

/* Force bus state to IDLE – must come AFTER enabling the master */
ldi r16, TWI_BUSSTATE_IDLE_gc   /* 0x01 */
sts TWI0_MSTATUS, r16
```

Writing TWI_BUSSTATE_IDLE_gc (= 0x01) to TWI0_MSTATUS forces the bus state machine to IDLE. Order matters: enabling the master sets the bus state to UNKNOWN (datasheet §26.3.2.2.2), and in the UNKNOWN state the peripheral will not initiate transactions. So the IDLE write must come *after* the TWI0_MCTRLA enable — if you force IDLE first, the enable immediately overwrites it back to UNKNOWN and the bus is stuck.

This is one of those TWI-specific startup details that feels odd until you remember the peripheral is tracking bus state, not just shifting bytes.

22.10 Master Write Transaction (Polling)

A complete write transaction — send one data byte to a slave device:

Step	Action
1.	Write (slave_addr << 1 0) to TWI0_MADDR → generates START + address
2.	Wait for WIF set
3.	Check RXACK == 0 (slave ACK); if RXACK == 1 → NACK, abort
4.	Write data byte to TWI0_MDATA
5.	Wait for WIF set
6.	Check RXACK == 0
7.	Write TWI_MCMD_STOP_gc to TWI0_MCTRLB → generates STOP

That list is longer than the SPI case because TWI has more protocol structure: START, address, ACK/NACK, data, ACK/NACK, STOP.

In assembly (using r24 = slave address, r22 = data byte):

```
/*
 * twi_write_byte – write one byte to TWI slave
 * r24 = 7-bit slave address (unshifted, 0x00–0x7F)
 * r22 = data byte
 * Returns: r24 = 0 on success, 1 on NACK
 * Clobbers: r16, r17
 */
twi_write_byte:
    /* Send START + address + W (R/W bit = 0) */
    lsl    r24                /* shift left: addr occupies bits[7:1], bit0=0 */
    sts    TWI0_MADDR, r24    /* write triggers START + 9-bit address phase */

    /* Wait for WIF */
.Lwif_addr:
    lds    r17, TWI0_MSTATUS
    sbrs   r17, TWI_WIF_bp    /* WIF = bit 6 */
    rjmp   .Lwif_addr

    /* Check RXACK */
    sbrc   r17, TWI_RXACK_bp /* RXACK = bit 4; skip ret if ACK (bit = 0) */
    rjmp   .Ltwi_nack

    /* Send data byte */
    sts    TWI0_MDATA, r22

    /* Wait for WIF again */
.Lwif_data:
    lds    r17, TWI0_MSTATUS
    sbrs   r17, TWI_WIF_bp
    rjmp   .Lwif_data

    /* Check RXACK for data */
    sbrc   r17, TWI_RXACK_bp
    rjmp   .Ltwi_nack

    /* Send STOP */
    ldi    r16, TWI_MCMD_STOP_gc /* 0x03 */
    sts    TWI0_MCTRLB, r16

    ldi    r24, 0              /* return 0 = success */
    ret
```

```
.Ltwi_nack:
    ldi    r16, TWI_MCMD_STOP_gc
    sts    TWI0_MCTRLB, r16 /* release bus even on failure */
    ldi    r24, 1          /* return 1 = NACK error */
    ret
```

22.10.1 SBRC vs SBRS for RXACK

RXACK = 0 means ACK received (success). The condition for *success* is bit = 0, so we use SBRC (Skip if Bit in Register is Cleared) to skip the NACK handler: if the bit is cleared (ACK), skip over `rjmp .Ltwi_nack`.

This is the opposite of USART's SBRS on DREIF — read the bit polarity carefully for each flag.

That is a general embedded rule: never assume status bits use the same success polarity across peripherals.

22.11 Example: Write to PCF8574 I²C I/O Expander

The PCF8574 is a popular 8-bit I²C I/O expander. Its 7-bit address is 0b0100xxx where xxx is set by hardware pins A2:A0. With A2:A0 = 0b000, the address is 0x20.

To write a byte to the PCF8574's output latch, a single data byte is sent after the address — no register address needed.

That simplicity is why expanders like the PCF8574 are common first TWI devices: the bus protocol is still there, but the device-side command format is minimal.

```
START | 0x20 W | ACK | 0b10101010 | ACK | STOP
      (slave)          (slave)
```

This drives outputs P7..P0 = 1,0,1,0,1,0,1,0 (alternating).

```
.equ PCF8574_ADDR, 0x20

main:
    /* TWI0 init (as shown in 13.9) */
    ldi    r16, TWI_BAUD_100K
    sts    TWI0_MBAUD, r16
    ldi    r16, TWI_ENABLE_bm
    sts    TWI0_MCTRLA, r16
    ldi    r16, TWI_BUSSTATE_IDLE_gc /* force idle AFTER enable */
    sts    TWI0_MSTATUS, r16

    /* Write 0xAA to PCF8574 */
    ldi    r24, PCF8574_ADDR /* 0x20 */
    ldi    r22, 0b10101010 /* output data */
    rcall twi_write_byte

.Lhalt:
    rjmp .Lhalt
```

22.11.1 Build

```
avr-gcc -mmcu=attiny3217 -nostartfiles -o pcf8574.elf pcf8574.S
avr-objcopy -O ihex pcf8574.elf pcf8574.hex
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -U flash:w:pcf8574.hex
```

22.12 Rendering Strings from Program Memory

Talking to a display is mostly about sending bytes, but the bytes you send are usually *text*. A string like "HELLO" has to live somewhere, and on a microcontroller the natural place is flash — it is constant and there is far more of it than SRAM.

On the ATtiny3217 (an AVRxt core) flash is mapped into the data address space at `MAPPED_PROGMEM_START` (= 0x8000), so a constant string can be read with an ordinary LD instead of LPM. This is the same mapped-flash technique introduced in the syntax chapter; here we put it to work feeding a display.

A NUL-terminated string is declared with `.asciz`:

```
.section .rodata
msg: .asciz "HELLO WORLD"      /* bytes 'H','E',..., 'D', 0x00 */
```

On this device the linker places `.rodata` in the flash-mapped window, so the label `msg` already carries its data-space address. Load it straight into Z with `lo8/hi8` — no manual + 0x8000 is needed — and walk the string with post-increment LD until the terminating zero:

```
/*
 * print_str – send a NUL-terminated flash string to a device
 * Z = address of the string (in the flash-mapped data window)
 * Calls putc (r17 = byte) once per character.
 */
print_str:
.Lps:
    ld    r17, Z+           /* fetch next byte, advance Z */
    tst   r17               /* terminator? */
    breq  .Lps_done
    rcall putc              /* device-specific: send one character */
    rjmp  .Lps
.Lps_done:
    ret
```

That `print_str` loop is the whole idea behind “string rendering”: iterate the bytes, hand each one to a `putc` that knows how to talk to the specific display. Everything below is just two different `putc` implementations — one for a character LCD, one for a graphical OLED.

The caller loads Z and calls it:

```
ldi    ZL, lo8(msg)
ldi    ZH, hi8(msg)
rcall  print_str
```

The same loop drives a UART, an EEPROM, an LCD, or an OLED. Only `putc` changes.

22.13 A Character LCD over I²C (HD44780 + PCF8574)

The HD44780 is the controller inside almost every 16x2 and 20x4 character LCD. It has its own character-generator ROM, so you send it ASCII bytes and it draws the glyphs — no font table needed on the AVR side. The catch is that a bare HD44780 needs 6–10 GPIO

pins. The popular “I²C LCD backpack” solves that by putting a **PCF8574** expander (the same chip from the previous section) in front of it, reducing the wiring to the two TWI lines.

So this example reuses `twi_write_byte` unchanged — every byte we push to the PCF8574 becomes the LCD’s parallel port pins.

22.13.1 Backpack Pin Mapping

The backpack wires the expander’s 8 outputs to the HD44780 in **4-bit mode**:

PCF8574 bit HD44780 signal

P0	RS (0 = command, 1 = data)
P1	RW (tied low – we only write)
P2	E (enable strobe; latches on the falling edge)
P3	Backlight (1 = on)
P4..P7	D4..D7 (the high data nibble)

Two consequences fall out of this map:

- The data pins are P4..P7, so a 4-bit nibble must be shifted into the **high** half of the byte before sending.
- There is no data bus to read or write directly — every change to a pin is a full PCF8574 write transaction over I²C.

22.13.2 Clocking a Nibble

In 4-bit mode the HD44780 takes each byte as two nibbles, high nibble first. A nibble is captured on the **falling edge of E**, so the sequence per nibble is: present the nibble with E high, then present the same nibble with E low.

PCF8574 write 1: [nibble<<4 | BL | RS | E=1] ← data set up, E high
 PCF8574 write 2: [nibble<<4 | BL | RS | E=0] ← E falls, LCD latches nibble

Because each of those two writes is a complete I²C transaction taking hundreds of microseconds at 100 kHz, the E pulse is enormously wider than the controller’s ~450 ns minimum. We get correct timing for free, without a single delay instruction in the strobe.

```
.equ LCD_ADDR, 0x27      /* common backpack 7-bit address */
.equ LCD_BL, 0x08        /* P3 backlight */
.equ LCD_EN, 0x04        /* P2 enable */
.equ LCD_RS, 0x01        /* P0 register select */

/*
 * lcd_write4 – push one nibble and strobe E
 * r18 = nibble (low 4 bits), r19 = RS bit (0 or LCD_RS)
 */
lcd_write4:
    mov    r20, r18
    swap   r20            /* nibble -> D4..D7 (high half) */
    andi   r20, 0xF0
    ori    r20, LCD_BL    /* keep backlight on */
    or     r20, r19       /* mix in RS */

    mov    r22, r20
    ori    r22, LCD_EN    /* E high */
    ldi    r24, LCD_ADDR
    rcall  twi_write_byte

    mov    r22, r20       /* E low – falling edge latches the nibble */
```

```
ldi    r24, LCD_ADDR
rcall  twi_write_byte
ret
```

A full byte is just two `lcd_write4` calls, and the RS bit chooses whether the byte is a command (cursor moves, clear, mode) or a character to display:

```
/* lcd_send – r17 = byte, r19 = RS bit */
lcd_send:
    mov    r16, r17          /* save byte across calls */
    mov    r18, r16
    swap   r18
    andi   r18, 0x0F         /* high nibble */
    rcall  lcd_write4
    mov    r18, r16
    andi   r18, 0x0F         /* low nibble */
    rcall  lcd_write4
    ret

lcd_cmd:                /* r17 = command */
    ldi    r19, 0
    rjmp   lcd_send
lcd_data:                /* r17 = character */
    ldi    r19, LCD_RS
    rjmp   lcd_send
```

Now `putc` for this display is simply `lcd_data`. Dropping it into the `print_str` pattern from the previous section gives `lcd_puts`:

```
/* lcd_puts – Z = flash string; prints it at the cursor */
lcd_puts:
.Lps:
    ld     r17, Z+
    tst    r17
    breq   .Lps_done
    rcall  lcd_data
    rjmp   .Lps
.Lps_done:
    ret
```

22.13.3 The Awkward Power-Up Sequence

The one genuinely fiddly part of an HD44780 is initialisation. The controller boots expecting 8-bit mode, and you must walk it down into 4-bit mode with a fixed sequence of single nibbles and minimum delays before normal commands work:

delay > 40 ms	(power-on)
write nibble 0x3	delay > 4.1 ms
write nibble 0x3	delay > 100 µs
write nibble 0x3	delay > 100 µs
write nibble 0x2	→ now in 4-bit mode
cmd 0x28	function set: 4-bit, 2 lines, 5x8 font
cmd 0x0C	display on, cursor off
cmd 0x06	entry mode: auto-increment
cmd 0x01	clear display (needs ~1.52 ms after)

This sequence is copied almost verbatim from the HD44780 datasheet, and it is the reason a display that “should just work” sometimes shows a row of black boxes: if the wake-up nibbles or their delays are wrong, the controller never leaves 8-bit mode.

The full annotated source — init, the nibble strobe, and a `delay_ms` busy-wait — is at `ch13_spi_twi/src/lcd_hd44780.S`. Build it the same way as the I²C example:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o lcd_hd44780.o src/lcd_hd44780.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o lcd_hd44780.elf lcd_hd44780.o
avr-objcopy -O ihex lcd_hd44780.elf lcd_hd44780.hex
```

The character LCD is the easy case: the display owns the font, so “rendering” is nothing more than streaming ASCII bytes. The OLED below is where the AVR has to draw the letters itself.

22.14 A Graphical OLED over I²C (SSD1306)

A 128x64 SSD1306 OLED has no character generator. It is a raw pixel buffer: 1024 bytes, organised as 8 horizontal **pages** of 128 columns, where each byte is a vertical strip of 8 pixels (LSB on top). To show text, we supply the font and convert each character into the column bytes that draw it.

That is the real “string rendering” example: flash holds a font table, and the code turns bytes of text into bytes of pixels.

22.14.1 Command vs Data Framing

Every SSD1306 transaction starts with one control byte that classifies the rest of the transaction:

```
control 0x00    the following bytes are COMMANDS
control 0x40    the following bytes are DISPLAY DATA (pixels)
```

In horizontal addressing mode (selected during init) the column pointer auto-increments after every data byte, so one I²C transaction can stream an entire line — or the whole screen. The combined `twi_write_byte` used for the PCF8574 sends exactly one data byte per START/STOP, which is wasteful here. Instead we split the transaction into primitives and keep it open across many bytes:

```
/* twi_start_w - START + (r24<<1)|W, wait WIF */
twi_start_w:
    lsl    r24
    sts    TWI0_MADDR, r24
.Lsw:
    lds    r17, TWI0_MSTATUS
    sbrs   r17, TWI_WIF_bp
    rjmp   .Lsw
    ret

/* twi_write - push data byte r22, wait WIF */
twi_write:
    sts    TWI0_MDATA, r22
.Lw:
    lds    r17, TWI0_MSTATUS
    sbrs   r17, TWI_WIF_bp
    rjmp   .Lw
    ret

/* twi_stop - STOP */
twi_stop:
    ldi    r17, TWI_MCMD_STOP_gc    /* 0x03 */
```



```

sts    TWI0_MCTRLB, r17
ret

```

Opening a data stream is then “start, send the 0x40 control byte, and leave the transaction open”:

```

oled_data_start:
    ldi    r24, 0x3C          /* SSD1306 address          */
    rcall twi_start_w
    ldi    r22, 0x40          /* control: data stream      */
    rjmp   twi_write

```

This decomposition is worth the trouble: the same three primitives also build the command stream that initialises the panel and the bulk write that clears it, so nothing here is single-use.

22.14.2 The Font Table in Flash

A 5x8 font stores 5 column bytes per glyph (a 6th blank column is added at print time as letter spacing). Laid out contiguously and indexed by char - 0x20, a glyph lives at font + (char - 0x20) * 5:

```

.section .rodata
font:
/* ... 0x20..0x47 ... */
.byte 0x7F,0x08,0x08,0x08,0x7F /* 'H' (0x48)          */
.byte 0x7F,0x49,0x49,0x49,0x41 /* 'E' (0x45 – shown out of order) */
.byte 0x7F,0x40,0x40,0x40,0x40 /* 'L' (0x4C)          */
.byte 0x3E,0x41,0x41,0x41,0x3E /* '0' (0x4F)          */
/* ... rest of the printable ASCII range ... */

```

Read a glyph by computing its byte offset and pointing Z at it in the flash-mapped window. The *5 is done with a hardware multiply, then the five columns are streamed straight into an open data transaction:

```

/*
 * oled_putc – render one glyph (5 font columns + 1 blank spacer)
 * r17 = character
 */
oled_putc:
    subi   r17, 0x20          /* index = char - 0x20          */
    ldi    r18, 5
    mul    r17, r18           /* r1:r0 = index * 5            */
    ldi    ZL, lo8(font)
    ldi    ZH, hi8(font)
    add    ZL, r0
    adc    ZH, r1
    clr    r1                  /* restore the r1 == 0 convention after MUL */

    rcall  oled_data_start
    ldi    r20, 5

.Lcol:
    ld     r22, Z+             /* next column byte from flash  */
    rcall  twi_write           /* stream it as pixel data      */
    dec    r20
    brne   .Lcol
    clr    r22                 /* blank column = inter-character gap */
    rcall  twi_write
    rjmp   twi_stop

```

Notice `clr r1` after the `MUL`: the AVR ABI assumes `r1` is always zero, and `MUL` is the one common instruction that disturbs it. Forgetting this is a classic bug that surfaces far away from the multiply.

With `oled_putc` as the `putc`, `oled_puts` is the same `print_str` loop once more — the string-rendering shape never changes, only the per-character worker. The full source, including the panel init command list, a screen clear, and a font, is at `src/oled/oled_ssd1306.S`:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o oled_ssd1306.o src/oled/oled_ssd1306.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o oled_ssd1306.elf oled_ssd1306.o
avr-objcopy -O ihex oled_ssd1306.elf oled_ssd1306.hex
```

22.14.3 LCD vs OLED: Who Owns the Font?

	HD44780 LCD	SSD1306 OLED
Font lives in	the display's ROM	a flash table on the AVR
You send	ASCII bytes	pixel columns per glyph
Per character	1 byte	5–6 bytes
Bus traffic	low	higher (font expansion)
Flexibility	fixed character set	any glyph, graphics, sizes

That trade is the whole story: the LCD is cheaper to drive because it hides the font; the OLED costs you the font and the bus bandwidth to expand it, and in return draws anything you can describe as pixels.

22.15 SPI vs TWI Comparison

Feature	SPI	TWI (I ² C)
Wires	4 (MOSI, MISO, SCK, CS×n)	2 (SDA, SCL) + pull-ups
Multi-slave	One CS pin per slave	Addressed (7-bit)
Duplex	Full duplex	Half duplex
Max speed here	833 kHz default, 1.667 MHz max	100 kHz / 400 kHz / 1 MHz
Protocol overhead	None (raw shift register)	START/STOP/ACK overhead
Hardware	Shift register + IF flag	State machine (WIF/RIF)
Complexity	Low	Medium
Typical devices	EEPROM, flash, ADC, display	Sensor ICs, I/O expanders

The practical split is usually this: use SPI when you want speed and simple byte streaming, and use TWI when you want several slower peripherals to share the same two wires.

22.16 Summary

SPI0 setup (master, mode 0, f/4, manual CS):

```
SPI0_CTRLB ← SPI_SSD_bm ; disable hardware SS
SPI0_CTRLA ← SPI_MASTER_bm | SPI_ENABLE_bm
```

SPI transfer:

```
STS SPI0_DATA, r16 ; start transfer
poll SPI_IF_bp in SPI0_INTFLAGS
LDS r16, SPI0_DATA ; get received byte, clears IF
```

25LC010A write sequence:

```
1. CS↓, send WREN (0x06), CS↑
```

2. CS↓, send WRITE (0x02), addr, data, CS↑
3. CS↓, send RDSR (0x05), dummy, CS↑ – check WIP (bit 0)
4. Repeat step 3 until WIP = 0 (max 5 ms)

TWI0 setup (master, 100 kHz):

```
TWI0_MBAUD    ← 12                ; ~98 kHz at 3.333 MHz (while disabled)
TWI0_MCTRLA   ← TWI_ENABLE_bm     ; enable master
TWI0_MSTATUS  ← TWI_BUSSTATE_IDLE_gc ; force idle (AFTER enable)
```

TWI master write:

```
STS TWI0_MADDR, (addr << 1)      ; START + address + W
wait WIF, check RXACK == 0
STS TWI0_MDATA, data_byte
wait WIF, check RXACK == 0
STS TWI0_MCTRLB, TWI_MCMD_STOP_gc ; STOP
```

RXACK polarity: 0 = ACK (slave present), 1 = NACK (slave absent/busy)

String rendering (same loop for every device):

```
LD r17, Z+                ; next byte from flash-mapped string (.rodata)
TST r17 / BREQ done       ; stop at the NUL terminator
RCALL putc                ; device-specific: lcd_data / oled_putc / USART tx
```

HD44780 over PCF8574 (4-bit): nibble -> P4..P7; pulse P2 (E); RS = P0
send byte = high nibble then low nibble, each strobed E high then E low

SSD1306 OLED: control 0x00 = commands, 0x40 = data stream
glyph at font + (char - 0x20) * 5; stream 5 columns + 1 blank as data
(remember CLR r1 after MUL)

22.17 Exercises

1. The `spi_transfer` function polls `SPI_IF_bp` in `SPI0_INTFLAGS`. At 3.333 MHz with `SCK = f/4`, how many CPU cycles elapse from writing `SPI0_DATA` until `IF` becomes set? How many poll iterations does the CPU execute during that time? (Use the `LDS+SBRS+RJMP` cycle count.)
2. The 25LC010A has a 16-byte page buffer. A page write starts with address 0x00 and fills up to 16 bytes before CS↑. Modify `eeeprom_write_byte` into `eeeprom_write_page` that accepts a pointer (Z) and a byte count (r20, max 16). Send `WREN`, then one `WRITE` transaction with all bytes inside the same CS-framed transfer. How does the CS framing differ from the single-byte version?
3. Change the SPI clock to the fastest value this 3.333 MHz CPU clock can generate while remaining below the 25LC010A's 10 MHz limit. Which combination of `PRESC` and `CLK2X` gives that `SCK`? Write the new `SPI0_CTRLA` value.
4. In `twi_write_byte`, the `BUSSTATE` field of `TWI0_MSTATUS` is not checked before sending. What value does it hold between transactions (after a `STOP`)? Under what conditions might starting a transaction with `BUSSTATE = BUSY` cause a problem? How would you guard against it?
5. Write `twi_read_byte`: read one byte from a TWI slave after sending its address with `R/W = 1`. After receiving the byte, send `NACK + STOP` (to tell the slave you don't want more bytes). Which `ACKACT` bit value selects `NACK`, which `MCMD` value issues `STOP`, and which `MSTATUS` flag indicates the byte is ready?
6. Using `avr-objdump -d spi_eeeprom.elf`, find the `STS SPI0_DATA, r16` instruction. What is its 32-bit encoding? Identify which bytes encode the destination address (0x0824) and which bytes encode the source register (r16 = R16 = register 16). Recall that `STS` with a 16-bit address uses the encoding `1001 001d dddd 0000 | kkkk kkkk kkkk kkkk`.

7. The `print_str` loop reads each byte with `LD r17, Z+` from the flash-mapped `.rodata` window. On this ATtiny3217 the linker gives a `.rodata` label its data-space address directly, so `lo8(msg)/hi8(msg)` load `Z` without adding `MAPPED_PROGMEM_START`. Build `lcd_hd44780.elf` and use `avr-nm` to find the address of `msg`. Which 0x8000-region address does it report, and what would `Z` contain if you *had* added 0x8000 by mistake?
8. `lcd_write4` issues two full `twi_write_byte` transactions per nibble, so a single character costs four I²C transactions. At ~98 kHz, estimate the time to write one character (each transaction $\approx$ START + address + 1 data byte + STOP $\approx$ 30 SCL clocks). How long does the 13-character demo string take? Why does this still comfortably satisfy the HD44780's E-pulse timing?
9. `oled_putc` computes the glyph offset with `MUL r17, r18` (`r18 = 5`) and then executes `CLR r1`. Remove the `CLR r1` and trace what happens on the *next* character: which later instruction relies on `r1` being zero, and how would the rendered text be corrupted? (Hint: look at the `ADC_ZH, r1` that forms the flash pointer.)
10. Extend the SSD1306 font to cover the digits '0'-'9' (0x30-0x39) and write `oled_put_dec`: given an 8-bit value in `r24`, render it as up to three decimal characters by repeated division by 10. Which existing routine does each digit reuse, and where do the glyph bytes for the digits sit relative to font?

Next: Chapter 22 — CRC and Data Integrity in Assembly

Chapter 23

CRC and Data Integrity in Assembly

CRC routines are a compact way to detect corrupted messages, tables, and code images. This chapter stays in assembly: a software CRC-8 routine for small data blocks, then the ATtiny3217 CRCSCAN peripheral for checking program Flash.

The two jobs are different:

Software CRC routine:

- Runs over bytes you choose.

- Useful for packets, EEPROM records, protocol frames, and small tables.

CRCSCAN peripheral:

- Runs over Flash sections selected by hardware configuration.

- Useful for startup or runtime program-memory integrity checks.

Microchip documents CRCSCAN on ATtiny3217 as a CRC-16-CCITT memory scanner that can check the entire Flash, the boot + application code section, or the boot section. It can also be configured through FUSE.SYSCFG0 to run during reset initialization before normal code execution starts.

23.1 CRC Basics

A cyclic redundancy check treats a byte stream as a polynomial over bits. Each incoming bit shifts the current remainder. When a one is shifted out, the remainder is XORed with a generator polynomial.

For the byte-oriented routines in this chapter:

CRC-8/SMBus:

- Width: 8 bits

- Polynomial: $0x07$ ($x^8 + x^2 + x + 1$)

- Initial CRC: $0x00$

- Bit order: MSB first

CRC-8 is small enough to write directly as a bit loop. It is not as strong as CRC-16, but it is useful for short records and teaching the mechanics.

For an 8-bit CRC accumulator:

for each input byte:

- `crc = crc XOR byte`

- repeat 8 times:

```

    shift crc left
    if the old bit 7 was one:
        crc = crc XOR polynomial

```

On AVR, LSL shifts bit 7 into SREG.C. That means the branch condition is already available:

```

lsl   r24      ; old bit 7 moves into C
brcc  no_xor   ; if old bit 7 was 0, skip feedback
eor   r24, r25 ; apply polynomial

```

23.2 Software CRC-8 Register Plan

The chapter example computes a CRC over bytes stored in Flash.

Input:

```

Z      byte address of first Flash byte
r20    byte count

```

Output:

```

r24    CRC result

```

Scratch:

```

r18    current input byte
r19    bit counter
r25    polynomial constant

```

Because this is a standalone assembly routine, the interface is simply the register contract above. There is no hidden runtime setup and no external language call boundary.

23.3 Flash Test Data

The example uses the byte sequence 01 02 03 04. For CRC-8/SMBus with polynomial 0x07 and initial value 0x00, the expected CRC is 0xE3.

```

.section .text
.Lmsg:
    .byte 0x01, 0x02, 0x03, 0x04
.balign 2

```

The `.balign 2` matters because the next instruction must start on an even byte address. Raw byte data in `.text` can leave the location counter odd.

23.4 CRC-8 Implementation

```

.equ CRC8_POLY, 0x07

.global crc8_flash
crc8_flash:
    ldi   r24, 0x00      /* CRC accumulator */
    ldi   r25, CRC8_POLY /* polynomial */

    tst   r20
    breq  .Lcrc_done

```

```

.Lcrc_byte:
    lpm    r18, Z+          /* read next Flash byte */
    eor    r24, r18         /* mix byte into CRC */
    ldi    r19, 8           /* 8 bits per byte */

.Lcrc_bit:
    lsl    r24              /* old bit 7 -> C */
    brcc   .Lcrc_no_xor
    eor    r24, r25

.Lcrc_no_xor:
    dec    r19
    brne   .Lcrc_bit

    dec    r20
    brne   .Lcrc_byte

.Lcrc_done:
    ret

```

Important details:

- LPM r18, Z+ reads program memory and advances the Flash byte pointer.
- EOR is used both to mix the input byte and to apply the polynomial.
- LSL sets carry from the old MSB, so no separate mask instruction is needed.
- r24 is reused as the accumulator and final result.

For four bytes, the bit loop runs 32 times. Each bit takes:

No feedback path: LSL + BRCC taken + DEC + BRNE

Feedback path: LSL + BRCC not taken + EOR + DEC + BRNE

The exact cycle count depends on the data because the feedback path includes the extra EOR and different branch timing.

23.5 Standalone Demo

The full source is [crc8.S](#). It computes the CRC and stores the result in SRAM for debugger inspection.

```

.section .bss
.global crc_result
crc_result:
    .skip 1

.global main
main:
    ldi    r16, hi8(RAMEND)
    out    SPH, r16
    ldi    r16, lo8(RAMEND)
    out    SPL, r16

    ldi    ZL, lo8(.Lmsg)
    ldi    ZH, hi8(.Lmsg)
    ldi    r20, 4
    rcall  crc8_flash

    sts    crc_result, r24

```

```
.Lhalt:
    rjmp .Lhalt
```

Build and inspect:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o crc8.o src/crc8.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o crc8.elf crc8.o
avr-objdump -d crc8.elf
```

Expected signs in the disassembly:

```
lpm    r18, Z+
eor    r24, r18
lsl    r24
brcc   ...
eor    r24, r25
sts    <crc_result>, r24
```

The expected stored value is 0xE3.

23.6 Table-Driven CRC

The bit loop is small, but it costs eight iterations per byte. A faster design uses a 256-byte lookup table in Flash:

```
index = crc XOR input_byte
crc = table[index]
```

On AVR, a table lookup uses Z and LPM. If the table starts on a 256-byte boundary, the index can be placed directly in ZL while ZH holds the page. If the table is not aligned, add the index to the full 16-bit table pointer.

Tradeoff:

Bit loop:

- Small code and no table.
- Cost grows with 8 bit-steps per input byte.

Table lookup:

- Larger Flash use: 256 bytes for CRC-8.
- Much faster per byte.

For short packets or rare checks, the bit loop is usually fine. For long streams, the table can be worth the Flash.

23.7 ATtiny3217 CRCSCAN Peripheral

The ATtiny3217 has a dedicated CRCSCAN peripheral at 0x0120.

Register	Address	Purpose

CRCSCAN_CTRLA	0x0120	Enable, NMI enable, reset
CRCSCAN_CTRLB	0x0121	Source selection and Flash access mode
CRCSCAN_STATUS	0x0122	Busy and OK flags

CTRLA bits:


```

Bit 7  RESET   CRCSCAN_RESET_bm   = 0x80
Bit 1  NMIEN   CRCSCAN_NMIEN_bm   = 0x02
Bit 0  ENABLE  CRCSCAN_ENABLE_bm  = 0x01

```

CTRLB fields:

```

SRC[1:0]
0x00  CRCSCAN_SRC_FLASH_gc        entire Flash
0x01  CRCSCAN_SRC_APPLICATION_gc  boot + application code section
0x02  CRCSCAN_SRC_BOOT_gc         boot section

```

```

MODE[5:4]
0x00  CRCSCAN_MODE_PRIORITY_gc    priority access to Flash

```

STATUS bits:

```

Bit 1  OK      CRCSCAN_OK_bm      = 0x02
Bit 0  BUSY    CRCSCAN_BUSY_bm    = 0x01

```

Microchip documents the hardware algorithm as CRC-16-CCITT:

Polynomial: $x^{16} + x^{12} + x^5 + 1$
 Initial checksum register value: 0xFFFF
 Input order: byte by byte, starting at byte 0, MSB first

The last two bytes of the selected Flash section must contain the precomputed 16-bit checksum. For full Flash, Microchip places CHECKSUM[15:8] at FLASHEND-1 and CHECKSUM[7:0] at FLASHEND. Boot and boot+application checks use the corresponding BOOTEND or APPEND fuse boundary. If the calculated value matches the placed checksum, CRCSCAN_STATUS.OK is set.

23.8 Starting a CRCSCAN in Assembly

The software sequence is short:

1. Write CRCSCAN_CTRLB to select the Flash source.
2. Write CRCSCAN_CTRLA with ENABLE.
3. Poll STATUS.BUSY until it clears.
4. Test STATUS.OK.

Example: scan entire Flash without NMI.

```

crcscan_flash:
    ldi    r16, CRCSCAN_SRC_FLASH_gc
    sts    CRCSCAN_CTRLB, r16

    ldi    r16, CRCSCAN_ENABLE_bm
    sts    CRCSCAN_CTRLA, r16

.Lcrcscan_busy:
    lds    r16, CRCSCAN_STATUS
    sbrc   r16, CRCSCAN_BUSY_bp
    rjmp   .Lcrcscan_busy

    sbrs   r16, CRCSCAN_OK_bp
    rjmp   .Lcrcscan_failed
    ret

.Lcrcscan_failed:
    rjmp   .Lcrcscan_failed

```

Microchip notes that a software-started scan begins after three cycles. In priority mode, CRCSCAN has priority access to Flash and stalls the CPU until the scan is complete. That makes runtime scans easy to reason about but not free; schedule them where a pause is acceptable.

One subtlety in the polling loop: because the scan does not begin until three cycles after the ENABLE write, a BUSY read issued too early can return 0 before the scan has started, letting the loop fall through prematurely. In the sequence above the STS/LDS spacing and the priority-mode CPU stall cover this gap, but if you restructure the code, insert a few cycles (or read STATUS twice) after enabling so the first BUSY poll cannot race the three-cycle start delay.

23.9 NMI on CRC Failure

CRCSCAN can trigger a Non-Maskable Interrupt when a scan fails:

```
ldi    r16, (CRCSCAN_ENABLE_bm | CRCSCAN_NMIEN_bm)
sts    CRCSCAN_CTRLA, r16
```

The CRCSCAN NMI vector is vector 1. On ATtiny3217, vector entries are 4-byte JMP entries, so vector 1 starts at byte address 0x0004:

```
__vectors:
    jmp    main                /* 0x0000 RESET */
    jmp    crcscan_nmi         /* 0x0004 CRCSCAN_NMI */
```

A CRC failure means program memory may be unreliable, so the NMI handler should be extremely conservative. A typical response is to put outputs in a safe state and stop execution or force a reset path. Do not treat a failed program-memory CRC like an ordinary recoverable event. Once CRCSCAN triggers an NMI, the NMI request remains active until a system reset and cannot be disabled in software.

23.10 Startup CRC Through Fuses

FUSE_SYSCFG0 contains the CRCSRC field at bits 7:6. Microchip documents that this field can make CRCSCAN run during reset initialization. If the startup CRC check fails, normal code execution is not allowed to start. The failure can still be observed through the debug interface.

The ATtiny3217 fuse encodings are:

```
CRCSRC[1:0]
0x00  CRCSRC_FLASH_gc    entire Flash
0x01  CRCSRC_BOOT_gc     boot section
0x02  CRCSRC_BOOTAPP_gc  boot + application code section
0x03  CRCSRC_NOCRC_gc    startup CRC disabled
```

That mode is different from a software-started runtime scan:

Runtime scan:

Code starts first, then software enables CRCSCAN.

Startup scan:

Reset sequence checks Flash before normal code execution.

Use startup CRC for stronger boot-time assurance. Use runtime CRC when the program needs to periodically verify a Flash region while running.

The Defensive Firmware chapter puts these to work — choosing the fuse pre-boot scan as a device’s first integrity gate and folding a run-time scan into a fail-safe boot sequence alongside the watchdog and EEPROM checks.

23.11 Summary

Software CRC-8:

Polynomial 0x07
 Initial value 0x00
 LSL moves old bit 7 into SREG.C
 BRCC decides whether to XOR the polynomial

ATtiny3217 CRCSCAN:

Registers: CTRLA=0x0120, CTRLB=0x0121, STATUS=0x0122
 Hardware algorithm: CRC-16-CCITT
 Polynomial: $x^{16} + x^{12} + x^5 + 1$
 Initial value: 0xFFFF
 Sources: entire Flash, boot + application code, or boot section
 STATUS.BUSY bit 0, STATUS.OK bit 1
 Optional NMI on failure
 Optional startup scan through FUSE_SYSCFG0.CRCSRC

23.12 Exercises

1. Step through `crc8_flash` for the first byte 0x01. After the EOR, what is the accumulator? Which bit iterations take the feedback path?
 2. Modify `crc8_flash` to compute over SRAM instead of Flash. Which instruction changes, and how does the pointer setup differ?
 3. Add a second test message in Flash and store two result bytes in SRAM.
 4. Rewrite the CRC-8 routine so the polynomial is passed in r21 instead of assembled as `CRC8_POLY`.
 5. Write a `crcscan_boot` routine that selects `CRCSCAN_SRC_BOOT_gc`, starts a scan, waits for `BUSY=0`, and returns `r24=0` on OK or `r24=1` on failure.
 6. Estimate the runtime cost of a CRCSCAN priority scan over 32 KB Flash using Microchip’s note that the peripheral fetches one 16-bit word every third main clock cycle in priority mode.
-

Next: Huffman Decoding from Flash

Chapter 24

Huffman Decoding from Flash

Huffman coding gives frequent symbols short bit-codes and rare symbols long ones, so a message that is mostly a few common bytes packs into fewer bits. This chapter uses it for one job: unpacking data that was compressed ahead of time and stored in Flash — canned strings, lookup tables, bitmap fonts.

The chapter is deliberately decode-only. Building an optimal Huffman code means counting symbol frequencies, repeatedly merging the two least-frequent nodes, and assigning codes by walking the resulting tree. That is pointer-heavy, allocation-heavy work that fits a desktop far better than a part with 2 KB of SRAM. Real firmware almost never builds the tree on the device. It does what this chapter does:

Host side (offline, build time):

- Count frequencies, build the tree, derive a canonical table.
- Emit the table and the packed stream as const data in Flash.

MCU side (this chapter, assembly):

- Walk a bit stream, follow the table, write decoded bytes to SRAM.
- No tree construction. No dynamic memory. Decode only.

This chapter covers:

1. What a Huffman code is, and why decode-only suits AVR
2. Prefix codes and the MSB-first bit-stream contract
3. Two table formats: an explicit tree, and a canonical (length) table
4. A tree-walk decoder in assembly
5. A canonical decoder in assembly
6. Reading tables and packed data from Flash on AVRxmega3
7. Validation, edge cases, and when not to reach for Huffman

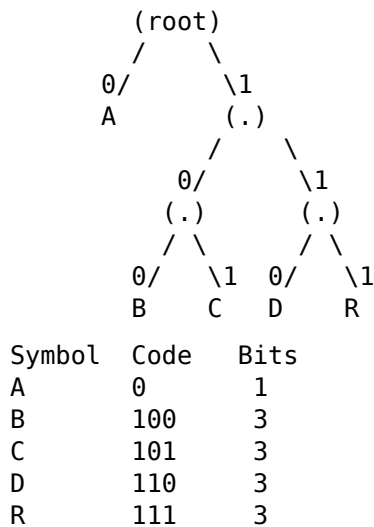
24.1 Huffman in One Picture

Take the message ABRACADABRA. The symbols and their frequencies are:

Symbol	Count
A	5
B	2
R	2
C	1
D	1

A Huffman tree merges the two least-frequent nodes repeatedly until one tree remains. Reading 0 for a left branch and 1 for a right branch gives each symbol a code. One optimal

tree for this message is:



The frequent A costs one bit; the rare symbols cost three. The whole message packs into 23 bits instead of 88.

The property that makes decoding possible is that this is a **prefix code**: no code is a prefix of another. A is 0, and every other code starts with 1, so the moment the decoder has read a complete code it knows the symbol ended — no separators are needed between codes.

When two nodes have equal weight there is a choice of which to merge, so more than one optimal tree exists. Different tools may produce different code *shapes* for the same input; all that matters is that the decoder uses the same table the encoder did. The companion script `build_tables.py` is the source of truth for the tables in this chapter.

24.2 The Bit-Stream Contract

Codes are packed into bytes **most-significant bit first**. The first bit of the message is bit 7 of the first byte. This matches the AVR shift instructions: `LSL` moves bit 7 into `SREG.C`, so reading the next bit and testing it is a single instruction, exactly as in the CRC chapter.

For ABRACADABRA the codes concatenate to 23 bits, packed into three bytes (the final byte padded with zeros):

A	B	R	A	C	A	D	A	B	R	A
0	100	111	0	101	0	110	0	100	111	0

bit:	0	1	0	0	1	1	1	0		1	0	1	0	1	1	0	0		1	0	0	1	1	1	0	(0)
byte:	0x4E									0xAC									0x9C							

Two facts the decoder cannot recover from the stream itself:

Where the message ends:

Byte count does not tell you the symbol count (the last byte is padded).
Store an explicit symbol count, or reserve an end-of-message symbol.

Which table was used:

The table is not in the stream. The decoder must hold the matching table.

The examples here store the symbol count as a build-time constant (`MSG_LEN`) and decode exactly that many symbols.

24.3 Two Ways to Store the Table

The same code can be stored two ways, and the chapter implements both:

Format	Flash cost	Decode work	Notes
-----	-----	-----	-----
Explicit tree	2 bytes / node	1 node / bit	easiest to follow
Canonical table	1 byte / symbol + 1 byte / length	1 pass / symbol	smallest; DEFLATE uses it

The explicit tree is the literal picture above: a node table you walk one bit at a time. The canonical table stores only how *long* each symbol's code is; the codes themselves are regenerated from the lengths, because canonical codes are assigned in a fixed order (by length, then by symbol). That is why a .png-style format or DEFLATE ships code lengths and not codes.

24.4 Reading Flash

Both decoders read two things from Flash: the table and the packed stream. As in the CRC chapter, that means LPM with the Z pointer. The wrinkle is that the decoders need two Flash pointers at once — one walking the stream, one indexing the table — but only Z can drive LPM.

The fix is to carry the stream pointer in X and copy it into Z only for the moment of the fetch, then save the advanced value back:

```
movw  r30, r26          /* Z = stream pointer (X)      */
lpm    r20, Z+           /* fetch next byte, advance Z    */
movw   r26, r30          /* save advanced pointer back to X */
```

Between fetches Z is free, so each table lookup recomputes Z from a base address held in registers. The tables and stream live in .text (Flash) as plain bytes, and pointers into them are loaded with lo8/hi8:

```
.section .text
.balign 2
.Lnodes:
    .byte 0x80|'A', 0x01    /* node0 */
    .byte 0x02, 0x03       /* node1 */
    .byte 0x80|'B', 0x80|'C' /* node2 */
    .byte 0x80|'D', 0x80|'R' /* node3 */
```

(On this AVRxmega3 part the whole Flash is also mapped into the data space at 0x8000, so the stream could instead be read with a plain LD from address + 0x8000, freeing Z entirely. Exercise 7 rewrites the bit reader that way; the listings here stay with LPM for consistency with the rest of the book.)

24.5 A Shared Bit Reader

Both decoders pull bits the same way. A byte is held in a buffer register with a count of how many bits remain. When the count reaches zero the next stream byte is fetched (via the X-to-Z LPM dance above). LSL delivers the next bit (MSB first) into carry:

```
if bits_left == 0:
    buffer = next stream byte
    bits_left = 8
```

```

shift buffer left      ; old bit 7 -> Carry
bits_left = bits_left - 1

```

DEC does not touch the carry flag, so the bit produced by LSL survives the decrement and is still available to the instruction that consumes it.

24.6 Decoder 1 — Walking the Tree

The explicit tree is stored two bytes per node: the child to take on a 0 bit, then the child to take on a 1 bit. Each child byte is either an **internal node index** (bit 7 clear) or a **leaf** (bit 7 set, with the 7-bit symbol in the low bits). Seven bits hold any ASCII symbol, which suits text. The root is node 0.

```

node0:  0 -> leaf 'A'    1 -> node1
node1:  0 -> node2      1 -> node3
node2:  0 -> leaf 'B'    1 -> leaf 'C'
node3:  0 -> leaf 'D'    1 -> leaf 'R'

```

Decoding is a walk from the root: read a bit, step to the named child, and if that child is a leaf, emit its symbol and return to the root.

```

node = root (0)
loop:
    read next bit
    entry = node_table[node*2 + bit]
    if entry is a leaf:
        emit entry's symbol
        node = root
    else:
        node = entry
    until all symbols decoded

```

The register contract:

Input:

X	packed stream, Flash byte address
Y	output buffer in SRAM
r25:r24	node-table base, Flash byte address
r23	number of symbols to decode

Scratch:

r0	zero register (carry propagation)
r17	current child entry
r20	bit buffer
r21	bits left in buffer
r22	current node index

The implementation:

```

.global huff_decode_tree
huff_decode_tree:
    clr    r0                      /* zero register */
    clr    r22                    /* node = root (index 0) */
    clr    r21                    /* bits left = 0 -> force a reload */
.Lt_bit:
    tst    r21
    brne   .Lt_have
    movw   r30, r26                /* Z = stream pointer */
    lpm    r20, Z+                /* fetch next stream byte, advance */

```

```

    movw    r26, r30                /* save advanced pointer back to X */
    ldi     r21, 8
.Lt_have:
    movw    r30, r24                /* Z = base + node*2 */
    add     r30, r22
    adc     r31, r0
    add     r30, r22
    adc     r31, r0
    lsl     r20                    /* next bit (MSB first) -> Carry */
    dec     r21                    /* DEC preserves Carry */
    adc     r30, r0                /* a 1 bit selects the right child (+1) */
    adc     r31, r0
    lpm     r17, Z                 /* child entry */
    sbrc    r17, 7                /* leaf? */
    rjmp    .Lt_leaf
    mov     r22, r17               /* internal node: descend */
    rjmp    .Lt_bit
.Lt_leaf:
    andi    r17, 0x7f             /* recover the 7-bit symbol */
    st      Y+, r17
    clr     r22                   /* back to the root */
    dec     r23
    brne    .Lt_bit
    ret

```

Worth noting:

- The node address is $\text{base} + \text{node} * 2 + \text{bit}$. The two ADD/ADC pairs double the node index; the ADC `r30, r0` after LSL folds the data bit in, because `r0` is zero and carry holds the bit.
- SBRC (skip if bit cleared) tests bit 7 of the child entry in one instruction: a leaf skips the RJMP and falls into `.Lt_leaf`.
- Cost is one table read per *bit*, so a three-bit symbol costs three reads. That is the price of the simplest possible table.

24.7 Decoder 2 — Canonical Decode

The canonical decoder stores no tree. It keeps two small tables:

```

counts[len]  number of codes that are exactly len bits long (len 0..MAXLEN)
symbols[]    the symbols, in canonical order: by length, then by symbol

```

For ABRACADABRA:

```

counts:  len 0..8 = 0, 1, 0, 4, 0, 0, 0, 0, 0
symbols:  A, B, C, D, R

```

One code of length 1 (A), four of length 3 (B C D R, in symbol order). Canonical codes are assigned by counting up within each length and shifting left when the length increases, so the decoder can regenerate them on the fly. It folds in one bit per length and asks whether the code so far lands inside the block of codes of that length:

```

code = first = index = 0
for len = 1 .. MAXLEN:
    code |= next_bit
    count = counts[len]
    if (code - first) < count:
        return symbols[index + (code - first)]

```



```

index += count
first = (first + count) << 1
code <=< 1

```

first is the numeric value of the first code of the current length; index is where that length's symbols begin in symbols[]. If the accumulated code is within count of first, the symbol is found and its position follows directly. Capping MAXLEN at 8 keeps every value in a single byte. (DEFLATE allows code lengths up to 15 and needs 16-bit arithmetic; the structure is identical.)

The register contract:

Input:

```

X      packed stream, Flash byte address
Y      output buffer in SRAM
r25:r24 counts[] base, Flash byte address
r3:r2  symbols[] base, Flash byte address
r16    number of symbols to decode

```

Scratch:

```

r0      zero register
r4      index          r17 code - first (temp)
r6      count          r18 len
r19     symbols left   r20 bit buffer    r21 bits left
r22     code           r23 first

```

The implementation:

```

.equ MAXLEN, 8

.global huff_decode_canon
huff_decode_canon:
    clr    r0
    mov    r19, r16                /* symbols remaining */
    clr    r21                    /* bits left = 0 -> force a reload */
.Lc_sym:
    clr    r22                    /* code = 0 */
    clr    r23                    /* first = 0 */
    clr    r4                     /* index = 0 */
    ldi    r18, 1                 /* len = 1 */
.Lc_len:
    tst    r21
    brne   .Lc_have
    movw   r30, r26                /* Z = stream pointer */
    lpm    r20, Z+
    movw   r26, r30
    ldi    r21, 8
.Lc_have:
    lsl    r20                    /* next bit (MSB first) -> Carry */
    dec    r21                    /* preserves Carry */
    adc    r22, r0                /* code |= bit (bit 0 was clear) */
    movw   r30, r24                /* count = counts[len] */
    add    r30, r18
    adc    r31, r0
    lpm    r6, Z
    mov    r17, r22                /* r17 = code - first */
    sub    r17, r23
    cp     r17, r6                /* (code - first) < count ? */
    brlo   .Lc_found
    add    r4, r6                /* index += count */

```

```

add    r23, r6                /* first += count          */
lsl    r23                    /* first <= 1            */
lsl    r22                    /* code <= 1            */
inc    r18
cpi    r18, MAXLEN+1
brlo   .Lc_len
ret                                /* overran MAXLEN: stream is corrupt */
.Lc_found:
add    r4, r17                /* index += (code - first) */
movw   r30, r2                /* Z = symbols base       */
add    r30, r4
adc    r31, r0
lpm    r17, Z                  /* the symbol             */
st     Y+, r17
dec    r19
brne   .Lc_sym
ret

```

Compared with the tree walk:

- The table is far smaller: one byte per symbol plus a short counts[] array, with no per-node child pointers.
- The inner work is a subtract and an unsigned compare (CP then BRL0) per length, not a table read per bit.
- counts[] is padded out to MAXLEN with zeros so the loop, which always runs len = 1..MAXLEN, never reads past the table even on a malformed stream. The RET after the MAXLEN check is the corrupt-input exit.

24.8 Standalone Demo

The full source is [huffman.S](#). It decodes the packed ABRACADABRA stream twice — once with each decoder — into two SRAM buffers, then transmits the decoded text over USART0 (polled, 9600 8N1) before halting:

```

main:
    ldi    r16, hi8(RAMEND)
    out    SPH, r16
    ldi    r16, lo8(RAMEND)
    out    SPL, r16

    /* tree decode into out_tree */
    ldi    XL, lo8(.Lstream)
    ldi    XH, hi8(.Lstream)
    ldi    YL, lo8(out_tree)
    ldi    YH, hi8(out_tree)
    ldi    r24, lo8(.Lnodes)
    ldi    r25, hi8(.Lnodes)
    ldi    r23, MSG_LEN
    rcall  huff_decode_tree

    /* canonical decode into out_canon */
    ldi    XL, lo8(.Lstream)
    ldi    XH, hi8(.Lstream)
    ldi    YL, lo8(out_canon)
    ldi    YH, hi8(out_canon)
    ldi    r24, lo8(.Lcounts)

```

```

ldi    r25, hi8(.Lcounts)
ldi    r18, lo8(.Lsymbols)
ldi    r19, hi8(.Lsymbols)
movw   r2, r18                /* r3:r2 = symbols base          */
ldi    r16, MSG_LEN
rcall  huff_decode_canon

/* USART0: 9600 8N1, TX only (polled) */
sbi    VPORTB_OUT, TXD_BIT    /* drive PB2 high before TXEN takes over */
sbi    VPORTB_DIR, TXD_BIT
ldi    r16, BAUD_LO
sts    USART0_BAUDL, r16
ldi    r16, BAUD_HI
sts    USART0_BAUDH, r16
ldi    r16, USART_TXEN_bm
sts    USART0_CTRLB, r16

/* transmit out_canon (CRLF afterward; see the full source) */
ldi    XL, lo8(out_canon)
ldi    XH, hi8(out_canon)
ldi    r18, MSG_LEN
.Lsend:
ld     r16, X+
rcall  usart_tx_byte
dec    r18
brne   .Lsend

.Lhalt:
rjmp   .Lhalt

```

The output path reuses the Curiosity Nano setup from the USART chapter: TXD is PB2, the byte is pushed to USART0_TXDATA0 once USART_DREIF shows the transmit buffer is free. Decoding into a buffer first keeps the two SRAM results inspectable; Exercise 5 wires the transmit into the decode loop so each symbol goes out as it is produced.

Build it:

```

avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o huffman.o src/huffman.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o huffman.elf huffman.o

```

Run it under simavr with a debugger attached and inspect both buffers (break at .Lhalt — find its address with avr-objdump -d huffman.elf):

```

simavr -m attiny3217 -g huffman.elf &
avr-gdb -batch \
  -ex "target remote :1234" \
  -ex "break *.Lhalt" \
  -ex "continue" \
  -ex "x/11cb &out_tree" \
  -ex "x/11cb &out_canon" \
  huffman.elf

```

Both buffers hold the same eleven bytes, and the same text appears on TXD:

```

41 42 52 41 43 41 44 41 42 52 41    =   A B R A C A D A B R A

```

24.9 Regenerating the Tables

The tables and stream in `huffman.S` were produced by `build_tables.py`, which does the offline half of the job:

```
python3 src/build_tables.py "ABRACADABRA"
```

It counts frequencies, builds the tree, derives canonical code lengths, and prints the `.Ln-`odes, `.Lcounts`, `.Lsymbols`, and `.Lstream` lines ready to paste into the assembly source. To compress a different message — a status string, a font, a table — run the script over it and replace the four data lines plus `MSG_LEN`. The decoders do not change.

24.10 When Not to Use Huffman

Huffman is not free, and on a small MCU it is often the wrong tool:

Tiny payloads:

The table can cost more Flash than it saves. A 20-byte string is not worth a tree plus a stream.

High-entropy data:

Already-compressed or near-random data does not have skewed symbol frequencies, so codes do not shrink.

Streaming with no host step:

If the data is produced on the device and never pre-analysed, there is no place to build the table.

For runs of repeated bytes — cleared regions, padded records, simple bitmaps — run-length encoding is simpler and usually wins, with no tree at all. Huffman earns its place when a fixed, skewed alphabet (text, a known symbol set) is decoded many times from a table you can prepare once at build time.

A decoder that walks off the end of a valid table only on corrupt input still needs a guard in production: the canonical decoder's `MAXLEN` check is that guard, and the tree decoder should be given only streams whose length matches `MSG_LEN`. The Defensive Firmware chapter covers treating external data as hostile, which applies directly to anything you decompress at runtime.

24.11 Summary

Decode-only:

Build the table on the host; the MCU never builds a tree.

Bit stream:

MSB first; LSL moves the next bit into `SREG.C`.

Store the symbol count or an end-of-message symbol separately.

Tree decoder (`huff_decode_tree`):

Node table, 2 bytes/node: child-on-0, child-on-1.

Leaf = bit7 set + 7-bit symbol; internal = node index.

One table read per bit.

Canonical decoder (`huff_decode_canon`):

`counts[len]` + `symbols[]` in canonical order.

Regenerates codes from lengths; one pass per symbol.
Smallest table; the format DEFLATE uses.

Flash access:

LPM via Z reads tables and stream; carry the stream pointer in X
and MOVW it into Z only to fetch the next byte.

24.12 Exercises

1. Hand-decode the first byte of the stream, 0x4E, with the tree table. How many symbols does it yield, and how many bits of the second byte are needed to finish the symbol that straddles the byte boundary?
 2. The tree decoder marks a leaf with bit 7, limiting symbols to 7 bits. Change the node format to allow full 8-bit symbols. What extra information does each entry now need, and how does the decode loop change?
 3. Add an end-of-message symbol to the alphabet and have `huff_decode_tree` stop when it emits that symbol, instead of counting `MSG_LEN` symbols. Regenerate the tables with `build_tables.py` over a message that includes it.
 4. Feed `huff_decode_canon` a deliberately truncated stream and confirm it exits through the `MAXLEN` path rather than running away. What does the output buffer contain in that case?
 5. The demo decodes into a buffer and then transmits it. Move the USART transmit inside `huff_decode_tree` so each symbol goes out as it is produced, with no output buffer at all. Which register in the decode loop holds the byte to send, and what must you preserve across the `usart_tx_byte` call?
 6. Compare the total Flash cost of the two table formats for a 16-symbol alphabet with a maximum code length of 6. Which is smaller, and by how much?
 7. Rewrite the shared bit reader to use the AVRxmega3 Flash mapping: point X at `.Lstream + 0x8000` and fetch stream bytes with `ld r20, X+` instead of the `MOVW/LPM/MOVW` sequence. What does this free up, and why does it work only because the stream lives below the 32 KB Flash boundary?
-

Next: Optimisation Techniques

Chapter 25

Optimisation Techniques

Assembly optimisation is useful only when it is measured. The ATtiny3217 is predictable enough that many improvements can be proven from the instruction listing alone: count the instructions on the hot path, apply the cycle counts from Microchip's AVR Instruction Set Manual, then compare the result with the flash and SRAM cost.

This chapter keeps the examples assembly-only. The benchmark harness in `src/bench_harness.S` calls the routines directly with registers, gives the linker real symbols to place, and leaves the result visible to `avr-objdump`, `avr-nm`, GDB, or MDB.

The techniques:

1. **Cycle counting** - build a baseline from the disassembly.
2. **Register residency** - keep hot values in registers, not SRAM.
3. **Loop unrolling** - trade flash for fewer branch and counter cycles.
4. **Flash lookup tables** - trade flash for much faster fixed-domain results.
5. **Size checks** - verify that speed did not quietly consume too much flash.

Measure first, optimise second.

25.1 Build and Inspect the Chapter Examples

Build from the chapter directory:

```
cd ch15_optimisation

avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -g3 -Wa,--gdwarf-2 \
  -nostartfiles -o bench.elf \
  src/bench_harness.S src/memcpy_naive.S src/memcpy_fast.S \
  src/lut_sin.S src/lut_interp.S src/horner.S
```

Inspect symbols and sizes:

```
avr-nm --size-sort --print-size bench.elf
```

Inspect instructions:

```
avr-objdump -d -S bench.elf
```

Useful focused views:

```
avr-objdump -d bench.elf | grep -A 20 '<memcpy_naive>'
avr-objdump -d bench.elf | grep -A 40 '<memcpy_fast>'
avr-objdump -d bench.elf | grep -A 12 '<sin8>'
avr-objdump -d bench.elf | grep -A 12 '<interp8>'
```

```
avr-objdump -d bench.elf | grep -A 18 '<horner8>'
avr-objdump -s -j .text bench.elf
```

The examples are linked into one ELF so the addresses and code sizes are real. No generated PDF rebuild is required for this chapter edit.

25.2 Cycle Reference for This Chapter

The ATtiny3217 uses the AVRxt timing column in Microchip’s AVR Instruction Set Manual. Always check the manual for instructions not listed here, especially when carrying code to older AVR, AVRxm, or reduced-core AVRrc devices.

Common timing used in this chapter:

Instruction	ATtiny3217 cycles	Notes
NOP	1	
MOV, MOVW	1	
LDI	1	r16-r31 only
CLR	1	encoded as EOR Rd,Rd
ADD, ADC	1	
AND, ANDI, EOR	1	
INC, DEC	1	
LSR	1	
TST	1	encoded as AND Rd,Rd
BREQ/BRNE not taken	1	
BREQ/BRNE taken	2	
RJMP	2	
RCALL	2	
RET	4	larger PC-width AVRr can differ
LD Rd, Z+	2	
ST X+, Rr	1	
LPM Rd, Z	3	
LPM Rd, Z+	3	
STS k, Rr	2	two-word instruction
MUL, MULS, MULSU	2	product in r1:r0; MULS/MULSU use r16-r23

Two details cause most bad cycle counts:

- A conditional branch costs 2 cycles when taken and 1 cycle when it falls through.
- Count the last loop iteration separately, because the loop branch usually falls through only once.

For a loop that executes N times:

```
branch cycles = (N - 1) * 2 + 1
```

25.3 Baseline: One-Byte Copy Loop

The file `src/memcpy_naive.S` copies one byte per loop iteration.

Register interface:

```
R25:R24 destination pointer
R23:R22 source pointer
R20      byte count
```

Inner loop:

```
.Lnaive_loop:
    ld    r18, Z+           /* 2 cycles */
    st    X+, r18           /* 1 cycle */
    dec   r20               /* 1 cycle */
    brne  .Lnaive_loop      /* 2 taken, 1 final fall-through */
```

Taken-path loop cost:

$2 + 1 + 1 + 2 = 6$ cycles per byte

For len = 16, the branch is taken 15 times and falls through once:

```
15 taken iterations: 15 * 6 = 90
1 final iteration:   2 + 1 + 1 + 1 = 5
inner-loop total:    95 cycles
```

Full subroutine cost for len = 16 on ATtiny3217:

```
setup:      movw + movw + tst + breq(not taken) = 4
copy loop:   95
ret:         4
subroutine:  103 cycles
with RCALL:  105 cycles from the harness call site
```

The important baseline is the loop body: **95 cycles for 16 bytes**, or about 5.94 cycles per byte including the final branch fall-through.

25.4 Keep Hot Values in Registers

SRAM is much slower than a register. Direct SRAM access also costs flash because LDS and STS are two-word instructions on this device family.

Bad pattern for hot loop state:

```
.Lloop:
    lds   r18, counter
    inc   r18
    sts   counter, r18
    dec   r20
    brne  .Lloop
```

That loop repeatedly pays for direct SRAM access. If the value is only needed by this routine, keep it in a register:

```
    clr   r18
.Lloop:
    inc   r18
    dec   r20
    brne  .Lloop
    sts   counter, r18      /* store once if an observable result is needed */
```

The same rule applies to pointers and accumulators. Plan register use before writing the loop:

```
Routine: block_sum
Z        source pointer
r20      byte count
r18      loaded byte
r24      accumulator/result
```


r1 zero source for ADC carry propagation when needed

Example:

```
.global block_sum
.type    block_sum, @function

block_sum:
    movw  ZL, r22
    clr   r24
    tst   r20
    breq  .Lsum_done

.Lsum_loop:
    ld     r18, Z+
    add    r24, r18
    dec    r20
    brne   .Lsum_loop

.Lsum_done:
    ret
.size block_sum, .-block_sum
```

No SRAM is touched inside the loop. The accumulator stays live in r24 until the routine returns.

When registers run short, do not spill in the inner loop by reflex. First try:

- reusing argument registers once their original value is no longer needed
- shortening the lifetime of temporary values
- splitting a large loop into smaller named phases
- saving a register once at entry and restoring it once at exit

One PUSH/POP pair outside a hot loop is often much cheaper than LDS/STS inside every iteration.

25.5 Loop Unrolling

Each iteration of the naive copy loop spends 3 cycles on loop overhead:

```
DEC    = 1
BRNE   = 2 when taken
```

For a tiny loop body, that overhead is a large fraction of the total. Unrolling processes multiple bytes per branch.

The file `src/memcpy_fast.S` processes four bytes per block:

```
.Lfast_block:
    ld     r18, Z+           /* 2 */
    ld     r19, Z+           /* 2 */
    ld     r22, Z+           /* 2 */
    ld     r23, Z+           /* 2 */
    st     X+, r18            /* 1 */
    st     X+, r19            /* 1 */
    st     X+, r22            /* 1 */
    st     X+, r23            /* 1 */
    dec    r20                /* 1 */
    brne   .Lfast_block      /* 2 taken, 1 final */
```

Taken-path block cost:

```
4 loads + 4 stores + DEC + BRNE
= 8 + 4 + 1 + 2
= 15 cycles per 4 bytes
= 3.75 cycles per byte
```

For len = 16, there are four blocks:

```
3 taken blocks:      3 * 15 = 45
1 final block:      8 + 4 + 1 + 1 = 14
block-loop total:   59 cycles
```

The fast routine also has setup and tail handling:

```
setup before block loop: 9 cycles
block loop:              59 cycles
tail test, no tail:      3 cycles
ret:                     4 cycles
subroutine:              75 cycles
with RCALL:              77 cycles from the harness call site
```

Compare the hot copy work:

```
naive inner loop, len=16: 95 cycles
unrolled block loop:      59 cycles
speedup in copy loop:    95 / 59 = 1.61x
```

Compare the full call from the harness:

```
naive with RCALL:        105 cycles
fast with RCALL:         77 cycles
full-call speedup:       105 / 77 = 1.36x
```

Both numbers matter. The first tells you what happens on long buffers where setup fades away. The second tells you what this exact len = 16 benchmark costs.

25.6 Tail Handling

Unrolling by four creates a remainder problem. Lengths such as 13 or 17 do not divide evenly by four.

The fast routine computes:

```
mov    r21, r20
andi   r21, 0x03          /* tail = len & 3 */
lsr    r20
lsr    r20                /* blocks = len >> 2 */
breq   .Lfast_tail        /* len < 4 */
```

Then the block loop copies groups of four and the tail loop copies the last 0-3 bytes:

```
.Lfast_tail:
    tst    r21
    breq   .Lfast_done
    ld     r18, Z+
    st     X+, r18
    dec    r21
    brne   .Lfast_tail
```

This is the price of a general-purpose unrolled routine. The fast path is cheaper per byte, but the control flow is larger than the naive loop.

Unrolling is usually worth it when:

- the loop runs many times
- the useful work per iteration is small
- flash budget is available
- the tail path is rare or short

It is often not worth it when:

- the loop normally runs only a few times
- the loop body already contains expensive work
- the routine is cold startup code
- flash space is the limiting resource

25.7 Flash Lookup Tables

Some computations have a small input domain. If every possible input fits in one byte, a 256-entry table can replace a long calculation.

The file `src/lut_sin.S` contains a 256-entry unsigned sine table:

```
.section .progmem.data,"a",@progbits
.global sin_table
sin_table:
    .byte 128,131,134,137,140,143,146,149
    /* full 256-entry table in src/lut_sin.S */
.size sin_table, .-sin_table
```

The table value is:

```
round(127.5 + 127.5 * sin(n * 2 * pi / 256))
```

Equivalently, `round(255 * (1 + sin(n * 2 * pi / 256)) / 2)`. The full 0-255 range is used: amplitude 127.5 around an offset of 127.5.

Rounded to bytes, angle 0 gives 128, angle 64 gives 255, angle 128 gives 128, and angle 192 gives 0.

The lookup routine:

```
.global sin8
.type sin8, @function

sin8:
    ldi ZL, lo8(sin_table)
    ldi ZH, hi8(sin_table)
    add ZL, r24
    adc ZH, r1
    lpm r24, Z
    ret
.size sin8, .-sin8
```

Cycle count on ATtiny3217:

```
LDI + LDI + ADD + ADC + LPM + RET
= 1 + 1 + 1 + 1 + 3 + 4
= 11 cycles
```

With RCALL from the harness: 13 cycles

The table costs 256 flash bytes. The routine costs 12 flash bytes. For a value that is used repeatedly, that is often a good trade.

25.7.1 Why ADC ZH, r1 Works

The ADD ZL, r24 instruction may carry from the low byte of the table address into the high byte. ADC ZH, r1 adds that carry to ZH.

This requires r1 to be zero. The chapter harness clears r1 before calling the routines:

```
clr    r1
```

If a standalone assembly program uses r1 as a zero source, it must preserve that convention itself. There is no startup code in these examples to do it for you.

25.7.2 Flash Address Limits

LPM addresses the low 64 KB of program memory. The ATtiny3217 has 32 KB of flash, so ordinary LPM is sufficient. Larger AVR devices may require ELPM and RAMPZ for tables above the low 64 KB region.

25.7.3 Verifying Table Placement

The default AVR linker script places program-memory input sections into the final flash image. In the linked ELF, inspect the final .text output section:

```
avr-nm -n bench.elf | grep sin_table
avr-objdump -s -j .text bench.elf
```

Do not assume that an input section name such as .progmem.data remains a separate output section after linking.

25.8 Interpolating Between Table Entries

The 256-entry sin_table spends 256 flash bytes to give an exact answer for every input. Most smooth functions do not need that. If the curve changes slowly, a coarse table plus a little arithmetic between entries reaches almost the same accuracy for a fraction of the flash.

The idea is linear interpolation. Split the input into two parts:

```
i  the index of the table entry at or below the input
f  how far the input lies between entry i and entry i+1, as a
   Q0.8 fraction (0 = exactly at T[i], 255 ~= just below T[i+1])
```

The interpolated value is the straight-line estimate between the two entries:

$$y = T[i] + (T[i+1] - T[i]) * f / 256$$

The file src/lut_interp.S applies this to the same sine curve, but stores only every 16th sample. That is 17 entries instead of 256:

```
.section .progmem.data,"a",@progbits
.global interp_table
interp_table:
    /* sin_table[0], [16], [32], ... [240], then wrap [256]=[0]=128 */
    .byte 128,176,218,246,255,246,218,176,128,79,37,9,0,9,37,79,128
.size interp_table,.-interp_table
```

The 17th entry (the wrap back to 128) is not decoration: it lets the routine read both $T[i]$ and $T[i+1]$ for any i in $0..15$ without a bounds check.

The lookup routine takes the index in R24 and the fraction in R22:

```
00000238 <interp8>:
238:  ec e7          ldi     r30, 0x7C          ; Z = interp_table (low 64K)
23a:  f1 e0          ldi     r31, 0x01
23c:  e8 0f          add     r30, r24          ; Z += i
23e:  f1 1d          adc     r31, r1          ; carry into ZH (r1 = 0)
240:  45 91          lpm     r20, Z+          ; r20 = T[i], Z -> T[i+1]
242:  54 91          lpm     r21, Z          ; r21 = T[i+1]
244:  54 1b          sub     r21, r20          ; r21 = delta = T[i+1] - T[i]
246:  56 03          mulsu   r21, r22          ; r1:r0 = delta(signed) * f(uns)
248:  41 0d          add     r20, r1          ; T[i] + ((delta*f) >> 8)
24a:  11 24          eor     r1, r1          ; clr r1 (MUL dirtied it)
24c:  84 2f          mov     r24, r20          ; return value
24e:  08 95          ret
```

The one subtle instruction is MULSU. The fraction f is always positive ($0..255$), but δ can be negative where the curve falls. MULSU multiplies a *signed* register by an *unsigned* one, which is exactly this case: a signed δ scaled by an unsigned fraction. The high byte of the product, R1, is the correction $(\delta * f) \gg 8$, carrying the right sign. Both MULSU operands must be registers r16..r23; r21 and r22 satisfy that.

This works only because consecutive table entries differ by no more than a signed byte can hold. For this table the steepest step is $128 - 79 = 49$, well inside $-128..127$. A table whose neighbouring entries can differ by more than 127 would overflow the SUB and need a 16-bit δ .

Cycle and space comparison against the dense table:

	sin8 (dense)	interp8 (coarse + interp)
Flash table	256 bytes	17 bytes
Routine	12 bytes	24 bytes
Cycles (no call)	11	20
Accuracy	exact	approximate

The harness calls `interp8` with $i = 0$, $f = 128$, the midpoint of the first segment:

```
T[0] = 128, T[1] = 176, delta = +48
y = 128 + (48 * 128) >> 8 = 128 + (6144 >> 8) = 128 + 24 = 152
```

It stores 152 in `interp_sample` for inspection. The true table value at that angle (`sin_table[8]`) is also 152 here, but interpolation is not generally exact: at a point where the curve bends, expect an error of a few least significant bits. That error is the price of the 15-fold shrink in table size.

A note on rounding. Taking the high byte of the product is a floor: it discards the low 8 fractional bits and so always rounds the correction down, biasing the result low by up to one count. If that bias matters, add $0x80$ to the 16-bit product before the shift (round to nearest); the extra ADD/ADC costs two cycles.

25.9 Horner's Method for Polynomials

Some functions are approximated not by a table but by a polynomial:

```
p(x) = c2*x^2 + c1*x + c0
```

Evaluated literally, that computes x^2 , two multiplies, and two adds, and it forms the large intermediate x^2 before scaling it back down. Horner's method rewrites the polynomial

so each term is reached by one multiply and one add:

$$p(x) = (c2*x + c1)*x + c0$$

A degree- n polynomial becomes n multiply-add steps with no separate powers. That is fewer multiplies, fewer temporaries, and in fixed-point arithmetic less rounding error, because the value never grows larger than it has to.

The file `src/horner.S` evaluates $2*x^2 + 3*x + 5$ for an unsigned 8-bit x , keeping a 16-bit accumulator. The coefficients live in flash, most-significant first, and stream through one loop with LPM $Z+$:

```
.section .progmem.data,"a",@progbits
poly_coeffs:
    .byte 2, 3, 5                                /* c2, c1, c0  -> 2*x^2 + 3*x + 5 */

00000250 <horner8>:
250:  ed e8      ldi     r30, 0x8D          ; Z = poly_coeffs
252:  f1 e0      ldi     r31, 0x01
254:  68 2f      mov     r22, r24          ; r22 = x (the multiplier)
256:  25 91      lpm     r18, Z+           ; acc low = c2
258:  33 27      eor     r19, r19         ; acc high = 0  ->  acc = c2
25a:  42 e0      ldi     r20, 0x02         ; two coefficients remain

0000025c <.Lhorner_loop>:
25c:  26 9f      mul     r18, r22          ; r1:r0 = accL * x
25e:  c0 01      movw    r24, r0           ; R25:R24 = accL*x
260:  36 9f      mul     r19, r22          ; r1:r0 = accH * x
262:  90 0d      add     r25, r0           ; fold (accH*x)<8 into high byte
264:  11 24      eor     r1, r1            ; clr r1 (product consumed)
266:  05 90      lpm     r0, Z+           ; r0 = next coefficient
268:  80 0d      add     r24, r0           ; acc low += coeff
26a:  91 1d      adc     r25, r1           ; acc high += carry (r1 = 0)
26c:  9c 01      movw    r18, r24          ; acc = acc*x + c
26e:  4a 95      dec     r20
270:  a9 f7      brne    .-22             ; back to .Lhorner_loop
272:  c9 01      movw    r24, r18          ; return acc in R25:R24
274:  08 95      ret
```

The inner step is $\text{acc} = \text{acc} * x + c$. Because acc is 16 bits and x is 8, the multiply is done in two halves: $\text{accL} * x$ gives the low partial product, and $\text{accH} * x$ contributes its low byte one position up. Only the low 16 bits of the product are kept, so the polynomial must be chosen so $p(x)$ fits in 16 bits over the input range you care about.

The harness calls `horner8` with $x = 12$:

```
acc = c2                = 2
acc = acc*x + c1 = 2*12 + 3 = 27
acc = acc*x + c0 = 27*12 + 5 = 329    ; = 0x0149
```

It stores 329 (low byte first) in `horner_sample`. Cross-check against the direct form: $2*144 + 36 + 5 = 329$.

Cost on the ATtiny3217:

Setup	8 cycles
Loop body	14 cycles * 2 iterations
Branch	+2 taken, +1 final fall-through
Return	movw + ret = 5 cycles
Total	44 cycles (excluding the RCALL from the caller)

For fixed-point work the coefficients and the accumulator carry an implied binary point, and each multiply is followed by a shift to rescale (see the fixed-point section of Chapter

26). Horner's form keeps that rescaling to one shift per term and avoids ever forming a high power that would overflow. Where a function needs trigonometry rather than a short polynomial, compare this with the table methods above and with the CORDIC routines in Chapter 29.

25.10 Size Results from the Current Sources

With the current chapter files, `avr-nm --size-sort --print-size bench.elf` reports the relevant symbols as:

```
0000018d 00000003 T poly_coeffs
0000022c 0000000c T sin8
00803820 00000010 B dst_fast
00803810 00000010 B dst_naive
00803800 00000010 B src_data
0000017c 00000011 T interp_table
000001e6 00000012 T memcpy_naive
00000238 00000018 T interp8
00000250 00000026 T horner8
000001f8 00000034 T memcpy_fast
00000192 00000054 T main
0000007c 00000100 T sin_table
```

Meaning:

Symbol	Size	Meaning
poly_coeffs	3 bytes	Flash coefficients for horner8.
sin8	12 bytes	Dense lookup routine only.
interp_table	17 bytes	Coarse 17-entry table used by interp8.
memcpy_naive	18 bytes	Smallest copy routine.
interp8	24 bytes	Interpolated lookup routine.
horner8	38 bytes	Polynomial evaluation routine.
memcpy_fast	52 bytes	Faster for larger lengths, 34 bytes larger.
main	84 bytes	Harness driver calling every routine.
sin_table	256 bytes	Flash table used by sin8.
src_data	16 bytes	SRAM source buffer for the harness.
dst_naive	16 bytes	SRAM destination for naive copy.
dst_fast	16 bytes	SRAM destination for unrolled copy.
sin_sample	1 byte	SRAM result of one sin8 lookup.
interp_sample	1 byte	SRAM result of one interp8 lookup.
horner_sample	2 bytes	SRAM result of one horner8 evaluation.

Speed is not free, and neither is space. The unrolled copy routine saves cycles by spending 34 extra flash bytes. The dense sine lookup spends 256 flash bytes to replace a long calculation with one table load. Going the other way, interp8 gives back most of that flash — 17 table bytes instead of 256 — by spending cycles and a little accuracy. horner8 shows the table-free end of the range: 38 bytes of code and three coefficients evaluate a

polynomial with no table at all. Which trade is right depends on whether flash, cycles, or accuracy is the scarce resource.

25.11 Reading avr-objdump Correctly

Example disassembly:

```
000001c4 <.lnaive_loop>:
1c4: 21 91      ld      r18, Z+
1c6: 2d 93      st      X+, r18
1c8: 4a 95      dec     r20
1ca: e1 f7      brne    .-8
```

Read it as:

```
1c4      byte address in flash
21 91    encoded instruction bytes, little-endian in the listing
ld ...   decoded instruction
```

AVR instructions are stored as 16-bit words, but the ELF view uses byte addresses. Most instructions advance by 2 bytes. Two-word instructions such as STS, LDS, CALL, and JMP advance by 4 bytes.

Use these commands often:

```
avr-objdump -d -S bench.elf
avr-objdump -h bench.elf
avr-nm -n bench.elf
avr-nm --size-sort --print-size bench.elf
```

If a symbol reports an unexpected size, check that the source has matching `.type` and `.size` directives.

25.12 Common Errors in Hand Optimisation

25.12.1 Optimising Cold Code

Speed work belongs on hot paths. A routine that runs once during setup is rarely worth making larger or harder to inspect. Use a timer, GPIO toggle, logic analyzer, or debugger stop points to identify where time is actually spent.

25.12.2 Ignoring Setup Cost

An unrolled loop can be faster in steady state but slower for tiny lengths. Do both calculations:

```
steady-state loop cost
full routine cost including setup, tail, and return
```

For `len = 16`, the unrolled copy is still faster. For very short lengths, the naive loop may win.

25.12.3 Forgetting the Final Branch

The last conditional branch usually costs one cycle, not two. Count taken and not-taken paths separately.

25.12.4 Using a Zero Register Without Clearing It

The lookup example uses `r1` as a zero source. In a fully standalone assembly program, clear it yourself before relying on it:

```
clr r1
```

If a routine temporarily uses `r1` for some other purpose, restore it to zero before returning to code that expects the zero-register convention.

25.12.5 Putting Hot State in SRAM

Repeated LDS/STS inside a tight loop is often the first thing to remove. Keep counters, pointers, accumulators, masks, and temporary bytes in registers for as long as their live ranges allow.

25.12.6 Unrolling Until the Source Becomes Fragile

Unrolling `x2` or `x4` is often manageable. Unrolling `x8` or `x16` can become hard to audit, especially when there is a tail path, status flags, or overlapping source and destination ranges. Code that cannot be reviewed reliably is not a professional optimisation.

25.13 Optimisation Checklist

Use this sequence before changing a routine:

1. Mark the exact hot loop or hot path.
2. Build the ELF.
3. Disassemble with `avr-objdump -d -S`.
4. Count cycles for the current path.
5. Record flash size for the current routine.
6. Apply one change.
7. Rebuild and recount.
8. Check flash size again.
9. Keep the change only if the gain is worth the size and complexity.

For AVR assembly, this discipline works well because the machine is small and the instruction timings are documented.

25.14 Source Notes

Facts in this chapter are based on:

- Microchip AVR Instruction Set Manual DS40002198C, especially the AVRxt cycle timing column used by ATtiny3217-class devices.
 - Microchip ATtiny3217 documentation for the 32 KB flash size that makes plain LPM sufficient for the chapter's lookup table.
 - The local linked ELF produced from `src/bench_harness.S`, `src/memcpy_naive.S`, `src/memcpy_fast.S`, `src/lut_sin.S`, `src/lut_interp.S`, and `src/horner.S`. The disassembly listings and symbol sizes shown above are copied from that ELF.
-

25.15 Exercises

1. Count the exact full-call cycle cost of `memcpy_naive` for `len = 8`, including `RCALL`, setup, loop, and `RET`.
2. Count the exact full-call cycle cost of `memcpy_fast` for `len = 8`. Does the unrolled routine still win?
3. Modify `memcpy_fast` to unroll by eight instead of four. Measure the new symbol size and calculate the new loop cost for `len = 64`.
4. Write `block_sum`, an assembly routine that sums `len` bytes from `Z` into `r24`. Compare the cycle count before and after unrolling by two.
5. Change the sine table to a 64-entry quarter-wave table and reconstruct the quadrant in code. How many flash bytes are saved, and how many cycles are added?
6. Use GDB or MDB to stop at `bench_done` and inspect `dst_naive`, `dst_fast`, `sin_sample`, `interp_sample`, and `horner_sample`. Explain what each byte proves about the benchmark.
7. Call `interp8` at several fractions across one segment and compare each result against the dense `sin_table`. Where in the segment is the interpolation error largest, and why?
8. Add a rounding term (`+0x80` before the shift) to `interp8`. Show the two extra instructions in the disassembly and confirm the cycle cost rises by two.
9. Extend `horner8` to a degree-3 polynomial by adding one coefficient. How many extra cycles does the third multiply-add cost, and does the routine's loop need any change at all?

Next: Chapter 24 — UPDI — The Unified Program and Debug Interface

Chapter 26

UPDI — The Unified Program and Debug Interface

Every program you write in this book eventually ends up on the ATtiny3217 as bytes in flash. UPDI is the hardware interface that puts those bytes there. It also provides the debugging channel that chapter 3A uses. Understanding UPDI from first principles — the wire protocol, the instruction set, the key mechanism, the register map — gives you a complete picture of what happens between typing avrdude and seeing your LED blink.

This chapter covers everything from the physical layer through the full programming sequence:

1. What UPDI is, and how it differs from the ISP interface it replaced.
2. The single-wire UART physical layer: frame formats, baud rate, guard time.
3. Three ways to enable UPDI on the RESET pin.
4. The eight UPDI instructions and their wire transactions.
5. Key-protected operations: chip erase, NVM programming, user-row write.
6. The full UPDI register map with every bit explained.
7. Hardware adapters: jtag2updi (legacy), SerialUPDI (diode or resistor method), and the on-board nEDBG on the Curiosity Nano.
8. Software tools: avrdude and pymcuprog command references.
9. On-chip debugging (OCD) capabilities exposed through UPDI.
10. Troubleshooting: error signatures, contention, fuse lock-out, HV recovery.

26.1 ISP vs UPDI: Why Microchip Changed the Interface

Classic AVR devices (ATmega328P and earlier) use the **In-System Programming (ISP)** interface for programming. ISP is SPI-based and requires four dedicated pins: SCK, MOSI, MISO, and RESET. Programming is synchronous — the host clocks every bit.

Modern AVR^s (tinyAVR 0/1/2-series, megaAVR 0-series, AVR DA/DB/DD/EA-series) released since 2016 use **UPDI** instead:

Feature	ISP (ATmega328P)	UPDI (ATtiny3217)
Pins used	SCK + MOSI + MISO + RST	One pin: RESET/PA0
Protocol	SPI (4-wire synchronous)	Half-duplex async UART
Baud detection	Fixed clock from host	Auto-baud from SYNCH char
Breakpoints	2 hardware only	2 hardware + unlimited software
Memory access	Sequential only	Full memory map random access

Fuse locking	SPIEN fuse	RSTPINCFG + HV override
OCD support	debugWIRE (separate)	Built-in, same pin
Max baud rate	1 MHz	900 kbps (@ 5 V)

UPDI consolidates programming and debugging onto one pin with no clock line, no separate MISO/MOSI, and no dedicated SPI peripheral required in the programmer.

26.2 The Physical Layer

26.2.1 The Single UPDI Pin

On the ATtiny3217, PA0 is a three-function pin labelled PA0/RESET/UPDI in the datasheet (physical pin 16 on the 20-SOIC package; pin 23 on the 24-VQFN). Which function is active depends on the RSTPINCFG bits in FUSE.SYSCFG0:

RSTPINCFG[1:0] PA0 pin function

0x0	GPIO – PA0 is general-purpose I/O; UPDI inaccessible without HV
0x1	UPDI – factory default; UPDI takes over the pin
0x2	RESET – external reset input; UPDI inaccessible without HV
Other	Reserved

Factory default: SYSCFG0 resets to 0xC4, placing RSTPINCFG[1:0] at 0b01 (0x1 = UPDI mode). A freshly manufactured ATtiny3217 always comes up in UPDI mode.

The UPDI pin has a constant internal pull-up when UPDI is active. The protocol is **half-duplex, single-wire, asynchronous UART** with:

- 8 data bits
- 1 parity bit (even parity, can be disabled)
- 2 stop bits
- Auto-baud (the baud rate is measured from each SYNCH character)
- No baud rate register — the programmer drives timing entirely

26.3 Frame Formats

The UPDI recognises five types of frames on the wire:

Frame	Bit pattern	Meaning
DATA	[St][D0..D7][P][S1][S2]	Normal 8-bit data byte
IDLE	12 high bits	Line idle / gap
BREAK	12 low bits	Reset UPDI to default state
SYNCH	DATA frame with value 0x55	Set baud rate for next transaction
ACK	DATA frame with value 0x40	Target confirms successful ST/STS

Every DATA frame: Start bit (low), 8 data bits LSB first, 1 even parity bit, 2 stop bits (high). Total: 12 bit-periods per byte.

26.3.1 BREAK Character

A BREAK is 12 consecutive low bits — the line held low for 12 bit-periods. BREAK resets the UPDI internal state machine back to idle and clears any error condition. It also resets the UPDI oscillator to the 4 MHz default.

To guarantee detection in all cases, the programmer sends **two consecutive BREAK frames**. The first BREAK may be missed if the UPDI is mid-transaction, but it will cause

a framing or contention error that aborts the transaction; the second BREAK is then detected cleanly.

Minimum BREAK duration at each clock setting:

UPDI CLKDIV	UPDI Clock	Minimum BREAK Duration
0x3 (default)	4 MHz	24.60 ms
0x2	8 MHz	12.30 ms
0x1	16 MHz	6.15 ms

26.3.2 SYNCH Character (0x55)

The SYNCH byte 0x55 has alternating 0/1 bits. The UPDI measures the time between transitions to derive the baud rate used by the programmer. This baud rate is then used for all subsequent frames in the transaction.

Every new instruction must be preceded by a SYNCH. The only exception is when using REPEAT: only the first instruction after a REPEAT needs a SYNCH.

26.3.3 Guard Time

After the programmer sends an instruction, the UPDI must change direction from receive to transmit before it can reply. The guard time is a configurable number of IDLE bits inserted before the first reply byte:

GTVAL[2:0] Guard Time (UPDI clock cycles)

0x0 (default)	128 cycles
0x1	64 cycles
0x2	32 cycles
0x3	16 cycles
0x4	8 cycles
0x5	4 cycles
0x6	2 cycles
0x7	Reserved

The programmer must wait for at least the guard time before sampling the reply. In practice, tools leave at least two guard-time cycles from the programmer side as well.

26.3.4 Baud Rate Limits

UPDI baud rate limits depend on VDD:

VDD Range	Max Baud Rate
1.8 V – 5.5 V	225 kbps
2.2 V – 5.5 V	450 kbps
2.7 V – 5.5 V	900 kbps

Practical tools typically use 115,200 or 230,400 bps for maximum compatibility. The ATtiny3217 Curiosity Nano's on-board programmer uses 225 kbps by default.

26.4 UPDI Enable Sequences

UPDI must be enabled before communication can begin. There are two methods.

26.4.1 Standard Enable (RSTPINCFG = 0x1)

When RSTPINCFG = 0x1 (factory default), PA0 is dedicated to UPDI and carries a constant pull-up. The programmer drives the pin low for at least 200 ns to trigger the enable handshake:

Programmer	UPDI Pin	UPDI State
Pull-up active		Pin held high by pull-up
Drive pin LOW (hold $\geq$ 200 ns)		Edge detector triggers
Release pin (Hi-Z)		UPDI holds low while clock starts
	← UPDI releases → Pin goes HIGH	
Send 0x55 SYNCH (within 16.4 ms of pin going high)		Clock ready, UPDI waiting Baud rate locked, UPDI enabled
First instruction		Ready to accept instructions

If SYNCH is not sent within 16.4 ms (65536 UPDI clock cycles at 4 MHz), UPDI disables itself and the sequence must restart.

Timing parameters (from Electrical Characteristics, §36.21):

Symbol	Description	Min	Max	Unit
TRES	Duration of Handshake/BREAK	10	200	μ s
TUPDI	Duration of UPDI.txd = 0	10	200	μ s
TDeb0	Duration of Debugger.txd = 0	200 ns	1	μ s
TDebZ	Duration of Debugger.txd = z	200	14000	μ s

26.4.2 High-Voltage (HV) Override

When RSTPINCFG = 0x0 (GPIO) or 0x2 (RESET), UPDI is inaccessible without HV. Applying a 12 V pulse to PA0 switches the pin to UPDI mode regardless of fuse settings. This is the only recovery path from GPIO mode.

Step	Action
1	Reset the device (recommended before HV sequence).
2	Apply 12 V pulse to PA0; hold for 100 μ s–1 ms, then tri-state.
3	Hold HV for $\geq$ 200 μ s, $\leq$ 14 ms.
4	Remove HV (tri-state).
5	Programmer drives PA0 LOW to release UPDI reset.
6	Release PA0 (Hi-Z); UPDI drives LOW until clock ready.
7	Wait for PA0 to go HIGH (UPDI clock ready).
8	Send SYNCH (0x55) within TDebZ.
9	Send NVMPROG key immediately after first SYNCH.
10	Program device; then write UPDIDIS to disable UPDI.

After an HV-enable session, the RESET pin remains in UPDI configuration until a Power-on Reset (POR). Issuing UPDIDIS does not restore PA0 to GPIO — only POR does. This means HV enables temporary UPDI access without permanently changing the fuse.

HV-specific timing (from figure 33-6 in the datasheet):

Symbol	Description	Min	Max	Unit
THV_ramp	HV rise time	10	4000	ns/ μ s (10 ns–4 ms)
TRES	HV pulse hold duration	10	200	μ s
TUPDI	UPDI holds pin low after HV edge	10	200	μ s
TDeb0	Programmer drives pin LOW	200 ns	1	μ s

TDebZ	Pin-high to first SYNCH start	200	14000	μs
-------	-------------------------------	-----	-------	----

26.5 UPDI Disabling

End every UPDI session by writing `UPDIDIS = 1` in `UPDI.CTRLB`. This:

- Issues a system reset, returning the CPU to the Run state.
- Lowers the UPDI clock request, reducing power consumption.
- Clears all UPDI KEYs and configuration.

If the session ends without disabling UPDI, the oscillator request remains active, causing higher idle power consumption.

26.6 The UPDI Instruction Set

The UPDI has eight instructions. Every instruction must be preceded by a SYNCH character (except instructions inside a REPEAT sequence after the first). The instruction is a single byte encoding an opcode and size fields.

Instruction	Opcode Bits	Purpose
LDS	0b00_aaaa_bb	Load (read) from data space, direct address
STS	0b01_aaaa_bb	Store (write) to data space, direct address
LD	0b00_1pp_aa_bb	Load from data space, indirect via pointer
ST	0b01_1pp_aa_bb	Store to data space, indirect via pointer
LDCS	0b10_0_ccccc	Load from UPDI CS register space
STCS	0b11_0_ccccc	Store to UPDI CS register space
REPEAT	0b10_00001_bb	Set repeat counter
KEY	0b11_00_s_c_1	Send key or receive SIB

Size fields:

Size A (address)	0x0 = 1 byte	0x1 = 2 bytes	0x2 = 3 bytes (>64 KB)
Size B (data)	0x0 = 1 byte	0x1 = 2 bytes	
CS Address	5-bit field, selects UPDI CS register (0x00–0x0C)		

26.6.1 LDS — Load Data Space, Direct Address

Read a byte or word from any address in the data space.

Wire sequence:

```
Programmer → UPDI: [SYNCH][LDS opcode][addr_lo][addr_hi]
UPDI → Programmer: (guard time) [data_byte]
```

If Size A = 2 bytes, addr is 2 bytes (covers tinyAVR ≤ 64 KB data space).

Example: read `PORTB.IN` at data address 0x0428 (`PORTB` base 0x0420 + `IN` offset 0x08):

Send: 0x55 0x04 0x28 0x04

Recv: <value of `PORTB.IN`>

Note: 0x04 is LDS with Size A = word (2 bytes), Size B = byte.

26.6.2 STS — Store Data Space, Direct Address

Write a byte or word to any address in the data space.

Wire sequence:

```

Programmer → UPDI: [SYNCH][STS opcode][addr_lo][addr_hi]
UPDI → Programmer: (guard time) [ACK 0x40]
Programmer → UPDI: [data_byte]
UPDI → Programmer: (guard time) [ACK 0x40]

```

The double-ACK confirms both the address phase and the data phase.

26.6.3 LD — Load Data Space, Indirect (via Pointer Register)

First write the pointer register with an ST to pointer address, then use LD with pointer post-increment to stream data. Most useful with REPEAT.

```

Set pointer: SYNCH + ST(ptr) + addr_lo + addr_hi → ACK
Read bytes: SYNCH + LD(*(ptr++)) → data_0, data_1, ..., data_n

```

26.6.4 ST — Store Data Space, Indirect (via Pointer Register)

Write the pointer register, then stream bytes to consecutive addresses. Most useful for writing flash page buffers.

```

Set pointer: SYNCH + ST(ptr) + addr_lo + addr_hi → ACK
Write bytes: SYNCH + ST(*(ptr++)) + data_0 → ACK
              (REPEAT handles subsequent bytes without extra SYNCH)

```

26.6.5 LDCS — Load UPDI Control/Status Register

Read one byte from the UPDI internal CS register space (not the device memory map). Used to check UPDI status, error signatures, and ASI state.

```

Wire sequence:
Programmer → UPDI: [SYNCH][LDCS opcode with 5-bit CS address]
UPDI → Programmer: (guard time) [register_value]

```

26.6.6 STCS — Store UPDI Control/Status Register

Write one byte to a UPDI internal CS register. No ACK is generated.

```

Wire sequence:
Programmer → UPDI: [SYNCH][STCS opcode with 5-bit CS address][data_byte]

Common uses: set GTVAL, enable IBDLY, write UPDIDIS to end session, change clock
speed via ASI_CTRLA.

```

26.6.7 REPEAT — Set Repeat Counter

Load a repeat count N. The next instruction executes N+1 times, with the overhead of SYNCH and opcode omitted after the first iteration.

```

Wire sequence:
Programmer → UPDI: [SYNCH][REPEAT opcode][repeat_count]
Programmer → UPDI: [SYNCH][next instruction + operands]
                  [data_1][data_2]...[data_N] (no SYNCH between repeats)

```

Maximum repeat count: 255 (giving 256 total executions). Only LD and ST with pointer post-increment (*(ptr++)) make sense with REPEAT. To abort a REPEAT in progress, send a BREAK.

Example: write 128 bytes to the flash page buffer for page 0, starting at 0x8000 (mapped flash start in data space on the ATtiny3217):

1. SYNCH + ST(ptr) + 0x00 + 0x80 → ACK (set pointer to 0x8000)
2. SYNCH + REPEAT + 0x7F (repeat 127 more times = 128 total)
3. SYNCH + ST(*(ptr++)) + byte_0 → ACK
 - byte_1 → ACK
 - byte_2 → ACK
 - ...
 - byte_127 → ACK

26.6.8 KEY — Send Activation Key or Read SIB

Unlock a protected feature by sending an 8-byte (64-bit) key. Also used to request the 16-byte System Information Block.

Wire sequence (send key):

Programmer → UPDI: [SYNCH][KEY opcode][key_7][key_6]...[key_0]
 (No response generated – check ASI_KEY_STATUS with LDCS to confirm.)

Wire sequence (request SIB):

Programmer → UPDI: [SYNCH][KEY opcode with SIB=1]
 UPDI → Programmer: (guard time) [sib_0][sib_1]...[sib_15]

Available keys (bytes transmitted LSB first):

Key Name	Hex Signature (byte 0 first)	Size
Chip Erase	4E 56 4D 45 72 61 73 65	64-bit
NVMPROG	4E 56 4D 50 72 6F 67 20	64-bit
USERROW-Write	4E 56 4D 55 73 26 74 65	64-bit

The System Information Block (SIB) identifies the device to the programmer:

Byte Range	Field	Content
[0–6]	Family_ID	ASCII, e.g. "tinyAVR" (7 bytes)
[7]	Reserved	–
[8–10]	NVM_VERSION	NVM controller version (3 bytes)
[11–13]	OCD_VERSION	OCD controller version (3 bytes)
[14]	Reserved	–
[15]	DBG_OSC_FREQ	Debug oscillator frequency class

26.7 Key-Protected Operations

Three operations require keys: chip erase, NVM programming, and user-row write. All three follow the same structure: send key → reset → poll status → perform operation → reset again.

26.7.1 Chip Erase

Chip erase clears all flash and lock bits. EEPROM is also erased unless the EESAVE fuse (SYSCFG0 bit 0) is set — except on a locked device, where EEPROM is always erased regardless of EESAVE. The User Row is never affected by chip erase. Chip erase is the only way to unlock a locked device.

Step Action

1. Send Chip Erase key with KEY instruction.
2. (Optional) LDCS ASI_KEY_STATUS → confirm CHIPERASE bit = 1.
3. STCS ASI_RESET_REQ ← 0x59 (assert system reset)

4. STCS ASI_RESET_REQ ← 0x00 (release system reset)
5. LDCS ASI_SYS_STATUS → poll until LOCKSTATUS = 0.
6. (Optional) LDCS ASI_SYS_STATUS → check ERASE_FAILED = 0 if present.
Note: the ERASE_FAILED bit appears in the chip erase procedure in the datasheet text (§33.3.8.1) but is absent from the ASI_SYS_STATUS register description. Checking LOCKSTATUS = 0 (step 5) is the reliable completion test.
7. Done. Device is unlocked; full UPDI bus access is restored.

Caution: the BOD is forced on during chip erase. If VDD is below the BOD threshold the erase will not complete.

26.7.2 NVM Programming

After chip erase (or on an already-unlocked device):

Step Action

-
1. (If locked) perform Chip Erase first.
 2. Send NVMPROG key with KEY instruction.
 3. (Optional) LDCS ASI_KEY_STATUS → confirm NVMPROG bit = 1.
 4. STCS ASI_RESET_REQ ← 0x59 (assert system reset; halts CPU)
 5. STCS ASI_RESET_REQ ← 0x00 (release system reset)
 6. LDCS ASI_SYS_STATUS → poll until NVMPROG bit = 1.
 7. Write data to flash via STS / ST + REPEAT targeting mapped flash.
(Use NVMCTRL commands: see Chapter 25 for the NVMCTRL write flow.)
 8. STCS ASI_RESET_REQ ← 0x59 (assert reset to end programming)
 9. STCS ASI_RESET_REQ ← 0x00 (release reset)
 10. Programming complete. UPDI regains access; CPU is running.

Writing to flash via UPDI follows exactly the NVMCTRL sequence from Chapter 25 (page-buffer fill → PAGEERASEWRITE command), except the CPU is halted and the programmer drives every store.

26.7.3 User Row Programming on a Locked Device

The USERROW is a 64-byte area of non-volatile memory distinct from flash. It can be updated even when the device lock bits are set — the device can be field-updated without revealing the program.

Step Action

-
1. Send USERROW-Write key with KEY instruction.
 2. (Optional) LDCS ASI_KEY_STATUS → confirm UROWWRITE bit = 1.
 3. STCS ASI_RESET_REQ ← 0x59 / 0x00 (reset cycle)
 4. LDCS ASI_SYS_STATUS → poll until UROWPROG = 1.
 5. Write 64 bytes of data to the first 64 bytes of RAM (address 0x3800, the start of SRAM on the ATtiny3217) using ST + REPEAT.
(Only the first 64 bytes of RAM are accessible; writes outside this range are silently ignored.)
 6. STCS ASI_SYS_CTRLA ← UROWWRITE_FINAL = 1 (commit RAM → user row)
 7. LDCS ASI_SYS_STATUS → poll until UROWPROG = 0.
 8. STCS ASI_KEY_STATUS ← UROWWRITE = 1 (invalidate key)
 9. STCS ASI_RESET_REQ ← 0x59 / 0x00 (final reset)
 10. User row programming complete.
-

26.8 The UPDI Register Map

UPDI registers live in an internal CS (Control/Status) space addressed by the 5-bit CS field in LDCS/STCS instructions, not in the normal I/O address space. The CPU cannot read them.

CS Addr	Register Name	Reset	Access	Description
0x00	STATUSA	0x10	R	Status A (UPDI revision)
0x01	STATUSB	0x00	R	Status B (error signature)
0x02	CTRLA	0x00	R/W	Control A (timing, parity)
0x03	CTRLB	0x00	R/W	Control B (disable UPDI)
0x04–06	Reserved	—	—	—
0x07	ASI_KEY_STATUS	0x00	R/W	Key activation status
0x08	ASI_RESET_REQ	0x00	R/W	System reset request
0x09	ASI_CTRLA	0x03	R/W	UPDI clock select
0x0A	ASI_SYS_CTRLA	0x00	R/W	System control (CLKREQ, UROWDONE)
0x0B	ASI_SYS_STATUS	0x01	R	System status
0x0C	ASI_CRC_STATUS	0x00	R	CRC check result

26.8.1 STATUSA (0x00) — Status A

Bit	Name	Reset	Access	
[7:4]	UPDIREV[3:0]	0x1	R	UPDI implementation revision
[3:0]	—	—	—	Reserved

The ATtiny3217 returns UPDIREV = 1 (reset value 0x10 = bits[7:4] = 1).

26.8.2 STATUSB (0x01) — Status B

Bit	Name	Reset	Access	
[2:0]	PESIG[2:0]	0x0	R	Error signature (cleared on read)

PESIG value Error type

0x0	No error (default)
0x1	Parity error (wrong parity bit sampled)
0x2	Frame error (wrong stop bits sampled)
0x3	Access layer time-out (ACC layer no response)
0x4	Clock recovery error (wrong start bit sampled)
0x5	Reserved
0x6	Bus error (address or access privilege violation)
0x7	Contention error (both sides driving the line)

When an error occurs, read STATUSB after a BREAK + SYNCH to find out what went wrong before retrying.

26.8.3 CTRLA (0x02) — Control A

Bit	Name	Reset	Access	Description
7	IBDLY	0	R/W	Inter-Byte Delay Enable. 1 = insert 2 IDLE bits between bytes in multi-byte LD(S) reads. Prevents overflow when the host processes data slowly.
6	—	—	—	Reserved
5	PARD	0	R/W	Parity Disable. 1 = ignore parity bit.

4	DTD	0	R/W	Use only for testing. Disable Time-Out Detection. 1 = disable 65536-cycle ACC-layer timeout.
3	RSD	0	R/W	Response Signature Disable. 1 = suppress ACK generation (faster bulk NVM writes when timing is known).
[2:0]	GTVAL	0x0	R/W	Guard Time Value (see table above).

26.8.4 CTRLB (0x03) — Control B

Bit	Name	Reset	Access	Description
4	NACKDIS	0	R/W	Disable NACK during system reset.
3	CCDETDIS	0	R/W	Disable contention detection.
2	UPDIDIS	0	R/W	Write 1 to disable UPDI, reset device, clear all keys and configuration.

26.8.5 ASI_KEY_STATUS (0x07)

Bit	Name	Reset	Access	Description
5	UROWWRITE	0	R/W	USERROW-Write key decoded and active. Write 1 to invalidate key after user row programming.
4	NVMPROG	0	R	NVMPROG key decoded. Cleared when NVMPROG mode starts.
3	CHIPERASE	0	R	Chip Erase key decoded. Cleared by the reset issued in erase sequence.

26.8.6 ASI_RESET_REQ (0x08)

Bit	Name	Reset	Access
[7:0]	RSTREQ	0x00	R/W

Value	Action
-------	--------

0x59	Assert system reset (CPU halted, peripherals running)
0x00	Release system reset (CPU resumes)
Other	Clears reset condition

The UPDI itself is NOT reset by this register — it continues running so the programmer can poll status.

26.8.7 ASI_CTRLA (0x09)

Bit	Name	Reset	Access	Description
[1:0]	UPDICKSEL	0x3	R/W	UPDI clock frequency: 0x0 = Reserved 0x1 = 16 MHz 0x2 = 8 MHz 0x3 = 4 MHz (default)

Use 16 MHz UPDI clock only when BOD is at its highest level.

26.8.8 ASI_SYS_CTRLA (0x0A)

Bit	Name	Reset	Access	Description
-----	------	-------	--------	-------------

1	UROWWRITE_FINAL	0	R/W	Write 1 when user-row RAM is ready to commit to NVM. Only writable when UROWWRITE key active.
0	CLKREQ	0	R/W	1 = keep system clock running even in sleep modes (default when UPDI is enabled). 0 = lower clock request.

26.8.9 ASI_SYS_STATUS (0x0B)

Bit	Name	Reset	Access	Description
5	RSTSYS	0	R	1 = system domain in reset. Cleared on read.
4	INSLEEP	0	R	1 = system domain in Idle or deeper sleep.
3	NVMPROG	0	R	1 = NVM Programming mode active; safe to write.
2	UROWPROG	0	R	1 = User Row Programming active.
0	LOCKSTATUS	1	R	1 = device is locked. 0 after chip erase.

26.8.10 ASI_CRC_STATUS (0x0C)

Bit	Name	Reset	Access
[2:0]	CRC_STATUS	0x0	R

Value Meaning

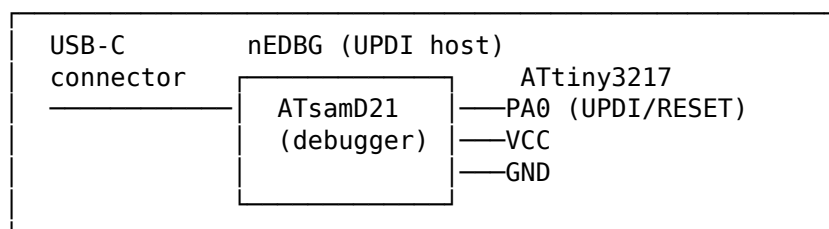
0x0	CRC not enabled
0x1	CRC enabled, still running
0x2	CRC done – PASSED
0x4	CRC done – FAILED

26.9 Hardware Adapters

26.9.1 The nEDBG: Curiosity Nano On-Board Programmer

The ATtiny3217 Curiosity Nano board includes a dedicated microcontroller (**nEDBG**, nano Embedded Debugger) that connects to the ATtiny3217's UPDI pin. No external hardware is needed.

Curiosity Nano (top view, simplified)



The USB connection to your PC presents as a CDC serial port and a HID device. avrdude uses the `curiosity_updi` programmer type to communicate with it. No diodes, no resistors, no extra hardware required.

26.9.2 jtag2updi (Legacy Firmware Programmer)

jtag2updi was the first widely-used DIY UPDI programmer. It is open-source firmware (by ElTangas, 2018–2020) that runs on a spare Arduino (typically an Arduino Nano or Uno with an ATmega328P or ATmega4809). The Arduino emulates a JTAG-to-UPDI bridge, which avrdude speaks to as if it were an Atmel ICE.

jtag2updi has largely been superseded by SerialUPDI (see below), but the firmware is still available at github.com/ElTangas/jtag2updi and works with avrdude’s jtag2updi programmer type.

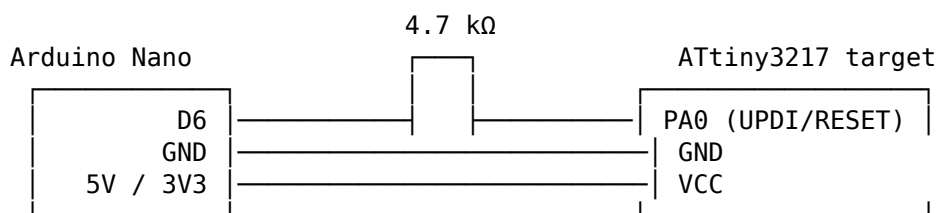
jtag2updi Circuit:

The key requirement is a series resistor between the Arduino’s digital output (Arduino pin 6 by default) and the target’s UPDI pin. This isolates the two microcontrollers and prevents bus contention.

Arduino (jtag2updi firmware)	Target ATtiny3217
GND	GND
5V or 3.3V	VCC (match target voltage)
Pin 6 (TX) — 4.7 kΩ	PA0 (UPDI)

Note: the 4.7 kΩ is appropriate specifically for jtag2updi because the firmware drives the line as a push-pull output with its own current limiting logic. Do NOT use this value with SerialUPDI adapters (see below).

Full jtag2updi schematic (through-hole, breadboard friendly):



(USB to PC for avrdude)

avrdude command for jtag2updi:

```
avrdude -c jtag2updi -p t3217 -P /dev/ttyUSB0 -b 115200 \
-U flash:w:firmware.hex:i
```

Limitations of jtag2updi: - Baud rate limited to approximately 115,200 bps in practice. - The firmware is complex and had known stability issues with some AVR targets. - Performance significantly slower than SerialUPDI at equivalent baud rates. - As of 2021, the jtag2updi project is in maintenance-only mode.

26.9.3 SerialUPDI: Direct USB Serial Adapter

SerialUPDI (by Spence Konde and Quentin Bolsée, 2020–2021) uses a standard USB-to-serial adapter directly as a UPDI programmer with minimal external circuitry. It is significantly faster and simpler than jtag2updi and is now the recommended DIY approach.

The key insight is that UPDI is a half-duplex single-wire UART. A serial adapter’s TX and RX lines can be combined with a small diode (or resistor) to create a single bidirectional line.

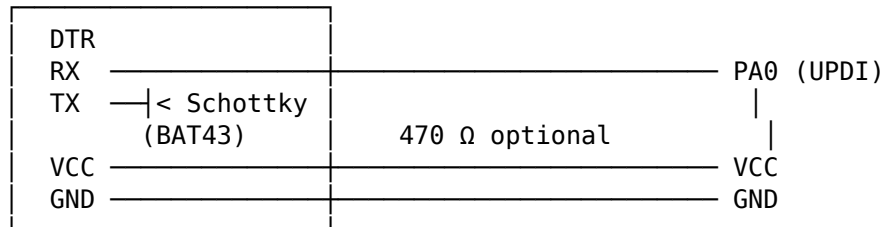
26.9.3.1 Required Hardware

1. A USB-to-serial adapter (CH340, CP2102, or FT232RL recommended).

2. One **Schottky diode** (BAT43, BAT54, 1N5817, or similar). A Schottky diode is strongly preferred over a regular silicon diode; the lower forward voltage drop avoids signal integrity problems at higher baud rates.
3. An optional 470 Ω series resistor (provides short-circuit protection).

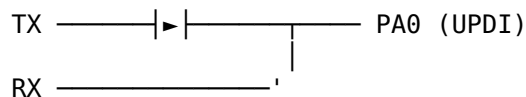
26.9.3.2 Recommended Wiring: Diode Method

USB Serial Adapter (with internal TX resistor 1–2 k Ω , e.g. CH340)



Band of diode points toward TX (current flows TX $\rightarrow$ junction $\rightarrow$ UPDI/RX).

ASCII schematic detail:

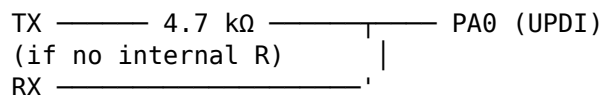


(diode band = ▶, toward TX side)

The diode allows TX to drive the UPDI line high/low, while the target's UPDI responses flow back on RX without the diode blocking them. RX and TX are tied together at the UPDI pin junction.

26.9.3.3 Wiring: Resistor Method (PyUPDI Classic)

If no Schottky diode is available, a resistor can be used. The total resistance (adapter's TX series resistor + external resistor) must sum to approximately **4.7 k Ω** :



If the adapter already has a 1–2 k Ω internal TX resistor, add enough to total 4.7 k Ω . The resistor method is less tolerant of voltage drops and high-speed signalling; prefer the diode method.

26.9.3.4 Adapter Voltage Considerations

- The ATtiny3217 runs at 3.3 V on the Curiosity Nano.
- Many CH340 adapters output 5 V logic levels even with a 3.3 V VCC jumper. Check your adapter datasheet.
- A 470 Ω series resistor on the UPDI line also provides some protection from level mismatch.

26.9.3.5 FTDI FT232RL Latency

The FT232RL defaults to a USB latency timer of 16 ms, which makes UPDI programming extremely slow (each page write requires multiple USB round trips). Set the latency timer to 1 ms before using with SerialUPDI:

- **Linux:** `echo 1 | sudo tee /sys/bus/usb-seria/devices/ttyUSB0/latency_timer`

- **Windows:** Device Manager → Ports → FT232R → Properties → Port Settings → Advanced → Latency Timer → 1 ms

26.9.3.6 avrdude Command for SerialUPDI

```
# Write flash at 230400 baud (general purpose speed)
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -b 230400 \
-U flash:w:firmware.hex:i

# Read flash to file
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -b 230400 \
-U flash:r:readback.hex:i

# Write fuses (set OSCCFG to select 20 MHz internal oscillator)
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -b 230400 \
-U fuse2:w:0x02:m

# Perform chip erase
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -b 230400 -e

# Windows: substitute COM3 (or whichever port) for /dev/ttyUSB0
```

26.9.3.7 Speed by Adapter (ATtiny3217, 128-byte page)

Adapter	115200 baud	230400 baud	460800 baud	Notes
CH340G	8.4 W/8.5 R	14.6 W/16.6 R	6.0 W/26.8 R	Best value
FT232RL	8.7 W/8.8 R	16.6 W/16.4 R	10.4 W/28.2 R	Needs 1 ms latency
CP2102	8.7 W/8.8 R	16.3 W/16.5 R	N/A	No 345600 by default

(kb/s; W = write, R = read. ATtiny with 128-byte pages measured on Linux.)

26.10 pymcuprog: Microchip's Python UPDI Tool

pymcuprog is Microchip's official Python implementation of the UPDI programming protocol. It is the reference implementation and the basis from which SerialUPDI was derived. Install via pip:

```
pip install pymcuprog
```

Common operations:

```
# Read device ID (confirm connection)
pymcuprog ping -t uart -u /dev/ttyUSB0 -d attiny3217

# Write flash from hex file
pymcuprog write -t uart -u /dev/ttyUSB0 -d attiny3217 \
--filename firmware.hex

# Erase flash
pymcuprog erase -t uart -u /dev/ttyUSB0 -d attiny3217

# Read fuses
pymcuprog read -t uart -u /dev/ttyUSB0 -d attiny3217 -m fuses

# Write a single fuse (FUSE2, index 2)
```



```
pymcuprog write -t uart -u /dev/ttyUSB0 -d attiny3217 \
  -m fuses --offset 2 --literal 0x02
```

pymcuprog is slower than the avrdude SerialUPDI implementation for large flash writes but is more portable and easier to script for custom workflows.

26.11 On-Chip Debugging (OCD) via UPDI

The same UPDI pin that programs the device also provides the debugging channel used in Chapter 5. The OCD controller is accessed through the UPDI ACC layer with no additional wiring.

OCD capabilities on the ATtiny3217:

Feature	Capability
Hardware breakpoints	2 (set via OCD registers; no flash writes needed)
Software breakpoints	Unlimited (BREAK opcode patched into flash)
Program counter readout	Real-time PC via UPDI CS register read
Stack pointer readout	SP available via data space read at any time
SREG readout	Status register readable at any time
Memory access while halted	Full read/write access to all memory-mapped I/O
Run/Stop/Reset control	Via ASI_SYS_STATUS and ASI_RESET_REQ
Non-intrusive monitoring	Can read PC/SP/SREG without halting CPU
Sleep mode debugging	CLKREQ bit keeps ACC layer accessible in sleep

The OCD debug key (required to unlock OCD Stop mode for full register access) is not published in the public datasheet but is available to tools like avarice and openOCD that speak to the ATtiny3217 via the Curiosity Nano’s nEDBG. The GDB + avarice flow described in Chapter 5 uses this path.

26.12 Fuses That Affect UPDI Access

The following fuses in FUSE.SYSCFG0 directly control UPDI accessibility:

SYSCFG0 address: 0x1285 (FUSE peripheral base 0x1280 + offset 0x05)

SYSCFG0 bits [3:2] = RSTPINCFG[1:0]

0x0	PA0 = GPIO. UPDI completely inaccessible. HV required to recover.
0x1	PA0 = UPDI (default). No external reset. Debugger connects freely.
0x2	PA0 = RESET. External reset active. UPDI inaccessible without HV.
0x3	Reserved.

Do not set RSTPINCFG = 0x0 unless you have a high-voltage programmer.

Check current fuse settings before programming them:

```
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -b 230400 \
  -U fuses:r:current_fuses.bin:r -v
```

The fuse layout for the ATtiny3217 (all 9 bytes, byte index 0–8):

Fuse Index	Name	Relevant Fields
0	WDTCFG	Watchdog config
1	BODCFG	Brown-out detector config

2	OSCCFG	Clock source (0x01=16 MHz, 0x02=20 MHz)
(no index 3)		
4	TCD0CFG	TCD0 config
5	SYSCFG0	RSTPINCFG (bits[3:2]), CRCSRC (bits[7:6]), EESAVE (bit 0)
6	SYSCFG1	SUT startup time (bits[2:0])
7	APPEND	Application code end (× 256 bytes)
8	BOOTEND	Boot section end (× 256 bytes)

26.13 Troubleshooting

26.13.1 Connection Fails Immediately

1. Verify RSTPINCFG ≠ 0x0 (GPIO mode). If it is, you need an HV programmer.
2. Check VCC and GND connections.
3. Try reducing baud rate (-b 57600).
4. On Linux, verify you are in the dialout group: `sudo usermod -aG dialout $USER`.

26.13.2 Error: “UPDI contention” or PESIG = 0x7

The UPDI line is being driven simultaneously by programmer and target. Common causes:
 - LED on the RX line of the USB serial adapter (the weak UPDI driver cannot overcome the LED load — remove the RX LED or use an adapter without one).
 - Series resistor on the target board too large (> 2.2 kΩ). Replace with 470 Ω or short it.
 - Wrong diode orientation.

26.13.3 Error: PESIG = 0x1 (Parity) or 0x2 (Frame)

Baud rate mismatch. Either the SYNCH character was corrupted, or the adapter is outputting a baud rate different from what avrdude requested. Try 57600 baud to rule out signal integrity issues.

26.13.4 Error: PESIG = 0x6 (Bus error)

Attempt to access a protected address. Usually means the NVMPROG key was not sent before trying to write flash, or the device lock bits are set. Run a chip erase first.

26.13.5 PESIG = 0x3 (Access Layer Timeout)

The UPDI ACC layer could not get bus access within 65536 UPDI clock cycles. Usually caused by the CPU being stuck in an infinite loop that monopolises the bus. Issue BREAK + system reset, then retry.

26.13.6 Device Locked (LOCKSTATUS = 1)

Send the Chip Erase key and perform the chip erase sequence. All flash and user data will be lost, but the device will be unlocked and reprogrammable.

26.13.7 Recovery from RSTPINCFG = 0x0 (No UPDI Access)

You need a high-voltage programmer capable of applying a 12 V pulse to PA0. Options:
 - MPLAB Snap (Microchip’s official programmer, supports HV UPDI).
 - JTAG2UPDI with HV hardware modifications.
 - Custom HV circuit: a MOSFET-based 12 V boost with precise timing control.

Once recovered via HV programming, immediately write correct fuses to restore RST-PINCFG = 0x1 or 0x2 before disconnecting.

26.14 Complete Wire-Level Walk-Through: Programming One Flash Page

To make the protocol concrete, here is the full UPDI byte sequence that writes a single 128-byte page of flash to address 0x0000 on the ATtiny3217 from scratch using SerialUPDI. Values are hexadecimal.

1. BREAK (×2): Two 12-bit low sequences ~25 ms each
2. SYNCH + LDCS STATUSA (read revision, confirm link):
Send: 55 80
Recv: 10 (UPDIREV = 1)
3. SYNCH + KEY (NVMPROG key, LSB first):
Send: 55 E4 4E 56 4D 50 72 6F 67 20
4. SYNCH + LDCS ASI_KEY_STATUS (confirm NVMPROG bit set):
Send: 55 87
Recv: 10 (bit 4 = NVMPROG active)
5. SYNCH + STCS ASI_RESET_REQ ← 0x59 (assert reset):
Send: 55 C8 59
6. SYNCH + STCS ASI_RESET_REQ ← 0x00 (release reset):
Send: 55 C8 00
7. SYNCH + LDCS ASI_SYS_STATUS (poll until NVMPROG=1):
Send: 55 8B
Recv: 08 (bit 3 = NVMPROG active; CPU halted)
8. Set NVMCTRL CMD = PAGEERASEWRITE (0x03):
SYNCH + STS addr=0x1000 data=0x03
Send: 55 44 00 10 03 (STS, 2-byte addr, 1-byte data; addr=NVMCTRL.CTRLA)
Recv: 40 40 (two ACKs)
9. Set pointer to mapped flash page 0 (0x8000 = mapped flash start):
SYNCH + ST(ptr) ← 0x8000:
Send: 55 6A 00 80 (ST ptr, 2-byte address 0x8000)
Recv: 40 (ACK)
10. SYNCH + REPEAT 127 (128 total writes):
Send: 55 A0 7F
11. SYNCH + ST(*(ptr++)) + 128 bytes of data:
Send: 55 64 <byte_0>
Recv: 40 (ACK for byte_0)
Send: <byte_1>
Recv: 40
... (continue for all 128 bytes)
12. Poll NVMCTRL.STATUS until FBUSY = 0 (write+erase complete):
SYNCH + LDS addr=0x1002 (NVMCTRL.STATUS):
Send: 55 04 02 10

Recv: <status> – poll until bit 0 (FBUSY) = 0

13. STCS ASI_RESET_REQ ← 0x59 / 0x00 (final reset):

Send: 55 C8 59

Send: 55 C8 00

14. STCS CTRLB ← UPDIDIS=1 (disable UPDI, return to normal run):

Send: 55 C3 04

The full 128-byte page write including all overhead typically completes in under 10 ms at 230400 baud plus the 4 ms PAGEERASEWRITE NVM time.

26.15 Summary

UPDI is a single-wire, half-duplex UART that replaces ISP on all modern AVR devices. It provides:

- Full read/write access to every byte in the device’s memory map.
- A key mechanism that unlocks three UPDI-initiated protected operations: chip erase, NVM programming, and user row write.
- Built-in on-chip debugging (OCD) on the same pin.
- Hardware-enforced protection via lock bits, bypassed only by chip erase.
- An HV override that can restore access even when the UPDI pin has been repurposed as a GPIO.

For daily development with the ATtiny3217 Curiosity Nano, the on-board nEDBG handles everything transparently. For standalone boards or custom hardware, a CH340-based SerialUPDI adapter with a single Schottky diode is all that is needed to program and debug.

The protocol details in this chapter — frame formats, instruction encodings, key signatures, register addresses — are the same across the entire modern AVR family: tinyAVR 0/1/2, megaAVR 0, and all AVR DA/DB/DD/EA parts. Once you know the ATtiny3217’s UPDI, you know UPDI everywhere.

Chapter 27

Self-Programming Flash: NVMCTRL on tinyAVR 1-Series

Classic AVR devices (ATmega328P and friends) use SPM from a classic boot section to write flash. The **ATtiny3217**, like other modern tinyAVR 1-series parts, takes a different approach: self-programming uses normal stores to the mapped Flash page buffer plus NVMCTRL commands, not the classic SPM flow, and there is no BOOTRST fuse. Instead, a memory-mapped peripheral called **NVMCTRL** (Non-Volatile Memory Controller) handles self-programming, while the BOOTEND and APPEND fuses divide flash into BOOT, APPCODE, and APPDATA sections with directional write protection.

This chapter covers everything needed to write assembly code that programs its own flash on the ATtiny3217 Curiosity Nano board:

1. Why AVRXMEGA3 replaced SPM with NVMCTRL.
2. How NVMCTRL registers and commands work.
3. The page buffer and the exact write sequence.
4. Configuration Change Protection (CCP) — the timing-critical unlock.
5. A complete 128-byte page-write routine in assembly.
6. A UART-driven self-update loop that receives Intel HEX and writes it to flash live.
7. The BOOTEND and APPEND fuses: defining protected flash sections.
8. Building, uploading, and common pitfalls.

This is one of the more hardware-specific chapters in the book. The goal is to turn “the chip can rewrite its own flash” from a mysterious feature into a sequence you can reason about step by step.

27.1 tinyAVR 1-Series vs Classic AVR: No SPM, NVMCTRL Instead

27.1.1 Architecture Family: AVRXMEGA3

The ATtiny3217 belongs to the **AVRXMEGA3** sub-architecture. The three key differences that affect self-programming are:

Feature	ATmega328P (classic)	ATtiny3217 (AVRXMEGA3)
Self-program flow	SPM-driven	ST to mapped Flash + NVMCTRL
Boot section	fixed classic boot area	configurable BOOT section
BOOTRST fuse	yes	none
Flash write ctrl.	SPMCSR register	NVMCTRL peripheral

Flash address space	separate word-address	byte-address, unified
Page size	128 bytes (64 words)	128 bytes (64 words)
Write privileges	boot section only	BOOT can write APPCODE/APPDATA; APPCODE can write APPDATA
UPDI programming	no	yes (PA0, replaces ISP)

The AVRXMEGA3 memory map is **unified**: flash, SRAM, and peripherals all share the same byte-addressed data space. Flash byte address 0 appears at data address 0x8000 when accessed by ordinary LD/ST instructions. The CPU's program counter still addresses executable flash from 0x0000. You will see both conventions in this chapter.

27.2 Microchip App Notes Used

The main Microchip reference for this chapter is **AN2634, Bootloader for tinyAVR 0- and 1-series, and megaAVR 0-series**. The relevant points from that application note are:

- tinyAVR 0/1-series and megaAVR 0-series devices program flash one page at a time.
- The page buffer has the same size as the flash page: 64 or 128 bytes depending on the device. The ATtiny3217 Flash page size is 128 bytes.
- The target page must be erased before the page buffer is written to flash. Programming an unerased page corrupts its contents.
- PAGEERASEWRITE starts the erase and write as one NVMCTRL command.
- The page buffer is automatically cleared after NVMCTRL commands execute.
- For tinyAVR 0/1-series, flash is mapped at data address 0x8000 for normal load/store access. Microchip's headers call this MAPPED_PROGMEM_START.
- BOOTEND and APPEND define BOOT, APPCODE, and APPDATA sections in 256-byte units.
- The documented write privileges are directional: BOOT can write APPCODE and APPDATA; APPCODE can write APPDATA; APPDATA cannot write flash or EEPROM.
- AN2634 warns against issuing CHIPERASE during self-programming because it erases the flash array.

The ATtiny3216/3217 data sheet adds two important details:

- The page buffer write uses the least significant address bits as the offset in the page buffer, and new page-buffer data is combined with previous buffer contents by a binary AND.
- After the self-programming CCP signature is written to CPU_CCP, the protected NVMCTRL command must be executed within the next four instructions.

27.2.1 What Replaces SPM?

On tinyAVR 1-series, you fill the flash **page buffer** by writing normally to flash addresses using ST (Store Indirect). There is no special opcode. The NVMCTRL peripheral watches these writes and accumulates them in a 128-byte buffer. When you issue a command to NVMCTRL_CTRLA, it commits the buffer to the actual flash array.

Classic AVR SPM flow:

```
load R1:R0 ← data word
write SPMCSR ← mode
SPM ← fills buffer
(repeat for each word)
write SPMCSR ← PGERS
SPM ← erases page
write SPMCSR ← PGWRT
SPM ← writes page
```

tinyAVR NVMCTRL flow:

```
(nothing special)
wait FBUSY = 0
ST Z+, rn ← fills buffer (just write!)
(repeat for each word)
CCP unlock (0x9D → 0x0034)
STS NVMCTRL_CTRLA ← 0x03
wait FBUSY = 0
```

The NVMCTRL approach is simpler to understand: write your data to the target flash addresses, then send the “erase+write” command. The hardware does the rest.

The important mental shift is that self-programming no longer looks like a special instruction stream. It looks like ordinary memory-mapped peripheral control wrapped around a careful write sequence.

27.2.2 Programming Interface: UPDI

The ATtiny3217 has no ISP pins. All external programming is done over **UPDI** (Unified Program and Debug Interface), a single-wire protocol on **PA0**. The Curiosity Nano board’s on-board debugger bridges USB to UPDI automatically. Use `pymcuprog` for the on-board debugger:

```
pymcuprog write -d attiny3217 -t uart -f firmware.hex
```

Once the self-update firmware is running, the device reprograms itself — no external programmer needed for firmware updates.

27.3 NVMCTRL Overview

27.3.1 Flash Parameters

ATtiny3217 flash layout

Total flash:	32 KB (32768 bytes)
Page size:	128 bytes
Number of pages:	256
Flash data space:	0x8000–0xFFFF (byte addresses when read as data)
LPM address range:	0x0000–0x7FFF (byte addresses for program memory reads)
SRAM:	0x3800–0x3FFF (2 KB)

The 128-byte page size is critical: every NVMCTRL flash write operation commits one page. If your update stream ends with a short final page, pad the unwritten bytes in the SRAM page image with 0xFF before issuing `PAGEERASEWRITE`.

That last point matters because it explains why page alignment and buffering show up everywhere in self-programming code. Flash is not rewritten one byte at a time, even if your firmware update logic receives bytes one at a time.

27.3.2 NVMCTRL Register Map

All NVMCTRL registers are outside the I/O space and require LDS/STS:

Register	Address	Width	Purpose
NVMCTRL_CTRLA	0x1000	1 byte	Command register – write command here
NVMCTRL_CTRLB	0x1001	1 byte	Config: BOOTLOCK, APCWP write-protect
NVMCTRL_STATUS	0x1002	1 byte	Status flags: FBUSY, EEBUSY, WRERROR
NVMCTRL_INTCTRL	0x1003	1 byte	Interrupt enable: EEREADY (bit 0)
NVMCTRL_INTFLAGS	0x1004	1 byte	Interrupt flags (write 1 to clear)
NVMCTRL_DATA	0x1006	2 bytes	Data register (UPDI fuse-write data)
NVMCTRL_ADDR	0x1008	2 bytes	Address of last-updated memory location

For flash programming, you only need `NVMCTRL_CTRLA` and `NVMCTRL_STATUS`.

27.3.3 NVMCTRL_STATUS — Status Register (0x1002)

Bit:	7	6	5	4	3	2	1	0
	—	—	—	—	—	WRERROR	EEBUSY	FBUSY

FBUSY — Flash Busy: 1 while a flash erase/write operation is in progress.
Poll this bit; do not issue a new command while FBUSY = 1.

EEBUSY — EEPROM Busy: 1 while an EEPROM operation is in progress.

WRERROR — Write Error: set if a write was attempted to a protected region.
Write 1 to clear (W1C).

27.3.4 NVMCTRL Commands (written to NVMCTRL_CTRLA)

Value	Name	Description
0x00	NOCMD	No operation
0x01	PAGEWRITE	Write page buffer to flash (no erase first)
0x02	PAGEERASE	Erase one flash page (Z must point into page)
0x03	PAGEERASEWRITE	Erase + write in one atomic step ← use this
0x04	PAGEBUFCLR	Clear the page buffer (all bytes → 0xFF)
0x05	CHIPERASE	Erase Flash + EEPROM (unless EESAVE); not used here
0x06	EEERASE	Erase EEPROM
0x07	WRITEFUSE	Write fuses – UPDI only, not issuable from app code

PAGEERASEWRITE (0x03) is the command you will use for firmware updates. It atomically erases the target page and writes the page buffer in one operation — safer than doing them separately, because a power failure between separate erase and write steps cannot leave a page half-erased.

In practice, that combined command is what makes the sequence robust enough to use in the field. Fewer separate steps means fewer partially completed states.

27.4 The Page Buffer and Write Sequence

27.4.1 How the Page Buffer Works

The NVMCTRL page buffer is a 128-byte staging area internal to the NVM controller. It is **not** directly addressable. You populate it by writing to the corresponding flash addresses in the data address space:

```
mapped_flash_address = 0x8000 + flash_byte_address
```

When the Z pointer points into flash address space and you execute ST Z+, Rn, the byte goes into the page buffer at the offset corresponding to Z's address. The hardware handles this automatically whenever the NVM controller recognises the write target as flash.

Page buffer write – step by step:

Flash page at byte address 0x0000 appears in data space at 0x8000.

```
Z = 0x8000          ; points to byte 0 of flash page 0
st Z+, r16          ; → page buffer[0] = r16
st Z+, r17          ; → page buffer[1] = r17
...                 ; fill all 128 bytes
st Z+, r30          ; → page buffer[127] = r30
                    ; Z = 0x8080 (past end of page)
```

The page buffer is erased to all ones after reset, after page write/erase commands, after a Clear Page Buffer command, and after wake-up from sleep. The least significant address

bits select the offset inside the page buffer. If you write the same page-buffer byte more than once, the resulting value is the binary AND of the old buffer value and the new value.

That AND behavior matters. A partial page update is not automatically merged with the existing flash page for you. For a robust partial update, either fill unwritten bytes with 0xFF when programming a fresh page, or read the old page into SRAM, modify the intended bytes, erase the flash page, and write the full merged 128-byte image.

27.4.2 Complete Flash Page Write Sequence

Step 1: Wait for any previous NVM operation to finish.
Poll NVMCTRL_STATUS.FBUSY until it reads 0.

Step 2: Clear any stale write error.
STS NVMCTRL_STATUS, 0x04 ; WRERROR is write-one-to-clear

Step 3: Fill the page buffer.
Set Z = 0x8000 + (target_page × 128) ; data-space address
ST Z+, r0 ; ST Z+, r1 ; ... ; 128 sequential writes

Step 4: Issue PAGEERASEWRITE (CCP-protected).
LDI r16, 0x03 ; PAGEERASEWRITE command, prepared first
LDI r17, 0x9D ; CCP_SPM_gc key, prepared first
STS 0x0034, r17 ; write to CCP register
STS NVMCTRL_CTRLA, r16 ; next instruction: protected command

Step 5: Wait for FBUSY to clear again.
Poll NVMCTRL_STATUS.FBUSY until 0.

27.4.3 The Z Pointer Convention

On ATtiny3217 the Z pointer (R31:R30) is a 16-bit register, and 0x8000 is well within that range. For page-buffer writes with ST Z+, Rn, set Z to the mapped program-memory address:

```
/* To write page at flash byte address 0x0200 (page 4): */
ldi ZL, lo8(0x8200)
ldi ZH, hi8(0x8200)
/* Now ST Z+, Rn writes to page buffer slots starting at offset 0 */
```

Equivalently:

```
ldi ZL, lo8(0x0200)
ldi ZH, hi8(0x0200)
subi ZH, -0x80 /* add mapped flash base 0x8000 */
```

The examples in Sections 16.5 and 16.6 use this mapped-address convention.

That convention keeps the assembly easier to follow because the same flash byte address can be used consistently through buffering, writing, and later jumps.

27.5 CCP — Configuration Change Protection

27.5.1 Why CCP Exists

Some NVMCTRL commands (and other sensitive operations like changing clock settings) can corrupt firmware if executed accidentally — for example, if a runaway pointer writes to NVMCTRL_CTRLA, or if a flash write executes during a brownout. **CCP** (Configuration

Change Protection) prevents accidental execution by requiring a timed unlock sequence before each protected write.

So CCP is not there to make your life difficult. It is there because an accidental flash-control write is serious enough that the hardware demands a deliberate two-step action.

27.5.2 The CCP Register (0x0034)

Address: 0x0034 (in I/O space: use OUT or STS)

CCP keys:

0x9D – CCP_SPM_gc Unlocks NVM Self-Programming commands
0xD8 – CCP_IOREG_gc Unlocks protected I/O register writes (e.g. CLKCTRL)

After writing a key, the protected write must occur within the next four CPU instructions. Interrupt requests are held pending during this short configuration-change window.

27.5.3 The Four-Instruction Rule

The CCP window opens the moment you write the key. Use the strictest practical rule in assembly: make the protected NVMCTRL_CTRLA write the very next instruction after the CPU_CCP write:

```
sts    CPU_CCP, r17
sts    NVMCTRL_CTRLA, r16
```

Assembly idiom:

```
ldi    r16, 0x03          /* PAGEERASEWRITE command */
ldi    r17, 0x9D          /* CCP_SPM_gc key */
sts     0x0034, r17        /* open the CCP window */
sts     NVMCTRL_CTRLA, r16 /* next instruction: protected write */
```

Critical: do not put any instruction between STS CPU_CCP and STS NVMCTRL_CTRLA — not even a NOP.

This is one of the rare cases where instruction spacing is not just an optimization detail. It is part of the functional correctness of the code.

27.6 Flash Write Routine in Assembly

The routine `nvm_write_page` writes one 128-byte page. It expects:

- **Z** (R31:R30): mapped flash data address of the target page: `0x8000 + flash_byte_address` (must be 128-byte aligned).
- **X** (R27:R26): data address of the 128-byte source buffer in SRAM.

It clobbers R16, R17, R18 (scratch) and leaves Z past the end of the written page. The source buffer pointer X is incremented and left past the last byte.

Leaving both pointers advanced is a small design choice, but it makes the routine easier to chain when writing consecutive pages from a stream.

27.6.1 Register Allocation

Z (R31:R30)	mapped flash destination – page base address, then advanced
X (R27:R26)	SRAM source pointer – 128-byte buffer of new content
R16	scratch: NVMCTRL command, status poll byte
R17	loop counter (128 iterations)
R18	data byte being copied into page buffer

27.6.2 Full Listing: src/nvmctrl_write.S

See src/nvmctrl_write.S for the complete annotated source. Key routines:

```
/*
 * nvm_wait – spin until NVMCTRL_STATUS.FBUSY = 0
 *
 * Clobbers: R16
 * Poll loop while busy: LDS (3) + SBRC (1) + RJMP (2) = 6 cycles per poll
 */
nvm_wait:
    lds    r16, NVMCTRL_STATUS          /* 3 cy: load status          */
    sbrc   r16, NVMCTRL_FBUSY_bp        /* 1 cy: skip next if FBUSY=0 */
    rjmp   nvm_wait                    /* 2 cy: still busy, loop     */
    ret                                     /* 4 cy                      */
```

SBRC Rd, b — Skip if Bit in Register is Cleared. Takes 1 cycle when not skipping, 2 when skipping (pipeline flush). NVMCTRL_FBUSY_bp = 0 (bit position 0 of NVMCTRL_STATUS).

```
/*
 * nvm_write_page – write 128-byte SRAM buffer to one flash page
 *
 * Entry:  Z = mapped flash data address (0x8000 + flash byte address)
 *         and 128-byte aligned
 *         X = SRAM pointer to 128-byte source buffer
 * Exit:   Z advanced by 128, X advanced by 128
 * Clobbers: R16, R17, R18
 */
nvm_write_page:
    /* Step 1: wait for any previous NVM operation */
    rcall nvm_wait                /* wait for FBUSY to clear */

    /* Step 2: clear previous write error (WRERROR is W1C) */
    ldi    r16, NVMCTRL_WRError_bm /* r16 = 0x04 */
    sts    NVMCTRL_STATUS, r16

    /* Step 3: fill page buffer from SRAM source */
    ldi    r17, 128                /* loop counter = page size */

.Lfill_loop:
    ld     r18, X+                 /* 2 cy: load byte from SRAM */
    st     Z+, r18                 /* write byte to flash page buffer */
    dec    r17                    /* 1 cy: decrement counter */
    brne   .Lfill_loop            /* 2 cy taken / 1 cy fall-through */
    /* Mapped-Flash page-buffer writes are NVM accesses; do not use SRAM ST
       timing for exact wall-clock estimates. */

    /* Step 4: rewind Z to page start for PAGEERASEWRITE */
    subi   ZL, lo8(128)           /* Z -= 128 */
    sbci   ZH, hi8(128)

    /* Step 5: CCP-protected PAGEERASEWRITE command */
    cli                                     /* 1 cy: disable interrupts */
    ldi    r16, NVMCTRL_CMD_PAGEERASEWRITE_gc /* r16 = 0x03 */
    ldi    r17, CCP_SPM_gc        /* r17 = 0x9D */
    sts    CPU_CCP, r17           /* 2 cy: unlock CCP window (cycles 1-2) */
    sts    NVMCTRL_CTRLA, r16     /* 2 cy: issue command (cycles 3-4) */
    sei                                     /* 1 cy: re-enable interrupts */

    /* Step 6: wait for completion */
    rcall nvm_wait
```

```

/* Z points to page start; advance past the page for caller convenience */
subi ZL, lo8(-128) /* Z += 128 */
sbc_i ZH, hi8(-128)
ret

```

27.6.3 Instruction Notes

Two instructions appear for the first time in this chapter:

SBIW Rd, K – Subtract Immediate from Word
 Operation: $Rd+1:Rd \leftarrow Rd+1:Rd - K$ ($K = 0..63$)
 Encoding: 1001 0111 KKdd KKKK (2 bytes)
 Registers: $d \in \{24,26,28,30\}$ only (R25:R24, X, Y, Z)
 Flags: C, Z, N, V, S
 Cycles: 2

ADIW Rd, K – Add Immediate to Word
 Operation: $Rd+1:Rd \leftarrow Rd+1:Rd + K$ ($K = 0..63$)
 Encoding: 1001 0110 KKdd KKKK (2 bytes)
 Registers: $d \in \{24,26,28,30\}$ only
 Flags: C, Z, N, V, S
 Cycles: 2

SBIW and ADIW are useful word operations, but their immediate range is 0..63. A 128-byte page adjustment must use another sequence. The source uses SUBI/SBCI pairs to subtract and add 128 from the 16-bit Z pointer.

27.7 Practical Firmware Update: UART-Driven Self-Update

With `nvm_write_page` in hand, we can build a self-update loop. The device receives new firmware as Intel HEX records over USART0, accumulates complete 128-byte pages in SRAM, then writes each page to flash.

At that point the problem stops being “how do I write flash?” and becomes a system problem: receive bytes reliably, group them into pages, commit them, and avoid damaging the updater itself.

27.7.1 Intel HEX Recap

```

:LLAAAATT[DD...]CC
| | | | |
| | | | | checksum (2 hex ASCII chars)
| | | | | data bytes (LL x 2 hex chars)
| | | | | record type: 00=data, 01=E0F
| | | | | address (4 hex ASCII chars, big-endian)
| | | | | byte count LL (2 hex ASCII chars)
↑ start code ':'

```

Checksum: two's complement of $(LL + \text{addr_hi} + \text{addr_lo} + \text{type} + \text{sum}(\text{data}))$
 The sum of all bytes in the record including checksum = 0x00 (mod 256).

27.7.2 Architecture of `selfupdate.S`

```

selfupdate_main:
|

```

```

├─ init: stack, USART0 (PB2 TX, PB3 RX, 9600 baud at 3.333 MHz)
├─ LED0 (PA3) on → indicates update mode active
├─ send banner "SU\r\n"
└─ record_loop:
    ├─ wait for ':' (discards noise before start code)
    ├─ recv_hex_byte × 4: byte_count, addr_hi, addr_lo, type
    ├─ if type = 0x01 (EOF): send "OK\r\n", wait FBUSY, IJMP to 0x0000
    ├─ if type = 0x00 (data):
        ├─ set Z = record address (flash byte address)
        ├─ receive LL data bytes → SRAM page buffer
        ├─ if buffer >= 128 bytes: call nvm_write_page
        └─ verify checksum → send '.' (ACK) or '!' (NACK)
    └─ repeat

```

27.7.3 Register Allocation for selfupdate.S

Z (R31:R30)	flash write address (maintained across records)
X (R27:R26)	SRAM page buffer pointer (buf_base .. buf_base+127)
R16	scratch / UART byte / NVMCTRL command
R17	checksum accumulator (zeroed at each record start)
R18	byte count from record header
R19	record type (00 or 01)
R24	address low byte from record header
R25	address high byte from record header
R20, R21	scratch in helpers
R15	saved high nibble in recv_hex_byte

27.7.4 USART0 Initialisation at 3.333 MHz

The ATtiny3217's default clock is 3.333 MHz (20 MHz internal oscillator with the default /6 prescaler in CLKCTRL). The USART0 fractional baud register for 9600 baud:

```

BAUD_REG = (f_cpu × 64) / (16 × baud)
          = (3333333 × 64) / (16 × 9600)
          = 213333312 / 153600
          = 1389 (0x056D)

```

```

.equ BAUD_REG, 1389                                /* 9600 baud @ 3.333 MHz, error < 0.01% */

usart_init:
    sbi    VPORTB_DIR, 2                            /* PB2 = TXD output */
    ldi    r16, lo8(BAUD_REG)
    sts    USART0_BAUDL, r16
    ldi    r16, hi8(BAUD_REG)
    sts    USART0_BAUDH, r16
    ldi    r16, USART_TXEN_bm | USART_RXEN_bm
    sts    USART0_CTRLB, r16                        /* enable TX and RX */
    ret
/* CTRLC defaults to 8N1 – no write needed */

```

Note: VPORTB_DIR is at I/O address 0x04 (within sbi/cbi range) — we use sbi rather than sts to set the single bit without disturbing other PORTB direction bits.

27.7.5 Full Listing: src/selfupdate.S

The complete source is at src/selfupdate.S. The core hex-receive loop and page-write call are shown below for reference:

```

.Ldata_loop:
    rcall recv_hex_byte      /* receive next data byte from HEX record */
    st      X+, r16          /* store in SRAM page buffer */
    add     r17, r16         /* accumulate checksum */
    dec     r18              /* decrement byte count */

    /* check if page buffer is full (128 bytes written) */
    mov     r20, XL
    andi    r20, 0x7F        /* XL & 127 – non-zero means not at boundary */
    brne    .Lcheck_done    /* not at 128-byte boundary: continue */

    /* page buffer is full: save X, write page, restore */
    movw    r20, XL          /* save X */
    rcall   nvm_write_page   /* writes page, advances Z by 128 */
    movw    XL, r20          /* restore X */

.Lcheck_done:
    tst     r18
    brne    .Ldata_loop     /* more bytes in this record */

```

27.7.6 Jumping to the Updated Application

After the EOF record is received and the last page is written:

```

.Leof:
    rcall   recv_hex_byte    /* consume EOF checksum byte (not verified) */
    rcall   nvm_wait         /* ensure final write is complete */
    /* turn off LED0 (PA3) to signal update complete */
    sbi     VPORTA_OUT, 3    /* PA3 high = LED off (active-low) */
    /* send "OK\r\n" */
    ldi     r16, 'O'
    rcall   usart_tx
    ldi     r16, 'K'
    rcall   usart_tx
    ldi     r16, '\r'
    rcall   usart_tx
    ldi     r16, '\n'
    rcall   usart_tx
    /* jump to new application */
    clr     ZL
    clr     ZH
    ijmp                                /* PC ← Z = 0x0000 (new firmware entry) */

```

IJMP (Indirect Jump): sets the program counter to Z (2 cycles). With Z = 0, execution continues from the reset vector of the freshly written firmware.

Conceptually, that final jump is just a handoff. The updater finishes its work, then transfers control to the program it just installed.

27.7.7 Uploading Firmware

Use the existing `src/send_hex.py` helper, or any serial terminal. The selfupdate firmware sends `SU\r\n` instead of `BL\r\n`:

The helper rewrites ordinary Intel HEX input into 128-byte page records before sending. That matches the small updater's assumptions: each data record starts on a page boundary, and a short final page is padded with `0xFF`.

```
# Flash the self-update firmware via the Curiosity Nano debugger (one-time):
pymcuprog write -d attiny3217 -t uart -f selfupdate.hex

# Reset the board – LED0 turns on (active-low, so PA3 = 0 = LED on)
# Then immediately send application firmware:
python3 src/send_hex.py /dev/ttyUSB0 app.hex
```

Expected output:

Waiting for self-updater on /dev/ttyUSB0...

Self-updater ready

record 1/18 OK

record 2/18 OK

...

record 18/18 OK

Upload complete. Application starting.

LED0 turns off when the update completes, and the new firmware starts.

27.8 BOOTEND Fuse: Protecting a Bootloader Region

27.8.1 The Problem

The self-update firmware in Section 25.6 writes to any flash address, including address 0x0000 — which is where the self-update firmware itself lives. If the update process is interrupted after erasing page 0 but before writing the new content, the device is bricked (no valid firmware at the reset vector). It also means buggy application firmware could accidentally overwrite the updater.

27.8.2 BOOTEND: Defining a Protected Region

The ATtiny3217 provides the **BOOTEND** and **APPEND** fuses. These divide flash into BOOT, APPCODE, and APPDATA sections in 256-byte units. For the simple updater layout, set **BOOTEND** to reserve space for the updater and leave **APPEND** = 0 so the rest of flash is APPCODE:

FUSES.BOOTEND value, with APPEND = 0:

0x01 – 256 bytes BOOT (pages 0–1)

0x02 – 512 bytes BOOT (pages 0–3)

0x04 – 1024 bytes BOOT (pages 0–7)

...

Boot section: 0x0000 .. (BOOTEND × 256 - 1)

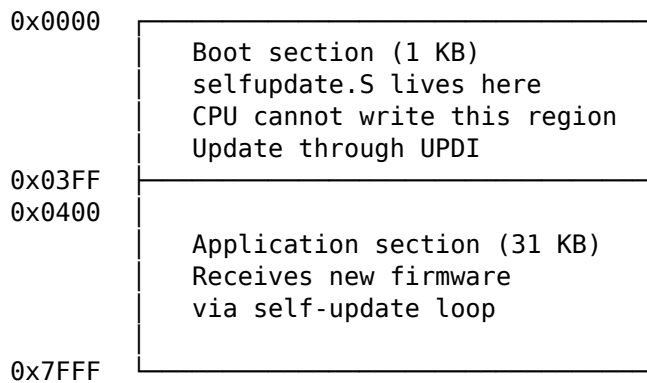
APPCODE section: (BOOTEND × 256) .. 0x7FFF

With **BOOTEND** = 0x04 (1 KB boot section), the self-update firmware occupies pages 0–7 (addresses 0x0000–0x03FF). The application firmware lives at 0x0400 onward. The NVMCTRL enforces section privileges. BOOT code can write APPCODE and APPDATA. APPCODE can write APPDATA. APPDATA cannot write flash or EEPROM. The CPU cannot write the BOOT section.

This is the safety boundary that turns a self-programming demo into something you can trust more. Without it, the updater can erase the very code that is needed to recover from a failed update.

27.8.3 Flash Layout With BOOTEND = 0x04

ATtiny3217 flash (32 KB) with **BOOTEND** = 0x04



27.8.4 Application Entry Point

When `BOOTEND = 0x04`, the application vector table starts at **0x0400**. The hardware Reset vector still starts execution at 0x0000; there is no `BOOTRST` fuse on this device. After a successful update, the self-update firmware must jump to the application's entry at 0x0400 rather than back to its own reset entry at 0x0000:

```
/* With BOOTEND = 0x04: application starts at 0x0400 */
.equ APP_START, 0x0400

.Leof:
    rcall recv_hex_byte
    rcall nvm_wait
    ldi   ZL, lo8(APP_START)
    ldi   ZH, hi8(APP_START)
    jmp  /* jump to application section start */
```

The application must be built with its `.text` section starting at 0x0400:

```
avr-gcc -mmcu=attiny3217 -nostartfiles \
    -Wl,--section-start=.text=0x0400 \
    -o app.elf app.S
```

If the bootloader uses interrupts while running from the BOOT section, it must set `CPUINT.CTRLA.IVSEL` to move the interrupt vector table to the start of Flash, then restore the application vector location before handing control to the application. The reset vector itself remains at 0x0000.

27.8.5 Run-Time Locks: BOOTLOCK and APCWP

`NVMCTRL_CTRLB` has two run-time lock bits that remain active until reset:

- `APCWP` prevents further writes to `APPCODE`.
- `BOOTLOCK` prevents reads and execution from `BOOT`.

Setting `APCWP` after the update completes provides defense-in-depth:

```
/* After successful update, lock application section against further writes */
ldi   r16, 0x01 /* APCWP bit */
sts   NVMCTRL_CTRLB, r16
```

This is cleared on every reset, so it does not permanently lock the device.

Think of `APCWP` as a temporary guardrail for the current boot session, not as a replacement for fuse-based protection.

27.9 Building and Flashing

27.9.1 Building `nvmctrl_write.S` (standalone test)

```
# Assemble the NVM write routine alone (produces an ELF for inspection)
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
    -o nvmctrl_write.elf src/nvmctrl_write.S

# Check sections and disassemble to verify instruction encoding
avr-objdump -h nvmctrl_write.elf
avr-objdump -d nvmctrl_write.elf
```

27.9.2 Building `selfupdate.S`

```
# Assemble the self-update firmware
# Without BOOTEND protection: starts at 0x0000
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
    -o selfupdate.elf src/selfupdate.S

# With BOOTEND=0x04: starts at 0x0000, app section starts at 0x0400
# The selfupdate firmware itself is the same; linker placement is 0x0000
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
    -o selfupdate.elf src/selfupdate.S

# Size check – .text must fit within chosen boot section (1 KB = BOOTEND 0x04)
avr-objdump -h selfupdate.elf
```

Expected section output excerpt:

Idx	Name	Size	VMA
1	.text	000001c6	00000000
2	.bss	00000080	00803800

0x1c6 is 454 bytes of flash code, which fits comfortably within the 1 KB BOOT section. The 128-byte .bss allocation is the SRAM page buffer.

27.9.3 Converting to HEX

```
avr-objcopy -O ihex selfupdate.elf selfupdate.hex
avr-objcopy -O ihex app.elf app.hex
```

27.9.4 Flashing with `pymcuprog`

```
# Program selfupdate firmware via UPDI (requires Curiosity Nano USB connection)
pymcuprog write -d attiny3217 -t uart -f selfupdate.hex

# Set BOOTEND fuse to 0x04 (1 KB boot section)
# WARNING: verify the exact pymcuprog fuse syntax for your installed version.
# BOOTEND is fuse byte 8; write value 0x04.

# Read back fuse to verify
pymcuprog read -d attiny3217 -t uart
```

Warning: BOOTEND is in fuse byte 8 on ATtiny3217. An incorrect value can make the device appear to run no code. Always read back fuses after writing. UPDI can always recover the device regardless of fuse settings (unlike ISP on ATmega parts where the BOOTRST fuse can complicate recovery).

That recovery path is important operationally: experimenting with self-update code is much less risky when you know UPDI can still get you back in.

27.9.5 Verifying the Disassembly

After building, verify the CCP sequence is encoded correctly:

```
avr-objdump -d selfupdate.elf | grep -A5 "CPU_CCP\|0034"
```

Look for two back-to-back sts instructions with addresses 0x0034 (CCP) and 0x1000 (NVMCTRL_CTRLA). No instruction should appear between them in the listing.

27.9.6 Building an Application for the Protected Layout

```
# Application must be linked at 0x0400 when BOOTEND = 0x04
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
    -Wl,--section-start=.text=0x0400 \
    -o app.elf app.S
avr-objcopy -O ihex app.elf app.hex
```

```
# Send via self-update firmware:
python3 src/send_hex.py /dev/ttyUSB0 app.hex
```

27.10 Common Pitfalls

27.10.1 Pitfall 1 — CCP Window Violated

Symptom: NVMCTRL_CTRLA write appears to succeed (no error) but no flash write occurs; WRError is set in STATUS.

Cause: One or more instructions were inserted between STS CPU_CCP, r17 and STS NVMCTRL_CTRLA, r16, pushing the protected write outside the four-instruction window. (Interrupts cannot be the cause here: the datasheet states all interrupts are *ignored* during the four cycles after the CCP key is written, so an ISR cannot fire inside the window.)

Fix: Ensure no instruction sits between the two STS calls. The CLI/SEI pair around the sequence is defensive — it does not protect the window itself (hardware already does that) but keeps an ISR from running just before the key write and disturbing the registers you prepared.

```
/* WRONG — ldi between STS instructions wastes a cycle inside the window */
sts CPU_CCP, r16
ldi r16, 0x03 /* ← this is inside the window, shrinks margin */
sts NVMCTRL_CTRLA, r16

/* CORRECT — load command value BEFORE writing CCP key */
ldi r16, 0x03 /* load command first (outside window) */
...
ldi r17, CCP_SPM_gc
sts CPU_CCP, r17 /* open window */
sts NVMCTRL_CTRLA, r16 /* issue command (within window) */
```

Note the subtlety: if r16 holds the CCP key when you write CPU_CCP, you must reload r16 with the command before the protected write, or use two registers. The corrected pattern above uses two registers to keep the command value ready before opening the CCP window.

That is a good example of why “almost correct” is not good enough here. A sequence can look visually right and still fail because one register was reused at the wrong moment.

27.10.2 Pitfall 2 — Page Buffer Not Aligned

Symptom: The first few bytes of a written page are corrupted or contain old flash content.

Cause: The mapped flash address passed to `nvm_write_page` is not 128-byte aligned. The page buffer fill loop always fills from byte 0 to byte 127 of the chosen page. If `Z` points into the middle of a page, the first `ST Z+, Rn` writes to a mid-page buffer slot. The earlier slots contain the current page-buffer state, not an automatic copy of the old flash page.

Fix: Always pass a 128-byte aligned address:

```
/* Align mapped Z to 128-byte page boundary: clear bits 6:0 */
andi ZL, 0x80 /* mask: 1000 0000 – clear bits 6:0 */
```

Page alignment bugs are especially deceptive because most of the write may look correct. Only the offsets near the page boundary reveal the mistake.

27.10.3 Pitfall 3 — Writing to Flash During Interrupt

Symptom: Application ISR fires while `nvm_write_page` is between the page buffer fill and the `PAGEERASEWRITE` command. The ISR reads stale flash content via `LPM` from the page being written.

Cause: The page buffer is not yet committed. If an ISR attempts to read instruction or data from flash in the middle of a `PAGEERASEWRITE`, it reads the erased (0xFF) state during the erase phase.

Fix: Disable global interrupts before the entire write sequence and re-enable after:

```
cli
rcall nvm_write_page
sei
```

The `nvm_write_page` routine already disables interrupts around the `CCP` write, but a complete `CLI/SEI` around the entire page write is safer if you have ISRs active.

27.10.4 Pitfall 4 — Forgetting to Wait for FBUSY Before Each Command

Symptom: Random corruption of flash pages, or `WRERROR` flag set frequently.

Cause: A second `PAGEERASEWRITE` command was issued before the first completed. `NVMCTRL_STATUS.FBUSY` must be 0 before issuing any command.

Fix: The `nvm_wait` routine at the start of `nvm_write_page` ensures this. If you issue `NVMCTRL` commands from multiple places in your code, always call `nvm_wait` first.

27.10.5 Pitfall 5 — Writing the Boot Section From Application Code

Symptom: Write to pages 0–(`BOOTEND`-based boundary) triggers `WRERROR` but no actual write.

Cause: With `BOOTEND` set, `NVMCTRL` enforces section privileges. The CPU cannot write the `BOOT` section through `NVMCTRL`.

Fix: This is intentional behaviour — the boot section protection is working correctly. If you need to update the self-update firmware, use `UPDI` with `pymcuprog`.

In other words, `WRERROR` here is a feature, not a malfunction.

27.10.6 Pitfall 6 — UPDI Port Floating (PA0)

Symptom: pymcuprog cannot connect; “timeout communicating with programmer.”

Cause: On the Curiosity Nano, PA0 is used for UPDI. If you configure PA0 as a GPIO output in your firmware, UPDI communication breaks. The on-board debugger drives PA0 with its own UPDI signalling.

Fix: Never configure PA0 as GPIO output. Leave PA0 at its power-on default (UPDI function). If your application needs an extra GPIO and PA0 was used, remap to a different pin.

27.10.7 Pitfall 7 — Baud Rate at Non-Default Clock

Symptom: Garbage characters received; UART communication fails after changing CLKCTRL prescaler.

Cause: The BAUD_REG constant was calculated for 3.333 MHz (default clock). If your firmware changes CLKCTRL_MCLKCTRLB to select a different frequency before calling `usart_init`, the baud rate will be wrong.

Fix: Recalculate BAUD_REG for the actual `f_cpu`, or initialise USART before changing the clock. The self-update firmware deliberately runs at the default clock to avoid this dependency.

27.11 Summary

ATTiny3217 self-programming (AVR_XMEGA3):

Flash geometry:

32 KB total, 128-byte pages, 256 pages
 Data-space flash base address: 0x8000
 Page buffer fill: ST Z+, Rn (plain store instruction)

NVMCTRL registers:

NVMCTRL_CTRLA 0x1000 command register
 NVMCTRL_STATUS 0x1002 FBUSY (bit 0), WRERROR (bit 2)

Write sequence for one 128-byte page:

1. Poll NVMCTRL_STATUS until FBUSY = 0
2. STS NVMCTRL_STATUS, 0x04 (clear WRERROR; write-one-to-clear)
3. 128 × ST Z+, Rn (fill page buffer at mapped flash address)
4. SUBI/SBCI Z, 128 (rewind Z to page base)
5. CLI
 - STS CPU_CCP, 0x9D (unlock CCP)
 - STS NVMCTRL_CTRLA, 0x03 (PAGEERASEWRITE as next instruction)
 - SEI
6. Poll NVMCTRL_STATUS until FBUSY = 0

CCP (Configuration Change Protection):

Register: 0x0034 (CPU_CCP)
 Key for NVMCTRL: 0x9D (CCP_SPM_gc)
 Window: next four CPU instructions
 Rule: STS CPU_CCP must be immediately followed by STS NVMCTRL_CTRLA

BOOTEND fuse (fuse byte 8):

0x04 = 1 KB boot section (selfupdate.S fits here)
 App section starts at BOOTEND × 256

Key instructions used:

```
SUBI/SBCI    - 16-bit pointer adjustment for +/-128
I JMP       - indirect jump through Z (2 cy)
```

Build:

```
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
-o selfupdate.elf src/selfupdate.S
avr-objcopy -O ihex selfupdate.elf selfupdate.hex
pymcuprog write -d attiny3217 -t uart -f selfupdate.hex
```

27.12 Exercises

1. The `nvm_wait` polling loop costs 6 cycles per iteration while `FBUSY` is set at 3.333 MHz. A `PAGEERASEWRITE` (ERWP) operation on ATtiny3217 takes approximately 4 ms typical (datasheet Table 36-38). How many loop iterations does `nvm_wait` execute while waiting? At 3.333 MHz, what is the duration of each iteration in microseconds? Is a timer-based timeout (to detect a stuck NVM controller) practical within the same loop?
 2. The CCP window is four CPU instructions. Why does the source place `STS NVMCTRL_CTRLA`, `r16` immediately after `STS CPU_CCP`, `r17` even though the hardware allows a small window?
 3. The `nvm_write_page` routine fills the page buffer with 128 iterations of `LD r18, X+ + ST Z+, r18`. Why is the exact cycle count for the `ST` portion not the same as a normal SRAM store? Estimate the loop overhead, then compare it with the millisecond-scale NVM erase/write time.
 4. The self-update firmware in `selfupdate.S` does not verify the EOF record's checksum. Add checksum verification: if the EOF checksum is incorrect, send '!', turn LED0 on (it was turned off on successful EOF), and loop back to `record_loop` without jumping to the application. What register holds the checksum accumulator at the point of the EOF handler, and what condition do you test?
 5. The current `selfupdate.S` jumps to address `0x0000` on EOF (the reset vector). If `B00TEND = 0x04`, the application section begins at `0x0400`. Modify the EOF handler to: (a) check that at least one page in the application section was written (track the highest written address in a register), (b) jump to `0x0400` instead of `0x0000`. Show the assembly changes.
 6. Implement a **write-verify** step: after writing each page with `nvm_write_page`, read back the 128 bytes using `LPM Z+, r16` and compare against the SRAM buffer. If any byte mismatches, retransmit '!' (NACK) and re-request the record. Where in the `selfupdate.S` main loop would you add this? What additional register or memory is needed?
 7. Add a page-address guard that rejects any incoming data record below `B00TEND * 256`. On rejection, send '!' and discard the record data. Which two header bytes do you compare, and where should the check occur?
 8. Using `avr-objdump -d selfupdate.elf`, locate the two `STS` instructions that form the CCP sequence. Record their addresses. What is the byte offset between them? Confirm that no other instruction appears at an address between them. Now deliberately insert a NOP between them in the source and rebuild — does `avr-objdump` show the NOP between the two `STS` instructions? Describe the hardware consequence of this change.
-

Next: Chapter 26 — Bit Manipulation & Fixed-Point Math

Chapter 28

Bit Manipulation & Fixed-Point Math

AVR has no hardware divider and no floating-point unit. The ATtiny3217 does include the AVR hardware multiplier, but shift-and-add multiplication remains useful for understanding the machine model and for AVR parts that omit MUL. When computation must be fast and deterministic — sensor scaling, display formatting, signal processing — the answers live in bit manipulation and fixed-point arithmetic.

This chapter covers:

1. **Shift-and-add multiply** — multiplication using only shifts and adds; useful as a portable fallback and as a teaching algorithm.
2. **Integer square root** — a digit-by-digit root using only shifts, adds, and compares, with no multiply or divide.
3. **Fixed-point Q8.8 arithmetic** — representing and operating on fractional numbers in 16-bit register pairs.
4. **BCD arithmetic** — binary-coded decimal for human-readable display.
5. **Practical example** — convert a 16-bit binary sensor reading to BCD for a four-digit 7-segment display, using all three techniques.

The common thread is that all of these techniques trade mathematical elegance for predictability. On a small MCU, that is usually a good trade.

28.1 Shift Operations

AVR provides five shift/rotate instructions. All take a single register operand and execute in **1 cycle**.

Instruction	Operation	Flags
LSL Rd	$C \leftarrow Rd7$; $Rd \leftarrow Rd \ll 1$; $Rd0 = 0$	C, Z, N, V, H
LSR Rd	$Rd7 = 0$; $Rd \leftarrow Rd \gg 1$; $C \leftarrow Rd0$	C, Z, N, V
ASR Rd	$Rd7$ unchanged; $Rd \leftarrow Rd \gg 1$; $C \leftarrow Rd0$	C, Z, N, V
ROL Rd	$C \leftarrow Rd7$; $Rd \leftarrow Rd \ll 1$; $Rd0 = \text{old } C$	C, Z, N, V, H
ROR Rd	$Rd7 = \text{old } C$; $Rd \leftarrow Rd \gg 1$; $C \leftarrow Rd0$	C, Z, N, V

LSL/LSR: logical shift, zero fill. LSL Rd multiplies by 2; LSR Rd divides by 2 (unsigned).

ASR: arithmetic shift right — sign-extends during right shift, preserving the sign of a signed value. ASR Rd divides a signed byte by 2 (rounds toward negative infinity).

ROL/ROR: rotate through carry. The carry flag acts as a 9th bit. This is how multi-byte shifts are chained:

```
/* Logical left shift a 16-bit value in R25:R24 by 1 bit */
lsl  r24      /* shift low byte left; bit 7 → C */
rol  r25      /* shift high byte left; C → bit 0 */
```

That is the key idea behind most “wide” arithmetic on AVR. You do not get a 16-bit or 32-bit shift instruction; you build one byte at a time and let carry connect the pieces.

Multi-byte right shift (logical):

```
/* Logical right shift R25:R24 by 1 bit */
lsr  r25      /* shift high byte right; bit 0 → C */
ror  r24      /* shift low byte right; C → bit 7 */
```

28.1.1 Register Pair Diagram — 16-bit LSL

Before: R25[b7..b0] R24[b7..b0] └ (bit 0 = 0 after LSL)

After:

R24: b6 b5 b4 b3 b2 b1 b0 0 ← LSL r24 (b7 of R24 → C)
 R25: b6 b5 b4 b3 b2 b1 b0 C ← ROL r25 (C → bit 0 of R25)
 C = old R25[b7]

28.2 Shift-and-Add Multiplication

The MUL instruction ($8 \times 8 \rightarrow 16$ unsigned) exists on ATtiny3217 and many other AVR devices. Some smaller or older AVR parts omit hardware multiply, so the shift-and-add algorithm is still worth knowing.

The shift-and-add algorithm works on any AVR: for each bit in the multiplier, if the bit is set, add the multiplicand (shifted appropriately) to the accumulator.

28.2.1 Algorithm — Unsigned $8 \times 8 \rightarrow 16$

```
product = 0
for bit = 0 to 7:
  if multiplier[bit] == 1:
    product += multiplicand << bit
  (equivalently: shift multiplicand left each iteration,
   add if LSB of multiplier is 1, then shift multiplier right)
```

28.2.2 Implementation

Instruction	Operands	Cycles	Flags
LSL Rd	Rd	1	C,Z,N,V,H
LSR Rd	Rd	1	C,Z,N,V
ROL Rd	Rd	1	C,Z,N,V,H
ROR Rd	Rd	1	C,Z,N,V
ADD Rd,Rr	Rd, Rr	1	C,Z,N,V,H
ADC Rd,Rr	Rd, Rr	1	C,Z,N,V,H
BRCC k	k (−64..63)	1/2	—
DEC Rd	Rd	1	Z,N,V

```

/*
 * mul8u – unsigned 8×8 → 16 multiply (shift-and-add)
 *
 * Entry:  R16 = multiplicand
 *         R17 = multiplier
 * Exit:   R25:R24 = R16 × R17 (unsigned 16-bit product)
 * Clobbers: R16, R17, R18, R20
 * Cycles: 71 + popcount(multiplier), including setup and RET on AVRxt
 */
mul8u:
    clr    r24                /* product = 0 */
    clr    r25
    clr    r20                /* multiplicand high byte = 0 */
    ldi    r18, 8             /* 8 bits to process */
.Lmul8u_loop:
    lsr    r17                /* multiplier >>= 1; old bit 0 → C */
    brcc   .Lmul8u_skip
    add    r24, r16           /* product_lo += multiplicand_lo */
    adc    r25, r20           /* product_hi += multiplicand_hi + carry */
.Lmul8u_skip:
    lsl    r16                /* multiplicand_lo <<= 1; old bit 7 → C */
    rol    r20                /* multiplicand_hi <<= 1; C → bit 0 */
    dec    r18
    brne   .Lmul8u_loop
    ret

```

The key detail is that the multiplicand is tracked as a 16-bit value (R20:R16), so left shifts preserve the carry-out from the low byte.

That is what makes the routine correct for products larger than 255. If you shifted only the low byte, the upper half of the multiplicand would vanish.

Trace — 3×5 (i.e. $0b00000011 \times 0b00000101 = 15$):

r16=3, r17=5, r20=0, r24=0, r25=0

Iter 1: lsr r17 → r17=2, C=1 (bit 0 of 5)
 add r24, r16 → r24=3; adc r25, r20 → r25=0
 lsl r16 → r16=6, C=0; rol r20 → r20=0

Iter 2: lsr r17 → r17=1, C=0 (bit 0 of 2)
 skip
 lsl r16 → r16=12; rol r20 → r20=0

Iter 3: lsr r17 → r17=0, C=1 (bit 0 of 1)
 add r24, r16 → r24=3+12=15; adc r25, r20 → r25=0
 lsl r16 → r16=24; rol r20 → r20=0

Iters 4-8: r17=0, C=0 each time → all skip

Result: r25:r24 = $0x000F$ = 15 OK

28.2.3 Signed Multiply — mul8s (8×8 → 16 signed)

Signed multiply extends naturally: run the unsigned loop for 7 bits, then treat the MSB of the multiplier as negative (two's complement weight -128):

```

/*
 * mul8s – signed 8×8 → 16 multiply
 *
 * Entry:  R16 = multiplicand (signed)

```



```

*      R17 = multiplier    (signed)
* Exit:  R25:R24 = R16 × R17 (signed 16-bit product)
* Clobbers: R16, R17, R18, R20
*/
mul8s:
    clr    r24
    clr    r25
    /* sign-extend R16 into R20 */
    mov    r20, r16
    lsl    r20          /* old bit 7 → C */
    sbc    r20, r20      /* r20 = 0xFF if negative, 0x00 if positive */
    ldi    r18, 7        /* process 7 unsigned bits */
.Lmul8s_loop:
    lsr    r17
    brcc   .Lmul8s_skip
    add    r24, r16
    adc    r25, r20
.Lmul8s_skip:
    lsl    r16
    rol    r20
    dec    r18
    brne   .Lmul8s_loop
    /* bit 7 of multiplier: subtract instead of add (two's complement) */
    lsr    r17          /* bit 7 → C */
    brcc   .Lmul8s_done
    sub    r24, r16      /* product -= (multiplicand << 7) */
    sbc    r25, r20
.Lmul8s_done:
    ret

```

Sign extension via sbc r20, r20: After `lsl r20`, the carry flag holds bit 7 of the original multiplicand. `sbc r20, r20` subtracts r20 and C from r20 itself:

C = 0 (positive): $r20 - r20 - 0 = 0x00$

C = 1 (negative): $r20 - r20 - 1 = 0xFF$ (wraps, giving all-ones)

One instruction propagates the sign to a full byte — no branch, no `clr`, no conditional move.

SBC Rd, Rr subtracts Rr and the carry flag from Rd. Here, after `lsr r17`, C holds the sign bit of the original multiplier. If set, we subtract the current (shifted) multiplicand — the two’s complement correction for the negative weight of bit 7.

This is one of those places where two’s complement stops being abstract. The top bit is not just “another positive weight”; it contributes a negative term, so the final step has to subtract instead of add.

28.3 Integer Square Root

Square root is the reverse of the squaring you just built from shifts and adds, and it can be computed the same way: with shifts, adds, subtracts, and compares, and no multiply or divide at all. That makes it cheap on any AVR, including the reduced-core parts that have no hardware multiplier.

The method is the binary version of long-hand square root, taught here as “digit-by-digit.” Instead of one decimal digit at a time it resolves the result two bits at a time, because each new result bit weights the radicand by a factor of four. Keep two 16-bit quantities:

```

res    the square root built so far
bit    a single set bit walking down the powers of four:
        0x4000, 0x1000, 0x0400, ... , 0x0001

```

At each step the trial value is `res + bit`. Compare it against what remains of `N`:

```

if N >= res + bit:
    N    -= res + bit          ; this bit belongs in the root
    res  = (res >> 1) + bit
else:
    res  = (res >> 1)          ; this bit does not
bit >>= 2

```

`bit` starts at `0x4000`, which is 4^7 , the largest power of four that fits in 16 bits. Running a fixed eight steps then covers every input from 0 to 65535. Starting `bit` too high for a small `N` costs nothing: those first steps simply take the `else` branch and leave `res` at zero, so no alignment pre-loop is needed and the loop count is constant.

The routine in `src/isqrt16.S` takes `N` in `R25:R24` and returns `floor(sqrt(N))` in `R24`:

```

isqrt16:
    clr    r22                /* res = 0 (R23:R22)          */
    clr    r23
    ldi    r20, 0x00          /* bit = 0x4000 (R21:R20)   */
    ldi    r21, 0x40
    ldi    r18, 8             /* fixed iteration count    */
.Lsq_loop:
    movw   r26, r22           /* sum = res (R27:R26)      */
    add    r26, r20           /* sum += bit               */
    adc    r27, r21
    cp     r24, r26           /* compare N : sum         */
    cpc    r25, r27
    brlo   .Lsq_else         /* N < sum -> no subtract   */
    sub    r24, r26           /* N -= sum                */
    sbc    r25, r27
    lsr    r23               /* res >>= 1               */
    ror    r22
    add    r22, r20           /* res += bit              */
    adc    r23, r21
    rjmp   .Lsq_next
.Lsq_else:
    lsr    r23               /* res >>= 1               */
    ror    r22
.Lsq_next:
    lsr    r21               /* bit >>= 2               */
    ror    r20
    lsr    r21
    ror    r20
    dec    r18
    brne   .Lsq_loop
    mov    r24, r22          /* return low byte of res   */
    ret

```

Three details are worth pointing out:

- The 16-bit compare is `CP` then `CPC`. `CP` sets the carry from the low-byte subtraction; `CPC` continues it through the high byte. After the pair, carry is set exactly when `N < sum`, which is what `BRL0` tests (`BRL0` is the unsigned “lower” branch, an alias for `BRCS`). The subtract that follows on the taken path repeats the same arithmetic with `SUB/SBC`, so the values always match the comparison.

- `bit >>= 2` is two LSR/ROR pairs across the register pair, not one. Each result bit is worth four times the next, so the probe moves down two binary places per step.
- `res` is a 16-bit pair during the computation, but the answer for any 16-bit input is at most 255 ($\text{sqrt}(65535) = 255.99\dots$), so only the low byte R22 is returned. The high byte stays zero.

Because the loop always runs eight times, timing is nearly constant: 137 cycles for $N = 0$ up to 177 cycles for $N = 65535$ including the RET, the small spread coming only from whether each step takes the subtract path. The routine is 56 bytes of flash.

Build and disassemble it the same way as the other routines in this chapter:

```
avr-gcc -mmcu=attiny3217 -nostartfiles -o isqrt16.elf src/isqrt16.S
avr-objdump -d isqrt16.elf
```

A few results to check the routine against; each is $\text{floor}(\text{sqrt}(N))$:

```
isqrt16(0)      = 0
isqrt16(4)      = 2
isqrt16(15)     = 3      floor(3.87)
isqrt16(16)     = 4
isqrt16(200)    = 14     floor(14.14)
isqrt16(65535) = 255
```

Where does an integer square root earn its place? The most common use is computing a vector magnitude, $\text{sqrt}(x*x + y*y)$, for example the length of an accelerometer reading or the amplitude of an I/Q sample. That is also the job the CORDIC vectoring mode does in Chapter 29 without any square root at all. The trade is the familiar one from Chapter 23: `isqrt16` is small and exact but needs the squares formed first; CORDIC avoids both the squaring and the root at the cost of a fixed-iteration rotation and a scaling constant. Pick the one whose cost — flash, cycles, or accuracy — matches what is scarce.

28.4 Fixed-Point Q8.8 Arithmetic

Fixed-point represents a real number as an integer scaled by a power of two. **Q8.8** uses a 16-bit register pair where the high byte is the integer part and the low byte is the fractional part:

The advantage is that the CPU still performs ordinary integer arithmetic. The “fractional” meaning comes from how you interpret the bits, not from special hardware support.

Q8.8 value = register_pair / 256

Bit layout (R25:R24 example):

```

R25 [b15..b8]  R24 [b7..b0]
┌── integer ──┐┌── fraction ──┐
│               │               │
│               │               │
└── (×1)        └── (×1/256)    ┘
```

Range: -128.0 to +127.99609375 (signed)
 0.0 to 255.99609375 (unsigned)

Resolution: $1/256 \approx 0.0039$

28.4.1 Converting to Q8.8

```
/* Integer 42 → Q8.8: shift left 8 bits (multiply by 256) */
ldi  r24, 0 /* fractional part = 0 */
```

```
ldi    r25, 42          /* integer part = 42 */
/* R25:R24 = 0x2A00 = 10752 raw = 42.0 in Q8.8 */

/* Fractional constant 0.5 → Q8.8 = 128 = 0x0080 */
ldi    r24, 128
clr    r25
/* R25:R24 = 0x0080 = 0.5 in Q8.8 */

/* Convert from Q8.8 back to integer (truncate) */
mov     r16, r25         /* integer part = high byte */
```

28.4.2 Q8.8 Addition

Identical to 16-bit integer addition — no adjustment needed:

That simplicity is one of the main reasons fixed-point is attractive. Once two values share the same scaling, addition and subtraction are just normal integer operations.

```
/* R25:R24 += R23:R22 (Q8.8 + Q8.8) */
add     r24, r22
adc     r25, r23
```

28.4.3 Q8.8 Multiplication (Q8.8 × Q8.8 → Q8.8)

Two Q8.8 values multiplied give a Q16.16 result in a 32-bit space. To get a Q8.8 result, extract bits [23:8] of the 32-bit product — effectively shift right 8 bits:

```
A = a15..a0 (Q8.8)    B = b15..b0 (Q8.8)
A × B = 32-bit Q16.16
Q8.8 result = (A × B) >> 8 = bits [23:8] of the 32-bit product
```

In practice, build the 32-bit product as four 8×8 partial products:

```
A = AH:AL    (high byte : low byte)
B = BH:BL
```

```
A × B = AH×BH (weight 216) + AH×BL (weight 28)
        + AL×BH (weight 28) + AL×BL (weight 20)
```

```
Q8.8 result = shift right 8 → take bits [23:8]:
bits [15:8] of (AH×BL + AL×BH) + AH×BH (lower byte) + carry from AL×BL
```

The ATtiny3217 is an AVRxt device with the hardware multiply instructions available; Microchip's instruction set manual lists MUL, MULS, and MULSU as two-cycle operations that write the 16-bit product to R1:R0. The routine below deliberately uses mul8u from Section 26.2 so the byte decomposition is visible and so the same listing still works on smaller AVR devices that omit hardware multiply.

The bookkeeping looks tedious, but the idea is straightforward: break the 16-bit multiply into byte-sized pieces, then keep only the middle 16 bits that correspond to the Q8.8 result.

```
/*
 * mulq88 — unsigned Q8.8 × Q8.8 → Q8.8 using mul8u
 *
 * Entry:  R25:R24 = A (Q8.8), R23:R22 = B (Q8.8)
 * Exit:   R25:R24 = A × B (Q8.8, truncated)
 */
mulq88:
    push    r12
    push    r13
    push    r14
```

```

push    r15

mov     r12, r24      /* AL */
mov     r13, r25      /* AH */
mov     r14, r22      /* BL */
mov     r15, r23      /* BH */

/* AL × BL */
mov     r16, r12
mov     r17, r14
rcall   mul8u
mov     r21, r25      /* result low byte starts with high(AL×BL) */
clr     r22           /* result high byte */

/* AL × BH */
mov     r16, r12
mov     r17, r15
rcall   mul8u
add     r21, r24
adc     r22, r25

/* AH × BL */
mov     r16, r13
mov     r17, r14
rcall   mul8u
add     r21, r24
adc     r22, r25

/* AH × BH */
mov     r16, r13
mov     r17, r15
rcall   mul8u
add     r22, r24      /* low byte of AH×BH contributes at result high byte */

mov     r24, r21
mov     r25, r22
pop     r15
pop     r14
pop     r13
pop     r12
ret

```

Trace — 1.5×1.5 (Q8.8: $0x0180 \times 0x0180 \rightarrow 0x0240 = 2.25$):

AH = 0x01, AL = 0x80 BH = 0x01, BL = 0x80

AL×BL: $0x80 \times 0x80 = 0x4000 \rightarrow r25:r24 = 0x40:0x00$
 r21 = 0x40 (high byte carries fractional weight)

AL×BH: $0x80 \times 0x01 = 0x0080 \rightarrow r25:r24 = 0x00:0x80$
 r21 += 0x80 → r21 = 0xC0

AH×BL: $0x01 \times 0x80 = 0x0080 \rightarrow r25:r24 = 0x00:0x80$
 r21 += 0x80 → r21 = 0x40, carry = 1
 r22 += 0x00 + carry → r22 = 0x01

AH×BH: $0x01 \times 0x01 = 0x0001 \rightarrow r25:r24 = 0x00:0x01$
 r22 += 0x01 → r22 = 0x02

Result: $r25:r24 \leftarrow r22:r21 = 0x02:0x40 = 0x0240 = 2.25$ in Q8.8 ✓

The two cross-terms ($AL \times BH$ and $AH \times BL$) both carry weight 2^8 — they accumulate into $r21$, which becomes the fractional byte of the result. Their combined carry feeds into $r22$, the integer byte. That is why bits [23:8] of the 32-bit product are the correct Q8.8 result.

Full listing in `src/fixedpoint.S`.

28.5 BCD Arithmetic

Binary-Coded Decimal stores each decimal digit as a 4-bit nibble. Two formats:

Unpacked BCD: one digit per byte (low nibble used, high nibble = 0)

Decimal 47 → byte 0x04, byte 0x07

Packed BCD: two digits per byte

Decimal 47 → single byte 0x47

28.5.1 Decimal Adjust on AVR

Unlike some 8-bit CPUs, AVR does **not** provide a general DAA instruction for packed-BCD correction. After a binary ADD/ADC, packed BCD must be corrected manually using the half-carry and carry flags.

28.5.2 H Flag and BCD Correction

The H (half-carry) flag is set when a carry propagates from bit 3 to bit 4 during addition — i.e., when the low nibble overflows. This is exactly the condition that requires BCD correction:

After binary ADD of two packed BCD bytes:

If low nibble > 9 OR H flag set: add 0x06 to correct low nibble

If high nibble > 9 OR C flag set: add 0x60 to correct high nibble

```
/*
 * bcd_add – add two single packed BCD bytes
 *
 * Entry:  R16 = BCD byte A (e.g. 0x47 for decimal 47)
 *         R17 = BCD byte B
 * Exit:   R16 = BCD sum (low byte), C = 1 if result > 99
 * Clobbers: R18, R19, R20
 */
bcd_add:
    add    r16, r17        /* binary add */
    clr    r18              /* correction byte: 0x00, 0x06, 0x60, or 0x66 */
    clr    r19              /* decimal carry-out marker */

    /* Capture H and C from the original ADD before later tests change SREG. */
    brhc   .Lbcd_no_h
    ori    r18, 0x06
.Lbcd_no_h:
    brcc   .Lbcd_no_c
    ori    r18, 0x60
    ldi    r19, 1
.Lbcd_no_c:

    /* Low nibble > 9 also needs +0x06. */
```

```

mov    r20, r16
andi   r20, 0x0F
cpi    r20, 10
brlo   .Lbcd_check_high
ori    r18, 0x06

.Lbcd_check_high:
/* High nibble > 9 also needs +0x60 and a decimal carry out. */
mov    r20, r16
andi   r20, 0xF0
cpi    r20, 0xA0
brlo   .Lbcd_done
ori    r18, 0x60
ldi    r19, 1
.Lbcd_done:
add    r16, r18
tst    r19
breq   .Lbcd_no_decimal_carry
sec
ret
.Lbcd_no_decimal_carry:
clc
ret

```

BRHS (Branch if Half-carry Set) and BRHC (Branch if Half-carry Clear) branch directly on the H flag.

That matters because packed BCD correction is really nibble arithmetic. The H flag tells you whether the low decimal digit overflowed even when the full byte did not.

Instruction	Branch condition	Cycles
BRHS k	H = 1	1/2
BRHC k	H = 0	1/2

28.6 Binary to BCD Conversion

Converting a multi-digit binary number to BCD for display is a recurring task. The **double-dabble** (shift-and-add-3) algorithm converts an N-bit binary number to packed BCD in N iterations, requiring no division:

Algorithm:

Place binary value in a shift register alongside N/4 BCD digits (all zero).

Repeat N times:

For each BCD digit: if digit > 4, add 3 to that digit.

Shift the entire combined register left by 1 bit.

After N iterations, BCD digits contain the decimal representation.

The “add 3” step ensures that a BCD digit that is about to overflow (≥ 5) will produce the correct carry into the next BCD digit after the shift. Shifting left doubles the digit, and $5 \times 2 = 10$ cannot fit in a single BCD nibble. Pre-adding 3 fixes this: $(5+3) \times 2 = 16 = 0b1_0000$, which leaves 0 in this nibble and carries 1 into the next-higher digit — exactly the decimal “10” that 5×2 should produce.

If double-dabble feels magical at first, focus on that one fact: digits 5..9 need help before the shift so that a binary left shift turns into a valid decimal carry.

28.6.1 16-bit Binary to 5-Digit BCD

A 16-bit value (0–65535) needs 5 BCD digits (0–9 each), packed into 3 bytes (two packed + one partial):

Packed BCD storage: [tens-thousands][thousands|hundreds][tens|ones]

Example: 65535 → 0x06, 0x55, 0x35

```

/*
 * bin16_to_bcd – convert 16-bit unsigned binary to 5 packed BCD digits
 *
 * Entry:  R25:R24 = 16-bit value (0–65535)
 * Exit:   R21:R20:R19 = packed BCD (5 digits)
 *         R21[3:0] = ten-thousands digit
 *         R20[7:4] = thousands digit
 *         R20[3:0] = hundreds digit
 *         R19[7:4] = tens digit
 *         R19[3:0] = ones digit
 * Clobbers: R16, R17, R22
 */
bin16_to_bcd:
    clr    r19          /* BCD digits [tens|ones] = 0 */
    clr    r20          /* BCD digits [thousands|hundreds] = 0 */
    clr    r21          /* BCD digit [ten-thousands] = 0 */
    ldi    r16, 16      /* loop counter: 16 bits to process */

.Lbcd_loop:
    /* For each BCD nibble: if > 4, add 3 */
    rcall  bcd_adjust   /* adjust R21:R20:R19 */

    /* Shift binary value and BCD digits left by 1 */
    lsl    r24          /* binary low byte: bit 7 → C */
    rol    r25          /* binary high byte: C → bit 0; bit 7 → C */
    rol    r19          /* BCD byte 0: C in, bit 7 → C */
    rol    r20          /* BCD byte 1: C in, bit 7 → C */
    rol    r21          /* BCD byte 2: C in (no further shift needed) */

    dec    r16
    brne   .Lbcd_loop
    ret

/*
 * bcd_adjust – add 3 to any BCD nibble > 4 in R21:R20:R19
 */
bcd_adjust:
    /* Check and adjust each nibble */
    /* R19 low nibble (ones) */
    mov    r22, r19
    andi   r22, 0x0F
    cpi    r22, 5
    brlo   .Lbcd_adj_r19h
    subi   r19, -3      /* add 3 to low nibble */

.Lbcd_adj_r19h:
    /* R19 high nibble (tens) */
    mov    r22, r19
    andi   r22, 0xF0
    cpi    r22, 0x50
    brlo   .Lbcd_adj_r20l
    subi   r19, -0x30

.Lbcd_adj_r20l:
    mov    r22, r20
    andi   r22, 0x0F
    cpi    r22, 5
    brlo   .Lbcd_adj_r20h
    subi   r20, -3

.Lbcd_adj_r20h:
    /* R20 high nibble (thousands) */
    mov    r22, r20
    andi   r22, 0xF0
    cpi    r22, 0x50
    brlo   .Lbcd_adj_r21
    subi   r20, -0x30

.Lbcd_adj_r21:
    /* R21 (ten-thousands) */
    mov    r22, r21
    andi   r22, 0x0F
    cpi    r22, 5
    brlo   .Lbcd_adj_r21h
    subi   r21, -3

.Lbcd_adj_r21h:
    /* R21 high nibble (no further adjustment needed) */
    mov    r22, r21
    andi   r22, 0xF0
    cpi    r22, 0x50
    brlo   .Lbcd_adj_r21h
    subi   r21, -0x30

```



```

.Lbcd_adj_r20l:
    /* R20 low nibble (hundreds) */
    mov    r22, r20
    andi   r22, 0x0F
    cpi    r22, 5
    brlo   .Lbcd_adj_r20h
    subi   r20, -3
.Lbcd_adj_r20h:
    /* R20 high nibble (thousands) */
    mov    r22, r20
    andi   r22, 0xF0
    cpi    r22, 0x50
    brlo   .Lbcd_adj_r21
    subi   r20, -0x30
.Lbcd_adj_r21:
    /* R21 low nibble (ten-thousands) – max digit is 6, never > 4 until late */
    mov    r22, r21
    andi   r22, 0x0F
    cpi    r22, 5
    brlo   .Lbcd_adj_done
    subi   r21, -3
.Lbcd_adj_done:
    ret

```

28.6.2 Build and Test

```

avr-gcc -mmcu=attiny3217 -nostartfiles \
        -o bin2bcd.elf src/bin16_to_bcd.S
avr-objdump -d bin2bcd.elf

```

To check the result, step the routine under `avr-gdb` on the Curiosity Nano (see Chapter 5) and read R21:R20:R19 after it returns, or trace the loop by hand from the disassembly.

28.7 Complete Example: Sensor Reading to 7-Segment Display

A 10-bit ADC reading (0–1023) from a temperature sensor is scaled to a °C value (0–99.9°C, one decimal place), then displayed on a four-digit 7-segment display interface connected over SPI.

28.7.1 Scaling: ADC → Temperature

Assume linear sensor: $0 \rightarrow 0.0^{\circ}\text{C}$, $1023 \rightarrow 99.9^{\circ}\text{C}$.

```
temp10 = (adc_reading × 63998 + 32768) >> 16
```

Where $63998 = \text{round}(999 / 1023 \times 65536)$. The result is an integer in units of 0.1°C , so 365 means 36.5°C . The added 32768 rounds before the final right shift.

The same scaling can be framed with a Q8.8 factor:

```
scale_factor = 99.9 × 256 / 1023 ≈ 25.0 → Q8.8 value 0x1900
temp = (adc × scale_factor) >> 8
```

But `adc` (0–1023) does not fit the 8-bit integer field of Q8.8, and the product `adc × 0x1900` reaches ~26 bits anyway, so this buys nothing over the 32-bit fixed-shift form used below

— which also folds the rounding constant in directly.

The full example is in `src/display.S`. Below are the key routines.

This example is intentionally end-to-end. It shows why bit tricks and numeric formats matter: eventually the value has to become something a human can read.

28.7.2 Register Map for `display.S`

R25:R24 ADC reading (10-bit, 0–1023)
 R23:R22 scaled temperature × 10 (e.g. 365 = 36.5°C)
 R21:R20:R19 BCD digits (five digits packed)
 R18 digit index (0–3)
 R17 segment data byte to SPI
 R16 scratch
 Z (R31:R30) pointer to 7-segment LUT in flash

28.7.3 7-Segment LUT

```
.section .rodata
seg7_lut:
/* segments: gfedcba (bit 6..0), active high */
.byte 0x3F /* 0: abcdef = 0b0111111 */
.byte 0x06 /* 1: bc     = 0b0000110 */
.byte 0x5B /* 2: abdeg  = 0b1011011 */
.byte 0x4F /* 3: abcdg  = 0b1001111 */
.byte 0x66 /* 4: bcfg    = 0b1100110 */
.byte 0x6D /* 5: acdfg    = 0b1101101 */
.byte 0x7D /* 6: acdefg   = 0b1111101 */
.byte 0x07 /* 7: abc      = 0b0000111 */
.byte 0x7F /* 8: all     = 0b1111111 */
.byte 0x6F /* 9: abcdfg   = 0b1101111
```

28.7.4 Scale ADC to Temperature

The implementation uses a 32-bit intermediate to avoid overflow (`adc × 63998` peaks at ~65 M, far past 16-bit range). Four 8×8 partial products with hardware MUL build the 32-bit result:

```
/*
 * scale_adc - 10-bit ADC reading → temperature × 10 (0–999)
 *
 * Formula: temp10 = (adc × 63998 + 32768) >> 16
 *          63998 = 0xF9FE = round(999 / 1023 × 65536)
 *          +32768 rounds to nearest before the final right-shift.
 *
 * Entry:  R25:R24 = ADC (0–1023)
 * Exit:   R23:R22 = temp × 10 (0–999)
 * Clobbers: R0, R1, R16–R21
 */
scale_adc:
    clr    r16                /* 32-bit accumulator R19:R18:R17:R16 = 0 */
    clr    r17
    clr    r18
    clr    r19
    clr    r21                /* zero for adc-with-zero carry propagation */

    ldi    r20, lo8(63998)
```

```

mul    r24, r20          /* ADC_lo × scale_lo → bits [15:0] */
movw   r16, r0

mul    r25, r20          /* ADC_hi × scale_lo → bits [23:8] */
add    r17, r0
adc    r18, r1
adc    r19, r21

ldi    r20, hi8(63998)

mul    r24, r20          /* ADC_lo × scale_hi → bits [23:8] */
add    r17, r0
adc    r18, r1
adc    r19, r21

mul    r25, r20          /* ADC_hi × scale_hi → bits [31:16] */
add    r18, r0
adc    r19, r1

ldi    r20, 0x80          /* round: add 0x00008000 before >> 16 */
add    r17, r20
adc    r18, r21
adc    r19, r21

clr    r1                /* restore zero-register convention after MUL */
movw   r22, r18          /* R23:R22 = bits [31:16] = temp × 10 */
ret

```

The final answer (0–999) fits in 10 bits, but the intermediate $\text{adc} \times 63998$ needs 26 bits — a 16-bit-only approach overflows silently. Keeping 32 bits through the multiply and discarding the bottom 16 after rounding is the standard fixed-point scaling pattern.

28.7.5 Extract BCD Digits and Drive SPI

```

/*
 * display_temp – convert R23:R22 (temp×10) to BCD and send to SPI display
 */
/* Digit layout: [blank][tens][ones].[tenths]
 * Entry: R23:R22 = temperature × 10 (0–999)
 */
display_temp:
    movw   r24, r22
    rcall  bin16_to_bcd /* R21:R20:R19 = BCD digits */

    /* temp×10 = abc in decimal corresponds to display "ab.c":
       hundreds digit = tens of degrees
       tens digit     = ones of degrees
       ones digit     = tenths of degrees. */

    /* Send digit 3: always blank for the 0.0..99.9 range */
    ldi    r17, 0x00 /* blank segment */
    rcall  spi_send

    /* Send digit 2: tens of degrees */
    mov    r16, r20
    andi   r16, 0x0F
    rcall  send_digit

```

```

/* Send digit 1: ones of degrees, with decimal point */
mov  r16, r19
swap r16
andi r16, 0x0F
rcall send_digit_dp

/* Send digit 0: tenths of degrees */
mov  r16, r19
andi r16, 0x0F
rcall send_digit
ret

/*
 * send_digit - look up 7-seg pattern for digit in R16, send via SPI
 */
send_digit:
    ldi  ZL, lo8(seg7_lut)
    ldi  ZH, hi8(seg7_lut)
    add  ZL, r16      /* index into LUT */
    adc  ZH, r1
    lpm  r17, Z       /* r17 = segment pattern */
    rjmp spi_send

/*
 * send_digit_dp - same as send_digit but also lights the decimal point
 *
 * Most 7-segment driver ICs map the decimal point to bit 7 of the
 * segment byte (the 'h' segment). OR-ing 0x80 sets that bit without
 * disturbing the digit pattern.
 */
send_digit_dp:
    ldi  ZL, lo8(seg7_lut)
    ldi  ZH, hi8(seg7_lut)
    add  ZL, r16      /* index into LUT */
    adc  ZH, r1
    lpm  r17, Z
    ori  r17, 0x80     /* set decimal-point bit */
    rjmp spi_send

/*
 * spi_send - send R17 via SPI (polling)
 */
spi_send:
    sts  SPDR, r17     /* load SPI data register */
.Lspi_wait:
    lds  r16, SPSR
    sbrs r16, SPIF     /* wait for SPIF (transfer complete) bit */
    rjmp .Lspi_wait
ret

```

28.7.6 Build

```

# display.S in this directory targets ATmega328P SPI hardware
avr-gcc -mmcu=atmega328p -nostartfiles \
    -o display.elf src/display.S src/fixedpoint.S src/mul8u.S src/bin16_to_bcd.S
avr-objcopy -O ihex display.elf display.hex

```

avr-size display.elf

28.8 Instruction Reference for This Chapter

Instruction	Syntax	Cycles	Flags	Notes
LSL	lsl Rd	1	C,Z,N,V,H	Rd ← Rd«1; C←Rd7
LSR	lsr Rd	1	C,Z,N,V	Rd ← Rd»1; C←Rd0
ASR	asr Rd	1	C,Z,N,V	Signed »1, sign extends
ROL	rol Rd	1	C,Z,N,V,H	Rotate left through C
ROR	ror Rd	1	C,Z,N,V	Rotate right through C
SWAP	swap Rd	1	—	Exchange high/low nibbles
MUL	mul Rd,Rr	2	Z,C	R1:R0 ← Rd×Rr unsigned
MULS	muls Rd,Rr	2	Z,C	Signed × signed
MULSU	mulsu Rd,Rr	2	Z,C	Signed × unsigned
BRHS	brhs k	1/2	—	Branch if H set
BRHC	brhc k	1/2	—	Branch if H clear
ADIW	adiw Rd,K	2	Z,C,N,V,S	Add immediate (0-63) to register pair

SWAP Rd is extremely useful for packed BCD: it moves the high nibble to the low position for digit extraction without shifting:

That is much cleaner than doing four single-bit shifts just to expose one decimal digit.

```
/* Extract high nibble of R16 as a value 0-15 */
swap r16          /* high nibble → low nibble */
andi r16, 0x0F    /* mask off high bits */
```

28.9 Common Pitfalls

28.9.1 Pitfall 1 — Using R1 Without Zeroing After MUL

MUL, Muls, and MULSU write the result to R1:R0. Many AVR assembly projects reserve R1 as a zero register because it makes multi-byte arithmetic and pointer math simpler. If your project follows that convention, copy the product out and restore R1 before code that expects it to be zero:

```
mul    r16, r17
mov    r24, r0      /* use result */
mov    r25, r1
clr    r1           /* restore project zero-register convention */
```

28.9.2 Pitfall 2 — ASR vs LSR on Signed Values

LSR fills the MSB with 0, which is wrong for negative values:

```
-8 in 8-bit two's complement: 0b11111000 = 0xF8
LSR → 0b01111100 = 0x7C = +124  WRONG
ASR → 0b11111100 = 0xFC = -4    CORRECT (-8 / 2)
```

Always use ASR for signed right shifts.

If you pick the wrong shift instruction on signed data, the result is not “slightly off.” It is usually catastrophically wrong.

28.9.3 Pitfall 3 — Q8.8 Overflow

Q8.8 integer range is ± 127 (signed) or 0–255 (unsigned). Multiplication of two large Q8.8 values easily overflows:

```
200.0 × 200.0 = 40000.0 — exceeds unsigned Q8.8 max (255.99)
```

Check that the mathematical range fits before choosing Q8.8. More *integer* bits buy range — Q12.4 reaches 4095.9; more *fraction* bits buy resolution — Q4.12 resolves 1/4096 but spans only 0–15.99. A product like $200 \times 200 = 40000$ exceeds every 16-bit fixed-point format and needs a 32-bit result.

Choosing a fixed-point format is mostly a range-versus-resolution trade. More fraction bits give you finer steps; more integer bits give you more headroom.

28.9.4 Pitfall 4 — Decide BCD Corrections From the Original Sum

Compute every correction from the **un-corrected** binary sum, then apply them. The two additions themselves commute — $+0x06$ then $+0x60$ lands on the same byte as $+0x60$ then $+0x06$ — so the order of the *adds* is irrelevant. The bug to avoid is *re-testing* a nibble after you have already modified it:

```
/* WRONG: test the high nibble AFTER the low-nibble fix has run.
   The +0x06 can carry into the high nibble and corrupt that decision. */
subi  r16, -0x06    /* low-nibble correction; may carry into bits 7:4 */
/* ...now testing the high nibble of r16 sees that carry — wrong basis... */

/* CORRECT: capture H and C and test both nibbles of the original sum
   first (as bcd_add does), then add the combined correction once. */
```

28.9.5 Pitfall 5 — Shift Count for Multi-Byte Values

AVR shifts operate on single registers only. There is no `lsl r25:r24` opcode. Multi-byte shifts require one instruction per byte:

```
/* 24-bit left shift (R22:R21:R20) */
lsl  r20
rol  r21
rol  r22
```

Forget the middle rol and bits disappear silently.

That is a recurring AVR pattern: multi-byte arithmetic usually fails by losing a carry in the middle, not by producing an obvious syntax error.

28.10 Summary

Shift recap:

- LSL / LSR – logical shift (unsigned), zero fill
- ASR – arithmetic right shift (signed), sign extend
- ROL / ROR – rotate through C; chain with LSL/LSR for multi-byte shifts
- SWAP – swap nibbles in 1 cycle; essential for packed BCD

Shift-and-add multiply (portable fallback when hardware MUL is unavailable):

8 iterations: lsr multiplier; brcc skip; add accumulator; shift 16-bit multiplicand

71..79 cycles for 8×8 vs 2 cycles for MUL on AVRxt devices such as ATtiny3217

Integer square root (digit-by-digit, no MUL or divide):

8 fixed steps: trial = res+bit; if N>=trial subtract and set bit; bit >>= 2
16-bit input → 8-bit result; 137..177 cycles; 56 bytes flash

Fixed-point Q8.8:

value = raw_int / 256
add: same as 16-bit integer add
multiply: 4 partial products, extract bits [23:8] of 32-bit result

BCD conversion (double-dabble):

16 iterations of: add-3-if-nibble>4, then shift-left
No division, no LUT, bounded data-dependent cycle count

Binary-to-BCD use cases:

ADC reading → decimal display
Timer counter → MM:SS display
Any sensor value needing human-readable output

Build:

```
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs -o ch17.elf src/mul8u.S src/isqrt16.S
avr-objdump -d ch17.elf # verify cycle counts
avr-size ch17.elf # check flash budget
```

Microchip source notes:

AVR Instruction Set Manual, device-core appendix: ATtiny3217 is AVRxt and its missing-instruction list does not include MUL/MULS/MULSU
AVR Instruction Set Manual, MUL/MULS/MULSU descriptions: 8×8 products write R1:R0 and execute in 2 cycles on AVRxt

28.11 Exercises

1. The `mul8u` routine takes 71..79 cycles, depending on multiplier bits. For a specific use case — multiplying an 8-bit value by the constant 17 — write a dedicated routine using only shifts, adds, and moves that computes the result much faster than the general routine. (Hint: $17 = 16 + 1$.)
2. Q4.12 format stores 4 integer bits and 12 fractional bits in 16 bits. Write the Q4.12 multiply: two Q4.12 values $\rightarrow$ Q4.12 result. Where do you extract bits from the 32-bit intermediate product? What is the new maximum integer value?
3. The `bin16_to_bcd` routine calls `bcd_adjust` 16 times — once per shift. Each `bcd_adjust` call checks all 5 nibbles. How many of those 5-nibble checks do useful work on iteration 1 (when all BCD bytes are 0)? Optimise `bcd_adjust` to skip nibbles that cannot yet be > 4 based on loop iteration count. How many instruction words does this save?
4. Implement `bcd16_add` — add two 4-digit packed BCD values (two bytes each, representing 0000-9999) with carry out. Use the H flag and manual correction. Test with $9999 + 0001$ (result: 0000, carry = 1).
5. The `seg7_lut` uses active-high segment encoding. Many 7-segment driver ICs use active-low (a 0 bit turns the segment on). Rewrite `send_digit` to invert the LUT byte before sending. Use a single `COM Rd` instruction (one's complement, 1 cycle) — do not rewrite the LUT itself.
6. Profile `display_temp` using `avr-objdump -d`. Count the cycles for the path where no digit is blank. Then count cycles for the path where the hundreds digit is zero (blanked). Which path is longer and by how much?
7. The double-dabble loop shifts the binary input value leftward until it is consumed. After 16 iterations, `R25:R24 = 0`. Verify this is always true regardless of the input value, and explain why it does not cause a problem that `R25:R24` is destroyed during the conversion.
8. `isqrt16` always runs eight iterations. Trace it for `N = 0` and confirm the first four steps take the `else` branch. Then add an alignment pre-loop that shifts `bit` down past leading zero pairs before the main loop, and decide whether the saved iterations are worth the extra code for a routine that is already nearly constant-time.
9. Extend the idea to a 32-bit input producing a 16-bit root. How many iterations are needed, what is the new starting value of `bit`, and which register pairs grow to hold `N`, `res`, and the trial value?

Next: Chapter 27 — Real-Time Patterns

Chapter 29

Real-Time Patterns

Real-time embedded systems must respond to events within bounded, predictable time. An AVR running bare-metal assembly has no OS scheduler, no kernel, and no hidden overhead — every cycle is accounted for. That is both its strength and its responsibility.

This chapter covers:

1. **Cooperative task scheduling** — a minimal round-robin scheduler in assembly; no dynamic allocation, no context switch overhead beyond register saves.
2. **Deterministic ISR latency analysis** — counting cycles from interrupt trigger to first ISR instruction; identifying and eliminating latency sources.
3. **Example: two-task scheduler with TCB0** — a TCB0 periodic interrupt fires every 1 ms; cooperative task switching drives an LED blinker and a USART heartbeat sender on the ATtiny3217 Curiosity Nano.

The main goal is not to build a tiny preemptive kernel. It is to show how timing, interrupts, and task structure fit together when you want behavior that stays bounded and explainable.

29.1 Real-Time Fundamentals

A real-time system is not necessarily fast — it is **predictable**. Two key properties:

Deadline: a task must complete before a specific time. Missing a deadline is a failure, regardless of whether the computation eventually finishes.

Determinism: given the same inputs, execution time is bounded and known in advance. The bound must hold under all interrupt patterns and all data values.

29.1.1 Two Scheduling Models

Preemptive: a higher-priority task can interrupt a lower-priority task at any instruction boundary. The interrupted task's full register state must be saved and restored, usually on a per-task stack.

Cooperative: tasks run to completion (or to an explicit yield point) before the scheduler runs the next task. No mid-task preemption. Context switch is cheaper; latency is bounded only by the longest task segment between yields.

That last clause is the defining tradeoff. Cooperative systems are simple because nothing interrupts a task halfway through, but that also means one task can delay everything behind it if it runs too long.

This chapter implements cooperative scheduling. It is appropriate when:

- All tasks have known, short run times.
- The system has only a few tasks (two to eight is typical on an 8-bit AVR).
- Worst-case response time = sum of all task segments + scheduler overhead, and that total meets all deadlines.

If you can write that sum down and it fits comfortably inside your timing budget, cooperative scheduling is often the cleanest answer on an AVR.

29.2 Cooperative Task Scheduler Design

29.2.1 Task Representation

Each task is a subroutine that runs, does some work, and returns. The scheduler calls each task in rotation. A task that needs to wait (e.g., for a UART byte) must either poll with a non-blocking check and return immediately, or use a flag set by an ISR.

Scheduler main loop:

```
loop forever:
    call task_0
    call task_1
    call task_2
    ...
```

This is the simplest possible cooperative scheduler. Its properties:

- **No stack switch:** each task runs on the same stack, called directly.
- **No saved context:** tasks return normally; all register state is the task's own responsibility (save what you use).
- **Tick-driven variant:** tasks are only called when a timer tick has elapsed, providing a fixed time base.

Notice what is missing: there is no saved "task state" in CPU registers and no separate per-task stack. These tasks are really just well-behaved subroutines that run on a schedule.

29.2.2 Tick-Driven Scheduler

A TCB0 periodic interrupt fires every 1 ms (the *tick*). Each tick sets a flag. The main loop spins until the flag is set, clears it, then runs all tasks:

```
tcb0_isr:
    ldi    r16, 1
    sts    tick_flag, r16
    reti

main_loop:
.lwait_tick:
    lds    r16, tick_flag
    tst    r16
    breq   .lwait_tick
    clr    r16
    sts    tick_flag, r16
    rcall  task_led
    rcall  task_uart
    rjmp   main_loop
```

In assembly, `tick_flag` is a byte in SRAM (`.bss`). The ISR sets it; the main loop polls and clears it.

That one-byte handoff is a classic embedded pattern: ISR records that an event happened, main-loop code does the slower work later.

29.2.3 Task Counter Pattern

Each task maintains its own counter and acts only when its counter reaches its period:

```
task_led:
    increment led_counter
    compare led_counter with LED_PERIOD
    branch to task_led_done when below period
    clear led_counter
    toggle LED
task_led_done:
    ret

task_uart:
    increment uart_counter
    compare uart_counter with UART_PERIOD
    branch to task_uart_done when below period
    clear uart_counter
    send heartbeat byte
task_uart_done:
    ret
```

With a 1 ms tick: - LED_PERIOD = 500 → LED toggles every 500 ms (1 Hz blink) - UART_PERIOD = 1000 → UART byte sent every 1 s

This pattern scales well because each task keeps its own notion of time without needing a complex global scheduler.

29.3 TCB0 Configuration — 1 ms Tick

On the ATtiny3217 Curiosity Nano the CPU runs at **3.333 MHz** (internal 20 MHz oscillator divided by 6, set by CLKCTRL_MCLKCTRLB).

TCB0 in **Periodic Interrupt mode** (CNTMODE = 0) counts up from 0, reloads from CCMP when it matches, and fires an interrupt on each match. With no prescaler (CLK_PER):

```
CCMP = (F_CPU / F_tick) - 1
      = (3 333 333 / 1000) - 1
      ≈ 3333.333 - 1
      = 3332 (0x0D04)
```

CCMP = 3332 gives a 3333-cycle period, about 999.9 μs at 3.333333 MHz. The remaining fractional cycle is far below the timing error of the default oscillator.

29.3.1 TCB0 Register Map

Address	Name	Description
0x0A40	TCB0_CTRLA	Control A – clock select, enable
0x0A41	TCB0_CTRLB	Control B – mode select
0x0A44	TCB0_EVCTRL	Event control
0x0A45	TCB0_INTCTRL	Interrupt enable (bit 0: CAPT)
0x0A46	TCB0_INTFLAGS	Interrupt flags (bit 0: CAPT; write 1 to clear)
0x0A4A	TCB0_CNTL	Counter low byte
0x0A4B	TCB0_CNTH	Counter high byte

0x0A4C TCB0_CCMP_L Compare/capture match low byte
 0x0A4D TCB0_CCMP_H Compare/capture match high byte

TCB0_CTRLA bit layout:

Bit	Name	Description
0	ENABLE	1 = timer runs
2:1	CLKSEL	00 = CLK_PER (no prescaler) 01 = CLK_PER/2 11 = CLK_TCA (use TCA0's clock output)
4	SYNCUPD	restart TCB when TCA0 restarts/overflows
6	RUNSTDBY	run in Standby sleep mode

TCB0_CTRLB bit layout:

Bit	Name	Description
2:0	CNTMODE	000 = Periodic Interrupt (INT mode) 001 = Timeout Check 010 = Input Capture on Event 011 = Input Capture Frequency Measurement 100 = Input Capture Pulse-Width Measurement 101 = Input Capture Frequency and Pulse-Width Measurement 110 = Single Shot 111 = 8-bit PWM

29.3.2 Initialization Code

```
/* tcb0_init – configure TCB0 for 1 ms periodic interrupt
 *
 * F_CPU = 3 333 333 Hz, CCMP = 3332 = 0x0D04
 * Clobbers: R16, R17
 */
tcb0_init:
    /* Set CCMP = 3332 (low byte first, high byte second – TCB locks on CCMPH write) */
    ldi r16, lo8(3332)      /* 0x04 */
    sts TCB0_CCMP_L, r16
    ldi r16, hi8(3332)      /* 0x0D */
    sts TCB0_CCMP_H, r16

    /* Mode: periodic interrupt (CNTMODE = 0) */
    ldi r16, 0
    sts TCB0_CTRLB, r16

    /* Enable CAPT interrupt */
    ldi r16, TCB_CAPT_bm     /* bit 0 */
    sts TCB0_INTCTRL, r16

    /* Enable timer, clock = CLK_PER (no prescaler) */
    ldi r16, TCB_ENABLE_bm  /* bit 0 */
    sts TCB0_CTRLA, r16

    ret
```

Instruction	Cycles
LDS Rd, k	3
STS k, Rr	2

LDS/STS access SRAM-mapped peripherals (addresses > 0x001F). On AVRxt, LDS costs 3 cycles and STS costs 2 cycles.

That is an easy place to lose cycles in timer setup and ISR code on newer AVR parts: many peripheral registers are no longer reachable with the short I/O instructions.

29.4 ATtiny3217 Interrupt Vector Table

Vectors are 4 bytes each (one JMP instruction). The table starts at address 0x0000 in flash.

Byte addr	Vector #	Source	Description
0x0000	0	RESET	Power-on, external, WDT, BOD reset
0x0004	1	CRCSCAN_NMI	CRC scan non-maskable interrupt
0x0008	2	BOD_VLM	Brown-out detector voltage level monitor
0x000C	3	PORTA_PORT	Port A pin-change
0x0010	4	PORTB_PORT	Port B pin-change
0x0014	5	PORTC_PORT	Port C pin-change
0x0018	6	RTC_CNT	RTC counter overflow/compare
0x001C	7	RTC_PIT	RTC periodic interrupt timer
0x0020	8	TCA0_LUNF/OVF	TCA0 low-byte underflow (split) / overflow (normal)
0x0024	9	TCA0_HUNF	TCA0 high-byte underflow (split mode)
0x0028	10	TCA0_CMP0	TCA0 compare match 0
0x002C	11	TCA0_CMP1	TCA0 compare match 1
0x0030	12	TCA0_CMP2	TCA0 compare match 2
0x0034	13	TCB0_INT	TCB0 capture/compare (our tick ISR)
0x0038	14	TCB1_INT	TCB1 capture/compare
0x003C	15	TCD0_OVF	TCD0 overflow
0x0040	16	TCD0_TRIG	TCD0 trigger
0x0044	17	AC0_AC	Analog comparator 0
0x0048	18	AC1_AC	Analog comparator 1
0x004C	19	AC2_AC	Analog comparator 2
0x0050	20	ADC0_RESRDY	ADC0 result ready
0x0054	21	ADC0_WCOMP	ADC0 window comparator match
0x0058	22	ADC1_RESRDY	ADC1 result ready
0x005C	23	ADC1_WCOMP	ADC1 window comparator match
0x0060	24	TWI0_TWIS	TWI0 slave
0x0064	25	TWI0_TWIM	TWI0 master
0x0068	26	SPI0_INT	SPI0
0x006C	27	USART0_RXC	USART0 receive complete
0x0070	28	USART0_DRE	USART0 data register empty
0x0074	29	USART0_TXC	USART0 transmit complete
0x0078	30	NVMCTRL_EE	NVMCTRL EEPROM ready

In assembly, the vector table uses jmp for each slot:

```
.section .vectors, "ax", @progbits
    jmp reset_handler      /* 0x0000 RESET */
    jmp default_isr        /* 0x0004 CRCSCAN_NMI */
    jmp default_isr        /* 0x0008 BOD_VLM */
    jmp default_isr        /* 0x000C PORTA_PORT */
    jmp default_isr        /* 0x0010 PORTB_PORT */
    jmp default_isr        /* 0x0014 PORTC_PORT */
    jmp default_isr        /* 0x0018 RTC_CNT */
    jmp default_isr        /* 0x001C RTC_PIT */
    jmp default_isr        /* 0x0020 TCA0_LUNF/OVF */
    jmp default_isr        /* 0x0024 TCA0_HUNF */
```

```

jmp    default_isr    /* 0x0028 TCA0_CMP0 */
jmp    default_isr    /* 0x002C TCA0_CMP1 */
jmp    default_isr    /* 0x0030 TCA0_CMP2 */
jmp    tcb0_isr       /* 0x0034 TCB0_INT ← our tick */
jmp    default_isr    /* 0x0038 TCB1_INT */
jmp    default_isr    /* 0x003C TCD0_OVF */
jmp    default_isr    /* 0x0040 TCD0_TRIG */
jmp    default_isr    /* 0x0044 AC0_AC */
jmp    default_isr    /* 0x0048 AC1_AC */
jmp    default_isr    /* 0x004C AC2_AC */
jmp    default_isr    /* 0x0050 ADC0_RESRDY */
jmp    default_isr    /* 0x0054 ADC0_WCOMP */
jmp    default_isr    /* 0x0058 ADC1_RESRDY */
jmp    default_isr    /* 0x005C ADC1_WCOMP */
jmp    default_isr    /* 0x0060 TWI0_TWIS */
jmp    default_isr    /* 0x0064 TWI0_TWIM */
jmp    default_isr    /* 0x0068 SPI0_INT */
jmp    default_isr    /* 0x006C USART0_RXC */
jmp    default_isr    /* 0x0070 USART0_DRE */
jmp    default_isr    /* 0x0074 USART0_TXC */
jmp    default_isr    /* 0x0078 NVMCTRL_EE */

```

The explicit-table style is a little longer, but it keeps real code and data out of vector-table slots and makes interrupt ownership obvious when you inspect the binary later.

29.5 ISR Prologue and Epilogue

Every ISR on AVR must:

1. **Save SREG** — instructions inside the ISR affect flags; they must be restored before RETI.
2. **Save any registers it uses** — if the ISR uses R16–R29 or any other register that the interrupted code relies on.
3. **Acknowledge the interrupt source correctly** — for TCB0, write 1 to TCB0_INTFLAGS bit 0 (TCB_CAPT_bm). Other peripherals may clear their flags by different means, such as a data-register read.
4. **Restore saved registers and SREG.**
5. **RETI** — return from interrupt and let CPUINT leave the active interrupt state.

That save/restore discipline is the price you pay for asynchronous execution. If the ISR leaves flags or registers disturbed, the interrupted code resumes in an impossible state.

29.5.1 Minimal Tick ISR

The tick ISR only sets a 1-byte flag — tick_flag in SRAM:

```

/*
 * tcb0_isr – TCB0 periodic interrupt (1 ms tick)
 *
 * Sets tick_flag = 1.
 * Saves/restores: SREG, R16.
 * Latency from interrupt trigger to first useful instruction: see §18.6.
 */
tcb0_isr:
    push    r16          /* 1 cycle: save R16 to stack on AVRxt */
    in      r16, _SFR_IO_ADDR(SREG) /* 1 cycle: save SREG */

```

```

push  r16                /* 1 cycle */

/* clear interrupt flag (write 1 to TCB_CAPT_bm = bit 0) */
ldi   r16, TCB_CAPT_bm
sts   TCB0_INTFLAGS, r16 /* 2 cycles */

/* set tick flag */
ldi   r16, 1
sts   tick_flag, r16     /* 2 cycles */

pop   r16                /* 2 cycles: restore SREG value */
out   _SFR_IO_ADDR(SREG), r16 /* 1 cycle: restore SREG */
pop   r16                /* 2 cycles: restore R16 */
reti                      /* 4 cycles on ATtiny3217 */

```

Total ISR body: $1+1+1+1+2+1+2+2+1+2+4 = \mathbf{18 \text{ cycles}}$ from first instruction to end of RETI. (On AVRxt, PUSH is 1 cycle but POP is 2 cycles.)

On AVRxt devices such as the ATtiny3217, RETI costs 4 cycles with the 16-bit program counter. Microchip's instruction set manual notes that CPUINT tracks interrupt state on AVRxm/AVRxt devices; RETI returns through that interrupt-controller state rather than being just a plain RET. Keep it only at the true end of the handler, after every register and flag has been restored.

29.6 ISR Latency Analysis

ISR latency = cycles from interrupt trigger condition to the first instruction of the ISR executing.

29.6.1 Sources of Latency

On ATtiny3217 (avrxtmega3), the CPU-side interrupt response after the request has been recognized is:

Source	Cycles
Peripheral/request synchronization	0–2
Finish current instruction	1–4
Push PC to stack	2
Jump from vector to ISR (jmp)	3
Total (best case)	6
Total (worst case with 2 sync cycles and a 4-cycle instruction)	11

Microchip documents a minimum CPU interrupt response of 6 cycles on devices with more than 8 KB of flash: 1 cycle to finish a single-cycle instruction, 2 cycles to push the PC, and 3 cycles for the vector jmp. Peripheral-level synchronization can add a small device-specific delay before the CPU begins that sequence. In practice:

Interrupt asserted during instruction N
 → request is synchronized into the CPU domain (0–2 cycles)
 → instruction N completes (1–4 cycles total, depending on instruction length)
 → CPU pushes PC (2 cycles)
 → CPU executes JMP at vector (3 cycles)
 → first ISR instruction executes (cycle 6–11 from assertion)

This is the right way to think about interrupt latency on AVR: not as one magic number, but as a short chain of small, countable delays.

29.6.2 Push/Pop Overhead

The prologue `push r16 / in r16, SREG / push r16` costs **3 cycles** before any useful work. For a time-critical ISR (e.g., one that must sample a pin within N cycles), count:

```
latency_to_useful_work = ISR_entry_latency + prologue_cycles
                        = 6 (best) + 3 = 9 cycles minimum
                        = 11 (worst) + 3 = 14 cycles worst case
```

At 3.333 MHz, 14 cycles = **4.2 µs** worst-case latency to the first useful instruction.

29.6.3 Eliminating Sources of Latency

Technique	Cycles saved
Keep ISRs short — move work to main loop	Reduces re-entry latency
Use VPORT for output (1-cycle <code>sbi/cbi</code>)	0 — saves body cycles, not entry
Avoid long instructions (<code>LPM</code> , <code>CALL</code>) near critical ISR entry points	0–4
Reduce number of registers pushed in prologue	2 per saved/restored register
Reserve a known scratch register for a dedicated ISR and document ownership	2 per omitted push/pop pair

For the tick ISR, latency is unimportant — the tick fires every 1 ms and a few microseconds of jitter is negligible. Latency matters for ISRs that must capture hardware events (e.g., `TCB0` in input-capture mode, or a pin-change ISR measuring pulse width).

So always ask “latency relative to what?” before optimizing an ISR. A scheduler tick and an edge-timestamp ISR care about very different things.

29.6.4 Measuring Latency

Toggle a debug pin at the start and end of the ISR, then use an oscilloscope or logic analyser to measure ISR entry and execution width relative to another observable system event:

```
tcb0_isr:
    sbi    VPORTB_OUT, 3    /* debug pin PB3 high — mark ISR entry    */
    push   r16
    ...
    cbi    VPORTB_OUT, 3    /* debug pin PB3 low  — mark ISR exit    */
    reti
```

`SBI VPORTB_OUT, 3` executes in **1 cycle** and does not affect `SREG` — safe before saving `SREG`, as long as the debug pin is not used by the application.

29.7 Complete Two-Task Scheduler

29.7.1 Hardware Setup — Curiosity Nano

LED0 = PA3, active low (HIGH = off, LOW = on)
 USART0 TX = PB2 (to on-board debugger virtual COM port)
 TCB0 tick = every 1 ms (CCMP = 3332, CLK_PER, F_CPU = 3.333 MHz)

29.7.2 Register Allocation

R16 – scratch in ISR, scheduler, and peripheral setup
 R24:R25 – 16-bit task counters while updating SRAM state
 R1 – zero register convention for carry propagation

SRAM variables (.bss):

```
tick_flag    1 byte – set by ISR, cleared by main loop
led_cnt      2 bytes – 16-bit counter for LED task
uart_cnt     2 bytes – 16-bit counter for UART task
```

Using SRAM variables (accessed via LDS/STS) ensures the ISR and main loop share data correctly — SRAM is in the data address space accessible to both.

It also keeps the design honest: anything shared between asynchronous code paths should live in memory, not only in a caller's temporary register plan.

29.7.3 Source Code

```
/* scheduler.S – two-task cooperative scheduler, ATtiny3217 Curiosity Nano
 *
 * Tasks:
 *   task_led – toggle LED0 (PA3) every 500 ms
 *   task_uart – send 'H' over USART0 every 1000 ms
 *
 * Tick: TCB0 periodic interrupt, 1 ms
 * F_CPU: 3 333 333 Hz (20 MHz / 6, default Curiosity Nano clock)
 */

#include <avr/io.h>

#define F_CPU      333333UL
#define LED_PIN    3           /* PA3, Curiosity Nano LED0 */
#define LED_PERIOD 500         /* ticks (= 500 ms) */
#define UART_PERIOD 1000       /* ticks (= 1 s) */

/* USART0 baud: 9600 at 3.333 MHz
 * BAUD register = 64 * F_CPU / (16 * baud) = 64 * 333333 / (16 * 9600)
 *               = 21333312 / 153600 ≈ 1389 = 0x056D
 */
#define BAUD_9600 1389

/* — Vector table — */
.section .vectors, "ax", @progbits
jmp reset_handler      /* 0x0000 RESET */
jmp default_isr        /* 0x0004 CRCSCAN_NMI */
jmp default_isr        /* 0x0008 BOD_VLM */
jmp default_isr        /* 0x000C PORTA_PORT */
jmp default_isr        /* 0x0010 PORTB_PORT */
jmp default_isr        /* 0x0014 PORTC_PORT */
jmp default_isr        /* 0x0018 RTC_CNT */
jmp default_isr        /* 0x001C RTC_PIT */
jmp default_isr        /* 0x0020 TCA0_LUNF/OVF */
jmp default_isr        /* 0x0024 TCA0_HUNF */
jmp default_isr        /* 0x0028 TCA0_CMP0 */
jmp default_isr        /* 0x002C TCA0_CMP1 */
jmp default_isr        /* 0x0030 TCA0_CMP2 */
jmp tcb0_isr           /* 0x0034 TCB0_INT */
jmp default_isr        /* 0x0038 TCB1_INT */
```

```

    jmp default_isr      /* 0x003C TCD0_OVF */
    jmp default_isr      /* 0x0040 TCD0_TRIG */
    jmp default_isr      /* 0x0044 AC0_AC */
    jmp default_isr      /* 0x0048 AC1_AC */
    jmp default_isr      /* 0x004C AC2_AC */
    jmp default_isr      /* 0x0050 ADC0_RESRDY */
    jmp default_isr      /* 0x0054 ADC0_WCOMP */
    jmp default_isr      /* 0x0058 ADC1_RESRDY */
    jmp default_isr      /* 0x005C ADC1_WCOMP */
    jmp default_isr      /* 0x0060 TWI0_TWIS */
    jmp default_isr      /* 0x0064 TWI0_TWIM */
    jmp default_isr      /* 0x0068 SPI0_INT */
    jmp default_isr      /* 0x006C USART0_RXC */
    jmp default_isr      /* 0x0070 USART0_DRE */
    jmp default_isr      /* 0x0074 USART0_TXC */
    jmp default_isr      /* 0x0078 NVMCTRL_EE */

/* — SRAM variables ————— */
.section .bss
tick_flag: .byte 0      /* 1 = tick has fired; 0 = not yet */
led_cnt:   .word 0      /* LED task counter (16-bit) */
uart_cnt:  .word 0      /* UART task counter (16-bit)

/* — TCB0 ISR ————— */
.section .text
tcb0_isr:
    push r16
    in r16, _SFR_IO_ADDR(SREG)
    push r16

    ldi r16, TCB_CAPT_bm
    sts TCB0_INTFLAGS, r16 /* clear interrupt flag */

    ldi r16, 1
    sts tick_flag, r16     /* signal tick to main loop */

    pop r16
    out _SFR_IO_ADDR(SREG), r16
    pop r16
    reti

default_isr:
    reti                  /* ignore unexpected interrupts */

/* — Initialization ————— */
reset_handler:
    /* Stack pointer: top of SRAM = 0x3FFF */
    ldi r16, hi8(RAMEND)
    out _SFR_IO_ADDR(SPH), r16
    ldi r16, lo8(RAMEND)
    out _SFR_IO_ADDR(SPL), r16
    clr r1                /* project zero register for ADC-with-carry use */

    /* LED0: PA3 output, initial state HIGH (LED off – active low) */
    sbi VPORTA_DIR, LED_PIN
    sbi VPORTA_OUT, LED_PIN

```

```

/* USART0: set TX pin PB2 idle-high, then output */
sbi  VPORTB_OUT, 2
sbi  VPORTB_DIR, 2

rcall tcb0_init
rcall usart0_init

sei                                     /* enable global interrupts */

rjmp  main_loop

/* — TCB0 setup — */
tcb0_init:
    ldi  r16, lo8(3332)
    sts  TCB0_CCMP_L, r16
    ldi  r16, hi8(3332)
    sts  TCB0_CCMP_H, r16
    ldi  r16, 0
    sts  TCB0_CTRLB, r16      /* CNTMODE = 0: periodic interrupt */
    ldi  r16, TCB_CAPT_bm
    sts  TCB0_INTCTRL, r16   /* enable CAPT interrupt */
    ldi  r16, TCB_ENABLE_bm
    sts  TCB0_CTRLA, r16     /* CLK_PER, enable */
    ret

/* — USART0 setup — */
usart0_init:
    /* BAUD = 1389 = 0x056D */
    ldi  r16, lo8(BAUD_9600)
    sts  USART0_BAUD_L, r16
    ldi  r16, hi8(BAUD_9600)
    sts  USART0_BAUD_H, r16
    /* CTRLB: TX enable (TXEN bit 6) */
    ldi  r16, USART_TXEN_bm
    sts  USART0_CTRLB, r16
    /* CTRLC: async USART, 8N1 (default reset value = 0x03 = 8-bit, 1 stop) */
    /* No explicit write needed – reset value is correct */
    ret

/* — Main loop — */
main_loop:
    /* wait for tick */
.Lwait_tick:
    lds  r16, tick_flag
    tst  r16
    breq .Lwait_tick

    /* clear flag */
    clr  r16
    sts  tick_flag, r16

    /* run tasks */
    rcall task_led
    rcall task_uart

    rjmp main_loop

```

```

/* — task_led ————— */
task_led:
    /* increment 16-bit counter */
    lds    r24, led_cnt
    lds    r25, led_cnt+1
    adiw   r24, 1
    sts    led_cnt, r24
    sts    led_cnt+1, r25

    /* compare to LED_PERIOD (500) */
    cpi    r24, lo8(LED_PERIOD)
    ldi    r16, hi8(LED_PERIOD)
    cpc    r25, r16
    brlo   .Lled_done          /* counter < period: nothing to do */

    /* reset counter */
    clr    r24
    clr    r25
    sts    led_cnt, r24
    sts    led_cnt+1, r25

    /* toggle PA3 via PORTA_OUTTGL (1-byte write toggles the pin) */
    ldi    r16, (1 << LED_PIN)
    sts    PORTA_OUTTGL, r16

.Lled_done:
    ret

/* — task_uart ————— */
task_uart:
    /* increment 16-bit counter */
    lds    r24, uart_cnt
    lds    r25, uart_cnt+1
    adiw   r24, 1
    sts    uart_cnt, r24
    sts    uart_cnt+1, r25

    /* compare to UART_PERIOD (1000) */
    cpi    r24, lo8(UART_PERIOD)
    ldi    r16, hi8(UART_PERIOD)
    cpc    r25, r16
    brlo   .Luart_done

    /* reset counter */
    clr    r24
    clr    r25
    sts    uart_cnt, r24
    sts    uart_cnt+1, r25

    /* send 'H' (0x48): wait for DREIF (Data Register Empty Flag) */
.Luart_wait_dreif:
    lds    r16, USART0_STATUS
    sbrs   r16, USART_DREIF_bp    /* skip next if DREIF=1 (ready) */
    rjmp   .Luart_wait_dreif

    ldi    r16, 'H'
    sts    USART0_TXDATA1, r16

```

```
.Luart_done:
    ret
```

29.7.4 Build

```
MCU=attiny3217
avr-gcc -mmcu=${MCU} -nostartfiles -nodefaultlibs -o scheduler.elf src/scheduler.S
avr-objdump -h scheduler.elf
avr-objcopy -O ihex scheduler.elf scheduler.hex
avrdude -p t3217 -c serialupdi -P /dev/ttyUSB0 -U flash:w:scheduler.hex:i
```

29.8 Timing Analysis — Worst-Case Response

With two tasks per tick, the cooperative overhead before the main loop re-checks `tick_flag` is:

```
scheduler_overhead = wait_tick_poll (1 iteration) + clear_flag
                   = (3 + 1 + 1) + (1 + 2) = ~8 cycles
```

```
task_led_max = lds×2 + adiw + sts×4 + cpi + ldi + cpc + brlo + ... toggle path
              ≈ 2×3 + 2 + 2×2 + 1 + 1 + 1 + 1 + clr×2 + 2×2 + ldi + sts + ret
              ≈ 29 cycles (toggle path, excluding RCALL)
              ≈ 21 cycles (no-toggle path, excluding RCALL)
```

```
task_uart_max (no-send path) ≈ 21 cycles (same structure as task_led)
task_uart_send_fast_path ≈ 34 cycles (USART immediately ready, excluding RCALL)
```

```
total_cycles_per_tick (when USART is immediately ready) ≈ 8 + 2 + 29 + 2 + 34 + 2 = ~77 cycles max
time_at_3.333MHz = 77 / 3 333 333 ≈ 23 μs
```

The tick period is 1 ms = 3333 cycles. On the fast path, the scheduler consumes ≈ 77 of those cycles per tick — about 2.3% CPU utilisation.

Worst-case tick latency (time between ISR firing and main loop responding):

```
worst_case_tick_latency_fast_path ≈ task_led_max + task_uart_send_fast_path
                                   ≈ 29 + 34 + RCALL overheads = ~67 cycles ≈ 20 μs
```

This occurs when both tasks do their full work in the tick that just preceded the one being measured. The scheduler handles one tick at a time; if both tasks fire simultaneously on the same tick, `tick_flag` is set for the next tick while the current work finishes.

Important caveat: the scheduler.S UART task is still **blocking**. If the code reaches `.Luart_wait_dreif` just after the USART has become busy, the loop can stall for almost a full character time. At 9600 baud with 8N1, one frame is about $10/9600 \text{ s} \approx \mathbf{1.04 \text{ ms}}$, which is longer than the 1 ms tick. So the fast-path cycle count above is useful for instruction budgeting, but it is **not** a strict worst-case bound for the blocking UART design.

That is the most important design lesson in the chapter: a system is not truly bounded just because its arithmetic paths are short. One blocking peripheral wait can dominate the whole schedule.

29.9 Non-Blocking Task Extension

The fix is to decouple task execution from peripheral transmit timing. The non-blocking variant in `src/scheduler_nb.S` uses:

- a small SRAM ring buffer,
- a short `uart_enqueue` routine used by the task, and
- the `USART0_DRE` interrupt to feed bytes into `TXDATA1` whenever the UART is ready for another byte.

Conceptually:

```
task_uart:
    once per second:
        enqueue "Hbeat\r\n"
        enable DRE interrupt

USART0_dre_isr:
    compare tx_tail with tx_head
    if equal, disable DRE interrupt and reti
    load tx_buf[tx_tail]
    write byte to TXDATA1
    advance tx_tail
    reti
```

This keeps `task_uart` short and bounded: it copies bytes into the queue and returns. The USART hardware timing is handled asynchronously by the DRE ISR. In the example implementation, bytes are dropped if the ring buffer is full; that policy keeps the task non-blocking and deterministic.

This is a typical real-time trade: it is often better to lose optional output than to let one slow device destroy the timing guarantees of the whole system.

Full implementation in `src/scheduler_nb.S`.

Two details matter in that source file:

- The heartbeat string is placed in `.text`, not `.rodata`, so LPM reads from a flash byte address. It is aligned after the 7-byte string so the following AVR instructions still start on a word boundary.
- `uart_enqueue` saves `R18` because the caller uses it as the heartbeat string loop counter. A non-blocking helper is only deterministic if its register ownership is documented and obeyed.

29.10 Scaling to More Tasks

The round-robin scheduler scales by adding more `rcall task_N` instructions in the main loop. Each task must be **short** — its worst-case execution time must fit within the tick budget:

```
sum(task_worst_case_cycles) < tick_period_cycles
sum ≤ 3333 cycles (1 ms at 3.333 MHz)
```

For a task that exceeds this (e.g., a DFT computation), split it across multiple ticks using a state machine:

```
dft_task:
    /* process one input sample per tick – spread N-point DFT over N ticks */
    load dft_state
    branch to the matching state label
SAMPLE_0:
```

```

    compute partial sum for k=0
    store SAMPLE_1 into dft_state
    ret
SAMPLE_1:
    compute partial sum for k=1
    store SAMPLE_2 into dft_state
    ret

```

Breaking work into per-tick slices is one of the most useful habits in small embedded systems. It keeps long algorithms compatible with a simple scheduler.

29.10.1 When to Move to a Preemptive Kernel

A cooperative scheduler works until either: 1. Any task's worst-case time can exceed the tick period, or 2. Tasks have different priorities that require preemption.

For those cases, a preemptive kernel provides priority scheduling and proper stack switching. The assembly patterns in this chapter (ISR prologue/epilogue, timer setup, ISR-to-task signalling) are the same low-level building blocks.

So even if you eventually move to a preemptive kernel, the reasoning here still matters. The kernel is built out of the same timer and interrupt mechanics.

29.11 Summary

Topic	Key Point
Cooperative scheduling	Tasks run to completion; scheduler calls them in rotation
Tick source	TCB0 periodic interrupt, CCMP=3332 for 1 ms at 3.333 MHz
ISR entry latency	6-11 cycles (hardware) + 3 cycles (prologue)
SREG save	Mandatory in any ISR that uses arithmetic or branch instructions
RETI	Returns through CPUINT interrupt state; costs 4 cycles on ATtiny3217
VPORT	1-cycle SBI/CBI for fast output; does not affect SREG
Tick budget	3333 cycles/ms; cooperative core uses < 80 cycles on the fast path
Scaling	Add <code>rcall task_N</code> ; split long tasks into per-tick state machines

Microchip source notes:

- ATtiny3216/3217 data sheet, TCB Periodic Interrupt mode: TCB counts to the capture value, restarts from BOTTOM, and generates a CAPT interrupt/event.
- ATtiny3216/3217 data sheet, CPUINT Interrupt Response Time: devices with more than 8 KB Flash use a 3-cycle `jmp` vector, after 1 cycle to finish a single-cycle instruction and 2 cycles to store the PC.
- AVR Instruction Set Manual, RETI: AVRxt devices use CPUINT interrupt state on return; RETI is 4 cycles with a 16-bit program counter.
- Microchip USART guidance: asynchronous normal mode uses $BAUD = 64 * f_{CLK_PER} / (16 * f_{BAUD})$.

Next: Chapter 28 — DDS and Phase Accumulator Math

Chapter 30

DDS and Phase Accumulator Math

Direct Digital Synthesis (DDS), also called a Numerically Controlled Oscillator (NCO), is a method of generating a periodic waveform from a fixed-rate clock using nothing more than an integer counter, an addition, and a table lookup. On an 8-bit AVR, where there is no floating-point hardware and no built-in waveform generator, DDS is the standard way to produce audio tones, carrier signals, PWM audio, or continuously variable-frequency clocking signals with fine frequency resolution and deterministic timing.

This chapter covers:

1. **The phase accumulator** — the integer counter that lies at the heart of every DDS implementation.
2. **The DDS frequency equation** — how accumulator width, sample rate, and the tuning word combine to determine output frequency.
3. **Choosing accumulator width** — the trade between resolution, register pressure, and ISR cost.
4. **Sample rate selection on ATtiny3217** — setting up TCB0 for a precise interrupt-driven sample clock.
5. **Waveform tables** — storing a sine cycle in flash and addressing it from the accumulator.
6. **AVR assembly implementation** — register plan, SRAM layout, ISR, and output to an 8-bit PWM channel.
7. **Frequency accuracy and phase truncation** — measuring output error, understanding spurious tones, and estimating SFDR.
8. **Phase and frequency modulation** — changing frequency mid-stream, sweeping through a chirp, and adding a phase offset.
9. **Two-tone mixing** — summing two independent DDS channels in the ISR.
10. **Common pitfalls** — Nyquist aliasing, overflow in tuning word computation, and interrupt jitter.

30.1 The Phase Accumulator

The phase accumulator is an N-bit unsigned integer held in SRAM. Each time the sample clock fires, a fixed value M is added to the accumulator:

```
accumulator = (accumulator + M) mod 2N
```

The modular reduction is free: when the accumulator overflows it wraps back to zero automatically, exactly like an N-bit unsigned integer on any CPU. There is no explicit division or modulo instruction needed.

30.1.1 Mapping Accumulator Value to Phase

Think of the accumulator as representing a phase angle that runs from 0 to 2π continuously:

$$\text{phase} = \text{accumulator} \times (2\pi / 2^N)$$

When the accumulator holds 0, the phase is 0° . When it holds $2^{(N-1)}$, the phase is 180° . When it overflows from $2^N - 1$ back to 0, it completes one full cycle.

This is identical to Binary Angular Measure (BAM), which Chapter 29 uses for the CORDIC angle format. A 16-bit phase accumulator is a 16-bit BAM counter.

30.1.2 Visualising the Accumulator

Three snapshots of a 4-bit accumulator ($N=4$, range 0–15) stepping by $M=3$:

Sample	Accumulator	Fractional phase ($\times 360^\circ/16$)
0	0	0.0°
1	3	67.5°
2	6	135.0°
3	9	202.5°
4	12	270.0°
5	15 $\rightarrow$ 0 (wrap)	$337.5^\circ \rightarrow 0^\circ$ (next cycle begins)
6	3	67.5°
...		

The accumulator completes one full revolution every $2^N / M$ samples. If the sample clock runs at f_s , the output repeats at $f_s \times M / 2^N$ Hz.

30.2 The DDS Frequency Equation

The fundamental relationship is:

$$f_{\text{out}} = f_s \times M / 2^N$$

Where:

f_{out}	output frequency (Hz)
f_s	sample rate (Hz) – how often the accumulator is updated
M	tuning word (also called phase increment) – the value added each sample
N	accumulator width in bits
2^N	accumulator modulus

Rearranging to find the tuning word M for a desired output frequency:

$$M = \text{round}(f_{\text{out}} \times 2^N / f_s)$$

The rounding means the output frequency may not be exactly f_{out} . The nearest achievable frequency is constrained by the resolution:

$$\Delta f = f_s / 2^N \quad (\text{Hz per tuning word step})$$

With a narrower accumulator the resolution is coarser; with a wider one it is finer. That trade is the central design decision in any DDS implementation.

30.2.1 Nyquist Limit

A real periodic signal reconstructed from discrete samples cannot exceed half the sample rate. The maximum usable output frequency is:

$$f_{\text{max}} = f_s / 2 \quad (\text{Nyquist limit})$$

The corresponding maximum tuning word is $M = 2^{(N-1)}$. Above this, the output aliases back to a lower frequency rather than increasing further.

30.3 Choosing the Accumulator Width

30.3.1 8-bit Accumulator

An 8-bit accumulator maps directly to a 256-entry table with no bit manipulation: the accumulator byte is the table index. It costs one register and one SRAM byte. The resolution is very coarse:

$$\Delta f = f_s / 256$$

At $f_s = 8$ kHz this is 31.25 Hz per step — useless for musical tones.

30.3.2 16-bit Accumulator

Two SRAM bytes, two registers, one ADD and one ADC per update. This is the standard choice when the output is audio or a control signal measured in Hz:

$$N = 16, 2^N = 65536$$

$$\Delta f = f_s / 65536$$

At $f_s \approx 8$ kHz: $\Delta f = 7992 / 65536 = 0.122$ Hz per step

The top 8 bits of the 16-bit accumulator are used as the table index, so the table still has 256 entries. The bottom 8 bits are sub-index precision that gives fine frequency control without enlarging the table. A 16-bit accumulator is the practical choice for this chapter.

30.3.3 32-bit Accumulator

Four SRAM bytes, four registers, four ADD/ADC instructions per update. The resolution becomes exceptional:

$$N = 32, 2^N = 4,294,967,296$$

$$\Delta f = f_s / 4,294,967,296$$

At $f_s \approx 8$ kHz: $\Delta f \approx 1.86$ μ Hz per step

A 32-bit accumulator is warranted when long stable tones must hold pitch within a few millihertz, or when generating a very slow ramp signal where coarse steps would cause audible pitch jumps. The ISR cost is roughly doubled compared to 16-bit. Section “32-bit Variant” below shows the register plan.

30.4 Sample Rate on ATtiny3217

The ATtiny3217 on the Curiosity Nano runs at 3,333,333 Hz (the internal 20 MHz RC oscillator divided by 6, set by CLKCTRL_MCLKCTRLB). TCB0 in Periodic Interrupt mode provides the sample clock.

30.4.1 TCB0 Sample Clock Calculation

Target sample rate: 8000 Hz (standard audio)

$$\begin{aligned} \text{CCMP} &= \text{round}(F_{\text{CPU}} / f_s) - 1 \\ &= \text{round}(3,333,333 / 8000) - 1 \end{aligned}$$

$$\begin{aligned}
 &= \text{round}(416.67) - 1 \\
 &= 417 - 1 \\
 &= 416 \quad (0x01A0)
 \end{aligned}$$

$$\begin{aligned}
 \text{Actual sample rate: } F_{\text{CPU}} / (\text{CCMP} + 1) \\
 &= 3,333,333 / 417 \\
 &= 7992 \text{ Hz}
 \end{aligned}$$

$$\text{Sample rate error: } (7992 - 8000) / 8000 \times 100\% = -0.10\%$$

A 0.1% sample rate error shifts every output frequency by 0.1%. For A4 at 440 Hz that is a 0.44 Hz error — inaudible for most purposes, and negligible compared to the quantisation error of the 16-bit tuning word.

30.4.2 TCB0 Register Map (from ATtiny3217 data sheet)

Address	Name	Value for 8 kHz
0x0A40	TCB0_CTRLA	0x01 (ENABLE=1, CLKSEL=CLK_PER)
0x0A41	TCB0_CTRLB	0x00 (CNTMODE=0, Periodic Interrupt)
0x0A45	TCB0_INTCTRL	0x01 (CAPT interrupt enable)
0x0A4C	TCB0_CCMPH	0xA0 (low byte of 416 = 0x01A0)
0x0A4D	TCB0_CCMPH	0x01 (high byte of 416 = 0x01A0)

In Periodic Interrupt mode, TCB0 counts from 0 to CCMP inclusive, then restarts and fires the CAPT interrupt. The period is CCMP+1 = 417 clock cycles.

30.5 Tuning Word Computation

Given N=16 and $f_s = 7992$ Hz:

$$M = \text{round}(f_{\text{out}} \times 65536 / 7992)$$

30.5.1 Common Musical Notes

Note	Frequency (Hz)	Tuning word M	Actual f_{out} (Hz)	Error (Hz)
C4	261.626	2145	261.58	-0.05
D4	293.665	2408	293.65	-0.01
E4	329.628	2703	329.63	-0.00
A4	440.000	3608	439.99	-0.01
C5	523.251	4291	523.28	+0.03

Calculation for A4:

$$\begin{aligned}
 M &= \text{round}(440 \times 65536 / 7992) \\
 &= \text{round}(28,835,840 / 7992) \\
 &= \text{round}(3608.09) \\
 &= 3608
 \end{aligned}$$

$$\begin{aligned}
 f_{\text{actual}} &= 7992 \times 3608 / 65536 \\
 &= 28,835,136 / 65536 \\
 &= 439.99 \text{ Hz}
 \end{aligned}$$

$$\text{Error} = 439.99 - 440 = -0.01 \text{ Hz} \quad (-0.002\%)$$

That error is far below the just-noticeable pitch difference for a typical listener (roughly 0.3% at 440 Hz), so 16-bit DDS with a 7992 Hz sample rate is adequate for audio synthesis at this scale.

30.5.2 Overflow Risk in Tuning Word Calculation

The intermediate product $f_{\text{out}} \times 65536$ can overflow a 16-bit integer for any frequency above 0.99 Hz. Always compute M in at least 32-bit arithmetic:

```
uint32_t M = (uint32_t)f_out_hz_times_1000 * 65536UL / 7992 / 1000;
```

Or, if computing offline and loading M as a constant, use Python:

```
f_s = 7992                # actual sample rate
N = 16
M = round(440 * (2**N) / f_s) # M = 3608
```

On AVR, the tuning word is set by the application before the DDS ISR starts and typically never recomputed in assembly. Frequency changes happen by writing a new precomputed M value to `phase_step` in SRAM.

30.6 The Waveform Table

30.6.1 Sine Table Format

The waveform table stores one complete cycle of the target waveform. For sine output to an 8-bit PWM channel, unsigned bytes in the range 0–255 are most convenient:

```
table[i] = round(127.5 + 127.5 * sin(2π × i / 256))
```

Selected entries:

Index	Angle	Value	Notes
0	0°	128	zero crossing, rising
32	45°	218	
64	90°	255	positive peak
96	135°	218	
128	180°	128	zero crossing, falling
160	225°	37	
192	270°	0	negative peak
224	315°	37	
255	358.59°	124	approaching next cycle

The value 128 corresponds to mid-scale PWM (0 V after low-pass filtering). The negative peak reaches 0, not 1, because $127.5 \times (1 - 1) = 0$ exactly.

30.6.2 Generating the Table

Use Python to compute the 256-entry table and paste it into the assembly source:

```
import math
table = [round(127.5 + 127.5 * math.sin(2 * math.pi * i / 256))
         for i in range(256)]
print(', '.join(str(v) for v in table))
```

Spot-checking the first ten entries:

```
i=0: 128   i=1: 131   i=2: 134   i=3: 137   i=4: 140
i=5: 143   i=6: 146   i=7: 149   i=8: 152   i=9: 155
```

Each step of 1 in the index advances the angle by $360^\circ/256 = 1.406^\circ$. The value increments approximately $127.5 \times 2\pi/256 \approx 3.13$ per step near the zero crossings and is flat near the peaks, exactly as expected for a sine wave.

30.6.3 Table Placement and Alignment

Aligning the 256-byte table to a 256-byte flash boundary allows the ISR to compute the table address with a single MOV rather than an addition:

```
.section .text
.balign 256          /* align to 256-byte address boundary */
sine_table:
    .byte 128, 131, 134, 137, 140, 143, 146, 149
    .byte 152, 155, 158, 162, 165, 167, 170, 173
    ...
    /* 256 entries total */
```

With this alignment, `lo8(sine_table)` is always 0x00. Inside the ISR:

```
ldi    ZH, hi8(sine_table) /* page address – fixed for the whole table */
mov     ZL, r_index        /* ZL = table index (top byte of accumulator) */
lpm     r_sample, Z        /* fetch the sample */
```

ZL will range over 0x00–0xFF, all within the aligned page, so ZH never changes during the fetch. If alignment is not possible (flash too fragmented), use the two-step add:

```
ldi    ZL, lo8(sine_table)
ldi    ZH, hi8(sine_table)
add     ZL, r_index
adc     ZH, r1              /* r1 = 0; propagate carry into ZH */
lpm     r_sample, Z
```

30.7 AVR Assembly Implementation

30.7.1 SRAM Variable Layout

```
.section .bss

/* 16-bit phase accumulator: two sequential bytes */
phase_accum: .byte 0 /* low byte */
              .byte 0 /* high byte – addressed as phase_accum+1 */

/* 16-bit phase step (tuning word M): two sequential bytes */
phase_step:  .byte 0 /* low byte */
              .byte 0 /* high byte – addressed as phase_step+1 */
```

In GAS assembly, `.byte 0` allocates one uninitialised byte in `.bss` (which the C runtime zeros before main, but in `-nostartfiles` builds you must zero it yourself in `reset_handler`). The two-byte layout lets `LDS r, phase_accum` and `LDS r, phase_accum+1` read the low and high bytes in sequential instructions. `phase_accum+1` is a valid GAS expression: the label's address plus one.

30.7.2 Register Plan

The DDS ISR needs:

```
r16    scratch: interrupt flag clear, ISR prologue/epilogue use
```

r17 phase accumulator high byte (table index after update)
 r18 phase accumulator low byte
 r19 phase step high byte
 r20 phase step low byte
 r21 sine sample fetched with LPM
 r30/r31 Z: flash pointer for LPM
 SREG saved/restored in prologue/epilogue

The ISR saves r16–r21 and Z. Since TCB0 fires every 417 cycles, the ISR must finish well within that budget.

30.7.3 Cycle Budget

Prologue (save SREG, r16–r21, r30–r31)	9 push + 1 in = 10 cycles
Clear TCB0 interrupt flag (LDI + STS)	1 + 2 = 3 cycles
Load accumulator (2 × LDS)	2 × 3 = 6 cycles
Load phase step (2 × LDS)	2 × 3 = 6 cycles
Add step to accumulator (ADD + ADC)	1+1 = 2 cycles
Store accumulator (2 × STS)	2 × 2 = 4 cycles
Set up Z and fetch sample (LDI + MOV + LPM)	1+1+3 = 5 cycles
Write sample to PWM (STS)	= 2 cycles
Epilogue (restore r30–r31, r16–r21, SREG + RETI)	9×2+1+4 = 23 cycles
Total	61 cycles
Budget (417 cycles per sample)	356 cycles remaining
CPU utilisation	61/417 = 14.6%

On AVRxt, PUSH is 1 cycle but POP is 2 cycles, so the 9-register restore in the epilogue costs 18 cycles. At 14.6%, the DDS ISR still leaves ample time for a cooperative-scheduler main loop to handle other tasks between samples.

30.7.4 TCB0 Initialisation

```

/* tcb0_dds_init – configure TCB0 for the DDS sample clock
 *
 * F_CPU = 3,333,333 Hz, target f_s = 8000 Hz, CCMP = 416 = 0x01A0
 * Actual f_s = 3,333,333 / 417 = 7992 Hz
 * Clobbers: R16
 */
tcb0_dds_init:
    ldi    r16, lo8(416)           /* 0xA0 */
    sts    TCB0_CCMPH, r16
    ldi    r16, hi8(416)          /* 0x01 */
    sts    TCB0_CCMPH, r16

    ldi    r16, 0                 /* CNTMODE = 0: Periodic Interrupt */
    sts    TCB0_CTRLB, r16

    ldi    r16, TCB_CAPT_bm        /* enable CAPT interrupt */
    sts    TCB0_INTCTRL, r16

    ldi    r16, TCB_ENABLE_bm      /* CLK_PER (no prescaler), enable */
    sts    TCB0_CTRLA, r16
    ret
  
```

TCB writes the CCMP register as two separate byte writes. The 16-bit value is buffered: the hardware copies the low byte to an internal buffer and latches both bytes atomically when the high byte is written. Always write CCMPH before CCMPH.

30.7.5 TCA0 8-bit PWM Initialisation

TCA0 in split mode provides two independent 8-bit PWM channels. The low counter drives WO0–WO2; the high counter drives WO3–WO5. Using the low counter on WO0 (PB0):

```
/* tca0_pwm_init – configure TCA0 low counter for 8-bit PWM on WO0 (PB0)
 *
 * PWM frequency = F_CPU / 256 = 3,333,333 / 256 = 13,021 Hz
 * Clobbers: R16
 */
tca0_pwm_init:
    /* Enable split mode */
    ldi    r16, TCA_SPLIT_SPLITM_bm
    sts    TCA0_SPLIT_CTRLD, r16

    /* Low counter: CLK_PER (no prescaler), enable.
     * TCA_SPLIT_CLKSEL_DIV1_gc = 0x00, so CTRLA = ENABLE_bm only. */
    ldi    r16, TCA_SPLIT_ENABLE_bm
    sts    TCA0_SPLIT_CTRLA, r16

    /* Enable WO0 output compare on low counter */
    ldi    r16, TCA_SPLIT_LCMP0EN_bm
    sts    TCA0_SPLIT_CTRLB, r16

    /* Set low counter period to 255 (8-bit PWM) */
    ldi    r16, 255
    sts    TCA0_SPLIT_LPER, r16

    /* Initial duty cycle = 128 (mid-scale) */
    ldi    r16, 128
    sts    TCA0_SPLIT_LCMP0, r16

    /* PB0 as output */
    sbi    VPORTB_DIR, 0
    ret
```

The PWM frequency is 13,021 Hz — about 1.6× the 7992 Hz sample rate, and roughly 3.3× the 3996 Hz Nyquist limit. Signals above the Nyquist limit alias; the PWM carrier itself aliases to audible frequencies unless filtered. Place an RC low-pass filter on PB0 before any audio output stage:

Cut-off frequency: $f_c = 1 / (2\pi \times R \times C)$

For $R = 10\text{ k}\Omega$ and $C = 10\text{ nF}$:

$$f_c = 1 / (2\pi \times 10,000 \times 10 \times 10^{-9}) = 1592\text{ Hz}$$

This passes audio up to 1.6 kHz and suppresses the 13 kHz PWM carrier.

For wider bandwidth (up to 3 kHz), use $R = 5.6\text{ k}\Omega$, $C = 10\text{ nF} \rightarrow f_c = 2.84\text{ kHz}$.

30.7.6 DDS Interrupt Service Routine

```
/* dds_isr – DDS sample clock ISR (TCB0 CAPT, fires at 7992 Hz)
 *
 * Each call:
 *   1. Adds phase_step to phase_accum (16-bit wrap)
 *   2. Uses high byte of accumulator as index into sine_table
 *   3. Writes the sine sample to TCA0_SPLIT_LCMP0 (8-bit PWM duty)
 *
 * Saves/restores: SREG, R16–R21, R30–R31 (Z)
```

```

* Cycle cost: ~61 cycles of the 417-cycle budget
*/
dds_isr:
/* — Prologue — */
push r16
in r16, _SFR_IO_ADDR(SREG)
push r16
push r17
push r18
push r19
push r20
push r21
push r30
push r31

/* — Clear TCB0 interrupt flag — */
ldi r16, TCB_CAPT_bm
sts TCB0_INTFLAGS, r16

/* — Load phase accumulator — */
lds r18, phase_accum /* low byte */
lds r17, phase_accum+1 /* high byte */

/* — Load phase step — */
lds r20, phase_step /* low byte */
lds r19, phase_step+1 /* high byte */

/* — Advance accumulator: accum += step (16-bit wrapping add) — */
add r18, r20 /* low: r18 = r18 + r20 */
adc r17, r19 /* high: r17 = r17 + r19 + C */

/* — Store updated accumulator — */
sts phase_accum, r18
sts phase_accum+1, r17

/* — Table lookup: sine_table[r17] — */
ldi ZH, hi8(sine_table) /* flash page address of table */
mov ZL, r17 /* index = high byte of accumulator */
lpm r21, Z /* r21 = sine_table[index] */

/* — Output sample to PWM duty register — */
sts TCA0_SPLIT_LCMP0, r21

/* — Epilogue — */
pop r31
pop r30
pop r21
pop r20
pop r19
pop r18
pop r17
pop r16
out _SFR_IO_ADDR(SREG), r16
pop r16
reti

```

The ordering of the low/high bytes follows the ATtiny3217 little-endian SRAM layout: low byte at the lower address, high byte at the higher address. Loading `phase_accum` gives the

low byte and `phase_accum+1` gives the high byte. The ADD/ADC pair updates the pair in one step: ADD updates the low byte and sets carry; ADC propagates carry into the high byte. Overflow in the high byte is silently discarded — that is the intended 16-bit wrap.

30.7.7 Reset Handler and Startup

```
/* reset_handler – entry point after power-on or reset */
reset_handler:
    /* Stack pointer */
    ldi    r16, hi8(RAMEND)
    out    _SFR_IO_ADDR(SPH), r16
    ldi    r16, lo8(RAMEND)
    out    _SFR_IO_ADDR(SPL), r16

    clr    r1                                /* zero register convention */

    /* Zero SRAM variables */
    clr    r16
    sts    phase_accum,    r16
    sts    phase_accum+1, r16
    sts    phase_step,    r16
    sts    phase_step+1,  r16

    /* Initialise peripherals */
    rcall  tca0_pwm_init
    rcall  tcb0_dds_init

    /* Load tuning word for A4 (440 Hz at 7992 Hz sample rate: M = 3608) */
    ldi    r16, lo8(3608)                    /* 0x18 */
    sts    phase_step, r16
    ldi    r16, hi8(3608)                    /* 0x0E */
    sts    phase_step+1, r16

    sei                                /* enable global interrupts */

.Lmain_idle:
    rjmp   .Lmain_idle                    /* main loop: all work done in ISR */
```

Writing the phase step after enabling the peripherals but before SEI avoids a race: the ISR cannot fire until after SEI, so the write is always visible before the first sample.

30.8 Complete Source Listing

The full buildable file is in `src/dds.S`. It includes the vector table, sine table, all initialisation routines, the DDS ISR, and the reset handler.

30.8.1 Vector Table

```
.section .vectors, "ax", @progbits
    jmp    reset_handler    /* 0x0000 RESET */
    jmp    default_isr      /* 0x0004 CRCSCAN_NMI */
    jmp    default_isr      /* 0x0008 BOD_VLM */
    jmp    default_isr      /* 0x000C PORTA_PORT */
    jmp    default_isr      /* 0x0010 PORTB_PORT */
```

```

jmp    default_isr    /* 0x0014 PORTC_PORT */
jmp    default_isr    /* 0x0018 RTC_CNT */
jmp    default_isr    /* 0x001C RTC_PIT */
jmp    default_isr    /* 0x0020 TCA0_LUNF/OVF */
jmp    default_isr    /* 0x0024 TCA0_HUNF */
jmp    default_isr    /* 0x0028 TCA0_CMP0 */
jmp    default_isr    /* 0x002C TCA0_CMP1 */
jmp    default_isr    /* 0x0030 TCA0_CMP2 */
jmp    dds_isr         /* 0x0034 TCB0_INT ← DDS sample clock */
jmp    default_isr    /* 0x0038 TCB1_INT */
jmp    default_isr    /* 0x003C TCD0_OVF */
jmp    default_isr    /* 0x0040 TCD0_TRIG */
jmp    default_isr    /* 0x0044 AC0_AC */
jmp    default_isr    /* 0x0048 AC1_AC */
jmp    default_isr    /* 0x004C AC2_AC */
jmp    default_isr    /* 0x0050 ADC0_RESRDY */
jmp    default_isr    /* 0x0054 ADC0_WCOMP */
jmp    default_isr    /* 0x0058 ADC1_RESRDY */
jmp    default_isr    /* 0x005C ADC1_WCOMP */
jmp    default_isr    /* 0x0060 TWI0_TWIS */
jmp    default_isr    /* 0x0064 TWI0_TWIM */
jmp    default_isr    /* 0x0068 SPI0_INT */
jmp    default_isr    /* 0x006C USART0_RXC */
jmp    default_isr    /* 0x0070 USART0_DRE */
jmp    default_isr    /* 0x0074 USART0_TXC */
jmp    default_isr    /* 0x0078 NVMCTRL_EE */

```

30.8.2 Build

```

MCU=attiny3217
avr-gcc -mmcu=${MCU} -nostartfiles -nodefaultlibs \
    -Wl,-Map=dds.map \
    -o dds.elf src/dds.S
avr-objdump -h dds.elf          # check section sizes and load addresses
avr-objdump -d dds.elf          # disassemble to verify ISR body
avr-objcopy -O ihex dds.elf dds.hex
avrdude -p t3217 -c serialupdi -P /dev/ttyUSB0 -U flash:w:dds.hex:i

```

The `-Wl, -Map=dds.map` flag writes a linker map. Inspect it to confirm that `sine_table` is placed at a 256-byte-aligned flash address (the `.balign 256` directive will force this, and the map file shows the actual result).

30.9 Frequency Accuracy and Phase Truncation

30.9.1 Two Sources of Frequency Error

A 16-bit DDS implementation on ATtiny3217 has two independent sources of frequency error:

1. Sample rate quantisation error.

The sample clock period must be an integer number of CPU cycles. At $F_{\text{CPU}} = 3,333,333$ Hz and target $f_s = 8000$ Hz, the ideal period is 416.667 cycles, which rounds to 417. The resulting sample rate is:

$$f_{s_actual} = 3,333,333 / 417 = 7992.4 \text{ Hz}$$

Every output frequency is scaled by this 0.10% error. For A4 at 440 Hz, that is a 0.44 Hz shift.

2. Tuning word quantisation error.

The tuning word M must be an integer. Rounding introduces an error of at most ± 0.5 steps, corresponding to $\pm \Delta f/2 = \pm 0.061$ Hz at 7992 Hz and $N=16$. This error is independent for each output frequency.

The combined worst-case frequency error is approximately:

$$|\text{error_max}| \approx f_{\text{out}} \times 0.001 + 0.061 \text{ Hz}$$

For A4: $\approx 0.44 + 0.06 = 0.50$ Hz. That is 0.11% — well within typical loudspeaker and hearing tolerance.

30.9.2 Phase Truncation and Spurious Tones

The phase accumulator has 16 bits of precision, but the waveform table has only 256 entries. Only the top 8 bits of the accumulator are used as the table index. The bottom 8 bits are discarded — this is called **phase truncation**.

Phase truncation creates a periodic phase-step error. That error manifests in the output spectrum as spurious tones, sometimes called spurs or phase truncation spurs (PTS). The amplitude of the worst-case spur is approximately:

$$\text{Spur level} \approx -6.02 \times (N - B) \quad \text{dBc}$$

Where:

$$\begin{aligned} N &= \text{accumulator bits} = 16 \\ B &= \text{table address bits} = 8 \\ N - B &= \text{truncated bits} = 8 \end{aligned}$$

$$\text{Spur level} \approx -6.02 \times 8 = -48 \text{ dBc}$$

“dBc” means decibels below the carrier (the wanted output tone). A -48 dBc spur is about 250 times smaller in amplitude than the carrier. For most audio and control applications this is inaudible and negligible, but it matters for communications use where spectral purity is required.

To reduce spurs without enlarging the table, use a wider accumulator (increase N) or interpolate between adjacent table entries (which is more expensive in the ISR but suppresses spurs dramatically).

30.9.3 Worst-Case Spur Frequency

Phase truncation spurs appear at frequencies related to the tuning word and the accumulator overflow rate. For a tuning word M and sample rate f_s :

$$\text{Spur frequencies} \approx k \times (f_s \times M / \text{gcd}(M, 2^N)) \quad \text{for integer } k$$

where $\text{gcd}()$ is the greatest common divisor of M and 2^N .

When M is odd, $\text{gcd}(M, 2^{16}) = 1$, and the only spur within the Nyquist band is the fundamental itself. When M shares factors of 2 with 2^{16} , multiple spurs appear at harmonics of a sub-fundamental. Choosing M to be odd or to share minimal common factors with 2^N reduces the spur density, though not their individual amplitude.

30.10 Phase and Frequency Modulation

30.10.1 Phase Modulation

A phase offset can be added to the accumulator at the table lookup step without changing the accumulator itself:

```
/* PM: add 16-bit phase offset from SRAM before the table lookup */
lds  r22, phase_offset          /* low byte of offset */
lds  r23, phase_offset+1        /* high byte of offset */
add  r18, r22                   /* r18 = accum_lo + offset_lo */
adc  r17, r23                   /* r17 = accum_hi + offset_hi + C */
/* now use r17 as the table index – the accumulator itself is unchanged */
```

The accumulated phase and the modulation offset are kept separate. This means the carrier oscillates at the frequency set by `phase_step`, and the modulation shifts the output phase each sample without altering the carrier rate.

A constant `phase_offset` shifts the output phase permanently (useful for generating a cosine from the same sine table: set offset = 0x4000 = 90°).

A time-varying `phase_offset` produces phase modulation (PM) or, with careful choice of offset, frequency modulation (FM):

FM: update `phase_offset` each sample by a small audio signal value
 → the carrier frequency swings above and below `f_out` at the audio rate

30.10.2 Frequency Modulation

The simplest frequency modulation writes a new `phase_step` value each sample. For a 1 Hz FM deviation, add $\pm \text{round}(1 \text{ Hz} \times 65536 / 7992) = \pm 8$ to `phase_step`.

For a small deviation, update `phase_step` from the ISR itself:

```
/* FM via time-varying phase step – add fm_delta each sample */
lds  r20, fm_delta              /* signed FM offset (low byte) */
lds  r21, fm_delta+1           /* signed FM offset (high byte) */

lds  r18, phase_step
lds  r19, phase_step+1
add  r18, r20
adc  r19, r21
sts  phase_step, r18
sts  phase_step+1, r19
```

`fm_delta` is written by the main loop based on an audio or control input. This is the pattern used by many simple FM synthesis chips on 8-bit systems.

30.10.3 Linear Frequency Sweep (Chirp)

A chirp increases or decreases the output frequency linearly over time. The phase step itself increments by a constant `chirp_rate` each sample:

```
/* Chirp: phase_step += chirp_rate each sample */
lds  r22, chirp_rate
lds  r23, chirp_rate+1
add  r18, r22
adc  r19, r23
sts  phase_step, r18
sts  phase_step+1, r19
```

If `chirp_rate` is positive, the output frequency rises; if negative (unsigned wrap), it falls. The rate of frequency change is:

$$\begin{aligned} df/dt &= \text{chirp_rate} \times \Delta f \times f_s \\ &= \text{chirp_rate} \times (7992 / 65536) \times 7992 \\ &\approx \text{chirp_rate} \times 975 \text{ Hz/s per tuning-word unit} \end{aligned}$$

For a chirp rate of 1 (one tuning-word step per sample), the frequency rises at 975 Hz/s. To cover the range from 200 Hz to 1000 Hz in 2 seconds:

$$\begin{aligned} \text{total frequency change} &= 1000 - 200 = 800 \text{ Hz} \\ \text{duration} &= 2 \text{ s} \\ \text{chirp_rate} &= \text{round}(800 / (2 \times 975)) = \text{round}(0.41) \rightarrow 0 - \text{too slow at 1 step} \end{aligned}$$

Use `chirp_rate = 1` and a slower duration:
 $\text{time} = 800 \text{ Hz} / (1 \times 975 \text{ Hz/s}) = 0.82 \text{ s}$

Or `chirp_rate = 4` and set starting M:
 $\text{time} = 800 / (4 \times 975) = 0.205 \text{ s}$

The chirp rate is a tuning parameter set before enabling the sweep. The main loop monitors a tick counter to stop the sweep at the target frequency.

30.11 Two-Tone Mixing

A second DDS tone requires a second phase accumulator and a second phase step in SRAM. At the table lookup, both samples are read and summed:

```
/* Second accumulator */
phase_accum2: .byte 0, 0
phase_step2:  .byte 0, 0
```

ISR extension after computing sample 1 in r21:

```
/* — Second oscillator ————— */
lds  r22, phase_accum2
lds  r23, phase_accum2+1
lds  r24, phase_step2
lds  r25, phase_step2+1
add  r22, r24
adc  r23, r25
sts  phase_accum2, r22
sts  phase_accum2+1, r23

ldi  ZH, hi8(sine_table)
mov  ZL, r23
lpm  r16, Z                /* r16 = sine sample for oscillator 2 */

/* — Mix: (sample1 + sample2) / 2 ————— */
/* Both r21 and r16 are unsigned 0-255 */
/* Sum can reach 510, so use 16-bit add then divide */
add  r21, r16              /* r21 = sample1 + sample2 (low byte) */
clr  r16
adc  r16, r1               /* r16 = carry from 8-bit overflow */
lsr  r16
ror  r21                  /* r21:r16 >> 1 → r21 = (sum) / 2 */
```

The result in r21 is the average of the two samples, range 0-255, suitable for the 8-bit PWM. Equal-power mixing of two full-scale tones prevents overflow.

For unequal amplitudes, scale one or both samples before summing. The same pattern extends to three or four oscillators within the 417-cycle ISR budget, though the total cycle count must be verified to stay within bounds.

The two-tone ISR costs approximately $61 + 28 = 89$ cycles (adding the second oscillator update and mix, plus the extra register saves it requires), leaving 328 cycles per sample. That is still about 21% CPU utilisation — manageable.

30.12 32-bit Accumulator Variant

When sub-Hz frequency resolution is needed — for example, generating a very slow periodic waveform for a mechanical actuator, or for tight frequency locking in a phase-locked loop — extend the accumulator to 32 bits.

30.12.1 SRAM Layout

```
phase_accum32: .byte 0, 0, 0, 0    /* 4 bytes, low to high */
phase_step32:  .byte 0, 0, 0, 0
```

30.12.2 Tuning Word

For A4 with the 32-bit accumulator:

```
M = round(440 × 2^32 / 7992)
  = round(440 × 4,294,967,296 / 7992)
  = round(1,889,785,610,240 / 7992)
  = round(236,459,661.4)
  = 236,459,661    (0x0E18168D)
```

30.12.3 ISR Accumulator Update

```
lds    r16, phase_accum32+0
lds    r17, phase_accum32+1
lds    r18, phase_accum32+2
lds    r19, phase_accum32+3

lds    r20, phase_step32+0
lds    r21, phase_step32+1
lds    r22, phase_step32+2
lds    r23, phase_step32+3

add    r16, r20
adc    r17, r21
adc    r18, r22
adc    r19, r23

sts    phase_accum32+0, r16
sts    phase_accum32+1, r17
sts    phase_accum32+2, r18
sts    phase_accum32+3, r19

/* Table index: top byte = r19 */
ldi    ZH, hi8(sine_table)
mov    ZL, r19
lpm    r21, Z
```

Cycle cost: $4 \times \text{LDS} + 4 \times \text{LDS} + 4 \times (\text{ADD}/\text{ADC}) + 4 \times \text{STS} + \text{LDI} + \text{MOV} + \text{LPM} = 12 + 12 + 4 + 8 + 1 + 1 + 3 = 41$ cycles for the accumulator and lookup, versus 23 cycles for the 16-bit version. The additional 18 cycles are a reasonable cost for the jump from 0.122 Hz resolution to 1.86 μHz resolution.

30.13 Common Pitfalls

30.13.1 Pitfall 1 — Aliasing Above Nyquist

Setting M above $2^{(N-1)} = 32768$ causes the output frequency to alias. With a 16-bit accumulator and $M = 40000$:

```
f_nominal = 7992 * 40000 / 65536 = 4878 Hz    (above Nyquist = 3996 Hz)
f_alias   = f_s - f_nominal = 7992 - 4878 = 3114 Hz
```

The output is not 4878 Hz but 3114 Hz — the mirror image below Nyquist. This is not an error in the DDS hardware; it is a mathematical property of discrete-time signals. The fix is to clamp M below $2^{(N-1)}$ before loading it into `phase_step`.

30.13.2 Pitfall 2 — Overflow in Tuning Word Calculation

The expression $f_{\text{out}} \times 2^N$ overflows a 16-bit integer for any frequency above 1 Hz when $N=16$. Always use 32-bit or wider arithmetic:

Wrong (16-bit overflow for any reasonable audio frequency):

```
M = (uint16_t)(440 * 65536) / 7992 → garbage
```

Correct:

```
M = (uint32_t)(440) * 65536UL / 7992 → 3608
```

In the assembly source, M is a precomputed constant. The overflow risk is in the host-side or C-side calculation, not in the ISR itself.

30.13.3 Pitfall 3 — Unaligned Sine Table

If `.balign 256` is missing or if the linker places the table at a non-aligned address, the `MOV ZL, r17` approach gives wrong results when `r17` rolls over. For example, if `sine_table` is at byte address `0x0180` (`ZH=0x01, ZL=0x80`), and the accumulator high byte is `0x00`, then `Z = 0x0100` — pointing before the table. When `ZL` wraps past `0xFF`, `Z` increments `ZH`, pointing one page past the end.

Verify alignment after building:

```
avr-objdump -h dds.elf | grep sine_table
```

or inspect the linker map for the `sine_table` address. If the address is not a multiple of 256, add `.balign 256` immediately before the `.byte` directives that define the table.

30.13.4 Pitfall 4 — Forgetting to Clear the TCB0 Interrupt Flag

`TCB0` in Periodic Interrupt mode does not clear its interrupt flag automatically. If `TCB0_INTFLAGS` is not written in the ISR, the interrupt immediately re-fires after `RETI`, consuming 100% of the CPU in the ISR. Always include:

```
ldi    r16, TCB_CAPT_bm
sts    TCB0_INTFLAGS, r16
```

Write 1 to the bit to clear it. Writing 0 has no effect.

30.13.5 Pitfall 5 — Phase Accumulator Not Zeroed Before Starting

The `.bss` section is zeroed by the C runtime startup code. In a `-nostartfiles` assembly build it is not — SRAM power-on state is undefined. The reset handler must zero `phase_accum` and `phase_step` explicitly before enabling the TCB0 interrupt. A non-zero initial `phase_step` can cause a brief burst of incorrect frequency output before the main loop loads the intended tuning word.

30.13.6 Pitfall 6 — Interrupt Latency Jitter Adds Phase Noise

If the DDS ISR competes with other ISRs for CPU time, the sample clock has jitter equal to the latency of whichever ISR runs first. Each jitter event shifts the phase of the output by a random amount, which adds broadband phase noise. To minimise this:

- Keep all other ISRs shorter than one sample period (417 cycles).
- If another interrupt source cannot be made that short, assign it a lower priority. On ATtiny3217, CPUINT supports two priority levels: normal (all ISRs by default) and high (one ISR assigned via `CPUINT_LVL1VEC`). Assign the DDS ISR to level 1 (high priority) for the best timing stability.

30.14 Relationship to CORDIC

Chapter 29 shows that the CORDIC algorithm can generate sine and cosine from a phase angle with no waveform table at all, at the cost of about 12 shift-add stages per sample. The two approaches trade differently:

Method	Flash (table)	Cycles/sample	Notes
256-byte LUT	256 bytes	~5 cycles	best speed, one output
CORDIC (12 stages)	~24 bytes	~80–120 cycles	sine + cosine, no table

For the DDS context, the LUT approach is almost always faster. CORDIC becomes attractive when flash is scarce, when both sine and cosine are needed from the same phase (e.g., I/Q generation), or when the phase is already in BAM16 from some other computation.

The phase accumulator described in this chapter is format-compatible with the BAM16 angles used by the CORDIC routines in Chapter 29. The high byte of the 16-bit DDS accumulator (0x00–0xFF) maps to angles 0°–360° in 256 steps. The CORDIC routines expect BAM16 (0x0000–0xFFFF). The relationship is:

$$\begin{aligned}\text{CORDIC_angle} &= \text{DDS_accumulator_high_byte} \times 256 \\ &= \text{DDS_accumulator} \times (0xFF00 / 0xFFFF) \approx \text{DDS_accumulator}\end{aligned}$$

More precisely: `CORDIC_angle_BAM16 = phase_accum (full 16-bit value)`

The full 16-bit DDS accumulator is already a valid BAM16 angle. To use CORDIC instead of the LUT in the DDS ISR, replace the LPM table lookup with a call to `cordic_sincos_bam16`, passing the accumulator as the angle argument. The sine output is then used as the PWM sample. The cycle cost rises from ~5 to ~80–120 cycles, which still fits within the 417-cycle budget for a single-channel DDS.

30.15 Summary

DDS core equation:

$$f_{\text{out}} = f_s \times M / 2^N$$

Tuning word for a target frequency:

$$M = \text{round}(f_{\text{out}} \times 2^N / f_s)$$

Resolution: $\Delta f = f_s / 2^N$

ATtiny3217 ($F_{\text{CPU}} = 3,333,333$ Hz) 16-bit DDS parameters:

Sample rate: CCMP = 416 $\rightarrow f_s = 7992$ Hz (target 8000 Hz, error -0.10%)

N: 16 (two SRAM bytes, two registers)

Resolution: $7992 / 65536 = 0.122$ Hz per tuning word step

Nyquist limit: 3996 Hz ($M < 32768$)

Phase spurs: ≈ -48 dBc (8 truncated bits, 256-entry table)

ISR cycle cost: ~ 61 cycles of a 417-cycle budget (14.6% CPU)

Table:

256 unsigned bytes in flash, .balign 256 for aligned-page ZL lookup

$\text{sine_table}[i] = \text{round}(127.5 + 127.5 \times \sin(2\pi \times i / 256))$

Phase modulation: add offset to accumulator copy before LPM (non-destructive)

Frequency mod: update phase_step from main loop or inner ISR

Chirp: increment phase_step by chirp_rate each sample

Two tones:

second accumulator + second step $\rightarrow$ average both samples before PWM write

total ISR cost ~ 89 cycles (21% CPU)

32-bit accumulator:

resolution ≈ 1.86 μHz at 7992 Hz sample rate

ISR cost ~ 79 cycles (additional ~ 18 cycles for the 4-byte add and stores)

Microchip source notes:

- ATtiny3216/3217 data sheet, TCB Periodic Interrupt mode: the TCB counter reloads from BOTTOM when it matches CCMP and generates a CAPT interrupt. CCMP is a buffered 16-bit register; write CCMPH first, CCMPH second.
- ATtiny3216/3217 data sheet, TCA Split Mode: each 8-bit counter has its own compare register (LCMP0-LCMP2, HCMP0-HCMP2) and corresponding output waveform pins. The PWM frequency in split mode is $\text{CLK_PER} / (\text{LPER} + 1)$.
- AVR Instruction Set Manual, LPM Rd, Z: loads the byte at the flash byte address in Z into Rd. On AVRxt devices including ATtiny3217, any register may be the destination. The Z register is not modified; Z+ post-increment form available but not needed here.
- AVR Instruction Set Manual, ADD Rd, Rr and ADC Rd, Rr: ADD sets carry on overflow; ADC adds carry into the result. Chaining one ADD and one or more ADC instructions implements arbitrary-width unsigned addition.

30.16 Exercises

1. Compute the tuning word M for the note G4 (392.000 Hz) using $f_s = 7992$ Hz and $N = 16$. What is the actual output frequency? What is the error in cents (1 cent = 1/100 of a semitone $\approx 0.058\%$ frequency change)?
2. The CCMP value 416 gives $f_s = 7992$ Hz. If you increase CCMP to 520 for a lower sample rate, what is the new f_s ? What is the maximum output frequency? What is the tuning word for 220 Hz (A3) at this sample rate?
3. Explain what happens to the output frequency when M is set to exactly 32768 (2^{15}). What does the waveform look like? What happens when $M = 32769$?
4. Modify the DDS ISR to read the phase step from an 8-bit potentiometer connected to an ADC input. The ADC result (0-1023) should map linearly to output frequencies 100 Hz-

- 4000 Hz. Write the tuning word conversion formula and describe where in the system the conversion runs (ISR or main loop).
5. Add a second sine table in flash for a square wave (table values: 0 for indices 0–127, 255 for indices 128–255). How many additional flash bytes does this cost? What change in the ISR lets the main loop select between the sine and square wave tables at run time?
 6. Measure the actual output frequency of the dds.S firmware using a frequency counter or oscilloscope, and compare it with the calculated $f_{\text{out}} = 439.99$ Hz for $M = 3608$. What sources of error remain between the measured and predicted values?

Next: Chapter 29 — CORDIC on AVR

Chapter 31

CORDIC on AVR

CORDIC (COordinate Rotation DIgital Computer) is one of the classic tricks for small processors. It computes trigonometric, inverse-trigonometric, and vector rotation results using only:

- shifts,
- adds/subtracts,
- comparisons, and
- a small table of constants.

That makes it a natural fit for AVR. The ATtiny3217 includes the AVR hardware multiplier, but Microchip's instruction set still has no general divide instruction and the device has no floating-point execution unit. When you need sine, cosine, vector angle, or vector magnitude with predictable execution time, CORDIC is often a better fit than general software floating-point math.

This chapter covers:

1. **Why CORDIC is useful on AVR** — where it beats general math routines, and where a lookup table is still better.
2. **Rotation mode** — computing sine and cosine from an angle.
3. **Vectoring mode** — reducing a vector to magnitude and angle.
4. **Fixed-point formats** — a practical AVR-friendly choice: Q1.15 for vector components and BAM16 for angles.
5. **Examples** — AVR assembly stage examples, register plans, and practical integration patterns.

The important theme is the same one that runs through the rest of the book: replace expensive, hard-to-bound math with small deterministic steps.

31.1 Why Use CORDIC on AVR?

CORDIC is attractive on AVR for three practical reasons:

1. **No floating-point execution unit.** General software floating-point trigonometry is much larger and harder to budget than a fixed shift-add kernel.
2. **Fixed iteration count.** A 12-stage CORDIC always does 12 stages. That is useful when you care about real-time bounds.
3. **One core, many functions.** The same shift-add structure can be used for sine/cosine, vector rotation, atan2 , and magnitude approximation.

Typical AVR uses:

- waveform generation in a DDS/NCO,

- phase tracking,
- I/Q demodulation,
- motor-control angle estimation,
- tilt/heading math from sensors, and
- display-oriented trig without a large general-purpose math package.

31.1.1 When Not to Use It

CORDIC is not magic. Sometimes a lookup table is better:

- If you only need sine values at 8-bit resolution, a 256-byte flash table is usually faster.
- If you need one-off trig rarely, a larger general-purpose math routine may be acceptable.
- If your MCU has enough flash and your timing budget is loose, a polynomial or LUT approach may be easier to maintain.

So the real question is not “is CORDIC clever?” It is “does repeated deterministic shift-add math fit this problem better than tables or general software arithmetic?”

31.2 The Core Idea

CORDIC works by applying a sequence of tiny rotations whose tangents are powers of two:

$\text{atan}(2^0)$, $\text{atan}(2^{-1})$, $\text{atan}(2^{-2})$, $\text{atan}(2^{-3})$, ...

Because $\tan(\theta_i) = 2^{-i}$, each stage can be written without a multiply:

```
x' = x plus_or_minus (y >> i)
y' = y minus_or_plus (x >> i)
z' = z minus_or_plus atan(2^-i)
```

The shifts replace a true multiply by $\tan(\theta_i)$.

There are two standard modes:

31.2.1 Rotation Mode

Start with a unit vector on the x-axis and rotate it toward the requested angle. After the final stage:

- x approximates $\cos(\theta)$
- y approximates $\sin(\theta)$

31.2.2 Vectoring Mode

Start with an input vector (x, y) and rotate it toward the x-axis. After the final stage:

- z approximates the vector angle (atan2)
- x approximates the vector magnitude, scaled by the CORDIC gain
- y approaches zero

This is why one algorithm can serve both “give me sine/cosine” and “tell me the angle/magnitude of this vector.”

31.3 Fixed-Point Choices for AVR

CORDIC is almost always done in fixed-point on AVR.

31.3.1 Vector Format: Q1.15

Use signed 16-bit values for x and y:

Q1.15: signed fraction in the range about -1.0 to +0.99997

```
0x4000 = +0.5
0x7FFF = +0.99997
0x0000 = 0.0
0xC000 = -0.5
0x8000 = -1.0
```

This is a good fit for sine and cosine because those values naturally live in the interval [-1, +1].

31.3.2 Angle Format: BAM16

For angles, a very AVR-friendly choice is **BAM16** (Binary Angular Measure):

```
Full turn = 65536 units
90°       = 16384 = 0x4000
180°      = 32768 = 0x8000
360°      = wraps to 0x0000
```

Why BAM16 works well:

- addition and wraparound are natural in 16-bit unsigned arithmetic,
- phase accumulators use the same representation,
- quadrants are easy to identify from the top two bits.

31.3.3 CORDIC Gain

The repeated pseudo-rotations scale the vector by a constant gain:

```
1 / K ≈ 1.646760258
K      ≈ 0.607252935
```

After 12 stages the gain is already close to the infinite-stage value:

```
K12 = 0.607252959
1/K12 = 1.646760193
```

For a 12-stage sine/cosine CORDIC, preload:

```
K_Q15 = round(0.607252959 × 32768) = 19898 = 0x4DBA
```

That means rotation mode typically starts with:

```
x = 0x4DBA ; +0.60725 in Q1.15
y = 0x0000
```

and the final result lands near properly scaled sine/cosine outputs.

31.4 The Angle Table

For 12 stages in BAM16:

i	atan(2 ⁻ⁱ)	BAM16 value
0	45.000°	0x2000
1	26.565°	0x12E4
2	14.036°	0x09FB

i	$\text{atan}(2^{-i})$	BAM16 value
3	7.125°	0x0511
4	3.576°	0x028B
5	1.790°	0x0146
6	0.895°	0x00A3
7	0.448°	0x0051
8	0.224°	0x0029
9	0.112°	0x0014
10	0.056°	0x000A
11	0.028°	0x0005

This table is tiny. Even stored as 16-bit words in flash, 12 stages only cost 24 bytes.

31.4.1 Convergence Range

Basic rotation-mode CORDIC converges only for angles near $\pm 99.88^\circ$, not over the full 360° range. On AVR, the usual fix is **quadrant reduction**:

1. Determine the quadrant from bits [15:14] of the BAM16 angle.
2. Reflect the angle into the first quadrant [0° , 90°].
3. Run the CORDIC core there.
4. Apply the final sign correction to sin and cos.

That is why BAM16 is convenient: the top two bits already tell you the quadrant.

One fixed-point detail matters at the quadrant boundaries: signed Q1.15 can represent -1.0 exactly (0x8000), but not $+1.0$. A production sine/cosine wrapper should special-case the exact axis angles (0° , 90° , 180° , 270°) or saturate positive full-scale outputs to 0x7FFF.

31.5 Rotation Mode for Sine and Cosine

Register-transfer view using Q1.15 vectors and BAM16 angles:

```
x = K_Q15
y = 0
z = reduced_angle      ; first quadrant, 0..90 degrees

repeat for stages i = 0..11:
    temp_x = arithmetic_shift_right(x, i)
    temp_y = arithmetic_shift_right(y, i)
    test sign bit of z
    when z is non-negative:
        x = x - temp_y
        y = y + temp_x
        z = z - atan_table[i]
    when z is negative:
        x = x + temp_y
        y = y - temp_x
        z = z + atan_table[i]
```

After the loop:

- x is the first-quadrant cosine
- y is the first-quadrant sine

Then apply quadrant signs. For exact axis angles, use the special-case or saturation rule above before applying the signs:

Quadrant	Reduced angle	cos sign	sin sign
0	angle	+	+
1	180°-angle	-	+
2	angle-180°	-	-
3	360°-angle	+	-

31.5.1 Example Values

With 12 stages and $K_{Q15} = 0x4DBA$, away from the exact axis-angle cases:

- 30° (0x1555) gives about $\cos \approx 0.8660$, $\sin \approx 0.5001$
- 45° (0x2000) gives about $\cos \approx 0.7070$, $\sin \approx 0.7072$
- 300° (0xD555) gives about $\cos \approx 0.5001$, $\sin \approx -0.8660$

That is easily good enough for control loops, DDS, UI graphics, and many sensor applications on AVR.

31.6 AVR Assembly Structure

An AVR-assembly CORDIC routine should be organized around three fixed pieces:

- quadrant reduction on the BAM16 input angle,
- a 12-entry atan table in flash, and
- either a counted loop or a fully unrolled set of stages.

31.6.1 Table Layout

One straightforward flash table is:

```
cordic_atan_bam16:
.word 0x2000, 0x12E4, 0x09FB, 0x0511
.word 0x028B, 0x0146, 0x00A3, 0x0051
.word 0x0029, 0x0014, 0x000A, 0x0005
```

Put this table in .text or another flash-resident section and align it to a word boundary. When reading it with LPM, remember that LPM fetches bytes; read the low byte and high byte separately, then advance Z to the next word.

For rotation mode, preload the vector with $K_{Q15} = 0x4DBA$:

```
ldi r24, lo8(0x4DBA) ; x low
ldi r25, hi8(0x4DBA) ; x high
clr r22 ; y low
clr r23 ; y high
```

31.6.2 Core Loop Skeleton

At a high level, the loop does this each stage:

```
1. form temp_x = x >> i      using signed shifts
2. form temp_y = y >> i      using signed shifts
3. if z >= 0:
    x = x - temp_y
    y = y + temp_x
    z = z - atan[i]
else:
    x = x + temp_y
```

```

    y = y - temp_x
    z = z + atan[i]
4. advance i and table pointer

```

That description is intentionally close to the register transfers you will actually code. On AVR, the main implementation choice is whether *i* is variable inside a loop or fixed in an unrolled stage sequence.

31.7 How This Maps to AVR Assembly

At the algorithm level, CORDIC is simple. At the assembly level, the main cost is the **variable arithmetic shift** by *i*.

For a general loop, $x \gg i$ and $y \gg i$ mean:

1. copy the 16-bit value to a temporary pair,
2. shift it right *i* times using ASR on the high byte and ROR on the low byte,
3. use the shifted temporary in the add/subtract step.

That is why many hand-tuned AVR CORDIC routines are either:

- partially unrolled, or
- fully unrolled.

When *i* is a compile-time constant, the shifts become fixed instruction sequences and the loop-control overhead disappears.

The file `src/cordic_helpers.S` contains a buildable support routine for the variable signed 16-bit shift:

```

/* Entry: R25:R24 = signed value, R17 = count. Exit: R25:R24 shifted. */
cordic_asr16:
    tst    r17
    breq   .Lasr16_done
.Lasr16_loop:
    asr    r25
    ror    r24
    dec    r17
    brne   .Lasr16_loop
.Lasr16_done:
    ret

```

31.7.1 Example: One Unrolled Stage (*i* = 3)

Suppose:

- R25:R24 = *x*
- R23:R22 = *y*
- R21:R20 = *z*

One positive-rotation stage for *i* = 3 looks like:

```

/* temp_y = y >> 3 (signed shift) */
movw   r18, r22          /* r19:r18 = y copy (MOVW needs even dest/src) */
asr     r19
ror     r18
asr     r19
ror     r18
asr     r19
ror     r18

```



```

/* x = x - temp_y */
sub    r24, r18
sbc    r25, r19

/* temp_x = old x >> 3
 * In a real implementation, preserve old x before modifying it. */

```

The mechanics are exactly the same as any other multi-byte signed shift on AVR: ASR the high byte, ROR the low byte, repeat as many times as needed.

31.7.2 Practical Register Plan

For a hand-written AVR version, a reasonable plan is:

```

R25:R24    x
R23:R22    y
R21:R20    z
R19:R18    temp shifted copy
R17        stage counter i
R16        scratch / table byte
Z          pointer into atan table

```

This is workable, but you can see why register pressure gets real quickly. CORDIC is a good example of a routine where AVR assembly quality depends as much on register planning as on the arithmetic itself.

31.8 Vectoring Mode: Magnitude and Angle

Vectoring mode starts with an input vector (x, y) and rotates it toward the x-axis:

```

z = 0
repeat for stages i = 0..11:
    temp_x = arithmetic_shift_right(x, i)
    temp_y = arithmetic_shift_right(y, i)
    test sign bit of y
    when y is non-negative:
        x = x + temp_x
        y = y - temp_y
        z = z + atan_table[i]
    when y is negative:
        x = x - temp_x
        y = y + temp_y
        z = z - atan_table[i]

```

After the last stage:

- z approximates the four-quadrant vector angle
- x approximates magnitude / K
- y is near zero

The magnitude is scaled upward by the CORDIC gain $1/K$, so you either:

- multiply the result by K afterwards,
- pre-scale the input vector by K, or
- calibrate the rest of the application around the scaled value.

This gain matters for overflow. A Q1.15 input vector with magnitude near 1.0 cannot safely be fed directly into a 16-bit vectoring core, because the final x wants to grow to

about 1.64676. Either pre-scale the vector by K, use a wider internal format such as 24-bit or 32-bit fixed point, or accept a smaller input range.

31.8.1 Where AVR Uses This

Vectoring mode is especially useful for:

- phase estimation from I/Q samples,
- converting Cartesian sensor axes to angle,
- simple magnitude estimation without a square root.

This is where CORDIC often beats ad-hoc math on AVR: one fixed-iteration loop replaces both angle extraction and square-root style magnitude calculation.

31.9 Example 1: DDS / NCO Sine Generator

A common AVR use is a numerically controlled oscillator:

1. keep a 16-bit phase accumulator,
2. add a phase step each sample period,
3. feed the phase into CORDIC rotation mode,
4. send the sine result to a PWM or DAC.

```
phase += phase_step;
call cordic_sincos_bam16
use high byte of sine as PWM sample
```

Why CORDIC can make sense here:

- no large sine table in flash,
- easy frequency control through `phase_step`,
- constant run time every sample.

If you need the very highest sample rate, a LUT is still usually faster. But if flash is tight and you want both sine and cosine from the same phase input, CORDIC is an attractive compromise.

31.10 Example 2: Tilt Angle from Two Axes

Suppose an AVR reads two already-filtered accelerometer channels:

```
x_axis = forward/back tilt component
y_axis = vertical tilt component
```

You want an angle estimate:

```
tilt = four-quadrant angle from x_axis and y_axis
```

Vectoring CORDIC gives the AVR a fixed-time integer path:

1. load (x, y) into Q1.15,
2. run vectoring mode,
3. interpret z in BAM16,
4. convert BAM16 to degrees only if the UI needs it.

That avoids general software floating-point math and keeps the control-loop representation in a format that is cheap to update.

31.11 Accuracy, Flash, and Cycle Tradeoffs

31.11.1 Iteration Count

More stages mean more accuracy and more cycles.

Typical choices on AVR:

- 8 stages: quick, coarse
- 12 stages: good practical middle ground
- 16 stages: near the limit of 16-bit usefulness

With Q1.15 outputs, 12 stages is usually enough because the result resolution is already limited by the 16-bit format.

31.11.2 LUT vs CORDIC

Rough comparison:

Method	Flash	Cycles	Strength
256-byte sine LUT	moderate	very low	fastest for sine only
CORDIC (12 stages)	small table + code	moderate, fixed	sine/cosine/angle/magnitude from one core
Polynomial fixed-point	small to moderate	data-dependent unless carefully bounded	good for one narrow function

The main advantage of CORDIC is not “fewest cycles.” It is that one deterministic integer core can cover several mathematically different jobs.

31.12 Common Pitfalls

31.12.1 Pitfall 1 — Forgetting Quadrant Reduction

Basic rotation mode does not converge over the full 360° range.

If you feed 210° directly into the core without reducing it first, the result will be wrong even if the stage math itself is perfect.

31.12.2 Pitfall 2 — Using Logical Right Shift on Signed Values

The shifts inside CORDIC are signed for x and y.

On AVR assembly:

```
asr    r25
ror    r24
```

is correct for a signed 16-bit right shift.

Using:

```
lsr    r25
ror    r24
```

would break negative vector components.

31.12.3 Pitfall 3 — Ignoring the Gain

CORDIC results are scaled. If you forget to preload with K in rotation mode, your sine/cosine amplitude will be too large. If you forget the gain in vectoring mode, your magnitude estimate will be off by a constant factor.

31.12.4 Pitfall 4 — Overflow in Narrow Formats

Q1.15 is ideal for sine/cosine outputs, but it is tight for vectoring-mode inputs.

If the input magnitude can approach 1.0, pre-scale by K or move to a wider fixed-point format before running vectoring CORDIC. Otherwise the internal $x = \text{magnitude} / K$ result can overflow even though both original components were inside $[-1, +1]$.

The same representation issue shows up at rotation-mode axis angles: $+1.0$ does not fit in signed Q1.15, so exact $\cos(0^\circ)$ or $\sin(90^\circ)$ must be saturated to $0x7FFF$ or handled as a special case.

31.12.5 Pitfall 5 — Hand-Writing the Loop Before Proving Correctness

CORDIC is simple enough to look obvious and subtle enough to get wrong.

On AVR, prove the math with a small set of known angles and compare the assembly against those expected values stage by stage. That is faster than debugging a broken fully-unrolled assembly version from scratch.

31.13 Summary

CORDIC on AVR:

- uses shifts + adds/subtracts + compare + small atan table
- no floating-point execution unit required
- fixed iteration count → deterministic timing

Useful formats:

- x, y = Q1.15 signed fractions
- angle = BAM16 (65536 units per turn)
- K_{Q15} = $0x4DBA$ for 12-stage rotation mode

Rotation mode:

- input = angle
- output = $\cos(\text{theta})$, $\sin(\text{theta})$

Vectoring mode:

- input = x, y vector
- output = angle and scaled magnitude

Main AVR tradeoff:

- slower than a small LUT for sine only
- more flexible than a LUT when you need sine/cosine/angle/magnitude

Build:

```
avr-gcc -mmcu=attiny3217 -nostartfiles -ndefaultlibs -o ch19.elf src/cordic_helpers.S
avr-objdump -h -d ch19.elf
```

This builds an inspection ELF for the helper routines. It is not a complete firmware image and should not be flashed by itself because it does not provide a reset vector or startup code.

Microchip source notes:

- ATtiny3217 product documentation lists an 8-bit AVR CPU with hardware multiplier, up to 20 MHz operation, 32 KB Flash, 2 KB SRAM, and 256 bytes EEPROM.
 - The AVR Instruction Set Manual device-core appendix lists ATtiny3217 as AVRxt. Its missing-instruction list does not include MUL, but the AVR instruction set has no general divide instruction.
 - The AVR Instruction Set Manual defines ASR as an arithmetic right shift and ROR as rotate right through carry; together they implement signed multi-byte right shifts for CORDIC stage terms.
-

31.14 Exercises

1. Implement an 8-stage CORDIC sine/cosine routine and compare its maximum error against the 12-stage version. How many cycles do you save?
 2. Store the BAM16 atan table in flash and read it with LPM through Z. Compare the flash cost and cycle cost against hard-coding the constants in a fully unrolled routine.
 3. Write an assembly helper that performs a signed 16-bit arithmetic right shift by a variable count in R17. Use ASR/ROR in a loop. How many cycles does it take for counts 0, 1, 4, and 8?
 4. Write a sine-only wrapper around the rotation-mode assembly core. Does a dedicated calling convention save any useful register moves, or are sine and cosine effectively “free” once the same 12 stages have already run?
 5. Implement vectoring mode for inputs pre-scaled by K and test it on a vector proportional to (0.6, 0.8). What BAM16 angle do you get, and how close is the corrected magnitude to 1.0?
 6. Compare three ways to generate a PWM sine on AVR:
 - a. 256-entry flash LUT
 - b. 12-stage CORDIC
 - c. fixed-point polynomial approximationMeasure flash usage with `avr-objdump -h` and inspect the call paths with `avr-objdump -d`.
-

Next: Chapter 30 — ADC and Analog Input

Chapter 32

ADC, Analog Comparator, and DAC

Digital inputs answer one question: is the pin low or high? Analog inputs answer a different question: how far is this voltage between ground and a reference voltage? The ADC converts that voltage into a number. The chapter closes with the other two analog peripherals that share the ADC's world — the Analog Comparator (AC0), which asks only “above or below?”, and the Digital-to-Analog Converter (DAC0), which turns a number back into a voltage.

The ATtiny3217 has two 10-bit ADC peripherals. This chapter uses **ADC0** and a single external input channel. The goal is practical bring-up: choose a pin, choose a reference, choose an ADC clock, start a conversion, wait for RESRDY, and read the result correctly.

32.1 The ADC Mental Model

An ADC conversion has four parts:

1. Select an input channel with `ADC0_MUXPOS`.
2. Select a voltage reference and ADC clock with `ADC0_CTRLA`.
3. Start a conversion by writing `ADC_STCONV_bm` to `ADC0_COMMAND`.
4. Wait for `ADC_RESRDY_bm`, then read `ADC0_RES` and `ADC0_RESH`.

For an ideal 10-bit single-ended conversion:

```
result = 1023 * Vin / Vref
Vin     = result * Vref / 1023
```

So with a 1.1 V reference:

```
0x000 = about 0 V
0x200 = about 0.55 V
0x3FF = about 1.1 V or higher
```

Input voltages above the selected reference saturate at the maximum result. The ADC does not tell you “too high”; it just returns the largest code.

32.2 ADC0 Register Path

The full register reference is in Appendix A. For a first polling conversion, these are the important registers:

Register	Purpose
VREF_CTRLA	Select internal reference voltage for ADC0
ADC0_CTRLA	Enable ADC, resolution, free-running mode
ADC0_CTRLC	Reference source and ADC clock prescaler
ADC0_CTRLD	Initial delay and sample delay
ADC0_SAMPCTRL	Extra sample length for high-impedance sources
ADC0_MUXPOS	Positive input channel selection
ADC0_COMMAND	Start conversion
ADC0_INTFLAGS	Result-ready flag
ADC0_RESL	Result low byte
ADC0_RESB	Result high byte

All ADC0 registers are memory-mapped above low I/O space, so use LDS and STS, not IN and OUT.

32.3 Choosing an Analog Pin

The ADC input mux names channels as AIN0, AIN1, and so on. On the ATtiny3217:

AIN0..AIN7	PA0..PA7
AIN8	PB5
AIN9	PB4
AIN10	PB1
AIN11	PB0

For simple experiments, avoid:

- PA0, because it is also UPDI.
- PA3, because the Curiosity Nano LED is on PA3.
- pins already used by USART, SPI, or TWI examples in your project.

This chapter uses AIN4 on PA4.

Before using a pin as an analog input, disable its digital input buffer:

```
ldi    r16, 0x04
sts    PORTA_PIN4CTRL, r16    ; ISC = input buffer disabled
```

That disconnects the digital input path from the analog signal. It reduces power and avoids the digital input buffer reacting to slow voltages between logic low and logic high.

32.4 Choosing the Reference

The reference voltage defines the top of the conversion range. ADC0 can use:

- the internal reference from VREF
- VDD
- an external reference on VREFA

For repeatable examples, this chapter uses the internal 1.1 V reference:

```
ldi    r16, (0x01 << 4)      ; ADC0REFSEL = 1.1 V, bits 6:4
sts    VREF_CTRLA, r16

ldi    r16, VREF_ADC0REFEN_bm ; force ADC0 reference on
sts    VREF_CTRLB, r16
```

Then select the internal reference source in ADC0_CTRLC by leaving REFSEL[1:0] = 0.

VREF_ADC0REFEN_bm is not an output-enable bit for a pin. It keeps the internal ADC0 reference running even when the ADC is not actively requesting it, which avoids first-sample startup surprises. Leave it clear when using an external VREFA reference.

The ADC0_CTRLC reference-select values are:

REFSEL	ADC reference source
0	internal reference from VREF
1	VDD
2	external VREFA pin
3	reserved

The practical rule is simple:

- Use an internal reference when you want the result to mean volts.
- Use VDD when you want the result to be ratiometric with the supply.
- Use an external reference when the board provides a precise analog reference.

Ratiometric measurement is useful for sensors powered by the same supply as the MCU. If both the sensor output and ADC reference move with VDD, the code can stay stable even when the exact supply voltage changes.

32.4.1 SAMPCAP

The SAMPCAP bit in ADC0_CTRLC (bit 6) selects the size of the internal sample-and-hold capacitor:

SAMPCAP	Recommended when
0	Reference voltage is below 1 V
1	Reference voltage is 1 V or higher

Set SAMPCAP = 1 when using the internal 1.1 V reference, VDD, or an external reference above 1 V. The datasheet requires it for the temperature sensor channel. Since the internal 1.1 V reference is the default choice for this chapter, SAMPCAP is included in the ADC0_CTRLC writes that follow.

32.5 Choosing the ADC Clock

For maximum 10-bit resolution, the datasheet specifies an ADC clock from 200 kHz to 1.5 MHz when using a 1.1 V or higher reference (100 kHz to 260 kHz with the 0.55 V reference). The ADC clock comes from CLK_PER through the ADC prescaler:

PRESC	Division
0	/2
1	/4
2	/8
3	/16
4	/32


```

5      /64
6      /128
7      /256

```

At the factory-default Curiosity Nano clock, CLK_PER is usually 20 MHz / 6, about 3.333 MHz. Good choices are:

```

DIV8      416.7 kHz
DIV16     208.3 kHz

```

This chapter uses DIV16. The write also includes SAMPCAP (bit 6) because the internal 1.1 V reference is above the 1 V threshold:

```

ldi  r16, (1 << 6) | 0x03 ; SAMPCAP=1, REFSEL=internal, PRESC=DIV16
sts  ADC0_CTRL, r16

```

A normal conversion takes 13 ADC clocks, plus any configured sampling delay and sample length. At 208.3 kHz, 13 clocks is about 62 us before software overhead.

When using or changing an internal reference, give the reference and input mux time to settle before the first sample. ADC0_CTRLD provides INITDLY for that purpose. This chapter uses 32 ADC clocks:

```

ldi  r16, 0x40 ; INITDLY = 32 CLK_ADC cycles
sts  ADC0_CTRLD, r16

```

32.6 Sampling Time

The ADC does not measure the pin forever. It briefly connects the input to an internal sample-and-hold capacitor, then converts the held voltage.

Low-impedance sources charge that capacitor quickly. High-impedance sources need more time. The datasheet calls out about 10 kOhm or less as the easy case. For larger source impedances, increase ADC0_SAMPCTRL.

The sample-time formula is:

```

SampleTime = (2 + SAMPDLY + SAMPLEN) / CLK_ADC

```

This chapter uses a small extra sample length:

```

ldi  r16, 8
sts  ADC0_SAMPCTRL, r16

```

That is conservative for breadboard experiments, potentiometers, and simple sensor outputs. It costs conversion time, but it makes first measurements less fragile.

32.7 The CALIB Register

ADC0_CALIB (offset 0x16) has one writable bit: DUTYCYC.

DUTYCYC	ADC clock duty cycle	Required when
1	25% high / 75% low	ADCclk ≤ 1.5 MHz (default)
0	50% high / 50% low	ADCclk > 1.5 MHz

The reset value is 0x01. That is the correct setting for 10-bit operation at any ADC clock up to 1.5 MHz, which covers every prescaler in the table above at the factory-default 3.333

MHz CPU clock. Most code never touches this register.

Only clear DUTYCYC if you push the ADC clock above 1.5 MHz for faster 8-bit conversions. That mode also requires $VDD \geq 2.7$ V; check the electrical characteristics section before using it.

32.7.1 Software Calibration

There are no hardware offset or gain trim registers. Accuracy on the ATtiny3217 ADC is determined by silicon and by the reference you choose. Two practical techniques apply in firmware:

Single-point offset correction. Read the GND channel (MUXPOS = 0x1F) and subtract the result from every real reading. On a well-designed board the GND reading is zero or very close to it. If it is not, you have a layout or decoupling problem to fix first.

Two-point gain correction. Apply two known voltages to the input (or use the internal reference on the INTREF channel with VDD as the ADC reference), read both, and compute a slope and intercept. This is a calibration step performed once, with the correction factors stored in EEPROM or the User Row.

For most embedded measurements, a stable reference voltage and good analog layout give sufficient accuracy without software correction.

32.8 Polling Conversion Routine

The simplest ADC routine starts one conversion and waits until the result is ready:

```
adc0_read_ain4:
    ldi    r16, ADC_STCONV_bm
    sts    ADC0_COMMAND, r16

.Lwait_adc:
    lds    r16, ADC0_INTFLAGS
    sbrs   r16, ADC_RESRDY_bp
    rjmp   .Lwait_adc

    lds    r24, ADC0_RESL
    lds    r25, ADC0_RESR

    ldi    r16, ADC_RESRDY_bm
    sts    ADC0_INTFLAGS, r16
    ret
```

r25:r24 now holds the right-adjusted 10-bit result. The useful bits are:

```
r25:r24 = 000000xx xxxxxxxx
           high    low
```

Read low byte first. That matches the normal AVR order for 16-bit peripheral results.

Reading ADC0_RESL and ADC0_RESR clears RESRDY on this device. Microchip's ADC examples also clear RESRDY by writing a 1 to the flag bit after the conversion is complete. The routine above does the same with `ADC0_INTFLAGS = ADC_RESRDY_bm`; after the result read this write is redundant but harmless, and it keeps the write-one-to-clear style explicit.

32.9 Complete Example: Read AIN4 and Drive an LED

The example reads PA4/AIN4. If the 10-bit result is at least half scale (≥ 512), it turns the Curiosity Nano LED off. If the result is below half scale, it turns the LED on.

The LED on PA3 is active-low:

```
PA3 = 0   LED on
PA3 = 1   LED off
```

Source file: src/adc_poll.S

```
#include <avr/io.h>

.equ LED_BIT,      3           ; PA3, active-low LED0
.equ ADC_PINCTRL,  0x04       ; PORT ISC input buffer disabled
.equ ADC_REF_1V1,  0x10       ; VREF ADC0REFSEL = 1.1 V, bits 6:4
.equ ADC_REFEN,    0x02       ; VREF ADC0REFEN
.equ ADC_PRESC_DIV16, 0x43    ; ADC0_CTRL: SAMPCAP=1, internal ref, prescaler /16
.equ ADC_INITDLY32, 0x40      ; ADC0_CTRLD: 32 CLK_ADC startup delay
.equ ADC_AIN4,     0x04       ; PA4 / AIN4
.equ ADC_SAMPLEN8,  8

.section .vectors, "ax", @progbits
.global __vectors
__vectors:
    jmp     main
    .rept 30
    jmp     default_isr
    .endr

.section .text
.global main
main:
    ldi     r16, hi8(RAMEND)
    out     SPH, r16
    ldi     r16, lo8(RAMEND)
    out     SPL, r16

    sbi     VPORTA_DIR, LED_BIT
    sbi     VPORTA_OUT, LED_BIT        ; LED off

    ldi     r16, ADC_PINCTRL
    sts     PORTA_PIN4CTRL, r16        ; disable digital input on PA4

    ldi     r16, ADC_REF_1V1
    sts     VREF_CTRLA, r16            ; internal 1.1 V reference for ADC0

    ldi     r16, ADC_REFEN
    sts     VREF_CTRLB, r16            ; force ADC0 reference on

    ldi     r16, ADC_PRESC_DIV16
    sts     ADC0_CTRL, r16

    ldi     r16, ADC_INITDLY32
    sts     ADC0_CTRLD, r16

    ldi     r16, ADC_SAMPLEN8
    sts     ADC0_SAMPCTRL, r16
```

```

ldi    r16, ADC_AIN4
sts    ADC0_MUXPOS, r16

ldi    r16, ADC_ENABLE_bm
sts    ADC0_CTRLA, r16

.Lmain_loop:
rcall  adc0_read_ain4

sbrs   r25, 1                      ; bit 9 set means result >= 512
rjmp   .Lbelow_half

sbi    VPORTA_OUT, LED_BIT         ; LED off
rjmp   .Lmain_loop

.Lbelow_half:
cbi    VPORTA_OUT, LED_BIT         ; LED on
rjmp   .Lmain_loop

adc0_read_ain4:
ldi    r16, ADC_STCONV_bm
sts    ADC0_COMMAND, r16

.Lwait_adc:
lds    r16, ADC0_INTFLAGS
sbrs   r16, ADC_RESRDY_bp
rjmp   .Lwait_adc

lds    r24, ADC0_RESL
lds    r25, ADC0_RESR

ldi    r16, ADC_RESRDY_bm
sts    ADC0_INTFLAGS, r16
ret

default_isr:
reti

```

Build:

```

avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
-o adc_poll.elf src/adc_poll.S
avr-objdump -d adc_poll.elf
avr-objcopy -O ihex adc_poll.elf adc_poll.hex

```

Hardware test:

- Connect a potentiometer between 1.1 V or a safe reference voltage and GND, with the wiper to PA4.
- Do not drive PA4 above VDD.
- If using the internal 1.1 V ADC reference, values above about 1.1 V saturate.

For a first bench test, a divided voltage below 1.1 V is easier to reason about than a full 0-5 V potentiometer.

32.10 ISR-Driven Conversion

32.10.1 When to Poll and When to Use an ISR

Polling is the right choice when the sample rate is low, the conversion is part of a simple foreground loop, and nothing else needs the CPU while waiting for RESRDY. The examples so far all use polling.

An ADC interrupt is better when:

- conversions happen periodically and other work must not stall
- a timer or event triggers sampling and the foreground should run freely
- the design already uses other interrupts and consistency favors ISRs

The hardware mechanism is the same either way: RESRDY in ADC0_INTFLAGS is set when a result is ready. In polling mode you spin on this flag. In interrupt mode you configure the ADC to request an interrupt when the flag sets.

32.10.2 Setting Up the Interrupt

Enable the RESRDY interrupt in ADC0_INTCTRL before calling SEI:

```
ldi    r16, ADC_RESRDY_bm      ; bit 0 = RESRDY interrupt enable
sts    ADC0_INTCTRL, r16
sei
```

The ADC0_RESRDY vector is _VECTOR(20) on ATtiny3217, at flash address 0x0050. In the vector table:

```
; in .section .vectors
; ...
; slot 20 (byte offset 0x50):
jmp    adc0_resrdy_isr
```

32.10.3 The ISR

The ISR reads both result bytes and stores them in a two-byte SRAM variable for the main loop to consume. Reading ADC0_RESL clears the RESRDY flag automatically on this device; the explicit write to INTFLAGS afterward is not required when reading both bytes, but it is harmless.

```
.section .bss
adc_latest_lo: .skip 1      ; low byte of most recent result
adc_latest_hi: .skip 1      ; high byte of most recent result

adc0_resrdy_isr:
    push    r16
    in      r16, SREG        ; save flags – arithmetic in main may be mid-compare
    push    r16

    lds     r16, ADC0_RESL    ; read low byte first – clears RESRDY flag
    sts     adc_latest_lo, r16
    lds     r16, ADC0_RESR
    sts     adc_latest_hi, r16

    pop     r16
    out     SREG, r16        ; restore flags exactly
    pop     r16
    reti
```

32.10.4 Reading the Result in the Main Loop

Both bytes are written in sequence by the ISR. A conversion interrupt between the two LDS reads in the main loop would update both bytes, leaving a torn result. Protect the read with a brief critical section:

```
cli
lds  r24, adc_latest_lo
lds  r25, adc_latest_hi
sei
; r25:r24 = latest 10-bit ADC result, consistent snapshot
```

For single-byte results (8-bit resolution mode) the critical section is unnecessary, since an 8-bit store and load are individually atomic.

32.10.5 Keeping the ISR Short

Do not perform filtering, scaling, BCD conversion, serial output, or display updates inside the ADC ISR. Those operations take many cycles and extend the time that other interrupts are blocked. Store the raw result and return. Let the foreground loop do the arithmetic.

32.11 Accumulation and Simple Filtering

ADC0 can accumulate multiple samples in hardware. That reduces interrupt or polling overhead when you want an averaged reading, but it changes the scale of the result.

For example, if 16 samples are accumulated:

```
raw accumulated range = 0 .. 16 * 1023 = 0 .. 16368
average                = accumulated >> 4
```

Hardware accumulation is useful for noise reduction, but it is not magic. It helps with random noise. It does not fix:

- a bad reference
- a floating input
- a source impedance too high for the sample time
- digital switching noise coupled into the analog wiring
- sampling at exactly the wrong point in a periodic waveform

For slow sensors, a simple foreground low-pass filter is often enough:

```
filtered = filtered + ((new_sample - filtered) >> n)
```

With $n = 3$, each new sample contributes one eighth of the correction. That kind of filter belongs in the fixed-point/control material later in the book.

32.11.1 Automatic Sampling Delay Variation (ASDV)

Hardware accumulation takes multiple samples in rapid succession. If a periodic interference source — switching regulator, clock harmonic, LCD multiplexing — has a frequency that aliases with the sample rate, it can add a fixed bias to every accumulated result instead of averaging out.

Setting ASDV (bit 4 of ADC0_CTRLD) randomises the interval between accumulated samples by incrementing SAMPDLY by one automatically after each sample. The sampling instant shifts by one ADC clock each time and wraps at 15. This spreads the interference across different phases so it averages out across accumulation bursts.

```
ldi    r16, (1 << 4) | (0x40) ; ASDV=1, INITDLY=32 cycles
sts    ADC0_CTRLD, r16
```

Use ASDV when hardware accumulation is enabled and you see unexpected fixed offsets in averaged results that do not improve with more samples.

32.12 Free-Running Mode

In single-conversion mode, software writes ADC_STCONV_bm each time it wants a sample. In free-running mode, software starts the first conversion and hardware immediately starts the next one after each result.

The control change is small:

```
ldi    r16, (ADC_ENABLE_bm | ADC_FREERUN_bm)
sts    ADC0_CTRLA, r16

ldi    r16, ADC_STCONV_bm
sts    ADC0_COMMAND, r16 ; starts the continuous stream
```

The read side still waits for RESRDY, reads the result, and clears the flag:

```
adc0_read_latest:
.lwait_adc:
    lds    r16, ADC0_INTFLAGS
    sbrs   r16, ADC_RESRDY_bp
    rjmp   .lwait_adc

    lds    r24, ADC0_RESL
    lds    r25, ADC0_RESH

    ldi    r16, ADC_RESRDY_bm
    sts    ADC0_INTFLAGS, r16
    ret
```

Use free-running mode when you want a steady stream from one channel. It is less convenient when changing channels, because after a channel change the next result may still reflect the previous channel selection. Discard one result after switching channels unless you have verified the timing carefully.

Source file: src/adc_freerun.S

32.13 Hardware Sample Accumulation

Hardware accumulation performs several conversions and adds the results into one result register value. Microchip's TB3209 examples use this pattern to average multiple samples with less software overhead.

For a 16-sample accumulation:

```
ldi    r16, 0x04 ; SAMPNUM = ACC16
sts    ADC0_CTRLB, r16
```

The raw result is now a sum:

```
raw range = 0 .. 16 * 1023
average   = raw >> 4
```

In assembly, right-shift the 16-bit result four times:

```
; r25:r24 = accumulated result
lsr    r25
ror    r24
lsr    r25
ror    r24
lsr    r25
ror    r24
lsr    r25
ror    r24
```

Hardware accumulation helps when noise is random and centered. It does not repair a floating input, poor grounding, a bad reference, or a source that cannot charge the sample-and-hold capacitor quickly enough.

Source file: `src/adc_accum16.S`

32.14 Window Comparator

The ADC window comparator lets hardware compare the result against a threshold or range. It can set WCMP and optionally request an interrupt when the result is below, above, inside, or outside the configured window.

For a simple low-threshold detector:

```
; WINLT = 0x0200 (half scale)
ldi    r16, 0x00
sts    ADC0_WINLTL, r16
ldi    r16, 0x02
sts    ADC0_WINLTH, r16

ldi    r16, 0x01 ; WINCM = BELOW
sts    ADC0_CTRL, r16
```

After each conversion, check `ADC_WCMP_bm` in `ADC0_INTFLAGS`:

```
lds    r16, ADC0_INTFLAGS
sbrc   r16, ADC_WCMP_bp ; skip next if WCMP = 0
rcall  handle_below_threshold

ldi    r16, (ADC_RESRDY_bm | ADC_WCMP_bm)
sts    ADC0_INTFLAGS, r16
```

The window comparator is useful when the CPU only cares about threshold crossings. It is also useful with accumulation: the comparison happens after the final sample in the accumulation burst, so thresholds apply to the accumulated value unless you scale them to match an averaged result.

Reading `ADC0_RES` clears `WCMP` as well as `RESRDY`, so code that cares about the window-comparator decision should read `ADC0_INTFLAGS` before reading the result bytes.

Source file: `src/adc_window.S`

32.15 Event-Triggered Conversion

Software-triggered conversions are easy, but they happen whenever the foreground loop reaches the start instruction. Event-triggered conversions move that start signal into hardware. A timer, RTC/PIT, pin edge, or other event generator can start the ADC at a repeatable time.

The ADC side only needs one bit:

```
ldi    r16, ADC_STARTEI_bm
sts    ADC0_EVCTRL, r16      ; ADC starts on incoming event
```

Then EVSYS connects a generator to the ADC0 user. On ATtiny3217, asynchronous user 1 is ADC0. The user value 0x06 selects ASYNCCH3:

```
ldi    r16, 0x0D             ; ASYNCCH3 generator = PIT_DIV1024
sts    EVSYS_ASYNCCH3, r16

ldi    r16, 0x06             ; ADC0 listens to ASYNCCH3
sts    EVSYS_ASYNCUSER1, r16
```

Finally wait until the PIT control register is not synchronizing, then enable the RTC PIT source:

```
.Lwait_pit_sync:
    lds    r16, RTC_PITSTATUS
    sbrc   r16, RTC_CTRLBUSY_bp
    rjmp   .Lwait_pit_sync

    ldi    r16, ((9 << 3) | RTC_PITEN_bm) ; PIT CYC1024 period select + enable
    sts    RTC_PITCTRLA, r16
```

After that, the foreground code can simply wait for RESRDY. No write to ADC0_COMMAND is needed for each sample; the PIT event starts conversions.

Event-triggered ADC is the clean path for periodic sampling because the sample time no longer depends on foreground loop timing.

Source file: src/adc_event_pit.S

32.16 Temperature Measurement

The ATtiny3217 has an on-chip temperature sensor accessible as a special ADC input channel. It reads a voltage that is linear with junction temperature. The slope and offset vary between individual devices because of process variation, so accurate conversion requires two factory calibration values from the Signature Row.

32.16.1 Signature Row Calibration Values

The Signature Row is a read-only flash region at base address 0x1100. Two bytes are relevant:

Address	Symbol	Type	Description
0x1120	SIGROW_TEMPSENSE0	uint8	gain/slope correction
0x1121	SIGROW_TEMPSENSE1	int8	offset correction (signed)

Read them with LDS from the mapped Signature Row address:

```
lds    r18, 0x1120    ; TEMPSENSE0 = gain (unsigned byte, typically ~230-270)
lds    r19, 0x1121    ; TEMPSENSE1 = offset (signed byte, typically -10 to +10)
```

32.16.2 Setup for Temperature Sampling

The temperature channel has stricter requirements than a normal external input:

Register	Setting	Reason
VREF_CTRLA	1.1 V reference	required; do not use VDD for temp sensor
ADC0_CTRLB	SAMPCAP=1, REFSEL=internal	required for 1.1 V reference; PRESC=DIV16
ADC0_CTRLD	INITDLY >= 32 us	reference and mux settle time
ADC0_SAMPCTRL	SAMPLEN >= 32 us	sensor output requires long sampling
ADC0_MUXPOS	0x1E	TEMPSENSE channel

At 208 kHz ADC clock, 32 μ s corresponds to about 7 ADC clocks. Using DLY32 for INITDLY and SAMPLEN = 31 both satisfy the requirement with margin:

```
; INITDLY = DLY32 (bits 7:5 = 0b010, value 0x40) and SAMPDLY = 0
ldi    r16, 0x40
sts    ADC0_CTRLD, r16

ldi    r16, 31
sts    ADC0_SAMPCTRL, r16

ldi    r16, 0x1E    ; TEMPSENSE channel
sts    ADC0_MUXPOS, r16
```

Then trigger a conversion the same way as a normal channel and read RESL / RESH when RESRDY is set.

32.16.3 Converting Raw ADC to Kelvin

The datasheet formula:

$$\text{Temp_K} = ((\text{adc_reading} - \text{TEMPSENSE1}) * \text{TEMPSENSE0} + 0x80) \gg 8$$

Subtract the signed offset from the raw result, multiply by the unsigned gain, add 0x80 for rounding, then shift right 8. The intermediate product can reach about 18 bits so the calculation needs a 24-bit accumulator.

```
; adc_temp_kelvin - convert TEMPSENSE raw reading to Kelvin
; Input:  r25:r24 = raw ADC result (10-bit, from TEMPSENSE channel)
; Output: r25:r24 = temperature in Kelvin (unsigned 16-bit)
; Clobbers: r0, r1, r18, r19, r20, r21, r22, r23
adc_temp_kelvin:
    lds    r18, 0x1120    ; SIGROW_TEMPSENSE0 = gain (unsigned)
    lds    r19, 0x1121    ; SIGROW_TEMPSENSE1 = offset (signed int8)

    ; sign-extend 8-bit offset to 16-bit r20:r19
    mov    r20, r19
    lsl    r20
    sbc    r20, r20    ; r20 = 0xFF if negative, 0x00 if positive

    ; r25:r24 = adc_reading - offset (sign-extended subtraction)
    sub    r24, r19
    sbc    r25, r20
```

```

; 16-bit x 8-bit multiply: 24-bit result in r21:r23:r22
; byte[0] = r22 (fractional, discarded after rounding)
; byte[1] = r23 (low Kelvin byte)
; byte[2] = r21 (high Kelvin byte, 0x01 for 256-358 K range)
clr    r21
mul    r24, r18                ; r1:r0 = low_byte * gain
mov    r22, r0                ; byte[0]
mov    r23, r1                ; byte[1] partial
mul    r25, r18                ; r1:r0 = high_byte * gain
add    r23, r0                ; byte[1] += low half of high partial product
adc    r21, r1                ; byte[2] = high half + carry

; add 0x80 to byte[0] for correct rounding before >> 8
ldi    r20, 0x80
add    r22, r20
ldi    r20, 0
adc    r23, r20
adc    r21, r20

; shift right 8: result = byte[2]:byte[1]
mov    r24, r23                ; low byte of Kelvin result
mov    r25, r21                ; high byte of Kelvin result
clr    r1                      ; restore zero register (MUL clobbered r1)
ret

```

Subtract 273 to convert Kelvin to Celsius:

```

rcall  adc_temp_kelvin          ; r25:r24 = Kelvin
subi   r24, lo8(273)
sbci   r25, hi8(273)
; r25:r24 = temperature in Celsius (signed 16-bit)
; typical range on ATtiny3217: -40 to +85, fits in int8 but use int16 for safety

```

Room temperature (25 C = 298 K) is the expected result for a bench test at approximately 20-25 C ambient with the MCU lightly loaded.

32.16.4 Supply Voltage Estimation

Reading the INTREF channel (MUXPOS = 0x1D) with VDD as the ADC reference (REFSEL = 0x01) produces a result proportional to the internal reference voltage divided by VDD:

```

result = 1023 * Vref_internal / VDD
VDD = 1023 * Vref_internal / result

```

With Vref_internal = 1.1 V:

```

VDD (mV) = (1023 * 1100) / result = 1125300 / result

```

This is useful as a battery level indicator without any external components. The accuracy depends on the tolerance of the internal reference (typically $\pm 2\%$, rising to about $\pm 5\%$ over the full temperature and voltage range), so treat the result as an estimate, not a precision measurement.

32.17 The Analog Comparator (AC0)

The ADC answers “what voltage is on this pin?” The Analog Comparator answers a simpler, faster question: “is this voltage above or below that one?” It compares a positive input against a negative input and produces a single bit — high when positive > negative. There

is no conversion clock and no result register to wait on: the answer is live in `AC0_STATUS`, updated continuously by analog hardware.

That makes the AC the right tool for threshold detection — an over-voltage trip, a zero-crossing detector, a light/dark switch — where a full ADC reading is more work than the problem needs.

32.17.1 Inputs and the OUT pin

The positive and negative inputs are each chosen from a small mux. On the 24-pin ATtiny3217 (datasheet sec.29):

MUXPOS	positive input	MUXNEG	negative input
0	AINP0 = PA7	0	AINN0 = PA6
1	AINP1 = PB5	1	AINN1 = PB4
2	AINP2 = PB1	2	VREF (shared DAC0/AC0 reference)
3	AINP3 = PB6	3	DAC (DAC0 output)

AC0 OUT = PA5

The two most useful negative inputs need no external parts: VREF compares against the internal reference voltage (set by `DAC0REFSEL`, see below), and DAC compares against the DAC0 output — a programmable threshold you can sweep in software.

32.17.2 Register path

The AC is four registers:

```
AC0_CTRLA    ENABLE(0) HYSMODE[2:1] LPMODE(3) INTMODE[5:4] OUTEN(6) RUNSTDBY(7)
AC0_MUXCTRLA MUXNEG[1:0] MUXPOS[4:3] INVERT(7)
AC0_INTCTRL  CMP(0) - interrupt enable
AC0_STATUS   CMP(0) = interrupt flag (write 1 to clear); STATE(4) = live output
```

STATE is the instantaneous comparator output — read it any time. CMP in STATUS is the *latched* interrupt flag, set on the edge selected by INTMODE and cleared by writing a 1 back to it. Enabling OUTEN also drives the result onto PA5 so it can be probed or routed to another peripheral.

32.17.3 Example: threshold detector on PA7

This compares AINP0 (PA7) against the internal 1.1 V reference and mirrors the result onto the on-board LED (PA3) — LED on while PA7 is above 1.1 V. The full runnable program is `src/ch20_adc/ac_compare.S`:

```
.equ AC_MUXPOS_PIN0,      (0x00 << 3)    /* MUXPOS = AINP0 (PA7) */
.equ AC_MUXNEG_VREF,      (0x02 << 0)    /* MUXNEG = internal VREF */
.equ VREF_DAC0REFSEL_1V1, 0x01          /* DAC0REFSEL = 1.1 V */
.equ PORT_ISC_INPUT_DISABLE, 0x04

main:
    /* 1. shared DAC0/AC0 reference = 1.1 V (VREF.CTRLA) */
    ldi r16, VREF_DAC0REFSEL_1V1
    sts VREF_CTRLA, r16

    /* 2. disable the digital input buffer on PA7 */
    ldi r16, PORT_ISC_INPUT_DISABLE
    sts PORTA_PIN7CTRL, r16

    /* 3. OUT pin PA5 and LED pin PA3 as outputs */
    sbi VPORTA_DIR, 5
```

```

sbi    VPORTA_DIR, 3

/* 4. mux: positive = AINP0, negative = VREF */
ldi    r16, AC_MUXPOS_PIN0 | AC_MUXNEG_VREF
sts    AC0_MUXCTRLA, r16

/* 5. enable AC0, drive result on PA5 */
ldi    r16, AC_ENABLE_bm | AC_OUTEN_bm
sts    AC0_CTRLA, r16

loop:
lds    r16, AC0_STATUS
sbrc   r16, AC_STATE_bp           /* STATE (bit 4) = live output */
rjmp   led_on
cbi    VPORTA_OUT, 3
rjmp   loop
led_on:
sbi    VPORTA_OUT, 3
rjmp   loop

```

As in the ADC examples, the analog input pin needs its **digital input buffer disabled** (`PORTA_PIN7CTRL = INPUT_DISABLE`) — otherwise the slowly varying analog level keeps toggling the digital input, wasting power. The `_bm` masks (`AC_ENABLE_bm`, `AC_OUTEN_bm`, `AC_STATE_bp`) are `#defines` and assemble directly; the `MUXPOS/MUXNEG` group configs are C enums in `iotn3217.h`, so their values are written with `.equ`.

To detect *edges* instead of polling the level, set `INTMODE` (e.g. `0x3 << 4` for rising edge), write `AC_CMP_bm` to `AC0_INTCTRL`, and `SEI`. The ISR clears the flag by writing `AC_CMP_bm` back to `AC0_STATUS`.

32.17.4 Hysteresis

A comparator looking at a slowly moving or noisy signal will *chatter* — flip back and forth rapidly as the input dithers across the threshold. The `HYSMODE` field adds hysteresis (10, 25, or 50 mV) so the input must cross a little past the threshold before the output flips back, cleaning up the transition:

<code>HYSMODE</code>	<code>hysteresis</code>
<code>0x0</code>	<code>off</code>
<code>0x1</code>	<code>±10 mV</code>
<code>0x2</code>	<code>±25 mV</code>
<code>0x3</code>	<code>±50 mV</code>

32.18 The Digital-to-Analog Converter (DAC0)

The DAC is the mirror image of the comparator: it turns a number into a voltage. `DAC0` is an 8-bit resistor-string converter that drives the `OUT` pin (`PA6`) to:

$$V_{out} = (\text{DAC0_DATA} / 256) * V_{REF}$$

The reference is the same shared `DAC0REFSEL` value the AC uses. With `VREF = 2.5 V`, `DATA = 128` gives mid-scale, 1.25 V; `DATA = 255` gives 2.49 V; full scale is one LSB (`VREF/256`, here ~9.8 mV) below `VREF`.

The DAC output can drive an external pin (`PA6`, with a 5 kΩ / 30 pF load limit), or feed the comparator and ADC internally — recall the AC's `MUXNEG = DAC` option, which turns `DAC0` into a *programmable* comparator threshold.

32.18.1 Register path and init order

DAC0 is two registers plus the VREF select:

VREF_CTRLA	DAC0REFSEL[2:0]	– reference voltage (shared with AC0)
VREF_CTRLB	DAC0REFEN(0)	– optional force-enable (faster startup)
DAC0_CTRLA	ENABLE(0) OUTEN(6) RUNSTDBY(7)	
DAC0_DATA	8-bit value, right-aligned	

The datasheet (sec.31) gives the order: select the reference, enable the output, load an initial DATA, then set ENABLE. `src/ch20_adc/dac_output.S` holds PA6 at ~1.25 V:

```
.equ VREF_DAC0REFSEL_2V5,    0x02          /* DAC0REFSEL = 2.5 V */
.equ PORT_ISC_INPUT_DISABLE, 0x04

main:
    /* 1. shared DAC0/AC0 reference = 2.5 V */
    ldi r16, VREF_DAC0REFSEL_2V5
    sts VREF_CTRLA, r16

    /* 2. disable the digital input buffer on the output pin PA6 */
    ldi r16, PORT_ISC_INPUT_DISABLE
    sts PORTA_PIN6CTRL, r16

    /* 3. initial value: 128 -> 1.25 V at VREF = 2.5 V */
    ldi r16, 128
    sts DAC0_DATA, r16

    /* 4. enable DAC0, route output to PA6 */
    ldi r16, DAC_ENABLE_bm | DAC_OUTEN_bm
    sts DAC0_CTRLA, r16

loop:
    rjmp loop          /* DAC holds PA6; write DATA to change it */
```

The DAC0REFSEL voltages are 0.55, 1.1, 2.5, 4.34, 1.5 V for field values 0x0..0x4 — note the order is **not** monotonic (1.5 V is 0x4), so read the value from the name, not the position. The reference cannot exceed VDD; on a 3.3 V board the 4.34 V setting clips. Writing a new byte to DAC0_DATA updates the output continuously — sweeping it in a loop produces a staircase, the basis for software-generated waveforms (see the DDS chapter for the table-driven version of this idea).

A note on DAC0_DATA: it is a single 8-bit register holding the value right-aligned, so a plain STS of a byte is all it takes — unlike the larger AVR DACs whose 10-bit DATA is split across DATAH/DATAL and left-aligned.

32.19 Common Pitfalls

- Forgetting to disable the digital input buffer on the analog pin.
- Using a reference lower than the input signal and wondering why the result is always near 0x3FF.
- Reading RESH first and getting inconsistent 16-bit results.
- Sampling a high-impedance source with the default sample length.
- Switching channels in free-running mode and assuming the very next result is already from the new channel.
- Using IN/OUT for ADC registers. ADC0 is memory-mapped; use LDS/STS.
- Forgetting that PA0 is UPDI and choosing it as a casual analog input.

- Not setting `SAMPCAP` when using the internal 1.1 V reference or `VDD`. The hardware still converts, but accuracy is degraded.
 - Changing `DUTYCYC` in `ADC0_CALIB` without checking the ADC clock frequency. The reset value (25% duty) is correct for 10-bit operation. Clearing it without raising the clock above 1.5 MHz produces incorrect conversions.
 - Reading `SIGROW_TEMPSENSE0` and `SIGROW_TEMPSENSE1` from the wrong addresses. The Signature Row base is `0x1100`; `TEMPSENSE0` is at `0x1120`, `TEMPSENSE1` at `0x1121`. Reading the wrong bytes produces a plausible-looking but wrong temperature.
 - Forgetting to restore `r1` to zero after `MUL` in the temperature conversion routine. GCC and many library routines depend on `r1 = 0`.
 - (AC) Reading `STATE` (bit 4) when you meant the latched flag `CMP` (bit 0), or vice versa. `STATE` is the live level; `CMP` is the edge flag you clear in an ISR.
 - (AC) Leaving the digital input buffer enabled on the comparator input pin — the dithering analog level toggles it continuously and wastes power.
 - (AC/DAC) Expecting `DAC0REFSEL` to be in voltage order. It is not: 1.5 V is field value `0x4`, after 4.34 V (`0x3`). Read the value from the `_gc` name.
 - (DAC) Selecting a reference above `VDD` (e.g. 4.34 V on a 3.3 V board). The output clips at `VDD`; high `DATA` values flatten out instead of climbing.
 - (DAC) Forgetting `OUTEN`. Without it `DAC0` still converts and drives the AC/ADC internally, but nothing appears on `PA6`.
-

32.20 Summary

ADC bring-up is mostly sequencing:

1. Configure the analog pin (disable digital input buffer).
2. Select the reference; set `SAMPCAP` if reference ≥ 1 V.
3. Select a valid ADC clock (200 kHz to 1.5 MHz for 10-bit with a 1.1 V or higher reference).
4. Configure startup delay (`INITDLY`) and sample length (`SAMPLEN`).
5. Select the channel in `ADC0_MUXPOS`.
6. Enable `ADC0`.
7. Start a conversion (`STCONV`).
8. Wait for `RESRDY` (poll or ISR).
9. Read `RESL` first, then `RESH`.
10. Clear `RESRDY` (reading the result bytes clears it automatically).

The `ADC0_CALIB` register needs no attention for 10-bit operation: the reset value of `DUTYCYC` = 1 (25% duty cycle) is correct at ADC clocks up to 1.5 MHz.

Calibration on this ADC is a software concern. Reading the `GND` channel checks for offset errors. Reading `INTREF` with a `VDD` reference estimates supply voltage. The temperature sensor uses `SIGROW` factory values to convert a raw reading to Kelvin.

Once the basic polling path works, the more advanced features are extensions of the same model: ISR replaces polling, events replace software starts, accumulation replaces manual averaging, `ASDV` reduces periodic interference, and the window comparator replaces software threshold checks.

32.21 Exercises

1. Change the example to use `AIN5` on `PA5`. Which `PORTA_PINnCTRL` register and `MUXPOS` value change?
2. Use `VDD` as the ADC reference instead of the internal reference. What does the result mean if the sensor is also powered from `VDD`? Should `SAMPCAP` be set?

3. Increase `ADC0_SAMPCTRL` from 8 to 16. How does that change conversion time at 208 kHz ADC clock? At 416 kHz (DIV8)?
4. Convert the polling example into an interrupt-driven example. Write the ISR that stores `RESL` and `RESH` into two SRAM bytes, and the critical-section read in the main loop.
5. Add a 16-sample hardware accumulation (`ADC0_CTRLB = 0x04`). What is the raw range of the accumulated result? Write the four-shift sequence to compute the average in assembly.
6. Enable `ASDV` alongside 16-sample accumulation. What does it change about how each of the 16 samples is timed?
7. Read the `GND` internal channel (`MUXPOS = 0x1F`) five times and average the results. What result should an ideal ADC return? If the measured value is consistently above zero, what does that indicate?
8. Read the `INTREF` channel (`MUXPOS = 0x1D`) with `VDD` as the ADC reference. Using the formula $VDD = 1125300 / \text{result}$, compute `VDD` in millivolts. What is the result when `VDD = 3.3 V`? When `VDD = 5.0 V`?
9. Implement `adc_temp_kelvin` from the Temperature Measurement section. Read `SIGROW.TEMPSENSE0` and `SIGROW.TEMPSENSE1` with two `LDS` instructions and verify the addresses are `0x1120` and `0x1121`. What temperature does the device report at room temperature?
10. The `DUTYCYC` bit in `ADC0_CALIB` resets to 1. Read back the register after reset and confirm the value. What would happen if you cleared it while keeping the ADC clock at 208 kHz?
11. (AC) Modify `ac_compare.S` to compare `AINP0` (PA7) against the DAC0 output instead of the fixed 1.1 V reference (`MUXNEG = DAC = 0x3`). What other peripheral must you bring up first, and what `DAC0_DATA` value sets the threshold to 1.0 V when the DAC reference is 2.5 V?
12. (AC) Add rising-edge interrupts to the comparator: set `INTMODE` for a rising edge, enable `AC_CMP_bm` in `AC0_INTCTRL`, and write the ISR. How does the ISR clear the flag, and which register and bit does it write?
13. (AC) Enable 25 mV hysteresis (`HYSMODE = 0x2`). For a triangle-wave input crossing the threshold slowly, how many millivolts must the input fall below the threshold before `STATE` returns to 0?
14. (DAC) Write a loop that walks `DAC0_DATA` from 0 to 255 and back, producing a triangle wave on PA6. At a 3.333 MHz CPU clock, roughly what output frequency results if each step is a single STS with no added delay?
15. (DAC) Compute the `DAC0_DATA` values for 0.5 V, 1.0 V, and 2.0 V outputs when the reference is 2.5 V. What is the smallest voltage step (1 LSB) at this reference?

Chapter 33

Integer Filters for ADC and Control

The ADC converts a voltage to a number, but that number is rarely clean. The internal 1.1 V reference has a tolerance of roughly $\pm 2\%$ (rising to about $\pm 5\%$ over the full temperature and voltage range). The sample-and-hold capacitor leaks charge during long conversions. Digital switching on the same supply rail couples noise into the analog circuitry. A sensor with high source impedance cannot charge the S/H capacitor fully in the default sample time. Even with a perfect hardware setup, thermal noise and quantisation noise are always present.

Software filters reduce that noise before the application acts on the result. They are also the building blocks of feedback control: a proportional-integral-derivative (PID) controller is three filter operations — a scaled identity, an integrator, and a differentiated low-pass — combined with a subtractor.

This chapter covers:

1. **Noise sources** on the ATtiny3217 ADC and where each filter type helps.
2. **Exponential Moving Average (EMA)** — the most important tool, with 16-bit and 24-bit fixed-point implementations.
3. **Box filter** — a running-window average using a circular buffer.
4. **Median filter** — a 3-sample compare-and-swap network for spike removal.
5. **Cascaded EMA** — two stages in series for a steeper low-pass rolloff.
6. **Integer PID structure** — proportional, integral, and derivative terms in fixed-point arithmetic, with anti-windup clamping.
7. **Filter selection** — a practical guide to choosing the right tool.
8. **Common pitfalls** — accumulator sizing, initialisation, and windup.

All filters in this chapter operate on 10-bit unsigned ADC results (0–1023) and use integer arithmetic with no floating-point operations.

33.1 Noise Sources and Filter Choices

Noise source	Character	Best filter
Thermal / quantisation	Broadband white	EMA or box filter
Supply-coupled switching	Periodic spike	Median filter
Ground bounce	Periodic spike	Median filter
Mechanical bounce	Sustained burst	EMA with large k
High-source-impedance	Slow settling	Add SAMPCTRL time; EMA helps
ADC INL / DNL	Systematic	Cannot be removed by software

The first step is always hardware: bypass capacitors, short ground paths, low-impedance sources, and correct SAMPCTRL settings from Chapter 30. Software filters cannot fix systematic hardware errors, but they substantially reduce random noise.

33.2 Exponential Moving Average

The exponential moving average (EMA), also called a first-order IIR low-pass filter, is the single most useful filter for ADC conditioning on a small MCU. It needs only one extra word of SRAM, runs in under 40 cycles, and has well-understood frequency-domain behaviour.

33.2.1 Algorithm

The update rule is:

$$y[n] = y[n-1] + (x[n] - y[n-1]) / 2^k$$

Where:

$x[n]$ new ADC sample (0..1023)
 $y[n]$ filtered output (0..1023)
 k shift parameter: larger k = more smoothing, slower response

The term $(x[n] - y[n-1]) / 2^k$ is an error-weighted correction. When the input suddenly changes, the output moves toward it in exponential steps of size $1/2^k$ per sample. Larger k makes the steps smaller and the filter slower.

33.2.2 Fixed-Point Implementation

The division by 2^k is an arithmetic right shift, which is exact and free on AVR. The challenge is representing $y[n]$ with enough precision to accumulate small corrections without truncating them away.

The solution is a **scaled accumulator**: keep the accumulator at $2^k \times y[n]$ rather than $y[n]$ itself. Each update adds the new sample and subtracts the shifted accumulator:

$$\text{accum}[n] = \text{accum}[n-1] - (\text{accum}[n-1] \gg k) + x[n]$$

$$\text{output} = \text{accum}[n] \gg k$$

This is equivalent to the definition because $\text{accum} \approx 2^k \times y$, so $\text{accum} \gg k \approx y$, and $\text{accum} - (\text{accum} \gg k) + x = \text{accum} \times (1 - 1/2^k) + x$.

33.2.3 Accumulator Width

The scaled accumulator converges to $2^k \times x$ at steady state. For a 10-bit ADC input (maximum 1023) and shift parameter k , the maximum accumulator value is:

$$\text{accum_max} = 1023 \times 2^k$$

k	accum_max	Bits required	Register choice
1	2046	11	16-bit (2 bytes)
2	4092	12	16-bit
3	8184	13	16-bit
4	16368	14	16-bit
5	32736	15	16-bit
6	65472	16	16-bit – maximum for 16-bit accumulator
7	130944	17	24-bit (3 bytes)
8	261888	18	24-bit

10	1047552	20	24-bit
14	16760832	24	24-bit – maximum for 24-bit accumulator

A 16-bit accumulator supports k up to 6. A 24-bit accumulator supports k up to 14. For most ADC conditioning, $k = 4$ (16-bit) or $k = 8$ (24-bit) are the practical starting points.

33.2.4 Cutoff Frequency

The EMA is a first-order low-pass filter. Its -3 dB cutoff frequency is:

$$f_c \approx f_s / (2^k \times 2\pi)$$

Where f_s is the sample rate (how often the filter is updated). At different sample rates:

k	f_c at $f_s=1000$ Hz	f_c at $f_s=100$ Hz	f_c at $f_s=7992$ Hz
-----	------------------------	-----------------------	------------------------

1	79.6 Hz	7.96 Hz	636 Hz
2	39.8 Hz	3.98 Hz	318 Hz
3	19.9 Hz	1.99 Hz	159 Hz
4	9.95 Hz	0.995 Hz	79.5 Hz
6	2.49 Hz	0.249 Hz	19.9 Hz
8	0.622 Hz	0.0622 Hz	4.97 Hz
10	0.155 Hz	0.0155 Hz	1.24 Hz

Approximate formula derivation: the EMA transfer function is $H(z) = \alpha / (1 - (1-\alpha)z^{-1})$ where $\alpha = 1/2^k$. For small α , the -3 dB radian frequency is $\omega_c \approx \alpha$, so $f_c \approx f_s \times \alpha / (2\pi) = f_s / (2^k \times 2\pi)$. The approximation is within a few percent for $k \geq 4$ (about 6% at $k = 3$).

33.2.5 Assembly: 16-bit Accumulator ($k = 4$)

```
.section .bss
ema_accum: .byte 0          /* 16-bit scaled accumulator, low byte */
           .byte 0          /* high byte – addressed as ema_accum+1 */

/* ema16_k4 – EMA low-pass, alpha = 1/16 (k = 4), 16-bit accumulator
 *
 *  $f_c \approx f_s / 101$  (e.g., 9.9 Hz at 1000 Hz sample rate)
 * Valid for 10-bit ADC input – accum_max = 16368 < 65536
 *
 * Entry: R25:R24 = new ADC sample (0..1023, unsigned)
 * Exit:  R25:R24 = filtered output (0..1023, unsigned)
 * Clobbers: R16, R17, R18, R19
 */
ema16_k4:
    /* Load scaled accumulator from SRAM */
    lds    r18, ema_accum    /* accum low byte */
    lds    r19, ema_accum+1  /* accum high byte */

    /* shift = accum >> 4 (copy, then shift right 4 times) */
    movw   r16, r18          /* r17:r16 = r19:r18 (copy of accum) */
    lsr    r17
    ror    r16
    lsr    r17
    ror    r16
    lsr    r17
    ror    r16
    lsr    r17
    ror    r16              /* r17:r16 = accum >> 4 */
```

```

/* accum = accum - shift + sample */
sub  r18, r16          /* accum_lo -= shift_lo (may borrow) */
sbc  r19, r17          /* accum_hi -= shift_hi - borrow */
add  r18, r24          /* accum_lo += sample_lo */
adc  r19, r25          /* accum_hi += sample_hi + carry */

/* Store updated accumulator */
sts  ema_accum, r18
sts  ema_accum+1, r19

/* Output = accum >> 4 */
movw r24, r18          /* r25:r24 = r19:r18 */
lsr  r25
ror  r24
lsr  r25
ror  r24
lsr  r25
ror  r24
lsr  r25
ror  r24          /* r25:r24 = accum >> 4 = filtered output */
ret

```

Cycle count: 2 LDS (6) + MOVW (1) + 4×(LSR+ROR) (8) + 2×SUB/SBC + 2×ADD/ADC (4) + 2 STS (4) + MOVW (1) + 4×(LSR+ROR) (8) + RET (4) = **36 cycles** plus 2 for the RCALL (AVRxt timing).

At 1 kHz sample rate: 38 of the 3333 cycles per sample period = 1.1% CPU.

The shift uses logical right shift (LSR/ROR) because the accumulator is unsigned. The signed arithmetic right shift (ASR/ROR) is used for signed values; here the accumulator cannot go negative, so LSR is correct.

33.2.6 Initialisation

The accumulator must be zeroed before the first sample, or pre-loaded with a known value. In a -nostartfiles build:

```

reset_handler:
...
clr  r16
sts  ema_accum, r16
sts  ema_accum+1, r16

```

A zero-initialised accumulator makes the filter converge from 0 toward the true signal, which can cause a startup transient. For a faster settling, pre-load with $\text{first_sample} \times 2^k$:

```

/* Pre-load EMA accumulator with first ADC reading × 16 (for k=4) */
rcall adc0_read          /* returns r25:r24 = 10-bit sample */
movw r18, r24          /* r19:r18 = sample */
ldi  r16, 0
ldi  r17, 0
/* multiply r19:r18 by 16 = shift left 4 */
lsl  r18
rol  r19
lsl  r18
rol  r19
lsl  r18
rol  r19
lsl  r18

```

```

rol    r19                                /* r19:r18 = sample × 16 */
sts    ema_accum, r18
sts    ema_accum+1, r19

```

Pre-loading eliminates the cold-start transient: the filter begins at the correct output rather than converging from zero.

33.2.7 Assembly: 24-bit Accumulator (k = 8)

For stronger filtering ($k = 8$, $f_c \approx f_s/1608$), the accumulator needs 24 bits.

```

.section .bss
ema_accum24: .byte 0      /* byte 0: lowest */
               .byte 0      /* byte 1: middle */
               .byte 0      /* byte 2: highest – ema_accum24+2 */

/* ema24_k8 – EMA low-pass, alpha = 1/256 (k = 8), 24-bit accumulator
 *
 * fc ≈ fs / 1608 (e.g., 0.62 Hz at 1000 Hz sample rate)
 * Valid for 10-bit ADC input – accum_max = 261888, fits in 24 bits (16777216)
 *
 * Entry: R25:R24 = new ADC sample (0..1023)
 * Exit:  R25:R24 = filtered output (0..1023)
 * Clobbers: R16, R17, R18, R19, R20, R21
 */
ema24_k8:
    /* Load 24-bit accumulator (byte 0 = least significant) */
    lds    r19, ema_accum24
    lds    r20, ema_accum24+1
    lds    r21, ema_accum24+2

    /* accum >> 8: byte N shifts down to byte N-1; byte 0 is discarded.
     * Shift value as three bytes: {0, b2, b1} = {0, r21, r20}.
     *
     * accum - (accum >> 8): subtract {0, b2, b1} from {b2, b1, b0}:
     *   byte0: b0 - b1
     *   byte1: b1 - b2 - borrow
     *   byte2: b2 - 0 - borrow
     */
    sub    r19, r20      /* byte0: b0 - b1 */
    sbc    r20, r21      /* byte1: b1 - b2 - C (r20 was b1, r21 = b2) */
    sbc    r21, r1       /* byte2: b2 - 0 - C (r1 = 0) */

    /* accum += sample (r25:r24 ≤ 1023, r25 ≤ 3) */
    add    r19, r24
    adc    r20, r25
    adc    r21, r1       /* propagate carry into high byte */

    /* Store updated accumulator */
    sts    ema_accum24, r19
    sts    ema_accum24+1, r20
    sts    ema_accum24+2, r21

    /* Output = accum >> 8 = top 16 bits {b2, b1} = {r21, r20}
     * r25:r24 = r21:r20 */
    movw   r24, r20      /* r24 = r20 (mid byte), r25 = r21 (high byte) */
    ret

```

The key insight in `ema24_k8` is that a right-shift by 8 on a 24-bit number reduces to moving

bytes down by one position. There is no bit-level shifting at all: the shift value is just the middle and high bytes with the low byte discarded. The `sub r19, r20 / sbc r20, r21 / sbc r21, r1` chain performs the subtraction in one pass without needing a temporary copy.

Cycle count: 3 LDS (9) + 3×(SUB/SBC) (3) + 3×(ADD/ADC) (3) + 3 STS (6) + MOVW (1) + RET (4) = **26 cycles** plus 2 for the RCALL (AVRxt timing).

The 24-bit version is actually *cheaper* than the 16-bit version because the shift-by-8 is free (byte movement).

33.3 Box Filter (N-Sample Moving Average)

A box filter (also called a sliding window average or rectangular FIR) averages the last N samples. It has a sharper rolloff than EMA at the cost of $N \times 2$ bytes of SRAM for the sample history.

33.3.1 Algorithm

```
sum = sum + new_sample - oldest_sample
buf[write_idx] = new_sample
write_idx = (write_idx + 1) mod N
```

```
output = sum / N
```

The circular buffer replaces the oldest sample with the new one and adjusts the running sum by the difference. Division by N is an arithmetic right shift when N is a power of two. The circular (ring) buffer itself — the wrapping write index, the power-of-two AND mask, and why those choices matter — is developed in full in the “Circular Buffer for High-Throughput RX” section of the USART chapter. Here the buffer holds 16-bit samples rather than received bytes, and there is a single owner (the sample loop), so the lock-free producer/consumer reasoning does not apply; the wrap-and-index mechanics are identical.

33.3.2 Implementation: N = 8 (M = 3 shifts)

For N = 8 and a 10-bit ADC:

```
sum_max = 8 × 1023 = 8184 < 65535 → 16-bit sum
buf:      8 × 2 bytes = 16 bytes of SRAM
idx:      1 byte, range 0..7
```

```
.section .bss
box_buf: .fill 16, 1, 0      /* 8 × 2-byte sample slots */
box_sum: .byte 0             /* running sum, low byte */
        .byte 0             /* running sum, high byte */
box_idx: .byte 0             /* circular buffer write index, 0..7 */

/* box8_update – 8-sample box filter (sliding window average)
 *
 * Entry: R25:R24 = new ADC sample (0..1023)
 * Exit:  R25:R24 = filtered output (0..1023)
 * Clobbers: R16, R17, R18, R19, R30, R31 (Z)
 */
box8_update:
    /* Load write index */
    lds    r16, box_idx      /* r16 = idx (0..7) */

    /* Point Z at box_buf[idx]: byte offset = idx × 2 */
    ldi    ZL, lo8(box_buf)
```

```

ldi    ZH, hi8(box_buf)
mov     r17, r16
lsl     r17                      /* r17 = idx × 2 */
add     ZL, r17
adc     ZH, r1                    /* Z = &box_buf[idx] (low byte of entry) */

/* Load oldest sample from that slot */
ld      r18, Z+                  /* r18 = old_lo, Z advances to high byte */
ld      r19, Z                    /* r19 = old_hi */

/* Store new sample in the same slot */
st      Z, r25                  /* write new_hi at high-byte position */
st      -Z, r24                  /* pre-decrement Z; write new_lo at low-byte */

/* Load running sum (Z is now free – LDS uses direct addressing, not Z) */
lds     r30, box_sum
lds     r31, box_sum+1

/* sum += new_sample - old_sample */
add     r30, r24
adc     r31, r25
sub     r30, r18
sbc     r31, r19

/* Store updated sum */
sts     box_sum, r30
sts     box_sum+1, r31

/* Advance index: idx = (idx + 1) & 7 */
inc     r16
andi    r16, 0x07
sts     box_idx, r16

/* Output = sum >> 3 */
movw    r24, r30                /* r25:r24 = r31:r30 */
lsr     r25
ror     r24
lsr     r25
ror     r24
lsr     r25
ror     r24                    /* r25:r24 = sum / 8 */
ret

```

The LD r18, Z+ / LD r19, Z pair reads the little-endian 16-bit sample from the circular buffer without modifying the entry. After both reads, Z points at the high-byte slot. ST Z, r25 writes the new high byte; ST -Z, r24 pre-decrements Z and writes the new low byte. After those two stores, the entry in SRAM holds the new sample in the correct little-endian layout.

LDS and STS use a direct 16-bit address encoded in the instruction word. They do not use or disturb the Z register. Loading the sum into r30:r31 does not clobber Z even though r30 and r31 are the Z register components, because after the buffer access, Z is no longer needed.

33.3.3 Box Filter Initialisation

Zero the sum, buffer, and index before use:

```

box_init:
    clr    r16
    sts    box_sum, r16
    sts    box_sum+1, r16
    sts    box_idx, r16
    /* Zero the 16-byte buffer with a loop */
    ldi    ZL, lo8(box_buf)
    ldi    ZH, hi8(box_buf)
    ldi    r17, 16
.Lbox_zero:
    st     Z+, r16
    dec    r17
    brne   .Lbox_zero
    ret

```

Like the EMA, zero initialisation causes a startup transient. Pre-fill the buffer with the first sample to avoid it.

33.3.4 Frequency Response

The box filter has a sinc-like frequency response. Its first null is at $f_c = f_s / N$. For $N = 8$ and $f_s = 1000$ Hz: first null at 125 Hz. The filter attenuates noise broadly up to that frequency, but has significant side lobes above it.

Box filters have poor sidelobe rejection compared to EMA. They are better suited to removing broadband noise when the sample count can be tuned to place the first null at the dominant interference frequency.

33.4 Median Filter (3-Sample)

A median filter returns the middle value of the last N samples, discarding outliers entirely. A 3-sample median removes isolated spikes with no smoothing of the underlying signal: valid consecutive readings pass through unchanged, but a single anomalous reading (EMI glitch, mechanical event) is removed.

33.4.1 Algorithm: Sorting Network

Sort three values a, b, c using a 3-swap network. After the network, the middle value b is the median. The compare-and-swap operation $\text{COMPARE_SWAP}(x, y)$ swaps x and y if $x > y$, ensuring $x \leq y$ afterwards:

```

COMPARE_SWAP(a, b)    →  $a \leq b$ 
COMPARE_SWAP(b, c)    →  $b \leq c$ 
COMPARE_SWAP(a, b)    →  $a \leq b$ 
After:  $a \leq b \leq c$ . Median =  $b$ .

```

Verification with $\{7, 3, 5\}$: 1. $\text{COMPARE_SWAP}(7, 3) \rightarrow \text{swap} \rightarrow \{3, 7, 5\}$ 2. $\text{COMPARE_SWAP}(7, 5) \rightarrow \text{swap} \rightarrow \{3, 5, 7\}$ 3. $\text{COMPARE_SWAP}(3, 5) \rightarrow \text{no swap} \rightarrow \{3, 5, 7\}$ Median = 5. ✓

33.4.2 SRAM Layout

The 3-sample median needs the two oldest samples in SRAM:

```

.section .bss
med_hist: .byte 0, 0, 0, 0 /* two 16-bit samples: oldest at +0, prev at +2 */

```


33.4.3 Assembly Implementation

```

/* med3_update - 3-sample median filter using a compare-and-swap sorting network
 *
 * Entry: R25:R24 = newest ADC sample (0..1023, unsigned)
 * Exit:  R25:R24 = median of last 3 samples (0..1023)
 * Clobbers: R18, R19, R20, R21, R22, R23
 */
med3_update:
/* Load history: a = oldest (med_hist), b = previous (med_hist+2) */
lds  r22, med_hist          /* a_lo
lds  r23, med_hist+1        /* a_hi
lds  r20, med_hist+2        /* b_lo
lds  r21, med_hist+3        /* b_hi

/* Shift history: oldest = prev, prev = new */
sts  med_hist, r20
sts  med_hist+1, r21        /* oldest ← prev */
sts  med_hist+2, r24
sts  med_hist+3, r25        /* prev ← new

/* c = new sample in r25:r24 */

/* — Sorting network ————— */

/* COMPARE_SWAP(a, b): if a >= b, swap(a, b)
 *
 * CP/CPC: compare a - b. Carry set (BRL0) iff a < b.
 * We want to skip the swap when a < b; swap otherwise (a >= b).
 * Swapping equal values is harmless and avoids an extra branch.
 */

/* Step 1: COMPARE_SWAP(a, b) → ensures a ≤ b afterwards */
cp   r22, r20
cpc  r23, r21              /* compare a (r23:r22) with b (r21:r20)
brlo .Lmed3_step2         /* a < b: already ordered, skip swap
movw r18, r22              /* temp = a
movw r22, r20              /* a = b
movw r20, r18              /* b = temp

.Lmed3_step2:
/* Step 2: COMPARE_SWAP(b, c) → ensures b ≤ c afterwards */
cp   r20, r24
cpc  r21, r25              /* compare b (r21:r20) with c (r25:r24)
brlo .Lmed3_step3         /* temp = b
movw r18, r20              /* b = c
movw r20, r24              /* c = temp (c not needed after this)

.Lmed3_step3:
/* Step 3: COMPARE_SWAP(a, b) → restores a ≤ b after step 2 may have changed b */
cp   r22, r20
cpc  r23, r21
brlo .Lmed3_done
movw r18, r22              /* temp = a (result discarded)
movw r22, r20              /* a = b (result discarded)
movw r20, r18              /* b = old a → median

```

```
.Lmed3_done:
    /* Median is in r21:r20 (b after sorting) */
    movw r24, r20          /* r25:r24 = median output */
    ret
```

MOVW Rd, Rr requires both Rd and Rr to be even-numbered registers. All register pairs used here (r18, r20, r22, r24) satisfy that constraint.

CP Rd, Rr; CPC Rd, Rr implements a 16-bit unsigned comparison. After CP r22, r20; CPC r23, r21, the carry flag is set if and only if $a < b$ (unsigned 16-bit). BRLO (branch if lower = branch if carry set) therefore branches when $a < b$.

The median filter adds 4 bytes of SRAM. It does not smooth the signal at all: valid readings pass through with zero delay. Use it as a pre-stage before EMA when the sensor produces occasional large spikes.

33.5 Cascaded EMA (Second-Order Low-Pass)

Running the output of one EMA through a second EMA produces a second-order low-pass filter with a steeper rolloff: the response falls at approximately 40 dB/decade above the cutoff rather than the 20 dB/decade of a single stage.

33.5.1 SRAM Layout

Two independent accumulators, one per stage. This example uses $k = 5$ for each stage. Cascading two identical single-pole stages puts the combined -3 dB point at about $0.64 \times$ a single stage's cutoff ($\approx$ the cutoff of a single-stage EMA with $k \approx 5.6$), but with a steeper second-order rolloff:

```
.section .bss
ema2_accum0: .byte 0, 0      /* Stage 1 accumulator (k=5, 16-bit) */
ema2_accum1: .byte 0, 0      /* Stage 2 accumulator (k=5, 16-bit) */
```

33.5.2 Implementation

```
/* ema2_k5_update – two cascaded EMA stages, k=5 each
 *
 * Second-order (40 dB/decade) rolloff; combined -3 dB cutoff  $\approx$  single-stage  $k \approx 5.6$ 
 *  $f_c \approx f_s / (2^5 \times 2\pi)$  per stage; cascaded  $f_c$  is lower ( $\approx 0.64 \times$  per-stage  $f_c$ )
 *
 * Entry: R25:R24 = new ADC sample (0..1023)
 * Exit:  R25:R24 = filtered output (0..1023)
 * Clobbers: R16, R17, R18, R19
 */
ema2_k5_update:
    /* — Stage 1: accum0 ————— */
    lds  r18, ema2_accum0
    lds  r19, ema2_accum0+1

    movw r16, r18
    lsr  r17          /* shift accum0 >> 5 (5 times) */
    ror  r16
    lsr  r17
    ror  r16
    lsr  r17
    ror  r16
```

```

lsr    r17
ror    r16
lsr    r17
ror    r16

sub    r18, r16
sbc    r19, r17
add    r18, r24
adc    r19, r25

sts    ema2_accum0, r18
sts    ema2_accum0+1, r19

/* Stage 1 output in r25:r24 = accum0 >> 5 */
movw   r24, r18
lsr    r25
ror    r24
lsr    r25
ror    r24
lsr    r25
ror    r24
lsr    r25
ror    r24
lsr    r25
ror    r24                      /* r25:r24 = stage-1 output */

/* — Stage 2: accum1 ————— */
lds    r18, ema2_accum1
lds    r19, ema2_accum1+1

movw   r16, r18
lsr    r17
ror    r16
lsr    r17
ror    r16
lsr    r17
ror    r16
lsr    r17
ror    r16
lsr    r17
ror    r16

sub    r18, r16
sbc    r19, r17
add    r18, r24
adc    r19, r25

sts    ema2_accum1, r18
sts    ema2_accum1+1, r19

movw   r24, r18
lsr    r25
ror    r24
lsr    r25
ror    r24
lsr    r25
ror    r24

```

```

lsr    r25
ror    r24
lsr    r25
ror    r24          /* r25:r24 = final filtered output */
ret

```

The cascaded EMA doubles the SRAM cost (4 bytes instead of 2) and roughly doubles the cycle cost (≈ 72 cycles instead of 36). The reward is a much steeper roll-off: noise above the cutoff is attenuated more aggressively.

Both stages must be initialised to zero (or pre-filled) before use.

33.6 Integer PID Structure

A PID controller computes an output u from a setpoint sp and measured value y :

$error = sp - y$

```

P_term = Kp × error
I_term = I_prev + Ki × error      (accumulator – may need anti-windup)
D_term = Kd × (y_prev - y)      (using output, not error, for the D term)

```

$u = P_term + I_term + D_term$

Using the output rather than the error for the derivative term prevents **derivative kick**: when the setpoint changes, the error jumps instantly and the derivative of the error is infinite. The output changes smoothly, so its derivative is well-behaved.

On AVR, each coefficient is stored as a fixed-point constant (see Chapter 12 for the multiply mechanics). This chapter uses a simplified representation where gains are powers of two so that multiplication reduces to a shift. For arbitrary gains, use the MUL/FMUL instructions and the Q-format scaling rules from Chapter 12.

33.6.1 Fixed-Point Representation

Use signed 16-bit values throughout. With a 10-bit ADC:

```

error range: -1023 to +1023 (signed, fits in 16 bits)
P_term:      -1023×Kp to +1023×Kp
I_accum:     must hold the running sum of many error samples

```

Typical gain scales:

```

Kp_shift = 0 (Kp = 1): P = error          range ±1023
Kp_shift = 2 (Kp = 4): P = error << 2    range ±4092
Ki_shift = k (Ki = 1/2^k): I += error >> k integrates slowly
Kd_shift = 0 (Kd = 1): D = (y_prev - y)    range ±1023

```

33.6.2 SRAM Layout

```

.section .bss
pid_I_accum: .byte 0, 0, 0, 0 /* 32-bit signed integral accumulator */
pid_y_prev:  .byte 0, 0      /* previous output sample (for D term) */
pid_I_min:   .byte 0, 0, 0, 0 /* anti-windup lower clamp (32-bit signed) */
pid_I_max:   .byte 0, 0, 0, 0 /* anti-windup upper clamp (32-bit signed) */
pid_out_min: .byte 0, 0      /* output clamp lower bound (signed 16-bit) */
pid_out_max: .byte 0, 0      /* output clamp upper bound (signed 16-bit) */

```

The integral accumulator is 32-bit because it sums many error samples. At $K_i = 1/16$ ($K_i \text{ shift} = 4$) and a sustained error of 512 for 60 seconds at 1 kHz: total integral = $60000 \times 512 / 16 = 1,920,000$, which needs 21 bits. A 32-bit accumulator is safe.

33.6.3 Proportional Term

```
/* P = error << Kp_shift (signed left shift)
 * Entry: R25:R24 = error (signed 16-bit, range -1023..+1023)
 * Exit:  R25:R24 = P_term (signed 16-bit, may overflow if Kp too large)
 * Kp_shift = 2 in this example (Kp = 4)
 */
pid_compute_P:
    lsl    r24
    rol    r25
    lsl    r24
    rol    r25                /* r25:r24 = error × 4 (signed) */
    ret
```

LSL/ROL propagates the sign bit correctly for signed left shift as long as no overflow occurs. Check that $|\text{error}| \times K_p < 32767$ before choosing $K_p \text{ shift}$.

33.6.4 Integral Term with Anti-Windup

```
/* I_accum += error >> Ki_shift (signed)
 * Then clamp I_accum to [pid_I_min, pid_I_max].
 *
 * Entry: R25:R24 = error (signed 16-bit)
 * Exit:  R25:R24 = I_term (low 16 bits of I_accum, scaled for output)
 * Clobbers: R16..R23
 * Ki_shift = 4 in this example (Ki = 1/16)
 */
pid_compute_I:
    /* Sign-extend error to 32 bits: r27:r26:r25:r24 = error */
    movw   r26, r24                /* r27:r26 = r25:r24 (copy error to r27:r26) */
    clr     r24                    /* r25:r24 = 0 for now */
    clr     r25
    sbrc    r27, 7                 /* check sign of original error (in r27 now) */
    ldi     r25, 0xFF              /* if negative, sign-extend high bytes */
    mov     r24, r25              /* r25:r24 = 0x0000 or 0xFFFF */

    /* Restore signed 32-bit value: r27:r26 was the original error */
    /* r27:r26:r25:r24 layout: byte0=r24(low), byte1=r25, byte2=r26, byte3=r27 */
    /* Rearrange to r23:r22:r21:r20 = 32-bit sign-extended error (low-first) */
    movw    r20, r26                /* r21:r20 = error_lo word */
    movw    r22, r24                /* r23:r22 = sign extension word (0x0000/0xFFFF) */

    /* Shift right by Ki_shift = 4 (signed: ASR high, ROR lower bytes) */
    asr     r23
    ror     r22
    ror     r21
    ror     r20
    asr     r23
    ror     r22
    ror     r21
    ror     r20
    asr     r23
    ror     r22
```

```

ror    r21
ror    r20
asr    r23
ror    r22
ror    r21
ror    r20                      /* r23:r22:r21:r20 = (signed error) >> 4 */

/* I_accum += error_step (32-bit signed addition) */
lds    r24, pid_I_accum
lds    r25, pid_I_accum+1
lds    r26, pid_I_accum+2
lds    r27, pid_I_accum+3

add    r24, r20
adc    r25, r21
adc    r26, r22
adc    r27, r23

sts    pid_I_accum,    r24
sts    pid_I_accum+1, r25
sts    pid_I_accum+2, r26
sts    pid_I_accum+3, r27

/* Anti-windup: the caller is responsible for clamping I_accum to
 * [pid_I_min, pid_I_max] and storing the clamped value back.
 * See the anti-windup section below.
 *
 * Return I_term = bytes 1 and 2 of I_accum (I_accum >> 8, low 16 bits).
 * r25 = byte1, r26 = byte2. MOVW requires even source; use two MOVs. */
mov    r24, r25                /* r24 = accum byte 1 */
mov    r25, r26                /* r25 = accum byte 2 */
ret

```

33.6.5 Derivative Term

```

/* D = Kd_shift * (y_prev - y) (using output measurement for smooth D)
 *
 * Entry:  R25:R24 = current measured output y (signed or unsigned 16-bit)
 * Exit:   R25:R24 = D_term
 * Clobbers: R16, R17, R18, R19
 * Kd_shift = 1 (Kd = 2)
 */
pid_compute_D:
/* Load y_prev */
lds    r18, pid_y_prev
lds    r19, pid_y_prev+1

/* D_raw = y_prev - y (previous minus current = negative rate of change of y) */
sub    r18, r24
sbc    r19, r25                /* r19:r18 = y_prev - y */

/* Store current y as new y_prev */
sts    pid_y_prev,    r24
sts    pid_y_prev+1, r25

/* D_term = D_raw × Kd: left shift by Kd_shift = 1 */
lsl    r18

```

```

rol    r19                                /* r19:r18 = D_raw × 2 */
movw   r24, r18                          /* return D_term in r25:r24 */
ret

```

In a real system, the derivative is usually filtered with an EMA before output because the discrete first-difference amplifies high-frequency noise. Call `ema16_k4` (or another suitable filter) on the result of `pid_compute_D` before adding it to the controller output.

33.6.6 Anti-Windup

Integral windup occurs when the controller output is saturated (e.g., the actuator is at its limit) but the integrator continues to accumulate error. When the error finally reverses, the integrator must “unwind” before the output can change direction, causing overshoot and slow recovery.

The simplest anti-windup strategy clamps the integral accumulator to a range:

```

/* pid_clamp_I – clamp pid_I_accum to [pid_I_min, pid_I_max]
 *
 * All values are 32-bit signed.
 * Clobbers: R16..R23
 */
pid_clamp_I:
/* Load current accumulator */
lds    r16, pid_I_accum
lds    r17, pid_I_accum+1
lds    r18, pid_I_accum+2
lds    r19, pid_I_accum+3 /* r19:r18:r17:r16 = I_accum (lo first) */

/* Compare with pid_I_min (load and compare signed 32-bit) */
lds    r20, pid_I_min
lds    r21, pid_I_min+1
lds    r22, pid_I_min+2
lds    r23, pid_I_min+3

/* Signed comparison: I_accum < I_min? (check sign of I_accum - I_min) */
cp     r16, r20
cpc    r17, r21
cpc    r18, r22
cpc    r19, r23 /* sets flags for I_accum - I_min */
brge   .Lclamp_check_max /* if I_accum >= I_min: check upper bound */

/* I_accum < I_min: clamp to I_min */
sts    pid_I_accum, r20
sts    pid_I_accum+1, r21
sts    pid_I_accum+2, r22
sts    pid_I_accum+3, r23
ret

.Lclamp_check_max:
/* Compare with pid_I_max */
lds    r20, pid_I_max
lds    r21, pid_I_max+1
lds    r22, pid_I_max+2
lds    r23, pid_I_max+3

cp     r16, r20
cpc    r17, r21

```

```

cpc    r18, r22
cpc    r19, r23          /* sets flags for I_accum - I_max          */
brlt   .Lclamp_done      /* if I_accum < I_max: within bounds      */

/* I_accum >= I_max: clamp to I_max */
sts    pid_I_accum, r20
sts    pid_I_accum+1, r21
sts    pid_I_accum+2, r22
sts    pid_I_accum+3, r23

.Lclamp_done:
ret

```

BRGE and BRLT are signed comparisons: they branch based on the combination of the N (negative) and V (overflow) flags after a CP/CPC sequence. The 32-bit signed comparison CP + CPC + CPC + CPC works correctly because each CPC propagates the carry and updates the V flag.

The clamp bounds in SRAM (pid_I_min, pid_I_max, pid_out_min, pid_out_max) should be set during initialisation based on the actuator range and the expected steady-state error. This example clamps the integral to ± 16384 and the output to ± 1023 (a stand-in for a 10-bit actuator command; pick values that match your real actuator):

```

/* Set I_min/I_max =  $\mp 16384$  (0xFFFFC000 / 0x00004000)
 * and out_min/out_max =  $\mp 1023$  (0xFC01 / 0x03FF, signed 16-bit) */
pid_init_clamps:
ldi    r16, 0x00
ldi    r17, 0xC0
ldi    r18, 0xFF
ldi    r19, 0xFF
sts    pid_I_min,    r16
sts    pid_I_min+1,  r17
sts    pid_I_min+2,  r18
sts    pid_I_min+3,  r19

ldi    r16, 0x00
ldi    r17, 0x40
ldi    r18, 0x00
ldi    r19, 0x00
sts    pid_I_max,    r16
sts    pid_I_max+1,  r17
sts    pid_I_max+2,  r18
sts    pid_I_max+3,  r19

/* out_min = -1023 = 0xFC01 */
ldi    r16, 0x01
ldi    r17, 0xFC
sts    pid_out_min,  r16
sts    pid_out_min+1, r17

/* out_max = +1023 = 0x03FF */
ldi    r16, 0xFF
ldi    r17, 0x03
sts    pid_out_max,  r16
sts    pid_out_max+1, r17
ret

```


33.6.7 Assembling the Full PID Output

```

/* pid_update – compute full PID output
 *
 * Entry: R25:R24 = setpoint (unsigned 16-bit, same scale as ADC reading)
 *        R23:R22 = measured value y (unsigned 16-bit)
 * Exit:  R25:R24 = controller output u (signed 16-bit)
 * Clobbers: many registers – this is a top-level routine
 */
pid_update:
/* Compute signed error = setpoint - y.
 * The P, I, and D helper routines between clobber r16..r27, so every value
 * that must survive across them is held on the stack, not in registers. */
sub    r24, r22
sbc    r25, r23          /* r25:r24 = sp - y (signed error) */
movw   r18, r24          /* r19:r18 = error (survives pid_compute_P) */

push   r22               /* save current y (for the D term) */
push   r23

/* P term (input: error in r25:r24) */
rcall  pid_compute_P     /* r25:r24 = P_term (clobbers only r24:r25) */
push   r24               /* save P_term on the stack */
push   r25

/* I term (input: error) */
movw   r24, r18          /* r25:r24 = error */
rcall  pid_compute_I     /* r25:r24 = I_term */
rcall  pid_clamp_I       /* clamp accumulator (leaves r25:r24 untouched) */
push   r24               /* save I_term on the stack */
push   r25

/* Recover terms. Stack top→bottom holds: I_term, P_term, y */
pop    r25               /* I_term high */
pop    r24               /* I_term low → r25:r24 = I_term */
pop    r19               /* P_term high */
pop    r18               /* P_term low → r19:r18 = P_term */
add    r24, r18
adc    r25, r19          /* r25:r24 = I_term + P_term */

pop    r21               /* y high */
pop    r20               /* y low → r21:r20 = y */

/* D term (input: current y); preserve the running sum across the call */
push   r24               /* save (I+P) low */
push   r25               /* save (I+P) high */
movw   r24, r20          /* r25:r24 = current y */
rcall  pid_compute_D     /* r25:r24 = D_term */

/* For a low-noise D term, filter it before summing, e.g.:
 * rcall ema16_k4 (optional: smooth the derivative) */

pop    r19               /* (I+P) high */
pop    r18               /* (I+P) low → r19:r18 = I_term + P_term */
add    r24, r18
adc    r25, r19          /* r25:r24 = P + I + D = raw controller output */

rcall  pid_clamp_out     /* clamp u to the actuator output range */

```

```
ret
```

33.6.8 Output Clamping

The integral clamp (`pid_clamp_I`) bounds one term; the *output* clamp bounds the sum. Real actuators have a finite range — a PWM duty cycle between 0 and TOP, a DAC code, a current limit — and the controller must never command beyond it. Clamping the final `u` is what makes the integral clamp meaningful: the two together implement saturation with anti-windup. Without the output clamp, a saturated command is silently truncated by the actuator's own limits and the integrator has no consistent bound to work against.

`pid_clamp_out` clamps the signed 16-bit output to `[pid_out_min, pid_out_max]`. It is the 16-bit analogue of `pid_clamp_I`: a signed CP/CPC compare against each bound, then a conditional MOVW to the bound when the output is outside it.

```
/* pid_clamp_out - clamp signed 16-bit output u to [pid_out_min, pid_out_max]
 *
 * Entry: R25:R24 = u (raw controller output, signed 16-bit)
 * Exit:  R25:R24 = u clamped to [pid_out_min, pid_out_max]
 * Clobbers: R18, R19
 */
pid_clamp_out:
    /* u < out_min? */
    lds    r18, pid_out_min
    lds    r19, pid_out_min+1
    cp     r24, r18
    cpc    r25, r19                /* flags for u - out_min */
    brge   .Lco_check_max         /* u >= out_min: check the upper bound */
    movw   r24, r18               /* u < out_min: clamp to out_min */
    ret

.Lco_check_max:
    /* u > out_max? */
    lds    r18, pid_out_max
    lds    r19, pid_out_max+1
    cp     r24, r18
    cpc    r25, r19                /* flags for u - out_max */
    brlt   .Lco_done              /* u < out_max: already within bounds */
    movw   r24, r18               /* u >= out_max: clamp to out_max */

.Lco_done:
    ret
```

As with the integral clamp, BRGE/BRLT make the comparison signed, so a negative output below `pid_out_min` is detected correctly rather than being treated as a large unsigned value. MOVW `r24, r18` copies the 16-bit bound into the output pair in one cycle.

In a real application, the PID structure is called from a timer ISR or a tick-driven scheduler task once per control sample period. The control period and gain values must be tuned to the specific plant (motor, heater, valve, etc.).

33.7 Filter Selection Guide

Requirement	Recommended filter
Broadband noise, low SRAM	EMA (k=4..6, 16-bit accum)
Broadband noise, strong filter	EMA (k=8, 24-bit accum) or cascaded EMA

Isolated spike removal	Median (3-sample) as pre-stage before EMA
Known noise frequency	Box filter (place null at noise frequency)
Smooth, fast-converging	EMA with large k; pre-load accumulator
Feedback control	EMA on sensor + PID with anti-windup
Derivative in control loop	First-difference, then EMA on D term

33.7.1 SRAM Cost Summary

Filter	SRAM bytes	Notes
EMA, 16-bit accum	2	$k \leq 6$
EMA, 24-bit accum	3	$k \leq 14$
Box, N=8	19	16-byte buffer + 2-byte sum + 1-byte idx
Box, N=16	35	
Median (3-sample)	4	2 × 16-bit history samples
Cascaded EMA ×2	4 or 6	two independent accumulators
PID	18	I_accum(4) + y_prev(2) + I clamps(8) + out clamps(4)

33.8 Common Pitfalls

33.8.1 Pitfall 1 — Using LSR/ROR on a Signed Value

The EMA accumulator is unsigned (all ADC results are non-negative). Use LSR/ROR for the right shift, not ASR/ROR. The ASR instruction preserves the sign bit and is correct only for signed arithmetic right shifts. The EMA `ema24_k8` example also uses `r1` (the zero register) in `sbc r21, r1` to propagate borrow without a literal operand. Never modify `r1` from outside the shift chain.

33.8.2 Pitfall 2 — Accumulator Overflow

A 16-bit accumulator overflows when $k > 6$ for a 10-bit ADC. Symptoms: output wraps around or jumps suddenly when the input is near the top of range.

Check before using: $k + 10 \leq \text{accumulator_width_bits}$ (the input needs about 10 bits, since $\log_2(1023) \approx 10$, and the scaling adds k bits). For $k=6$: $6 + 10 = 16 \rightarrow$ exactly at the limit. ✓ for $k \leq 6$; use 24-bit for $k=7+$.

33.8.3 Pitfall 3 — Not Initialising the Accumulator

In a `-nostartfiles` build, `.bss` is not zeroed automatically. If the EMA accumulator starts at a random SRAM value, the filter output will be incorrect for many samples. Always zero all filter state in `reset_handler`, or pre-load the accumulator with `first_sample × 2^k` for instant convergence.

33.8.4 Pitfall 4 — Updating the Filter from Two Places

If both the ISR and the main loop call the filter update, the accumulator can be read-modify-written non-atomically. Symptom: occasional wrong output readings. Fix: update the filter in one place only. If the ISR stores the raw ADC result, let the main loop call the filter update on that stored value.

33.8.5 Pitfall 5 — Box Filter Zero Not Re-Zeroed After Reset

The box filter has 16 bytes of buffer plus the running sum. If only the sum is zeroed on reset but the buffer retains stale SRAM values, the running sum accumulates garbage from the first N updates. Zero the entire buffer and the sum index before use.

33.8.6 Pitfall 6 — Integral Windup Without Clamping

Omitting the anti-windup clamp on the integral accumulator causes the integrator to drift arbitrarily far during saturation. On an 8-bit AVR, 32-bit overflow is silent and wraps the accumulator. The controller then behaves correctly in the short term but with a large hidden bias that eventually flips sign and causes violent instability.

Always set `pid_I_min` and `pid_I_max` to values that correspond to the maximum useful integral contribution, and call `pid_clamp_I` after every update.

33.8.7 Pitfall 7 — Derivative Amplifying Noise

A 10-bit ADC with ± 1 LSB noise produces first-differences of up to ± 2 LSB per sample. At a 1 kHz sample rate and $K_d = 1$, that is a D-term noise band of ± 2 . At $K_d = 16$ it is ± 32 — roughly 3% of full scale, visible as jitter in the actuator output.

Always low-pass filter the derivative with a small EMA ($k = 2..4$) before use. The low-pass on D limits the noise bandwidth without significantly delaying response to genuine rate-of-change events.

33.9 Summary

EMA (exponential moving average) — the primary ADC filter

```
accum = accum - (accum >> k) + sample
output = accum >> k
fc ≈ fs / (2k × 2π)
16-bit accum for k ≤ 6; 24-bit accum for k ≤ 14
```

Box filter (N-point average) — broadband noise at fixed N

```
running sum updated with circular buffer
N = 2M for efficient division (M right shifts)
SRAM: 2N + 3 bytes
```

Median filter (3-sample) — spike rejection, no smoothing

```
3-swap compare-and-swap sorting network
output = middle value of last 3 samples
SRAM: 4 bytes
```

Cascaded EMA — 40 dB/decade roll-off for strong filtering

```
apply EMA twice; doubles SRAM and cycle cost
```

PID structure

```
P = error >> p_shift          (or << for gain > 1)
I += (error >> Ki_shift)      with anti-windup clamp
D = (y_prev - y) << Kd_shift  then EMA on D
u = clamp(P + I + D, out_min, out_max)
Integral accumulator: 32-bit signed; clamp it AND clamp the output u
```

Initialisation matters:

```
zero all filter state in reset_handler (not done by -nostartfiles)
pre-load EMA accum with first_sample × 2k for instant convergence
set pid_I_min / pid_I_max before enabling the controller
```

Signed arithmetic:

```
LSR/ROR for unsigned accumulator right shifts (EMA, box output)
ASR/ROR for signed right shifts (PID integral shift, D-term gain)
BRGE/BRLT for signed 32-bit comparison after CP+CPC+CPC+CPC chain
```

Microchip source notes:

- ATtiny3217 data sheet, ADC result registers: ADC0_RESL is the low byte of the 10-bit result; ADC0_RESH holds bits 9:8. Read RESL first to latch both bytes.
- AVR Instruction Set Manual, LSR Rd: shifts Rd right by 1, zero into MSB, MSB into carry. Use for unsigned right shift.
- AVR Instruction Set Manual, ASR Rd: shifts Rd right by 1, MSB preserved (sign extension). Use for signed right shift.
- AVR Instruction Set Manual, CP Rd, Rr / CPC Rd, Rr: signed comparison flags are N $\oplus$ V (negative after borrow $\neq$ sign bit of result). BRGE/BRLT (branch if greater-or-equal / less-than, signed) use this combined flag.
- AVR Instruction Set Manual, MOVW Rd, Rr: copies a register pair. Both Rd and Rr must be even-numbered registers (R0, R2, ... R30).

33.10 Exercises

1. Implement `ema16_k6` ($k = 6$, $\alpha = 1/64$). What is the maximum safe input value for a 16-bit accumulator? Show that $1023 \times 64 = 65472$ fits in 16 bits. What is the cutoff frequency at $f_s = 1000$ Hz?
2. Modify `box8_update` for $N = 16$. What changes: the buffer size, the sum width, the index mask, and the shift count? Does the sum still fit in a 16-bit register?
3. Trace through `med3_update` for the input sequence 1023, 0, 512 (newest first). What is the output? Repeat for 0, 0, 1023. Does the filter behave as expected?
4. Implement `ema24_k10`. What are the three LDS addresses? Verify that the output `movw` picks up the correct two bytes of the 24-bit accumulator.
5. The `pid_clamp_I` routine uses BRGE (branch if greater or equal, signed) after a four-instruction CP/CPC/CPC/CPC chain. Explain why BRSH (branch if same or higher, unsigned) would give wrong results for negative accumulator values.
6. Connect a potentiometer to PA4 and log the raw and EMA-filtered ADC output over USART at 1 kHz. Use different k values (3, 5, 7) and compare the settling time and noise level on each setting. What k gives a useful balance between noise rejection and response speed for a slow-changing mechanical input?

Next: Appendices

Chapter 34

Event System and Configurable Custom Logic

The peripherals in the previous chapters all needed the CPU to move data: read a flag, set a bit, start a conversion. This chapter covers two peripherals that let other peripherals talk to each other *directly*, with the CPU asleep or busy elsewhere:

- The **Event System (EVSYS)** routes a signal (“event”) from one peripheral to another over dedicated channels. A timer overflow can start an ADC conversion, or a pin change can reset a counter, with zero instructions executed.
- **Configurable Custom Logic (CCL)** is a small block of programmable logic gates. Each look-up table (LUT) implements any Boolean function of up to three inputs, optionally clocked or filtered, and its inputs and outputs can be other peripherals or pins.

Together they form the chip’s “digital fabric”: glue logic that used to require external 74-series chips, now built in and wired in software.

34.1 Chapter structure (read this first)

This chapter covers:

1. Why hardware event routing matters (latency, power, determinism)
2. EVSYS architecture: generators, channels, users
3. Asynchronous vs synchronous channels on the ATtiny3217
4. Example: mirror a pin to an event-output pin (no CPU)
5. Example: trigger an ADC conversion from a timer event
6. CCL architecture: LUTs, truth tables, inputs, the sequential block
7. Example: a LUT as a simple gate driving a pin
8. Example: feeding a LUT from an event (EVSYS + CCL together)
9. What you learned, register quick reference, exercises

34.2 Why Route Events in Hardware?

Consider starting an ADC conversion every time a timer overflows. The software way is an interrupt service routine:

```
timer overflow -> ISR fires -> push regs -> write ADC START -> pop regs -  
> reti
```

Every step costs cycles, and the time between the overflow and the actual start *jitters* depending on what the CPU was doing. For sampling at a precise rate, jitter is noise.

The hardware way is one event channel:

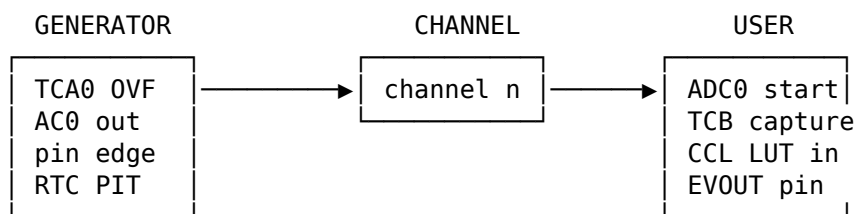
timer overflow -> event channel -> ADC start (fixed latency, no code)

The connection is configured once at startup. After that it works while the CPU sleeps, which also saves power. The trade-offs:

	Software (ISR)	Event System
Latency	variable (jitter)	fixed, deterministic
CPU cost	cycles per event	zero after setup
Works in sleep	only if ISR wakes CPU	yes (async channels)
Flexibility	arbitrary code	fixed routing + CCL logic

34.3 EVSYS Architecture

The Event System has three roles. A peripheral that produces a signal is a **generator**. A peripheral that consumes one is a **user**. Between them runs a **channel**:



You configure two things:

1. **Which generator drives a channel** — write the generator's selection value into that channel register.
2. **Which channel a user listens to** — write the channel into that user's register.

One generator can feed many users (they all select the same channel). A channel carries exactly one generator at a time.

34.4 Asynchronous vs Synchronous Channels

The ATtiny3217 (tinyAVR 1-series) splits channels into two kinds. This split is the most common stumbling block, so it is worth understanding before writing any code.

Async channels ASYNCCH0..ASYNCCH3 pass events with no clock; work in sleep, can carry very short pulses and edges.

Sync channels SYNCCH0..SYNCCH1 events are sampled on the peripheral clock; needed by users that expect a clocked event.

A user can only listen to the channel type it supports. Async users select an async channel via their ASYNCUSERn register; sync users select a sync channel via SYNCUSERn. Match the channel type to both the generator's nature and the user's requirement.

EVSYS registers (tinyAVR 1-series layout):

EVSYS.ASYNCCH0..3	pick the generator for each async channel
EVSYS.SYNCCH0..1	pick the generator for each sync channel
EVSYS.ASYNCUSER0..12	pick the async channel each async user listens to
EVSYS.SYNCUSER0..1	pick the sync channel each sync user listens to
EVSYS.ASYNCSTROBE	fire a software event on async channels
EVSYS.SYNCSTROBE	fire a software event on sync channels

Note: this layout differs from the unified CHANNEL/USER model on the megaAVR 0-series and newer AVR DA/DB parts. Code from those families does not port directly.

34.4.1 Generators (what can drive a channel)

Verified against the ATtiny3217 datasheet (section 15). Peripheral generators (CCL, AC0, TCD0, RTC) are available on every async channel; the PORT pin generators differ per channel. Timers TCA0/TCB0 are *sync*-only generators.

ASYNC generators (ASYNCCH0..3)

OFF	CCL_LUT0	CCL_LUT1
AC0_OUT	RTC_OVF	RTC_CMP
TCD0_CMPBCLR	TCD0_CMPASET	
TCD0_CMPBSET	TCD0_PROGEV	
PORT pins:	CH0=PORTA	CH1=PORTB
	CH2=PORTC	CH3=PORTA

SYNC generators (SYNCCH0..1)

OFF	TCB0
TCA0_OVF_LUNF	TCA0_HUNF
TCA0_CMP0	TCA0_CMP1
TCA0_CMP2	
PORTC pins, then	PORTB pins

34.4.2 Users (what can listen to a channel)

Each user has a fixed index; you write the channel number into that user's register. Async users select an async channel; sync users a sync channel.

ASYNC users (write to ASYNCUSERn)

0	TCB0	7	TCD0_EV1
1	ADC0	8	EV0UTA (PA2)
2	CCL_LUT0_EV0	9	EV0UTB (PB2)
3	CCL_LUT1_EV0	10	EV0UTC (PC2)
4	CCL_LUT0_EV1	11	TCB1
5	CCL_LUT1_EV1	12	ADC1
6	TCD0_EV0		

SYNC users (write to SYNCUSERn)

0	TCA0
1	USART0

For a CCL LUT, its first event input (EVENT0; the datasheet calls it EVENTA) arrives through the ..._EV0 user, and the second (EVENT1 / EVENTB) through the ..._EV1 user — e.g. LUT0's EVENT0 is ASYNCUSER2.

34.4.3 A trap: `_gc` constants do not assemble

The code in this chapter uses the readable group-config names like `EVSYS_ASYNCCH0_RTC_OVF_gc` for clarity. Be aware of a catch when you move to a .S file: in `iotn3217.h` these `_gc` constants are defined as members of a C enum, not as `#define` macros. The assembler only runs the C preprocessor, so it never sees enum names — assembling a file that uses a bare `_gc` fails with “*undefined reference*”. (The `_bm` and `_bp` bit constants, by contrast, are `#define` macros and work fine in assembly.)

The fix is to define the value yourself with `.equ`, taking the number from the datasheet register table. For example, `RTC_OVF` is generator value `0x08` for an async channel:

```
.equ ASYNCCH_RTC_OVF, 0x08      /* datasheet sec.15, ASYNCCHn generator */
ldi  r16, ASYNCCH_RTC_OVF
sts  EVSYS_ASYNCCH0, r16
```

The companion `src/ch20b_event_ccl/` files use this `.equ` style throughout and assemble cleanly with a stock `avr-gcc`. The register *names* (`EVSYS_ASYNCCH0`, `CCL_TRUTH0`, ...) are plain `#defines` and need no such treatment.

34.5 Example: Mirror a Pin to an Event-Output Pin

The simplest possible event: copy one pin to another with no CPU involvement. We use PA6 as the source (an async PORTA generator on `ASYNCCH0`) and route it to the event-output user `EV0UTA`, which drives the dedicated PA2 event pin.


```

/* --- generator: PA6 edge/level drives async channel 0 --- */
ldi  r16, EVSYS_ASYNCCH0_PORTA_PIN6_gc
sts  EVSYS_ASYNCCH0, r16          /* ASYNCCH0 carries PA6 */

/* --- user: event output A (PA2) listens to async channel 0 --- */
ldi  r16, EVSYS_ASYNCUSER8_ASYNCCH0_gc /* EVOUTA is an async user */
sts  EVSYS_ASYNCUSER8, r16

/* PA2 must be an output for EVOUT to drive it */
sbi  VPORTA_DIR, 2                /* PA2 = output */

```

After this runs, PA2 follows PA6 directly. No interrupt, no loop. EVOUTA is async user 8 and drives PA2; async channel 0's pin generators are the PORTA pins (so PA6 = PORTA_PIN6). Both facts are from the verified tables above.

34.6 Example: Trigger an ADC Conversion From a Timer Event

A periodic, jitter-free sample clock. Here the audit pays off: on the ATtiny3217 **ADC0 is an asynchronous user** (ASYNCUSER1), so it can only listen to an *async* channel. The TCA0/TCB0 overflow events are *synchronous* generators — they live on SYNCCH and cannot feed an async user directly. So we use an async periodic source instead: the **RTC overflow** (RTC_OVF), which is an async generator. (RTC setup is its own topic; here we add only the event wiring and the ADC's event trigger.)

```

/* --- generator: RTC overflow drives async channel 0 --- */
ldi  r16, EVSYS_ASYNCCH0_RTC_OVF_gc
sts  EVSYS_ASYNCCH0, r16

/* --- user: ADC0 start (ASYNCUSER1) listens to async channel 0 --- */
ldi  r16, EVSYS_ASYNCUSER1_ASYNCCH0_gc /* ADC0 is an async user */
sts  EVSYS_ASYNCUSER1, r16

/* --- ADC: enable event-triggered start --- */
ldi  r16, ADC_STARTEI_bm          /* start conversion on event input */
sts  ADC0_EVCTRL, r16

```

Every RTC overflow now launches a conversion at a fixed delay. The CPU only has to read ADC0.RES when RESRDY is set (or have *that* drive another event). This is the backbone of regular-rate sampling for the digital filters in the next chapter.

This async/sync mismatch is exactly the trap the previous section warned about: always check whether your generator and user are on the same channel type before wiring them. The reference tables below list which side of the fence each one is on.

34.7 CCL Architecture

Configurable Custom Logic gives you programmable gates. The ATtiny3217 has two look-up tables, LUT0 and LUT1, plus one sequential-logic block that can pair the two LUTs into a flip-flop or latch.

Each LUT is a 3-input Boolean function defined by an 8-entry **truth table**:

IN2	IN1	IN0	TRUTH bit
0	0	0	bit 0
0	0	1	bit 1
0	1	0	bit 2

0	1	1		bit 3
1	0	0		bit 4
1	0	1		bit 5
1	1	0		bit 6
1	1	1		bit 7

The output is whatever bit you placed at the row addressed by the three inputs. Any 3-input logic function is just the right 8-bit TRUTH_n value. A few useful ones:

Function (of IN₁,IN₀; IN₂=0) TRUTH value Notes

Buffer	OUT = IN ₀	0xAA	IN ₀ to output
Inverter	OUT = !IN ₀	0x55	
AND	OUT = IN ₀ & IN ₁	0x88	
OR	OUT = IN ₀ IN ₁	0xEE	
XOR	OUT = IN ₀ ^ IN ₁	0x66	

Each LUT's inputs are selected independently from a menu: a pin (IO), an event channel (EVENT0/EVENT1 — the datasheet labels these EVENTA/EVENTB, but the device header names them EVENT0/EVENT1), the other LUT's output (LINK), its own filtered output (FEED-BACK), or a peripheral (AC0, TCB0, TCA0 waveform, USART/SPI, ...). Selection lives in LUTnCTRLB (IN₀, IN₁) and LUTnCTRLC (IN₂).

When a LUT input is set to IO, or its output is enabled with OUTEN, it uses a fixed pin. On the 24-pin ATtiny3217 (from the datasheet's Multiplexed Signals table):

	IN ₀	IN ₁	IN ₂	OUT (default)	OUT (alternate, via PORTMUX)
LUT0	PA0	PA1	PA2	PA4	PB4
LUT1	PC3	PC4	PC5	PA7	PC1

On the 20-pin SOIC package PC4 and PC5 are not bonded out, so LUT1's IN₁/IN₂ IO inputs are unavailable there — route those through an event channel instead.

CCL registers:

CCL.CTRLA	module enable, run-in-standby
CCL.SEQCTRL0	sequential block mode (off / D-FF / JK / latch / RS)
CCL.LUTnCTRLA	LUT enable, output-to-pin enable, clock source, filter, edge
CCL.LUTnCTRLB	input select for IN ₀ and IN ₁
CCL.LUTnCTRLC	input select for IN ₂
CCL.TRUTH _n	the 8-bit truth table

Important ordering rule: the LUT control and truth registers are **enable-protected**. Configure LUTnCTRLB/C and TRUTH_n *before* you set the ENABLE bit in LUTnCTRLA, and before the module ENABLE in CCL.CTRLA.

34.8 Example: A LUT as a Simple Gate Driving a Pin

Make LUT0 compute OUT = IN₀ AND IN₁ from two input pins and drive its output pin, entirely in hardware. We use IO inputs and the AND truth value 0x88.

```
/* configure inputs FIRST (enable-protected) */
ldi r16, CCL_INSEL0_IO_gc | CCL_INSEL1_IO_gc
sts CCL_LUT0CTRLB, r16          /* IN0 = pin, IN1 = pin */
ldi r16, CCL_INSEL2_MASK_gc     /* IN2 unused -> masked to 0 */
sts CCL_LUT0CTRLC, r16

/* truth table: AND of IN1, IN0 */
ldi r16, 0x88
sts CCL_TRUTH0, r16
```

```

/* enable LUT0 and route its output to the LUT0 pin */
ldi  r16, CCL_ENABLE_bm | CCL_OUTEN_bm
sts  CCL_LUT0CTRLA, r16

/* enable the CCL module */
ldi  r16, CCL_ENABLE_bm
sts  CCL_CTRLA, r16

```

The output pin now equals the AND of the two LUT0 input pins, updating with only gate-propagation delay. With I0 inputs and OUTEN, LUT0 uses fixed pins: IN0 = PA0, IN1 = PA1, output = PA4 (the alternate output PB4 is selectable via PORTMUX). So this gate reads PA0 and PA1 and drives their AND onto PA4 — no CPU, no code.

34.9 Example: Feeding a LUT From an Event (EVSYS + CCL Together)

The two peripherals combine: a CCL input can be an EVSYS channel, so logic can react to any generator the Event System can route. Here LUT1's IN0 comes from event input EVENT0 (the datasheet's EVENTA), letting the LUT gate (say) an AC0 or timer signal that arrives over EVSYS.

```

/* route some generator onto an async channel, then to the LUT's EVENT0 */
ldi  r16, EVSYS_ASYNCCH2_AC0_OUT_gc
sts  EVSYS_ASYNCCH2, r16

/* CCL LUTs consume events via dedicated EV users; pick the channel */
ldi  r16, EVSYS_ASYNCUSER3_ASYNCCH2_gc /* CCL LUT1 EV0 user */
sts  EVSYS_ASYNCUSER3, r16

/* LUT1 IN0 = EVENT0 (datasheet: EVENTA) */
ldi  r16, CCL_INSEL0_EVENT0_gc | CCL_INSEL1_MASK_gc
sts  CCL_LUT1CTRLB, r16
ldi  r16, CCL_INSEL2_MASK_gc
sts  CCL_LUT1CTRLC, r16

ldi  r16, 0xAA /* OUT = IN0 (buffer the event) */
sts  CCL_TRUTH1, r16

ldi  r16, CCL_ENABLE_bm | CCL_OUTEN_bm
sts  CCL_LUT1CTRLA, r16
ldi  r16, CCL_ENABLE_bm
sts  CCL_CTRLA, r16

```

This is the pattern behind hardware-only signal conditioning: comparator or timer signal -> EVSYS -> CCL gate/filter -> output pin or another peripheral, all without the CPU.

34.10 What You Learned

- The **Event System** routes signals between peripherals over channels, with fixed latency and zero CPU cost after setup. Roles are **generator**, **channel**, and **user**.
- The ATtiny3217 has **asynchronous** channels (no clock, work in sleep, carry edges) and **synchronous** channels (sampled on a clock). A user must select a channel of the type it supports.
- Configuration is two writes: generator -> channel register, and channel -> user register.
- **CCL** provides programmable gates. Each **LUT** is any 3-input Boolean function set by an 8-bit **truth table**, with inputs chosen from pins, events, the other LUT, feedback, or

peripherals.

- LUT config registers are **enable-protected**: set inputs and truth table before enabling the LUT and the module.
- EVSYS and CCL **combine**: a LUT input can be an event channel, building hardware signal paths that never touch the CPU.

34.11 Register Quick Reference

EVSYS.ASYNCCH0..3	generator select for async channels
EVSYS.SYNCCH0..1	generator select for sync channels
EVSYS.ASYNCUSER0..12	async channel select per user
EVSYS.SYNCUSER0..1	sync channel select per user
EVSYS.ASYNCSTROBE	software event on async channels
EVSYS.SYNCSTROBE	software event on sync channels
CCL.CTRLA	module ENABLE, RUNSTDBY
CCL.SEQCTRL0	sequential block mode
CCL.LUTnCTRLA	ENABLE, OUTEN, CLKSRC, FILTSEL, EDGEDET
CCL.LUTnCTRLB	INSEL0, INSEL1
CCL.LUTnCTRLC	INSEL2
CCL.TRUTHn	8-bit truth table

34.12 Exercises

1. Write the TRUTHn value for a 3-input majority function (output is 1 when at least two of IN0, IN1, IN2 are 1). Verify it against the truth-table diagram.
2. Configure LUT0 as an inverter ($OUT = !IN0$) driven by an input pin and observe the output pin. What truth value did you use, and why is it the bitwise complement of the buffer value?
3. Route the RTC periodic-interrupt (PIT) event to the ADC start user so the ADC samples at the PIT rate. Which channel type (async or sync) is required, and why?
4. Combine both peripherals: route AC0's output through an EVSYS channel into a CCL LUT that buffers it to a pin. Compare the pin's response time to doing the same job with an AC0 interrupt that toggles the pin in software.
5. Use the sequential-logic block: set SEQCTRL0 to D-flip-flop mode and feed LUT0 (D) and LUT1 (clock). What register sequence enables it, and what ordering rule must you respect?

Next: Appendix A — Register Reference

EEPROM and Nonvolatile Data

RAM forgets everything at power-off. Flash remembers, but it is the program – you do not rewrite it casually at run time. Between them sits EEPROM: a small block of nonvolatile memory meant exactly for the data an application needs to **keep** – calibration constants, a device serial number, user settings, a run-hours counter. The ATtiny3217 has ***256 bytes*** of it.

This chapter covers reading and writing that EEPROM from assembly through the Nonvolatile Memory Controller (NVMCTRL), and the durability concerns – write endurance, safe update patterns, and integrity checks – that separate a toy example from firmware you can ship.

The EEPROM Mental Model

EEPROM on this device is **memory-mapped** at address ``0x1400``. That single fact makes half the job trivial: **reading** EEPROM is an ordinary ``LDS`` from ``0x1400 + offset``. There is no command, no busy wait, no special sequence. The byte is just there in the address space.

Writing is where the controller comes in. You cannot store directly to EEPROM the way you store to RAM, because a nonvolatile cell must be electrically erased and re-written – a slow operation (a few milliseconds) that the hardware sequences for you. Writing therefore has two halves:

EEPROM layout (ATtiny3217) base address : 0x1400 size : 256 bytes page size : 64 bytes (4 pages)

Read : LDS from 0x1400 + offset (immediate, like any SRAM read) Write: fill page buffer
-> issue NVM command (milliseconds, sequenced by NVMCTRL)

The key mechanism is the **page buffer**: a staging area you fill with normal store instructions. Storing to a mapped EEPROM address does not change EEPROM – it loads that byte into the page buffer and marks it as "touched." A subsequent NVM command transfers the buffer into the array. Crucially for EEPROM, **only the bytes you touched** are erased and written, so you can update a single byte without disturbing the other 63 in its page.

NVMCTRL Register Path

EEPROM writes use four NVMCTRL registers plus the CPU's Configuration Change Protection register:

NVMCTRL_STATUS FBUSY(0) EEBUSY(1) WRERROR(2) — poll EEBUSY before/after a write NVMCTRL_CTRLA CMD[2:0] — the command to execute CPU_CCP — unlock key, written first (mapped EEPROM) 0x1400 + offset — page buffer fill / direct read

The command set (in ``NVMCTRL_CTRLA`` ``CMD``) that matters for EEPROM:

CMD value name effect 0x00 NONE no command 0x01 WP write page buffer to the array (no erase) 0x02 ER erase the addressed page 0x03 ERWP erase + write in one step <- the simple choice for EEPROM 0x04 PBC clear the page buffer 0x06 EEER erase the entire EEPROM

``ERWP`` (erase-and-write page) is the one-step option and what most EEPROM code uses. The split ``ER`` / ``WP`` pair exists for Flash, where separating the slow erase from the write can shorten time-critical sections; for a single EEPROM byte the distinction rarely matters.

Configuration Change Protection (CCP)

``NVMCTRL_CTRLA`` is a protected register. You cannot simply store a command to it – the write is silently ignored unless you first arm Configuration Change Protection. The sequence the datasheet (sec. 9.3.2.4) mandates is:

1. Confirm no write is in progress (``EEBUSY``/``FBUSY`` clear in ``NVMCTRL_STATUS``).
2. Write the **SPM key**, ``0x9D``, to ``CPU_CCP``.
3. **Within the next four instructions**, write the command to ``NVMCTRL_CTRLA``.

The four-instruction window is a hardware safety latch: it makes an accidental

command write (from a wild pointer or a glitch) astronomically unlikely, because the stray write would also have to be immediately preceded by the exact unlock key. Miss the window and the protected write is dropped.

Reading and Writing a Byte

The companion file ``src/ch21_eeprom/eeprom_rw.S`` writes ``0x2A`` to EEPROM offset 0, reads it back, and lights the LED if the round-trip succeeded. Reading is the easy half:

```
``asm
.equ EEPROM_BASE, 0x1400

ldi    ZL, lo8(EEPROM_BASE)
ldi    ZH, hi8(EEPROM_BASE)
ld     r20, Z          /* r20 = EEPROM[0]; no wait, no command */
```

The write is the subroutine below. r18 is the offset, r19 the data:

```
.equ NVM_CMD_ERWP, 0x03      /* erase+write page */
.equ CCP_SPM_KEY, 0x9D      /* CCP unlock key for NVMCTRL.CTRLA */

eeprom_write_byte:
/* 1. wait until any previous EEPROM write has finished */
1: lds    r16, NVMCTRL_STATUS
   sbrc   r16, NVMCTRL_EEBUSY_bp
   rjmp   1b

/* 2. point Z at EEPROM_BASE + offset and store the byte (fills buffer).
 * EEPROM_BASE low byte is 0x00 and offset <= 0xFF -> no carry to ZH. */
ldi    ZL, lo8(EEPROM_BASE)
ldi    ZH, hi8(EEPROM_BASE)
add    ZL, r18
st     Z, r19

/* 3+4. unlock CCP, then issue ERWP within the 4-instruction window */
ldi    r16, CCP_SPM_KEY
sts    CPU_CCP, r16          /* arm CCP for the next 4 instructions */
ldi    r16, NVM_CMD_ERWP
sts    NVMCTRL_CTRLA, r16    /* protected write: erase+write the page */
ret
```

Build and confirm it assembles to the expected sequence:

```
$ avr-gcc -mmcu=attiny3217 -nostartfiles eeprom_rw.S -o eeprom_rw.elf
$ avr-objdump -d eeprom_rw.elf
...
24: ldi    r16, 0x9D          ; CCP key
26: sts    0x0034, r16        ; CPU.CCP  <- arm
2a: ldi    r16, 0x03          ; ERWP      (instruction 1 of 4)
2c: sts    0x1000, r16        ; NVMCTRL.CTRLA (instruction 2) -> inside window
```

A few details that bite people:

- **The EEBUSY wait comes first.** Issuing a command while a previous EEPROM write is still in progress sets `WRERROR`. Poll `EEBUSY` *before* starting, not only after.
- **No carry handling is needed here** only because `EEPROM_BASE` is page-aligned to `0x1400` (low byte `0x00`) and the offset is a single byte. For a 16-bit offset you would add `ZL / adc ZH`.

- **The CPU keeps running during an EEPROM write.** Unlike Flash, an EEPROM write does not stall the core — your code continues immediately after the command store. That is why the *next* write must re-check EEBUSY.
- **Interrupts and the CCP window.** If an interrupt fires between the CPU_CCP write and the command store, the handler’s instructions consume the four-instruction budget and the protected write can fall outside it. In code that runs with interrupts enabled, bracket the unlock-and-command pair with `cli / sei` (saving SREG if the caller’s interrupt state is unknown).

34.13 Write Endurance and Wear

EEPROM cells wear out. The ATtiny3217 is rated for roughly **100,000 erase/write cycles per byte**. That is plenty for settings changed occasionally, and easy to exhaust by accident. A counter updated once per second to the *same* byte burns through 100,000 cycles in about a day.

Two habits keep you well clear of the limit:

Write only on change. Read the current byte first; skip the write if the new value matches. Most “save the settings” calls in real firmware write nothing, because nothing changed:

```
/* eeprom_update_byte: write only if different (saves a cell cycle) */
eeprom_update_byte:
    ldi    ZL, lo8(EEPROM_BASE)
    ldi    ZH, hi8(EEPROM_BASE)
    add    ZL, r18
    ld     r16, Z
    cp     r16, r19
    breq   9f                                /* already equal -> do nothing */
    rcall  eeprom_write_byte
9:  ret
```

This is exactly what `avr-libc`’s `eeprom_update_byte` does, and it should be your default over a blind write.

Spread wear across cells (wear leveling). For a value that genuinely changes often — a run-hours counter, a “last used channel” — do not pin it to one byte. Rotate it through a ring of *N* bytes, tagging each record so you can find the most recent on power-up. With a 16-byte ring you multiply effective endurance by 16. The simplest scheme stores a monotonically increasing sequence number alongside each value; the live record is the one with the highest sequence number, and the next write lands in the following slot.

34.14 Integrity: Surviving a Botched Write

EEPROM’s worst failure is not wear — it is a write interrupted by a brownout or reset, which can leave a multi-byte value half-updated. A calibration constant that is half old and half new is worse than either.

Two standard defenses:

Checksum the block. Store a one-byte checksum (or CRC) after a parameter block. On startup, recompute it; if it does not match, the block is corrupt and you fall back to defaults instead of trusting garbage. A simple additive checksum is often enough:

```
/* checksum: sum bytes [Z .. Z+r17-1], result in r16 (two's-complement form
 * so that the stored byte makes the total sum zero). */
```

```

eeprom_checksum:
    clr    r16
1:  ld     r0, Z+
    add    r16, r0
    dec    r17
    brne   1b
    neg    r16                /* stored value -> whole block sums to 0 */
    ret

```

Write the validity marker last. When updating a block, write the payload first and a “valid” flag (or the matching checksum) only after the payload is safely committed. A reset midway then leaves the flag clear, and startup correctly treats the block as invalid rather than partially valid. Order matters: the marker is a commit point, so it must be the final write.

For values that must survive *any* interruption, combine both ideas with the wear ring above: write a complete record (payload + sequence number + checksum) to the next slot, and only a fully written, checksum-valid record with the highest sequence number is ever read back. A torn write simply leaves an invalid slot that the reader skips.

34.15 Erasing

To clear a single page back to 0xFF, touch any byte in it and issue ER (CMD = 0x02). To wipe the whole EEPROM in one operation, use EEER (CMD = 0x06) — useful in a factory-reset path. Both follow the same CCP-unlock-then-command sequence as a write. After an erase, an unwritten EEPROM byte reads as 0xFF, which is the natural “blank / never written” sentinel: code that treats 0xFF as “use default” needs no separate erased-flag.

34.16 Common Pitfalls

- Storing to 0x1400 + offset and expecting it to take effect immediately. That only fills the page buffer; without the NVM command nothing reaches EEPROM.
 - Skipping the EEBUSY check and issuing a command mid-write, setting WRERROR.
 - Missing the four-instruction CCP window — usually by letting an interrupt land between the CPU_CCP write and the command store. Guard with cli/sei.
 - Writing the SPM key with the wrong value. It is 0x9D for NVMCTRL_CTRLA; 0xD8 (CCP_IOREG) is a *different* key for protected I/O registers and will not unlock the NVM command.
 - Blind writes in a hot loop, burning endurance. Read-before-write (update semantics) and, for frequently changed values, wear leveling.
 - Treating EEPROM like RAM for a fast-changing variable. A 3 ms write is far too slow for per-sample storage, and the endurance limit makes it unwise anyway.
 - Forgetting that the page buffer is **cleared on wake from sleep** — fill it and issue the command without sleeping in between.
-

34.17 Summary

EEPROM (ATtiny3217): 256 bytes, mapped at 0x1400, 64-byte pages, ~100k cycles/byte.

```

Read   : LDS from 0x1400 + offset.                (immediate)
Write  : 1. wait EEBUSY clear (NVMCTRL_STATUS)
        2. store byte(s) to 0x1400 + offset (fills page buffer)

```


3. CPU_CCP = 0x9D (SPM key)
4. NVMCTRL_CTRLA = 0x03 (ERWP) within 4 instructions of step 3
(CPU keeps running; only touched bytes are written)

Commands: WP=0x01 ER=0x02 ERWP=0x03 PBC=0x04 EEER=0x06.

Durability:

- read-before-write (update) to skip no-op writes
- wear-level frequently changed values across a ring of bytes
- checksum parameter blocks; write the validity marker last
- erased / never-written bytes read as 0xFF

EEPROM is simple to read and only slightly more involved to write — the CCP unlock plus the EEBUSY wait are the whole ceremony. The engineering that matters is not the write sequence but the *policy* around it: writing rarely, spreading wear, and never trusting a block you have not checksummed.

34.18 Exercises

1. Modify `eprom_rw.S` to write the four bytes 0xDE 0xAD 0xBE 0xEF to offsets 0–3 and read them back. Do they all land in the same page? How many NVM commands does the most efficient version issue, and why can it be fewer than four?
2. Implement `eprom_update_byte` from the chapter and call it twice with the same value. Using a counter in SRAM incremented inside `eprom_write_byte`, confirm the second call performs no actual write.
3. The CCP window is four instructions. Rewrite the unlock-and-command pair using out/in to `_SFR_IO_ADDR(CPU_CCP)` instead of `sts`. Does it still fit the window? What is the instruction-count difference?
4. Write `eprom_read_block` and `eprom_write_block` that move `r17` bytes between SRAM (pointed to by `X`) and EEPROM (offset in `r18`). Handle a block that straddles a 64-byte page boundary.
5. Add the additive checksum routine to a 16-byte parameter block. Corrupt one byte by hand (write a different value) and confirm the recomputed checksum no longer matches.
6. Design a wear-leveled 8-slot ring for a single run-hours counter. Each slot holds a value and a 1-byte sequence number. Write the startup routine that scans the ring and returns the slot index holding the highest sequence number, accounting for the sequence number wrapping past 255.
7. The whole-EEPROM erase command is EEER (0x06). Write a `factory_reset` routine that issues it with the correct CCP sequence, then verify offset 0 reads back as 0xFF.
8. Using `avr-objdump -d`, confirm that the `sts NVMCTRL_CTRLA` in your write routine falls within four instructions of the `sts CPU_CCP`. What is the byte distance between the two stores in the encoded program?

Chapter 35

Fuses and Device Configuration

Some of a microcontroller's behavior is decided before your code runs at all. Whether the watchdog is already armed at boot, what frequency the main oscillator starts at, whether the UPDI pin is a programming interface or a GPIO, whether EEPROM survives a chip erase — these are set by **fuses**: a small block of nonvolatile configuration latched into hardware at every reset.

Fuses sit between hardware straps and software registers. Like hardware straps, they are read once at reset and you cannot change them from the running program. Like registers, they are just bytes with named bit-fields. This chapter covers reading them at run time, the fuses that matter most on the ATtiny3217, the program-and-recover workflow with a UPDI programmer, and the one mistake that can brick a board.

35.1 What a Fuse Is

At reset, the device copies each fuse byte into the corresponding peripheral's configuration. FUSE.WDTCFG becomes the reset value of WDT.CTRLA; FUSE.OSCCFG selects the base clock; FUSE.SYSCFG0 decides what the UPDI/RESET pin does. After that copy, the fuses are passive: the *registers* drive the hardware, and the fuses just wait for the next reset.

Two consequences follow, and they are the whole mental model:

Read at run time : YES — fuses are memory-mapped read-only at 0x1280 (LDS).
Write at run time : NO — fuse programming is a UPDI/PDI operation, not an instruction your firmware can execute.

That second point surprises people coming from classic AVR. On the tinyAVR 1-series the NVM FUSEWRITE command is **PDI-only** — it works through the UPDI programmer, not from code running on the core. So fuses are configured by your *toolchain* (avrdude over UPDI), and your firmware can only *read* them.

35.2 The Fuse Map

The ATtiny3217 fuses are memory-mapped starting at 0x1280:

addr	fuse	controls
0x1280	WDTCFG	watchdog period/window loaded into WDT.CTRLA at reset
0x1281	BODCFG	brown-out detector: level, and active/sleep operation
0x1282	OSCCFG	base oscillator frequency (16 or 20 MHz) + osc lock
0x1284	TCD0CFG	TCD0 default compare outputs

```

0x1285 SYSCFG0      reset-pin config, EEPROM-save, CRC source
0x1286 SYSCFG1      start-up time (SUT)
0x1287 APPEND       application code section end (boot/app boundary)
0x1288 BOOTEND      boot section end (in 256-byte blocks)

```

The `avr-libc` header gives a symbol for each: `FUSE_WDTCFG`, `FUSE_OSCCFG`, `FUSE_SYSCFG0`, and so on — all LDS-able.

35.2.1 The fuses that bite

A handful are worth knowing in detail because they change how the chip boots or, in one case, whether you can reprogram it at all:

- **OSCCFG.FREQSEL** — selects the internal oscillator base: 16 MHz or 20 MHz. The factory default on Curiosity Nano boards is 20 MHz, which the main clock prescaler then divides (the book’s /6 gives 3.333 MHz). Get this wrong and every timing calculation — baud rates, timer periods — is off by the 16/20 ratio. `OSLOCK` can lock the calibration.
- **SYSCFG0.RSTPINCFG** — the dangerous one. The shared UPDI/RESET/PA0 pin can be **UPDI** (0x1, default), **GPIO** (0x0), or **RESET** (0x2). UPDI mode is what lets your programmer talk to the chip. Set it to GPIO or RESET and the ordinary UPDI connection stops working — recovery then needs a high-voltage UPDI pulse that most simple programmers cannot produce. Treat this fuse as radioactive (see “Recovery and UPDI-Safe Habits” below).
- **SYSCFG0.EESAVE** — when set, EEPROM contents are preserved across a chip erase. Useful for keeping calibration data while reflashing firmware.
- **WDTCFG** — if non-zero, the watchdog is *already enabled when your code starts*, and `WDT.STATUS.LOCK` is set, so firmware cannot turn it off. This is a safety feature for fielded products (covered in the Watchdog chapter).
- **BOOTEND / APPEND** — define the boot and application code section boundaries (in 256-byte units), used when a bootloader must live in a write-protected region. With `BOOTEND = 0` the whole flash is one application section.

35.3 Reading a Fuse at Run Time

Because fuses are memory-mapped, reading one is a single LDS. The companion file `src/ch22_fuses/fuse_read.S` reads `SYSCFG0`, isolates the `RSTPINCFG` field, and lights the LED if the UPDI pin is still in UPDI mode:

```

.equ RSTPINCFG_gm, 0x0C      /* SYSCFG0 bits 3:2 */
.equ RSTPINCFG_UPDI, (0x1 << 2) /* UPDI mode, in place */

main:
    sbi    VPORTA_DIR, 3

    lds    r16, FUSE_SYSCFG0    /* read the live fuse value */
    andi   r16, RSTPINCFG_gm
    cpi    r16, RSTPINCFG_UPDI
    brne   done                /* not UPDI -> LED off */
    sbi    VPORTA_OUT, 3        /* UPDI mode -> LED on */

done:
    rjmp   done

```

Reading fuses at run time is occasionally genuinely useful: firmware can adapt to how it was configured — for example, scaling its baud-rate constants by checking whether `OSCCFG` selected 16 or 20 MHz, instead of hard-coding one assumption.

35.4 Programming Fuses (UPDI Workflow)

Since firmware cannot write fuses, you set them with `avrdude` over the Curiosity Nano's onboard nEDBG UPDI bridge. Fuses are addressed by index (the offset from 0x1280): fuse 2 is 0SCCFG, fuse 5 is SYSCFG0, and so on.

Read the current fuses:

```
$ avrdude -c pkobn_updi -p attiny3217 -U fuse5:r:-:h
0xf6
```

Read-modify-write a single fuse — for example, set EESAVE (bit 0 of SYSCFG0) while leaving the other bits as the default 0xF6:

```
$ avrdude -c pkobn_updi -p attiny3217 -U fuse5:w:0xf7:m
```

The golden rule of the write step is **read-modify-write, never blind-write**. A fuse byte packs several unrelated fields; writing a value you computed by hand risks clearing a field you did not mean to touch. Always start from the byte you read back.

35.5 Recovery and UPDI-Safe Habits

The one fuse field that can lock you out is SYSCFG0.RSTPINCFG. Here is the failure mode and how to stay clear of it:

- **What goes wrong.** Writing RSTPINCFG = GPIO (0x0) or RESET (0x2) repurposes the UPDI pin. The next time you try to program the chip, the programmer cannot establish a UPDI link, because that pin is no longer a UPDI interface.
- **Why it is hard to undo.** Re-enabling UPDI requires a **high-voltage UPDI** sequence (a 12 V pulse on the pin to force it back). The nEDBG on a Curiosity Nano does not do HV-UPDI; you would need a dedicated HV-capable programmer.
- **How to avoid it.** Leave RSTPINCFG at UPDI unless you have a concrete need for the pin and an HV programmer on hand. If you must use it as RESET or GPIO, do that change *last*, on a board you are willing to risk, and keep an HV-capable tool available.

A few more UPDI-safe habits:

- **Keep EESAVE in mind before a chip erase.** If you rely on EEPROM calibration surviving a reflash, set EESAVE first; otherwise chip erase clears EEPROM too.
- **Change one field at a time** and verify by reading the fuse back, so a surprising result is easy to attribute.
- **Do not set 0SCLOCK** while you are still iterating on clock configuration — it locks the oscillator calibration.

35.6 Common Pitfalls

- Trying to write a fuse from firmware. FUSEWRITE is PDI-only; the store does nothing. Fuses change only through a UPDI programmer.
- Blind-writing a fuse byte and wiping an unrelated field in the same byte. Read-modify-write.
- Setting RSTPINCFG away from UPDI without an HV-UPDI programmer — the classic way to “brick” a tinyAVR for ordinary tools.
- Assuming the base clock is 16 MHz when 0SCCFG selected 20 MHz (or vice versa). Every baud and timer constant inherits the error.
- Forgetting EESAVE and losing EEPROM calibration on the next chip erase.
- Confusing fuse *index* with fuse *address*. `avrdude` uses the index (0-8); the memory-mapped address is 0x1280 + index.

35.7 Summary

Fuses (ATtiny3217): nonvolatile config latched into registers at reset.

Read : LDS from 0x1280 + index (FUSE_* symbols). Read-only at run time.

Write : UPDI programmer only (avrdude -U fuseN:w:0xNN:m). FUSEWRITE is PDI-only.

Map: 0x1280 WDTCFG 0x1281 BODCFG 0x1282 OSCCFG 0x1284 TCD0CFG
0x1285 SYSCFG0 0x1286 SYSCFG1 0x1287 APPEND 0x1288 BOOTEND

Key fields:

OSCCFG.FREQSEL	16 or 20 MHz base clock
SYSCFG0.RSTPINCFG	0=GPIO 1=UPDI(default) 2=RESET <- only UPDI keeps you safe
SYSCFG0.EESAVE	keep EEPROM across chip erase
WDTCFG	watchdog enabled (and LOCKed) at boot if non-zero
BOOTEND/APPEND	boot/app section boundaries (256-byte units)

Workflow: read -> modify one field -> write -> read back to verify.

Hazard: RSTPINCFG != UPDI can require HV-UPDI to recover.

Fuses are short to read and rarely changed, but they set the stage everything else runs on. The discipline that matters is conservatism: read before you write, touch one field at a time, and never move RSTPINCFG off UPDI without a way back.

35.8 Exercises

1. Read FUSE_OSCCFG in assembly and branch on the FREQSEL field (bits 1:0): set a register to 16 or 20 to record the detected base frequency. What value does a stock Curiosity Nano report?
2. Use avrdude to read all of fuses 0–8 into a hex dump. Identify the SYSCFG0 value and decode its RSTPINCFG, EESAVE, and CRCSRC fields by hand.
3. Write the avrdude command that sets EESAVE without disturbing any other bit in SYSCFG0, given that the current value reads back as 0xF6. What is the new byte?
4. Explain, in terms of the reset-time copy, why changing WDTCFG to a non-zero value means firmware can no longer disable the watchdog. Which status bit enforces this?
5. The default SYSCFG0 is 0xF6. Decode every field of that byte. Which bits are reserved, and what are RSTPINCFG and CRCSRC set to by default?
6. A colleague set RSTPINCFG = RESET and can no longer program the board with the nEDBG. Describe the recovery procedure and the hardware it requires. What habit would have prevented the situation?

Chapter 36

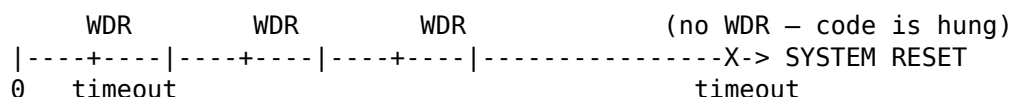
Watchdog and Reset Recovery

Firmware fails in ways you did not plan for: a peripheral wedges, a buffer pointer runs wild, a brown-out leaves a state machine stranded, a `rcall` chain overflows the stack. On a desktop the OS kills the process. On a bare-metal microcontroller there is no OS — so the device must rescue *itself*. That is the watchdog's job: a hardware timer that issues a system reset if the program stops checking in. Paired with the reset controller, which records *why* the last reset happened, it turns a hung system into a recoverable one.

This chapter covers arming and petting the watchdog from assembly, the reset controller's cause flags, software-triggered resets, and a fault-counter policy that keeps a device from looping forever on the same fault.

36.1 The Watchdog Mental Model

The Watchdog Timer (WDT) is a free-running counter on its own clock. Your program must periodically reset that counter — “pet the dog” — with the WDR instruction. If the counter ever reaches its timeout before the next WDR, the hardware concludes the program is stuck and forces a system reset:



Three properties make it trustworthy:

- **Independent clock.** The WDT runs from the 1.024 kHz output of the internal ultra-low-power oscillator (OSCULP32K), *not* the main clock. So it keeps counting and can still fire even if the main clock fails — exactly when you most need it.
- **Runs in sleep.** It continues in any sleep mode whose clock source is active.
- **Approximate.** That low-power oscillator is not precise; the datasheet warns the timeout can vary device to device. Choose a period with margin — do not design for the WDT firing at exactly its nominal time.

Eleven periods are available, from 8 ms to 8.2 s, selected by the `PERIOD` field in `WDT.CTRLA`:

PERIOD	timeout	PERIOD	timeout
0x1	8 ms	0x7	0.5 s
0x2	16 ms	0x8	1.0 s
0x3	32 ms	0x9	2.0 s
0x4	64 ms	0xA	4.1 s
0x5	0.128 s	0xB	8.2 s
0x6	0.256 s	0x0	OFF

36.2 Arming the Watchdog

WDT.CTRLA is protected two ways, both of which shape the setup code:

1. **CCP.** Like all configuration-change-protected registers, you must write an unlock key to CPU.CCP and then the protected register within four instructions. The watchdog's key is **IOREG (0xD8)** — note that this is a *different* key from the SPM (0x9D) key the EEPROM/NVM controller uses. The key depends on the register, not on the operation.
2. **Synchronization.** WDT.CTRLA lives in the slow WDT clock domain, so a write must synchronize across clock domains. Writing while SYNCBUSY is set is not allowed — wait for it to clear first.

The companion file `src/ch23_watchdog_reset/wdt_demo.S` arms a 1 s normal-mode watchdog and pets it in the main loop:

```
.equ CCP_IOREG_KEY,    0xD8      /* CPU.CCP key for WDT.CTRLA */
.equ WDT_PERIOD_1KCLK, 0x08      /* ~1.0 s */

/* wait for any pending CTRLA synchronization to finish */
1: lds    r16, WDT_STATUS
   sbrc   r16, WDT_SYNCBUSY_bp
   rjmp   1b

   ldi    r16, CCP_IOREG_KEY
   sts    CPU_CCP, r16           /* arm CCP for the next 4 instructions */
   ldi    r16, WDT_PERIOD_1KCLK
   sts    WDT_CTRLA, r16        /* protected write: enable WDT, ~1 s */

loop:
   wdr                                /* pet the dog */
   /* ... real work here; must complete a lap in well under 1 s ... */
   rjmp   loop
```

The contract you take on by enabling the watchdog: **every path through the main loop must reach a WDR comfortably within the timeout.** That is a design constraint, not a detail. A long blocking operation — a slow EEPROM block write, a busy-wait on a sensor — either needs its own WDR partway through, or a timeout chosen to cover it. The right period is “longer than the slowest legitimate lap, shorter than a user would tolerate a hang,” with margin for the oscillator’s imprecision.

36.2.1 Where to pet — and where not to

Resist the temptation to sprinkle WDR everywhere or to pet from a timer ISR. If the watchdog is petted by an interrupt that keeps firing while the main loop is hung, it defeats the entire purpose — the dog never barks at the actual fault. Pet from the *main control flow*, at a point that is only reached when real forward progress has happened.

36.2.2 Window mode and the lock

Two hardening options exist for fielded products:

- **Window mode.** Setting a non-zero WINDOW field defines a *closed* period at the start of each timeout during which a WDR is illegal and itself triggers a reset. This catches the opposite failure from a hang: code stuck in a tight loop that pets *too often*. The total period becomes closed + open.
- **Lock.** Writing 1 to LOCK in WDT.STATUS freezes the configuration — CTRLA can no longer be changed, so firmware cannot disable the watchdog. If the WDTCFG fuse enabled the watchdog at boot, LOCK is set automatically. This is the belt-and-suspenders configuration for safety-critical designs.

36.3 Knowing Why You Reset: RSTCTRL

After any reset, the program starts from the same vector regardless of cause. The Reset Controller records the cause in `RSTCTRL.RSTFR` (Reset Flag Register), a set of sticky flags you read — and clear — at startup:

RSTFR bit	flag	cause
0	PORF	power-on reset
1	BORF	brown-out reset
2	EXTRF	external reset (RESET pin)
3	WDRF	watchdog reset
4	SWRF	software reset
5	UPDIRF	UPDI (programmer) reset

The flags accumulate and are cleared by writing 1s back (write-one-to-clear). The idiom is: read once, save the cause, clear all flags, then branch:

```
lds    r16, RSTCTRL_RSTFR    /* read all reset flags */
sts    RSTCTRL_RSTFR, r16    /* clear them (write 1 to clear) */
sbrcl  r16, RSTCTRL_WDRF_bp  /* was this a watchdog reset? */
sbi    VPORTA_OUT, 3         /* yes -> show it on the LED */
```

Always clear RSTFR at startup. Because the flags are sticky, an uncleared PORF from the original power-up would still be set after a later watchdog reset, and your cause-decoding logic would see *both* and likely mis-handle the event. Read, save, clear — once, early.

Knowing the cause lets boot behavior adapt: a clean PORF runs full initialization; a WDRF might skip a slow self-test to recover quickly, or log the fault; a BORF might re-check supply-dependent peripherals.

36.4 Software Reset

Sometimes firmware *wants* to reset — to apply a new configuration cleanly, or to bail out of an unrecoverable state in a controlled way. `RSTCTRL.SWRR` provides it: writing 1 to the SWRE bit triggers an immediate system reset. Like the watchdog register, `SWRR` is CCP-protected with the `IOREG` key:

```
software_reset:
    ldi    r16, 0xD8          /* CCP IOREG key */
    sts    CPU_CCP, r16
    ldi    r16, RSTCTRL_SWRE_bm /* = 0x01 */
    sts    RSTCTRL_SWRR, r16  /* resets now; never returns */
    ret
```

A software reset sets `SWRF` in `RSTFR`, so the next boot can distinguish a deliberate restart from a watchdog rescue — useful for telling “I chose to reboot” apart from “I was hung.”

36.5 A Fault-Counter Policy

A watchdog that simply resets a faulting device can produce an infinite loop: boot → hit the same fault → hang → watchdog reset → boot. Recovering *gracefully* means noticing that you keep failing and changing behavior.

The standard pattern couples the reset cause with a small nonvolatile counter (EEPROM is the natural home — see the EEPROM chapter):

```
on boot:
    read RSTFR, save cause, clear RSTFR
    if cause == WDRF (or BORF):
        n = eeprom_read(FAULT_COUNT)
        n = n + 1
        eeprom_update(FAULT_COUNT, n)
        if n >= THRESHOLD:
            enter safe mode          # minimal, known-good behavior; stop retrying
    else if cause == PORF:
        eeprom_update(FAULT_COUNT, 0)  # a clean power-up clears the streak
    ... normal init ...
    eeprom_update(FAULT_COUNT, 0)      # reaching steady state clears it too
```

The shape that matters: **count consecutive fault resets, reset the count on a clean boot or on reaching steady state, and degrade to a safe mode once the count crosses a threshold.** Safe mode might disable the misbehaving peripheral, fall back to default configuration, or sit blinking an error code — anything that stops the device from re-entering the same fault forever. Use `eeprom_update` (not a blind write) so a device that boots cleanly thousands of times does not wear out the counter byte.

36.6 Common Pitfalls

- Petting the watchdog from a timer ISR. If the ISR keeps running while the main loop is hung, the watchdog never fires. Pet from the main control flow.
- Using the wrong CCP key. `WDT.CTRLA` and `RSTCTRL.SWRR` need `I0REG (0xD8)`, not the `NVM SPM key (0x9D)`.
- Writing `WDT.CTRLA` while `SYNCBUSY` is set — the write is disallowed. Wait for it to clear.
- Forgetting to clear `RSTFR` at startup, so stale sticky flags confuse later cause-decoding.
- Choosing a timeout too close to the longest legitimate loop lap and getting spurious resets — made worse by the `OSCULP32K`'s imprecision. Leave margin.
- Expecting the watchdog timeout to be exact. It is a low-power oscillator; design for a range, not a number.
- Resetting on every fault with no counter, so a persistent fault loops forever. Add a fault counter and a safe-mode threshold.

36.7 Summary

Watchdog (ATtiny3217): independent 1.024 kHz oscillator, runs in sleep, survives main-clock failure. 11 periods, 8 ms .. 8.2 s.

Arm: wait `SYNCBUSY=0` -> `CPU_CCP = 0xD8 (I0REG)` -> `WDT_CTRLA = PERIOD` (within 4 instr)

Pet: `WDR` instruction, from the main control flow, every lap < timeout.

Harden: `WINDOW` (closed period) catches pet-too-often; `LOCK` freezes config.

Reset cause: `RSTCTRL_RSTFR` – `PORF BORF EXTRF WDRF SWRF UPDIRF` (write 1 to clear).

Read, save, clear at startup – always.

Software reset: `CPU_CCP = 0xD8` -> `RSTCTRL_SWRR = SWRE(0x01)`. Sets `SWRF`.

Recovery policy: count consecutive fault resets in EEPROM; clear on clean boot / steady state; degrade to safe mode past a threshold.

The watchdog is the difference between a product that needs a power-cycle and one that rescues itself. Arming it is a few protected writes; using it well is a design habit — bounded loop laps, a single honest pet point, and a fault policy that stops the device from failing the same way forever.

36.8 Exercises

1. Change `wdt_demo.S` to a 0.25 s timeout. Which `PERIOD` value do you write, and how does the main loop's timing budget change?
2. Add a deliberate hang: after 10 loop iterations, branch into an infinite loop with no WDR. Predict what happens, then confirm by checking `WDRF` on the next boot.
3. Enable Window mode with an 8 ms closed window and a 1 s open period. What does a WDR issued 4 ms into the period do? Write the `CTRLA` value.
4. Write the startup reset-decode routine that sets a register to a small code (0=POR, 1=BOR, 2=EXT, 3=WDT, 4=SW) based on `RSTFR`, handling the case where more than one flag is set by priority.
5. Implement `software_reset` and call it after writing a new configuration to EEPROM. Verify the next boot reports `SWRF` and not `WDRF`.
6. Implement the fault-counter policy against the EEPROM routines from the EEPROM chapter. Use a 4-byte wear-leveled counter and define a safe mode that disables the LED-driving peripheral after three consecutive watchdog resets.
7. Why does the watchdog run from `OSCULP32K` instead of the main clock? Give a concrete failure the independent clock lets the watchdog catch that a main-clock-derived timer could not.

Chapter 37

Defensive Firmware

The previous three chapters each added one safety mechanism: EEPROM integrity checks, conservative fuse configuration, and watchdog recovery. Defensive firmware is the discipline of *combining* them — designing a system that detects its own corruption, refuses to run code it cannot trust, initializes hardware into known states, and degrades predictably instead of failing in surprising ways.

This chapter covers the pieces not yet seen: verifying Flash integrity with the CRCSCAN peripheral, testing RAM at power-on, embedding version and identity metadata, initializing peripherals defensively, and orchestrating a boot sequence that pulls the watchdog, reset-cause, and integrity checks into one coherent startup. Where a topic was covered earlier — fault counters, EEPROM checksums — this chapter references it rather than repeating it.

37.1 Why Defensive Firmware

A microcontroller in the field faces hazards a desktop program never does: supply brown-outs mid-write, electromagnetic interference flipping a RAM bit, Flash that degrades after years of heat, a bug that corrupts a pointer. You cannot prevent all of these. What you *can* do is ensure that when something goes wrong, the device notices and responds — rather than executing corrupted code as if nothing happened.

The mindset has three habits:

- **Trust nothing you have not verified.** Check Flash before running it; check EEPROM before using it; check that a reset was clean before assuming state.
 - **Make every state explicit.** Do not rely on a peripheral's reset value or on a variable being whatever it was; set what you depend on.
 - **Have a fallback for every check.** A detection with no defined response is just a more elaborate crash.
-

37.2 Verifying Flash: the CRCSCAN Peripheral

The most direct threat is corrupted program memory — a Flash cell that has degraded, or a botched firmware update. The ATtiny3217 has dedicated hardware for catching it: **CRCSCAN**, which computes a CRC-16 over a Flash section and compares it against a checksum stored in the last bytes of that section.

The CRC chapter covers the peripheral in depth — the register map, the CRC-16-CCITT algorithm, where the pre-calculated checksum must be placed, the three-cycle start delay,

the NMI-on-failure option, and the fuse-driven startup scan. This section is about *using* it defensively; refer there for the mechanics. The two facts that drive the defensive design choice:

- **A fuse-driven pre-boot scan is the strongest guarantee.** The CRCSRC field in FUSE.SYSCFG0 (see the Fuses chapter) makes the hardware verify Flash *before the CPU executes a single instruction*; on failure the CPU never starts. Corrupted firmware cannot run at all, and it costs nothing at run time. This is the first choice for a fielded device — prefer it over any run-time check.
- **A software scan covers what the pre-boot scan cannot** — re-verifying a section *after* a run-time update, or periodically while running. That is the niche for an in-application scan.

37.2.1 Applying a run-time scan

The companion file `src/ch24_defensive/crc_selfcheck.S` puts the CRC chapter’s scan sequence to work as a defensive check — select the section, enable, wait for BUSY to clear (Priority mode stalls the CPU, so this usually falls straight through), and branch on OK:

```
ldi    r16, CRCSCAN_SRC_FLASH    /* entire Flash; Priority mode = 0 */
sts    CRCSCAN_CTRLB, r16
ldi    r16, CRCSCAN_ENABLE_bm    /* start the scan */
sts    CRCSCAN_CTRLA, r16
1: lds    r16, CRCSCAN_STATUS      /* wait for BUSY to clear */
sbrc   r16, CRCSCAN_BUSY_bp
rjmp   1b
sbrc   r16, CRCSCAN_OK_bp         /* OK = 1 -> Flash verified */
sbi    VPORTA_OUT, 3             /* (LED as a visible pass indicator) */
```

The defensive caveat to remember: **OK = 1 requires the appended checksum.** The example assembles and the logic is right, but on hardware OK stays 0 until a post-build tool has written a valid CRC into the section’s last bytes. A failing self-check is therefore ambiguous — it could mean genuine corruption *or* a missing checksum step — so a real product must guarantee the checksum is appended in its build, then treat OK = 0 as a true integrity failure and route to safe mode. For the “fail loud” variant, arm NMIIEN so a failure traps to the non-maskable interrupt instead of returning (mechanics in the CRC chapter).

37.3 Testing RAM

Flash holds the code; RAM holds everything the code works with — the stack, variables, buffers. CRCSCAN proves the code is intact, but says nothing about the memory that code runs in. A stuck bit, a cell that cannot hold a one, or two addresses shorted together will corrupt data silently. A power-on RAM test is the defensive answer: before trusting working memory, prove every cell can hold both states and is independent of its neighbours.

A naive test — write a value, read it back — catches almost nothing. The cell holds the value you just wrote because it is still on the bus; the test passes even on faulty memory. Real RAM faults are about *relationships*:

Stuck-at fault: a cell is frozen at 0 or 1 regardless of writes.
Coupling fault: writing one cell flips another.
Address fault: two addresses decode to the same physical cell.

The industry-standard test is **March C-**, a sequence of six “march elements” that walk the whole region writing and verifying in alternating directions. The alternating order is the point: it exposes coupling and address-order faults that a single sweep would step right over.

M0	any direction	write 0	(initialise every cell to 0)
M1	ascending	read 0, write 1	(each cell was 0; now make it 1)
M2	ascending	read 1, write 0	
M3	descending	read 0, write 1	
M4	descending	read 1, write 0	
M5	any direction	read 0	(final confirmation)

Each cell is read expecting the value the previous element left, then written to the opposite value. Any read that does not match the expected pattern is a detected fault. Using 0x00 and 0xFF as the two patterns exercises all eight bits of every byte at once.

37.3.1 The Catch-22 of Testing Your Own Memory

A RAM test is **destructive** — it overwrites the region — and the obvious implementation uses RCALL, which pushes a return address onto the stack, which is RAM. You cannot march over the bytes your own stack is sitting in without destroying the return address and crashing.

The defensive resolutions, in order of thoroughness:

Test a region that is not the stack:

Simplest. Test buffers and the .bss/.data region, leave the stack alone.

Test RAM in halves:

Put the stack in half A, march half B; move the stack to B, march A.

Stackless test before startup:

Run a register-only march (no RCALL, no pushes) out of the reset handler before the stack pointer is even set, then test the whole array.

The example below takes the first approach: it tests a .bss buffer, well away from the stack near RAMEND.

37.3.2 March C- in Assembly

The full source is [ram_test.S](#). The core routine marches an arbitrary region given a start pointer and a length, and returns pass/fail:

In: X = region start (SRAM), r25:r24 = length in bytes (> 0)

Out: r24 = 0 pass, 1 fail

The six elements call three small helpers — an ascending fill, an ascending/descending read-modify-write, and a final read-only check. Each read/modify/write helper returns the verdict in the carry flag (SEC on mismatch), so the caller branches with a single BRCS:

```
.global ram_march
ram_march:
    movw    r20, r26                /* save start -> r21:r20 */
    movw    r22, r24                /* save length -> r23:r22 */

    ldi     r19, 0x00               /* M0: write 0 ascending */
    rcall   fill_asc

    ldi     r18, 0x00               /* M1: up    r0, w1 */
    ldi     r19, 0xFF
    rcall   rmw_asc
    brcs    .Lfail

    ldi     r18, 0xFF               /* M2: up    r1, w0 */
    ldi     r19, 0x00
    rcall   rmw_asc
```

```

brcs .Lfail

ldi r18, 0x00          /* M3: down r0, w1 */
ldi r19, 0xFF
rcall rmw_desc
brcs .Lfail

ldi r18, 0xFF          /* M4: down r1, w0 */
ldi r19, 0x00
rcall rmw_desc
brcs .Lfail

ldi r18, 0x00          /* M5: read 0 ascending */
rcall check_asc
brcs .Lfail

ldi r24, 0             /* every element passed */
ret
.Lfail:
ldi r24, 1
ret

```

The ascending read/modify/write helper is the heart of it — read the cell, compare against the expected pattern, and only on a match write the new pattern and advance:

```

rmw_asc:
    movw r26, r20          /* X = saved start */
    movw r24, r22          /* counter = saved length */
.Lra:
    ld r16, X              /* read current cell */
    cp r16, r18            /* == expected pattern? */
    brne .Lrw_fail
    st X+, r19             /* write new pattern, X++ */
    sbiw r24, 1
    brne .Lra
    clc                   /* whole pass matched */
    ret
.Lrw_fail:
    sec                   /* a cell held the wrong value */
    ret

```

The descending variant is identical except it starts the pointer at `start + length - 1` and walks down. Reading the cell *before* overwriting it is what makes the test meaningful: it confirms the value the previous element wrote actually survived.

37.3.3 Wiring It Into the Boot

The demo runs the march at power-on and lights LED0 only if RAM passed:

```

ldi XL, lo8(test_buf)
ldi XH, hi8(test_buf)
ldi r24, lo8(TEST_LEN)
ldi r25, hi8(TEST_LEN)
rcall ram_march
sts ram_result, r24      /* 0 = pass, 1 = fail */
tst r24
brne .Lhalt              /* fail: leave LED off, stop */
cbi VPORTA_OUT, LED_BIT /* pass: LED on */

```

In a real product the failing branch goes to safe mode, not a halt — but the shape is the

same as the other defensive checks: detect, then route to a defined response. Because the test is destructive, it belongs *first*, before any RAM holds anything worth keeping.

A march is the thorough choice; for a faster boot a single **walking-ones** pass (write a 1 in each bit position, confirm only that bit reads back) or an **address-uniqueness** test (write each cell a value derived from its address, read all back) catches the common stuck-bit and addressing faults at a fraction of the cost. March C- is the one to reach for when a latent RAM fault is a safety problem.

37.4 Version and Identity Metadata

When a defensive device logs a fault or talks to a host, it needs to say *which* firmware and *which* unit. Two sources, no external storage required:

Factory device ID and serial number live in the Signature Row, memory-mapped and read with plain LDS:

```
lds    r16, SIGROW_DEVICEID0    /* 0x1100: manufacturer/device id bytes */
lds    r17, SIGROW_DEVICEID1
lds    r18, SIGROW_DEVICEID2
lds    r19, SIGROW_SERNUM0      /* 0x1103: start of the unique serial */
```

The device ID confirms you are running on the part you think you are (a cheap guard against a firmware image flashed to the wrong variant). The serial number is a factory-unique value — useful as a device identifier in logs or on a bus without burning your own EEPROM bytes.

Your firmware version should be embedded at a known place so a host — or a bootloader — can read it without running the application. Put a small version record in a dedicated Flash section at a fixed address (via the linker), so it can be inspected even when the main code is suspect. Pair it with the CRC above: a host reads the version, then trusts it only if the CRC of the section checks out.

37.5 Safe Peripheral Initialization

Reset values are documented, but defensive code does not lean on them. Two programs sharing a peripheral, a soft reset that does not clear every register, a future chip revision with a different default — any of these can leave a register holding something other than what you assumed. The habits:

Configure before enabling. Set a peripheral’s mode, source, and parameters *first*, and flip its ENABLE bit last. Enabling first can produce a glitch — a spurious edge on an output, a conversion with the wrong reference — in the window before configuration lands. Every example in this book follows this order deliberately.

Set what you depend on; do not assume. If your code needs a pin to be an input with the buffer disabled, write that explicitly, even if it is the reset state. The cost is a few instructions at boot; the benefit is that the code is correct regardless of how it was entered.

Disable what you do not use. Unused peripherals left enabled waste power and can drive shared pins unexpectedly. An explicit “everything off” baseline before selectively enabling what you need is easier to reason about than inheriting a partially-configured state.

Verify writes that matter. For a configuration where a wrong value is dangerous — not for routine register writes — read the register back and confirm it took. This catches a

bus fault or a locked register (for instance, a watchdog whose LOCK bit silently dropped your CTRLA write).

Respect synchronization and protection. Honor SYNCBUSY before writing clock-domain-crossing registers, and follow the CCP unlock sequence for protected ones — skipping either leaves the register silently unchanged, which a defensive init must not assume succeeded.

37.6 Orchestrating a Defensive Boot

The mechanisms compose into a startup sequence. Each step has a check and a fallback; nothing downstream runs until the things it depends on are verified:

```

reset
|
v
[1] read RSTCTRL.RSTFR -> save cause, clear flags          (Watchdog chapter)
|
v
[2] Flash integrity: CRCSCAN OK? (or rely on the fuse pre-boot scan)
    no -> safe mode: drive outputs safe, signal error, halt
    | yes
    v
[2b] RAM test: March C- over working memory (destructive -> run early)
     fail -> safe mode: RAM is unreliable, do not run on it
     | pass
     v
[3] reset cause == WDRF / BORF ?
     yes -> bump EEPROM fault counter (update, not blind write)
         counter >= threshold -> safe mode (stop retrying) (Watchdog ch.)
     |
     v
[4] validate EEPROM config block via its checksum          (EEPROM chapter)
     bad -> load defaults, optionally re-write the block
     |
     v
[5] safe peripheral init: disable-all -> configure -> enable
     |
     v
[6] arm the watchdog, enter the main loop (which pets it) (Watchdog chapter)

```

The structure is the lesson, not any single line. Read *why* you reset before assuming state. Verify code before running it and data before using it. Count repeated failures so a persistent fault degrades to a safe mode instead of looping forever. Initialize into known states. And only then arm the watchdog and start real work. A device built this way turns the failures it cannot prevent into events it survives.

37.7 Common Pitfalls

- Enabling CRCSCAN and expecting OK = 1 without appending the checksum at the section end. The scan has nothing valid to compare against.
- Confusing the CRCSCAN SRC setting with the CRCSRC fuse. The fuse drives the *pre-boot* scan; CTRLB.SRC selects the section for a *software* scan.
- Arming NMIEN and then trying to clear it in software. It is clearable only by a system reset — that is the point.

- Trusting peripheral reset values across a soft or watchdog reset, which may not re-initialize every register the way a power-on reset does.
 - Enabling a peripheral before configuring it, producing a startup glitch.
 - A detection with no fallback — verifying Flash or config but having no defined behavior for the failing case, so the device crashes anyway.
 - Putting integrity checks so deep in the boot flow that corrupted code has already run before the check happens. Verify early.
 - A “RAM test” that just writes then immediately reads a cell — it passes on faulty memory because the value is still on the bus. Use a march.
 - Marching over the region that holds your own stack: the RCALL/PUSH return address is destroyed mid-test. Test non-stack regions, or test in halves.
-

37.8 Summary

Defensive firmware = compose the per-peripheral safety nets into one boot policy.

Flash integrity (CRCSCAN — mechanics in the CRC chapter):

- prefer the fuse pre-boot scan (CRCSRC in SYSCFG0): CPU never starts on failure
- software scan (CTRLB.SRC -> CTRLA.ENABLE -> poll BUSY -> check OK) for re-checking after an update or periodically; OK=1 needs the appended checksum

RAM integrity (March C-, ram_test.S): six march elements (w0; up r0w1; up r1w0; down r0w1; down r1w0; r0) catch stuck-at, coupling, and address faults a naive write-then-read misses. Destructive and stack-using: run early, on a non-stack region (or test in halves / stackless).

Identity/version: SIGROW DEVICEID0..2 (0x1100) and SERNUM0 (0x1103); embed a firmware version record at a fixed Flash address, guarded by the CRC.

Safe init: configure-before-enable; set what you depend on; disable the unused; verify critical writes; honor SYNCBUSY and CCP.

Defensive boot: reset cause -> Flash CRC -> RAM test -> fault counter -> EEPROM config check -> safe peripheral init -> arm watchdog -> main loop.

Defensive firmware is less about any one register than about a stance: assume the hardware and your own memory can be wrong, check before you trust, and define what happens when a check fails. The ATtiny3217 gives you the pieces — CRCSCAN, the reset controller, the watchdog, EEPROM integrity, conservative fuses; this chapter is about wiring them into a system that fails safe.

37.9 Exercises

1. Modify `crc_selfcheck.S` to scan only the boot section (SRC = BOOT = 0x2) instead of the whole Flash. Which fuse defines where that section ends, and where would its checksum be stored?
2. Add NMIEN to the scan and write an NMI handler at the NMI vector that drives PA3 low and halts. Why can the handler not simply clear the condition and return to normal execution?
3. Read all three SIGROW_DEVICEID bytes and compare them against the known ATtiny3217 signature. Branch to an error state if they do not match. What does a mismatch most likely indicate?
4. Sketch a linker arrangement that places a 4-byte firmware version record (major, minor, patch, reserved) at a fixed Flash address. How would a host read it, and how does the

CRC make that read trustworthy?

5. Take the defensive-boot outline and implement steps 1–3 in assembly, reusing the reset-cause decode and the EEPROM fault counter from the Watchdog and EEPROM chapters. Define a concrete safe mode (e.g. slow LED blink, peripherals disabled).
6. The chapter says to verify “writes that matter” but not every write. Give two register writes where read-back verification is justified and two where it is unnecessary overhead, and explain the difference.
7. Explain why the fuse-driven pre-boot CRC scan is a stronger guarantee than the software scan in `crc_selfcheck.S`. What class of failure can the software scan never catch, no matter where you place it?
8. A “write 0x55, read it back, write 0xAA, read it back” test passes on a cell that is permanently stuck at neither value yet shorted to its neighbour. Walk through which March C- element catches that coupling fault and why the alternating direction (M3/M4 descending) matters.
9. `ram_march` is destructive and uses `RCALL`. Rewrite the M0 fill and M1 as a stackless, register-only loop suitable for running from the reset handler before `SPL/SPH` are set. What must you avoid using, and how do you return?
10. Extend the demo to test all of SRAM in two halves: place the stack in the upper half, march the lower half, relocate the stack, then march the upper half. What is the smallest region you can never test this way?

Chapter 38

C and Assembly Integration

Most real firmware is not all assembly. The productive pattern is C for the bulk of the program — state machines, protocol handling, the parts where clarity matters more than cycles — with assembly reserved for the handful of routines where you need exact timing or instruction-level control. Making the two halves work together cleanly is the subject of this chapter.

The contract that lets a C compiler and your hand-written assembly call each other is the **Application Binary Interface (ABI)**: which registers carry arguments, where return values go, and which registers a function may clobber versus must preserve. Get the ABI right and the two languages interleave seamlessly; get it wrong and you get corruption that is maddening to debug. Everything here is verified against the `avr-gcc` this book uses (16.1.0) by compiling C and reading the generated code.

38.1 The `avr-gcc` ABI

38.1.1 Register classes

Every register falls into one of two classes, and the distinction is the whole game:

Call-used (caller-saved, "scratch"): `r0`, `r18-r27`, `r30`, `r31`

A function may freely modify these. If the caller needs a value kept across a call, the caller must save it.

Call-saved (callee-saved, "preserved"): `r2-r17`, `r28`, `r29`

A function that uses any of these must restore them before returning. The caller can assume they survive a call.

Special:

`r0` temporary — clobbered by some instructions (`MUL` result low, `LPM`); scratch.
`r1` the zero-register — must be 0 at all times; restore with `CLR r1` after `MUL`.

If your assembly routine touches `r2-r17`, `r28`, or `r29`, push them in the prologue and pop them in the epilogue. If it only uses scratch registers (`r18-r27`, `r30`, `r31`, `r0`), it needs no save/restore at all — which is why small leaf routines are so cheap.

38.1.2 Argument passing

Arguments are allocated **starting at `r25` and working downward**, each argument rounded up to an even number of registers (so everything starts on an even register). A value wider than one byte occupies a register *group*, low byte in the lower-numbered register.

Compiling `callee(uint16_t a, uint16_t b, uint8_t c, uint32_t d)` and reading the generated call site shows the allocation exactly:

```
a (16-bit) -> r24:r25      (low:high)
b (16-bit) -> r22:r23
c (8-bit)  -> r20           (rounded to the r20:r21 slot; value in r20)
d (32-bit) -> r16:r19      (r16 low .. r19 high)
```

An 8-bit argument still consumes its even-aligned slot, so the next argument starts two registers down. Arguments that run past r8 are passed on the stack, but that is rare — most functions fit in registers.

38.1.3 Return values

```
8-bit      -> r24
16-bit     -> r24:r25
32-bit     -> r22:r25
64-bit     -> r18:r25
```

A pointer is a 16-bit value, so it is returned in r24:r25 like any other 16-bit type.

38.2 Calling Assembly from C

To expose an assembly routine to C, give it a `.global` symbol and follow the ABI. The companion file `src/ch25_c_asm_integration/asm_funcs.S` defines several; the simplest adds two 16-bit values:

```
/* uint16_t asm_add16(uint16_t a, uint16_t b)
 *   a = r25:r24, b = r23:r22, return r25:r24 */
.global asm_add16
asm_add16:
    add    r24, r22
    adc    r25, r23
    ret
```

It clobbers only r24/r25 (call-used) and the result lands where C expects a 16-bit return, so there is nothing to save. On the C side you only need a prototype that matches the ABI the assembly assumes:

```
extern uint16_t asm_add16(uint16_t a, uint16_t b);

uint16_t s = asm_add16(0x1234, 0x1111); /* -> 0x2345 */
```

Build the two together — the linker resolves the symbol:

```
$ avr-gcc -mmcu=attiny3217 -Os driver.c asm_funcs.S -o app.elf
```

The prototype is load-bearing. C uses it to decide which registers to load the arguments into; if the prototype says `uint8_t` where the assembly expects a 16-bit value in a register pair, the high byte is never set and you read garbage. The compiler cannot check your assembly against your claim — matching them is your responsibility.

38.3 Passing Pointers and Arrays

A pointer is just a 16-bit argument, so it arrives in a register pair and you move it into a pointer register (X, Y, or Z) to dereference. Arrays decay to a pointer to their first element, exactly as in C. `asm_fill` takes a buffer pointer, a value, and a length:

```

/* void asm_fill(uint8_t *p, uint8_t val, uint8_t n)
 *   p = r25:r24, val = r22, n = r20 */
.global asm_fill
asm_fill:
    movw    r26, r24           /* X = p */
    tst     r20
    breq    2f
1:  st      X+, r22
    dec     r20
    brne    1b
2:  ret

extern void asm_fill(uint8_t *p, uint8_t val, uint8_t n);

uint8_t buf[8];
asm_fill(buf, 0xAA, sizeof buf); /* C passes &buf[0], 0xAA, 8 */

```

asm_sum shows the mirror case — reading through a pointer and returning a scalar — plus the one housekeeping rule that trips people up:

```

/* uint8_t asm_sum(const uint8_t *p, uint8_t n) ; p=r25:r24, n=r22, ret r24 */
.global asm_sum
asm_sum:
    movw    r30, r24           /* Z = p */
    clr     r24                /* accumulator = return register */
    tst     r22
    breq    4f
3:  ld      r0, Z+
    add     r24, r0
    dec     r22
    brne    3b
4:  clr     r1                 /* keep the zero-register zero on the way out */
    ret

```

Because this routine borrows r0, and because the C code on either side assumes r1 is always zero, the discipline is: scratch registers are free to use, but **leave r1 as zero** when you return. Any routine that runs MUL (which writes r1:r0) must clr r1 before returning to C, or the C side’s next implicit “r1 is zero” assumption breaks in a way that is very hard to trace.

To return a *struct* or multiple values, the usual approach is to pass a pointer to a caller-provided struct and have the assembly write through it — the same as passing any other pointer.

38.4 Calling C from Assembly

The reverse direction works too: load the arguments into the ABI registers and rcall the C function, reading the result from the return registers.

```

ldi    r24, lo8(1000)        /* arg: uint16_t period = 1000 */
ldi    r25, hi8(1000)
rcall  set_period            /* a C function: void set_period(uint16_t) */

```

Two preconditions matter when *your* assembly is the top-level environment (for example a -nostartfiles program that then calls into C): the C code assumes the **stack pointer is initialized** and **r1 is zero**. The normal C startup code (crt) sets both up before main. If you bypass that startup, establish them yourself — set SP to RAMEND and clr r1 — before calling any C function, or the first stack push or zero-register use will misbehave.

38.5 Inline Assembly

For a single instruction or a tiny sequence, a whole separate .S file is overkill. GNU C **inline assembly** drops instructions into a C function while letting the compiler handle register allocation through a constraint system:

```
__asm__ volatile ("instructions"
                  : output operands
                  : input operands
                  : clobber list);
```

A minimal example swaps the nibbles of a byte in place:

```
static inline uint8_t nibble_swap(uint8_t x){
    __asm__ ("swap %0" : "+r"(x));
    return x;
}
```

%0 refers to the first operand. "+r"(x) says: allocate any register for x, and it is both read and written (+). The compiler picks the register and emits the moves around your instruction.

38.5.1 Constraints

The constraint letter tells the compiler what kind of operand the instruction needs. The ones you reach for on AVR:

```
r  any register (r0-r31)
d  an "ldi-capable" register (r16-r31)      - required by LDI, ANDI, etc.
e  a pointer register: X, Y, or Z (r26/r28/r30)
w  an adiw-capable even pair: r24, r26, r28, r30
M  a compile-time 8-bit constant (0-255)
I  a 6-bit positive constant (0-63), e.g. for IN/OUT/ADIW
```

Modifiers on output operands: = write-only, + read-write. So "=d"(out) is a write-only operand that must land in an ldi-capable register.

A two-operand form with distinct input and output:

```
uint8_t hi;
__asm__ ("mov %0, %1\n\t"
        "swap %0\n\t"
        "andi %0, 0x0F"
        : "=d"(hi)          /* %0 out, ldi-capable for the andi */
        : "r"(value));      /* %1 in, any register */
```

38.5.2 Clobbers, volatile, and "memory"

The clobber list names what your code disturbs that the compiler did not allocate:

- A scratch register you used explicitly (e.g. "r0").
- "cc" if you change the condition flags — many AVR instructions do, and the compiler must know not to rely on flags across your block.
- "memory" if your code reads or writes memory the compiler cannot see (an arbitrary pointer, a peripheral register). It forces the compiler to flush values to memory before the block and reload after, and acts as a barrier against reordering.

volatile tells the compiler the statement has a side effect and must not be deleted (even if its outputs look unused) or moved. Use it for anything that touches hardware, busy-waits,

or must execute exactly where written — a nop delay, an sei, a peripheral poke. A pure computation with declared outputs does *not* need volatile; the data dependency keeps it alive.

```
__asm__ volatile (" ::: \"memory\"); /* a pure compiler barrier */
```

The empty-template, "memory"-clobber form above emits no instructions but stops the compiler from moving memory accesses across that point — useful for ordering around hardware without a heavier cli/sei.

38.6 Sharing Data and Linker Symbols

Symbols cross the C/assembly boundary in both directions. A .global label in assembly is callable or readable from C; a C extern declaration names it.

A variable defined in C is reachable from assembly by its name (memory-mapped, so LDS/STS):

```
volatile uint8_t shared_flag; /* C definition */

lds    r16, shared_flag      /* read the C variable from assembly */
ori    r16, 0x01
sts    shared_flag, r16
```

Going the other way, data defined in an assembly .section with a .global label is declared extern in C. This is also how you place something at a specific address or in a specific memory: define a named section in assembly (or via a variable attribute in C) and position it with the linker script (see the linker appendix). The symbol is the contract; the types on each side must agree, because nothing checks them for you.

38.7 Common Pitfalls

- A C prototype that does not match the assembly's register expectations — the classic being uint8_t versus uint16_t, which leaves a high byte unset.
- Clobbering a call-saved register (r2-r17, r28, r29) without saving it, corrupting the caller's state.
- Returning to C after a MUL without clr r1, breaking the zero-register invariant the compiler depends on.
- Forgetting "cc" in an inline-asm clobber list when your instructions change flags, so the compiler trusts a stale condition.
- Omitting "memory" when inline asm touches memory the compiler cannot see, so it caches a value across your block.
- Leaving off volatile on a hardware or timing-critical inline asm, and finding the optimizer deleted or moved it.
- Calling a C function from a -nostartfiles assembly program without first setting up SP and clearing r1.
- Returning a 16-bit value only in r24 and leaving r25 stale.

38.8 Summary

avr-gcc ABI (verified against avr-gcc 16.1.0):

Call-used (scratch): r0, r18-r27, r30, r31	– free to clobber
Call-saved (preserve): r2-r17, r28, r29	– save/restore if used

`r1 = 0` always (`clr r1` after `MUL`). `r0` = temporary.

Args: allocated `r25` downward, even-aligned groups, low byte in lower register.
 16-bit `a, b, ...` -> `r24:r25`, `r22:r23`, ... 8-bit takes the even reg.

Return: 8b `r24` | 16b `r24:r25` | 32b `r22:r25` | 64b `r18:r25`. Pointer = 16-bit.

Call asm from C: `.global label + matching extern prototype`; obey the ABI.

Call C from asm: load arg regs, `rcall`, read return regs; ensure `SP` set + `r1=0`.

Pointers: arrive in a register pair; `movw` into `X/Y/Z` to dereference.

Inline asm: `__asm__ volatile ("...": out : in : clobbers);`
 constraints `r/d/e/w/M/I` ; modifiers = (write) + (read-write)
 clobbers: "cc" (flags), "memory" (unseen accesses), explicit regs
 volatile: keep it, do not move it (hardware / timing / side effects)

The ABI is a short list of rules, but it is unforgiving: the compiler trusts your prototype and your promise to preserve call-saved registers, and it cannot check either. Follow the register classes, keep `r1` zero, match prototypes to expectations, and the boundary between C and assembly becomes invisible — each language doing the part it is best at.

38.9 Exercises

1. Write `asm_max16(uint16_t a, uint16_t b)` returning the larger value. Which registers hold the arguments and the result, and does it need to save any call-saved registers?
2. Compile a C function `int16_t scale(int16_t x, uint8_t shift)` at `-Os` and read the generated assembly. Confirm `x` is in `r24:r25` and `shift` is in `r22`. Where is the return value?
3. Write `asm_strlen(const char *s)` returning a `uint8_t` length. Which pointer register do you use, and what is the return register?
4. Implement `nibble_swap` from the chapter, then add a version using a separate input and output operand ("`=d`" / "`r`"). Disassemble both and compare.
5. Add a one-cycle `nop`-based delay as `volatile` inline asm. Remove the `volatile` and explain, from the disassembly, what the optimizer does to it.
6. A routine multiplies two bytes with `MUL` and returns the low byte to C but forgets `clr r1`. Describe a concrete bug this causes in C code compiled later in the program, and why it is hard to localize.
7. Define a `volatile uint16_t` in C and write an assembly routine that increments it atomically (with interrupts disabled). What does the "memory" clobber accomplish if you instead did the increment with inline asm?
8. Call a C function `void log_event(uint8_t code)` from an assembly ISR. What must the ISR do that an ordinary `rcall` from C would not, regarding call-used registers and `SREG`?

Chapter 39

Polynomial and Rational Approximation

A microcontroller with no floating-point unit still has to compute sines, square roots, logarithms, and the rest. The answer is approximation: replace a function that has no closed-form integer recipe with a polynomial or a ratio of polynomials that a fixed-point CPU *can* evaluate, accurate enough for the job and cheap enough for the cycle budget.

Chapter 15 introduced Horner's method for evaluating a polynomial efficiently. This chapter is about choosing *which* polynomial — why the textbook Taylor series is usually the wrong one, how a minimax fit does better with the same number of terms, how range reduction shrinks the problem before you spend a single coefficient, and when a *rational* approximation (one division) beats piling on more polynomial terms. Every error figure here is measured, not asserted: the companion files build and a host model sweeps the full input range.

39.1 Why Not Taylor

The Taylor series is the approximation everyone learns first, and it is the wrong default for embedded work. Taylor expands a function about a *single point* and is extremely accurate there — but the error grows as you move away, piling up at the far end of your interval. You are paying for accuracy near $x = 0$ that you do not need, while the worst case (the end of the range) goes uncontrolled.

Concretely, approximating $\sin(x)$ on $[0, \pi/2]$ with a degree-5 odd polynomial:

	max error over $[0, \pi/2]$	
Taylor $(x - x^3/6 + x^5/120)$	4.5e-3	(~148 LSB of Q15)
Minimax (least-squares fit)	1.6e-4	(~5 LSB of Q15)

Same three terms, same evaluation cost — and the minimax fit is roughly **28×** more accurate, because it spreads the error evenly across the whole interval instead of dumping it all at the end.

39.2 Minimax and Equioscillation

The best polynomial of a given degree, in the sense that matters here, is the one that *minimizes the maximum error* over the interval — the **minimax** polynomial. Its defining

feature is **equioscillation**: the error wiggles up and down, touching its maximum magnitude the same number of times with alternating sign. No local stretch of the interval is worse than any other, so there is no slack left to trade away.

You do not derive these coefficients on the device. You compute them once, on a host, with the **Remez exchange algorithm** (which converges to the true minimax polynomial) — or, for a smooth function, a dense **least-squares fit**, which lands very close to minimax and is a one-liner with a numerical library. Then you round the coefficients to your fixed-point format and bake them into the firmware as constants. The sine coefficients used below came from a least-squares fit on a 20,000-point grid:

```
sin(x) ~= c1*x + c3*x^3 + c5*x^5
c1 = +0.999786    c3 = -0.165833    c5 = +0.007568
```

A related host-side trick is **Chebyshev economization**: take a longer Taylor series and re-express it in Chebyshev polynomials, where the high-order terms are tiny and can be dropped with a known, bounded penalty — a quick way to shave a term without a full Remez run.

39.3 Range Reduction

The single most effective accuracy tool is not a better polynomial — it is a *smaller interval*. A polynomial that must be accurate over $[0, 2\pi]$ needs many terms; one that only has to cover $[0, \pi/2]$ needs few, because periodicity and symmetry let you reconstruct the rest:

```
sin(x):  reduce x mod 2π, then use quadrant symmetry to fold into [0, π/2]
2^x:     split x into integer + fractional parts; the integer part is a shift,
        only the fraction in [0,1) needs the polynomial
log2(x): normalize x = m·2^e (m in [1,2)); e is the exponent, approximate log2(m)
```

Always reduce first. A degree-3 polynomial on a well-reduced argument routinely beats a degree-7 polynomial applied to the raw input — for fewer cycles.

39.4 Evaluating in Fixed Point

Once you have coefficients, evaluating them in integer arithmetic is the careful part. Three things to track:

- **Q-format and overflow.** Every value carries an implied binary point. A product of a Q_m and a Q_n value is $Q_{(m+n)}$; you must shift it back to keep the running result in range. Intermediate values can exceed ± 1 even when the final result does not, so pick formats for the *intermediates*, not just the answer.
- **Rounding direction.** A right shift is the cheap “divide by a power of two,” but an *arithmetic* shift rounds toward $-\infty$, not toward zero. That matters for signed values; the host model must use the same rule as the assembly or the two will disagree by an LSB.
- **Saturation.** A minimax polynomial can overshoot the true function slightly at the extremes. If the true value is near the top of your format, that overshoot wraps. Clamp the result.

39.4.1 Worked example: Q15 sine

src/ch26_approx/q15_sin.S takes an angle in **Q14** (so the range $[0, \pi/2]$ fits, since $\pi/2 \approx 1.57 > 1$) and returns $\sin(x)$ in **Q15**. The coefficients are stored in Q15, and the evaluation is Horner with explicit re-scaling at each step:

```

x2 = (x * x) >> 15      ; Q14*Q14 = Q28 -> Q13
p  = c5                  ; Q15
p  = c3 + (p*x2) >> 13   ; Q15*Q13 = Q28 -> Q15
p  = c1 + (p*x2) >> 13   ; Q15
r  = (p*x) >> 14        ; Q15*Q14 = Q29 -> Q15, then clamp

```

The arithmetic rests on two helpers: a signed 16×16→32 multiply (the standard AVR201 sequence, which uses `mulsu` and the carry flag to sign-extend the partial products) and a 32-bit arithmetic shift. The core of the routine:

```

/* x2 = (x*x) >> 15 */
movw r22, r14          /* x in r15:r14 */
movw r20, r14
rcall smul32           /* r19:r16 = x*x (signed 32-bit) */
ldi r20, 15
rcall asr32
movw r12, r16          /* x2 -> r13:r12 */

/* p = c3 + (p*x2)>>13, with p starting at c5 ... (repeated for c1) */
movw r22, r24
movw r20, r12
rcall smul32
ldi r20, 13
rcall asr32
ldi r24, lo8(-5434)    /* c3 in Q15 */
ldi r25, hi8(-5434)
add r24, r16
adc r25, r17

```

A degree-5 minimax polynomial overshoots 1.0 by just **4 LSB** near $x = \pi/2$, which would wrap the Q15 result negative — so the routine ends with a saturating clamp:

```

tst r25                /* result is non-negative on [0, pi/2]; */
brpl 1f                /* bit 15 set means it crept above Q15 max */
ldi r24, 0xFF
ldi r25, 0x7F          /* clamp to 0x7FFF */
1:

```

Verification. The host model `sinref.c` reproduces these exact integer operations and sweeps all 25,737 valid inputs: worst-case error **7.6 LSB of Q15** (2.3×10^{-4}). An instruction-level simulation that models the AVR carry flag confirms the assembled routine matches that model **bit-for-bit on every input** — and confirms the AVR201 multiply is correct across 300,000 random signed pairs. The routine assembles to 144 bytes.

39.5 Rational Approximation

Sometimes a polynomial of *any* practical degree cannot match a function's shape — anything with an asymptote or a sharp bend resists polynomials, which are smooth and unbounded. A **rational** approximation, a ratio of two polynomials $P(x) / Q(x)$, fits these far better for the same number of coefficients, because the denominator can bend the curve in ways a numerator alone cannot. The price is one division.

The classic example is `arctan`:

```

atan(x) ~ x / (1 + 0.28125 * x^2)    on [-1, 1]
      0.28125 = 9/32  (a denominator-friendly constant)

```

Measured over $[-1, 1]$:

`max |error|`

rational $x / (1 + (9/32)x^2)$	4.9e-3 rad	(0.28 deg)	– one divide
degree-3 odd polynomial	9.2e-3 rad	(0.53 deg)	– no divide

The single-division rational form is about **2× more accurate** than a degree-3 polynomial here, and it captures arctan’s flattening toward $\pm\pi/4$ that a low-degree polynomial cannot. In fixed point you compute the numerator and denominator as small polynomials (Horner, as above), then divide — using the multi-byte division routine from the divider appendix. Choosing the constant as 9/32 makes the $0.28125 \cdot x^2$ term a multiply by 9 and a 5-bit shift, avoiding a general multiply.

The trade-off is real: division is the most expensive integer operation on an AVR (no hardware divide; tens of cycles for a multi-byte routine). A rational form wins when it saves *more* polynomial terms than the division costs, or when the function’s shape leaves no good polynomial at any sane degree.

39.6 Choosing an Approach

Need a transcendental function, no FPU:

1. Range-reduce first — always. It buys the most accuracy per cycle.
2. Smooth, well-behaved on the reduced interval? -> minimax polynomial (Horner)
3. Has an asymptote / sharp bend / must match shape tightly? -> rational P/Q
4. Derive coefficients on the host (Remez or least-squares); ship them as fixed-point constants.
5. Pick Q-formats for the intermediates, match the shift rounding in your reference model, and saturate the result.

A handful of functions have better dedicated algorithms — sqrt by the digit-by-digit method (Chapter 17), sin/cos/atan by CORDIC (Chapter 19) when you need many of them or lack a multiplier. Polynomial and rational approximation is the general-purpose tool for everything else, and often the fastest when a hardware multiplier is available.

39.7 Common Pitfalls

- Reaching for Taylor because it is familiar, and paying for accuracy at $x = 0$ you do not need while the interval’s far end is the worst case.
- Skipping range reduction and then fighting the error with extra polynomial terms — far more expensive than folding the argument first.
- Letting an intermediate product overflow its Q-format even though the final result is in range. Choose formats for the intermediates.
- Using an arithmetic shift in the assembly but truncation-toward-zero in the host model (or vice versa), so the two disagree by an LSB and “verification” is meaningless.
- Forgetting that a minimax polynomial can overshoot at the extremes; not saturating, and watching the result wrap.
- Choosing a rational form for a smooth function where a polynomial would do — and paying for a division you did not need.
- Deriving coefficients in floating point and forgetting to re-optimize after rounding to fixed point; the rounded coefficients are slightly different and the error bound should be re-measured on the integer model, as done here.

39.8 Summary

Approximate transcendentals on an FPU-less MCU with polynomials / rationals.

Polynomial: minimax (equioscillating) beats Taylor for the same degree – ~28x here for degree-5 sine. Derive on host (Remez / least-squares), ship as fixed-point constants, evaluate with Horner.

Range reduction first: fold the argument into a small interval (periodicity, symmetry, exponent split). Biggest accuracy-per-cycle win.

Fixed-point: track Q-formats through every product ($Q_m \cdot Q_n = Q_{m+n}$), shift back), match shift-rounding between asm and reference model, saturate overshoot.

Worked: q15_sin.S – Q14 in, Q15 out, degree-5 odd minimax, 7.6 LSB worst case, verified bit-for-bit against a host model over all inputs, 144 bytes.

Rational $P(x)/Q(x)$: one division buys shape a polynomial cannot match – $\text{atan} \approx x/(1+(9/32)x^2)$, 0.28 deg vs 0.53 deg for a degree-3 polynomial. Use when it saves more terms than the divide costs, or for asymptotic shapes.

Dedicated alternatives: sqrt (Ch.17 digit-by-digit), sin/cos/atan (Ch.19 CORDIC).

The skill is not evaluating a polynomial — Chapter 15 covered that — but choosing one worth evaluating: minimax over Taylor, a reduced argument over a wide one, a rational form when shape demands it, and a fixed-point scheme whose error you have actually measured rather than hoped for.

39.9 Exercises

1. Re-fit the degree-5 sine coefficients yourself (least-squares on a dense grid of $[0, \pi/2]$), round to Q15, and run `sinref.c` with your values. Can you beat 7.6 LSB? What happens to the error if you drop the x^5 term?
2. The routine clamps a 4-LSB positive overshoot. Extend the clamp to full signed saturation (both $+0x7FFF$ and $-0x8000$) so the routine is safe for inputs slightly outside $[0, \pi/2]$. Which extra instructions are needed?
3. Implement range reduction for a full-circle sine: take an angle in Q12 turns $[0, 1)$, fold it into $[0, \pi/2]$ using quadrant symmetry, call `q15_sin`, and fix the sign. How many comparisons and negations does the fold cost?
4. Write a Q15 2^f for f in $[0, 1)$ using a degree-2 minimax polynomial. Derive the coefficients on the host and measure the error. Why is degree 2 often enough here?
5. Implement the rational `atan` in fixed point: numerator x , denominator $1 + (9/32)x^2$, using the multi-byte divide from the divider appendix. Measure the error against `atan` with a host model and compare cycle count to a degree-5 polynomial of the same accuracy.
6. The AVR201 `smul32` uses the carry flag from `mulsu` to sign-extend. Trace one negative×positive case by hand and show how the `sbc r19, r2` produces the correct upper bytes. Why would replacing `mulsu` with `mul` break it?
7. Compare evaluating the sine polynomial with Horner versus the naive form (computing x^3 and x^5 separately). Count multiplies and the worst-case intermediate magnitude for each. Why does Horner also help with overflow?

Chapter 40

Fixed-Point Matrix Transforms

Rotating a sensor reading into a level frame, steering a two-axis gimbal, projecting a point for a small display — these are matrix transforms, and on a chip with no floating-point unit they run in fixed point. The mathematics is the same linear algebra as anywhere else; the engineering is keeping the arithmetic in range and the precision honest while every multiply is an integer operation.

This chapter builds the core operation — a matrix times a vector — in fixed point, with the accumulation and scaling done right, then extends it to composing transforms and to the homogeneous and projection matrices used for control and graphics. The 2×2 routine here is build-verified and checked bit-for-bit against a host model; the coefficients for its rotations come from the sine work of the previous chapter.

40.1 Matrices in Fixed Point

A transform matrix holds coefficients, and for the common cases those coefficients live in $[-1, 1]$ — the entries of a rotation matrix are sines and cosines. That makes **Q15** the natural format: each entry is a signed 16-bit value scaled by 2^{15} , and a vector coordinate is an integer (Q0) or another fixed-point value.

One representation trap appears immediately. Q15 spans $[-1, +0.99997]$ — it **cannot represent +1.0 exactly** (that would be 32768, which overflows a signed 16-bit value to -32768). A rotation by 0° has $\cos = 1.0$; convert it naively and the matrix entry flips sign, turning the identity into a reflection. The fix is a **saturating conversion** — clamp $+1.0$ to 32767:

```
int16_t q15(double x){
    long v = lround(x * 32768.0);
    if(v > 32767) v = 32767;      /* +1.0 -> 0.99997, not -1.0 */
    if(v < -32768) v = -32768;
    return (int16_t)v;
}
```

The one-LSB error this introduces at ± 1.0 is negligible; the sign flip it prevents is catastrophic. (If exact ± 1.0 matters, use Q14 for the matrix and carry the extra headroom.)

40.2 The Core Operation: Matrix $\times$ Vector

Every transform reduces to a matrix-vector product, and every element of the result is a **dot product** of a matrix row with the vector:

$$\begin{array}{|c|c|} \hline | v0' | & | M0 \ M1 | \\ \hline | v1' | = & | M2 \ M3 | \\ \hline \end{array} \begin{array}{|c|} \hline | v0 | \\ \hline | v1 | \\ \hline \end{array} \quad \begin{array}{l} v0' = M0*v0 + M1*v1 \\ v1' = M2*v0 + M3*v1 \end{array}$$

With Q15 matrix entries and Q0 vector coordinates, each product $M*v$ is Q15, and the dot product must be scaled back by $\gg 15$ to return to Q0. The question is *when* to shift, and it has one right answer:

Accumulate the full-width products, then shift once.

Shifting each product *before* adding throws away the low bits of every term, and that rounding error accumulates. Measured over random Q15 coefficients and coordinates, the gap between the two strategies grows with the length of the dot product:

dot length	max round-once - round-each
2	1 unit
3	2 units
4	3 units
8	7 units

The pattern is length – 1 LSB of avoidable error — small for a 2×2 , real for a larger transform or a chain of them. Keeping a wider accumulator costs nothing on a CPU that already produces a 32-bit product from a 16×16 multiply.

40.3 Worked Example: 2×2 Transform

src/ch27_matrix/mat2_vmul.S implements $v' = M \cdot v$ for a Q15 matrix and an integer vector, accumulating each row's two products in a 32-bit register before the single shift:

```
void mat2_vmul(const int16_t *M, const int16_t *vin, int16_t *vout);
/* vout[0] = (M0*v0 + M1*v1) >> 15
   vout[1] = (M2*v0 + M3*v1) >> 15 */
```

The inner loop reuses the signed $16 \times 16 \rightarrow 32$ multiply (smul32) and 32-bit arithmetic shift (asr32) from the approximation chapter. The accumulation is the part to notice — the first product is stashed in four registers, the second is added to it at full width, and only the 32-bit sum is shifted:

```
.Lrow:
    ld     r22, X+           /* A = M[k] */
    ld     r23, X+
    movw   r20, r14          /* B = v0 */
    rcall  smul32             /* P0 -> r19:r18:r17:r16 */
    movw   r4, r16            /* stash P0 (32-bit) in r7:r6:r5:r4 */
    movw   r6, r18

    ld     r22, X+           /* A = M[k+1] */
    ld     r23, X+
    movw   r20, r12          /* B = v1 */
    rcall  smul32             /* P1 -> r19:r18:r17:r16 */
    add    r16, r4            /* acc = P0 + P1, full 32-bit */
    adc    r17, r5
    adc    r18, r6
    adc    r19, r7

    ldi    r20, 15
    rcall  asr32              /* one shift on the sum */
    st     Y+, r16
    st     Y+, r17
```

```
dec    r24
brne   .Lrow
```

To rotate a point by θ , fill M with $\{\cos, -\sin, \sin, \cos\}$ in Q15 (using the saturating convert) and call the routine. The cosines and sines come from the Q15 sine of the previous chapter, or a lookup table, or CORDIC.

Verification. The host model `mat2ref.c` mirrors these exact integer operations and sweeps rotation matrices through a full 360° : worst-case coordinate error **1.0 unit** against floating-point rotation — pure rounding. An instruction-level simulation that models the AVR carry flag confirms the assembled routine reproduces the model **bit-for-bit across 400,000 random in-domain cases**. The routine is 150 bytes.

40.4 Staying in Range

Fixed-point transforms have two overflow boundaries, and you size your inputs to respect both:

- **The accumulator.** Each product $M \cdot v$ is up to $32767 \times |v|$. Two of them sum in the 32-bit accumulator, which is safe as long as the sum stays below 2^{31} . With full-magnitude Q15 entries that means $|v_0| + |v_1|$ must stay under about 65,000 — never an issue for sane coordinates, but the reason the accumulator is 32-bit and not 16.
- **The result.** $v' = (M \cdot v) \gg 15$ must fit back into a 16-bit coordinate. For a rotation ($|entries| \leq 1$), $|v'| \leq \sqrt{2} \cdot |v|$, so keeping $|v| \leq 16383$ (one bit of headroom) guarantees the result fits. A transform that *scales up* needs more headroom, or a wider output type.

The general habit: know the magnitude bound of your input coordinates, work out the worst-case result, and choose the vector's fixed-point format so it cannot overflow. The `mat2_vmul` routine is exact for any Q15 matrix and any vector within that domain — and silently wraps outside it, like all fixed-point code.

40.5 Composing Transforms

Applying transform A then B to many vectors is $B \cdot (A \cdot v)$ — but doing it as two matrix-vector products per vector wastes work. Because matrix multiplication is associative, precompute the **composite** $C = B \cdot A$ once and apply the single C to every vector:

rotate-then-translate, scale-then-rotate, ... : $C = B * A$ (once, on the host or at setup), then $v' = C * v$ per point.

Two cautions. First, **order matters** — $B \cdot A \neq A \cdot B$; the rightmost matrix acts on the vector first. Second, each multiply in forming C is itself a fixed-point dot product with the same accumulate-then-shift rule, and composing many transforms compounds rounding — so where you can, compute the composite in higher precision (or on a host) and ship the final fixed-point matrix as constants, exactly as the previous chapter shipped polynomial coefficients.

40.6 Beyond 2×2: Homogeneous and Projection

Rotation alone is linear, but **translation** is not — you cannot add an offset with a 2×2 multiply. The standard fix is **homogeneous coordinates**: extend the 2D point to $(x, y, 1)$ and use a 3×3 matrix whose third column carries the translation:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos & -\sin & tx \\ \sin & \cos & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Now a single 3×3 matrix-vector multiply does rotate-and-translate together, and composing several such matrices folds an entire chain of moves into one. The evaluation is the same dot-product machinery, just three terms per row instead of two — and the length – 1 rounding argument now says round once over all three.

Projection adds the one operation fixed point dreads: division. A perspective projection divides by depth ($x_{\text{screen}} = x / z$), so after the matrix multiply you perform a fixed-point divide (the divider appendix) per point. Division is the expensive step, so projection code minimizes it — one divide per vertex, the reciprocal $1/z$ computed once and reused for both x/z and y/z .

For pure rotation of a *single* vector, a matrix is not even required: **CORDIC** (Chapter 19) rotates by shift-and-add without a multiplier. The matrix form wins when you transform many vectors by the same rotation (amortizing the cos/sin cost) or when you compose rotation with scaling and translation into one operator.

40.7 Common Pitfalls

- Converting +1.0 to Q15 without saturating, overflowing to –1.0, and turning a 0° rotation into a reflection.
 - Shifting each product before summing, accumulating length – 1 LSB of avoidable rounding error per dot product.
 - Using a 16-bit accumulator and overflowing on the sum of two products. The accumulator must hold the full 32-bit width.
 - Letting the result exceed the coordinate format’s range — fine inside the domain, silent wrap outside it. Bound your input magnitudes.
 - Swapping multiplication order when composing ($A \cdot B$ vs $B \cdot A$) and applying the transforms in the wrong sequence.
 - Reaching for a matrix to rotate one vector when CORDIC would do it without a multiply, or doing many per-vector divides in a projection instead of one reciprocal reused.
 - Composing a long chain of fixed-point matrices on-device and accumulating rounding; precompute the composite at higher precision where possible.
-

40.8 Summary

Fixed-point transforms = matrix-vector products in integer arithmetic.

Represent matrix entries in Q15 (rotations are sin/cos in [-1,1]); SATURATE the +1.0 case to 32767 (Q15 cannot hold +1.0) or use Q14 for exact +-1.

Core op: each result element is a dot product of a matrix row with the vector. ACCUMULATE full 32-bit products, SHIFT ONCE (>>15). Round-each loses up to (length-1) LSB.

Worked: mat2_vmul.S – 2x2 Q15 matrix x integer vector, 32-bit accumulate, 150 bytes; max 1.0-unit rotation error, verified bit-for-bit over 400k cases.

Range: accumulator is 32-bit (sum of two products < 2³¹); bound |v| so the result fits the coordinate type (rotation: |v| <= 16383 is safe).

Compose: $C = B \cdot A$ once, then $C \cdot v$ per point. Order matters ($B \cdot A \neq A \cdot B$). Prefer

computing composites at higher precision / on the host.

Bigger: 3x3 homogeneous matrices add translation; projection adds one divide per vertex (reuse $1/z$). Single-vector rotation: CORDIC (Ch.19) needs no multiply. Coefficients (sin/cos): the approximation chapter (Ch.26) or a LUT.

A fixed-point transform is just the linear algebra you already know with two extra disciplines bolted on: choose formats so nothing overflows, and round once at the end of each dot product rather than at every term. Get those right — as the verified 2×2 routine does — and rotations, scalings, and projections all reduce to the same accumulate-and-shift core.

40.9 Exercises

1. Build and run `mat2ref.c`. Then change `q15` to *not* saturate and rerun — explain the error spike at 0° , 90° , 180° , and 270° in terms of the Q15 representation of ± 1.0 .
2. Extend `mat2_vmul.S` to a 3×3 homogeneous transform `mat3_vmul`, accumulating three products per row. How many registers do you need to stash partial sums, and where does the length - 1 rounding rule now apply?
3. Compose a rotation by 30° with a scale of 0.5 by forming $C = S \cdot R$ in Q15 on the host, then verify $C \cdot v$ equals scaling the rotated vector. Does $R \cdot S$ give the same matrix? When would it?
4. Determine the largest $|v|$ for which `mat2_vmul` cannot overflow the int16 result, for a matrix that scales by 1.5 (entries up to 1.5 — which Q-format must the matrix use, since 1.5 exceeds Q15?).
5. Implement a 2D point rotation that fills the matrix from `q15_sin` (Chapter 26) for an arbitrary angle, then rotates a test vector. Compare its accuracy and cycle count to a CORDIC rotation (Chapter 19) of the same vector.
6. The accumulator sums two 32-bit products. Construct the worst-case Q15 matrix and vector that bring the accumulator closest to 2^{31} without overflowing, and show the corresponding bound on $|v_0| + |v_1|$.
7. A perspective projection computes x/z and y/z . Show why computing $1/z$ once and multiplying is both faster and more consistent than two separate divides, and estimate the cycle saving using the divider appendix's routine.

Chapter 41

Build and Linker Workflow

Every example so far has been one source file and one `avr-gcc` command. Real firmware is several source files — a main, a few driver modules, a math library — compiled separately and linked together. This chapter is about that workflow: how the toolchain turns a pile of `.S` and `.c` files into one flashable image, how to read the map file the linker leaves behind, how to drop code you do not use, and how to make the build reproducible.

The reference appendices cover the pieces in depth — GAS directives (Appendix C), the linker script (Appendix D), and binary inspection (Appendix F). This chapter is the *process* that ties them together, built around a small two-file project you can run. Every command and every byte count below is real output from that project under `avr-gcc 16.1.0`.

41.1 The Pipeline

`avr-gcc` is a *driver* — a single command that runs four stages in turn. You rarely invoke the stages by hand, but knowing them tells you where to look when something breaks:

```
.c / .S  --[preprocess]--> expanded source    (cpp: #include, #define, #if)
          --[compile]----> .s assembly          (only for .c; ccl)
          --[assemble]----> .o object            (as: machine code + symbols)
          --[link]-----> .elf executable       (ld: resolve symbols, place code)
```

You can stop after any stage: `-E` preprocess only, `-S` stop at assembly, `-c` stop at the object file, and no flag to go all the way to a linked ELF. The two that matter daily are `-c` (compile/assemble one file to a `.o`) and the final link.

A `.S` file (capital S) is preprocessed before assembly, which is why `#include <avr/io.h>` and `#define` work in it; a lowercase `.s` skips the preprocessor. Always name AVR assembly `.S`.

41.2 Separate Compilation

Splitting a program into modules and compiling each to its own object file is **separate compilation**. It is worth doing for two reasons: a one-line change recompiles only one file instead of the whole program, and modules become reusable. The example project is a main plus a small vector library:

```
$ avr-gcc -mmcu=attiny3217 -c main.S -o main.o
$ avr-gcc -mmcu=attiny3217 -c vec.S -o vec.o
```

Each `.o` holds machine code plus a **symbol table**: symbols it *defines* (`main`, `vec_add`) and symbols it *references but does not define* (`main` calls `vec_add`, which lives in `vec.o`). Neither object is complete on its own — the link step resolves the cross-references:

```
$ avr-gcc -mmcu=attiny3217 -nostartfiles main.o vec.o -o app.elf
```

The linker reads every object, matches each undefined reference to a definition somewhere, lays the sections out at addresses, and writes the ELF. A symbol that is referenced but never defined is the classic *undefined reference* error; a symbol defined twice is a *multiple definition*. `.global` is what makes a symbol visible across object files — without it a label is local to its own file and the linker cannot see it (this is the same mechanism the C/assembly chapter used to call assembly from C).

For a larger collection of reusable routines, bundle the objects into a static **archive** (`avr-ar rcs libfoo.a a.o b.o ...`) and link against it; the linker pulls in only the members that are actually referenced.

41.3 Dropping Unused Code: `--gc-sections`

Linking `vec.o` brings in *both* its functions, even if `main` calls only one. To let the linker discard the unused one, two things must line up: each function goes in its **own section**, and the linker is told to **garbage-collect** sections nothing references.

In assembly you place each function in its own section explicitly:

```
.section .text.vec_add,"ax",@progbits
.global vec_add
vec_add:
...

.section .text.vec_scale,"ax",@progbits /* a separate, collectable section */
.global vec_scale
vec_scale:
...
```

(For C, `-ffunction-sections` does this automatically.) Then link with `--gc-sections`, naming the entry point so the linker knows where “used” starts:

```
$ avr-gcc -mmcu=attiny3217 -nostartfiles -e main \
    main.o vec.o -o app.elf -Wl,--gc-sections
```

`main` uses `vec_add` but not `vec_scale`, and the size drops accordingly:

	text	data	bss	filename	
without GC:	34	0	2	app.elf	
with GC:	26	0	2	app.elf	(vec_scale, 8 bytes, discarded)

`-Wl,` is how the gcc driver passes an option through to the linker; multiple linker options are comma-separated. On a 32 KB part eight bytes is nothing, but on a real project with a fat library you did not write, `--gc-sections` routinely reclaims kilobytes.

41.4 Reading the Map File

Ask the linker for a **map file** and it writes down every decision it made — what went where, and why. Generate it with `-Wl,-Map`:

```
$ avr-gcc ... main.o vec.o -o app.elf -Wl,--gc-sections,-Map,app.map
```

The map answers the questions you actually have. *Where did this symbol land, and which object provided it?*

.text.vec_add	0x00000014	0x6 vec.o
	0x00000014	vec_add

vec_add is at address 0x14, occupies 6 bytes, and came from vec.o. *What did --gc-sections throw away?* The map lists it explicitly:

Discarded input sections		
.text	0x00000000	0x0 vec.o
...		

And the **Memory Configuration** block at the top shows the address spaces the linker is placing into — the same regions Appendix D’s linker script defines:

Name	Origin	Length	Attributes
text	0x00000000	0x00100000	xr
data	0x00802000	0x0000ffa0	rw!x
eeprom	0x00810000	0x00010000	rw!x
fuse	0x00820000	0x00000400	rw!x

When a binary is mysteriously large, or a symbol ended up somewhere unexpected, or you need to know which object pulled in a heavy dependency, the map file is the first place to look — far faster than disassembling.

41.5 A Project Makefile

Tie it together with make, so a build is one command and only changed files recompile. The example project’s Makefile:

```
MCU      := attiny3217
SRCS     := main.S vec.S
OBJS     := $(SRCS:.S=.o)
TARGET   := app

ASFLAGS  := -mmcu=$(MCU)
LDFLAGS  := -mmcu=$(MCU) -nostartfiles -e main \
            -Wl,--gc-sections,-Map,$(TARGET).map

all: $(TARGET).hex

%.o: %.S                                # separate compilation: each .S -> .o
    avr-gcc $(ASFLAGS) -c $< -o $@

$(TARGET).elf: $(OBJS)                  # link the objects
    avr-gcc $(LDFLAGS) $^ -o $@

$(TARGET).hex: $(TARGET).elf            # flashable image
    avr-objcopy -O ihex $< $@

clean:
    rm -f $(OBJS) $(TARGET).elf $(TARGET).hex $(TARGET).map
```

The pattern rule %.o: %.S and the prerequisite lists let make rebuild incrementally — touch one source and only it is reassembled before the relink:

```
$ touch vec.S && make
avr-gcc -mmcu=attiny3217 -c vec.S -o vec.o          # only vec recompiles
```

```
avr-gcc -mmcu=attiny3217 -nostartfiles -e main ... main.o vec.o -o app.elf
avr-objcopy -O ihex app.elf app.hex
```

main.o is untouched because main.S did not change — make compared timestamps and skipped it. For C sources, add `-MMD` to the compile flags and `-include $(OBJDIR:.o=.d)` so header changes also trigger the right rebuilds.

41.6 Reproducible Builds

A reproducible build produces a **byte-identical** image from the same sources, on any machine, at any time — essential for verifying that a shipped binary matches its source, and for caching. Two practical points on AVR:

- **Ship the .hex, not the .elf, as the canonical artifact.** An ELF can embed build paths and other host-specific bytes; the Intel HEX produced by `avr-objcopy` is just the flashable bytes. Building the example twice and comparing the hex confirms it:

```
$ avr-objcopy -O ihex r1.elf r1.hex
$ avr-objcopy -O ihex r2.elf r2.hex
$ cmp r1.hex r2.hex && echo identical
identical                # byte-for-byte, same sha256
```

- **Pin everything that affects output.** The same toolchain version, the same flags, and no embedded timestamps. If a path leaks into the binary, map it away with `-ffile-prefix-map=$(PWD)=.`; for archives, `avr-ar` is deterministic by default (the `s` modifier writes a sorted index). Record the toolchain version alongside the artifact so a future rebuild can match it.

A reproducible .hex plus the CRCSCAN integrity check from the Defensive Firmware chapter is a strong pairing: you can prove a device is running exactly the image you built.

41.7 Common Pitfalls

- Naming assembly `.s` (lowercase) and losing the preprocessor, so `#include` and `#define` silently do nothing. Use `.S`.
 - A label that is not `.global`, so another object's reference to it fails to resolve — *undefined reference* at link time.
 - Expecting `--gc-sections` to drop unused functions without per-function sections (own `.section` in assembly, `__function_sections` in C) and an entry point (`-e main`, or the normal C startup) to anchor liveness analysis.
 - Forgetting `-Wl`, and passing a linker option to the compiler driver, which does not understand it.
 - Treating the ELF as the reproducible artifact when host paths or timestamps leak into it; compare the .hex.
 - Linking objects built for mismatched options or assuming an object built for one MCU works for another.
 - No header dependencies in the Makefile, so a changed header does not trigger a rebuild and you flash stale code. Use `-MMD`.
-

41.8 Summary

`avr-gcc` is a driver over four stages: preprocess -> compile -> assemble -> link.

-E / -S / -c stop early; .S preprocesses, .s does not (always use .S).

Separate compilation: each module -> .o (defines + undefined symbols); the link resolves cross-references. .global exposes a symbol across objects. Bundle reusable objects in an archive (avr-ar) and the linker pulls only what's used.

--gc-sections drops unreferenced sections: needs per-function sections (.section .text.name in asm / -ffunction-sections in C) + an entry point (-e main). Saved 8 bytes here by discarding an unused library function.

Map file (-Wl,-Map): symbol -> address -> object, discarded sections, memory configuration. The first tool for "why is it big / where did this go".

Makefile: pattern rule %.o: %.S, prerequisite lists for incremental rebuilds, objcopy to a flashable .hex. Add -MMD for header dependencies (C).

Reproducible builds: ship the .hex (not the ELF); pin toolchain + flags; map away build paths (-ffile-prefix-map). Pairs with the CRCSCAN integrity check.

Pass linker options through the driver with -Wl,opt1,opt2.

The build workflow is not glamorous, but it is where a project stops being a demo and becomes maintainable: modules that compile independently, a linker you can interrogate through its map, dead code that removes itself, and an image you can rebuild bit-for-bit. The reference appendices have the details; this is the loop you run every day.

41.9 Exercises

1. Build the two-file project by hand: assemble main.S and vec.S to objects, then link them. Now delete the .global on vec_add and rebuild. What exact error does the linker give, and at which stage?
2. Add a third function vec_sub to vec.S in its own section, call it from main, and confirm from the map file where it lands. Then stop calling it and confirm --gc-sections removes it.
3. Generate the map with and without --gc-sections and diff them. Which sections appear under "Discarded input sections" only in the GC build?
4. Convert vec.S/vec.o into a static archive libvec.a with avr-ar. Link main.o against the archive and confirm only the referenced members are pulled in.
5. Build the project twice into r1.hex and r2.hex and compare their sha256 sums. Then introduce a deliberate difference (a changed constant) and confirm the hash changes. Why is the .hex a better artifact to hash than the .elf?
6. Add a flash target to the Makefile that runs avrdude (Appendix E) on the produced .hex. What prerequisite should it depend on so make flash rebuilds first if a source changed?
7. Using -Wl,-Map, find the total .text size two ways: from avr-size and from the map file. Do they agree? Account for any difference.

Appendix A

ATtiny3217 Register Reference

Quick lookup for registers used throughout this book. The ATtiny3217 is an AVRXMEGA3 device. All peripheral registers are memory-mapped and require LDS/STS unless noted otherwise.

VPORT exception: Virtual PORT registers at 0x0000–0x001F are within the I/O space and can be accessed with IN/OUT, SBI/CBI, SBIS/SBIC in a single cycle. All other peripheral registers require LDS/STS.

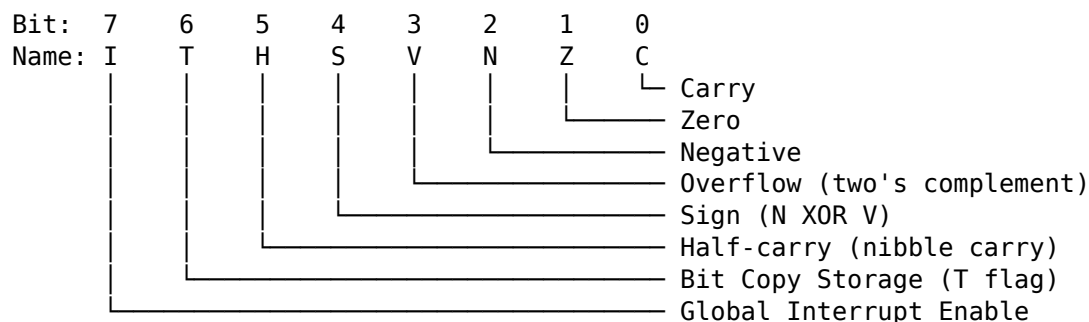
CPU registers: SREG (0x003F), SPH (0x003E), SPL (0x003D) are accessible with IN/OUT using their I/O addresses directly (no +0x20 offset needed on AVRXMEGA3 — these addresses are already the memory-mapped addresses, and the I/O address equals the memory address for registers at 0x0000–0x003F).

A.1 CPU Registers

Register	Mem Addr	Access	Description
SREG	0x003F	IN/OUT	Status Register (I T H S V N Z C)
SPH	0x003E	IN/OUT	Stack Pointer High
SPL	0x003D	IN/OUT	Stack Pointer Low
CCP	0x0034	LDS/STS	Configuration Change Protection

SREG, SPH, and SPL sit within the range 0x0000–0x003F and are accessible via IN/OUT using the address directly. The CCP register requires LDS/STS.

A.1.1 SREG Bit Layout



A.1.2 Configuration Change Protection (CCP)

Certain registers are protected against accidental writes. Before writing to a protected register, write the appropriate key to CCP. The protected register must then be written within 4 CPU cycles.

Key	Value	Protects
IOREG	0xD8	I/O registers (e.g. CLKCTRL, BOD, OSC settings)
SPM	0x9D	NVM (flash/EEPROM) write commands

Example — disable clock prescaler to run at 20 MHz:

```
ldi    r16, 0xD8          ; CCP key for I/O register changes
sts    0x0034, r16        ; write to CCP
sts    0x0061, r1         ; write 0 to CLKCTRL_MCLKCTRLB (clear PEN)
                        ; must complete within 4 cycles of CCP write
```

A.2 CLKCTRL — Clock Controller

Base address: 0x0060. All registers require LDS/STS and CCP protection where noted.

Register	Mem Addr	CCP?	Description
CLKCTRL_MCLKCTRLA	0x0060	Yes	Main clock source select
CLKCTRL_MCLKCTRLB	0x0061	Yes	Main clock prescaler
CLKCTRL_MCLKLOCK	0x0062	Yes	Lock register (prevents further changes)
CLKCTRL_MCLKSTATUS	0x0063	No	Status (read-only)

A.2.1 MCLKCTRLA — Main Clock Source Select

Bit: 7 6:2 1:0
 CLKOUT — CLKSEL

CLKSEL Source

0x0	20 MHz internal oscillator ← factory default
0x1	32 KHz internal ULP oscillator
0x2	32.768 KHz external crystal (XOSC32K)
0x3	External clock on CLKI pin

Setting CLKOUT (bit 7) routes the main clock to the CLKOUT pin.

A.2.2 MCLKCTRLB — Main Clock Prescaler

Bit: 7:5 4:1 0
 — PDIV PEN

PEN Prescaler enable (1 = prescaler active, 0 = disabled)

PDIV Division factor (when PEN=1)

0x0	/2
0x1	/4
0x2	/8
0x3	/16
0x4	/32
0x5	/64
0x8	/6 ← factory default (20 MHz / 6 = 3.333 MHz)
0x9	/10

0xA /12
 0xB /24
 0xC /48

Factory default: PDIV=0x8 (divide by 6), PEN=1 → **3.333 MHz** system clock.

To run at full **20 MHz**: write 0x00 to MCLKCTRLB (clears PEN) — requires CCP key 0xD8 first. To run at **10 MHz**: write PEN=1, PDIV=0x0 (divide by 2).

A.2.3 MCLKSTATUS — Status (read-only)

Bit: 7 6 5 4 3 2 1 0
 EXTS XOSC32KS OSC32KS OSC20MS – – – SOSC

Bit name	Bit	Description
EXTS	7	1 = external clock (EXTCLK) has started
XOSC32KS	6	1 = 32.768 KHz crystal oscillator is stable
OSC32KS	5	1 = internal ULP 32 KHz oscillator is stable
OSC20MS	4	1 = 20 MHz oscillator is stable and running
SOSC	0	1 = main clock source switch is in progress

A.3 SLPCTRL — Sleep Controller

Base address: 0x0050.

Register	Mem Addr	Description
SLPCTRL_CTRLA	0x0050	Sleep mode select and enable

A.3.1 CTRLA — Sleep Control

Bit: 7:3 2:1 0
 – SMODE SEN

SMODE Sleep mode

0x0 Idle
 0x1 Standby
 0x2 Power-down

SEN Sleep enable. Must be 1 before executing SLEEP.

A.4 BOD — Brown-Out Detector / Voltage Level Monitor

Base address: 0x0080. Some BOD_CTRLA fields are loaded from fuses. The SLEEP field is CCP-protected when changed at run time.

Register	Mem Addr	Description
BOD_CTRLA	0x0080	BOD sample frequency and active/sleep modes
BOD_CTRLB	0x0081	BOD threshold level (fuse-loaded)
BOD_VLMCTRLA	0x0088	VLM threshold above BOD level
BOD_INTCTRL	0x0089	VLM interrupt enable/configuration
BOD_INTFLAGS	0x008A	VLM interrupt flag (W1C)
BOD_STATUS	0x008B	VLM status

A.4.1 BOD_CTRLA

Bit:	7:5	4	3:2	1:0
	—	SAMPFREQ	ACTIVE	SLEEP

SAMPFREQ 0 = 1 kHz sampled mode, 1 = 125 Hz sampled mode

ACTIVE Active/Idle BOD mode (fuse-loaded, not changed by software)

0x0 Disabled

0x1 Continuous

0x2 Sampled

0x3 Continuous, wake-up halted until BOD is ready

SLEEP Standby/Power-down BOD mode (runtime writable through CCP)

0x0 Disabled

0x1 Continuous

0x2 Sampled

A.4.2 VLM Registers

BOD_VLMCTRLA.VLMLVL

0x0 VLM threshold 5% above BOD level

0x1 VLM threshold 15% above BOD level

0x2 VLM threshold 25% above BOD level

BOD_INTCTRL

bit 0 VLMIE: enable VLM interrupt

bits 2:1 VLMCFG: interrupt below, above, or crossing VLM threshold

BOD_INTFLAGS

bit 0 VLMIF: write 1 to clear

A.5 GPIO Registers

The AVRXMEGA3 GPIO model is completely different from classic AVR. Each port has a full set of direction/output/input/control registers at a fixed base address. Atomic set/clear/toggle registers eliminate the need for read-modify-write sequences.

A.5.1 Port Base Addresses

Port	Base Addr
------	-----------

PORTA	0x0400
-------	--------

PORTB	0x0420
-------	--------

PORTC	0x0440
-------	--------

A.5.2 Per-Port Register Map (offset from base)

Offset	Register	Description
+0x00	PORTx_DIR	Data direction register (1=output, 0=input)
+0x01	PORTx_DIRSET	Write 1 to set direction bits (atomic)
+0x02	PORTx_DIRCLR	Write 1 to clear direction bits (atomic)
+0x03	PORTx_DIRTGL	Write 1 to toggle direction bits (atomic)
+0x04	PORTx_OUT	Output value register
+0x05	PORTx_OUTSET	Write 1 to set output bits (atomic)

+0x06	PORTx_OUTCLR	Write 1 to clear output bits (atomic)
+0x07	PORTx_OUTTGL	Write 1 to toggle output bits (atomic)
+0x08	PORTx_IN	Input value (reads current pin state)
+0x09	PORTx_INTFLAGS	Pin-change interrupt flags (WIC)
+0x0A	PORTx_PORTCTRL	Port control register
+0x10	PORTx_PIN0CTRL	Pin 0 config (pull-up, invert, ISC)
+0x11	PORTx_PIN1CTRL	Pin 1 config
+0x12	PORTx_PIN2CTRL	Pin 2 config
+0x13	PORTx_PIN3CTRL	Pin 3 config
+0x14	PORTx_PIN4CTRL	Pin 4 config
+0x15	PORTx_PIN5CTRL	Pin 5 config
+0x16	PORTx_PIN6CTRL	Pin 6 config
+0x17	PORTx_PIN7CTRL	Pin 7 config

A.5.3 PORTA Absolute Addresses

Register	Mem Addr	Description
PORTA_DIR	0x0400	Direction
PORTA_DIRSET	0x0401	Set direction (atomic)
PORTA_DIRCLR	0x0402	Clear direction (atomic)
PORTA_DIRTGL	0x0403	Toggle direction (atomic)
PORTA_OUT	0x0404	Output value
PORTA_OUTSET	0x0405	Set output (atomic)
PORTA_OUTCLR	0x0406	Clear output (atomic)
PORTA_OUTTGL	0x0407	Toggle output (atomic)
PORTA_IN	0x0408	Input value
PORTA_INTFLAGS	0x0409	Interrupt flags
PORTA_PORTCTRL	0x040A	Port control
PORTA_PIN0CTRL	0x0410	PA0 config (UPDI pin – do not reconfigure)
PORTA_PIN1CTRL	0x0411	PA1 config (SPI MOSI default)
PORTA_PIN2CTRL	0x0412	PA2 config (SPI MISO default)
PORTA_PIN3CTRL	0x0413	PA3 config (LED0 – Curiosity Nano)
PORTA_PIN4CTRL	0x0414	PA4 config (SPI SS default)
PORTA_PIN5CTRL	0x0415	PA5 config
PORTA_PIN6CTRL	0x0416	PA6 config
PORTA_PIN7CTRL	0x0417	PA7 config

A.5.4 PORTB Absolute Addresses

Register	Mem Addr	Description
PORTB_DIR	0x0420	Direction
PORTB_DIRSET	0x0421	Set direction (atomic)
PORTB_DIRCLR	0x0422	Clear direction (atomic)
PORTB_DIRTGL	0x0423	Toggle direction (atomic)
PORTB_OUT	0x0424	Output value
PORTB_OUTSET	0x0425	Set output (atomic)
PORTB_OUTCLR	0x0426	Clear output (atomic)
PORTB_OUTTGL	0x0427	Toggle output (atomic)
PORTB_IN	0x0428	Input value
PORTB_INTFLAGS	0x0429	Interrupt flags
PORTB_PORTCTRL	0x042A	Port control
PORTB_PIN0CTRL	0x0430	PB0 config (TWI SDA default)
PORTB_PIN1CTRL	0x0431	PB1 config (TWI SCL default)
PORTB_PIN2CTRL	0x0432	PB2 config (USART0 TX – Curiosity Nano)
PORTB_PIN3CTRL	0x0433	PB3 config (USART0 RX – Curiosity Nano)
PORTB_PIN4CTRL	0x0434	PB4 config

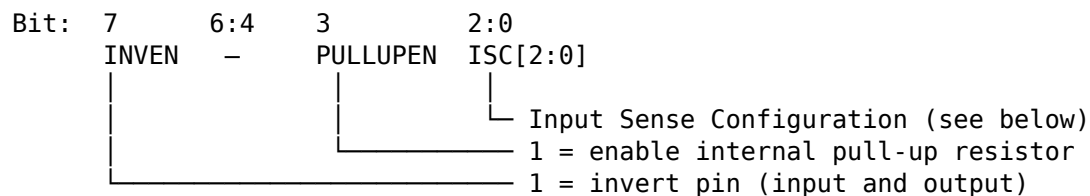
PORTB_PIN5CTRL	0x0435	PB5 config
PORTB_PIN6CTRL	0x0436	PB6 config
PORTB_PIN7CTRL	0x0437	PB7 config

A.5.5 PORTC Absolute Addresses

Register	Mem Addr	Description
PORTC_DIR	0x0440	Direction
PORTC_DIRSET	0x0441	Set direction (atomic)
PORTC_DIRCLR	0x0442	Clear direction (atomic)
PORTC_DIRTGL	0x0443	Toggle direction (atomic)
PORTC_OUT	0x0444	Output value
PORTC_OUTSET	0x0445	Set output (atomic)
PORTC_OUTCLR	0x0446	Clear output (atomic)
PORTC_OUTTGL	0x0447	Toggle output (atomic)
PORTC_IN	0x0448	Input value
PORTC_INTFLAGS	0x0449	Interrupt flags
PORTC_PORTCTRL	0x044A	Port control
PORTC_PIN0CTRL	0x0450	PC0 config
PORTC_PIN1CTRL	0x0451	PC1 config
PORTC_PIN2CTRL	0x0452	PC2 config
PORTC_PIN3CTRL	0x0453	PC3 config
PORTC_PIN4CTRL	0x0454	PC4 config
PORTC_PIN5CTRL	0x0455	PC5 config
PORTC_PIN6CTRL	0x0456	PC6 config (not present on 20-pin package)
PORTC_PIN7CTRL	0x0457	PC7 config (not present on 20-pin package)

A.5.6 PINnCTRL Bit Layout

Each PORTx_PINnCTRL register configures one pin independently.



ISC[2:0] Input Sense Configuration

0x0	INTDISABLE	No interrupt; digital input buffer enabled
0x1	BOTHEDGES	Interrupt on both rising and falling edges
0x2	RISING	Interrupt on rising edge only
0x3	FALLING	Interrupt on falling edge only
0x4	INPUT_DISABLE	Digital input buffer disabled (reduces power)
0x5	LEVEL	Interrupt on low level

A.5.7 Virtual PORT Registers (VPORT) — Fast I/O

VPORTs mirror the key PORT registers into I/O space (0x0000–0x001F). They can be accessed with IN/OUT (1 cycle), SBI/CBI, and SBIS/SBIC. Writing to VPORT_IN toggles the corresponding output bit (same as OUTTGL).

Register	Addr	Mirrors
VPORTA_DIR	0x0000	PORTA_DIR (1=output)
VPORTA_OUT	0x0001	PORTA_OUT
VPORTA_IN	0x0002	PORTA_IN (write 1 to toggle output)

VPORTA_INTFLAGS 0x0003 PORTA_INTFLAGS

VPORTB_DIR 0x0004 PORTB_DIR
 VPORTB_OUT 0x0005 PORTB_OUT
 VPORTB_IN 0x0006 PORTB_IN
 VPORTB_INTFLAGS 0x0007 PORTB_INTFLAGS

VPORTC_DIR 0x0008 PORTC_DIR
 VPORTC_OUT 0x0009 PORTC_OUT
 VPORTC_IN 0x000A PORTC_IN
 VPORTC_INTFLAGS 0x000B PORTC_INTFLAGS

Use VPORT registers with IN/OUT for single-cycle access. Use the full PORT registers with LDS/STS when you need atomic set/clear/toggle or pin config.

A.6 Curiosity Nano Pin Assignments

Signal	Port Pin	Mem Addr (PINnCTRL)	Notes
LED0	PA3	0x0413	Active low – drive LOW to light
USART0 TX	PB2	0x0432	To on-board debugger virtual COM port
USART0 RX	PB3	0x0433	From on-board debugger virtual COM port
UPDI	PA0	0x0410	Programming/debug – do not reconfigure

A.6.1 LED and Button Quick Reference

LED0 on (light): STS PORTA_OUTCLR, (1<<3) ; clear PA3 (active low)
 LED0 off (dark): STS PORTA_OUTSET, (1<<3) ; set PA3
 LED0 toggle: STS PORTA_OUTTGL, (1<<3) ; toggle PA3

The ATtiny3217 Curiosity Nano does not provide a dedicated user pushbutton. For button examples, wire an external active-low button to a free GPIO and enable that pin's internal pull-up.

A.7 TCA0 — Timer/Counter Type A (16-bit)

Base address: 0x0A00 (single-slope mode, the default). TCA0 is a general-purpose 16-bit timer supporting normal, CTC, and PWM modes.

Register	Mem Addr	Description
TCA0_SINGLE_CTRLA	0x0A00	Prescaler select (CLKSEL), enable (ENABLE)
TCA0_SINGLE_CTRLB	0x0A01	Waveform mode (WGMODE), compare enables
TCA0_SINGLE_CTRLC	0x0A02	Force compare outputs
TCA0_SINGLE_CTRLD	0x0A03	Split mode enable
TCA0_SINGLE_CTRLCLR	0x0A04	Control E clear
TCA0_SINGLE_CTRLSET	0x0A05	Control E set
TCA0_SINGLE_INTCTRL	0x0A06	Interrupt enable (OVF, CMP0, CMP1, CMP2)
TCA0_SINGLE_INTFLAGS	0x0A07	Interrupt flags (W1C)
TCA0_SINGLE_EVCTRL	0x0A09	Event control
TCA0_SINGLE_DBGCTRL	0x0A0E	Debug control
TCA0_SINGLE_TEMP	0x0A0F	Temp register for 16-bit read/write
TCA0_SINGLE_CNT	0x0A20	Counter value (16-bit, write low byte first)
TCA0_SINGLE_PER	0x0A26	Period / TOP value (16-bit)

TCA0_SINGLE_CMP0	0x0A28	Compare/capture 0 (16-bit)
TCA0_SINGLE_CMP1	0x0A2A	Compare/capture 1 (16-bit)
TCA0_SINGLE_CMP2	0x0A2C	Compare/capture 2 (16-bit)
TCA0_SINGLE_PERBUF	0x0A36	Period buffer (double-buffered)
TCA0_SINGLE_CMP0BUF	0x0A38	Compare 0 buffer
TCA0_SINGLE_CMP1BUF	0x0A3A	Compare 1 buffer
TCA0_SINGLE_CMP2BUF	0x0A3C	Compare 2 buffer

A.7.1 CTRLA — Prescaler and Enable

Bit: 7:4 3:1 0
 — CLKSEL ENABLE

CLKSEL	Prescaler
0x0	DIV1 (no prescaling)
0x1	DIV2
0x2	DIV4
0x3	DIV8
0x4	DIV16
0x5	DIV64
0x6	DIV256
0x7	DIV1024

Example — 1 Hz overflow at 3.333 MHz with DIV64 (prescaled clock = 52,083 Hz, PER = 52082):

```
ldi r16, 0x0B           ; CLKSEL=DIV64 (bits 3:1 = 101), ENABLE=1
sts 0x0A00, r16         ; TCA0_SINGLE_CTRLA
ldi r16, lo8(52082)
sts 0x0A26, r16         ; TCA0_SINGLE_PER low byte
ldi r16, hi8(52082)
sts 0x0A27, r16         ; TCA0_SINGLE_PER high byte
ldi r16, 0x01           ; OVF interrupt enable
sts 0x0A06, r16         ; TCA0_SINGLE_INTCTRL
```

A.7.2 CTRLB — Waveform Mode

WGMODE[2:0] (bits 2:0)	Mode
0x0	Normal (count up, overflow at PER)
0x1	Frequency (toggle W0, reset at CMP0)
0x2	Single-slope PWM
0x5	Dual-slope PWM (overflow at BOTTOM and TOP)
0x6	Dual-slope PWM (overflow at TOP only)
0x7	Dual-slope PWM (overflow at BOTTOM only)

A.8 TCB0 / TCB1 — Timer/Counter Type B (16-bit)

Type B timers are 16-bit, single-channel timers suited for periodic interrupts, input capture, and PWM. TCB0 and TCB1 share the same register layout.

TCB0 base: 0x0A40

TCB1 base: 0x0A50

Offset	Register	Description
--------	----------	-------------

+0x00	TCBn_CTRLA	Clock select, enable
+0x01	TCBn_CTRLB	Compare/capture mode, output enable
+0x04	TCBn_EVCTRL	Event control
+0x05	TCBn_INTCTRL	CAPT interrupt enable (bit 0)
+0x06	TCBn_INTFLAGS	CAPT interrupt flag (bit 0, W1C)
+0x07	TCBn_STATUS	RUN flag (bit 0, read-only)
+0x08	TCBn_DBGCTRL	Debug control
+0x09	TCBn_TEMP	Temp register for 16-bit read/write
+0x0A	TCBn_CNTL	Counter low byte
+0x0B	TCBn_CNTH	Counter high byte
+0x0C	TCBn_CCMP_L	Compare/capture low byte
+0x0D	TCBn_CCMP_H	Compare/capture high byte

A.8.1 CTRLA — Clock Select and Enable

Bit: 7:3 2:1 0
 — CLKSEL ENABLE

CLKSEL Clock source

0x0	CLKPER	(same as system clock)
0x1	CLKPER/2	
0x2	TCA0	(TCB clocked from TCA0 overflow)

A.8.2 CTRLB — Mode (CMODE)

CMODE[2:0] (bits 2:0) Mode

0x0	Periodic Interrupt	← default, most common
0x1	Timeout Check	
0x2	Input Capture on Event	
0x3	Input Capture Frequency Measurement	
0x4	Input Capture Pulse-Width Measurement	
0x5	Input Capture Freq+PW Measurement	
0x6	Single Shot	
0x7	8-bit PWM	

In Periodic Interrupt mode (CMODE=0), the counter counts up to CCMP, fires the CAPT interrupt, then resets to zero. CCMP sets the period (TOP).

Example — 1 kHz periodic interrupt at 3.333 MHz:

```
ldi r16, lo8(3332)      ; CCMP = (f_cpu / f_irq) - 1 = 3332
sts 0x0A4C, r16         ; TCB0_CCMP_L
ldi r16, hi8(3332)
sts 0x0A4D, r16         ; TCB0_CCMP_H
ldi r16, 0x01           ; CAPT interrupt enable
sts 0x0A45, r16         ; TCB0_INTCTRL
ldi r16, 0x01           ; CLKSEL=CLKPER, ENABLE=1
sts 0x0A40, r16         ; TCB0_CTRLA
```

A.9 USART0

Base address: 0x0800. On the Curiosity Nano board, USART0 is connected to the on-board debugger (PB2=TX, PB3=RX), providing a virtual COM port over USB.

Register	Mem Addr	Description
----------	----------	-------------

USART0_RXDATA	0x0800	Receive data (low byte, 8-bit data here)
USART0_RXDATAH	0x0801	Receive data high + error flags
USART0_TXDATA	0x0802	Transmit data (write byte here to send)
USART0_TXDATAH	0x0803	Transmit data high (9-bit mode only)
USART0_STATUS	0x0804	Status flags
USART0_CTRLA	0x0805	Interrupt enables
USART0_CTRLB	0x0806	Receiver/transmitter enable
USART0_CTRLC	0x0807	Frame format (default = 8N1)
USART0_BAUDL	0x0808	Baud rate register low byte
USART0_BAUDH	0x0809	Baud rate register high byte
USART0_CTRLD	0x080A	Auto-baud, IRCOM
USART0_DBGCTRL	0x080B	Debug control
USART0_EVCTRL	0x080C	Event control
USART0_TXPLCTRL	0x080D	TX pin LINAUTO
USART0_RXPLCTRL	0x080E	RX pin LINAUTO

A.9.1 STATUS Bits

Bit: 7 6 5 4 3 2 1:0
 RXCIF TXCIF DREIF FERR BUFOVF PERR –

RXCIF Receive Complete Interrupt Flag (1 = unread data in RXDATA)
 TXCIF Transmit Complete Interrupt Flag (1 = TX shift register empty)
 DREIF Data Register Empty Flag (1 = OK to write next byte to TXDATA)
 FERR Frame Error
 BUFOVF Buffer Overflow
 PERR Parity Error

A.9.2 CTRLA — Interrupt Enables

Bit: 7 6 5 4:2 1 0
 RXCIE TXCIE DREIE – LBME ABEIE

RXCIE Receive Complete IE
 TXCIE Transmit Complete IE
 DREIE Data Register Empty IE (triggers when ready for next transmit byte)

A.9.3 CTRLB — Enable Bits

Bit: 7 6 5:4 3 2 1:0
 RXEN TXEN MPCM RXMODE ODME –

RXEN 1 = enable receiver
 TXEN 1 = enable transmitter
 RXMODE 0x0=NORMAL, 0x1=CLK2X, 0x2=GENAUTO, 0x3=LINAUTO

A.9.4 CTRLC — Frame Format

Default value: 0x03 = 8N1 (8 data bits, no parity, 1 stop bit)

Bit: 7:6 5:4 3 2:0
 CMODE PMODE SBMODE CHSIZE

CMODE 0x0=ASYNCHRONOUS, 0x1=SYNCHRONOUS, 0x2=IRCOM, 0x3=MSPI
 PMODE 0x0=DISABLED, 0x2=EVEN, 0x3=ODD
 SBMODE 0=1 stop bit, 1=2 stop bits

CHSIZE 0x0=5-bit, 0x1=6-bit, 0x2=7-bit, 0x3=8-bit, 0x4-0x6=reserved, 0x7=9-bit

A.9.5 Baud Rate Formula and Common Values

Normal asynchronous mode:

$$\begin{aligned}\text{BAUD_REG} &= (\text{f_cpu} \times 64) / (16 \times \text{baud_rate}) \\ &= (\text{f_cpu} \times 4) / \text{baud_rate}\end{aligned}$$

Baud Rate	f_cpu = 3.333 MHz BAUD_REG (hex)	f_cpu = 20 MHz BAUD_REG (hex)
9600	0x056D (1389)	0x2098 (8344)
19200	0x02B7 (694)	0x104C (4172)
38400	0x015B (347)	0x0826 (2086)
57600	0x00E2 (226)	0x0571 (1393)
115200	0x0074 (116)	0x02B9 (697)

A.10 SPI0

Base address: 0x08C0. Default pin assignments: MOSI=PA1, MISO=PA2, SCK=PA3, SS=PA4.

Register	Mem Addr	Description
SPI0_CTRLA	0x08C0	Control A: DORD, MASTER, CLK2X, PRESC[1:0], ENABLE
SPI0_CTRLB	0x08C1	Control B: BUFEN, SSD, MODE[1:0]
SPI0_INTCTRL	0x08C2	Interrupt enables: IE, SSIE, TXCIE, RXCIE
SPI0_INTFLAGS	0x08C3	Interrupt flags: IF(7), WRCOL(0)
SPI0_DATA	0x08C4	TX/RX data register

A.10.1 CTRLA Bits

Bit:	7	6	5	4:3	2	1	0
	DORD	MASTER	CLK2X	PRESC	—	—	ENABLE

DORD 0=MSB first, 1=LSB first

MASTER 0=slave, 1=master

CLK2X 1=double clock speed

PRESC 0x0=DIV4, 0x1=DIV16, 0x2=DIV64, 0x3=DIV128

ENABLE 1=enable SPI

A.10.2 SPI Clock Rates

PRESC	CLK2X=0	CLK2X=1
0x0	f_cpu / 4	f_cpu / 2
0x1	f_cpu / 16	f_cpu / 8
0x2	f_cpu / 64	f_cpu / 32
0x3	f_cpu / 128	f_cpu / 64

A.11 TWI0 — Two-Wire Interface (I2C)

Base address: 0x08A0. Default pins: SDA=PB0, SCL=PB1. Requires external 4.7 kΩ pull-up resistors on SDA and SCL lines.

Register	Mem Addr	Description
TWI0_CTRLA	0x08A0	SDA hold time, fast-mode plus enable
TWI0_DUALCTRL	0x08A1	Dual-mode control
TWI0_MCTRLA	0x08A2	Master: RIEN, WIEN, QCEN, TIMEOUT, SMEN, ENABLE
TWI0_MCTRLB	0x08A3	Master: FLUSH, ACKACT, MCMD[1:0]
TWI0_MSTATUS	0x08A4	Master status flags
TWI0_MBAUD	0x08A5	Master baud rate
TWI0_MADDR	0x08A6	Master address (write triggers START condition)
TWI0_MDATA	0x08A7	Master data register (read/write)
TWI0_SCTRLA	0x08A8	Slave: DIEN, APIEN, PIEN, PMEN, SMEN, ENABLE
TWI0_SCTRLB	0x08A9	Slave: ACKACT, SCMD[1:0]
TWI0_SSTATUS	0x08AA	Slave status flags
TWI0_SADDR	0x08AB	Slave address
TWI0_SDATA	0x08AC	Slave data
TWI0_SADDRMASK	0x08AD	Slave address mask

A.11.1 MSTATUS Bits

Bit:	7	6	5	4	3	2	1:0
	RIF	WIF	CLKHOLD	RXACK	ARBLIST	BUSERR	BUSSTATE

RIF	Read Interrupt Flag
WIF	Write Interrupt Flag
CLKHOLD	Clock hold (SCL stretched by master)
RXACK	Received ACK (0=ACK received, 1=NACK)
ARBLIST	Arbitration lost
BUSERR	Bus error
BUSSTATE	0x0=UNKNOWN, 0x1=IDLE, 0x2=OWNER, 0x3=BUSY

A.11.2 MCTRLB — Master Command

MCMD[1:0]	Command
0x0	NOACT No action
0x1	REPSTART Issue repeated START
0x2	RECVTRANS Continue (ACK) / send byte
0x3	STOP Issue STOP condition

A.11.3 Baud Rate Formula

$$\text{BAUD} = (\text{f_cpu} / \text{f_scl} - 10) / 2 \quad (\text{for f_cpu in Hz, f_scl in Hz})$$

100 kHz at 3.333 MHz:	BAUD = (3333333 / 100000 - 10) / 2 ≈ 12
400 kHz at 3.333 MHz:	BAUD = (3333333 / 400000 - 10) / 2 ≈ 2 (use fast-mode+)
100 kHz at 20 MHz:	BAUD = (20000000 / 100000 - 10) / 2 = 95
400 kHz at 20 MHz:	BAUD = (20000000 / 400000 - 10) / 2 = 20

A.12 ADC0

Base address: 0x0600. The ATtiny3217 ADC is 10-bit (or 8-bit selectable).

Register	Mem Addr	Description
ADC0_CTRLA	0x0600	Enable, resolution, free-run, left-adjust
ADC0_CTRLB	0x0601	Sample accumulation (SAMPNUM)
ADC0_CTRLC	0x0602	Prescaler, reference select, sample capacitor
ADC0_CTRLD	0x0603	Init delay, auto-sample delay
ADC0_CTRLF	0x0604	Window comparator mode
ADC0_SAMPCTRL	0x0605	Sample length extension
ADC0_MUXPOS	0x0606	Input mux (selects channel)
ADC0_COMMAND	0x0608	Write bit 0 (STCONV) to start conversion
ADC0_EVCTRL	0x0609	Event trigger enable
ADC0_INTCTRL	0x060A	RESRDY(0) and WCMP(1) interrupt enables
ADC0_INTFLAGS	0x060B	Interrupt flags (W1C)
ADC0_DBGCTRL	0x060C	Debug run control
ADC0_TEMP	0x060D	Temporary register
ADC0_RESL	0x0610	Result low byte
ADC0_RESB	0x0611	Result high byte
ADC0_WINLT	0x0612	Window comparator low threshold
ADC0_WINHT	0x0614	Window comparator high threshold
ADC0_CALIB	0x0616	Calibration

A.12.1 CTRLA Bits

Bit:	7	6:5	4	3	2	1	0
	—	—	LEFTADJ	RESSEL	FREERUN	RUNSTDBY	ENABLE

LEFTADJ 1=result left-adjusted in 16-bit register (8-bit in RESB)

RESSEL 0=10-bit result, 1=8-bit result

FREERUN 1=continuous conversions

ENABLE 1=enable ADC

A.12.2 CTRLC — Prescaler and Reference

Bit:	7:6	5:4	3	2:0
	—	REFSEL	SAMPCAP	PRESC

REFSEL 0x0=INTREF (internal), 0x1=VDDI02/10, 0x2=reserved, 0x3=VREFA (external)

SAMPCAP 1=reduced sample capacitance (for high-impedance sources)

PRESC ADC clock prescaler (see table below)

PRESC Division

0x0	DIV2
0x1	DIV4
0x2	DIV8
0x3	DIV16
0x4	DIV32
0x5	DIV64
0x6	DIV128
0x7	DIV256

ADC requires 50–1500 kHz clock. At 3.333 MHz use DIV8 (417 kHz) or DIV16 (208 kHz). At 20 MHz use DIV32 (625 kHz) or DIV64 (312 kHz).

A.12.3 MUXPOS — Input Mux

MUXPOS Channel

0x00	AIN0 (PA0 — also UPDI, avoid)
------	-------------------------------

0x01	AIN1 (PA1)
0x02	AIN2 (PA2)
0x03	AIN3 (PA3)
0x04	AIN4 (PA4)
0x05	AIN5 (PA5)
0x06	AIN6 (PA6)
0x07	AIN7 (PA7)
0x08	AIN8 (PB5)
0x09	AIN9 (PB4)
0x0A	AIN10 (PB1)
0x0B	AIN11 (PB0)
0x1B	Reserved for ADC0 / PTC
0x1C	DAC0
0x1D	INTREF (internal reference voltage)
0x1E	TEMPSENSE (internal temperature sensor)
0x1F	GND

A.12.4 Internal Reference Voltage

The internal reference is configured via VREF peripheral (base 0x00A0):

Register	Mem Addr	Description
VREF_CTRLA	0x00A0	ADC0 and DAC0 reference select
VREF_CTRLB	0x00A1	ADC0 reference force enable
VREF_CTRLA bits 6:4 (ADC0REFSEL):		
0x0	0.55V	
0x1	1.1V	
0x2	2.5V	
0x3	4.3V	
0x4	1.5V	

Default internal reference: 1.1V.

A.13 NVMCTRL — Non-Volatile Memory Controller

Base address: 0x1000. Writing flash or EEPROM requires writing the CCP_SPM key (0x9D) to CCP before the command, then completing the write within 4 cycles.

Register	Mem Addr	Description
NVMCTRL_CTRLA	0x1000	Command register (write CCP key first)
NVMCTRL_CTRLB	0x1001	Boot lock, application code write protect
NVMCTRL_STATUS	0x1002	Status flags (read-only)
NVMCTRL_INTCTRL	0x1003	EEREADY interrupt enable
NVMCTRL_INTFLAGS	0x1004	EEREADY interrupt flag (W1C)
NVMCTRL_DATA	0x1006	Data register (16-bit)
NVMCTRL_ADDR	0x1008	Address register (16-bit)

A.13.1 CTRLA — NVM Commands

CMD[2:0] (bits 2:0)	Command	
0x00	NOCMD	No operation
0x01	PAGEWRITE	Write page buffer to flash
0x02	PAGEERASE	Erase one page of flash

0x03	PAGEERASEWRITE	Erase and write in one operation
0x04	PAGEBUFCLR	Clear page write buffer
0x05	CHIPERASE	Erase entire chip (flash + EEPROM)
0x06	EEERASE	Erase EEPROM
0x07	FUSEWRITE	Write fuse bytes

A.13.2 STATUS Bits

Bit:	2	1	0
	WRERROR	EEBUSY	FBUSY

FBUSY 1 = flash write/erase in progress
 EEBUSY 1 = EEPROM write/erase in progress
 WRERROR 1 = write error occurred

A.13.3 Memory Layout

Region	Start	End	Size	Notes
Flash	0x0000	0x7FFF	32 KB	Program memory (word-addressed by PC)
SRAM	0x3800	0x3FFF	2 KB	Data memory (avr-ld origin 0x803800)
EEPROM	0x1400	0x14FF	256 B	(memory-mapped for read; use NVMCTRL to write)
Fuses	0x1280	0x128A		Written via UPDI/avrdude

Flash page size: 64 bytes. EEPROM page size: 32 bytes.

A.14 Interrupt Vector Table

All vectors are 4-byte (2-word) entries. Use JMP (4-byte instruction) for targets anywhere in the 32 KB flash, or RJMP + NOP (2+2 bytes) to keep the same slot size.

Vector	Byte	Addr	Name	Description
0	0x0000		RESET	Power-on / external / WDT reset
1	0x0004		CRCSCAN_NMI	CRC scan non-maskable interrupt
2	0x0008		BOD_VLM	Brown-out detection voltage level monitor
3	0x000C		PORTA_PORT	Port A pin-change interrupt
4	0x0010		PORTB_PORT	Port B pin-change interrupt
5	0x0014		PORTC_PORT	Port C pin-change interrupt
6	0x0018		RTC_CNT	RTC overflow / compare match
7	0x001C		RTC_PIT	RTC periodic interrupt timer
8	0x0020		TCA0_OVF / TCA0_LUNF	TCA0 overflow (single) / low underflow (split)
9	0x0024		TCA0_HUNF	TCA0 high byte underflow (split mode)
10	0x0028		TCA0_CMP0 / TCA0_LCMP0	TCA0 compare 0 match
11	0x002C		TCA0_CMP1 / TCA0_LCMP1	TCA0 compare 1 match
12	0x0030		TCA0_CMP2 / TCA0_LCMP2	TCA0 compare 2 match
13	0x0034		TCB0_INT	TCB0 capture interrupt
14	0x0038		TCB1_INT	TCB1 capture interrupt
15	0x003C		TWI0_TWIS	TWI0 slave interrupt
16	0x0040		TWI0_TWIM	TWI0 master interrupt
17	0x0044		SPI0_INT	SPI0 transfer complete
18	0x0048		USART0_RXC	USART0 receive complete
19	0x004C		USART0_DRE	USART0 data register empty
20	0x0050		USART0_TXC	USART0 transmit complete
21	0x0054		AC0_AC	Analog comparator interrupt
22	0x0058		ADC0_RESRDY	ADC0 result ready

23	0x005C	ADC0_WCMP	ADC0 window comparator match
24	0x0060	CCL_CCL	Configurable custom logic interrupt
25	0x0064	NVMCTRL_EE	EEPROM ready

Minimal startup with vector table in assembly:

```
.section .vectors, "ax", @progbits
jmp _start          ; 0x0000 RESET – must be first
jmp _unhandled      ; 0x0004 CRCSCAN_NMI
; ... one jmp per vector ...

.section .text
_unhandled:
    rjmp _unhandled    ; spin on unhandled interrupts

_start:
; ... init stack, SRAM, then jump to main
```

A.15 Fuses

Fuses are configuration bytes in the FUSES region (base 0x1280), written via UPDI using avrdude. Factory values are shown. Unlike ATmega, there is no separate CKDIV, BOOTRST, or BOOTSZ fuse; the boot/application split is set by BOOTEND and APPEND instead. The fuse offset within the region (0-based) is also the number avrdude uses for the fuseN memory name.

Off	Fuse	Mem Addr	Factory	Description
0x0	WDTCFG	0x1280	0x00	WINDOW[7:4], PERIOD[3:0] – watchdog
0x1	BODCFG	0x1281	0x00	LVL[7:5], SAMPFREQ[4], ACTIVE[3:2], SLEEP[1:0] – brown-out detector
0x2	OSCCFG	0x1282	0x02	OSLOCK[7], FREQSEL[1:0] – oscillator
0x3	(reserved)	0x1283	–	
0x4	TCD0CFG	0x1284	0x00	TCD0 compare/fault default outputs
0x5	SYSCFG0	0x1285	0xC4	CRCSRC[7:6], TOUTDIS[4], RSTPINCFG[3:2], EESAVE[0] – reset pin, CRC, EEPROM save
0x6	SYSCFG1	0x1286	0x07	SUT[2:0] – start-up time
0x7	APPEND	0x1287	0x00	Application code end page (256-byte units)
0x8	BOOTEND	0x1288	0x00	Boot section end page (0 = no boot section)
0xA	LOCKBIT	0x128A	0xC5	0xC5 = unlocked; any other value locks

A.15.1 OSCCFG (0x1282)

Bit: 7 2:0
 OSLOCK FREQSEL[1:0]

FREQSEL 0x1 = 16 MHz internal oscillator base
 0x2 = 20 MHz internal oscillator base ← factory default
 OSLOCK 1 = lock OSC20M calibration registers

Factory default OSCCFG = 0x02 (FREQSEL=0x2 → 20 MHz base). The clock prescaler in CLKCTRL_MCLKCTRLB further divides this: 20 MHz / 6 = 3.333 MHz at reset.

A.15.2 SYSCFG0 (0x1285)

Bit: 7:6 4 3:2 0
 CRCSRC TOUTDIS RSTPINCFG EESAVE

CRCSRC 0x0=full Flash, 0x1=boot, 0x2=boot+app, 0x3=no CRC (default 0x3)
 TOUTDIS 0 = NVM-write block after POR enabled, 1 = disabled
 RSTPINCFG 0x0=GPIO, 0x1=UPDI, 0x2=RESET (default 0x1 = UPDI)
 EESAVE 0 = EEPROM erased on chip erase, 1 = preserved

Factory default SYSCFG0 = 0xC4 (CRCSRC=no-CRC, RSTPINCFG=UPDI, EESAVE=erase).

A.15.3 Writing Fuses with avrdude

avrdude accepts the fuse names directly for this device:

Write a single fuse (OSCCFG = 20 MHz base)

```
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -b 115200 \
-U osccfg:w:0x02:m
```

Restore factory fuse values

```
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -b 115200 \
-U wdtcfg:w:0x00:m -U bodcfg:w:0x00:m -U osccfg:w:0x02:m \
-U tcd0cfg:w:0x00:m -U syscfg0:w:0xC4:m -U syscfg1:w:0x07:m \
-U append:w:0x00:m -U bootend:w:0x00:m
```

Warning: Setting RSTPINCFG to GPIO or RESET, or writing an invalid LOCKBIT key, can disable the UPDI programming interface or lock the device, effectively bricking it. Never modify SYSCFG0 or LOCKBIT unless certain.

Appendix B

AVR Instruction Set Summary

Complete instruction set for AVR (classic core, enhanced core with MUL). Columns: **Op-code** — mnemonic and operands; **Operation** — what it does; **Cycles** — clock cycles **for the AVRxt core** (the ATtiny3217's family; these differ from classic AVRe timings for several load/store and call instructions — branch cycles shown as not-taken/taken); **Flags** — SREG bits modified (I T H S V N Z C).

Rd = destination register (R0-R31 unless noted). Rr = source register. K = 8-bit constant (0-255). k = relative branch offset (−64..63 for 7-bit, −2048..2047 for 12-bit). b = bit index (0-7). A = I/O address (0-63). q = displacement (0-63) for Y/Z indirect with displacement.

B.1 How to Read the AVR Instruction Set Manual

This appendix is a quick reference. The Microchip instruction set manual is the authoritative source. If you only use the summary table, you can write code. If you learn to read the manual pages directly, you can answer harder questions:

- Why does this instruction only work on r16–r31?
- Why is this branch 1 cycle sometimes and 2 cycles other times?
- Why does SBIS skip differently from BRNE?
- Why does OUT work here but STS is required there?
- Why does an instruction update Z but not C?

The manual is not hard once you know what each field is trying to tell you. The problem is that it reads like a hardware document, not a tutorial.

B.1.1 What One Instruction Page Contains

Each instruction in the manual is usually presented in the same structure:

- **Description / operation:** what value changes logically.
- **Syntax:** the assembly form, such as ADD Rd, Rr.
- **Operands:** legal ranges, such as 0 ≤ d ≤ 31.
- **Program Counter:** how PC changes after execution.
- **Opcode:** the encoded bit pattern.
- **Status Register (SREG) and Boolean Formula:** which flags change and why.
- **Words / Cycles:** code size and execution time, often by AVR core family.

When reading a page, do not read it top to bottom like prose. Read it in this order instead:

1. **Syntax:** what form the instruction has.
2. **Operands:** which registers or constants are legal.
3. **Operation:** what value is computed.

4. **SREG effects**: which later branches or compares it can support.
5. **Words / Cycles**: whether it is a good fit for tight loops.
6. **Opcode**: only when you want to understand encoding limits or disassembly.

That order matches how you actually use the information while programming.

B.1.2 Start with Syntax, but Trust the Operand Limits

Beginners often read `LDI Rd, K` as “load any register with any 8-bit constant”. The syntax alone does not tell the whole story. The **Operands** field does:

```
LDI Rd, K
Operands: 16 <= d <= 31, 0 <= K <= 255
```

That means `ldi r0, 1` is impossible, not merely discouraged. The reason is in the opcode encoding: `LDI` only has four bits for the register number, so the encoded register is really `Rd - 16`.

This pattern appears all over AVR:

- `LDI`, `ANDI`, `ORI`, `SUBI`, `SBCI`, `CPI` only work on `r16–r31`.
- `ADIW` and `SBIW` only work on specific register pairs.
- `MOVW` only uses even-numbered register pairs.
- `SBI`, `CBI`, `SBIC`, `SBIS` only work on low I/O addresses.

If the syntax looks general but the operand range is narrow, believe the range. The encoding is the real reason.

B.1.3 Read “Words” Before “Cycles”

AVR documentation uses **words**, not bytes, as the natural unit of program storage. One word is 16 bits, so:

- **1 word** = 2 bytes
- **2 words** = 4 bytes

This matters because AVR instructions are either one word or two words. That affects all of these at once:

- code size in flash
- instruction fetch behaviour
- skip-instruction timing
- absolute vs relative addressing choices

Examples:

- `OUT` is one word; `STS` is two words.
- `RJMP` is one word; `JMP` is two words.
- `RCALL` is one word; `CALL` is two words.

So when the manual says an instruction is “Words: 2”, translate that immediately into “this costs 4 bytes and may also affect skip timing”.

B.1.4 Relative Addresses Are Not Absolute Addresses

AVR manuals use several similar-looking operand letters:

- `K` often means an immediate constant.
- `k` often means a relative code offset.
- `q` means a small displacement from `Y` or `Z`.
- `A` means a low I/O address.

Case matters. `K` and `k` are not interchangeable.

For branches and relative calls:

- RJMP *k* means jump relative to the current PC
- RCALL *k* means call relative to the current PC
- BRNE *k* means branch a short relative distance if $Z = 0$

The manual usually writes this as:

```
PC <- PC + k + 1
```

That + 1 exists because the PC normally advances to the next instruction word before the relative offset is applied. The important practical result is:

- RJMP and RCALL do not encode a full absolute address.
- BRxx instructions have much shorter range than RJMP.
- Disassemblers may show targets as labels, but the hardware stores offsets.

B.1.5 Read the SREG Section as “What Can I Test Next?”

The **Status Register (SREG)** is not just bookkeeping. It determines what branches are meaningful after an instruction.

The bits are:

- I: global interrupt enable
- T: transfer bit for BST / BLD
- H: half-carry, mainly for nibble/BCD-style carries
- S: sign bit, defined as $N \text{ xor } V$
- V: signed overflow
- N: negative
- Z: zero
- C: carry / borrow

When reading an instruction page, ask:

- Does this instruction update flags at all?
- Which flags does it update?
- Are those flags enough for unsigned comparison, signed comparison, or both?

Examples:

- ADD updates Z, C, N, V, S, H, so it supports later arithmetic decisions.
- INC does **not** update C, so it is not a drop-in replacement for ADD *Rd*, 1 if carry matters.
- TST is really a flag-producing logical test, useful before BREQ or BRMI.
- CP changes flags without changing registers, which is why branches usually follow it directly.

The fastest way to understand a flag table is to map it to branch usage:

- For **unsigned** comparisons, care mostly about C and Z.
- For **signed** comparisons, care mostly about S, V, N, and Z.
- For equality, care about Z.

That is why AVR has both unsigned-style branches (BRLO, BRSH) and signed-style branches (BRLT, BRGE).

B.1.6 Do Not Panic About the Boolean Formulas

The manual often gives flag equations in Boolean form. Those formulas are precise, but you usually do not need to derive them by hand while writing ordinary code.

Use them in three situations:

1. When a flag result surprises you.
2. When writing multi-byte arithmetic where carry/borrow chains matter.
3. When reviewing whether two instructions are truly interchangeable.

For most programming, this simpler interpretation is enough:

- C: carry out of bit 7 for addition, borrow behaviour for subtraction
- Z: result was zero
- N: result bit 7 is 1
- V: signed overflow happened
- S: signed comparison helper, $N \text{ xor } V$
- H: carry from bit 3 to bit 4

If you are not doing decimal-style arithmetic or bit-exact arithmetic tricks, H is usually less important than Z, C, and S.

B.1.7 Cycles Are Family-Specific

The instruction set manual often shows cycle counts across multiple AVR core families. This book targets the ATtiny3217, which is an **AVRxt** device, so the AVRxt column is the one that matters most here.

Do not assume:

- a timing number from an older ATmega necessarily matches AVRxt
- all loads and stores cost the same on every AVR family
- every manual example was written with your exact core in mind

When timing matters, read the correct family column first, then read any note attached to it. Notes are where the manual hides the details that break naive assumptions.

B.1.8 Skip Instructions Need Extra Attention

CPSE, SBRC, SBRS, SBIC, and SBIS do not branch. They **skip the next instruction** if the condition is true.

That sounds simple, but the manual’s word count matters here:

- If the next instruction is 1 word, the skip jumps over 2 bytes.
- If the next instruction is 2 words, the skip jumps over 4 bytes.
- That is why skip instructions have multiple cycle counts.

So this is safe:

```
sbrc r16, 0
rjmp bit_was_set
```

But this behaves differently from a true branch because the skip is based on the size of the *next* instruction, not on a stored relative destination.

When reading the manual, any cycle count like 1/2/3 or 2/3/4 should make you ask: “is this a skip instruction, and is the next instruction one or two words long?”

B.1.9 Learn the Address Spaces Separately

AVR documentation is much easier once you stop treating “memory” as one thing. The manual switches among several address spaces:

- **register file:** r0–r31
- **I/O space:** low peripheral addresses used by IN / OUT
- **data space:** SRAM and memory-mapped locations used by LD / ST / LDS / STS
- **program memory:** flash, read with LPM

Many confusing instruction choices become obvious once you ask which space is being accessed.

Example:

- OUT PORTA_DIR, r16 only works if that register is in low I/O space.
- STS PORTA_DIR, r16 works with a data-space address, but costs more.

- LPM is for flash, not SRAM.

If two instructions seem to do the same job, check whether they are really talking to different address spaces.

B.1.10 Aliases and Pseudo-Instructions Can Hide the Real Opcode

Assembly source often uses friendly names that are not separate hardware instructions.

Examples:

- CLR Rd is assembled as EOR Rd, Rd.
- SER Rd is assembled as LDI Rd, 0xFF.
- LSL Rd is effectively ADD Rd, Rd.
- named branches like BREQ and BRNE are readable aliases for specific BRBS / BRBC conditions.

That matters because the manual may document one spelling while the assembler accepts another. If something looks inconsistent, check whether you are dealing with:

- a real opcode
- an alias mnemonic
- a pseudo-instruction expanded by the assembler

B.1.11 A Good Manual-Reading Workflow

When you meet an unfamiliar instruction, use this checklist:

1. Read the syntax and operand limits.
2. Identify the address space involved: register, I/O, SRAM, or flash.
3. Check whether it is 1 word or 2 words.
4. Check which SREG flags it updates.
5. Check whether the cycle count depends on branch taken/not taken, skip size, or AVR core family.
6. Check whether there is a shorter equivalent instruction for your case.

That method is enough to decode most of the manual quickly.

B.1.12 What This Means for the Rest of the Book

Use Chapter 6 to learn the common instruction patterns, then come back to this appendix when you need the full table. Use the Microchip manual when a question depends on one of these details:

- exact flag behaviour
- exact operand limits
- instruction encoding constraints
- family-specific cycle counts
- special notes about program memory, skip timing, or core differences

Once you start reading the manual this way, it stops looking like a wall of tables and starts looking like a set of answers to very specific questions.

B.2 Arithmetic and Logic

Opcode	Operation	Cycles	Flags
ADD Rd, Rr	$Rd \leftarrow Rd + Rr$	1	H S V N Z C
ADC Rd, Rr	$Rd \leftarrow Rd + Rr + C$	1	H S V N Z C

ADIW Rd, K	$Rd+1:Rd \leftarrow Rd+1:Rd + K$ (0–63)	2	S V N Z C
SUB Rd, Rr	$Rd \leftarrow Rd - Rr$	1	H S V N Z C
SUBI Rd, K	$Rd \leftarrow Rd - K$	1	H S V N Z C
SBC Rd, Rr	$Rd \leftarrow Rd - Rr - C$	1	H S V N Z C
SBCI Rd, K	$Rd \leftarrow Rd - K - C$	1	H S V N Z C
SBIW Rd, K	$Rd+1:Rd \leftarrow Rd+1:Rd - K$ (0–63)	2	S V N Z C
AND Rd, Rr	$Rd \leftarrow Rd \text{ AND } Rr$	1	S V N Z
ANDI Rd, K	$Rd \leftarrow Rd \text{ AND } K$ (R16–R31)	1	S V N Z
OR Rd, Rr	$Rd \leftarrow Rd \text{ OR } Rr$	1	S V N Z
ORI Rd, K	$Rd \leftarrow Rd \text{ OR } K$ (R16–R31)	1	S V N Z
EOR Rd, Rr	$Rd \leftarrow Rd \text{ XOR } Rr$	1	S V N Z
COM Rd	$Rd \leftarrow 0xFF - Rd$ (bitwise NOT)	1	S V N Z C
NEG Rd	$Rd \leftarrow 0x00 - Rd$ (two's comp)	1	H S V N Z C
INC Rd	$Rd \leftarrow Rd + 1$	1	S V N Z
DEC Rd	$Rd \leftarrow Rd - 1$	1	S V N Z
MUL Rd, Rr	$R1:R0 \leftarrow Rd \times Rr$ (unsigned)	2	Z C
MULS Rd, Rr	$R1:R0 \leftarrow Rd \times Rr$ (signed)	2	Z C
	Rd, Rr: R16–R31 only		
MULSU Rd, Rr	$R1:R0 \leftarrow Rd \times Rr$ (sgn $\times$ uns)	2	Z C
	Rd, Rr: R16–R23 only		
FMUL Rd, Rr	$R1:R0 \leftarrow (Rd \times Rr) \ll 1$ (uns)	2	Z C
FMULS Rd, Rr	$R1:R0 \leftarrow (Rd \times Rr) \ll 1$ (sgn)	2	Z C
FMULSU Rd, Rr	$R1:R0 \leftarrow (Rd \times Rr) \ll 1$ (s $\times$ u)	2	Z C

B.3 Shift and Rotate

Opcode	Operation	Cycles	Flags
LSL Rd	$Rd \leftarrow Rd \ll 1$; $Rd0=0$; $C \leftarrow Rd7$	1	H S V N Z C
LSR Rd	$Rd \leftarrow Rd \gg 1$; $Rd7=0$; $C \leftarrow Rd0$	1	S V N Z C
ASR Rd	$Rd \leftarrow Rd \gg 1$; $Rd7$ unchanged	1	S V N Z C
ROL Rd	$Rd \leftarrow Rd \ll 1$; $Rd0=C$; $C \leftarrow Rd7$	1	H S V N Z C
ROR Rd	$Rd \leftarrow Rd \gg 1$; $Rd7=C$; $C \leftarrow Rd0$	1	S V N Z C
SWAP Rd	$Rd[7:4] \leftrightarrow Rd[3:0]$	1	—

B.4 Bit Operations

Opcode	Operation	Cycles	Flags
SBI A, b	$I/O[A][b] \leftarrow 1$ (A: 0–31)	1	—
CBI A, b	$I/O[A][b] \leftarrow 0$ (A: 0–31)	1	—
BST Rd, b	$T \leftarrow Rd[b]$	1	T
BLD Rd, b	$Rd[b] \leftarrow T$	1	—
SEC	$C \leftarrow 1$	1	C
CLC	$C \leftarrow 0$	1	C
SEN	$N \leftarrow 1$	1	N
CLN	$N \leftarrow 0$	1	N
SEZ	$Z \leftarrow 1$	1	Z
CLZ	$Z \leftarrow 0$	1	Z
SEI	$I \leftarrow 1$ (global interrupt on)	1	I
CLI	$I \leftarrow 0$ (global interrupt off)	1	I
SES	$S \leftarrow 1$	1	S
CLS	$S \leftarrow 0$	1	S

SEV	$V \leftarrow 1$	1	V
CLV	$V \leftarrow 0$	1	V
SET	$T \leftarrow 1$	1	T
CLT	$T \leftarrow 0$	1	T
SEH	$H \leftarrow 1$	1	H
CLH	$H \leftarrow 0$	1	H
BSET b	$SREG[b] \leftarrow 1$	1	(b)
BCLR b	$SREG[b] \leftarrow 0$	1	(b)

B.5 Compare and Test

Opcode	Operation	Cycles	Flags
CP Rd, Rr	$Rd - Rr$ (flags only)	1	H S V N Z C
CPC Rd, Rr	$Rd - Rr - C$ (flags only)	1	H S V N Z C
CPI Rd, K	$Rd - K$ (flags only, R16–R31)	1	H S V N Z C
TST Rd	Rd AND Rd (flags only)	1	S V N Z

B.6 Branch Instructions

Opcode	Condition	Cycles	Notes
RJMP k	—	2	$PC \leftarrow PC + k + 1$; k: -2048..2047
JMP k	—	3	$PC \leftarrow k$; 32-bit instruction
IJMP	—	2	$PC \leftarrow Z$ (16-bit)
EIJMP	—	2	$PC \leftarrow EIND:Z$ (>128KB flash)
RCALL k	—	2	Push PC, $PC \leftarrow PC + k + 1$
CALL k	—	3	Push PC, $PC \leftarrow k$; 32-bit
ICALL	—	2	Push PC, $PC \leftarrow Z$
EICALL	—	3	Push PC, $PC \leftarrow EIND:Z$
RET	—	4	Pop PC
RETI	—	4	Pop PC, $I \leftarrow 1$
BRBS b, k	$SREG[b] = 1$	1/2	Branch if bit set; 1=not taken, 2=taken
BRBC b, k	$SREG[b] = 0$	1/2	Branch if bit clear
/* Named branch mnemonics (aliases for BRBS/BRBC) */			
BREQ k	$Z = 1$	1/2	Branch if equal
BRNE k	$Z = 0$	1/2	Branch if not equal
BRCS k	$C = 1$	1/2	Branch if carry set
BRCC k	$C = 0$	1/2	Branch if carry clear
BRSH k	$C = 0$	1/2	Branch if same or higher (unsigned $\geq$)
BRL0 k	$C = 1$	1/2	Branch if lower (unsigned $<$)
BRMI k	$N = 1$	1/2	Branch if minus
BRPL k	$N = 0$	1/2	Branch if plus
BRGE k	$S = 0$	1/2	Branch if greater or equal (signed $\geq$)
BRLT k	$S = 1$	1/2	Branch if less than (signed $<$)
BRHS k	$H = 1$	1/2	Branch if half-carry set
BRHC k	$H = 0$	1/2	Branch if half-carry clear
BRTS k	$T = 1$	1/2	Branch if T set
BRTC k	$T = 0$	1/2	Branch if T clear
BRVS k	$V = 1$	1/2	Branch if overflow set

BRVC k	V = 0	1/2	Branch if overflow clear
BRIE k	I = 1	1/2	Branch if interrupt enabled
BRID k	I = 0	1/2	Branch if interrupt disabled

B.6.1 Skip Instructions

Opcode	Condition	Cycles	Notes
CPSE Rd, Rr	Rd = Rr → skip next	1/2/3	skip 1 or 2-word instruction
SBRC Rd, b	Rd[b] = 0 → skip	1/2/3	
SBRS Rd, b	Rd[b] = 1 → skip	1/2/3	
SBIC A, b	I/O[A][b] = 0 → skip	1/2/3	A: 0–31 only
SBIS A, b	I/O[A][b] = 1 → skip	1/2/3	A: 0–31 only

Skip cycle counts: 1 = no skip; 2 = skip 1-word instruction; 3 = skip 2-word instruction (LDS, STS, CALL, JMP). On AVRxt SBIC/SBIS cost the same as SBRC/SBRS (1/2/3) — unlike older AVR_{Re}/AVR_{Rxm} cores, there is no extra I/O read penalty.

B.7 Load and Store

Opcode	Operation	Cycles	Notes
MOV Rd, Rr	Rd ← Rr	1	
MOVW Rd, Rr	Rd+1:Rd ← Rr+1:Rr	1	Rd, Rr: even registers
LDI Rd, K	Rd ← K (R16–R31 only)	1	
LDS Rd, k	Rd ← SRAM[k]	3	k: 16-bit address
STS k, Rr	SRAM[k] ← Rr	2	k: 16-bit address
/* X register (R27:R26) indirect */			
LD Rd, X	Rd ← SRAM[X]	2	
LD Rd, X+	Rd ← SRAM[X]; X ← X+1	2	
LD Rd, -X	X ← X-1; Rd ← SRAM[X]	2	
ST X, Rr	SRAM[X] ← Rr	1	
ST X+, Rr	SRAM[X] ← Rr; X ← X+1	1	
ST -X, Rr	X ← X-1; SRAM[X] ← Rr	1	
/* Y register (R29:R28) indirect */			
LD Rd, Y	Rd ← SRAM[Y]	2	
LD Rd, Y+	Rd ← SRAM[Y]; Y ← Y+1	2	
LD Rd, -Y	Y ← Y-1; Rd ← SRAM[Y]	2	
LDD Rd, Y+q	Rd ← SRAM[Y+q] q: 0–63	2	
ST Y, Rr	SRAM[Y] ← Rr	1	
ST Y+, Rr	SRAM[Y] ← Rr; Y ← Y+1	1	
ST -Y, Rr	Y ← Y-1; SRAM[Y] ← Rr	1	
STD Y+q, Rr	SRAM[Y+q] ← Rr q: 0–63	1	
/* Z register (R31:R30) indirect */			
LD Rd, Z	Rd ← SRAM[Z]	2	
LD Rd, Z+	Rd ← SRAM[Z]; Z ← Z+1	2	
LD Rd, -Z	Z ← Z-1; Rd ← SRAM[Z]	2	
LDD Rd, Z+q	Rd ← SRAM[Z+q] q: 0–63	2	
ST Z, Rr	SRAM[Z] ← Rr	1	
ST Z+, Rr	SRAM[Z] ← Rr; Z ← Z+1	1	
ST -Z, Rr	Z ← Z-1; SRAM[Z] ← Rr	1	
STD Z+q, Rr	SRAM[Z+q] ← Rr q: 0–63	1	

On AVRxt, indirect/displacement stores (ST, STD) take **1** cycle and LDS takes **3**; these differ from the classic AVRc values (2 each). LD, LDD, STS, POP, IN, and OUT are unchanged.

B.7.1 Load from Program Memory (Flash)

Opcode	Operation	Cycles	Notes
LPM	$R0 \leftarrow \text{FLASH}[Z]$	3	Z = byte address
LPM Rd, Z	$Rd \leftarrow \text{FLASH}[Z]$	3	
LPM Rd, Z+	$Rd \leftarrow \text{FLASH}[Z]; Z \leftarrow Z+1$	3	>64KB flash devices
ELPM	$R0 \leftarrow \text{FLASH}[\text{RAMPZ}:Z]$	3	
ELPM Rd, Z	$Rd \leftarrow \text{FLASH}[\text{RAMPZ}:Z]$	3	
ELPM Rd, Z+	$Rd \leftarrow \text{FLASH}[\text{RAMPZ}:Z]; Z++$	3	Boot section only
SPM	$\text{FLASH}[Z] \leftarrow R1:R0$	varies	

B.8 I/O Instructions

Opcode	Operation	Cycles	Notes
IN Rd, A	$Rd \leftarrow \text{I/O}[A]$	1	A: 0–63
OUT A, Rr	$\text{I/O}[A] \leftarrow Rr$	1	A: 0–63
PUSH Rr	$\text{SRAM}[SP] \leftarrow Rr; SP \leftarrow SP-1$	1	
POP Rd	$SP \leftarrow SP+1; Rd \leftarrow \text{SRAM}[SP]$	2	

B.9 Miscellaneous

Opcode	Operation	Cycles	Notes
NOP	No operation	1	
SLEEP	Enter sleep mode	1	MCU-defined behaviour
WDR	Reset watchdog timer	1	
BREAK	Stop for on-chip debug system	1	

B.10 Instruction Encoding Summary

Most AVR instructions are 16-bit (one word). 32-bit instructions:

32-bit instructions (2 words):

- LDS Rd, k – word 1: opcode+Rd; word 2: 16-bit address k
- STS k, Rr – word 1: opcode+Rr; word 2: 16-bit address k
- CALL k – word 1: 0x940E+kH; word 2: kL (22-bit address)
- JMP k – word 1: 0x940C+kH; word 2: kL (22-bit address)

All other instructions are 16-bit. The assembler handles encoding automatically.

B.11 GCC ABI Register Conventions

Registers	Role	Save by
-----------	------	---------

R0	Temporary, MUL result	Caller (assume destroyed)
R1	Always zero (GCC)	Callee must restore if modified
R2–R17	Call-saved	Callee (push/pop if used)
R18–R27	Call-clobbered	Caller (assume destroyed)
R28–R29 (Y)	Frame pointer	Callee
R30–R31 (Z)	Call-clobbered	Caller

Return values:

8-bit: R24

16-bit: R25:R24

32-bit: R25:R24:R23:R22 (little-endian: R22=lowest byte)

Arguments (first to last):

arg1: R25:R24 (or R24 for 8-bit)

arg2: R23:R22

arg3: R21:R20

arg4: R19:R18

More: pushed on stack (rightmost first)

After MUL/MULS/MULSU: always restore R1 = 0 before returning or calling any C function. R0 is implicitly scratch.

Appendix C

GAS Directive Reference

GNU Assembler (avr-as) directives used in AVR assembly. Directives begin with `.` and are not instructions — they control how the assembler lays out output.

C.1 Section Directives

Directive	Effect
<code>.section name [,"flags"[@type]]</code>	Switch to named section. Common flags: a = allocatable w = writable x = executable Types: @progbits (data), @nobits (BSS, zero-init)
<code>.text</code>	Switch to .text (code, flash)
<code>.data</code>	Switch to .data (initialised SRAM variables)
<code>.bss</code>	Switch to .bss (zero-initialised SRAM)
<code>.rodata</code>	Read-only data (constants in flash on AVR)

Common section usage on AVR:

```
.section .text          /* executable code (flash) */
.section .data          /* initialised variables (SRAM; copied from flash at startup) */
.section .bss           /* zero-initialised variables (SRAM) */
.section .rodata        /* constants (flash; access via LPM) */
.section .vectors,"ax",@progbits /* interrupt vector table */
.section .noinit,"aw",@nobits  /* SRAM variables NOT initialised at reset */
```

C.2 Symbol Directives

Directive	Effect
<code>.global symbol</code>	Export symbol (visible to linker / other files)
<code>.local symbol</code>	Mark symbol local (file scope only)
<code>.weak symbol</code>	Weak symbol: overridden if another file defines it
<code>.type symbol, @function</code>	Mark symbol as function (for debugging/ELF info)
<code>.type symbol, @object</code>	Mark symbol as data object
<code>.size symbol, size</code>	Set symbol size (bytes)

```
.set    symbol, expr      Define symbol = expr (reassignable)
.equ    symbol, expr      Define symbol = expr (reassignable; synonym of .set)
.equiv  symbol, expr      Like .equ, but errors if symbol already defined
```

Example:

```
.global uart_init
.type   uart_init, @function
uart_init:
    /* ... */
    ret
.size   uart_init, . - uart_init /* size = current address minus symbol start */
```

C.3 Data Directives

Directive	Size per item	Example
<code>.byte expr[,...]</code>	1 byte	<code>.byte 0x3F, 42, 'A'</code>
<code>.2byte expr[,...]</code>	2 bytes LE	<code>.2byte 0x1234</code> /* = 0x34, 0x12 in memory */
<code>.word expr[,...]</code>	2 bytes LE	<code>.word 1000</code> /* AVR word = 2 bytes */
<code>.long expr[,...]</code>	4 bytes LE	<code>.long 0xDEADBEEF</code>
<code>.quad expr[,...]</code>	8 bytes LE	<code>.quad 0</code>
<code>.ascii "str"</code>	n bytes	<code>.ascii "hello"</code> /* no null terminator */
<code>.asciz "str"</code>	n+1 bytes	<code>.asciz "hello"</code> /* null terminated */
<code>.string "str"</code>	n+1 bytes	<code>.string "hello"</code> /* alias for .asciz */
<code>.space n [,fill]</code>	n bytes	<code>.space 16, 0xFF</code> /* fill with 0xFF */
<code>.zero n</code>	n zero bytes	<code>.zero 64</code> /* alias for .space n, 0 */
<code>.skip n [,fill]</code>	n bytes	<code>.skip 4</code> /* alias for .space */
<code>.fill reps,size,val</code>		<code>.fill 8, 1, 0xAA</code> /* 8 bytes of 0xAA */

C.4 Alignment Directives

Directive	Effect
<code>.align n</code>	Align to 2 ⁿ byte boundary (with NOP padding in .text)
<code>.balign n [,fill]</code>	Align to n byte boundary
<code>.p2align n</code>	Align to 2 ⁿ boundary (same as .align on most targets)
<code>.org expr</code>	Set location counter to expr (absolute address)

On AVR, `.align 1` aligns to 2-byte boundary (one AVR word). `.align 2` aligns to 4 bytes. Useful before jump tables and 16-bit data.

C.5 Conditional Assembly

Directive	Effect
<code>.ifdef sym</code>	Include if sym is defined
<code>.ifndef sym</code>	Include if sym is NOT defined
<code>.if expr</code>	Include if expr != 0
<code>.else</code>	Else branch
<code>.elif expr</code>	Else-if
<code>.endif</code>	End conditional block

```
.error "msg"      Abort assembly with message
.warning "msg"    Emit warning, continue
```

Example — select multiply implementation at assemble time:

```
#ifdef NOMUL
    rcall mul8u_shiftadd
#else
    mul    r16, r17
    movw   r24, r0
    clr    r1
#endif
```

When using `avr-gcc` as the front-end, `#ifdef/#define` (C preprocessor) work in `.S` files. For plain `avr-as`, use `.ifdef/.equ`.

C.6 Macro Directives

Directive	Effect
<code>.macro name [params]</code>	Begin macro definition
<code>.endm</code>	End macro definition
<code>.exitm</code>	Exit macro early
<code>.rept n</code>	Repeat enclosed block <i>n</i> times
<code>.endr</code>	End <code>.rept</code> block
<code>.irp sym, values</code>	Iterate: <i>sym</i> takes each value in turn
<code>.endr</code>	
<code>.irpc sym, chars</code>	Iterate over characters of a string
<code>.endr</code>	

Example — macro for a 16-bit load:

```
.macro ldw reg_h, reg_l, value
    ldi  \reg_l, lo8(\value)
    ldi  \reg_h, hi8(\value)
.endm

/* Usage */
ldw    r25, r24, 1999    /* expands to: ldi r24, lo8(1999); ldi r25, hi8(1999) */
```

Example — `.rept` for vector table padding:

```
__vectors:
    rjmp reset_handler
    .rept 25
    rjmp dummy_isr        /* fill remaining 25 vectors */
    .endr
```

C.7 Include Directives

Directive	Effect
<code>.include "file"</code>	Textually include file at this point
<code>#include <file></code>	C preprocessor include (only when assembled via <code>avr-gcc</code>)

For AVR with `avr-gcc -x assembler-with-cpp` (the default for `.S` files):

```
#include <avr/io.h>          /* defines I/O register names, bit names, RAMEND, etc. */
```

<avr/io.h> selects the correct device header based on -mmcu= flag.

C.8 Listing and Debug Directives

Directive	Effect
.file "name"	Set source file name (for debug info)
.line n	Set source line number
.loc file line [col]	DWARF location (used by avr-gcc; rarely hand-written)
.stabs "str",n,n,n,n	STABS debug info (legacy)
.ident "string"	Embed comment string in .comment ELF section

C.9 Expression Operators

GAS expressions can use these operators in directives and immediate operands:

Operator	Example	Meaning
+	.byte 1+2	Addition
-	.byte 5-3	Subtraction
*	.word 16*1000	Multiplication
/	.byte 255/2	Integer division
%	.byte 17%10	Modulo
<<	.word 1<<8	Left shift
>>	.word 256>>1	Right shift
&	.byte 0xFF & val	Bitwise AND
	.byte flags mask	Bitwise OR
^	.byte a ^ b	Bitwise XOR
~	.byte ~mask	Bitwise NOT
lo8(x)	ldi r16, lo8(val)	Low byte of 16-bit value
hi8(x)	ldi r16, hi8(val)	High byte of 16-bit value
hlo8(x)		Byte 2 (bits 23:16) of 32-bit value
hhi8(x)		Byte 3 (bits 31:24) of 32-bit value
pm(x)	.word pm(label)	Program memory word address (byte addr >> 1)
.	(dot)	Current location counter (address)

C.10 Common Patterns

C.10.1 Define a constant

```
.equ F_CPU,    16000000
.equ BAUD,    9600
.equ UBRR_V,  (F_CPU / (16 * BAUD)) - 1  /* = 103 */
```

C.10.2 Reserve SRAM variable

```
.section .bss
my_var: .byte 0      /* 1-byte variable */
my_word: .2byte 0    /* 2-byte variable, aligned */
my_buf: .space 64    /* 64-byte buffer */
```

C.10.3 Initialised variable (copied to SRAM at startup by avr-libc __init)

```
.section .data
init_val: .byte 42
```

C.10.4 Constant in flash (accessed via LPM)

```
.section .rodata
my_table:
.byte 0, 1, 1, 2, 3, 5, 8, 13 /* Fibonacci */
```

C.10.5 Local labels

```
func:
    ldi    r16, 10
1:      dec    r16          /* local label – referenced as 1f (forward) or 1b (back) */
    brne   1b             /* branch back to label 1 */
    ret
```

Local labels (numeric) can be reused across the file; 1f means “next 1: going forward”, 1b means “previous 1: going backward”. They do not pollute the symbol table.

Appendix D

Linker Script Walkthrough

The linker script tells `avr-ld` where to place each section in the output ELF file and ultimately in the device's address space. For most projects `avr-gcc` selects a built-in script automatically via `-mmcu=`. Understanding linker scripts is necessary when:

- Writing a bootloader (section placement at a specific flash address).
- Placing variables in a specific SRAM region.
- Using `.noinit` for variables that survive a software reset.
- Adding custom sections (e.g., a parameter page at a known flash address).

Note on the target device: the worked examples below use the ATmega328P because its memory map is the one most readers have seen. The concepts are identical for the book's ATtiny3217, but two numbers change: flash is still 32 KB (0x0000-0x7FFF), while SRAM is 2 KB at hardware address 0x3800, so the `avr-ld` data origin is **0x803800** (not 0x800100) and `RAMEND` is 0x803FFF. Substitute those values, and `-mmcu=attiny3217`, when adapting a script.

D.1 Anatomy of a Linker Script

A linker script has three main constructs:

`MEMORY { }` – declares memory regions (name, origin, length)
`SECTIONS { }` – maps input sections to output sections and memory regions
`PROVIDE() / ENTRY()` – define special symbols and the entry point

D.1.1 Minimal AVR Linker Script

```
/* minimal_avr.ld – bare-bones script for ATmega328P */

MEMORY
{
    flash    (rx)  : ORIGIN = 0x000000, LENGTH = 32K
    sram      (rw!x): ORIGIN = 0x800100, LENGTH = 2K
}

SECTIONS
{
    /* Interrupt vector table – must be at flash origin */
    .vectors :
    {
        KEEP(*(.vectors))
    }
}
```



```

} > flash

/* Executable code */
.text :
{
    *(.text)
    *(.text.*)
} > flash

/* Read-only data (constants accessed via LPM) */
.rodata :
{
    *(.rodata)
    *(.rodata.*)
} > flash

/* Initialised data: stored in flash, copied to SRAM at startup */
.data :
{
    _data_start = .;
    *(.data)
    *(.data.*)
    _data_end = .;
} > sram AT > flash          /* VMA in sram, LMA in flash */

/* Zero-initialised data: only reservation in SRAM */
.bss :
{
    _bss_start = .;
    *(.bss)
    *(.bss.*)
    *(COMMON)
    _bss_end = .;
} > sram

/* Variables preserved across software reset (not zeroed at startup) */
.noinit (NOLOAD) :
{
    *(.noinit)
} > sram

/* Stack: grows downward from RAMEND */
_stack_top = ORIGIN(sram) + LENGTH(sram) - 1;
}

```

D.2 MEMORY Region Flags

r – readable
 w – writable
 x – executable
 ! – negate following flags

Common combinations:

(rx) – read-execute (flash)
 (rw!x) – read-write, not executable (SRAM)

(rx!w) – read-execute, not writable (flash, explicit)

D.3 Address Spaces on AVR

AVR uses a **Harvard architecture** with separate address spaces:

Flash (program memory):

avr-ld uses byte addresses with origin 0x000000
 ATmega328P: 0x000000–0x007FFF (32 KB)

SRAM (data memory):

avr-ld uses byte addresses with origin 0x800000 + hardware_offset
 ATmega328P hardware SRAM starts at 0x0100
 avr-ld SRAM origin: 0x800100

Registers: 0x0000–0x001F (R0–R31) – not in linker script
 I/O: 0x0020–0x005F – not in linker script
 Ext I/O: 0x0060–0x00FF – not in linker script
 SRAM: 0x0100–0x08FF (2 KB on ATmega328P)

The 0x800000 offset is an avr-ld convention to distinguish data addresses from code addresses in the ELF file. The hardware only sees the lower 16 bits.

D.4 VMA vs LMA

VMA (Virtual Memory Address): where the section lives at runtime (SRAM for .data).

LMA (Load Memory Address): where the section is stored in the ELF output (flash for .data, since flash is what gets programmed).

```
.data :
{
  _data_start = .;          /* VMA: SRAM address where data lives at runtime */
  *(.data)
  _data_end = .;
} > sram AT > flash        /* LMA: in flash, immediately after .rodata */
```

The startup code (__init in avr-libc) copies .data from its LMA (flash) to its VMA (SRAM) before main() runs:

```
/* avr-libc __init pseudo-code */
ldi ZL, lo8(_data_lma_start) /* source: flash LMA */
ldi ZH, hi8(_data_lma_start)
ldi YL, lo8(_data_start)    /* dest: SRAM VMA */
ldi YH, hi8(_data_start)
1: lpm r0, Z+
st Y+, r0
cpi ZL, lo8(_data_lma_end)
brne 1b
```

If you use -nostartfiles, **you must perform this copy yourself** if any .data variables are used. The simplest approach: avoid initialised variables entirely, or use .equ constants in flash and load them explicitly.

D.5 KEEP()

The garbage-collector (`--gc-sections`) removes unreferenced sections. `KEEP()` prevents this:

```
KEEP(*(.vectors))    /* never discard the vector table */
```

When building with `avr-gcc -Wl,--gc-sections` (common for size optimisation), sections not reachable from the entry point are removed. `KEEP()` forces retention regardless of reachability.

D.6 Bootloader Placement

Place the bootloader at the boot section start address by overriding `.text` origin:

```
avr-gcc -Wl,--section-start=.text=0x7C00 ...
```

Or in a dedicated linker script:

```
/* bootloader.ld */
MEMORY
{
    boot_flash (rx) : ORIGIN = 0x007C00, LENGTH = 1K
    sram        (rw!x): ORIGIN = 0x800100, LENGTH = 2K
}

SECTIONS
{
    .text : { *(.vectors) *(.text) *(.text.*) } > boot_flash
    .bss  : { *(.bss) } > sram
}

avr-gcc -mmcu=atmega328p -nostartfiles \
    -T bootloader.ld \
    -o bootloader.elf bootloader.S
```

D.7 Parameter Page at Fixed Flash Address

Store calibration data at a known, fixed flash address so the application can find it regardless of code size changes:

```
SECTIONS
{
    /* ... normal sections ... */

    .params :
    {
        *(.params)
    } > flash
    . = 0x7000; /* force next section to start at 0x7000 */
    .calibration :
    {
        KEEP(*(.calibration))
    } > flash
}
```

In source:

```
.section .calibration
cal_offset: .word 0x0032    /* factory calibration: ADC offset */
cal_gain:   .word 0x0100    /* Q8.8 gain factor */
```

Access at runtime:

```
ldi  ZL, lo8(cal_offset)
ldi  ZH, hi8(cal_offset)
lpm  r24, Z+
lpm  r25, Z    /* r25:r24 = cal_offset value */
```

D.8 .noinit Section

Variables in `.noinit` are **not zeroed** at startup — they retain whatever value was in SRAM before reset. Useful for reset reason tracking:

```
.section .noinit,"aw",@nobits
reset_reason: .byte 0    /* written before software reset; survives it */
```

Application writes a code before resetting:

```
ldi  r16, 0xAA
sts  reset_reason, r16
/* ... trigger watchdog reset ... */
```

After reset, check `reset_reason` before the BSS clear to determine why the device restarted. BSS clear (zeroing `.bss`) happens in `__init` — `.noinit` is placed in a separate section that `__init` never touches.

D.9 Useful Linker Symbols

These symbols are defined by the default `avr-gcc` linker script and available in assembly via `lo8()/hi8()`:

Symbol	Value
<code>__data_start</code>	VMA start of <code>.data</code> in SRAM
<code>__data_end</code>	VMA end of <code>.data</code> in SRAM
<code>__data_load_start</code>	LMA of <code>.data</code> in flash (copy source for <code>__init</code>)
<code>__bss_start</code>	Start of <code>.bss</code> in SRAM
<code>__bss_end</code>	End of <code>.bss</code> in SRAM
<code>__heap_start</code>	Start of heap (after <code>.bss</code>)
<code>__stack</code>	Initial stack pointer value (= <code>RAMEND</code>)
<code>__TEXT_REGION_ORIGIN__</code>	Start of <code>.text</code> (usually 0)
<code>__DATA_REGION_ORIGIN__</code>	Start of SRAM (usually <code>0x800100</code>)

Access in assembly:

```
ldi  r26, lo8(__bss_start)
ldi  r27, hi8(__bss_start)
ldi  r24, lo8(__bss_end - __bss_start) /* BSS length */
```

D.10 Inspecting Linker Output

```
# Show section layout (addresses, sizes)
avr-objdump -h firmware.elf

# Show all symbols with addresses
avr-nm --numeric-sort firmware.elf

# Show memory map (verbose linker output)
avr-gcc -Wl,-Map=firmware.map ... && cat firmware.map

# Show disassembly with source interleaved
avr-objdump -dS firmware.elf

# Verify flash usage
avr-size --mcu=atmega328p --format=avr firmware.elf
```

Sample avr-size output with AVR format:

AVR Memory Usage

Device: atmega328p

Program: 486 bytes (1.5% Full)
(.text + .data + .bootloader)

Data: 3 bytes (0.1% Full)
(.data + .bss + .noinit)

D.11 Common Linker Errors

Error	Cause / Fix
undefined reference to `symbol'	Symbol not defined in any linked file; add the .S or .o file that defines it.
relocation truncated to fit: R_AVR_13_PCREL against `symbol'	Branch offset exceeds range (e.g. RJMP only reaches $\pm 2\text{KB}$); use JMP or reorganise code.
section `.text' will not fit in region `flash'	Total code exceeds flash; reduce size or use a larger device.
cannot find entry symbol reset_handler	Define reset_handler or use -e reset_handler linker flag.
multiple definition of `__vector_N'	Two .S files both define the same ISR vector entry; remove the duplicate.

Appendix E

avrdude Flashing Cheat-Sheet

avrdude is the standard tool for programming AVR microcontrollers. It communicates with the device via a programmer (USB, serial, SPI ISP) and can read/write flash, EEPROM, and fuse bytes.

E.1 Basic Syntax

```
avrdude -p PARTNO -c PROGRAMMER [-P PORT] [-b BAUD] -U MEMTYPE:OP:FILE[:FORMAT]
```

-p PARTNO Device (e.g. m328p, t3217, m2560). See Section E.4 for list.
-c PROGRAMMER Programmer type. See Section E.3.
-P PORT Port (e.g. /dev/ttyUSB0, /dev/ttyACM0, usb).
-b BAUD Baud rate (for serial programmers; default depends on programmer).
-U Memory operation (can be repeated for multiple operations).
-v Verbose output. -vv for more detail.
-n Dry run: do not write anything.
-e Erase flash before programming.
-F Override signature check (use with caution).

E.1.1 -U Flag Format

MEMTYPE:OP:FILE[:FORMAT]

MEMTYPE:

flash – program memory
eeprom – EEPROM
lfuse – low fuse byte
hfuse – high fuse byte
efuse – extended fuse byte
lock – lock bits
signature – device signature (read only)
calibration – oscillator calibration (read only)

OP:

r – read from device, write to FILE
w – write FILE to device
v – verify device matches FILE

FORMAT (optional, auto-detected if omitted):

i – Intel HEX

s – Motorola S-record
r – raw binary
e – ELF
h – hex string
b – byte value (for single-byte ops like fuses)
m – immediate value (e.g. 0xDE)

E.2 Most-Used Commands

E.2.1 Flash a HEX file

```
# Arduino Uno (ATmega328P, USB-serial bootloader on /dev/ttyUSB0)
avrdude -p m328p -c arduino -P /dev/ttyUSB0 -b 115200 \
        -U flash:w:firmware.hex:i

# USBasp programmer (no port needed)
avrdude -p m328p -c usbasp \
        -U flash:w:firmware.hex:i

# AVRISP mkII (USB)
avrdude -p m328p -c avrispmkII \
        -U flash:w:firmware.hex:i
```

E.2.2 Read flash to file

```
avrdude -p m328p -c usbasp \
        -U flash:r:backup.hex:i
```

E.2.3 Verify only (no write)

```
avrdude -p m328p -c usbasp \
        -U flash:v:firmware.hex:i
```

E.2.4 Erase then write

```
avrdude -p m328p -c usbasp -e \
        -U flash:w:firmware.hex:i
```

E.2.5 Write EEPROM

```
avrdude -p m328p -c usbasp \
        -U eeprom:w:data.hex:i
```

E.2.6 Read fuse bytes

```
avrdude -p m328p -c usbasp \
        -U lfuse:r:-:h -U hfuse:r:-:h -U efuse:r:-:h
# Output: e.g. 0xff 0xd9 0x07 (lfuse hfuse efuse)
# '-' = stdout, 'h' = hex string format
```

E.2.7 Write fuse bytes

```
# WARNING: incorrect fuse values can brick the device.
# Always calculate fuses with an online calculator and verify before writing.

# Set hfuse = 0xDE (BOOTRST=0, BOOTSZ=10 for 1KB bootloader section)
avrdude -p m328p -c usbasp \
        -U hfuse:w:0xDE:m

# Restore ATmega328P factory fuses (lfuse=0xFF, hfuse=0xD9, efuse=0x07)
avrdude -p m328p -c usbasp \
        -U lfuse:w:0xFF:m \
        -U hfuse:w:0xD9:m \
        -U efuse:w:0x07:m
```

Warning: Fuses control clock source, brown-out detection, bootloader region, and JTAG. Setting CKSEL bits for an external crystal when no crystal is connected will make the device unresponsive — recovery requires an ISP programmer that provides a clock signal (High-Voltage Serial Programming).

E.2.8 Multiple operations in one command

```
avrdude -p m328p -c usbasp \
        -U flash:w:firmware.hex:i \
        -U eeprom:w:eeprom_data.hex:i
```

E.3 Common Programmers

Programmer ID	Description
arduino	Arduino bootloader via serial (auto-reset on DTR)
arduino-ft232r	Arduino via FTDI FT232R cable
avrisp	Atmel AVRISP v1 (serial)
avrisp2	Atmel AVRISP mkII (USB HID); also: avrispmkII
usbasp	USBasp (cheap USB ISP clone, widely available)
usbtiny	USBTiny / Adafruit USBTinyISP
dragon_isp	AVR Dragon in ISP mode
dragon_jtag	AVR Dragon in JTAG mode
jtag2isp	JTAGICE mkII in ISP mode
jtag2	JTAGICE mkII
atmelice_isp	Atmel-ICE in ISP mode
atmelice_pdi	Atmel-ICE in PDI mode (XMEGA)
atmelice_updi	Atmel-ICE in UPDI mode (tinyAVR/megaAVR 0/1/2-series)
pymcuprog	Microchip MPLAB Snap / Curiosity Nano via pymcuprog bridge
serialupdi	UPDI via USB-serial adapter (cheap, 1-wire)

E.3.1 UPDI Devices (ATtiny3217 and tinyAVR 1-series)

The ATtiny3217 uses UPDI (Unified Program and Debug Interface), a single-wire protocol. Use serialupdi with a USB-serial adapter connected to the UPDI pin via a 4.7kΩ resistor:

```
# ATtiny3217 via serialupdi on /dev/ttyUSB0
avrdude -p t3217 -c serialupdi -P /dev/ttyUSB0 \
        -U flash:w:firmware.hex:i
```


Read fuses

```
avrdude -p t3217 -c serialupdi -P /dev/ttyUSB0 \
  -U fuse0:r--:h -U fuse1:r--:h -U fuse2:r--:h \
  -U fuse4:r--:h -U fuse5:r--:h -U fuse6:r--:h -U fuse7:r--:h
```

UPDI wiring (1-wire):

```

USB-serial TX ┌───┐
                │   4.7kΩ ─ UPDI pin (ATtiny3217 pin 16)
USB-serial RX └───┘
USB-serial GND ─── GND

```

Alternatively, use the Atmel-ICE or MPLAB PICKit 4 for UPDI.

E.4 Device Part Numbers

Part Number	Device
m328p	ATmega328P (Arduino Uno, Nano)
m328pb	ATmega328PB
m2560	ATmega2560 (Arduino Mega)
m32u4	ATmega32U4 (Arduino Leonardo, Pro Micro)
m168p	ATmega168P
m88p	ATmega88P
m48p	ATmega48P
t3217	ATtiny3217 (book target for tinyAVR 1-series)
t1616	ATtiny1616
t816	ATtiny816
t416	ATtiny416
t85	ATtiny85
t84	ATtiny84
t45	ATtiny45
t13	ATtiny13
x128a4u	ATxmegal28A4U

Full list: `avrdude -p ?`

E.5 Fuse Byte Reference (ATmega328P)

This section is ATmega328P only. Classic ATmega parts use three fuse bytes (LFUSE/HFUSE/EFUSE) with the bit fields below. The book's ATtiny3217 does **not** have these — it uses the named fuses (WDTCFG, BODCFG, OSCCFG, SYSCFG0/1, APPEND, BOOTEND, LOCKBIT) documented in Appendix A. Do not apply the values below to the ATtiny3217.

E.5.1 Low Fuse (LFUSE)

Bit	Name	Description
7	CKDIV8	Divide clock by 8 (0 = enabled; factory default)
6	CKOUT	Clock output on PB0 (0 = enabled)
5:4	SUT1:0	Start-up time (10 = default 14CK + 65ms)
3:0	CKSEL3:0	Clock source (0010 = internal 8MHz RC; 1111 = external crystal)

Factory default (internal RC, /8): LFUSE = 0x62
 No divide (internal 8MHz full speed): LFUSE = 0xE2
 External 16MHz crystal (Arduino): LFUSE = 0xFF

E.5.2 High Fuse (HFUSE)

Bit	Name	Description
7	RSTDISBL	Disable reset pin (0 = PC6 becomes I/O; DANGEROUS)
6	DWEN	debugWIRE enable (0 = enabled; disables ISP when set)
5	SPIEN	SPI programming enable (0 = enabled; keep this 0)
4	WDTON	Watchdog always on (0 = always on)
3	EESAVE	Preserve EEPROM during chip erase (0 = preserved)
2:1	BOOTSZ1:0	Boot section size (11=256B, 10=512B, 01=1KB, 00=2KB)
0	BOOTRST	Boot reset vector (0 = reset jumps to boot section)

Factory default: HFUSE = 0xD9
 (SPI enabled, WDT off, EEPROM erased, 2KB boot, reset to 0x0000)
 Bootloader (1KB boot, reset to boot section): HFUSE = 0xDE

E.5.3 Extended Fuse (EFUSE)

Bit	Name	Description
2:0	BODLEVEL2:0	Brown-out detection level 111 = BOD disabled (factory) 110 = 1.8V 101 = 2.7V 100 = 4.3V

Factory default: EFUSE = 0x07 (BOD disabled; only bits 2:0 used)

E.5.4 Online Fuse Calculator

The Engbedded AVR Fuse Calculator provides a GUI for calculating fuse values. Search “AVR fuse calculator” — cross-check with the device datasheet before programming.

E.6 Generating Intel HEX from ELF

```
avr-objcopy -O ihex firmware.elf firmware.hex

# EEPROM section separately
avr-objcopy -O ihex -j .eeprom \
  --set-section-flags=.eeprom=alloc,load \
  --change-section-lma .eeprom=0 \
  firmware.elf eeprom.hex
```

E.7 Complete Build + Flash Script

```
#!/bin/bash
# build_flash.sh - assemble, link, and flash ATmega328P
```

```

set -e

MCU=atmega328p
PORT=/dev/ttyUSB0
BAUD=115200
SRC=firmware.S

avr-gcc -mmcu=${MCU} -nostartfiles -o firmware.elf ${SRC}
avr-size firmware.elf
avr-objcopy -O ihex firmware.elf firmware.hex
avrdude -p ${MCU} -c arduino -P ${PORT} -b ${BAUD} \
        -U flash:w:firmware.hex:i
echo "Done."

# For ATtiny3217 via serialupdi
MCU=attiny3217
PORT=/dev/ttyUSB0

avr-gcc -mmcu=${MCU} -nostartfiles -o firmware.elf firmware.S
avr-objcopy -O ihex firmware.elf firmware.hex
avrdude -p ${MCU} -c serialupdi -P ${PORT} \
        -U flash:w:firmware.hex:i

```

E.8 Troubleshooting

Problem	Likely cause / fix
avrdude: stk500_recv(): timeout	Wrong port, wrong baud, board not reset. Try -P /dev/ttyACM0, check USB cable.
avrdude: Expected signature for ATmega328P is 1E 95 0F	Wrong -p part number; use avrdude -p ? to list all; read signature with -U signature:r:-:h.
avrdude: verification error	Flash write failed; bad connection, insufficient power, or write-protected region.
avrdude: error: usbasp, Error: Could not find USB device	USBasp firmware too old for target; update USBasp firmware, or add -B 10 flag (slow SCK).
avrdude: initialization failed rc=-1; check connections and try again, or use -F	Target not responding; check power, RESET pin, JTAG enable fuse (JTAGEN=0 disables ISP on some devices).
Can't open device /dev/ttyUSB0 Permission denied	Add user to dialout group: sudo usermod -a -G dialout \$USER Then log out and back in.

E.8.1 USBasp Slow Clock

If ISP communication fails with a freshly-soldered board or a device that has been running at low speed (CKDIV8 fuse set):

```

# Slow down SCK to 125 kHz (-B = bitclock period in µs; -B 8 = 125kHz SCK)
avrdude -p m328p -c usbasp -B 8 \
        -U lfuse:w:0xFF:m # then reprogram fuses for 16MHz crystal

```

E.8.2 Arduino Auto-Reset Circuit

Arduino boards use a 100 nF capacitor on DTR/RST. avrdude -c arduino toggles DTR to reset the MCU before uploading. If the reset does not trigger:

```
# Manual: press reset button 2 seconds before avrdude starts, then release
avrdude -p m328p -c arduino -P /dev/ttyUSB0 -b 115200 \
        -U flash:w:firmware.hex:i
```

Or disable the auto-reset capacitor (cut the RESET EN trace on the Arduino PCB) when running a custom bootloader that does not expect the timing.

Appendix F

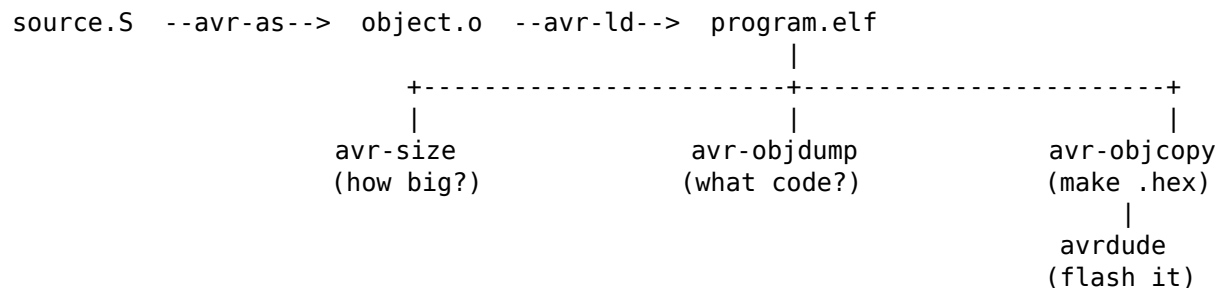
Inspecting Your Binaries

Assembling and linking turn your source into an ELF file, but an ELF file is not the end of the story. Before you flash a part you usually want to answer three questions:

1. How much flash and RAM did this use?
2. Did the assembler and linker actually produce the machine code I expected?
3. How do I turn the ELF into something a programmer can write to flash?

The GNU AVR toolchain answers each of these with a small, sharp tool: `avr-size`, `avr-objdump`, and `avr-objcopy`. The assembler (`avr-as`, see Appendix C), the linker (`avr-ld`, see Appendix D), the debugger (`avr-gdb`, see Chapter 5), and `avrdude` (Appendix E) are covered elsewhere. This appendix covers those three core tools and three supporting tools—`avr-nm`, `avr-readelf`, and `avr-addr2line`—that help you understand and diagnose the ELF the linker produces.

The whole loop looks like this:



F.1 The worked example

Every command in this appendix runs against the same small program so you can match output to source. It blinks the user LED on PA3 of the ATtiny3217 Curiosity Nano with a crude busy-wait delay.

```
; blink.S - ATtiny3217: blink the LED on PA3 with a busy-wait delay.
;
; VPORTA is in the low I/O space, so we can reach it with the
; single-cycle bit instructions sbi/cbi. Addresses are written out
; by hand here so the file is self-contained.
```

```
.equ    VPORTA_DIR, 0x00        ; VPORTA.DIR - data direction
.equ    VPORTA_OUT, 0x01        ; VPORTA.OUT  - output drive

.section .text
.global main
```

```

main:
    sbi      VPORTA_DIR, 3          ; PA3 as output
loop:
    sbi      VPORTA_OUT, 3         ; LED on
    rcall    delay
    cbi      VPORTA_OUT, 3         ; LED off
    rcall    delay
    rjmp     loop

delay:
    ldi      r24, lo8(40000)
    ldi      r25, hi8(40000)
1:    sbiw    r24, 1
    brne     1b
    ret

```

Build it in one step with `avr-gcc` acting as the assembler-and-linker driver:

```
avr-gcc -mmcu=attiny3217 -nostartfiles -e main blink.S -o blink.elf
```

The `-nostartfiles` flag drops the C runtime so the listings stay short and start at address 0. This is safe on the ATtiny3217 because the AVRXMEGA3 core resets the stack pointer to `RAMEND` in hardware; on classic AVR's the runtime does that job and you would leave `-nostartfiles` off. A normal build (without `-nostartfiles`) prepends the interrupt vector table and startup code, which shifts every address you see below but changes nothing about how the tools are used.

F.2 `avr-size` - how much flash and RAM

Run `avr-size` with no options for the classic “Berkeley” format:

```
$ avr-size blink.elf
   text    data     bss      dec     hex filename
    22      0       0       22     16 blink.elf
```

The columns mean:

Column	Meaning
text	Code and read-only data that lives in flash.
data	Initialised variables. Counts TWICE: the initial values are stored in flash, and a copy lives in RAM at run time.
bss	Zero-initialised variables. RAM only, costs no flash.
dec	text + data + bss, in decimal.
hex	The same total in hexadecimal.

The data column is the classic trap: a byte of initialised data costs you one byte of flash AND one byte of RAM. For flash budgeting, flash used = text + data; for RAM budgeting, RAM used = data + bss.

The friendlier view tells you how full the device is. Pass `-C` and the part name:

```
$ avr-size -C --mcu=attiny3217 blink.elf
AVR Memory Usage
-----
Device: attiny3217
```

```

Program:      22 bytes (0.1% Full)
(.text + .data + .bootloader)
```

```
Data:         0 bytes (0.0% Full)
```

```
(.data + .bss + .noinit)
```

Here Program is the flash figure (the ATtiny3217 has 32 KB) and Data is the RAM figure (2 KB). This is the form to watch as a project grows; when Program approaches 100% you are out of flash.

A wrinkle worth knowing: the percentages and the device name come from `avr-size`'s own internal device table, and not every build of the tool knows the newer tinyAVR parts. The `avr-size` bundled with Microchip's XC8 toolchain, for instance, prints "Device: Unknown" and omits the "% Full" figures for attiny3217. The raw byte counts are always correct; if you want the percentages, use the `avr-size` that ships with a device-aware `avr-gcc/avr-libc` toolchain. The Berkeley output (`avr-size` with no `-C`) never depends on the device table and is portable across builds.

A second gotcha: `avr-size` is part of *binutils*, not GCC. Some popular AVR toolchains are GCC-only builds that bundle just enough *binutils* for the compiler to run and omit `avr-size` (along with `addr2line`, `readelf`, and `strings`). If `avr-size` is "command not found", install a full *binutils* for AVR — or remember that `avr-objdump -h` already prints every section's size, so you can always read the byte counts from there.

A useful habit is to print the size on every build so a sudden jump is obvious. Make this the last line of your build rule:

```
avr-size -C --mcu=attiny3217 blink.elf
```

F.3 `avr-objdump` - see the machine code

`avr-objdump` is the most valuable tool in this appendix for an assembly programmer: it shows the exact bytes the assembler emitted and disassembles them back into mnemonics. It is how you confirm that an instruction encoded the way Appendix B says it should, and how you check what the linker did to relative branches.

Disassemble the code section with `-d`:

```
$ avr-objdump -d blink.elf
```

```
blink.elf:      file format elf32-avr
```

Disassembly of section `.text`:

```
00000000 <__ctors_end>:
   0:  03 9a          sbi      0x00, 3          ; 0

00000002 <loop>:
   2:  0b 9a          sbi      0x01, 3          ; 1
   4:  03 d0          rcall   .+6              ; 0xc <delay>
   6:  0b 98          cbi      0x01, 3          ; 1
   8:  01 d0          rcall   .+2              ; 0xc <delay>
  a:  fb cf          rjmp    .-10             ; 0x2 <loop>

0000000c <delay>:
   c:  80 e4          ldi      r24, 0x40        ; 64
   e:  9c e9          ldi      r25, 0x9C        ; 156

00000010 <L1^B1>:
  10:  01 97          sbiw     r24, 0x01        ; 1
  12:  f1 f7          brne     .-4              ; 0x10 <L1^B1>
  14:  08 95          ret
```

The first block is labelled `<__ctors_end>` rather than

: the linker places that constructor-table marker at address 0, our main lands at the same address, and avr-objdump prints whichever symbol it finds there first. The code is still your main.

Read one line left to right:

address	raw bytes	mnemonic + operands	; objdump comment
4:	03 d0	rcall .+6	; 0xc <delay>

- address: byte offset of the instruction in flash.
- raw bytes: the machine-code word, little-endian. 03 d0 is the 16-bit word 0xd003.
- mnemonic: the decoded instruction. Note rcall shows .+6, a PC-relative offset in bytes from the NEXT instruction, not the symbol you wrote.
- comment: avr-objdump helpfully resolves the target to an address and symbol, 0xc .

Three things worth pointing out in this listing:

- The numeric local label 1: shows up as <.L1^B1>, which is how GAS spells its internal name for a numeric local label. You wrote 1:/1b; the assembler keeps it under that mangled name rather than a label you would recognise.
- sbi 0x00, 3 and sbi 0x01, 3 are your VPORTA_DIR and VPORTA_OUT writes. objdump prints the raw I/O address because the .equ names are not in the binary; this is exactly why VPORT registers must sit in low I/O for sbi/cbi to reach them (see Chapter 16).
- The two rcall delay lines encode to different offsets (.+6 and .+2) even though they call the same routine, because the offset is measured from each call site. This is the relative-branch behaviour discussed in Chapter 14.

If you assemble with debug info (-g), avr-objdump can interleave your source lines with the disassembly using -S:

```
avr-gcc -mmcu=attiny3217 -nostartfiles -e main -g blink.S -o blink.elf
avr-objdump -S blink.elf
```

Each block of machine code is then preceded by the source line that produced it, which is the clearest way to see how one line of assembly maps to one instruction.

To see the layout of the file rather than the code, use -h for section headers:

```
$ avr-objdump -h blink.elf
```

```
blink.elf:      file format elf32-avr
```

Sections:

Idx	Name	Size	VMA	LMA	File off	Algn
0	.data	00000000	00803800	00000016	0000008a	2**0
	CONTENTS, ALLOC, LOAD, DATA					
1	.text	00000016	00000000	00000000	00000074	2**1
	CONTENTS, ALLOC, LOAD, READONLY, CODE					

This confirms .text is 0x16 (22) bytes and lives at address 0. VMA is the run-time address and LMA is the load address. Notice .data: it is empty here (this program has no initialised variables), but its VMA is 0x00803800 - the AVR data-space address of RAM, 0x3800, with the 0x800000 tag avr-objdump uses to mark “this lives in the data address space” - while its LMA is 0x16, right after .text in flash. That VMA/LMA split is exactly the case where initialised values are stored in flash (LMA) but used from RAM (VMA); it is how you reason about the copy-down that startup code performs.

Common avr-objdump flags:

Flag	Purpose
-d	Disassemble executable (code) sections.
-D	Disassemble ALL sections, including data.
-S	Interleave source with disassembly (needs -g at build time).

F.5 avr-nm - list symbols and their addresses

avr-nm reads an object file or ELF and prints every symbol: its value (usually an address in flash or RAM), type, and name. The fastest way to answer “where did the linker put delay?” or “is this label visible to other files?” is avr-nm.

The full output of avr-nm on blink.elf runs to about thirty lines because the linker script populates boundary and region symbols (type A, uppercase). The entries that come from your source are:

value	type	name
00000000	T	main
00000002	t	loop
0000000c	t	delay
00000010	t	.L11
00000000	a	VPORTA_DIR
00000001	a	VPORTA_OUT

Type codes worth knowing:

Code	Meaning
T	Symbol in .text (code), global -- visible at link time.
t	Symbol in .text (code), local -- invisible outside the file.
D	Symbol in .data (initialised variable), global.
d	Symbol in .data, local.
B	Symbol in .bss (zero-initialised variable), global.
b	Symbol in .bss, local.
A	Absolute constant (linker-script boundary or region size).
a	Absolute constant, local. .equ symbols appear with this type.
U	Undefined -- referenced here but not defined in this file.

Uppercase is global; lowercase is local. main is T because the source declares .global main. delay and loop are t because they have no .global directive and cannot be linked against from another file.

Two details worth noticing:

- VPORTA_DIR and VPORTA_OUT appear as type a. A .equ constant is not code or data: it is a named value resolved entirely at assembly time. avr-nm includes it in the symbol table so a debugger can display the name when it appears in an expression.
- .L11 is avr-nm’s rendering of the numeric local label 1: on line 33 of the source. avr-objdump renders the same symbol as <.L1^B1> – both are the assembler’s internal name for the same label, displayed differently by the two tools.

Add -n to sort by address instead of alphabetically. This shows the flash layout in the order the linker placed things, which is easier to read when checking function order or looking for gaps:

```
$ avr-nm -n blink.elf | grep " t "
00000002 t loop
0000000c t delay
00000010 t .L11
```

Lowercase t selects only local text symbols. Using [tT] would also produce the many linker-generated global T entries that clutter the sorted output. To locate a specific global symbol, grep by name:

```
$ avr-nm blink.elf | grep " main$"
00000000 T main
```

Add -u to print only undefined symbols. An empty result means the ELF has no unresolved references and will link cleanly:

```
$ avr-nm -u blink.elf
(no output -- all symbols are defined)
```

For multi-file projects, run `avr-nm -u` on each `.o` before linking to find which module references a missing symbol before the linker errors.

F.6 avr-readelf - inspect the ELF structure

`avr-objdump` works on section contents; `avr-readelf` works on the ELF container itself. The distinction matters when you want to check architecture metadata, section flags, or program-header layout rather than the bytes those sections hold.

The `-h` flag prints the ELF header:

```
$ avr-readelf -h blink.elf
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF32
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  EXEC (Executable file)
  Machine:                               Atmel AVR 8-bit microcontroller
  Version:                               0x1
  Entry point address:                   0x0
  Start of program headers:              52 (bytes into file)
  Start of section headers:              1264 (bytes into file)
  Flags:                                  0x67, avr:103
  Size of this header:                   52 (bytes)
  Size of program headers:               32 (bytes)
  Number of program headers:              2
  Size of section headers:               40 (bytes)
  Number of section headers:              6
  Section header string table index: 5
```

The fields to verify for AVR work:

Field	What to check
Machine	Should read "Atmel AVR 8-bit microcontroller". A non-AVR ELF will not disassemble correctly and <code>avrdude</code> will refuse it.
Entry point	Address where execution begins after reset. <code>0x0</code> here because <code>-nostartfiles</code> places <code>main</code> at the start of flash.
Flags: avr:NNN	The architecture variant number. 103 is AVRXMEGA3, the family containing the ATtiny3217. A mismatch between this number and your target device means the code was not compiled for the right core.

The `-S` flag prints section headers – the same information as `avr-objdump -h` but in a fixed-column format that is easier to parse with shell scripts or Python:

```
$ avr-readelf -S blink.elf
There are 6 section headers, starting at offset 0x4f0:
```

Section Headers:

[Nr]	Name	Type	Addr	Off	Size	ES	Flg	Lk	Inf	Al
[0]		NULL	00000000	000000	000000	00		0	0	0
[1]	.data	PROGBITS	00803800	00008a	000000	00	WA	0	0	1
[2]	.text	PROGBITS	00000000	000074	000016	00	AX	0	0	2

```

[ 3] .symtab    SYMTAB    00000000 00008c 000240 10      4   9   4
[ 4] .strtab    STRTAB    00000000 0002cc 0001fd 00      0   0   1
[ 5] .shstrtab  STRTAB    00000000 0004c9 000027 00      0   0   1

```

The Flg column records section flags:

Flag	Meaning
A	Alloc -- the section occupies device memory at run time.
X	Execute -- the CPU can fetch and run instructions from it.
W	Write -- the section is writable (RAM).

.text is AX: code in flash that is executable. .data is WA: RAM that is writable. The three remaining sections (.symtab, .strtab, .shstrtab) carry no flags; they exist only in the ELF file and avr-objcopy drops them before producing the .hex.

Building with -g adds several .debug_* sections (also flagged as no-alloc). They do not change what goes into flash; they are debug metadata consumed by avr-gdb and avr-addr2line.

avr-readelf is most useful when a linker script section directive or **attribute**((section("..."))) is not behaving as expected. The -S output shows exactly where each section landed, its actual size, and whether it received the flags you intended.

F.7 avr-addr2line - translate an address to source

avr-addr2line takes an address from a listing, a debugger, or a crash log and tells you the source file and line that produced the code at that address. It is faster than scrolling through avr-objdump -S when you only need to cross-reference a handful of addresses.

The ELF must include debug information. Build with -g:

```
avr-gcc -mmcu=attiny3217 -nostartfiles -e main -g blink.S -o blink.elf
```

Pass one or more addresses with -e to name the ELF file:

```
$ avr-addr2line -e blink.elf 0x10
/path/to/blink.S:33
```

Address 0x10 is the sbiw instruction at line 33 – the body of the countdown loop. avr-addr2line resolved it without opening the full disassembly.

You can give several addresses in one call, which is useful when reviewing output from avr-gdb's info registers:

```
$ avr-addr2line -e blink.elf 0x04 0x08 0x10
/path/to/blink.S:24
/path/to/blink.S:26
/path/to/blink.S:33
```

The -f flag adds the enclosing function name above each result. addr2line takes that name from the ELF symbol table: it reports the nearest function-typed (STT_FUNC) symbol at or before the address. A plain assembly label is not function-typed unless you mark it with a .type directive:

```
.type main, @function
.type delay, @function
```

avr-gcc emits that marking automatically for every C function. A hand-written .S file like blink.S has none, so its labels stay untyped and addr2line -f prints ?? in place of the name. The file and line still resolve correctly – those come from the DWARF .debug_line table that -g always generates; only the function name depends on the symbol type. Add a .type directive to any .S subroutine you want addr2line -f to name.

If `avr-addr2line` is missing from your toolchain – it is part of GNU binutils and some GCC-only packages omit it – the same information is available inside `avr-gdb`: load the ELF and run `info line *0x10`. `avr-addr2line` is the faster option when you are not already in a debug session.

F.8 Other tools at a glance

avr-gcc as driver. The build command throughout this appendix is `avr-gcc` rather than `avr-as`. `avr-gcc` is a compiler driver: it detects a `.S` file and passes it to `avr-as`, then passes the resulting `.o` to `avr-ld`. It is more convenient than calling the tools directly for single-file projects because `-mmcu=attiny3217` is written once and the driver selects the right linker script automatically. Appendices C and D cover `avr-as` and `avr-ld` directly; use them when you need options the driver does not expose.

avr-ar and avr-ranlib. These pack multiple `.o` files into a static library (`.a` archive) and index it so the linker can extract only the objects it needs. For most bare-metal assembly projects you list the `.o` files directly on the linker command line and need no archive. If you build a reusable collection of subroutines shared across several projects, create and index the archive with:

```
avr-ar cr libfoo.a util1.o util2.o
avr-ranlib libfoo.a
```

Then link against it with `avr-ld ... -L. -lfoo`.

F.9 Putting it together

A typical build rule chains all of this:

```
blink.elf: blink.S
    avr-gcc -mmcu=attiny3217 -nostartfiles -e main -g blink.S -o blink.elf
    avr-size -C --mcu=attiny3217 blink.elf
```

```
blink.hex: blink.elf
    avr-objcopy -O ihex -j .text -j .data blink.elf blink.hex
```

```
disasm: blink.elf
    avr-objdump -S blink.elf
```

With these three tools you can answer, for any program, exactly how much room it takes, exactly what machine code it became, and produce exactly the file your programmer expects, closing the gap between “it assembled” and “it runs on the chip.”